CSS

MDN Web Docs (Mozilla) — Português do Brasil

Fonte oficial: https://developer.mozilla.org/pt-BR/docs/Web/CSS
Gerado para o projeto doc-frog

CSS

CSS (***Cascading Style Sheets* ou Folhas de Estilo em Cascata)** é uma linguagem de estilo usada para descrever a apresentação de um documento escrito em HTML ou em XML (incluindo várias linguagens em XML como SVG, MathML ou XHTML). O CSS descreve como elementos são mostrados na tela, no papel, na fala ou em outras mídias.

CSS é uma das principais linguagens da **open web** e é padronizada em navegadores web de acordo com as especificação da W3C. Desenvolvido em níveis, o CSS1 está atualmente obsoleto, o CSS2.1 é uma recomendação e o CSS3, agora dividido em pequenos módulos, está progredindo para a sua padronização.

- Introdução CSS

  Se você é novo no desenvolvimento web, certifique-se de ler nosso artigo CSS básico para aprender o que é CSS e como usá-lo.

- Tutoriais CSS

  Nossa área de aprendizado CSS contém diversos tutoriais para levá-lo do nível iniciante ao avançado, cobrindo todos os fundamentos.

- Referências CSS

  Nossa extensa referência CSS para desenvolvedores web experientes descreve todas as propriedades e conceitos do CSS.

Tutoriais

Nossa área de aprendizado CSS contempla múltiplos módulos que ensinam CSS desde o início — não é necessário conhecimento prévio.

Introdução ao CSS

Esse módulo inicia com o funcionamento básico do CSS, incluindo seletores e propriedades, escrever regras CSS, aplicar CSS ao HTML, como especificar comprimento, cor, e outras unidades em CSS, cascata e herança, o básico do modelo de caixas, e depuração de CSS.

Estilização do texto

Esse módulo discute os fundamentos da estilização de textos, incluindo a definição de fontes, negritos, itálicos, linhas e espaçamentos de letras, sombras e outras propriedades de textos.

Estilização de caixas

Esse módulo foca na estilização de caixas, um dos passos fundamentais para o layout de uma página web. Nesse módulo nós recapitulamos o modelo de caixas, então abordamos o controle do layout das caixas definindo margens, bordas e preenchimento, cores de fundo customizadas, imagens e outras características extravagantes como sombras e filtros em caixas.

Layout CSS

Neste ponto nós já vimos os fundamentos do CSS, como estilizar texto e estilizar e manipular caixas que contém seu conteúdo. Agora veremos como posicionar suas caixas no lugar certo em relação à janela de exibição e um ao outro. Nós cobrimos os pré-requisitos necessários e agora você pode se aprofundar no layout do CSS com diferentes configurações de exibição, métodos tradicionais de layout envolvendo float e posicionamento, e modernas ferramentas para layout como flexbox.

Referências

- Referências CSS: Essa extensa referência para desenvolvedores web experientes descreve todas as propriedades e conceitos do CSS.
- Conceitos chave do CSS:
  - A sintaxe e formas da linguagem
  - Especificidade, herança e cascata
  - Box model e colapso de margem
  - O containing block
  - Empilhamento e block-formatting context
  - Valores iniciais, computados, usados e reais
  - Propriedades abreviadas do CSS
  - CSS Flexible Box Layout
  - CSS Grid Layout
  - Media queries

Livro de receitas

O livro de receitas de layout CSS tem como objetivo juntar receitas para padrões comuns de layout, coisas que você pode precisar para implementar em seus sites. Além de prover código você pode usá-lo como ponto de partida em seus projetos, essas receitas destacam as diferentes formas de

utilização das especificações de layout e as escolhas que você pode fazer como desenvolvedor.

Ferramentas para desenvolvimento de CSS

- Você pode usar o W3C CSS Validation Service para verificar se o seu CSS é válido. Essa é uma ferramenta indispensável de depuração.
- As Ferramentas do Firefox para desenvolvedores permitem ver e editar o CSS de uma página ao vivo usando as ferramentas de Inspeção e Editor de Estilo.
- O complemento Web Developer para Firefox também permite a observação e edição de CSS ao vivo dos sites visualizados.
- A comunidade web criou várias outras pequenas ferramentas CSS para você usar.

Veja também

- CSS demos: Inspire-se explorando criativos exemplos das mais recentes tecnologias CSS em ação.
- Linguagens web nas quais o CSS é frequentemente aplicado: HTML, SVG, MathML, XHTML e XML.
- Tecnologias Mozilla que usam extensivamente o CSS: XUL, complementos e temas do Firefox e Thunderbird.

Usando animações CSS

> **Experimental: Esta é uma tecnologia experimental**
> Verifique a tabela de compatibilidade entre Navegadores cuidadosamente antes de usar essa funcionalidade em produção.

Animações CSS tornam possível animar transições de um estilo CSS para outro. Animações se consistem de dois componentes: um estilo descrevendo a animação e um set de keyframes que indicam o estado final e inicial do estilo CSS da animação, bem como possíveis waypoints intermediários ao longo do caminho.

Existem três vantagens chave para animações CSS além das técnicas tradicionais de animação dirigidas por script:

1. São de fácil utilização para animações simples; você pode criá-las sem mesmo ter que conhecer JavaScript.
2. As animações executam bem, mesmo sobre moderada carga do sistema. Animações simples podem normalmente ser executadas precariamente em JavaScript (a não ser que sejam bem feitas). A ferramenta de renderização pode usar frame-skipping e outras técnicas para manter a performance o mais estável possível.
3. Deixando o navegador controlar a sequência de animação permite ao navegador otimizar a performance e eficiência em, por exemplo, reduzir a frequência de update de animações correndo em abas que não estão visíveis no momento.

Configurando a animação

Para criar uma sequência de animação CSS, você estiliza o elemento que deseja animar com a propriedade `animation` ou suas sub-propriedades. Isso permite que você configure a sincronização da animação, bem como outros detalhes de como a de como a sequência de animação deveria progredir. Isso **não** configura a aparência atual da animação, que é feita usando a regra com parênteses (at-rule) `@keyframes` como descrito em Defining the animation sequence using keyframes abaixo.

As sub-propriedades da propriedade `animation` são:

`animation-delay`

: Configura o delay entre o tempo em que o elemento é carregado e o inicio da sequência de animação.

`animation-direction`

: Configura se a animação deve ou nao alternar a direção em cada execução durante a sequência ou voltar ao ponto inicial e se repetir.

`animation-duration`

: Configura o tempo que uma animação deveria levar para completar um ciclo.

`animation-iteration-count`

: Configura o numero de vezes que uma animação deveria se repetir; você pode especificar infinito para repetir a animação indefinidamente.

`animation-name`

: Especifica o nome da regra com parênteses (at-rule) `@keyframes` at-rule descrevendo os keyframes da animação.

`animation-play-state`

: Permite voce pausar e resumir a sequência da animação.

`animation-timing-function`

Configura a sincronização da animação; que é, como a animação transita por keyframes, por estabilizar curvas de aceleração.

`animation-fill-mode`

Configura que valores são aplicados pela animação antes e depois de se executar.

Definindo a sequência de animação usando keyframes

Uma vez que você configurou a sincronização da animação, você precisa definir a aparência da animação. Isso é feito por estabelecer duas ou mais keyframes usando a regra com parênteses (at-rule) `@keyframes`. Cada keyframe descreve como o elemento animado deveria se renderizar a um tempo dado durante a sequência de animação.

Como a sincronização da animação é definida por um estilo CSS que configura a animação, keyframes usam uma `<percentage>` para indicar o tempo durante a sequência de animação que eles fazem parte. 0% indica o primeiro momento da sequência de animação, enquanto 100% indica o estado final da animação. Esses dois tempos devem ser especificados para que o navegador então saiba onde a animação deve começar e parar; por serem tão importantes, esses dois tempos tem expressões equivalentes especiais: from e to.

Você pode opcionalmente incluir keyframes adicionais que descrevem passos intermediários ao longo do caminho do ponto inicial ao ponto final da animação.

Exemplos

> **Nota:** Os exemplos aqui não usam nenhum prefixo nas propriedades de animação CSS. Navegadores mais antigos podem precisar de prefixos; os exemplos ao vivo que você pode clicar pra ver em seu navegadores também incluem as versões prefixadas -webkit.

Fazendo o texto deslizar através da janela do navegador

Esse exemplo simples estiliza o elemento `<p>` onde o elemento então desliza para dentro vindo de fora da lateral direita da janela do navegador.

Perceba que animações como essa podem fazer com que a página se torne mais larga que a janela do navegador. Para evitar esse problema coloque o elemento a ser animado dentro de um container, e atribua `overflow:hidden` ao container.

```
p {
  animation-duration: 3s;
  animation-name: slidein;
}

@keyframes slidein {
  from {
    margin-left: 100%;
    width: 300%;
  }

  to {
    margin-left: 0%;
    width: 100%;
  }
}
```

O estilo para o elemento `<p>` aqui especifica que a animação deveria levar 3 segundos para executar do início ao fim, usando a propriedade `animation-duration`, e que o nome da regra com parênteses (at-rule)`@keyframes` definindo os keyframes para a sequência de animação é nomeado por "slidein".

Se quiséssemos quaisquer estilização customizada no elemento `<p>` para aparecer em navegadores que não suportam animações CSS, incluiríamos aqui também; no entanto, nesse caso não queremos nenhuma estilização customizada a não ser o efeito da animação.

Os keyframes são definidos utilizando-se as regras`@keyframes`. Neste caso, utilizamos apenas dois keyframes. O primeiro ocorre no progresso de 0% (ou seja, o primeiro keyframe da animação, através do pseudônimo from). Nesta etapa, configuramos a margem esquerda do elemento para ser 100% - quer dizer, como a margem está à esquerda e com valor 100%, o elemento irá se deslocar para o seu limite, ou seja, para a parte direita – e sua largura será de 300%, ou seja, 3 vezes a largura do seu tamanho original. Isto faz com que o elemento, em seu primeiro frame da animação, seja "empurrado" para fora do limite da parte direita da janela do navegador.

O segundo (e último) keyframe ocorre na etapa 100% do progresso (ou seja, o último keyframe da animação, através do pseudônimo to). A margem esquerda está com valor de 0% e a largura do elemento está com valor de 100%. Isto resulta na animação do elemento `<p>`, que entra gradativamente na área de conteúdo até atingir uma margem esquerda de 0%.

```
<p>
  A Lagarta e Alice olharam-se por algum tempo em silêncio: finalmente, a
```

```
  Lagarta tirou o narguilé da boca e dirigiu-se a ela com uma voz lânguida e
  sonolenta.
</p>
```

Adicionando outro keyframe

Vamos adicionar outro keyframe à animação do exemplo anterior. Digamos que nós queremos que o tamanho da fonte aumente durante o movimento da direita para a esquerda por um determinado momento, e que depois ele reduzisse ao seu tamanho original. Você precisaria simplesmente adicionar este keyframe:

```
75% {
  font-size: 300%;
  margin-left: 25%;
  width: 150%;
}
```

O código completo fica da seguinte forma:

```
p {
  animation-duration: 3s;
  animation-name: slidein;
}

@keyframes slidein {
  from {
    margin-left: 100%;
    width: 300%;
  }

  to {
    margin-left: 0%;
    width: 100%;
  }
}
```

```
<p>
  A Lagarta e Alice olharam-se por algum tempo em silêncio: finalmente, a
  Lagarta tirou o narguilé da boca e dirigiu-se a ela com uma voz lânguida e
  sonolenta.
</p>
```

Isso indica ao navegador que até atingir a etapa 75% do progresso da sequência da animação o elemento deve ter 25% no valor da sua margem esquerda e sua largura deve ser de 150%.

Faça repetir-se

Para fazer a animação se repetir, simplesmente use a propriedade`animation-iteration-count` para indicar a quantidade de vezes que a animação deve se repetir. Neste caso, vamos usar `infinite` para que a animação se repita indefinidamente:

```
p {
  animation-duration: 3s;
  animation-name: slidein;
  animation-iteration-count: infinite;
}
```

Adicionando ao código existente:

```
@keyframes slidein {
  from {
    margin-left: 100%;
    width: 300%;
  }

  to {
    margin-left: 0%;
    width: 100%;
  }
}
```

```
<p>
  A Lagarta e Alice olharam-se por algum tempo em silêncio: finalmente, a
  Lagarta tirou o narguilé da boca e dirigiu-se a ela com uma voz lânguida e
  sonolenta.
</p>
```

Fazendo a animação se mover para trás e para frente

Com o exemplo anterior, fizemos a animação se repetir, mas é muito estranho tê-la pulando lá do início toda vez que a animação inicia. O que nós realmente queremos é que a animação se mova para trás e para frente por toda tela. Isso é facilmente realizado se adicionarmos o valor alternate à propriedade `animation-direction`:

```
p {
  animation-duration: 3s;
  animation-name: slidein;
  animation-iteration-count: infinite;
  animation-direction: alternate;
}
```

```
@keyframes slidein {
  from {
    margin-left: 100%;
    width: 300%;
  }

  to {
    margin-left: 0%;
    width: 100%;
  }
}
```

```
<p>
  A Lagarta e Alice olharam-se por algum tempo em silêncio: finalmente, a
  Lagarta tirou o narguilé da boca e dirigiu-se a ela com uma voz lânguida e
  sonolenta.
</p>
```

Usando eventos de animação

Você pode ter controle adicional sobre animações — como também informações úteis sobre elas — através do uso de eventos de animação. Esses eventos, representados pelo objeto `AnimationEvent` , podem ser usados para detectar quando animações iniciam, terminam, e começam uma nova iteração. Cada evento inclui o tempo no qual ele ocorreu como também o nome da animação que lançou o evento.

Nós vamos modificar o exemplo de deslizamento de texto para gerar alguma informação sobre cada evento de animação quando ele ocorrer, para que possamos perceber como eles funcionam.

Adicionando o CSS

Nós começamos criando o CSS para a animação. Essa animação vai durar por 3 segundos, se chamar "slidein", repetir 3 vezes, e alternar a direção cada vez. No `@keyframes`, a largura (width) e a margem esquerda (margin-left) são manipulados para fazer o elemento deslizar na tela.

```
.slidein {
  -moz-animation-duration: 3s;
  -webkit-animation-duration: 3s;
  animation-duration: 3s;
  -moz-animation-name: slidein;
  -webkit-animation-name: slidein;
  animation-name: slidein;
  -moz-animation-iteration-count: 3;
  -webkit-animation-iteration-count: 3;
  animation-iteration-count: 3;
  -moz-animation-direction: alternate;
  -webkit-animation-direction: alternate;
  animation-direction: alternate;
}

@-moz-keyframes slidein {
  from {
    margin-left: 100%;
```

```
    width: 300%;
  }

  to {
    margin-left: 0%;
    width: 100%;
  }
}

@-webkit-keyframes slidein {
  from {
    margin-left: 100%;
    width: 300%;
  }

  to {
    margin-left: 0%;
    width: 100%;
  }
}

@keyframes slidein {
  from {
    margin-left: 100%;
    width: 300%;
  }

  to {
    margin-left: 0%;
    width: 100%;
  }
}
```

Adicionando animação a lista de eventos

Nós vamos usar o código JavaScript para escutar todos três possíveis eventos de animação. Esse código configura nossos escutadores de evento; nós o chamamos quando o documento é primeiramente carregado para configurar as coisas.

```
var e = document.getElementById("watchme");
e.addEventListener("animationstart", listener, false);
e.addEventListener("animationend", listener, false);
e.addEventListener("animationiteration", listener, false);

e.className = "slidein";
```

Isso é simplesmente código padrão; você pode ter detalhes sobre como ele funciona na documentação para `element.addEventListener()`. A última coisa que este código faz é atribuir a classe no elemento o qual estaremos animando para "deslizar" (slidein); nós fazemos isso para iniciar a animação.

Por que? Porque o evento `animationstart` dispara assim que a animação começa, e no nosso caso, isso acontece antes do nosso código rodar. Então nós mesmos vamos iniciar a animação através de atribuição da classe do elemento para o estilo que é animado depois do ocorrido.

Recebendo os eventos

Os eventos são entregues à função `listener()`, a qual é mostrada abaixo.

```
function listener(e) {
  var l = document.createElement("li");
  switch (e.type) {
    case "animationstart":
      l.innerHTML = "Started: elapsed time is " + e.elapsedTime;
      break;
    case "animationend":
      l.innerHTML = "Ended: elapsed time is " + e.elapsedTime;
      break;
    case "animationiteration":
      l.innerHTML = "New loop started at time " + e.elapsedTime;
      break;
  }
  document.getElementById("output").appendChild(l);
}
```

Esse código também é bem simples. Ele simplemente olha no `event.type` para determinar qual tipo de evento de animação ocorreu, então adiciona uma nota apropriada no `<ul>` (lista não ordenada) que estamos usando para logar esses eventos.

A saída, quando tudo foi dito e feito, parece com algo assim:

- Started: elapsed time is 0
- New loop started at time 3.01200008392334
- New loop started at time 6.00600004196167
- Ended: elapsed time is 9.234000205993652

Perceba que os tempos são bem próximos, mas não exatamente iguais, àqueles esperados dado o tempo estabelecido quando a animação foi configurada. Perceba também que após a ultima iteração da animação, o evento `animationiteration` não é enviado; ao invés disso, o evento `animationend` é enviado.

O HTML

Apenas por questão de completude, aqui está o HTML que exibe o conteúdo da pagina, incluindo a lista na qual o script insere informação sobre os eventos recebidos:

```
<h1 id="watchme">Veja-me mover</h1>
<p>
  Este exemplo mostra como usar animações CSS para fazer o elemento
  <code>h1</code>
  se mover pela página
</p>
<p>
  Além disso, emitimos algum texto sempre que um evento de animação dispara,
  para que você possa vê-los em ação.
</p>
<ul id="output"></ul>
```

Veja também

- `AnimationEvent`
- Detectando suporte de animação CSS

Gerador de Border-image

Esta ferramenta pode ser utilizada para gerar o valor `border-image` em CSS3.

Gerador de Border-radius

Esta ferramenta pode ser utilizada para gerar o efeito `border-radius` em CSS3.

```
<div id="container">
  <div class="group section">
    <div id="preview" class="col span_12">
      <div id="subject">
        <div
          id="top-left"
          class="radius-container"
          data-X="left"
          data-Y="top"></div>
        <div
          id="top-right"
          class="radius-container"
          data-X="right"
          data-Y="top"></div>
        <div
          id="bottom-right"
          class="radius-container"
          data-X="right"
          data-Y="bottom"></div>
        <div
          id="bottom-left"
          class="radius-container"
          data-X="left"
          data-Y="bottom"></div>

        <div id="radius-ui-sliders">
```

```
<div
  id="tlr"
  class="ui-input-slider"
  data-topic="top-left"
  data-unit=" px"
  data-sensivity="2"></div>

<div
  id="tlw"
  class="ui-input-slider"
  data-topic="top-left-w"
  data-unit=" px"
  data-sensivity="2"></div>

<div
  id="tlh"
  class="ui-input-slider"
  data-topic="top-left-h"
  data-unit=" px"
  data-sensivity="2"></div>

<div
  id="trr"
  class="ui-input-slider"
  data-topic="top-right"
  data-unit=" px"
  data-sensivity="2"></div>

<div
  id="trw"
  class="ui-input-slider"
  data-topic="top-right-w"
  data-unit=" px"
  data-sensivity="2"></div>

<div
  id="trh"
  class="ui-input-slider"
  data-topic="top-right-h"
  data-unit=" px"
  data-sensivity="2"></div>

<div
  id="brr"
  class="ui-input-slider"
  data-topic="bottom-right"
  data-unit=" px"
  data-sensivity="2"></div>

<div
  id="brw"
  class="ui-input-slider"
  data-topic="bottom-right-w"
  data-unit=" px"
  data-sensivity="2"></div>

<div
  id="brh"
  class="ui-input-slider"
  data-topic="bottom-right-h"
  data-unit=" px"
  data-sensivity="2"></div>

<div
  id="blr"
  class="ui-input-slider"
  data-topic="bottom-left"
  data-unit=" px"
  data-sensivity="2"></div>

<div
  id="blw"
  class="ui-input-slider"
```

```
            data-topic="bottom-left-w"
            data-unit=" px"
            data-sensivity="2"></div>

          <div
            id="blh"
            class="ui-input-slider"
            data-topic="bottom-left-h"
            data-unit=" px"
            data-sensivity="2"></div>
        </div>
      </div>
    </div>
  </div>
  <div id="controls" class="group section">
    <div class="group section">
      <div id="dimensions">
        <div
          class="ui-input-slider"
          data-topic="width"
          data-info="width"
          data-unit=" px"
          data-min="150"
          data-max="700"
          data-sensivity="1"></div>

        <div
          class="ui-input-slider"
          data-topic="height"
          data-info="height"
          data-unit=" px"
          data-min="75"
          data-max="350"
          data-sensivity="1"></div>
      </div>

      <div id="output"></div>
    </div>

    <div class="group section">
      <div id="radius-lock">
        <div class="info">rounded corner</div>
        <div class="ui-checkbox" data-topic="top-left"></div>
        <div class="ui-checkbox" data-topic="top-right"></div>
        <div class="ui-checkbox" data-topic="bottom-right"></div>
        <div class="ui-checkbox" data-topic="bottom-left"></div>
      </div>

      <div id="unit-selection">
        <div class="info">select border units</div>
      </div>
    </div>
  </div>
</div>
```

```
/*  GRID OF TEN
 * ========================================================================== */

.span_12 {
  width: 100%;
}

.span_11 {
  width: 91.46%;
}

.span_10 {
  width: 83%;
}

.span_9 {
  width: 74.54%;
}
```

```
.span_8 {
  width: 66.08%;
}

.span_7 {
  width: 57.62%;
}

.span_6 {
  width: 49.16%;
}

.span_5 {
  width: 40.7%;
}

.span_4 {
  width: 32.24%;
}

.span_3 {
  width: 23.78%;
}

.span_2 {
  width: 15.32%;
}

.span_1 {
  width: 6.86%;
}

/*  SECTIONS
 * ========================================================================== */

.section {
  clear: both;
  padding: 0px;
  margin: 0px;
}

/*  GROUPING
 * ========================================================================== */

.group:before,
.group:after {
  content: "";
  display: table;
}

.group:after {
  clear: both;
}

.group {
  zoom: 1; /* For IE 6/7 (trigger hasLayout) */
}

/*  GRID COLUMN SETUP
 * ========================================================================== */

.col {
  display: block;
  float: left;
  margin: 1% 0 1% 1.6%;
}

.col:first-child {
  margin-left: 0;
} /* all browsers except IE6 and lower */

/*
```

```
 * UI Component
 */

.ui-input-slider-container {
  height: 20px;
  margin: 10px 0;
  font-family: "Segoe UI", Arial, Helvetica, sans-serif;
  -moz-user-select: none;
  user-select: none;
}

.ui-input-slider-container * {
  float: left;
  height: 100%;
  line-height: 100%;
}

/* Input Slider */

.ui-input-slider > input {
  margin: 0;
  padding: 0;
  width: 50px;
  text-align: center;

  -moz-box-sizing: border-box;
  -webkit-box-sizing: border-box;
  box-sizing: border-box;
}

.ui-input-slider-info {
  width: 90px;
  padding: 0px 10px 0px 0px;
  text-align: right;
  text-transform: lowercase;
}

.ui-input-slider-left,
.ui-input-slider-right {
  width: 16px;
  cursor: pointer;
  background: url("arrows.png") center left no-repeat;
}

.ui-input-slider-right {
  background: url("arrows.png") center right no-repeat;
}

.ui-input-slider-name {
  width: 90px;
  padding: 0 10px 0 0;
  text-align: right;
  text-transform: lowercase;
}

.ui-input-slider-btn-set {
  width: 25px;
  background-color: #2c9fc9;
  border-radius: 5px;
  color: #fff;
  font-weight: bold;
  line-height: 14px;
  text-align: center;
}

.ui-input-slider-btn-set:hover {
  background-color: #379b4a;
  cursor: pointer;
}

/*
 * UI Component
 */
```

```
/* Checkbox */

.ui-checkbox {
  text-align: center;
  font-size: 16px;
  font-family: "Segoe UI", Arial, Helvetica, sans-serif;
  line-height: 1.5em;
  color: #fff;

  -moz-user-select: none;
  -webkit-user-select: none;
  -ms-user-select: none;
  user-select: none;
}

.ui-checkbox > input {
  display: none;
}

.ui-checkbox > label {
  font-size: 12px;
  padding: 0.333em 1.666em 0.5em;
  height: 1em;
  line-height: 1em;

  background-color: #888;
  background-image: url("disabled.png");
  background-position: center center;
  background-repeat: no-repeat;

  color: #fff;
  border-radius: 3px;
  font-weight: bold;
  float: left;
}

.ui-checkbox .text {
  padding-left: 34px;
  background-position: center left 10px;
}

.ui-checkbox .left {
  padding-right: 34px;
  padding-left: 1.666em;
  background-position: center right 10px;
}

.ui-checkbox > label:hover {
  cursor: pointer;
}

.ui-checkbox > input:checked + label {
  background-image: url("checked.png");
  background-color: #379b4a;
}

body {
  max-width: 1000px;
  margin: 0 auto;

  font-family: "Segoe UI", Arial, Helvetica, sans-serif;

  -moz-box-sizing: border-box;
  -webkit-box-sizing: border-box;
  box-sizing: border-box;

  -moz-user-select: none;
  -webkit-user-select: none;
  -ms-user-select: none;
  user-select: none;
}
```

```
#container {
  width: 100%;

  -moz-box-sizing: border-box;
  -webkit-box-sizing: border-box;
  box-sizing: border-box;
}

/*****************************************************************************/
/*****************************************************************************/
/*
 * Preview Area
 */

#preview {
  height: 500px;
  border: 1px solid #ccc;
  border-radius: 3px;
  text-align: center;
  overflow: hidden;
  position: relative;
}

#preview .content {
  width: 100%;
  height: 100%;
  display: block;
}

#preview input {
  color: #333;
  border: 1px solid #ccc;
  border-radius: 3px;
}

#subject {
  width: 400px;
  height: 150px;
  margin: 0 auto;
  border: 3px solid #c60;
  background: #fff;
  position: relative;
}

.radius {
  width: 50%;
  height: 50%;
  border: 1px solid #ccc;
  display: none;
  position: absolute;
  z-index: 1;
  -moz-box-sizing: border-box;
  -webkit-box-sizing: border-box;
  box-sizing: border-box;
}

.handle {
  width: 16px;
  height: 16px;
  position: absolute;
  z-index: 2;
}

.handle-top-left {
  top: -12px;
  left: -12px;
  cursor: se-resize;
  background: url("resize-handle.png") top left no-repeat;
}

.handle-top-right {
  top: -12px;
  right: -12px;
```

```
  cursor: sw-resize;
  background: url("resize-handle.png") top right no-repeat;
}

.handle-bottom-right {
  bottom: -12px;
  right: -12px;
  cursor: nw-resize;
  background: url("resize-handle.png") bottom right no-repeat;
}

.handle-bottom-left {
  bottom: -12px;
  left: -12px;
  cursor: ne-resize;
  background: url("resize-handle.png") bottom left no-repeat;
}

.radius-container {
  position: absolute;
  display: block;
  z-index: 1;

  -moz-box-sizing: border-box;
  -webkit-box-sizing: border-box;
  box-sizing: border-box;
}

/* TOP LEFT */
#top-left {
  top: 0;
  left: 0;
}

#top-left .radius {
  border-top-left-radius: 100%;
  top: 0;
  left: 0;
}

/* TOP RIGHT */
#top-right {
  top: 0;
  right: 0;
}

#top-right .radius {
  border-top-right-radius: 100%;
  top: 0;
  right: 0;
}

/* BOTTOM RIGHT */
#bottom-right {
  bottom: 0;
  right: 0;
}

#bottom-right .radius {
  border-bottom-right-radius: 100%;
  bottom: 0;
  right: 0;
}

/* BOTTOM lEFT */
#bottom-left {
  bottom: 0;
  left: 0;
}

#bottom-left .radius {
  border-bottom-left-radius: 100%;
  bottom: 0;
```

```
}

/* INPUT SLIDERS */

#preview .ui-input-slider {
  margin: 10px;
  position: absolute;
  z-index: 10;
}

#radius-ui-sliders {
  width: 100%;
  height: 100%;
  min-height: 75px;
  min-width: 150px;
  padding: 20px 50px;
  top: -20px;
  left: -50px;
  position: relative;
}

#tlr {
  top: -30px;
  left: -50px;
  display: none;
}

#tlw {
  top: -30px;
  left: 30px;
}

#tlh {
  top: 20px;
  left: -50px;
}

#trr {
  top: -30px;
  right: -50px;
  display: none;
}

#trw {
  top: -30px;
  right: 30px;
}

#trh {
  top: 20px;
  right: -50px;
}

#brr {
  bottom: -30px;
  right: -50px;
  display: none;
}

#brw {
  bottom: -30px;
  right: 30px;
}

#brh {
  bottom: 20px;
  right: -50px;
}

#blr {
  bottom: -30px;
  left: -50px;
  display: none;
```

```
}

#blw {
  bottom: -30px;
  left: 30px;
}

#blh {
  bottom: 20px;
  left: -50px;
}

#preview .ui-input-slider-left,
#preview .ui-input-slider-right {
  visibility: hidden;
}

#preview .ui-input-slider-container:hover .ui-input-slider-left {
  visibility: visible;
}

#preview .ui-input-slider-container:hover .ui-input-slider-right {
  visibility: visible;
}

/*
 *
 */

#unit-selection {
  width: 200px;
  height: 75px;
  margin: 30px 30px 0 0;
  padding: 30px;
  border: 3px solid #555;
  border-radius: 10px;
  position: relative;
  float: right;
}

#unit-selection .info {
  height: 20%;
  width: 100%;
  line-height: 20%;
  font-size: 20px;
  text-align: center;
  position: relative;
  top: 40%;
}

#unit-selection .dropdown {
  width: 50px;
  height: 20px;
  margin: 10px;
  padding: 0;
  border-radius: 3px;
  position: absolute;
  overflow: hidden;
}

#unit-selection select {
  width: 50px;
  height: 20px;
  marign: 0;
  padding: 0 0 0 10px;
  background: #555;
  border: 1px solid #555;
  border: none;
  color: #fff;
  float: left;
}

#unit-selection select option {
```

```
  background: #fff;
  color: #333;
}

#unit-selection select:hover {
  cursor: pointer;
}

#unit-selection .dropdown:before {
  content: "";
  width: 18px;
  height: 20px;
  display: block;
  background-color: #555;
  background-image: url("dropdown.png");
  background-position: center center;
  background-repeat: no-repeat;
  top: 0px;
  right: 0px;
  position: absolute;
  z-index: 1;
  pointer-events: none;
}

#unit-selection .unit-top-left {
  top: 0;
  left: 0;
  display: none;
}

#unit-selection .unit-top-left-w {
  top: -22px;
  left: 30px;
}

#unit-selection .unit-top-left-h {
  top: 20px;
  left: -37px;
}

#unit-selection .unit-top-right {
  top: 0;
  right: 0;
  display: none;
}

#unit-selection .unit-top-right-w {
  top: -22px;
  right: 30px;
}

#unit-selection .unit-top-right-h {
  top: 20px;
  right: -37px;
}

#unit-selection .unit-bottom-right {
  bottom: 0;
  right: 0;
  display: none;
}

#unit-selection .unit-bottom-right-w {
  bottom: -22px;
  right: 30px;
}

#unit-selection .unit-bottom-right-h {
  bottom: 20px;
  right: -37px;
}

#unit-selection .unit-bottom-left {
```

```
  bottom: 0;
  left: 0;
  display: none;
}

#unit-selection .unit-bottom-left-w {
  bottom: -22px;
  left: 30px;
}

#unit-selection .unit-bottom-left-h {
  bottom: 20px;
  left: -37px;
}

/*****************************************************************************/
/*****************************************************************************/

#radius-lock {
  width: 200px;
  height: 75px;
  margin: 30px 0 0 30px;
  padding: 30px;
  border: 3px solid #555;
  border-radius: 10px;
  position: relative;
  float: left;
}

#radius-lock .ui-checkbox {
  color: #fff;
  position: absolute;
}

#radius-lock .ui-checkbox > label {
  height: 20px;
  width: 34px;
  padding: 0;
}

#radius-lock .info {
  height: 20%;
  width: 100%;
  line-height: 20%;
  font-size: 20px;
  text-align: center;
  position: relative;
  top: 40%;
}

#radius-lock [data-topic="top-left"] {
  top: 10px;
  left: 10px;
}

#radius-lock [data-topic="top-right"] {
  top: 10px;
  right: 10px;
}

#radius-lock [data-topic="bottom-right"] {
  bottom: 10px;
  right: 10px;
}

#radius-lock [data-topic="bottom-left"] {
  bottom: 10px;
  left: 10px;
}

/**
 * Controls
 */
```

```
#dimensions {
  width: 200px;
  color: #444;
  float: left;
}

#dimensions input {
  background: #555;
  color: #fff;
  border: none;
  border-radius: 3px;
}

#output {
  width: 500px;
  padding: 10px 0;
  margin: 10px 0;
  color: #555;
  text-align: center;
  border: 1px dashed #999;
  border-radius: 3px;
  -moz-user-select: text;
  -webkit-user-select: text;
  -ms-user-select: text;
  user-select: text;

  float: right;
}
```

```
"use strict";

/**
 * UI-InputSliderManager
 */

var InputSliderManager = (function InputSliderManager() {
  var subscribers = {};
  var sliders = [];

  var InputComponent = function InputComponent(obj) {
    var input = document.createElement("input");
    input.setAttribute("type", "text");

    input.addEventListener("click", function (e) {
      this.select();
    });

    input.addEventListener("change", function (e) {
      var value = parseInt(e.target.value);

      if (isNaN(value) === true) setValue(obj.topic, obj.value);
      else setValue(obj.topic, value);
    });

    subscribe(obj.topic, function (value) {
      input.value = value + obj.unit;
    });

    return input;
  };

  var SliderComponent = function SliderComponent(obj, sign) {
    var slider = document.createElement("div");
    var startX = null;
    var start_value = 0;

    slider.addEventListener("click", function (e) {
      setValue(obj.topic, obj.value + obj.step * sign);
    });

    slider.addEventListener("mousedown", function (e) {
      startX = e.clientX;
```

```
    start_value = obj.value;
    document.body.style.cursor = "e-resize";
    document.addEventListener("mousemove", sliderMotion);
  });

  document.addEventListener("mouseup", function (e) {
    document.removeEventListener("mousemove", sliderMotion);
    document.body.style.cursor = "auto";
    slider.style.cursor = "pointer";
  });

  var sliderMotion = function sliderMotion(e) {
    slider.style.cursor = "e-resize";
    var delta = ((e.clientX - startX) / obj.sensivity) | 0;
    var value = delta * obj.step + start_value;
    setValue(obj.topic, value);
  };

  return slider;
};

var InputSlider = function (node) {
  var min = node.getAttribute("data-min") | 0;
  var max = node.getAttribute("data-max") | 0;
  var step = node.getAttribute("data-step") | 0;
  var value = node.getAttribute("data-value") | 0;
  var topic = node.getAttribute("data-topic");
  var unit = node.getAttribute("data-unit");
  var name = node.getAttribute("data-info");
  var sensivity = node.getAttribute("data-sensivity") | 0;

  this.min = min;
  this.max = max > 0 ? max : 100;
  this.step = step === 0 ? 1 : step;
  this.topic = topic;
  this.node = node;
  this.unit = unit;
  this.sensivity = sensivity > 0 ? sensivity : 5;

  var input = new InputComponent(this);
  var slider_left = new SliderComponent(this, -1);
  var slider_right = new SliderComponent(this, 1);

  slider_left.className = "ui-input-slider-left";
  slider_right.className = "ui-input-slider-right";

  if (name) {
    var info = document.createElement("span");
    info.className = "ui-input-slider-info";
    info.textContent = name;
    node.appendChild(info);
  }

  node.appendChild(slider_left);
  node.appendChild(input);
  node.appendChild(slider_right);
  node.className = "ui-input-slider ui-input-slider-container";

  this.input = input;
  sliders[topic] = this;
  setValue(topic, value);
};

var setValue = function setValue(topic, value, send_notify) {
  var slider = sliders[topic];
  if (slider === undefined) return;

  if (value > slider.max) value = slider.max;
  if (value < slider.min) value = slider.min;

  slider.value = value;
  slider.node.setAttribute("data-value", value);
```

```
    if (send_notify !== undefined && send_notify === false) {
      slider.input.value = value + slider.unit;
      return;
    }

    notify.call(slider);
  };

  var setMax = function setMax(topic, value) {
    var slider = sliders[topic];
    if (slider === undefined) return;

    slider.max = value;
    setValue(topic, slider.value);
  };

  var setMin = function setMin(topic, value) {
    var slider = sliders[topic];
    if (slider === undefined) return;

    slider.min = value;
    setValue(topic, slider.value);
  };

  var setUnit = function setUnit(topic, unit) {
    var slider = sliders[topic];
    if (slider === undefined) return;

    slider.unit = unit;
    setValue(topic, slider.value);
  };

  var getNode = function getNode(topic) {
    return sliders[topic].node;
  };

  var subscribe = function subscribe(topic, callback) {
    if (subscribers[topic] === undefined) subscribers[topic] = [];
    subscribers[topic].push(callback);
  };

  var unsubscribe = function unsubscribe(topic, callback) {
    subscribers[topic].indexOf(callback);
    subscribers[topic].splice(index, 1);
  };

  var notify = function notify() {
    for (var i in subscribers[this.topic]) {
      subscribers[this.topic][i](this.value);
    }
  };

  var init = function init() {
    var elem = document.querySelectorAll(".ui-input-slider");
    var size = elem.length;
    for (var i = 0; i < size; i++) new InputSlider(elem[i]);
  };

  return {
    init: init,
    setMax: setMax,
    setMin: setMin,
    setUnit: setUnit,
    getNode: getNode,
    setValue: setValue,
    subscribe: subscribe,
    unsubscribe: unsubscribe,
  };
})();

/**
 * UI-ButtonManager
 */
```

```
var ButtonManager = (function CheckBoxManager() {
  var subscribers = [];
  var buttons = [];

  var CheckBox = function CheckBox(node) {
    var topic = node.getAttribute("data-topic");
    var state = node.getAttribute("data-state");
    var name = node.getAttribute("data-label");
    var align = node.getAttribute("data-text-on");

    state = state === "true";

    var checkbox = document.createElement("input");
    var label = document.createElement("label");

    var id = "checkbox-" + topic;
    checkbox.id = id;
    checkbox.setAttribute("type", "checkbox");
    checkbox.checked = state;

    label.setAttribute("for", id);
    if (name) {
      label.className = "text";
      if (align) label.className += " " + align;
      label.textContent = name;
    }

    node.appendChild(checkbox);
    node.appendChild(label);

    this.node = node;
    this.topic = topic;
    this.checkbox = checkbox;

    checkbox.addEventListener(
      "change",
      function (e) {
        notify.call(this);
      }.bind(this),
    );

    buttons[topic] = this;
  };

  var getNode = function getNode(topic) {
    return buttons[topic].node;
  };

  var setValue = function setValue(topic, value) {
    try {
      buttons[topic].checkbox.checked = value;
    } catch (error) {
      console.log(error);
    }
  };

  var subscribe = function subscribe(topic, callback) {
    if (subscribers[topic] === undefined) subscribers[topic] = [];

    subscribers[topic].push(callback);
  };

  var unsubscribe = function unsubscribe(topic, callback) {
    subscribers[topic].indexOf(callback);
    subscribers[topic].splice(index, 1);
  };

  var notify = function notify() {
    for (var i = 0; i < subscribers[this.topic].length; i++)
      subscribers[this.topic][i](this.checkbox.checked);
  };
```

```
  var init = function init() {
    var elem = document.querySelectorAll(".ui-checkbox");
    var size = elem.length;
    for (var i = 0; i < size; i++) new CheckBox(elem[i]);
  };

  return {
    init: init,
    setValue: setValue,
    subscribe: subscribe,
    unsubscribe: unsubscribe,
  };
})();

window.addEventListener("load", function () {
  BorderRadius.init();
});

var BorderRadius = (function BorderRadius() {
  function getElemById(id) {
    return document.getElementById(id);
  }

  /**
   * Shadow dragging
   */
  var PreviewMouseTracking = (function Drag() {
    var active = false;
    var lastX = 0;
    var lastY = 0;
    var subscribers = [];

    var init = function init(id) {
      var elem = getElemById(id);
      elem.addEventListener("mousedown", dragStart, false);
      document.addEventListener("mouseup", dragEnd, false);
    };

    var dragStart = function dragStart(e) {
      if (e.button !== 0) return;

      active = true;
      lastX = e.clientX;
      lastY = e.clientY;
      document.addEventListener("mousemove", mouseDrag, false);
    };

    var dragEnd = function dragEnd(e) {
      if (e.button !== 0) return;

      if (active === true) {
        active = false;
        document.removeEventListener("mousemove", mouseDrag, false);
      }
    };

    var mouseDrag = function mouseDrag(e) {
      notify(e.clientX - lastX, e.clientY - lastY);
      lastX = e.clientX;
      lastY = e.clientY;
    };

    var subscribe = function subscribe(callback) {
      subscribers.push(callback);
    };

    var unsubscribe = function unsubscribe(callback) {
      var index = subscribers.indexOf(callback);
      subscribers.splice(index, 1);
    };

    var notify = function notify(deltaX, deltaY) {
      for (var i in subscribers) subscribers[i](deltaX, deltaY);
```

```
  };

  return {
    init: init,
    subscribe: subscribe,
    unsubscribe: unsubscribe,
  };
})();

var subject;
var units = ["px", "%"];
var output = null;

var UnitSelector = function UnitSelector(topic) {
  this.container = document.createElement("div");
  this.select = document.createElement("select");
  for (var i in units) {
    var option = document.createElement("option");
    option.value = i;
    option.textContent = units[i];
    this.select.appendChild(option);
  }

  this.container.className = "dropdown " + "unit-" + topic;
  this.container.appendChild(this.select);
};

UnitSelector.prototype.setValue = function setValue(value) {
  this.salect.value = value;
};

var RadiusContainer = function RadiusContainer(node) {
  var radius = document.createElement("div");
  var handle = document.createElement("div");
  var x = node.getAttribute("data-x");
  var y = node.getAttribute("data-y");
  var active = false;

  this.id = node.id;
  this.node = node;
  this.radius = radius;
  this.handle = handle;
  this.width = 100;
  this.height = 50;
  this.size = 0;
  this.rounded = false;

  this.unitX = 0;
  this.unitY = 0;
  this.unitR = 0;

  this.maxW = 100;
  this.maxH = 100;
  this.maxR = 100;

  this.topic = y + "-" + x;

  var sliderW = InputSliderManager.getNode(this.topic + "-w");
  var sliderH = InputSliderManager.getNode(this.topic + "-h");
  var sliderR = InputSliderManager.getNode(this.topic);

  this.setUnitX(this.unitX);
  this.setUnitY(this.unitY);
  this.setUnitR(this.unitR);

  this.updateWidth();
  this.updateHeight();
  this.updateRadius();

  if (x === "left") this.resizeX = 1;
  if (x === "right") this.resizeX = -1;
  if (y === "top") this.resizeY = 1;
  if (y === "bottom") this.resizeY = -1;
```

```
radius.className = "radius";

var unit_selector = document.getElementById("unit-selection");
var unitW = new UnitSelector(this.topic + "-w");
var unitH = new UnitSelector(this.topic + "-h");
var unitR = new UnitSelector(this.topic);

unit_selector.appendChild(unitW.container);
unit_selector.appendChild(unitH.container);
unit_selector.appendChild(unitR.container);
node.appendChild(radius);
subject.appendChild(handle);

unitW.select.addEventListener(
  "change",
  function (e) {
    this.setUnitX(e.target.value | 0);
  }.bind(this),
);

unitH.select.addEventListener(
  "change",
  function (e) {
    this.setUnitY(e.target.value | 0);
  }.bind(this),
);

unitR.select.addEventListener(
  "change",
  function (e) {
    this.setUnitR(e.target.value | 0);
  }.bind(this),
);

if (x === "left" && y == "top") handle.className = "handle handle-top-left";
if (x === "right" && y == "top")
  handle.className = "handle handle-top-right";
if (x === "right" && y == "bottom")
  handle.className = "handle handle-bottom-right";
if (x === "left" && y == "bottom")
  handle.className = "handle handle-bottom-left";

handle.addEventListener(
  "mousedown",
  function (e) {
    active = true;
    this.radius.style.display = "block";
    PreviewMouseTracking.subscribe(this.updateContainer.bind(this));
  }.bind(this),
);

document.addEventListener(
  "mouseup",
  function (e) {
    this.radius.style.display = "none";
    if (active === true)
      PreviewMouseTracking.unsubscribe(this.updateContainer.bind(this));
  }.bind(this),
);

InputSliderManager.subscribe(this.topic + "-w", this.setWidth.bind(this));
InputSliderManager.subscribe(this.topic + "-h", this.setHeight.bind(this));
InputSliderManager.subscribe(this.topic, this.setRadius.bind(this));

ButtonManager.subscribe(
  this.topic,
  function (value) {
    this.rounded = value;
    if (value === true) {
      unitW.container.style.display = "none";
      unitH.container.style.display = "none";
      unitR.container.style.display = "block";
```

```
        sliderW.style.display = "none";
        sliderH.style.display = "none";
        sliderR.style.display = "block";
        this.setUnitR(this.unitR);
        this.updateRadius();
      }

      if (value === false) {
        unitW.container.style.display = "block";
        unitH.container.style.display = "block";
        unitR.container.style.display = "none";
        sliderW.style.display = "block";
        sliderH.style.display = "block";
        sliderR.style.display = "none";
        this.setUnitX(this.unitX);
        this.setUnitY(this.unitY);
        this.updateWidth();
        this.updateHeight();
      }

      this.updateBorderRadius();
    }.bind(this),
  );

  this.updateBorderRadius();
};

RadiusContainer.prototype.updateWidth = function updateWidth() {
  this.node.style.width = this.width + units[this.unitX];
  var value = Math.round(this.width / 2);
  InputSliderManager.setValue(this.topic + "-w", value, false);
};

RadiusContainer.prototype.updateHeight = function updateHeight() {
  this.node.style.height = this.height + units[this.unitY];
  var value = Math.round(this.height / 2);
  InputSliderManager.setValue(this.topic + "-h", value, false);
};

RadiusContainer.prototype.updateRadius = function updateRadius() {
  var value = Math.round(this.size / 2);
  this.node.style.width = this.size + units[this.unitR];
  this.node.style.height = this.size + units[this.unitR];
  InputSliderManager.setValue(this.topic, value, false);
};

RadiusContainer.prototype.setWidth = function setWidth(value) {
  this.radius.style.display = "block";
  this.width = 2 * value;
  this.node.style.width = this.width + units[this.unitX];
  this.updateBorderRadius();
};

RadiusContainer.prototype.setHeight = function setHeight(value) {
  this.radius.style.display = "block";
  this.height = 2 * value;
  this.node.style.height = this.height + units[this.unitY];
  this.updateBorderRadius();
};

RadiusContainer.prototype.setRadius = function setRadius(value) {
  this.radius.style.display = "block";
  this.size = 2 * value;
  this.node.style.width = this.size + units[this.unitR];
  this.node.style.height = this.size + units[this.unitR];
  this.updateBorderRadius();
};

RadiusContainer.prototype.setUnitX = function setUnitX(value) {
  this.unitX = value;
  if (this.unitX === 0) this.maxW = 2 * subject.clientWidth;
  if (this.unitX === 1) this.maxW = 200;
  InputSliderManager.setUnit(this.topic + "-w", units[this.unitX]);
```

```
  InputSliderManager.setMax(this.topic + "-w", this.maxW / 2);
};

RadiusContainer.prototype.setUnitY = function setUnitY(value) {
  this.unitY = value;
  if (this.unitY === 0) this.maxH = 2 * subject.clientHeight;
  if (this.unitY === 1) this.maxH = 200;
  InputSliderManager.setUnit(this.topic + "-h", units[this.unitY]);
  InputSliderManager.setMax(this.topic + "-h", this.maxH / 2);
};

RadiusContainer.prototype.setUnitR = function setUnitR(value) {
  this.unitR = value;

  if (this.unitR === 0)
    this.maxR = 2 * Math.min(subject.clientHeight, subject.clientWidth);

  if (this.unitR === 1) this.maxR = 200;

  InputSliderManager.setUnit(this.topic, units[this.unitR]);
  InputSliderManager.setMax(this.topic, this.maxR / 2);
};

RadiusContainer.prototype.updateUnits = function updateUnits(unit) {
  if (this.rounded) {
    this.setUnitR(this.unitR);
    return;
  }

  if (unit === 0) this.setUnitX(this.unitX);

  if (unit === 1) this.setUnitY(this.unitY);
};

RadiusContainer.prototype.composeBorderRadius =
  function composeBorderRadius() {
    if (this.rounded === true) {
      var unit = units[this.unitR];
      var value = Math.round(this.size / 2);
      return value + unit;
    }

    var unitX = units[this.unitX];
    var unitY = units[this.unitY];
    var valueX = Math.round(this.width / 2);
    var valueY = Math.round(this.height / 2);

    if (valueX === valueY && this.unitX === this.unitY) return valueX + unitX;

    return valueX + unitX + " " + valueY + unitY;
  };

RadiusContainer.prototype.updateBorderRadius = function updateBorderRadius() {
  var radius = this.composeBorderRadius();
  var corner = 0;

  if (this.topic === "top-left") {
    subject.style.borderTopLeftRadius = radius;
    corner = 0;
  }

  if (this.topic === "top-right") {
    subject.style.borderTopRightRadius = radius;
    corner = 1;
  }

  if (this.topic === "bottom-right") {
    subject.style.borderBottomRightRadius = radius;
    corner = 2;
  }

  if (this.topic === "bottom-left") {
    subject.style.borderBottomLeftRadius = radius;
```

```
    corner = 3;
  }

  Tool.updateOutput(corner, radius);
};

RadiusContainer.prototype.updateContainer = function updateContainer(
  deltaX,
  deltaY,
) {
  if (this.rounded === true) {
    this.size += this.resizeX * deltaX + this.resizeY * deltaY;
    if (this.size < 0) this.size = 0;
    if (this.size > this.maxR) this.size = this.maxR;
    this.updateRadius();
    this.updateBorderRadius();
    return;
  }

  if (deltaX) {
    this.width += this.resizeX * deltaX;
    if (this.width < 0) this.width = 0;
    if (this.width > this.maxW) this.width = this.maxW;
    this.updateWidth();
  }

  if (deltaY) {
    this.height += this.resizeY * deltaY;
    if (this.height < 0) this.height = 0;
    if (this.height > this.maxH) this.height = this.maxH;
    this.updateHeight();
  }

  if (deltaX || deltaY) this.updateBorderRadius();
};

/**
 * Tool Manager
 */
var Tool = (function Tool() {
  var preview;
  var preview_ui;
  var radius_containers = [];
  var border_width = 3;
  var borders1 = [null, null, null, null];
  var borders2 = [0, 0, 0, 0];

  var updateUIWidth = function updateUIWidth(value) {
    var pwidth = subject.parentElement.clientWidth;
    var left = (pwidth - value) / 2;
    subject.style.width = value + "px";

    for (var i = 0; i < 4; i++) radius_containers[i].updateUnits(0);
  };

  var updateUIHeight = function updateUIHeight(value) {
    var pheight = subject.parentElement.clientHeight;
    var top = (pheight - value) / 2;
    subject.style.height = value + "px";
    subject.style.top = top - border_width + "px";

    for (var i = 0; i < 4; i++) radius_containers[i].updateUnits(1);
  };

  var updatePreviewUIWidth = function updatePreviewUIWidth() {
    var p = subject.parentElement.clientWidth;
    var v = preview_ui.clientWidth;
    console.log(p, v, (p - v) / 2);
    preview_ui.style.left = (p - v) / 2 + "px";
  };

  var updatePreviewUIHeight = function updatePreviewUIHeight() {
    var p = subject.parentElement.clientHeight;
```

```
      var v = preview_ui.clientHeight;
      console.log(p, v, (p - v) / 2);
      preview_ui.style.top = (p - v) / 2 + "px";
    };

    var updateOutput = function updateOutput(corner, radius) {
      var values = radius.split(" ");

      borders1[corner] = values[0];
      borders2[corner] = values[0];

      if (values.length === 2) borders2[corner] = values[1];

      var border_1_value = borders1.join(" ");
      var border_2_value = borders2.join(" ");
      var border_radius = "border-radius: " + border_1_value;

      if (border_2_value !== border_1_value)
        border_radius += " / " + border_2_value;

      border_radius += ";";
      output.textContent = border_radius;
    };

    var init = function init() {
      preview = getElemById("preview");
      subject = getElemById("subject");
      output = getElemById("output");
      preview_ui = getElemById("radius-ui-sliders");

      var elem = document.querySelectorAll(".radius-container");
      var size = elem.length;
      for (var i = 0; i < size; i++)
        radius_containers[i] = new RadiusContainer(elem[i]);

      InputSliderManager.subscribe("width", updateUIWidth);
      InputSliderManager.subscribe("height", updateUIHeight);

      InputSliderManager.setValue("width", subject.clientWidth);
      InputSliderManager.setValue("height", subject.clientHeight);
    };

    return {
      init: init,
      updateOutput: updateOutput,
    };
  })();

  /**
   * Init Tool
   */
  var init = function init() {
    ButtonManager.init();
    InputSliderManager.init();
    PreviewMouseTracking.init("preview");
    Tool.init();
  };

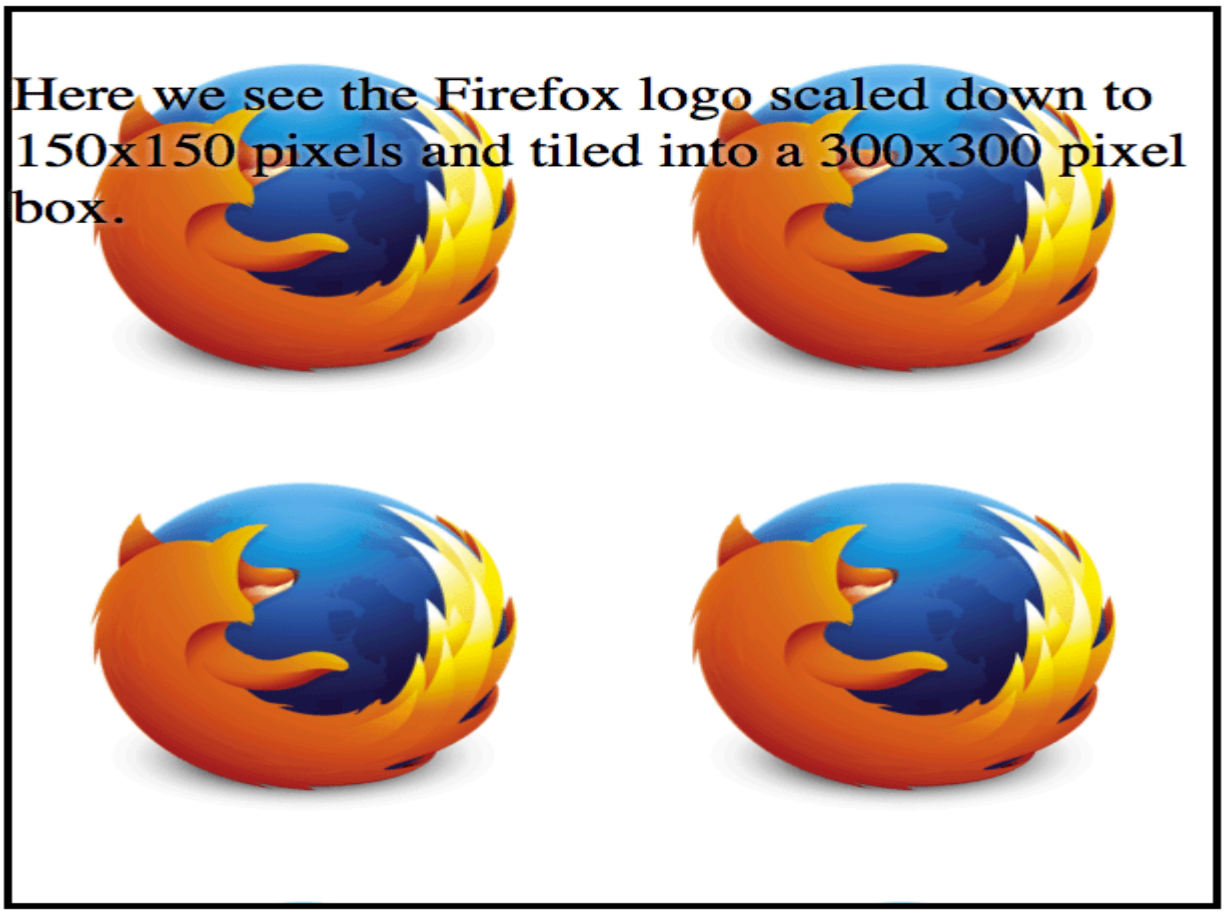
  return {
    init: init,
  };
})();
```

Alterando a escala das imagens de background

A propriedade CSS `background-size` possibilita o ajuste das imagens do background, ao invés do comportamento padrão do navegador de mostrar a imagem no seu tamanho real. Você pode tanto aumentar como diminuir a imagem.

Duplicando uma imagem grande

Vamos considerar uma imagem grande, a image da logo do Firefox com 2982x2808 . Nós queremos (por alguma razão, envolvendo um site com um design ruim) quatro cópia desta imagem em um quadrado de 300x300 pixel, resultando nesse visual:



Isto pode ser conseguido usando o seguinte CSS:

```
.square {
  width: 300px;
  height: 300px;
  background-image: url(firefox_logo.png);
  border: solid 2px;
  text-shadow: white 0px 0px 2px;
  font-size: 16px;
  background-size: 150px;
}
```

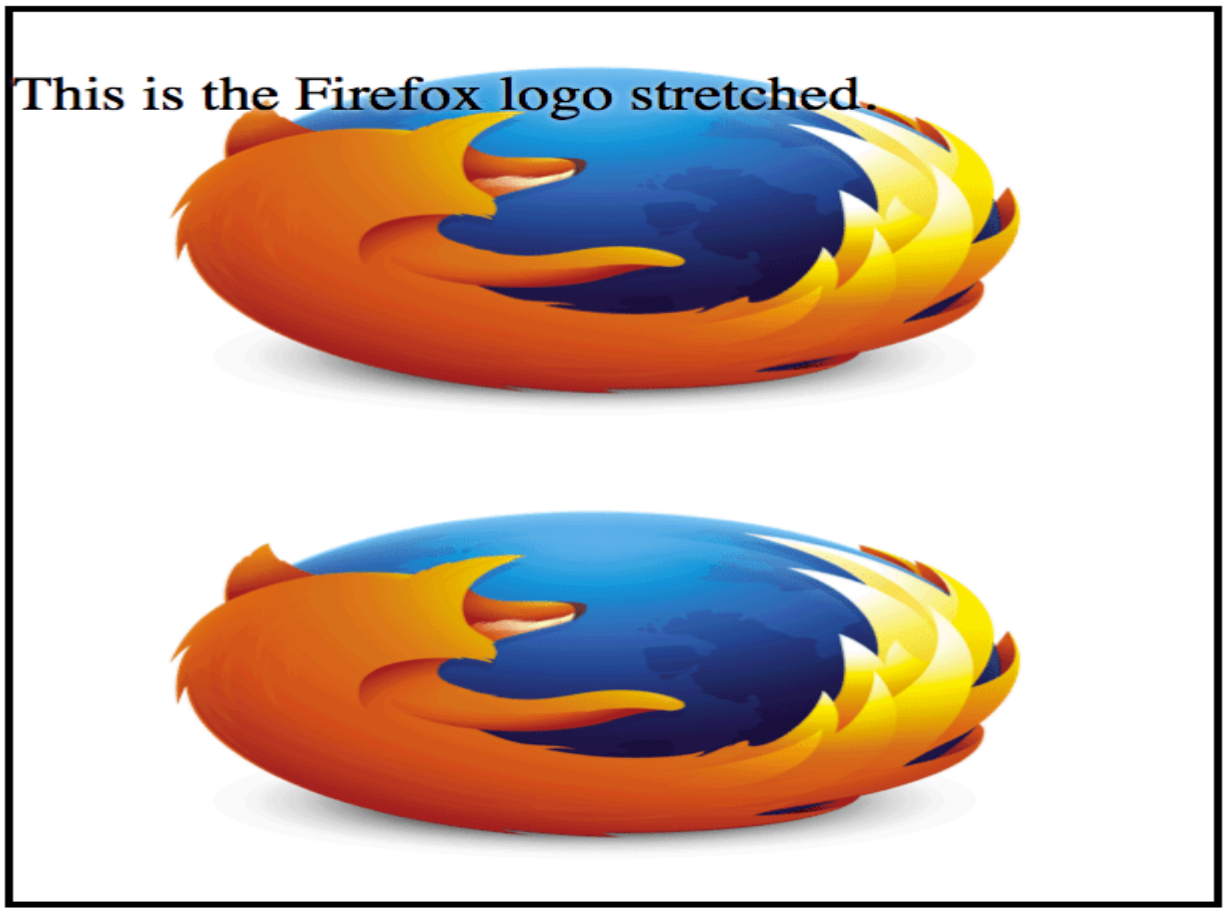
O `background-size` não precisa mais de nenhum prefixo, mas você pode considerar a adição de uma versão pré-fixada se você está focando em browsers muito antigos.

Esticando uma imagem

Você também pode especificar ambos os tamanhos, horizontal e vertical da imagem, assim:

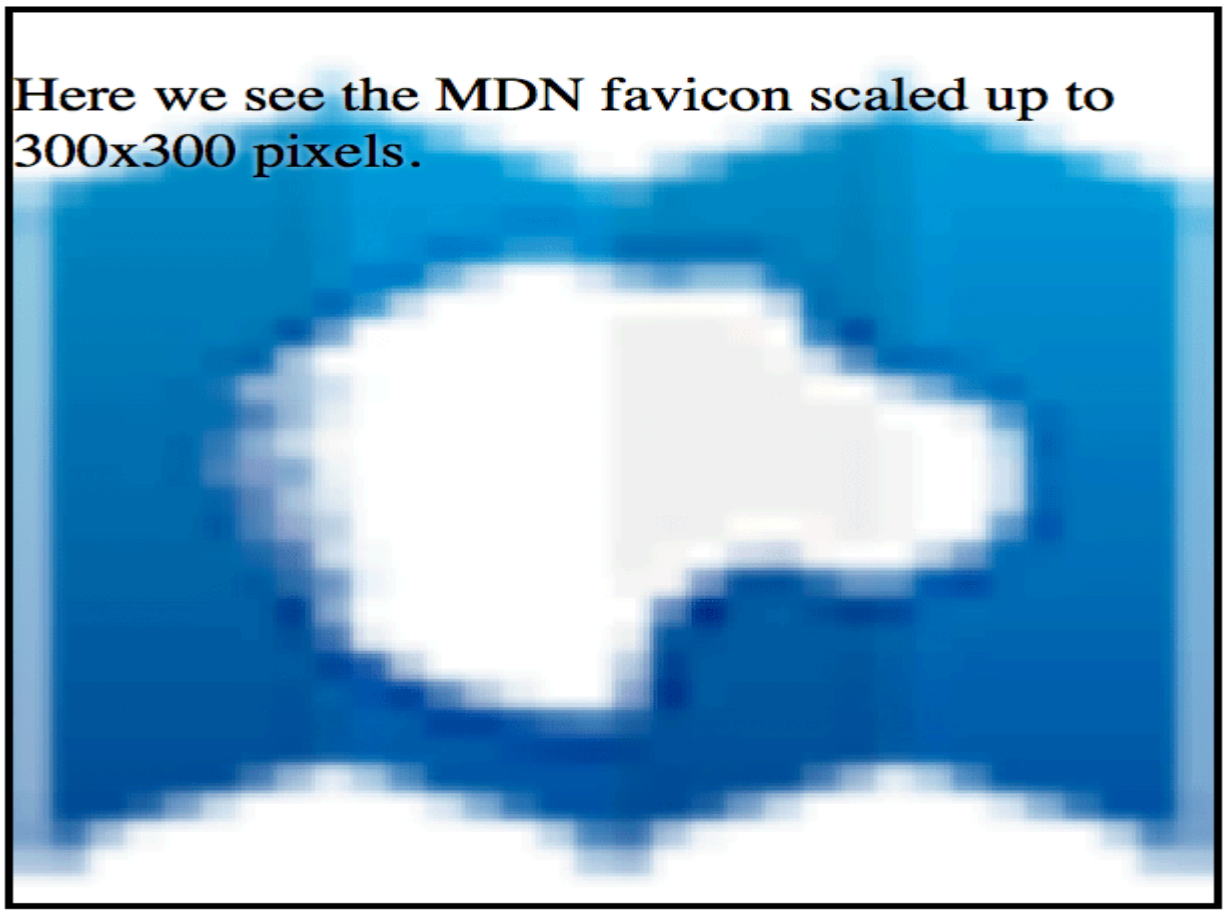
```
background-size: 300px 150px;
```

O resultado fica assim:



Aumentando escala de uma imagem

Na outra extremidade do espectro, é possível dimensionar-se uma imagem no fundo. Aqui nós aumentamos a escala de um favicon de pixel 32x32 para 300x300 pixels:



```
.square2 {
  width: 300px;
  height: 300px;
  background-image: url(favicon.png);
  background-size: 300px;
  border: solid 2px;
  text-shadow: white 0px 0px 2px;
  font-size: 16px;
}
```

Como você pode ver, o CSS é, na verdade, essencialmente idêntico, salvo o nome do arquivo de imagem.

Valores especiais: "contain" e "cover"

Da mesma maneira que o `<length>`, a propriedade CSS de `background-size` oferece dois valores de tamanho especial, contain e cover. Vamos dar uma olhada nestes.

<code>contain</code>

O valor contain especifica que, independentemente do tamanho da caixa que contém, a imagem de fundo deve ser dimensionado de modo a que cada lado seja tão grande quanto possível ao mesmo tempo que não exceda o comprimento do lado correspondente do recipiente. Tente redimensionar a janela usando um navegador que suporta imagens de fundo de escala (como o Firefox 3.6 ou posterior) para ver isso em ação no exemplo vivo abaixo.

```
<div class="bgSizeContain">
  <p>Tente redimensionar a janela e ver o que acontece.</p>
</div>
```

```
.bgSizeContain {
  height: 200px;
  background-image: url(firefox_logo.png);
  background-size: contain;
  border: 2px solid darkgray;
  color: #000;
  text-shadow: 1px 1px 0 #fff;
}
```

<code>cover</code>

O valor cover especifica que a imagem de fundo deve ser dimensionado de modo que seja tão pequena quanto possível ao mesmo tempo assegurar que ambas as dimensões são maiores do que ou igual à dimensão correspondente do recipiente.

Os exemplos à seguir usam HTML & CSS:

```
<div class="bgSizeCover">
  <p>Tente redimensionar a janela e ver o que acontece.</p>
</div>
```

```
.bgSizeCover {
  height: 200px;
  background-image: url('firefox_logo.png');
  background-size: cover;
  border: 2px solid darkgray;
  color: #000; text-shadow: 1px 1px 0 #fff;
```

Veja Também

- `background-size`
- `background`

Multiple backgrounds

Com CSS3, você pode aplicar aos elementos multiplos planos de fundo. Estes ficam em camadas empilhadas uma acima da outra onde o o primeiro fundo dado será desenhado no topo e apenas o último fundo da lista poderá definir uma cor sólida de fundo.

Especificar planos de fundo múltplos é fácil:

```
.minhaClasse {
  background: fundo1, fundo2, ..., fundoN;
}
```

Você pode fazer isso com a propriedade reduzida `background` e também com as propriedade individuais, com a excessão de `background-color`. Isto é, as seguintes propriedades de plano de fundo podem ser especificadas com uma lista, uma por fundo: `background`, `background-attachment`, `background-clip`, `background-image`, `background-origin`, `background-position`, `background-repeat`, `background-size`.

Exemplo

Neste exemplo, três planos de fundos estão empilhados: o logo do Firefox, um degradê linear, e uma imagem com flores:

```
.multi_bg_example {
  background:
    url(http://demos.hacks.mozilla.org/openweb/resources/images/logos/firefox-48.png),
    -moz-linear-gradient(left, rgba(255, 255, 255, 0), rgba(255, 255, 255, 1)),
    url(http://demos.hacks.mozilla.org/openweb/resources/images/patterns/flowers-pattern.jpg);
  background-repeat: no-repeat, no-repeat, repeat;
  background-position:
    bottom right,
    left,
    right;
}
```

Captura de Tela	Demonstração
	

Como pode ver, o logo do firefox (listado primeiro) está no topo, seguido do gradiente que está uma camada acima do fundo florido. Cada uma das sub-propriedade subsequente, (`background-repeat` e `background-position`) se aplicam aos fundos correspondentes. Então o primeiro valor para `background-repeat` se aplica ao primeiro plano de fundo (o mais da frente), e assim por adiante.

Veja também

- Usando degradês

CSS Basic User Interface

CSS Basic User Interface é um módulo CSS que permite definir a renderização e a funcionalidade de recursos relacionados à interface do usuário.

Referência

Propriedades

- `appearance`
- `box-sizing`
- `cursor`
- `ime-mode`
- `nav-down`
- `nav-left`
- `nav-right`
- `nav-up`
- `outline`
- `outline-width`
- `outline-style`
- `outline-color`
- `outline-offset`
- `resize`
- `text-overflow`
- `user-select`

Guias

Usando valores URL na propriedade `cursor`

> Explica como uma URL pode ser usada com a propriedade `cursor` para criar cursores personalizados.

CSS Box Model

CSS Box Model (Modelo de Caixa CSS) é um módulo de CSS que define caixas retangulares, incluindo preenchimento (padding) e margem (margin), que são gerados por elementos e dispostos de acordo com o modelo de formato visual.

Referência

Propriedades

Propriedades que controlam o fluxo de conteúdo em uma caixa

- `box-decoration-break`
- `box-sizing`
- `overflow`
- `overflow-x`
- `overflow-y`

Propriedades que controlam o tamanho de uma caixa

- `height`
- `width`
- `max-height`
- `max-width`
- `min-height`
- `min-width`

Propriedades que controlam as margens de uma caixa

- `margin`
- `margin-bottom`
- `margin-left`
- `margin-right`
- `margin-top`

Propriedades que controlam o preenchimento (padding) de uma caixa

- `padding`
- `padding-bottom`
- `padding-left`
- `padding-right`
- `padding-top`

Outras propriedades

- `box-shadow`
- `visibility`

Guias e ferramentas

Introdução ao modelo de caixa CSS

> Explicações de uma concepção fundamental no CSS, o modelo de caixa: descreve o sentido de margem, preenchimento, tambem como as diferentes áreas da caixa.

Dominando o colapso da margem

> Na maioria dos casos, duas margens adjacentes são colapsadas em uma. Este artigo descreve quando isso acontecwe e como controlar isso.

Gerador de sombra de caixa

> Uma ferramenta interativa que permite criar sombras visualmente e gerar a sintaxe necessária para a propriedade `box-shadow`.

Box model

Resumo

Em uma página WEB, cada elemento é representado como um *box* retangular. Determinar o tamanho, propriedades - como sua cor, fundo, estilo das bordas - e a posição desses *boxes* é o objetivo do mecanismo de renderização.

No CSS, cada um desses *boxes* retangulares é descrita usando o *box model* padrão. Este modelo descreve o conteúdo do espaço ocupado por um elemento. Cada box possui 4 edges: **margin edge**, **border edge**, **padding edge** e **content edge**.

A **área de conteúdo** (content area) é a área ocupada pelo conteúdo real do elemento. Ele frequentemente possui um fundo, uma cor de fonte ou uma imagem (nessa ordem, uma imagem opaca esconde a cor de fundo) e é localizada dentro do *content edge*; suas dimensões são a largura do conteúdo, ou largura do box de conteúdo, e altura do conteúdo, ou altura do box de conteúdo.

Se a propriedade CSS `box-sizing` está configurada como padrão, as propriedades CSS `width`, `min-width`, `max-width`, `height`, `min-height` e `max-height` controlam o tamanho do conteúdo.

A **área de preenchimento** (padding area) estende-se para a borda em torno do enchimento. Quando a área de conteúdo tem um fundo, cor ou imagem, isso será estendido para a área de preenchimento, por esse motivo, você pode pensar o preenchimento como a extensão do conteúdo. O preenchimento está localizado dentro do *padding edge,* e suas dimensões são a *largura do padding-box* e a *altura do padding-box.*

O espaço entre os edges de preenchimento e conteúdo podem ser controlados utilizando as seguintes propriedades CSS `padding-top`, `padding-right`, `padding-bottom`, `padding-left` e na forma generalizada `padding`.

A **área de borda** (border area**)** estende a área de preenchimento para a área que contém as bordas. Esta é a área de dentro do *border edge*, e suas dimensões são a largura e a altura do *border-box.* Esta área depende do tamanho da borda que está definido pela propriedade `border-width` ou pela propriedade `border`.

A **área de margem** (margin area) estende a área de borda com um espaço vazio utilizado para separar o elemento dos elementos vizinhos. Esta é a área de dentro do *margin edge,* e suas dimensões são a largura e a altura do *margin-box*.

O tamanho da área de margem é controlada utilizando as seguintes propriedades CSS `margin-top`, `margin-right`, `margin-bottom`, `margin-left` e na forma generalizada `margin`.

Quando ocorre um colapso de margens, a área de margem não está claramente definida, uma vez que as margens são compartilhadas entre os boxes.

Finalmente, note que, para elementos não substituídos inline, o total de espaço ocupado (para a altura da linha) é determinado pela propriedade `line-height`, mesmo que a borda e o padding aparecerem visualmente em torno do conteúdo.

Veja também

- Referência CSS
- Propriedades CSS relacionadas: `box-sizing`, `background-clip`, `height`, `max-height`, `min-height`, `width`, `max-height`, `min-height`, `padding`, `padding-top`, `padding-right`, `padding-bottom`, `padding-left`, `border`, `border-top`, `border-right`, `border-bottom`, `border-left`, `border-width`, `border-top-width`, `border-right-width`, `border-bottom-width`, `border-left-width`, `margin`, `margin-top`, `margin-right`, `margin-bottom`, `margin-left`

Dominando margin collapsing

As margens superior e inferior dos blocos às vezes são combinadas(colapsadas/reduzidas) para uma única margem cujo o tamanho é a maior das margens (se os elementos tiverem a mesma margem, uma delas não será somada), combinado a ele, um comportamento conhecido como **margin collapsing**. Note que as margens de elementos flutuantes e posicionados de forma absoluta nunca colapsam.

Margin collapsing ocorre em três casos básicos:

Irmãos adjacentes

> As margens de irmãos adjacentes são colapsadas (execeto quando o último irmão precisar ser limpado devido ao uso de floats em elementos anteriores). Por exemplo:
>
> ```
> A margem inferior deste parágrafo é colapsada...
> ... com margem superior deste parágrafo.
> ```

Pai e primeiro/último filho

> Se não houver border, padding, parte inline, contexto de formatação de bloco criado ou livre para separar o `margin-top` de um bloco do `margin-top` do seu primeiro bloco filho, ou nenhum border, padding, conteúdo inline, `height`, `min-height`, ou `max-height` para separar o `margin-bottom` de um bloco do `margin-bottom` do seu último filho, então essas margens colapsam. A margem colapsada acaba fora do pai.

Blocos vázios

> Se não houver border, padding, conteúdo inline, `height`, ou `min-height` para separar um bloco `margin-top` de sua `margin-bottom`, então as margens superior e inferior são colapsadas.

Algumas coisas podem ser observadas:

- Margin collapsing mais complexo (de mais de duas margens) ocorrem quando esses casos são combinados.
- Essas regras se aplicam até mesmo a margens que são zero, portanto a margem e um primeiro/último filho termina fora de seu pai(de acordo com as regras acima) sendo a margem do pai zero ou não.
- Quando margens negativas são envolvidas, o tamanho da margem colapsada é a soma da maior margem positiva e da menor margem negativa(a mais negativa).
- Se ambos são negativos, o valor negativo maior será usado. Essa definição se aplica a elementos adjacentes e a elementos aninhados.

Margens de elementos flutuantes e posicionados de forma absoluta nunca colapsam.

Veja Também

- CSS Reference

Herança

Resumo

O resumo de cada definição de propriedade do CSS diz se tal propriedade é herdada por padrão ("Inherited": sim) ou é não herdada por padrão ("Inherited": não). Isso controla o que ocorre quando nenhum valor é especificado para uma propriedade em um elemento.

Propriedades herdadas

Quando nenhum valor de uma **propriedade herdada** foi especificado em um elemento, o elemento obtém o valor computado desta propriedade em seu elemento pai. Apenas o elemento raiz do documento obtém o valor inicial dado no resumo da propriedade.

Um exemplo típico de uma propriedade herdada é a propriedade `color`. Dadas as seguintes regras de estilo:

```
p {
  color: green;
}
```

e o markup:

```
<p>Este paragrafo tem <em>texto enfatizado</em> dentro dele.</p>
```

As palavras "texto enfatizado" vão aparecer em verde, já que o elemento `em` herdou o valor da propriedade `color` do elemento `p`. Ele *não* obtém o valor inicial da propriedade (o qual é a cor usada pelo elemento raiz quando a página não especifica nenhuma cor).

Propriedades não herdadas

Quando nenhum valor para uma **propriedade não herdada** (algumas vezes chamada de **reset property** em código Mozilla) é especificado em um elemento, o elemento obtém o valor inicial desta propriedade (como especificado no resumo da propriedade).

Um exemplo típico de uma propriedade não herdada é a propriedade `border`. Dadas as seguintes regras de estilo:

```
p {
  border: medium solid;
}
```

e o markup

```
<p>Este paragrafo tem <em>texto enfatizado</em> dentro dele.</p>
```

As palavras "texto enfatizado" não vão ter uma borda (já que o valor inicial de `border-style` é `none`).

Notas

A palavra-chave `inherit` permite aos autores especificar explicitamente a herança. Ela funciona tanto em propriedades herdadas como não herdadas.

Veja também

- Referência CSS
- `inherit`

Valor atual

The **actual value** of a CSS property is the used value of that property after any necessary approximations have been applied. For example, a user agent that can only render borders with a whole-number pixel width may round the thickness of the border to the nearest integer.

Veja também

- CSS Reference

Propriedades shorthand

Definição

Propriedades shorthand (tradução literal: Propriedades abreviadas) são propriedades de CSS que deixam você setar os valores de outras propriedades de CSS simultaneamente. Usando shorthand property, um desenvolvedor web pode escrever folhas de estilo mais concisas e legíveis,economizando tempo e energia.

A especificação CSS define como shorthand properties, o grupo de propriedades com o mesmo tema. Ex: a propriedade CSS `background` é uma propriedade shorthand que é capaz de definir o valor de `background-color`, `background-image`, `background-repeat`, e `background-position`. Assim também,como as propriedades comuns e relacionadas a fonte podem ser definidas e usadas com o shorthand `font`,e os diferentes margins em volta de um box podem ser definidas com o shorthand `margin`.

Casos específicos

Mesmo sendo convenientes para se usar, existem alguns casos que precisam ser analisados antes de usar:

1. Um valor que não especificado é setado seu valor inicial. Isso parece uma anedota, mas realmente significa que sobrescreve valores previamente setados. Por exemplo:

   ```
   background-color: red;
   background: url(images/bg.gif) no-repeat top right;
   ```

   não irá setar a cor do background para vermelho, mas para `background-color`'s padrão, transparente, como a segunda regra tem precedência.

2. Apenas propriedades individuais são herdadas. Como valores que faltam são substituidos pelo seu valor inicial, é impossivel herdar valores individuais os omitindo. O valor `inherit` pode ser aplicado a uma propriedade, mas num total e não como uma palavra-chave para um valor ou outro. Isso significa que a unica forma de um valor específico ser herdado é usando a propriedade em longhand com o valor `inherit`.

3. Propriedades shorthand tentam não forçar uma ordem específica para os valores das propriedades que elas substituem. Isso funciona bem quando essas propriedades usam valores de diferentes tipos, como a ordem não tem importância, isso funciona bem quando essas propriedades usam valores de diferentes tipos, como a ordem não tem importância, mas isso não funciona tão facilmente quando várias propriedades podem ter valores idênticos. O tratamento destes casos está agrupado em várias categorias:

   1. Propriedades shorthand que lidam com valores da borda de um box, como `border-style`, `margin` or `padding`, sempre usam um consistente 1-para-4 valor de sintaxe representando essas bordas:

	***A sintaxe de um valor*: `border-width: 1em` — O valor único representa todas as bordas**
	As sintaxes de dois valores: `border-width: 1em 2em` — O primeiro valor representa a vertical, isto é, as bordas superior e inferior, a segunda as horizontais, que é a esquerda e a direita.
	As sintaxes de três valores: `border-width: 1em 2em 3em` — O primeiro valor representa a borda superior, a segunda, a horizontal, que é esquerda e direita, e o terceiro valor a borda inferior
	As sintaxes de quatro valores: `border-width: 1em 2em 3em 4em` — Os quatro valores representam as margens superior, direita, inferior e esquerda, respectivamente, sempre nessa ordem, que é no sentido horário começando pelo topo (a letra inicial de Top-Right-Bottom-Left corresponde à ordem da consoante da palavra *trouble*: TRBL)

   2. Também parecido, shorthand que lidam com os cantos de um box `border-radius`, sempre usam 1-para-4-valor de sintaxe representando esses cantos:

	***A sintaxe de um valor*: `border-radius: 1em` — O valor único representa todos os cantos**
	As sintaxes de dois valores: `border-radius: 1em 2em` — O primeiro valor representa o canto superior esquerdo e inferior direito, o segundo o superior direito e inferior esquerdo
	As sintaxes de três valores: `border-radius: 1em 2em 3em` — O primeiro valor representa o canto superior esquerdo, o segundo o canto superior direito eo inferior esquerdo e o terceiro o canto inferior direito
	As sintaxes de quatro valores: `border-radius: 1em 2em 3em 4em` — Os quatro valores representam os cantos superior esquerdo, superior direito, inferior direito e inferior esquerdo respectivamente, sempre nessa ordem, que é no sentido horário começando no canto superior esquerdo.

Propriedades de background

Um background com as seguintes declarações:

```
background-color: #000;
background-image: url(images/bg.gif);
background-repeat: no-repeat;
background-position: top right;
```

Pode ser resumido em uma unica declaração:

```
background: #000 url(images/bg.gif) no-repeat top right;
```

(A forma shorthand é o equivalente as propriedades longhand acima, mais `background-attachment: scroll`, em CSS3, algumas propriedades adicionais.)

Propriedades de fonte

As seguintes declarações:

```
font-style: italic;
font-weight: bold;
font-size: .8em;
line-height: 1.2;
font-family: Arial, sans-serif;
```

Podem ser resumidas a:

```
font: italic bold .8em/1.2 Arial, sans-serif;
```

Essa declaração shorthand é o equivalente as declarações longhand acima, mais `font-variant: normal` e `font-size-adjust: none` (CSS2.0 / CSS3), `font-stretch: normal` (CSS3).

Propriedades de borda

Com a borda, o width (largura), color (cor), podem ser simplificados em uma declaração. Por exemplo,

```
border-width: 1px;
border-style: solid;
border-color: #000;
```

Podem ser escritas como

```
border: 1px solid #000;
```

Propriedades de margin e padding

As versões shorthand de margin e padding funcionam da mesma maneira. As seguintes declarações CSS:

```
margin-top: 10px;
margin-right: 5px;
margin-bottom: 10px;
margin-left: 5px;
```

são as mesmas da seguinte declaração (note que os valores estão em ordem horária: top, right, bottom e left(TRBL, mesmas consoantes de TRBL "trouble"))

```
margin: 10px 5px 10px 5px;
```

Veja também

- CSS Reference
- Propriedades shorthand : `background`, `font`, `margin`, `border`, `border-top`, `border-right`, `border-bottom`, `border-left`, `border-width`, `border-color`, `border-style`, `transition`, `animation`, `transform`, `padding`, `list-style`, `border-radius`.

Especificidade

O Conceito

A especificação é a maneira de como os navegadores definem quais valores de propriedades são os mais relevantes para o elemento a ser utilizado. A especificação é baseada apenas nas regras impostas na composição de diferentes tipos de seletores.

Como isso é calculado?

A espeficicação é calculada na concatenação da contagem de cada tipo de seletor. Não é um peso aplicado na expressão correspondente.

No caso de igualdade de especificação, a última declaração encontrada no CSS é aplicada ao elemento.

> **Nota:** O fato de elementos estarem próximos na árvore do documento não tem efeito sobre a especificação.

Ordem crescente de especificação

A seguinte lista de seletores está incluida na especificação:

- Seletores Universais
- Tipo de Seletores
- Classes seletoras
- Atributos Seletores
- Pseudo-classes
- Seletores ID
- Estilo Inline

A exceção <code>!important</code>

Quando a regra `!important` é utilizada na declaração do estilo substitui qualquer outra declaração feita no CSS, onde quer que esteja na lista de declaração. Contudo, `!important` não tem nada a ver com especificação.

A exceção <code>:not</code>

A pseudo-classe de negação `:not` não é considerada uma pseudo-classe no cálculo de especificação. Contudo, seletores colocados na pseudo-class de negação são entendidos como seletores normais.

Aqui está um trecho de CSS:

```
div.outer p {
  color: orange;
}
div:not(.outer) p {
  color: lime;
}
```

Usado com o seguindo código HTML:

```
<div class="outer">
  <p>Esta é a div outer.</p>
  <div class="inner">
    <p>Este texto está na div inner.</p>
  </div>
</div>
```

Irá aparecer na tela assim:

Esta é a div outer.

Este texto está na div inner.

Especificação Form-based

A especificação é baseada na forma de um seletor. No seguinte caso, o seletor contém os atributos no algoritmo de determinação de especificação, embora ele selecione um ID.

A seguir veja as declarações de estilo:

```
* #foo {
  color: green;
}
*[id="foo"] {
  background: purple;
}
```

Usado com esta marcação:

```
<p id="foo">Eu sou um simples texto.</p>
```

Vai acabar parecendo algo como:

Eu sou um simples texto.

Porque coincide com o mesmo elemento, mas o seletor de ID tem uma especificação superior.

Estrutura aproximada

A seguir a declaração do estilo:

```
body h1 {
  color: green;
}
html h1 {
  color: purple;
}
```

Com o HTML seguinte::

```
<html>
  <body>
    <h1>Aqui está o título!</h1>
  </body>
</html>
```

Vamos ter algo como:

Aqui está o título!

Veja Também

- Espcificação de Seletores CSS - https://www.w3.org/TR/selectors/#specificity

Utilizando propriedades CSS personalizadas (variáveis)

Propriedades personalizadas (às vezes chamadas de **variáveis CSS** ou **variáveis em cascata**) são entidades definidas por autores CSS que contêm valores específicos a serem reutilizados em um documento. Eles são definidos usando a notação de propriedade personalizada (por exemplo, `--main-color: black;`) e são acessados usando a função `var()` (por exemplo, `cor: var(--main-color);`).

Sites complexos têm quantidades muito grandes de CSS, geralmente com muitos valores repetidos. Por exemplo, a mesma cor pode ser usada em centenas de lugares diferentes, exigindo pesquisa global e substituição se essa cor precisar ser alterada. As propriedades personalizadas permitem que um valor seja armazenado em um local e, em seguida, referenciado em vários outros locais. Um benefício adicional são os identificadores semânticos. Por exemplo, `--main-text-color` é mais fácil de entender do que `#00ff00`, especialmente se esta mesma cor também for usada em outros contextos.

As propriedades personalizadas estão sujeitas à cascata e herdam seu valor de seu pai.

Uso básico

A declaração de uma propriedade personalizada é feita usando um nome de propriedade personalizada que começa com um hífen duplo (`--`) e um valor de propriedade que pode ser qualquer valor CSS válido. Como qualquer outra propriedade, isso é escrito dentro de um conjunto de regras, assim:

```
element {
  --main-bg-color: brown;
}
```

Observe que o seletor fornecido ao conjunto de regras define o escopo no qual a propriedade personalizada pode ser usada. Uma prática recomendada comum é definir propriedades personalizadas na pseudoclasse `:root`, para que ela possa ser aplicado globalmente em seu documento HTML:

```
:root {
  --main-bg-color: brown;
}
```

No entanto, isso nem sempre precisa ser o caso: talvez você tenha um bom motivo para limitar o escopo de suas propriedades personalizadas.

> **Nota:** Os nomes das propriedades personalizadas diferenciam maiúsculas de minúsculas — `--my-color` será tratado como uma propriedade personalizada separada de `--My-color`.

Conforme mencionado anteriormente, você usa o valor da propriedade personalizada especificando o nome da propriedade personalizada dentro da função `var()`, no lugar de um valor de propriedade regular:

```
element {
  background-color: var(--main-bg-color);
}
```

Primeiros passos com propriedades personalizadas

Vamos começar com este CSS que aplica a mesma cor a elementos de classes diferentes:

```
.one {
  color: white;
  background-color: brown;
  margin: 10px;
  width: 50px;
  height: 50px;
  display: inline-block;
}

.two {
  color: white;
  background-color: black;
  margin: 10px;
  width: 150px;
  height: 70px;
  display: inline-block;
}
.three {
  color: white;
  background-color: brown;
  margin: 10px;
  width: 75px;
}
.four {
  color: white;
  background-color: brown;
  margin: 10px;
  width: 100px;
}

.five {
```

```
  background-color: brown;
}
```

Vamos aplicá-lo a este HTML:

```
<div>
  <div class="one">1:</div>
  <div class="two">2: Texto <span class="five">5 - mais texto</span></div>
  <input class="three" />
  <textarea class="four">4: Lorem Ipsum</textarea>
</div>
```

Isso produz o seguinte resultado:

Usando a pseudo-classe :root

Observe o CSS repetitivo no exemplo acima. A cor de fundo é definida como 'marrom' em vários lugares. Para algumas declarações CSS, é possível declarar isso mais alto na cascata e deixar que a herança CSS resolva esse problema naturalmente. Para projetos não triviais, isso nem sempre é possível. Ao declarar uma propriedade personalizada na pseudoclasse `:root` e usá-la quando necessário em todo o documento, um autor de CSS pode reduzir a necessidade de repetição:

```
:root {
  --main-bg-color: brown;
}

.one {
  color: white;
  background-color: var(--main-bg-color);
  margin: 10px;
  width: 50px;
  height: 50px;
  display: inline-block;
}

.two {
  color: white;
  background-color: black;
  margin: 10px;
  width: 150px;
  height: 70px;
  display: inline-block;
}
.three {
  color: white;
  background-color: var(--main-bg-color);
  margin: 10px;
  width: 75px;
}
.four {
  color: white;
  background-color: var(--main-bg-color);
  margin: 10px;
  width: 100px;
}

.five {
  background-color: var(--main-bg-color);
}
```

```
<div>
  <div class="one"></div>
  <div class="two">Texto <span class="five">- mais texto</span></div>
  <input class="three" />
  <textarea class="four">Lorem Ipsum</textarea>
</div>
```

Isso leva ao mesmo resultado do exemplo anterior, mas permite uma declaração canônica do valor da propriedade desejada; muito útil se você quiser alterar o valor em toda a página posteriormente.

Herança de propriedades personalizadas

As propriedades personalizadas são herdadas. Isso significa que, se nenhum valor for definido para uma propriedade personalizada em um determinado elemento, o valor de seu pai será usado. Pegue este HTML:

```
<div class="one">
  <div class="two">
    <div class="three"></div>
    <div class="four"></div>
  </div>
</div>
```

... com o seguinte CSS:

```
.two {
  --test: 10px;
}

.three {
  --test: 2em;
}
```

Neste caso, os resultados de `var(--test)` são:

- Para o elemento `class="two"`: `10px`
- Para o elemento `class="three"`: `2em`
- Para o elemento `class="four"`: `10px` (herdado de seu pai)
- Para o elemento `class="one"`: *invalid value*, que é o valor padrão de qualquer propriedade personalizada

Lembre-se de que essas são propriedades personalizadas, não variáveis reais, como você pode encontrar em outras linguagens de programação. O valor é calculado onde é necessário, não armazenado para uso em outras regras. Por exemplo, você não pode definir uma propriedade para um elemento e esperar recuperá-la na regra do descendente de um irmão. A propriedade é definida apenas para o seletor correspondente e seus descendentes, como qualquer CSS normal.

Valores alternativos de propriedade personalizada

Usando a função `var()`, você pode definir vários **valores alternativos** quando a variável dada ainda não está definida; isso pode ser útil ao trabalhar com Elementos personalizados e Shadow DOM.

> **Nota:** os valores alternativos não são usados para corrigir a compatibilidade do navegador. Se o navegador não oferecer suporte a propriedades personalizadas de CSS, o valor de fallback não ajudará. É apenas um backup para o navegador que suporta propriedades personalizadas CSS para escolher um valor diferente se a variável fornecida não estiver definida ou tiver um valor inválido.

O primeiro argumento para a função é o nome da propriedade personalizada a ser substituída. O segundo argumento para a função, se fornecido, é um valor de fallback, que é usado como valor de substituição quando a propriedade personalizada referenciada é inválido. A função aceita apenas dois parâmetros, atribuindo tudo o que segue a primeira vírgula como segundo parâmetro. Se esse segundo parâmetro for inválido, o fallback falhará. Por exemplo:

```
.two {
  /* Vermelho se --my-var não estiver definido */
  color: var(--my-var, red);
}

.three {
  /* rosa se --my-var e --my-background não estiverem definidos */
  background-color: var(--my-var, var(--my-background, pink));
}

.three {
  /* Inválido: "--meu-fundo, rosa" */
  background-color: var(--my-var, --my-background, pink);
}
```

Incluir uma propriedade personalizada como fallback, conforme visto no segundo exemplo acima, é a maneira correta de fornecer mais de um fallback. A técnica pode causar problemas de desempenho, pois leva mais tempo para analisar as variáveis.

> **Nota:** A sintaxe do fallback, como a de propriedades personalizadas, permite vírgulas. Por exemplo, `var(--foo, red, blue)` define um fallback de `red, blue` — qualquer coisa entre a primeira vírgula e o final da função é considerado um valor fallback.

Tratamento de propriedades personalizadas inválidas

Cada propriedade CSS pode receber um conjunto definido de valores. Se você tentar atribuir um valor a uma propriedade que está fora de seu conjunto de valores válidos, ela será considerada *inválida*.

Quando o navegador encontra um valor inválido para uma propriedade normal, ele descarta o valor e os elementos recebem os valores que teriam se a declaração simplesmente não existisse.

No entanto, quando os valores das propriedades personalizadas são analisados, o navegador ainda não sabe onde eles serão usados, portanto, deve considerar quase todos os valores como *válidos*.

Infelizmente, esses valores válidos podem ser usados, através da notação funcional `var()`, em um contexto onde eles podem não fazer sentido. Propriedades e variáveis personalizadas podem levar a declarações CSS inválidas, levando ao novo conceito de *válido no tempo computado*.

Quando o navegador encontra uma substituição `var()` inválida, então o inicial ou herdado valor da propriedade é usado.

Os próximos dois exemplos ilustram isso.

Propriedades normais inválidas

Neste exemplo, tentamos aplicar um valor de `16px` à propriedade `color`. Como isso é inválido, o CSS é descartado e o resultado é como se a regra não existisse, então a regra `color: blue` anterior é aplicada em seu lugar e o parágrafo é azul.

HTML

```
<p>Este parágrafo é inicialmente preto.</p>
```

CSS

```
p {
  color: blue;
}

p {
  color: 16px;
}
```

Resultado

Propriedades personalizadas inválidas

Este exemplo é exatamente como o último, exceto que usamos uma propriedade personalizada.

Como esperado, o navegador substitui o valor de `--text-color` no lugar de `var(--text-color)`, mas `16px` não é um valor de propriedade válido para `color`. Após a substituição, a propriedade não faz mais sentido. O navegador lida com essa situação em duas etapas:

1. Verifique se a propriedade `color` é herdável. É, mas este `<p>` não tem nenhum pai com a propriedade `color` definida. Então passamos para a próxima etapa.
2. Defina o valor para seu **valor inicial padrão**, que é preto.

HTML

```
<p>Este parágrafo é inicialmente preto.</p>
```

CSS

```
:root {
  --text-color: 16px;
}

p {
  color: blue;
}

p {
```

```
  color: var(--text-color);
}
```

Resultado

Valores em JavaScript

Para usar os valores das propriedades personalizadas em JavaScript, é como as propriedades padrão.

```
// obtém a variável do estilo inline
element.style.getPropertyValue("--my-var");

// obtém variável de qualquer lugar
getComputedStyle(element).getPropertyValue("--my-var");

// define a variável no estilo inline
element.style.setProperty("--my-var", jsVar + 4);
```

Veja também

- Sintaxe de propriedade personalizada
- `var()`

Aplicando cores a elementos HTML usando CSS

Este artigo é uma cartilha que apresenta cada uma das maneiras pelas quais as cores CSS podem ser usadas em HTML.

O uso da cor é uma forma fundamental de expressão humana. As crianças experimentam a cor antes mesmo de terem a destreza manual para desenhar. Talvez seja por isso que a cor é uma das primeiras coisas que as pessoas muitas vezes querem experimentar quando aprendem a desenvolver sites. Com CSS, há várias maneiras de adicionar cor aos seus elementos HTML para criar exatamente a aparência desejada.

Vamos abordar a maior parte do que você precisa saber ao usar cores, incluindo uma lista do que você pode colorir e quais propriedades CSS estão envolvidas, como você descreve as cores, e como realmente usar cores em folhas de estilo e em scripts. Também veremos como deixar o usuário escolher uma cor.

Em seguida, encerraremos as coisas com uma breve discussão sobre como usar cores com sabedoria: como selecionar cores apropriadas, tendo em mente as necessidades de pessoas com diferentes capacidades visuais.

Coisas que podem ter cor

No nível dos elementos, tudo em HTML pode ter cores aplicadas. Em vez disso, vamos ver as coisas em termos dos tipos de coisas que são desenhadas nos elementos, como texto e bordas e assim por diante. Para cada um, veremos uma lista das propriedades CSS que aplicam cores a eles.

Em um nível fundamental, a propriedade `color` define a cor de primeiro plano do conteúdo de um elemento HTML e a propriedade `background-color` define a cor de fundo do elemento. Estes podem ser usados em praticamente qualquer elemento.

Texto

Sempre que um elemento é renderizado, essas propriedades são usadas para determinar a cor do texto, seu plano de fundo e quaisquer decorações no texto.

`color`

: A cor a ser usada ao desenhar o texto e quaisquer decorações de texto (como a adição de sublinhados ou sublinhados, linhas tachadas, e assim por diante.

`background-color`

: A cor de fundo do texto.

`text-shadow`

: Configura um efeito de sombra aplicado ao texto. Entre as opções para os aspectos da sombra, está a cor base da sombra (que será então desfocada e mesclada com o plano de fundo, com base nos outros parâmetros). Veja Sombras projetadas em texto para descobrir mais.

`text-decoration-color`

: Por padrão, as decorações de texto (como sublinhados, tachados, etc.) usam a propriedade `color` como suas cores. No entanto, você pode substituir esse comportamento e usar uma cor diferente para eles com a propriedade `text-decoration-color`.

`text-emphasis-color`

A cor a ser usada ao desenhar símbolos de ênfase adjacentes a cada caractere no texto. Isso é usado principalmente ao desenhar texto para idiomas do Leste Asiático.

`caret-color`

A cor a ser usada ao desenhar o caret (às vezes chamado de cursor de entrada de texto) dentro do elemento. Isso só é útil em elementos que são editáveis, como `<input>` e `<textarea>` ou elementos cujo atributo HTML `contenteditable` está definido.

Caixas

Cada elemento é uma caixa com algum tipo de conteúdo e tem um fundo e uma borda além de qualquer conteúdo que a caixa possa ter.

Borders

Consulte a seção Borders para obter uma lista das propriedades CSS que você pode usar para definir as cores das bordas de uma caixa.

`background-color`

A cor de fundo a ser usada em áreas do elemento que não possuem conteúdo de primeiro plano.

`column-rule-color`

A cor a ser usada ao desenhar a linha que separa as colunas do texto.

`outline-color`

A cor a ser usada ao desenhar um contorno ao redor da parte externa do elemento. Esse contorno é diferente da borda, pois não recebe espaço reservado para ele no documento (portanto, pode se sobrepor a outro conteúdo). Geralmente é usado como indicador de foco, para mostrar qual elemento receberá eventos de entrada.

Bordas

Qualquer elemento pode ter uma borda desenhada em torno dele. Uma borda de elemento básico é uma linha desenhada ao redor das bordas do conteúdo do elemento. Consulte Propriedades do Box para saber mais sobre a relação entre elementos e suas bordas, e o artigo Estilizando bordas usando CSS para saber mais sobre como aplicar estilos a bordas.

Você pode usar a propriedade abreviada `border`, que permite configurar tudo sobre o borda de uma só vez (incluindo recursos não coloridos de bordas, como sua largura, estilo (sólido, tracejado, etc.), e assim por diante.

`border-color`

Especifica uma única cor a ser usada para cada lado da borda do elemento.

`border-left-color`, `border-right-color`, `border-top-color` e `cor da borda inferior`

Permite definir a cor do lado correspondente da borda do elemento.

`border-block-start-color` e `border-block-end-color`

Com estes, você pode definir a cor usada para desenhar as bordas que estão mais próximas do início e do fim do bloco que a borda circunda. Em um modo de escrita da esquerda para a direita (como a forma como o inglês é escrito), a borda inicial do bloco é a borda superior e o final do bloco é a parte inferior. Isso difere do início e do fim embutidos, que são as bordas esquerda e direita (correspondendo a onde cada linha de texto na caixa começa e termina).

`border-inline-start-color` e `border-inline-end-color`

Permitem colorir as bordas da borda mais próximas ao início e ao final do início das linhas de texto dentro da caixa. De que lado isso está irá variar dependendo das propriedades `writing-mode`, `direction` e `text-orientation`, que são normalmente (mas nem sempre) usado para ajustar a direcionalidade do texto com base no idioma exibido. Por exemplo, se o texto da caixa estiver sendo renderizado da direita para a esquerda, a cor da borda em linha será aplicada ao lado direito da borda.

Outras formas de usar cores

CSS não é a única tecnologia da web que suporta cores. Existem tecnologias gráficas disponíveis na web que também suportam cores.

A API do Canvas

Permite desenhar gráficos de bitmap 2D em um elemento `<canvas>`. Veja nosso tutorial do Canvas para saber mais.

SVG (Gráficos vetoriais escaláveis)

permite desenhar imagens usando comandos que desenham formas, padrões e linhas específicos para produzir uma imagem. Os comandos SVG são formatados como XML e podem ser incorporados diretamente em uma página da Web ou podem ser colocados na página usando o elemento `<img>`, como qualquer outro tipo de imagem.

WebGL

A Biblioteca de gráficos da Web é uma API baseada em OpenGL ES para desenhar gráficos 2D e 3D de alto desempenho na Web. Veja nosso tutorial WebGL para saber mais.

Como descrever uma cor

Para representar uma cor em CSS, você precisa encontrar uma maneira de traduzir o conceito analógico de "cor" em uma forma digital que um computador possa usar. Isso geralmente é feito dividindo a cor em componentes, como quanto de cada um de um conjunto de cores primárias deve ser misturado ou quão brilhante é a cor. Como tal, existem várias maneiras de descrever cores em CSS.

Para uma discussão mais detalhada de cada um dos tipos de valor de cor, veja a referência para a unidade CSS `<color>`.

Palavras-chave

Um conjunto de nomes de cores padrão foi definido, permitindo que você use essas palavras-chave em vez de representações numéricas de cores se você optar por fazer isso e houver uma palavra-chave representando a cor exata que você deseja usar. As palavras-chave de cores incluem as cores primárias e secundárias padrão (como `red`, `blue` ou `orange`), tons de cinza (de `black` a `white`, incluindo cores como `darkgray` e `lightgrey`), e uma variedade de outras cores misturadas, incluindo `lightseagreen`, `cornflowerblue` e `rebeccapurple`.

Veja Palavras-chave de cores para uma lista de todas as palavras-chave de cores disponíveis.

valores RGB

Existem três maneiras de representar uma cor RGB em CSS.

Notação de string hexadecimal

A notação de string hexadecimal representa uma cor usando dígitos hexadecimais para representar cada um dos componentes de cor (vermelho, verde e azul). Também pode incluir um quarto componente: o canal alfa (ou opacidade). Cada componente de cor pode ser representado como um número entre 0 e 255 (0x00 e 0xFF) ou, opcionalmente, como um número entre 0 e 15 (0x0 e 0xF). Todos os componentes *devem* ser especificados usando o mesmo número de dígitos. Se você usar a notação de um dígito, a cor final será calculada usando o dígito de cada componente duas vezes; ou seja, `"#D"` torna-se `"#DD"` ao desenhar.

Uma cor em notação de string hexadecimal sempre começa com o caractere `"#"`. Depois disso vêm os dígitos hexadecimais do código de cores. A string não diferencia maiúsculas de minúsculas.

`"#rrggbb"`

Especifica uma cor totalmente opaca cujo componente vermelho é o número hexadecimal `0xrr`, o componente verde é `0xgg` e o componente azul é `0xbb`.

`"#rrggbbaa"`

Especifica uma cor cujo componente vermelho é o número hexadecimal `0xrr`, componente verde é `0xgg` e composição azulente é `0xbb`. O canal alfa é especificado por `0xaa`; quanto menor for esse valor, mais translúcida será a cor.

`"#rgb"`

Especifica uma cor cujo componente vermelho é o número hexadecimal `0xrr`, o componente verde é `0xgg` e o componente azul é `0xbb`.

`"#rgba"`

Especifica uma cor cujo componente vermelho é o número hexadecimal `0xrr`, o componente verde é `0xgg` e o componente azul é `0xbb`. O canal alfa é especificado por `0xaa`; quanto menor for esse valor, mais translúcida será a cor.

Como exemplo, você pode representar a cor opaca azul brilhante como `"#0000ff"` ou `"#00f"`. Para torná-lo 25% opaco, você pode usar `"#0000ff44"` ou `"#00f4"`.

Notação funcional RGB

A notação funcional RGB (Vermelho/Verde/Azul), como a notação de string hexadecimal, representa cores usando seus componentes vermelho, verde e azul (assim como, opcionalmente, um componente de canal alfa para opacidade). No entanto, em vez de usar uma string, a cor é definida usando a função CSS `rgb()`. Esta função aceita como parâmetros de entrada os valores dos componentes vermelho, verde e azul e um quarto parâmetro opcional, o valor do canal alfa.

Os valores legais para cada um desses parâmetros são:

`vermelho`, `verde` e `azul`

Cada um deve ter um valor `<integer>` entre 0 e 255 (inclusive), ou um `<percentage>` de 0% a 100%.

`alfa`

O canal alfa é um número entre 0,0 (totalmente transparente) e 1,0 (totalmente opaco). Você também pode especificar uma porcentagem em que 0% é igual a 0,0 e 100% é igual a 1,0.

Por exemplo, um vermelho brilhante que é 50% opaco pode ser representado como `rgb(255, 0, 0, 0.5)` ou `rgb(100%, 0, 0, 50%)`.

Notação funcional HSL

Designers e artistas geralmente preferem trabalhar usando o método de cores HSL (Hue/Saturation/Luminosity). Na web, as cores HSL são representadas usando a notação funcional HSL. A função CSS `hsl()` é muito semelhante à função `rgb()` em uso de outra forma.

O diagrama abaixo mostra um cilindro de cor HSL. Hue define a cor real com base na posição ao longo de uma color wheel circular que representa as cores do espectro visível. A saturação é uma porcentagem de quanto do caminho entre ser um tom de cinza e ter a quantidade máxima possível do tom fornecido. À medida que o valor da luminância (ou luminosidade) aumenta, a cor passa do mais escuro possível para o mais claro possível (do preto para o branco). Imagem cortesia do usuário SharkD na Wikipedia, distribuído sob a licença CC BY-SA 3.0.

O valor do componente de matiz (H) de uma cor HSL é um ângulo do vermelho ao redor do amarelo, verde, ciano, azul e magenta (terminando de volta ao vermelho novamente em 360°) que identifica qual é a cor base. O valor pode ser especificado em qualquer unidade `<angle>` suportada por CSS, incluindo graus (`deg`), radianos (`rad`), gradianos (`grad`), ou voltas (`virar`). Mas isso não controla o quão viva ou opaca, ou quão brilhante ou escura é a cor.

O componente de saturação (S) da cor especifica qual porcentagem da cor final é composta pelo matiz especificado. O restante é definido pelo nível de cinza fornecido pelo componente de luminância (L).

Pense nisso como criar a cor de tinta perfeita:

1. Você começa com a tinta base que é a intensidade máxima possível para uma determinada cor, como o azul mais intenso que pode ser representado pela tela do usuário. Este é o componente **matiz** (H): um valor que representa o ângulo ao redor do color wheel para o matiz vívido que queremos usar como base.
2. Em seguida, selecione uma tinta em tons de cinza que corresponda ao brilho que você deseja que a cor seja; esta é a luminância. Você quer que seja muito brilhante e quase branco, ou muito escuro e mais próximo do preto, ou algo no meio? Isso é especificado usando uma porcentagem, onde 0% é perfeitamente preto e 100% é perfeitamente branco (independentemente da saturação ou matiz). Entre os valores há uma área cinza literal.
3. Agora que você tem uma tinta cinza e uma cor perfeitamente viva, precisa misturá-las. O componente de saturação (S) da cor indica qual porcentagem da cor final deve ser composta por aquela cor perfeitamente vívida. O restante da cor final é composto pela tinta cinza que representa a saturação.

Você também pode incluir opcionalmente um canal alfa, para tornar a cor menos de 100% opaca.

Aqui estão algumas cores de amostra em notação HSL:

```
table {
  border: 1px solid black;
  font:
    16px "Open Sans",
    Helvetica,
    Arial,
    sans-serif;
  border-spacing: 0;
  border-collapse: collapse;
}

th,
td {
  border: 1px solid black;
  padding: 4px 6px;
  text-align: left;
}

th {
  background-color: hsl(0, 0%, 75%);
}
```

```
<table>
  <thead>
    <tr>
      <th scope="col">Color in HSL notation</th>
      <th scope="col">Example</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td><code>hsl(90deg, 100%, 50%)</code></td>
      <td style="background-color: hsl(90deg, 100%, 50%);"> </td>
    </tr>
    <tr>
      <td><code>hsl(90, 100%, 50%)</code></td>
      <td style="background-color: hsl(90, 100%, 50%);"> </td>
    </tr>
    <tr>
      <td><code>hsl(0.15turn, 50%, 75%)</code></td>
      <td style="background-color: hsl(0.15turn, 50%, 75%);"> </td>
    </tr>
    <tr>
      <td><code>hsl(0.15turn, 90%, 75%)</code></td>
      <td style="background-color: hsl(0.15turn, 90%, 75%);"> </td>
    </tr>
    <tr>
      <td><code>hsl(0.15turn, 90%, 50%)</code></td>
      <td style="background-color: hsl(0.15turn, 90%, 50%);"> </td>
    </tr>
    <tr>
      <td><code>hsl(270deg, 90%, 50%)</code></td>
      <td style="background-color: hsl(270deg, 90%, 50%);"> </td>
    </tr>
  </tbody>
</table>
```

> **Nota:** Quando você omite a unidade do matiz, assume-se que está em graus (`deg`).

Notação funcional HWB

Assim como a notação funcional HSL acima, a função hwb() usa o mesmo valor de matiz. Mas, em vez de claridade e saturação, você especifica valores de brancura e negritude em porcentagens. Os valores **não** são separados por vírgula e um valor alfa opcional pode ser incluído (deve ser precedido por uma barra `/`).

Aqui estão alguns exemplos de uso da notação HWB:

```
/* Todos esses exemplos especificam vários tons de verde-limão. */
hwb(90 10% 10%)
hwb(90 10% 10%)
hwb(90 50% 10%)
hwb(90deg 10% 10%)
hwb(1.5708rad 60% 0%)
hwb(.25turn 0% 40%)

/* Mesmo verde-limão, mas com valor alfa */
hwb(90 10% 10% / 0.5)
hwb(90 10% 10% / 50%)
```

Usando cores

Agora que você sabe quais propriedades CSS existem que permitem aplicar cores a elementos e os formatos que você pode usar para descrever cores, você pode juntar tudo isso para começar a usar cores. Como você deve ter visto na lista em Coisas que podem ter cor, há muitas coisas que você pode colorir com CSS. Vejamos isso de dois lados: usando a cor dentro de um stylesheet e adicionando e alterando a cor usando o código JavaScript para alterar os estilos dos elementos.

Especificando cores em folhas de estilo

A maneira mais fácil de aplicar cores aos elementos — e a maneira como você normalmente faz isso — é especificar as cores no CSS que é usado ao renderizar os elementos. Embora não usemos todas as propriedades mencionadas anteriormente, veremos alguns exemplos. O conceito é o mesmo em qualquer lugar que você use cores.

Vamos dar uma olhada em um exemplo, começando pelos resultados que estamos tentando alcançar:

HTML

O HTML responsável pela criação do exemplo acima é mostrado aqui:

```
<div class="wrapper">
  <div class="box boxLeft">
    <p>Esta é a primeira caixa.</p>
  </div>
  <div class="box boxRight">
    <p>Esta é a segunda caixa.</p>
  </div>
</div>
```

Isso é bem simples, usando um `<div>` como um wrapper em torno do conteúdo, que consiste em mais dois `<div>`s, cada um com um estilo diferente com um único parágrafo (`<p>`) em cada caixa.

A mágica acontece, como sempre, no CSS, onde aplicaremos as cores para definir o layout do HTML acima.

CSS

Veremos o CSS para criar os resultados acima um por vez, para que possamos revisar as partes interessantes uma a uma.

```
.wrapper {
  width: 620px;
  height: 110px;
  margin: 0;
  padding: 10px;
  border: 6px solid mediumturquoise;
}
```

A classe `.wrapper` é usada para atribuir estilos ao `<div>` que inclui todos os nossos outros conteúdos. Isso estabelece o tamanho do contêiner usando `width` e `height` bem como seu `margin` e `padding` .

De mais interesse para nossa discussão aqui é o uso da propriedade `border` para estabelecer uma borda ao redor da borda externa do elemento. Essa borda é uma linha sólida, com 6 pixels de largura, na cor `mediumturquoise`.

Nossas duas caixas coloridas compartilham várias propriedades em comum, então, a seguir, estabelecemos uma classe, `.box`, que define essas propriedades compartilhadas:

```
.box {
  width: 290px;
  height: 100px;
  margin: 0;
  padding: 4px 6px;
  font:
    28px "Marker Felt",
    "Zapfino",
    cursive;
  display: flex;
  justify-content: center;
  align-items: center;
}
```

Resumidamente, `.box` estabelece o tamanho de cada caixa, bem como a configuração da fonte utilizada dentro dela. Também aproveitamos o CSS Flexbox para centralizar facilmente o conteúdo de cada caixa. Ativamos o modo `flex` usando `display: flex` e definimos `justify-content` e `align-items` para `center`. Então podemos criar uma classe para cada umas duas caixas que definem as propriedades que diferem entre as duas.

```
.boxLeft {
  float: left;
  background-color: rgb(245, 130, 130);
  outline: 2px solid darkred;
}
```

A classe `.boxLeft` — que, inteligentemente, é usada para estilizar a caixa à esquerda — flutua a caixa para a esquerda e, em seguida, configura as cores:

- A cor de fundo da caixa é definida alterando o valor da propriedade CSS `background-color` para `rgb(245, 130, 130)`.

- Um contorno é definido para a caixa. Ao contrário do `border` mais comumente usado, `outline` não afeta o layout; ele desenha por cima de qualquer coisa que esteja fora da caixa do elemento em vez de abrir espaço como `border`. Este contorno é uma linha vermelha escura sólida com dois pixels de espessura. Observe o uso da palavra-chave `darkred` ao especificar a cor.
- Observe que não estamos definindo explicitamente a cor do texto. Isso significa que o valor de `color` será herdado do elemento mais próximo que o define. Por padrão, isso é preto.

```
.boxRight {
  float: right;
  background-color: hsl(270deg, 50%, 75%);
  outline: 4px dashed rgb(110, 20, 120);
  color: hsl(0deg, 100%, 100%);
  text-decoration: underline wavy #88ff88;
  text-shadow: 2px 2px 3px black;
}
```

> **Nota:** ao tentar mostrá-lo no Safari, ele não será exibido corretamente. Porque o Safari não suporta `text-decoration: underline wavy #88ff88`.

Finalmente, a classe `.boxRight` descreve as propriedades únicas da caixa que é desenhada à direita. Ele está configurado para flutuar a caixa para a direita para que ela apareça ao lado da caixa anterior. Em seguida, as seguintes cores são estabelecidas:

- O `background-color` é definido usando o valor HSL especificado usando `hsl(270deg, 50%, 75%)`. Esta é uma cor roxa média.
- O `outline` da caixa é usado para especificar que a caixa deve ser colocada em uma linha tracejada de quatro pixels de espessura cuja cor é um roxo um pouco mais profundo (`rgb(110, 20, 120)`).
- A cor do primeiro plano (texto) é especificada definindo a propriedade `color` para `hsl(0deg, 100%, 100%)`. Esta é uma das muitas maneiras de especificar a cor branca.
- Adicionamos uma linha ondulada verde sob o texto com `text-decoration`.
- Finalmente, um pouco de sombra é adicionada ao texto usando `text-shadow`. Seu parâmetro `color` é definido como `black`.

Deixando o usuário escolher uma cor

Há muitas situações em que seu site pode precisar permitir que o usuário selecione uma cor. Talvez você tenha uma interface de usuário personalizável ou esteja implementando um aplicativo de desenho. Talvez você tenha texto editável e precise deixar o usuário escolher a cor do texto. Ou talvez seu aplicativo permita que o usuário atribua cores a pastas ou itens. Embora historicamente tenha sido necessário implementar seu próprio seletor de cores, o HTML agora oferece suporte para navegadores fornecerem um para seu uso por meio do `<input>`, usando `"color"` como o valor de seu atributo `type`.

O elemento `<input>` representa uma cor apenas na notação de string hexadecimal abordada acima.

Exemplo: Escolhendo uma cor

Vejamos um exemplo simples, no qual o usuário pode escolher uma cor. Conforme o usuário ajusta a cor, a borda ao redor do exemplo muda para refletir a nova cor. Após finalizar e escolher a cor final, o valor do seletor de cores é exibido.

> **Nota:** no macOS, você indica que finalizou a seleção da cor fechando a janela do seletor de cores.

HTML

O HTML aqui cria uma caixa que contém um controle seletor de cores (com um rótulo criado usando o elemento `<label>`) e um elemento de parágrafo vazio (`<p>`) no qual vamos produzir algum texto do nosso código JavaScript.

```
<div id="box">
  <label for="colorPicker">Cor da borda:</label>
  <input type="color" value="#8888ff" id="colorPicker" />
  <p id="output"></p>
</div>
```

CSS

O CSS estabelece um tamanho para a caixa e alguns estilos básicos para as aparências. A borda também é estabelecida com uma largura de 2 pixels e uma cor de borda.

```
#box {
  width: 500px;
  height: 200px;
```

```
  border: 2px solid rgb(245, 220, 225);
  padding: 4px 6px;
  font:
    16px "Lucida Grande",
    "Helvetica",
    "Arial",
    "sans-serif";
}
```

JavaScript

O script aqui lida com a tarefa de atualizar a cor inicial da borda para corresponder ao valor do seletor de cores. Em seguida, dois manipuladores de eventos são adicionados para lidar com a entrada do elemento `<input type="color">`.

```
const colorPicker = document.getElementById("colorPicker");
const box = document.getElementById("box");
const output = document.getElementById("output");

box.style.borderColor = colorPicker.value;

colorPicker.addEventListener(
  "input",
  (event) => {
    box.style.borderColor = event.target.value;
  },
  false,
);

colorPicker.addEventListener(
  "change",
  (event) => {
    output.innerText = `Cor definida como ${colorPicker.value}.`;
  },
  false,
);
```

O evento `input` é enviado toda vez que o valor do elemento muda; ou seja, toda vez que o usuário ajusta a cor no seletor de cores. Cada vez que esse evento chega, definimos a cor da borda da caixa para corresponder ao valor atual do seletor de cores.

O evento `change` é recebido quando o valor do seletor de cores é finalizado. Nós respondemos definindo o conteúdo do elemento `<p>` com o ID `"output"` para uma string que descreve a cor finalmente selecionada.

Usando cores com sabedoria

Fazer as escolhas certas ao selecionar as cores ao projetar um site pode ser um processo complicado, especialmente se você não estiver bem fundamentado em arte, design ou pelo menos teoria básica de cores. A escolha errada de cores pode tornar seu site pouco atraente ou, pior ainda, deixar o conteúdo ilegível devido a problemas de contraste ou cores conflitantes. Pior ainda, se usar as cores erradas pode resultar na inutilização do seu conteúdo por pessoas com certos problemas de visão, principalmente daltonismo.

Encontrando as cores certas

Criar as cores certas pode ser complicado, especialmente sem treinamento em arte ou design. Felizmente, existem ferramentas disponíveis que podem ajudá-lo. Embora eles não possam substituir ter um bom designer ajudando você a tomar essas decisões, eles definitivamente podem ajudá-lo a começar.

Cor básica

O primeiro passo é escolher sua **cor base**. Esta é a cor que de alguma forma define o seu site ou o assunto do site. Assim como associamos o verde à bebida Mountain Dew e pode-se pensar na cor azul em relação ao céu ou ao oceano, escolhendo uma cor base apropriada para representar seu site é um bom lugar para começar. Há muitas maneiras de selecionar uma cor base; algumas ideias incluem:

- Uma cor que esteja naturalmente associada ao tema do seu conteúdo, como a cor existente identificada com um produto ou ideia ou uma cor representativa da emoção que você deseja transmitir.
- Uma cor que vem de imagens associadas ao assunto do seu conteúdo. Se você estiver criando um site sobre um determinado item ou produto, escolha uma cor que esteja fisicamente presente nesse item.
- Navegue por sites que permitem que você veja muitas paletas de cores e imagens existentes para encontrar inspiração.

Ao tentar decidir sobre uma cor base, você pode descobrir que as extensões do navegador que permitem selecionar cores do conteúdo da Web podem ser particularmente úteis. Alguns deles são projetados especificamente para ajudar nesse tipo de trabalho. Por exemplo, o site ColorZilla

oferece uma extensão (Chrome / [Firefox](https:// www.colorzilla.com/firefox/)) que oferece uma ferramenta conta-gotas para escolher cores da web. Também pode obter médias das cores dos pixels em áreas de vários tamanhos ou até mesmo em uma área selecionada da página.

> **Nota:** A vantagem da média de cores pode ser que, muitas vezes, o que parece uma cor sólida é, na verdade, um número surpreendentemente variado de cores relacionadas, todas usadas em conjunto, misturando-se para criar o efeito desejado. Escolher apenas um desses pixels pode resultar na obtenção de uma cor que, por si só, parece muito fora do lugar.

Confeccionando a paleta

Depois de decidir a cor base, existem muitas ferramentas on-line que podem ajudá-lo a criar uma paleta de cores apropriadas para usar junto com a cor base, aplicando a teoria das cores à cor base para determinar as cores adicionadas apropriadas. Muitas dessas ferramentas também suportam a visualização das cores filtradas para que você possa ver como elas seriam para pessoas com várias formas de daltonismo. Consulte Cor e acessibilidade para uma breve explicação de por que isso é importante.

Alguns exemplos (todos gratuitos para uso a partir do momento em que esta lista foi revisada pela última vez):

- Ferramenta seletora de cores do MDN
- Paletton
- Roda de cores on-line do Adobe Color CC

Ao projetar sua paleta, lembre-se de que, além das cores que essas ferramentas normalmente geram, você provavelmente também precisará adicionar algumas cores neutras centrais, como branco (ou quase branco), preto (ou quase preto), e algum número de tons de cinza.

> **Nota:** Normalmente, é muito melhor usar o menor número de cores possível. Ao usar cores para acentuar em vez de adicionar cores a tudo na página, você mantém seu conteúdo fácil de ler e as cores que você usa têm muito mais impacto.

Recursos da teoria das cores

Uma revisão completa da teoria das cores está além do escopo deste artigo, mas há muitos artigos sobre a teoria das cores disponíveis, bem como cursos que você pode encontrar em escolas e universidades próximas. Alguns recursos úteis sobre a teoria das cores:

Ciência das cores (Khan Academy em associação com a Pixar)

> Um curso online que apresenta conceitos como o que é a cor, como ela é percebida e como usar as cores para expressar ideias. Apresentado por artistas e designers da Pixar.

Teoria das cores na Wikipedia

> Entrada da Wikipedia sobre teoria das cores, que tem muitas informações excelentes do ponto de vista técnico. Não é realmente um recurso para ajudá-lo com o processo de seleção de cores, mas ainda está cheio de informações úteis.

Cor e acessibilidade

Existem várias maneiras pelas quais a cor pode ser um problema accessibility. O uso impróprio ou descuidado de cores pode resultar em um site ou aplicativo que uma porcentagem de seu público-alvo pode não ser capaz de usar adequadamente, resultando em perda de tráfego, perda de negócios e possivelmente até mesmo um problema de relações públicas. Portanto, é importante considerar cuidadosamente o uso da cor.

Você deve fazer pelo menos uma pesquisa básica sobre daltonismo. Existem vários tipos; o mais comum é o daltonismo vermelho-verde, que faz com que as pessoas não consigam diferenciar entre as cores vermelho e verde. Existem outras, também, que vão desde a incapacidade de distinguir certas cores até a total incapacidade de ver as cores.

> **Nota:** A regra mais importante: nunca use a cor como a única maneira de saber algo. Se, por exemplo, você indicar sucesso ou falha de uma operação alterando a cor de uma forma de branco para verde para sucesso e vermelho para falha, os usuários com daltonismo vermelho-verde não poderão usar seu site corretamente. Em vez disso, talvez use texto e cor juntos, para que todos possam entender o que está acontecendo.

Para obter mais informações sobre daltonismo, consulte os seguintes artigos:

- Medline Plus: Daltonismo (Instituto Nacional de Saúde dos Estados Unidos)
- Academia Americana de Oftalmologia: O que é daltonismo?
- Daltonismo e Web Design (Usability.gov: Departamento de Saúde e Serviços Humanos dos Estados Unidos)

Exemplo de design de paleta

Vamos considerar um exemplo rápido de seleção de uma paleta de cores apropriada para um site. Imagine que você está construindo um site para um novo jogo que acontece no planeta Marte. Então, vamos fazer uma pesquisa no Google por fotos de Marte. Muitos bons exemplos de coloração marciana lá. Evitamos cuidadosamente as maquetes e as fotos dos filmes. E decidimos usar uma foto tirada por um dos landers de Marte que a humanidade estacionou na superfície nas últimas décadas, já que o jogo se passa na superfície do planeta. Usamos uma ferramenta de seleção de cores para selecionar uma amostra da cor que escolhemos.

Usando uma ferramenta conta-gotas, identificamos uma cor que gostamos e determinamos que a cor em questão é `#D79C7A`, que é uma cor vermelho-alaranjada apropriada que é tão estereotipada da superfície marciana.

Tendo selecionado nossa cor base, precisamos construir nossa paleta. Decidimos usar Paletton para criar as outras cores que precisamos. Ao abrir Paletton, vemos:

Em seguida, inserimos o código hexadecimal da nossa cor (`D79C7A`) na caixa "Base RGB" no canto inferior esquerdo da ferramenta:

Agora vemos uma paleta monocromática baseada na cor que escolhemos na foto de Marte. Se você precisar de muitas cores relacionadas por algum motivo, é provável que sejam boas. Mas o que realmente queremos é uma cor de destaque. Algo que irá aparecer ao lado da cor base. Para encontrar isso, clicamos no botão "adicionar complementar" abaixo do menu que permite selecionar o tipo de paleta (atualmente "Monocromática"). Paletton calcula uma cor de destaque apropriada; clicar na cor de destaque no canto inferior direito nos diz que essa cor é `#508D7C`.

Se não estiver satisfeito com a cor que lhe é proposta, pode alterar o esquema de cores, para ver se encontra algo de que goste mais. Por exemplo, se não gostarmos da cor azul-esverdeada proposta, podemos clicar no ícone do esquema de cores Triad, que nos apresenta o seguinte:

Esse azul acinzentado no canto superior direito parece muito bom. Clicando nele, descobrimos que é `#556E8D`. Isso seria usado como a cor de destaque, para ser usado com moderação para destacar as coisas, como em títulos ou no destaque de guias ou outros indicadores no site:

Agora temos nossa cor base e nosso sotaque. Além disso, temos alguns tons complementares de cada um, caso precisemos deles para gradientes e afins. As cores podem então ser exportadas em um nvários formatos para que você possa utilizá-los.

Depois de ter essas cores, você provavelmente ainda precisará selecionar cores neutras apropriadas. A prática comum de design é tentar encontrar o ponto ideal onde haja contraste suficiente para que o texto seja nítido e legível, mas não contraste suficiente para se tornar áspero para os olhos. É fácil ir longe demais de uma forma ou de outra, portanto, certifique-se de obter feedback sobre suas cores depois de selecioná-las e ter exemplos delas em uso disponíveis. Se o contraste for muito baixo, seu texto tenderá a ficar desbotado pelo fundo, deixando-o ilegível, mas se o contraste for muito alto, o usuário poderá achar seu site berrante e desagradável de se ver.

Cor, fundos, contraste e impressão

O que parece bom na tela pode parecer muito diferente no papel. Além disso, a tinta pode ser cara e, se um usuário estiver imprimindo sua página, ele não precisará necessariamente de todos os fundos e usar sua preciosa tinta quando tudo o que importa é o próprio texto. A maioria dos navegadores, por padrão, remove imagens de fundo ao imprimir documentos.

Se as cores e imagens de fundo foram selecionadas com cuidado e/ou são cruciais para a utilidade do conteúdo, você pode usar a propriedade CSS `print-color-adjust` para informar ao navegador que não deve fazer ajustes na aparência do conteúdo.

O valor padrão de `print-color-adjust`, `economy`, indica que o navegador tem permissão para fazer alterações de aparência conforme necessário para tentar otimizar a legibilidade e/ou economia de impressão do conteúdo, dado o tipo do dispositivo de saída para o qual o documento está sendo desenhado.

Você pode definir `print-color-adjust` como `exact` para informar ao navegador que o elemento ou elementos nos quais você o usa foram projetados especificamente para funcionar melhor com as cores e imagens deixadas como estão. Com este conjunto, o navegador não alterará a aparência do elemento e o desenhará conforme indicado pelo seu CSS.

> **Nota:** Não há garantia, porém, de que `print-color-adjust: exact` resultará em seu CSS sendo usado exatamente como fornecido. Se o navegador fornecer as preferências do usuário para alterar a saída (como uma caixa de seleção "não imprimir fundos" em uma caixa de diálogo de impressão), isso substituirá o valor de `print-color-adjust`.

Veja também

- Desenhando gráficos
- Gráficos na web
- ferramenta seletora de cores do MDN

Seletor de cores

Esta ferramenta facilita a criação, ajuste e experimentação com cores personalizadas para uso na web. Ela também facilita a conversão entre vários formatos de cores suportados por CSS, incluindo cores HEXA, RGB (vermelho, verde, azul) e HSL (tonalidade, saturação, luminosidade). Controle sobre o canal alfa também é suportado nos formatos RGB (rgba) e (hsla).

Enquanto você ajusta os parâmetros que definem a cor, ela é apresentada nos 3 formatos padrão para CSS web. Além disso, com base na seleção de cor atual, é gerada uma paleta para HSL, HSV e também alfa. O selecionador de cor estilo "conta-gotas" pode alternar os estilos HSL e HSV. Você também pode testar as cores e ver como elas se sobrepõem umas sobre às outras.

As cores geradas por você acima podem ser usadas em qualquer lugar em que a cor é usada em CSS e HTML, a menos que seja indicado de outra forma.

Veja também

Para mais aplicações de cores, veja:

- Aplicando cores para elementos HTML com CSS
- Fundamentação de textos e estilo de fonte
- Estilização de bordas com CSS
- Mudando estilos de cores de fundo com CSS
- Cores e contrastes de cores

Using CSS counters

Contadores CSS são em essência, as variáveis mantidas pelo CSS cujos valores podem ser incrementado por regras do CSS para controlar quantas vezes eles são usados.Isso permite ajustar a aparência do conteúdo com base na sua colocação no documento. contadores CSS são uma implementação de Contadores automáticos e numeração em CSS 2.1.

O valor de um contador é manipulado através da utilização de `counter-reset`. `counter-increment` pode ser exibido em uma página usando o `counter()` ou `counters()` função da propriedade de `conteúdo`.

Usando contadores

Para usar um contador, tem quer definir um valor para ele (ele é 0 default). Para adicionar o valor do contador em um elemento, use a função `counter()`. O CSS abaixo adiciona "Section [o valor do contador]:" no início de cada elemento h3.

```
body {
  counter-reset: section; /* Set the section counter to 0 */
}
h3::before {
  counter-increment: section; /* Increment the section counter*/
  content: "Section" counter(section) ": "; /* Display the counter */
}
```

Exemplo:

```
<h3>Introduction</h3>
<h3>Body</h3>
<h3>Conclusion</h3>
```

Nesting counters

Um contador CSS pode ser especialmente útil para fazer listas descritas porque uma nova instância de um contador CSS é criado automaticamente em elementos filho . Usando a função `counters()`, uma corda pode ser inserido entre diferentes níveis de contadores aninhados. Por exemplo, esta CSS :

```
ol {
  counter-reset: section; /* Creates a new instance of the
                             section counter with each ol
                             element */
  list-style-type: none;
}
li::before {
  counter-increment: section; /* Increments only this instance
                             of the section counter */
  content: counters(section, ".") " "; /* Adds the value of all instances
                             of the section counter separated
```

```
                                        by a ". */
  /* if you need to support < IE8 then
                                        make sure there is no space after
                                        the ',' */
}
```

Combinado com o seguinte HTML:

```
<ol>
  <li>item</li>          <!-- 1     -->
  <li>item               <!-- 2     -->
    <ol>
      <li>item</li>      <!-- 2.1   -->
      <li>item</li>      <!-- 2.2   -->
      <li>item           <!-- 2.3   -->
        <ol>
          <li>item</li>  <!-- 2.3.1 -->
          <li>item</li>  <!-- 2.3.2 -->
        </ol>
        <ol>
          <li>item</li>  <!-- 2.3.1 -->
          <li>item</li>  <!-- 2.3.2 -->
          <li>item</li>  <!-- 2.3.3 -->
        </ol>
      </li>
      <li>item</li>      <!-- 2.4   -->
    </ol>
  </li>
  <li>item</li>          <!-- 3     -->
  <li>item</li>          <!-- 4     -->
</ol>
<ol>
  <li>item</li>          <!-- 1     -->
  <li>item</li>          <!-- 2     -->
</ol>
```

Produz este resultado:

Veja mais

- `contain`
- `counter-reset`
- `counter-set`
- `counter-increment`
- `@counter-style`
- CSS counter styles module
- CSS lists and counters module

CSS Display

CSS Display É um módulo de CSS que define como a árvore da caixa de formatação CSS é gerada a partir da árvore de elementos do documento e define as propriedades que a controlam.

Referência

CSS Propriedades

- `display`

CSS tipos de dados

- `<display-outside>`
- `<display-inside>`
- `<display-listitem>`
- `<display-box>`
- `<display-internal>`
- `<display-legacy>`

Guias

CSS Fluxo Layout (<code>display: block</code>, <code>display: inline</code>)

- Layout de bloco e embutido no fluxo normal
- Layout de fluxo e Overflow
- Layout de fluxo e modos de gravação
- Contextos de formatação explicados
- No fluxo e fora do fluxo

<code>display: flex</code>

- Conceitos básicos de flexbox
- Alinhando itens em um container Flex
- Relações de controle de itens flexíveis ao longo do eixo principal
- Combinações Flexbox entre navegadores
- Dominando o agrupamento de itens flexíveis
- Pedido de itens flexíveis
- Relação do flexbox com outros métodos de layout
- Compatibilidade com versões anteriores do Flexbox
- Casos de uso típicos do Flexbox

<code>display: grid</code>

- Conceitos básicos de layout de grade
- Relação com outros métodos de layout
- Posicionamento baseado em linha
- Áreas de modelo de grade
- Layout usando linhas de grade nomeadas
- Posicionamento automático no layout da grade
- Alinhamento de caixa no layout da grade
- Grades, valores lógicos e modos de escrita
- Layout e acessibilidade da grade CSS
- Layout de grade CSS e aprimoramento progressivo
- Realizando layouts comuns usando grades

Compatibilidade com navegadores

CSS Flexible Box Layout

CSS Flexible Box Layout é um módulo do CSS que define um layout multicoluna otimizado para o design de interfaces de usuário, para isso busca lidar com o layout dos itens de maneira unidimensional. No modelo de layout do Flexbox, os filhos de um contêiner flex podem ser dispostos em qualquer direção, e podem "flexibilizar" seus tamanhos, crescendo para preencher o espaço vazio ou diminuindo para evitar o transbordamento do elemento pai. O alinhamento horizontal e vertical dos filhos podem ser facilmente manipulados.

Exemplo Básico

No exemplo a seguir um contêiner foi definido como `display: flex`, o que significa que os três items filhos dele se tornaram flex. O valor de `justify-content` foi definido como `space-between` para espaçar os itens uniformemente no eixo principal. Uma quantidade igual de espaço é colocada entre cada item com a esquerda e direita dos itens sendo niveladas com as bordas do contêiner flex. Você também pode perceber que os items estão alongados no eixo transversal, isso acontece por causa do valor padrão de `align-items` ser `stretch`(esticar). Os itens se estendem até a altura do contêiner flex, fazendo com que eles sejam tão altos quanto o item mais alto.

Referência

Propriedades CSS

- `flex`
- `flex-basis`
- `flex-direction`
- `flex-flow`
- `flex-grow`
- `flex-shrink`
- `flex-wrap`
- `order`

Propriedades de Alinhamento

A propriedade `align-content`, `align-self`, `align-items` e `justify-content` apareceram inicialmente na especificação do Flexbox, mas agora estão definidas em Box Alignment. As especificações do Flexbox agora referência a especificação do Box Alignment para definições atualizadas. Propriedades de alinhamento adicional agora também são definidas em Box Alignment.

- `justify-content`
- `align-content`
- `align-items`
- `align-self`
- `place-content`
- `place-items`
- `row-gap`
- `column-gap`
- `gap`

Entradas do glossário

- Flexbox
- Flex Container
- Flex Item
- Main Axis
- Cross Axis
- Flex

Guia

Conceitos básicos de flexbox

Uma visão geral sobre as funcionalidades do Flexbox

Relação entre o Flexbox e os outros métodos de layout

Como o Flexbox está relacionado aos outros métodos de layout, e outras especificações CSS

Alinhando itens em um contêiner flex

Como as propriedades Box Alignment trabalham com Flexbox.

Ordenando itens flex

Explicando as diferentes formas de mudar a order e direção dos itens, e cobrindo os possíveis problemas em fazer isso.

Proporções de Controle de itens flex ao longo do eixo principal

Explicando as propriedades flex-grow, flex-shrink e flex-basis.

Dominando a embalagem de itens flex

Como criar contêiners flex com múltiplas linhas e controlar a exibição dos itens nessas linhas.

Casos típicos de uso do Flexbox

Padrão de projeto comuns que são casos de uso típicos do Flexbox.

Retrocompatibilidade do Flexbox

Status do Flexbox nos browsers, problemas de interoperabilidade e suporte para browsers antigos e versões das especificações

Veja também

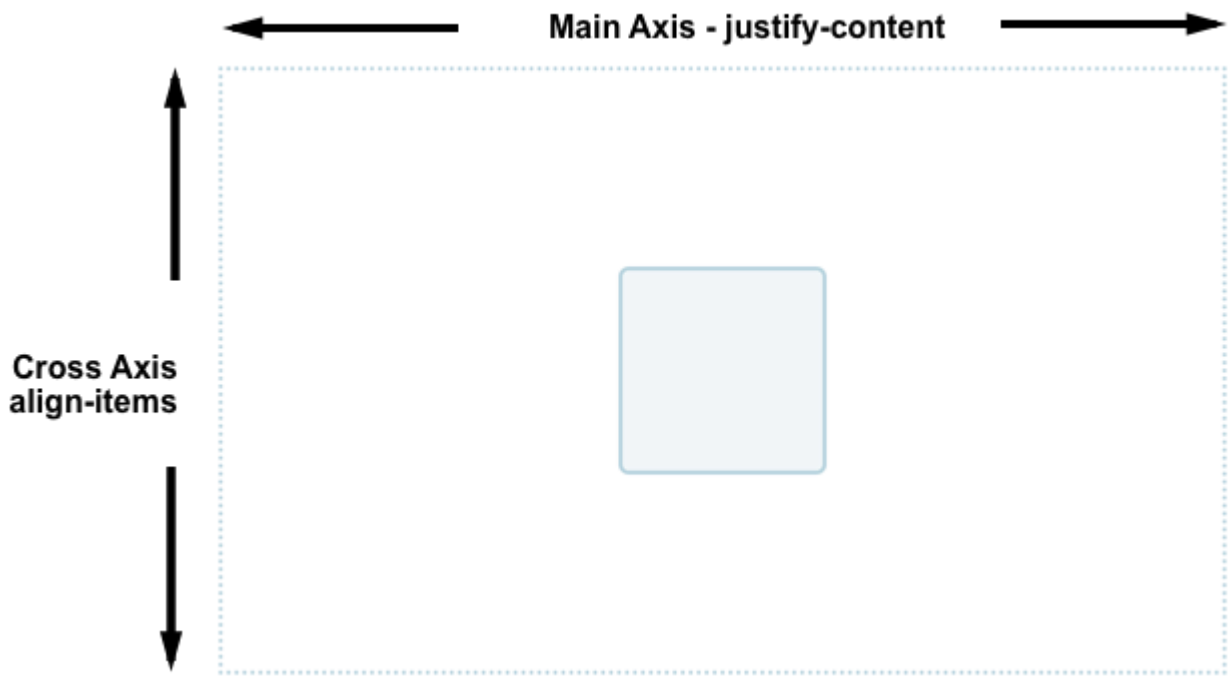
Flexbugs

Uma lista curada pela comunidade de bugs do Flexbox nos navegadores e soluções alternativas

Aligning Items in a Flex Container

Uma das razões pela qual flexbox rapidamente despertou interesse dos desenvolvedores web é que ela trouxe capacidade de alinhamento apropriada para web pela primeira vez. Tem capacidade de alinhamento vertical apropriada, e assim finalmente podemos de modo fácil centralizar uma caixa. Neste guia, veremos detalhadamente como as propriedades de alinhamento e justificação funcionam no Flexbox.

Para centralizar nossa caixa nós usamos a propriedade `align-items` para alinhar nosso item no eixo transversal, que neste caso é o eixo do bloco indo verticalmente. Nós usamos `justify-content` para alinhar o item no eixo principal, que neste caso é o eixo indo horizontalmente.



Você pode dar uma olhada no código deste exemplo abaixo. Altere o tamanho do container ou elemento aninhado e o elemento aninhado sempre permanece centralizado.

Propriedades que controlam o alinhamento

As propriedades que nós veremos neste guia são as seguintes.

- `justify-content` — controla o alinhamento de todos os itens no eixo principal.
- `align-items` — controla o alinhamento de todos os itens no eixo transversal.
- `align-self` — controla o alinhamento individual de um item no eixo transversal.
- `align-content` — descrito na especificação como "packing flex lines"; controla o espaço entre as linhas no eixo transversal.

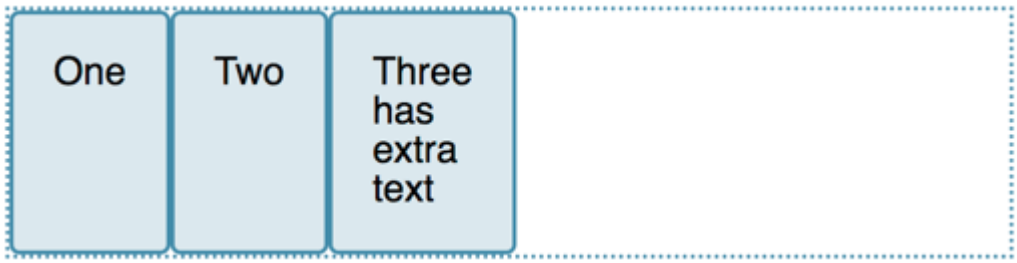
Nós também descobriremos como margens automáticas podem ser utilizadas para o alinhamento no flexbox.

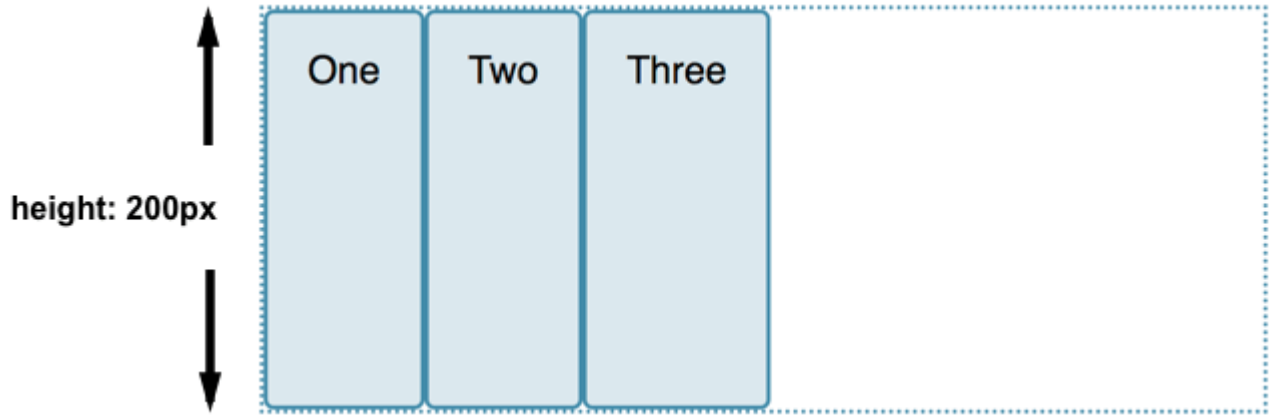
> **Nota:** The alignment properties in Flexbox have been placed into their own specification — CSS Box Alignment Level 3. It is expected that this spec will ultimately supersede the properties as defined in Flexbox Level One.

The Cross Axis

As propriedades `align-items` e `align-self` controlam o alinhamento dos nossos itens no eixo transversal, em colunas se `flex-direction` é `row` e entre linhas se `flex-direction` é `column`.

Nós estamos usando o alinhamento transversal na maioria dos exemplos. Se adicionarmos `display: flex` para um contêiner, todos os itens filhos se tornam itens flexíveis organizados em uma linha. Todos irão se esticar para serem tão altos quanto o item mais alto, pois este item define a altura dos itens no eixo transversal. Se o seu contêiner possui uma altura definida, então os itens se estenderão para esta altura, independemente da quantidade de conteúdo que está dentro deste item.





A razão pela qual os itens passam a ter a mesma altura é que o valor inicial do `align-items`, a propriedade que controla o alinhamento do eixo transversal, está definida como `stretch`.

Nós podemos usar outros valores para controlar como os itens se alinham:

- `align-items: flex-start`
- `align-items: flex-end`
- `align-items: center`
- `align-items: stretch`
- `align-items: baseline`

No exemplo abaixo, o valor do `align-items` está definido como `stretch`. Tente outros valores e veja como todos os itens se alinham no contêiner.

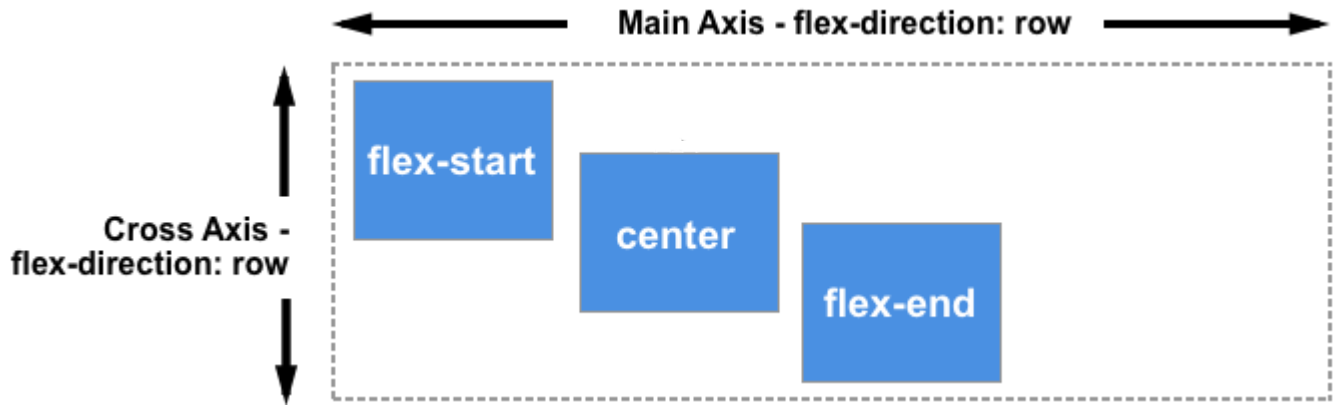
Alinhando um item com <code>align-self</code>

A propriedade `align-items` adiciona a propriedade `align-self` em todos os itens flex como um grupo. Ou seja, você pode explicitamente declarar a propriedade `align-self` para impactar um único item. A propriedade `align-self` aceita todos os mesmo valores que `align-items` mais o valor `auto`, que irá resetar o valor que está configurado no container flex.

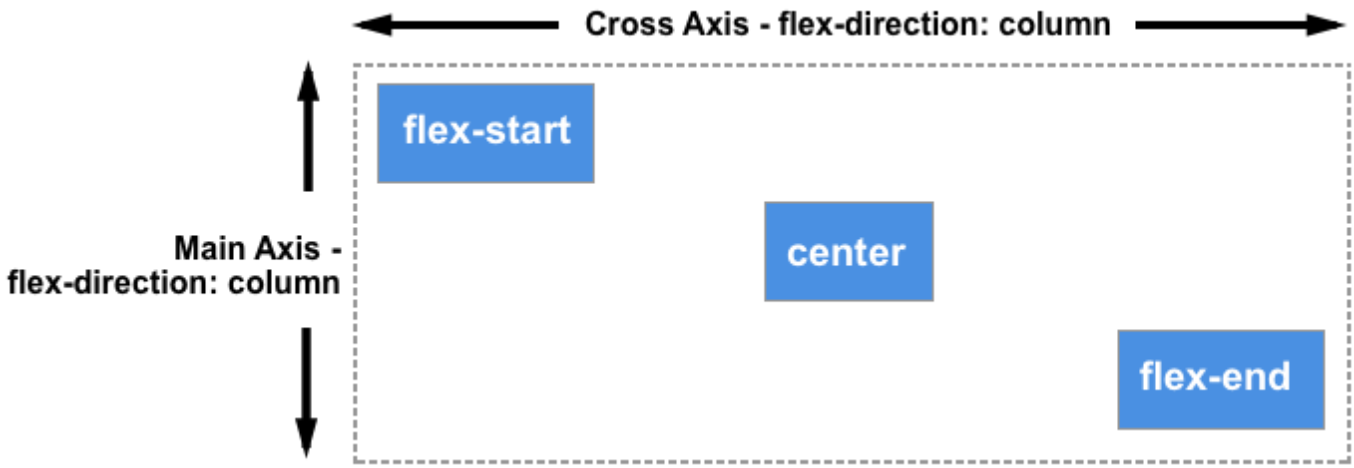
Neste próximo exemplo, o container flex possui `align-items: flex-start`, que implica em todos os itens estarem alinhados ao início do eixo transversal. Foquei no primeiro item usando o seletor `first-child` e configurei este item para `align-self: stretch`; outro item foi selecionado usando a classe `selected` e `align-self: center`. Você pode mudar o valor do `align-items` ou mudar o valor do `align-self` para focar em itens individuais para ver como funciona.

Changing the main axis

So far we have looked at the behaviour when our `flex-direction` is `row`, and while working in a language written top to bottom. This means that the main axis runs along the row horizontally, and our cross axis alignment moves the items up and down.



If we change our `flex-direction` to column, `align-items` and `align-self` will align the items to the left and right.



You can try this out in the example below, which has a flex container with `flex-direction: column` yet otherwise is exactly the same as the previous example.

Aligning content on the cross axis — the align-content property

So far we have been aligning the items, or an individual item inside the area defined by the flex-container. If you have a wrapped multiple-line flex container then you might also want to use the `align-content` property to control the distribution of space between the rows. In the specification this is described as packing flex lines.

For `align-content` to work you need more height in your flex container than is required to display the items. It then works on all the items as a set, and dictates what happens with that free space, and the alignment of the entire set of items within it.

The `align-content` property takes the following values:

- `align-content: flex-start`
- `align-content: flex-end`
- `align-content: center`
- `align-content: space-between`
- `align-content: space-around`
- `align-content: stretch`
- `align-content: space-evenly` (not defined in the Flexbox specification)

In the live example below, the flex container has a height of 400 pixels, which is more than needed to display our items. The value of `align-content` is `space-between`, which means that the available space is shared out *between* the flex lines, which are placed flush with the start and end of the container on the cross axis.

Try out the other values to see how the `align-content` property works.

Once again we can switch our `flex-direction` to `column` in order to see how this property behaves when we are working by column. As before, we need enough space in the cross axis to have some free space after displaying all of the items.

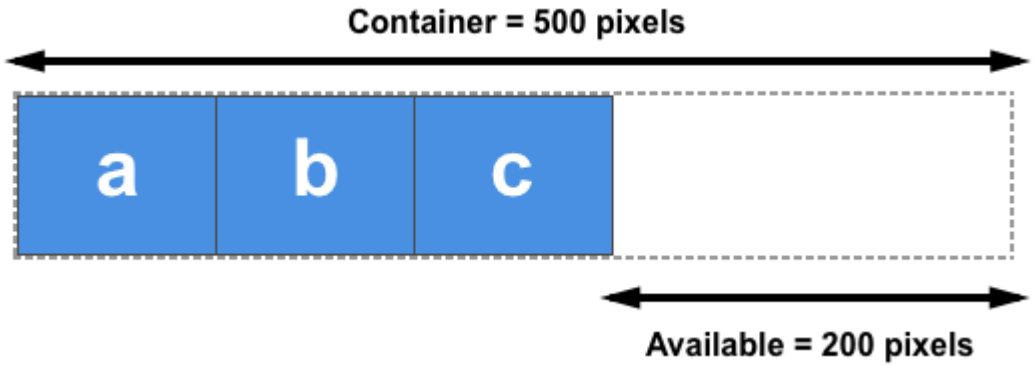
> **Nota:** the value `space-evenly` is not defined in the flexbox specification and is a later addition to the Box Alignment specification. Browser support for this value is not as good as that of the values defined in the flexbox spec.

See the documentation for `justify-content` on MDN for more details on all of these values and browser support.

Aligning content on the main axis

Now that we have seen how alignment works on the cross axis, we can take a look at the main axis. Here we only have one property available to us — `justify-content`. This is because we are only dealing with items as a group on the main axis. With `justify-content` we control what happens with available space, should there be more space than is needed to display the items.

In our initial example with `display: flex` on the container, the items display as a row and all line up at the start of the container. This is due to the initial value of `justify-content` being `flex-start`. Any available space is placed at the end of the items.



The `justify-content` property accepts the same values as `align-content`.

- `justify-content: flex-start`
- `justify-content: flex-end`
- `justify-content: center`
- `justify-content: space-between`
- `justify-content: space-around`
- `justify-content: stretch`
- `justify-content: space-evenly` (not defined in the Flexbox specification)

In the example below, the value of `justify-content` is `space-between`. The available space after displaying the items is distributed between the items. The left and right item line up flush with the start and end.

If the main axis is in the block direction because `flex-direction` is set to `column`, then `justify-content` will distribute space between items in that dimension as long as there is space in the flex container to distribute.

Alignment and Writing Modes

Remember that with all of these alignment methods, the values of `flex-start` and `flex-end` are writing mode-aware. If the value of `justify-content` is `start` and the writing mode is left-to-right as in English, the items will line up starting at the left side of the container.



However if the writing mode is right-to-left as in Arabic, the items will line up starting at the right side of the container.



The live example below has the `direction` property set to `rtl` to force a right-to-left flow for our items. You can remove this, or change the values of `justify-content` to see how flexbox behaves when the start of the inline direction is on the right.

Alignment and flex-direction

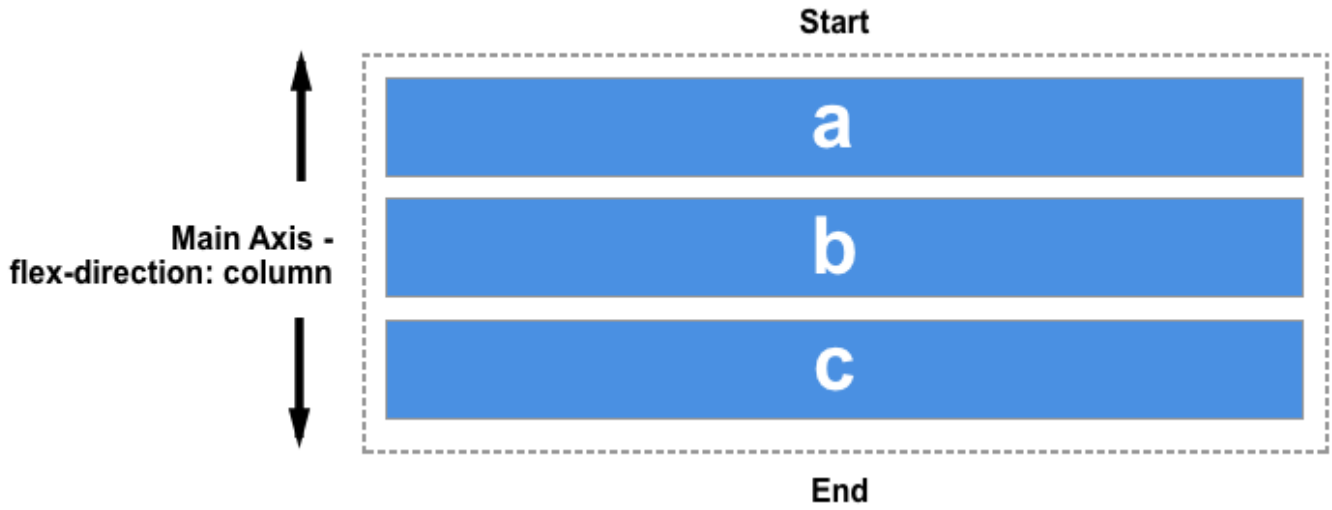
The start line will also change if you change the `flex-direction` property — for example using `row-reverse` instead of `row`.

In this next example I have items laid out with `flex-direction: row-reverse` and `justify-content: flex-end`. In a left to right language the items all line up on the left. Try changing `flex-direction: row-reverse` to `flex-direction: row`. You will see that the items now move to the right hand side.

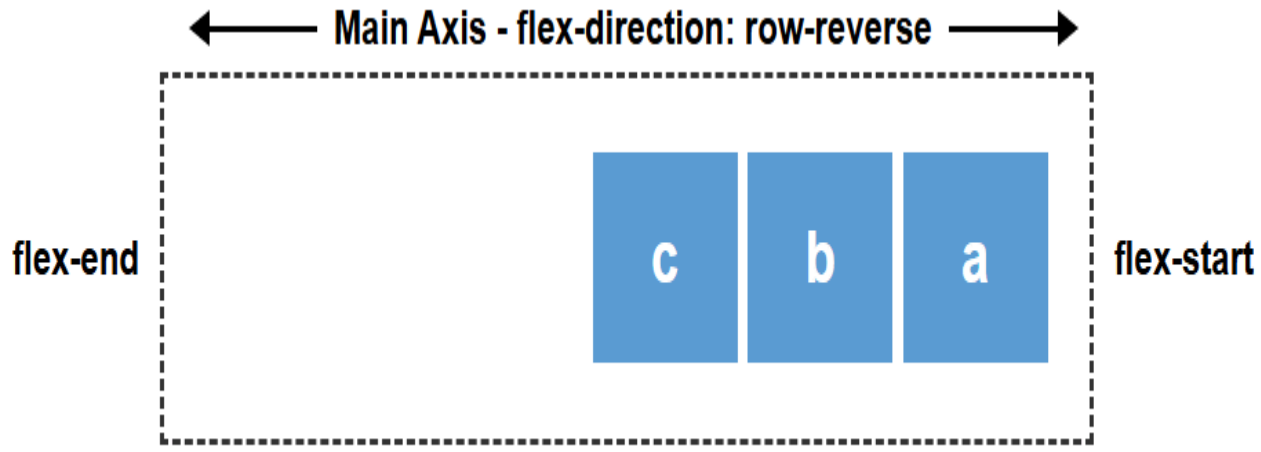
While this may all seem a little confusing, the rule to remember is that unless you do something to change it, flex items lay themselves out in the direction that words are laid out in the language of your document along the inline, row axis. `flex-start` will be where the start of a sentence of text would begin.

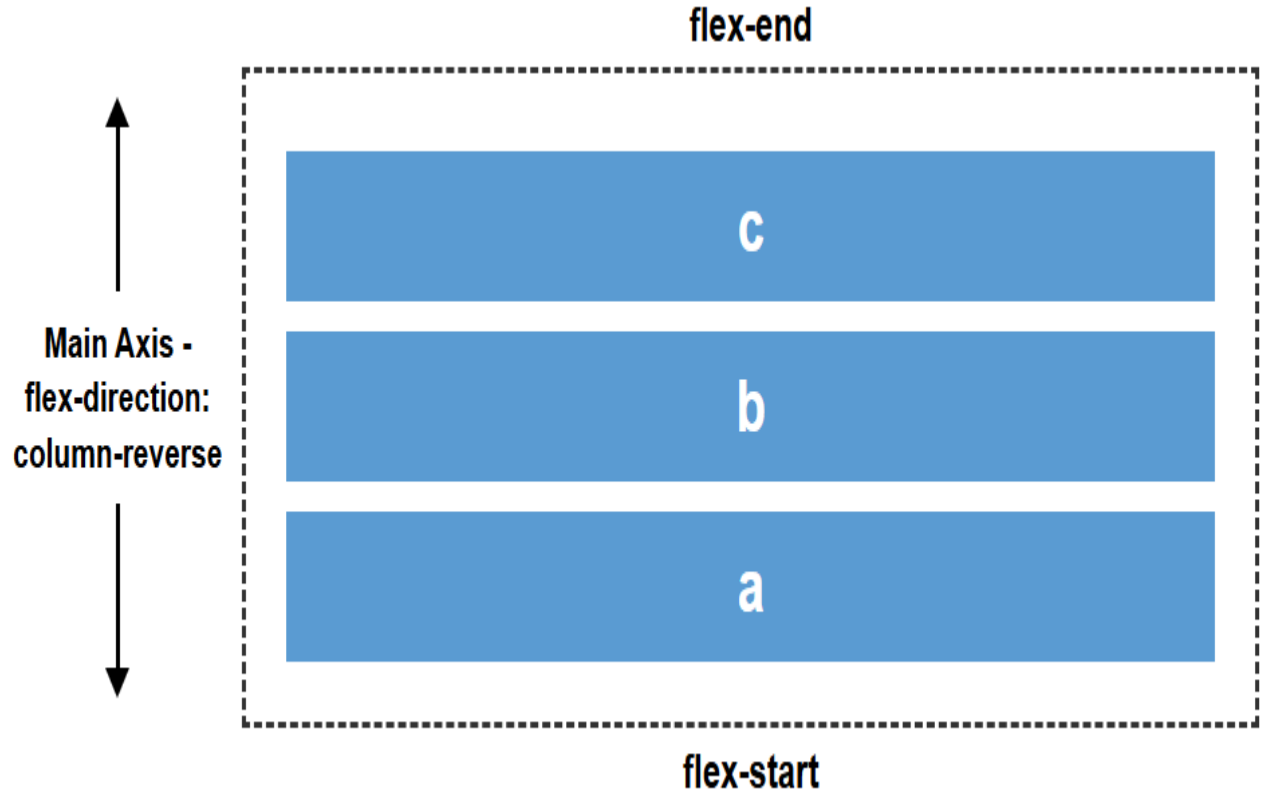


You can switch them to display in the block direction for the language of your document by selecting `flex-direction: column`. Then `flex-start` will then be where the top of your first paragraph of text would start.



If you change flex-direction to one of the reverse values, then they will lay themselves out from the end axis and in the reverse order to the way words are written in the language of your document. `flex-start` will then change to the end of that axis — so to the location where your lines would wrap if working in rows, or at the end of your last paragraph of text in the block direction.





Using auto margins for main axis alignment

We don't have a `justify-items` or `justify-self` property available to us on the main axis as our items are treated as a group on that axis. However it is possible to do some individual alignment in order to separate an item or a group of items from others by using auto margins along with flexbox.

A common pattern is a navigation bar where some key items are aligned to the right, with the main group on the left. You might think that this should be a use case for a `justify-self` property, however consider the image below. I have three items on one side and two on the other. If I were able to use `justify-self` on item *d*, it would also change the alignment of item *e* that follows, which may or may not be my intention.



Instead we can target item 4 and separate it from the first three items by giving it a `margin-left` value of `auto`. Auto margins will take up all of the space that they can in their axis — it is how centering a block with margin auto left and right works. Each side tries to take as much space as it can, and so the block is pushed into the middle.

In this live example, I have flex items arranged simply into a row with the basic flex values, and the class `push` has `margin-left: auto`. You can try removing this, or adding the class to another item to see how it works.

Future alignment features for Flexbox

At the beginning of this article I explained that the alignment properties currently contained in the Level 1 flexbox specification are also included in Box Alignment Level 3, which may well extend these properties and values in the future. We have already seen one place where this has happened, with the introduction of the `space-evenly` value for `align-content` and `justify-content` properties.

The Box Alignment module also includes other methods of creating space between items, such as the `column-gap` and `row-gap` feature as seen in CSS Grid Layout. The inclusion of these properties in Box Alignment means that in future we should be able to use `column-gap` and `row-gap` in flex layouts too, and in Firefox 63 you will find the first browser implementation of the gap properties in flex layout.

My suggestion when exploring flexbox alignment in depth is to do so alongside looking at alignment in Grid Layout. Both specifications use the alignment properties as detailed in the Box Alignment specification. You can see how these properties behave when working with a grid in the MDN article Box Alignment in Grid Layout, and I have also compared how alignment works in these specifications in my Box Alignment Cheatsheet.

See Also

- Box Alignment
- Box Alignment in Flexbox
- Box Alignment in Grid Layout

Conceitos básicos de flexbox

O *Flexible Box Module*, geralmente chamado de *flexbox*, foi projetado tanto como um modelo de *layout* unidimensional quanto como um método capaz de organizar espacialmente os elementos em uma interface, além de possuir capacidades avançadas de alinhamento. Este artigo fornece um resumo das principais funcionalidades do *flexbox*, as quais exploraremos com mais detalhes no restante deste guia.

Quando se descreve o flexbox como sendo unidimensional, enfatiza-se o fato de que ele lida com *o layout* em uma dimensão de cada vez - seja uma linha ou uma coluna. Isto pode ser comparado com o modelo bidimensional de CSS - Layout de Grade, que permite o controle simultâneo das colunas e linhas.

Os eixos do flexbox

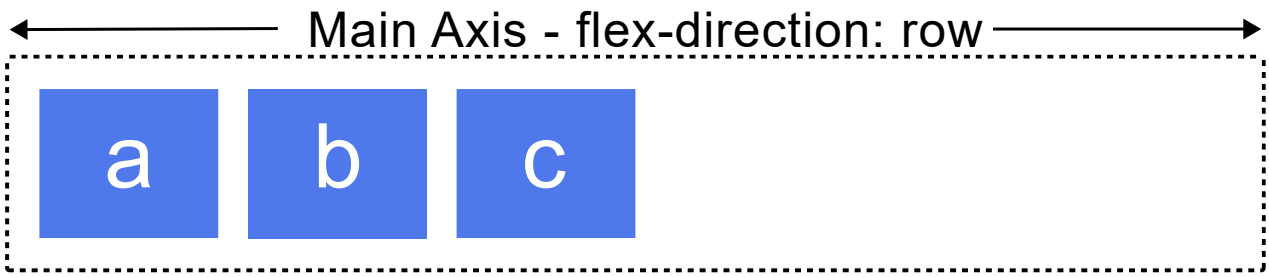
Ao se utilizar o *flexbox*, é preciso ter em mente que todas as operações realizadas relacionam-se a dois eixos: o eixo principal e o eixo transversal. O eixo principal é definido através da propriedade `flex-direction` e o eixo transversal encontra-se na direção perpendicular a ele. Como esses eixos são as engrenagens fundamentais do flexbox é necessário compreender minuciosamente o seu funcionamento.

Eixo principal

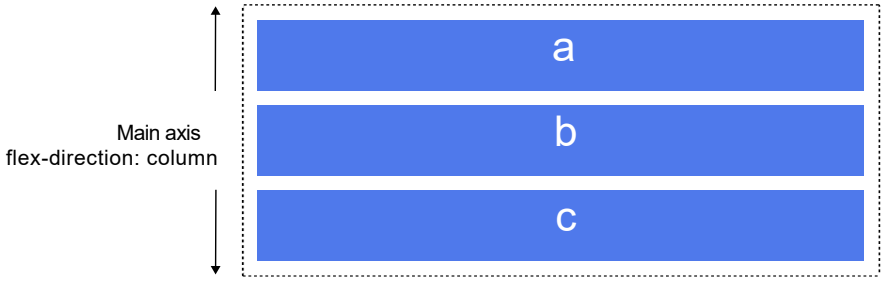
Conforme descrito acima, a propriedade `flex-direction` define a direção do eixo principal e pode tem quatro valores possíveis:

- `row`
- `row-reverse`
- `column`
- `column-reverse`

Se o valor escolhido for `row` (linha) ou `row-reverse` (linha reversa), seu eixo principal se moverá ao longo da linha — na **direção inline**.

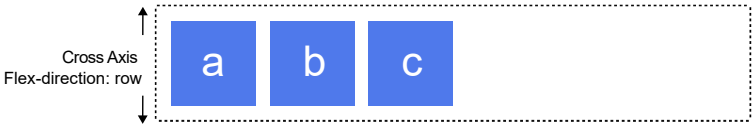


Se o valor escolhido for `column` (coluna) ou `column-reverse` (coluna reversa) e o eixo principal se moverá do topo até o fim da página — na **direção block**.

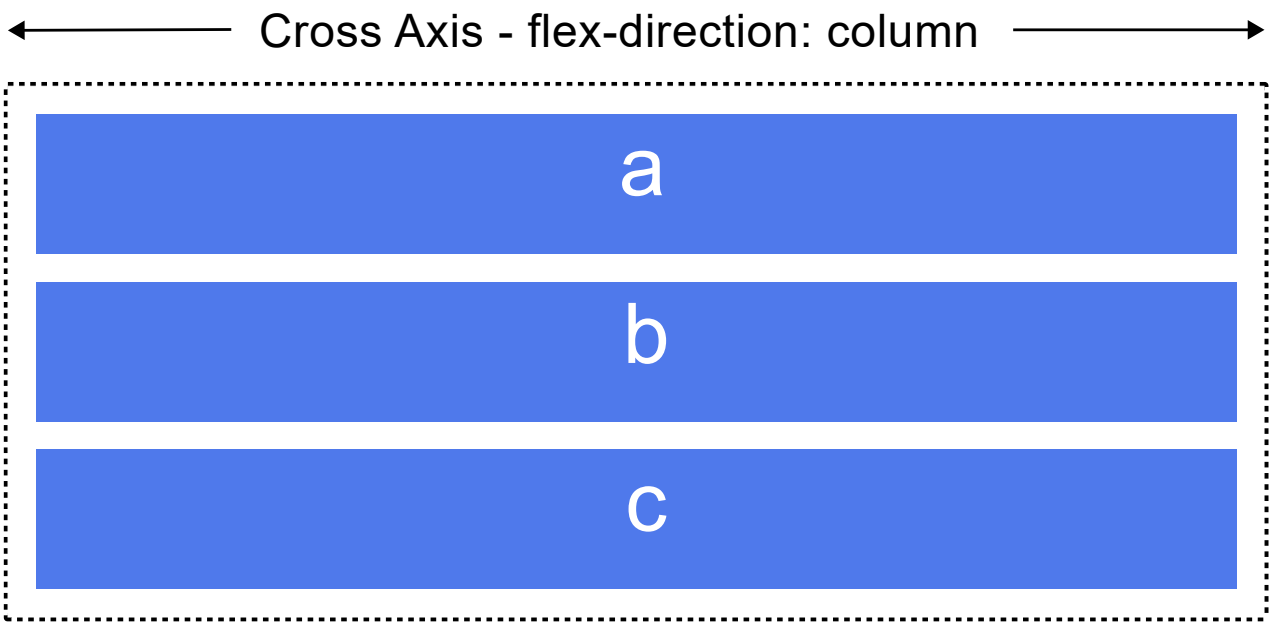


Eixo transversal

O eixo transversal é perpendicular ao eixo principal, logo, se a propriedade `flex-direction` estiver definida nas linhas, como `row` ou `row-reverse`, o eixo transversal estará na direção das colunas, como `column` ou `column-reverse`.



Se o eixo principal for definido nas colunas, como `column` ou `column-reverse`, então o eixo transversal estará na direção das linhas, como `row` ou `row-reverse`.



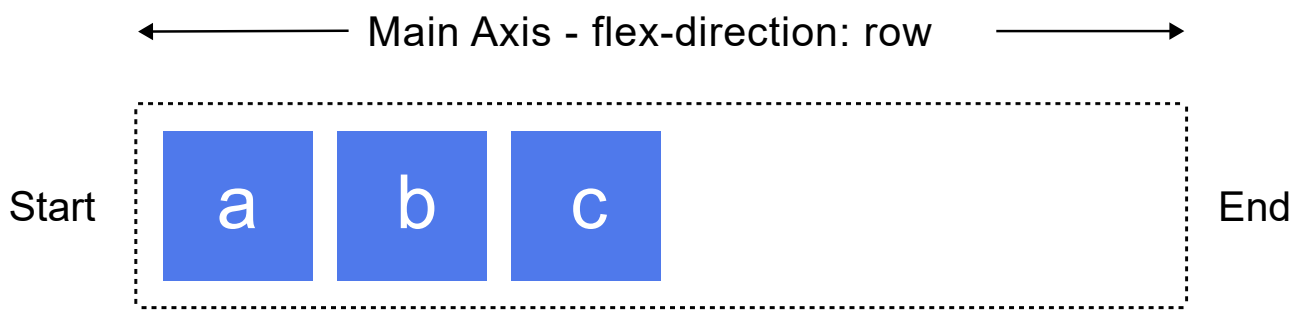
Compreender a diferença entre os eixos principal e perpendicular é o que importa quando começamos a observar o alinhamento ou justificação dos itens flexíveis (flex items); o *flexbox* possui propriedades que alinham e justificam o conteúdo ao longo de um eixo ou de outro.

Linhas de Início e Fim

Outra área crítica em termos de compreensão é como o Flexbox não faz nenhuma premissa sobre o modo de escrita do documento. No passado, o CSS utilizava fortemente os modos de escrita horizontal e da esquerda para a direita. Métodos modernos de layout abrangem uma variedade de modos de escrita e, portanto, não assumimos mais que uma linha de texto começará no canto superior esquerdo do documento e sigará para o lado direito, com novas linhas aparecendo uma após a as outra.

Essa discussão sobre a relação entre o *flexbox* e a especificação do modo de escrita será abordada um artigo posterior, contudo, a descrição a seguir explica brevemente porque não se fala sobre esquerda e direita/ acima e abaixo quando descreve-se a direção para a qual os elementos *flex* fluem.

Se o valor da propriedade `flex-direction` for `row (linha)`, considerando o estilo de escrita ocidental, a borda inicial do eixo principal estará localizada à esquerda e a borda final, à direita.



Considerando um idioma como o Árabe, que possui um estilo de escrita oriental, teremos o inverso: a borda inicia do eixo principal estará localizada à direita e a borda final, à esquerda.



Em ambos os casos, a borda inicial do eixo transversal está na parte superior do contêiner flex e a borda final, na parte inferior, visto que ambos os idiomas têm um estilo de de escrita horizontal.

Após um tempo de prática, pensar em termos de início e final ao invés de esquerda e direita se tornará natural e será útil ao lidar com outros métodos de layout, como CSS Grid, que seguem os mesmos padrões.

Contêiner Flex

A área de um documento que faz uso do *flexbox* é chamada de **contêiner *flex***. Para criar essa estrutura, define-se o valor da propriedade `display` do elemento representado pelo contêiner como `flex` ou `inline-flex`. Desse modo, os elementos-filhos desse contêiner tornar-se-ão do tipo *flex*. Como todas as propriedades no CSS possuem valores padrão, ao criar um contêiner flex, os elementos nele contidos apresentarão o seguinte comportamento:

- Exibição em linha (o padrão do `flex-direction` é `row`).
- Alinhamento na borda inicial do eixo principal.
- Não há expansão no eixo principal, mas pode haver contração.
- Haverá expansão vertical para preencher a altura do eixo transversal.
- A propriedade `flex-basis` estará definida como `auto`.
- A propriedade `flex-wrap` estará definida como `nowrap`.

O resultado final é que todos os elementos serão alinhados em uma linha, usando o tamanho do conteúdo como o tamanho no eixo principal. Se houver mais itens do que é possível caber no container, não haverá uma quebra de linha; ao invés disso, irão ultrapassar o limite horizontal da página. Se alguns elementos forem mais altos que outros, todos os itens se estenderão ao longo do eixo transversal para preencher seu tamanho total.

É possível conferir essas questões no exemplo prático abaixo. Tente editar ou adicionar mais itens para testar o comportamento inicial do Flexbox.

Propriedade <code>flex-direction</code>

A propriedade `flex-direction` permite alterar a direção na qual os elementos flex serão exibidos ao longo do eixo principal. Definindo a propriedade `flex-direction` como `row-reverse` (linha reversa) ainda teremos os elementos dispostos em uma linha, entretanto, as linhas inicial e final serão trocadas.

Se mudarmos a `flex-direction` para a `column` (coluna), o eixo principal exibirá os elemento em uma coluna. Trocando o valor para `column-reverse` (coluna reversa) fará com que as linhas inicial e final sejam novamente trocadas.

No exemplo prático abaixo tem-se a propriedade `flex-direction` definida como `row-reverse`. Experimente os outros valores - `row`, `column` e `column-reverse` - para ver o que acontece com o conteúdo.

Quebra de linha com "flex-wrap"

Embora o *flexbox* seja um modelo unidimensional, é possível fazer com que elementos *flex* sejam agrupados em múltiplas linhas. Ao fazer isso, considera-se cada linha como um novo contêiner *flex*. Qualquer distribuição espacial ocorrerá ao longo essa linha, sem referência às linhas de ambos os lados. Para gerar a quebra automática das linhas adicione a propriedade `flex-wrap` com o valor `wrap`. Assim, se elementos forem muito grandes para serem exibidos em uma única linha, eles serão agrupados em outras linhas.

O exemplo prático abaixo contém elementos *flex* aos quais foi dada uma determinada largura, cuja soma totaliza um valor grande demais para caber no container. Visto que a propriedade `flex-wrap` está definida como `wrap`, os itens serão reorganizados em mais de uma linha. Trocando-se para `nowrap`, que também é o valor inicial, eles encolherão para caber no contêiner, porque estão usando valores iniciais de flexbox que permitem que os itens encolham. O uso do `nowrap` causaria um vazamento se os itens não encolhessem ou não diminuíssem o suficiente para caber.

Saiba mais sobre o empacotamento de itens flexíveis no guia Masterização de Empacotamento de Itens Flexíveis (em Inglês).

Propriedade abreviada flex-flow

Pode-se combinar as propriedades `flex-direction` e `flex-wrap` de forma abreviada através da propriedade `flex-flow`. O primeiro valor especificado é o `flex-direction` e o segundo valor é o `flex-wrap`.

No exemplo prático abaixo, tente alterar o primeiro valor para um dos valores permitidos para a propriedade `flex-direction` - `row`, `row-reverse`, `column` ou `column-reverse`, e também altere o segundo para `wrap` e `nowrap`.

Expansão, encolhimento e tamanho dos elementos flex

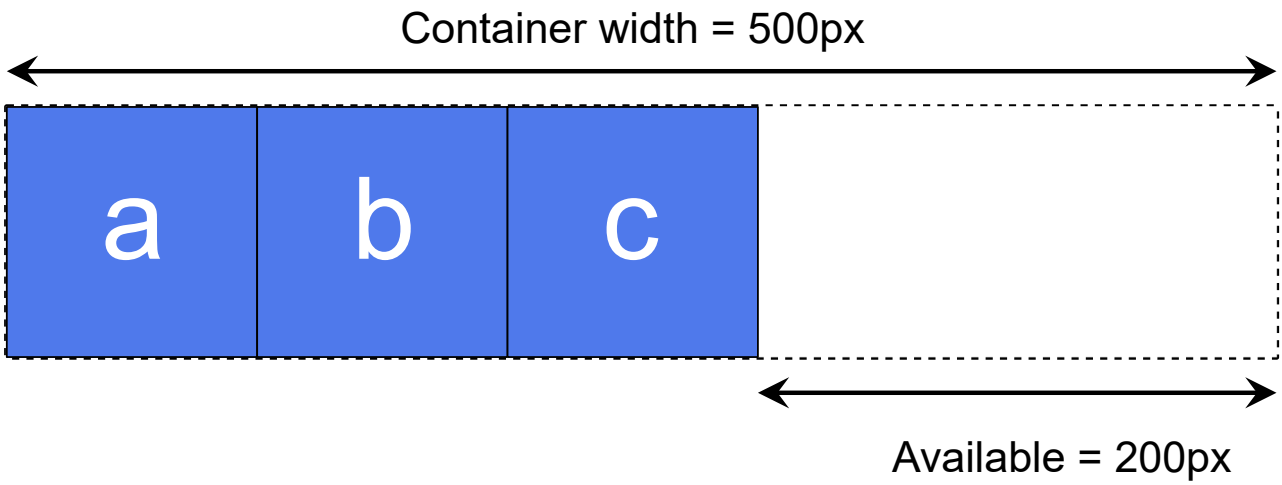
Para ter mais controle sobre os elementos *flex* é possível alterá-los diretamente utilizando as três propriedades abaixo:

- `flex-grow`
- `flex-shrink`
- `flex-basis`

Na presente seção, examinar-se-á brevemente tais propriedades. Para se aprofundar neste conteúdo sugere-se o tutorial Taxas de Controle de Elementos Flex ao Longo do Eixo Principal (em inglês).

Antes que essas propriedades possam fazer sentido, é preciso compreender o conceito de **espaço disponível**. Quando se modifica o valor das propriedades acima, altera-se a forma que o espaço disponível é distribuído entre os elementos. Tal conceito de espaço disponível também é relevante quando se trata do alinhamento.

Conforme o exemplo abaixo, se houver três elementos com 100 pixels de comprimento em um contêiner de 500 pixels, então o espaço total necessário para acomodá-los será de 300 pixels. Desse modo, sobrarão 200 pixels de espaço útil. Se os valores iniciais não forem modificados, então o *flexbox* posicionará esse espaço após o último item.



Se for necessário que os elementos cresçam proprocionamente ou não e preencham o espaço disponível, deverá haver método que faça essa distribuição espacial entre eles, conforme será visto nas seções subsequentes.

Propriedade <code>flex-basis</code>

A propriedade `flex-basis` define o tamanho inicial dos elementos, em unidades de *pixel*, antes que o espaço remanescente seja redistribuído. O valor inicial desta propriedade é `auto` — neste caso o navegador observa se os itens possuem o mesmo tamanho. No exemplo acima, todos os itens têm uma largura de 100 pixels, que é utilizada como padrão na propriedade `flex-basis`.

Se os elementos não possuírem um tamanho padrão, então as dimensões dos seus conteúdos (imagem, texto, etc) serão passadas como parâmetro para propriedade flex-basis. É por isso que quando escreve-se `display: flex` no elemento-pai para criar o contêiner, todos os elementos-filhos se organizam em linha e ocupam apenas o espaço necessário para exibir seu conteúdo.

Propriedade <code>flex-grow</code>

Com a propriedade `flex-grow` definida como um inteiro positivo, os elementos *flex* podem crescer ao longo do eixo principal, a partir do valor mínimo estabelecido no `flex-basis`. Isto fará com que o elemento se estique e ocupe qualquer espaço disponível nesse eixo ou uma proporção dele, caso outros elementos-irmãos também possam crescer.

Atribuir o valor 1 à propriedade `flex-grow` fará com que o espaço disponível no contêiner *flex* seja igualmente distribuído entre todos os elementos do exemplo acima. Logo, os elementos-filhos irão se expandir para preencher o contêiner no sentido do eixo principal.

Como visto no parágrafo anterior, a propriedade `flex-grow` pode ser empregada para distribuir o espaço proporcionalmente entre os elementos de um contêiner, contudo, se atribuirmos ao primeiro elemento o valor 2 e 1 aos elementos restantes, duas partes serão dadas ao primeiro elemento (100px de 200px totais) e uma parte para cada um dos outros dois elementos (50px de 200px totais).

Propriedade <code>flex-shrink</code>

Enquanto a propriedade `flex-grow` permite aumentar a largura dos elementos dentro do contêiner para completar o espaço disponível no eixo principal, a propriedade `flex-shrink` faz o oposto, controlando a redução dos mesmos. Caso não haja espaço suficiente para acomodar todos os elementos e o valor da propriedade `flex-shrink` seja um inteiro positivo, a largura pode ser reduzida a um valor menor do que a definida na propriedade `flex-basis`. Assim como na propriedade flex-grow, diferentes valores podem ser atribuídos a um elemento de modo que ele encolha mais do que os outros - um elemento cuja propriedade `flex-shrink` receba um valor inteiro maior irá diminuir mais do que os seus irmão que tenham valores menores.

O tamanho mínimo do elemento será levado em consideração ao se calcular a quantidade real de encolhimento que ocorrerá, o que significa que a propriedade flex-shrink se comporta de modo potencialmente menos consistente do que a propriedade flex-grow. Examinar-se-á mais detalhadamente o funcionamento desse algoritmo no artigo Taxas de Controle de Elementos Flex ao Longo do Eixo Principal.

> **Nota:** Note que os valores para as propriedades `flex-grow` e `flex-shrink` são proporcionais. Particularmente, se tivermos todos os nossos elementos definidos como flex: `1 1 200px` e então quisermos que um deles cresça o dobro, temos de definir o elemento como flex: `2 1 200px`. Entretanto, podemos escrever flex: `10 1 200px` e flex: `20 1 200px` se quisermos.

Abreviatura para os valores das propriedades flex

As propriedades `flex-grow`, `flex-shrink`, and `flex-basis` raramente são empregas de forma individual. Usualmente, elas são combinadas através da propriedade de abreviação `flex`. A abreviatura `flex` permite definir os três valores na seguinte ordem: `flex-grow`, `flex-shrink`, `flex-basis`.

O exemplo prático abaixo permite que sejam testados diferentes valores com a propriedade de abreviação `flex`; lembre-se que o primeiro campo corresponde à propriedade `flex-grow`, onde um valor inteiro e positivo faz-se o elemento crescer. O segundo campo é a propriedade `flex-shrink` e, ao contrário do anterior, um valor inteiro e positivo pode fazer os elementos encolherem, mas somente se o seu comprimento total ultrapassar o limite horizontal do contêiner, no sentido do eixo principal. O último campo contém a propriedade `flex-basis`, que define o valor base, em unidades de *pixel*, para aumentar e diminuir o elemento quando da aplicação das propriedades anteriores.

Há ainda alguns valores de abreviação predefinidos, que cobrem a maioria dos casos de uso. São aplicados com frequência em turoriais e, normalmente, suprem todas as necessidades práticas. Os valores predefinidos podem ser vistos abaixo:

- `flex: initial`
- `flex: auto`
- `flex: none`
- `flex: <positive-number>`

Definir `flex: initial` reseta os elementos para valores-padrão do Flexbox, sendo equivale a `flex: 0 1 auto`. Neste ultimo caso, o valor da propriedade `flex-grow` é 0, então os elementos não irão crescer mais do que o tamanho-base definido na propriedade `flex-basis`. o valor da propriedade `flex-shrink` é 1, indicando que o elemento pode ser reduzido caso seja necessário, para evitar que o limite do contêiner seja ultrapassado. Por fim, o valor da propriedade `flex-basis` é `auto` e assim será usad o tamanho mínimo necessário para preencher a dimensão do eixo principal.

Definir `flex: auto` é equivalente a `flex: 1 1 auto`. Essa configuração é semelhante a `flex:initial`, mas nesse caso os elementos podem aumentar para preencher o contêiner ou diminuir se necessário, para evitar o transbordamento lateral da tela.

Definir `flex: none` irá criar um elemento flex totalmente inflexível, sendo o equivalente a escrever `flex: 0 0 auto`. O elementos não poderão crescer ou diminuir, mas serão criados usando o *flexbox* com a propriedade `flex-basis` com o valor `auto`.

Outra abreviação normalmente vista em tutoriais é `flex: 1` ou `flex: 2` e assim por diante, o que equipara-se a `flex: 1 1 0`. Os elementos podem crescer ou diminuir a partir da propriedade `flex-basis` com valor nulo.

Teste essas formas abreviadas no exemplo prático abaixo:

Alinhamento, justificação e distribuição de espaço livre entre os elementos

Um atributo chave do *flexbox* é a capacidade de alinhar e justificar os elementos *flex* nos eixos principal e transversal e distribuir o espaço entre eles.

Propriedade <code>align-items</code>

A propriedade `align-items` irá alinhar os elementos no eixo transversal.

O valor inicial desta propriedade é `stretch` e é por essa razão que, por padrão, os elementos *flex* se estendem até a maior altura. De fato, eles se esticam para preencher o contêiner flex - o item mais alto define a altura deste.

Pode-se definir a propriedade `align-items` como `flex-start`, de modo que os elementos fiquem alinhados com topo do contêiner, `flex-end` para alinhá-los a partir da base ou `center`, para que o alinhamento seja centralizado.

Teste essa propriedade e seus possíveis valores no exemplo prático abaixo — colocou-se uma determinada altura no contêiner flex, de modo que se perceba como os elementos podem ser movidos no interior do mesmo. Veja o que acontece ao definir cada um dos possíveis valores da propriedade **align-items**:

- `stretch`
- `flex-start`
- `flex-end`
- `center`

Propriedade <code>justify-content</code>

A propriedade `justify-content` é empregada para alinhar os elementos ao longo do eixo principal, cuja direção (`row` ou `column`) é definida a partir da propriedade `flex-direction`. O valor inicial é `flex-start`, que alinha os elementos rente à borda esquerda do contêiner, mas também pode ser definido como `flex-end`, que resulta em um alinhamento oposto, rente à borda direita do contêiner, ou `center`, para alinhá-los ao centro.

O valor `space-between` pode ser usado pode ser usado para ocupar todo o espaço livre após a disposição dos itens e dividí-lo igualmente entre os itens, para que haja a mesma quantidade de espaço entre cada elemento. Para gerar uma quantidade igual de espaço à direita e à esquerda, usa-se o valor `space-around`.

Tente os seguintes valores da propriedada `justify-content` no exemplo prático abaixo:

- `stretch`
- `flex-start`
- `flex-end`
- `center`
- `space-around`
- `space-between`

No artigo Alinhando Elementos em um Contêiner Flex (em inglês) tais propriedades serão abordadas mais detalhadamente, de modo a compreender melhor o seu funcionamento. Contudo, os exemplos simples abordados aqui serão úteis na maioria dos casos.

Próximos passos

Após ler este artigo, você deve ser capaz de compreender as características básicas do Flexbox. No próximo artigo, iremos examinar como essa especificação se relaciona com outras partes do CSS (em inglês).

CSS Grid Layout

> **Nota:** CSS Grid será suportado por vários navegadores até meados de 2017. O suporte em navegadores antigos pode ser obtido habilitando-se uma flag que permite o uso da API. Nesse caso não se esqueça de consultar e fazer referência a cada propriedade e funcionalidade da especificação para certificar-se da sua compatibilidade, bem como para obter maiores informações.

CSS Grid layout é uma especificação do W3C projetada para proporcionar um método bi-dimensional para criação de layout CSS que oferece a possibilidade de "layoutar" itens da página com uso de linhas e colunas. CSS grid poderá ser usado para obter uma infinidade de diferentes layouts. O diferencial desse método de criação de layouts reside na possibilidade de se dividir uma página em grandes regiões e de se definir o relacionamento em termos de medidas, posicionamento e camadas entre os diferentes componentes da marcação HTML.

Tal como as tabelas, grid layout permite ao autor alinhar os componentes da página fazendo uso de linhas e colunas. Contudo ao contrário das tabelas, grid layout não se baseia em estruturação de conteúdos, possibilitando uma enorme variedade de layouts, o que não é possível de se obter com uso de tabelas. Por exemplo: elementos filhos de um grid container podem ser posicionados à vontade se sobrepondo ou mudando de layer, de modo idêntico como se faz com uso das funcionalidades do nosso já conhecido posicionamento CSS.

Exemplo básico

HTML

```
<div class="wrapper">
  <div class="box a">a</div>
  <div class="box b">b</div>
  <div class="box c">c</div>
  <div class="box d">d</div>
  <div class="box e">e</div>
  <div class="box f">f</div>
</div>
```

CSS

```
.wrapper {
  display: grid;
  grid-template-columns: 100px 100px 100px;
  grid-gap: 10px;
  background-color: #fff;
  color: #444;
}

.box {
  background-color: #444;
  color: #fff;
  border-radius: 5px;
  padding: 20px;
  font-size: 150%;
}
```

Referência

Propriedades CSS

- `grid-template-columns`
- `grid-template-rows`
- `grid-template-areas`
- `grid-template`
- `grid-auto-columns`
- `grid-auto-rows`
- `grid-auto-flow`
- `grid`
- `grid-row-start`
- `grid-column-start`
- `grid-row-end`
- `grid-column-end`
- `grid-row`
- `grid-column`
- `grid-area`
- `grid-row-gap`
- `grid-column-gap`
- `grid-gap`

Funções CSS

- `repeat()`
- `minmax()`

Glossário

- Grid Lines
- Grid Tracks
- Grid Cell
- Grid Area
- Gutters
- Grid row
- Grid column

Guias

- Conceitos básicos de Grid Layout
- Relação do Grid Layout com outros métodos de layout
- Layout usando grid lines nomeadas
- Grid template areas
- Posicionamento Automático CSS Grid Layout
- Alinhamento Box no CSS Grid Layout
- CSS Grid, Valores Lógicos e Modos de Escrita
- CSS Grid Layout e Acessibilidade

- CSS Grid e Aprimoramento Progressivo
- Realizando layouts comuns usando CSS Grid

Veja também

- Glossário:
  - Grid lines
  - Grid tracks
  - Grid cell
  - Grid areas
  - Gutters
  - Grid row
  - Grid column

Conceitos básicos de layout de grade

O layout de grade CSS introduz um sistema de grade bidimensional ao CSS. As grades podem ser usadas para dispor áreas maiores da página ou pequenos elementos da interface da pessoa usuária. Este artigo apresenta o layout de grade CSS e a nova terminologia que faz parte da especificação Layout de Grade CSS Nível 1. Os recursos mostrados nesta visão geral serão então explicados em maiores detalhes nas demais seções deste guia.

O que é uma grade?

Uma grade é um conjunto de linhas horizontais e verticais que se cruzam definindo colunas e linhas. Elementos podem ser colocados na grade dentro dessas linhas e colunas. O layout de grade CSS tem os seguintes recursos:

Tamanhos de trilha fixos e flexíveis

Você pode criar uma grade com tamanhos de trilha fixos – usando pixels, por exemplo. Isso define a grade com os pixels especificados que cabem no layout desejado. Você também pode criar uma grade com tamanhos flexíveis com porcentagens ou com a unidade `fr` criada para essa finalidade.

Posicionamento de itens

Você pode posicionar itens na grade com precisão usando números de linha, nomes ou selecionando uma área da grade. A grade também contém um algoritmo para controlar o posicionamento de itens que não possuem uma posição explícita na grade.

Criação de trilhas adicionais para armazenar conteúdo

Você pode definir uma grade explícita com um layout de grade. A especificação do layout de grade é flexível o suficiente para adicionar linhas e colunas adicionais quando necessário. Recursos como adicionar "quantas colunas couberem em um contêiner" estão incluídos.

Controle de alinhamento

A grade contém recursos de alinhamento para que possamos controlar como os itens se alinham uma vez colocados em uma área da grade e como toda a grade é alinhada.

Controle de conteúdo sobreposto

Mais de um item podem ser posicionados em uma célula ou área da grade e eles podem se sobrepor parcialmente. Essa disposição em camadas pode então ser controlada com a propriedade `z-index`.

A grade é uma especificação poderosa que, quando combinada com outras partes do CSS, como flexbox, pode ajudar você a criar layouts que antes eram impossíveis de construir com CSS. Tudo começa criando uma grade no seu **contêiner de grade**.

Contêiner de grade

Criamos um *contêiner de grade* declarando `display: grid` ou `display: inline-grid` em um elemento. Assim que fazemos isso, todos os *filhos diretos* desse elemento se tornam *itens de grade*.

Neste exemplo, temos um elemento div contentor com uma classe wrapper e, dentro, há cinco elementos filhos.

```
<div class="wrapper">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
</div>
```

Vamos tornar o `.wrapper` um contêiner de grade.

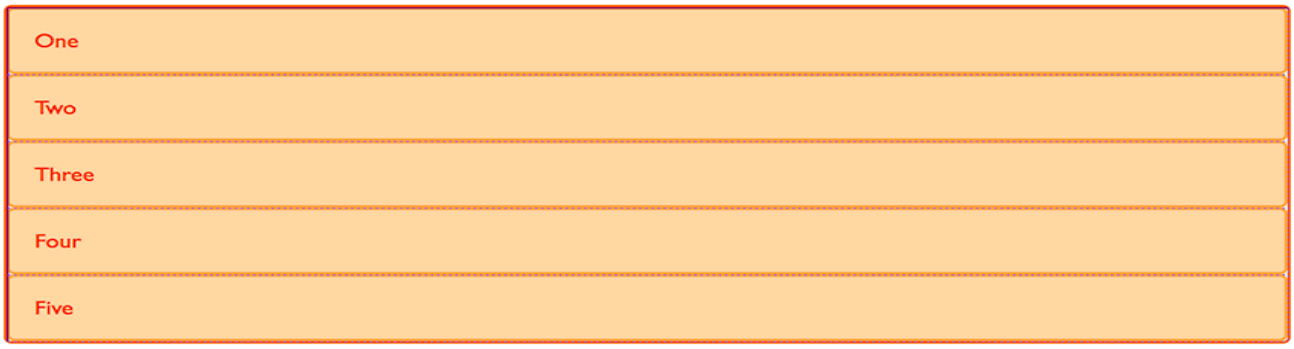
```
.wrapper {
  display: grid;
}
```

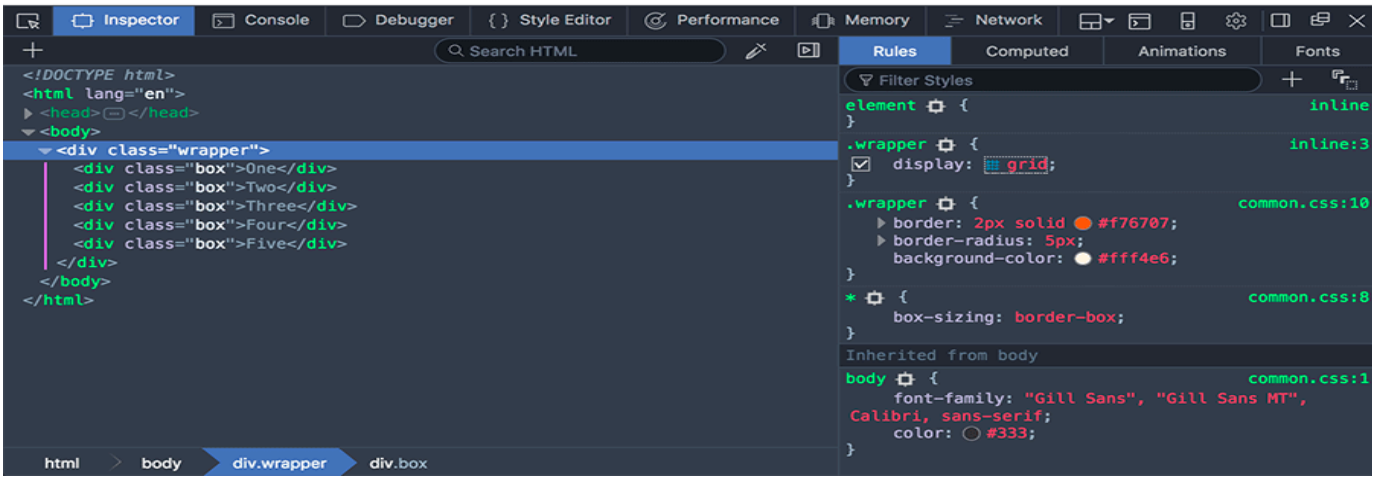
```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

Todos os elementos filhos diretos agora são itens de grade. Em um navegador web, você não verá nenhuma diferença em relação a como esses itens eram exibidos antes de transformá-los em uma grade, pois a grade criou uma grade de coluna única para os itens. Neste ponto, você pode achar útil trabalhar com o Inspetor de Grade, disponível como parte das Ferramentas da Pessoa Desenvolvedora do Firefox. Se você visualizar este exemplo no Firefox e inspecionar a grade, verá um pequeno ícone ao lado do valor `grid`. Clique nele e a grade neste elemento será sobreposta na janela do navegador.



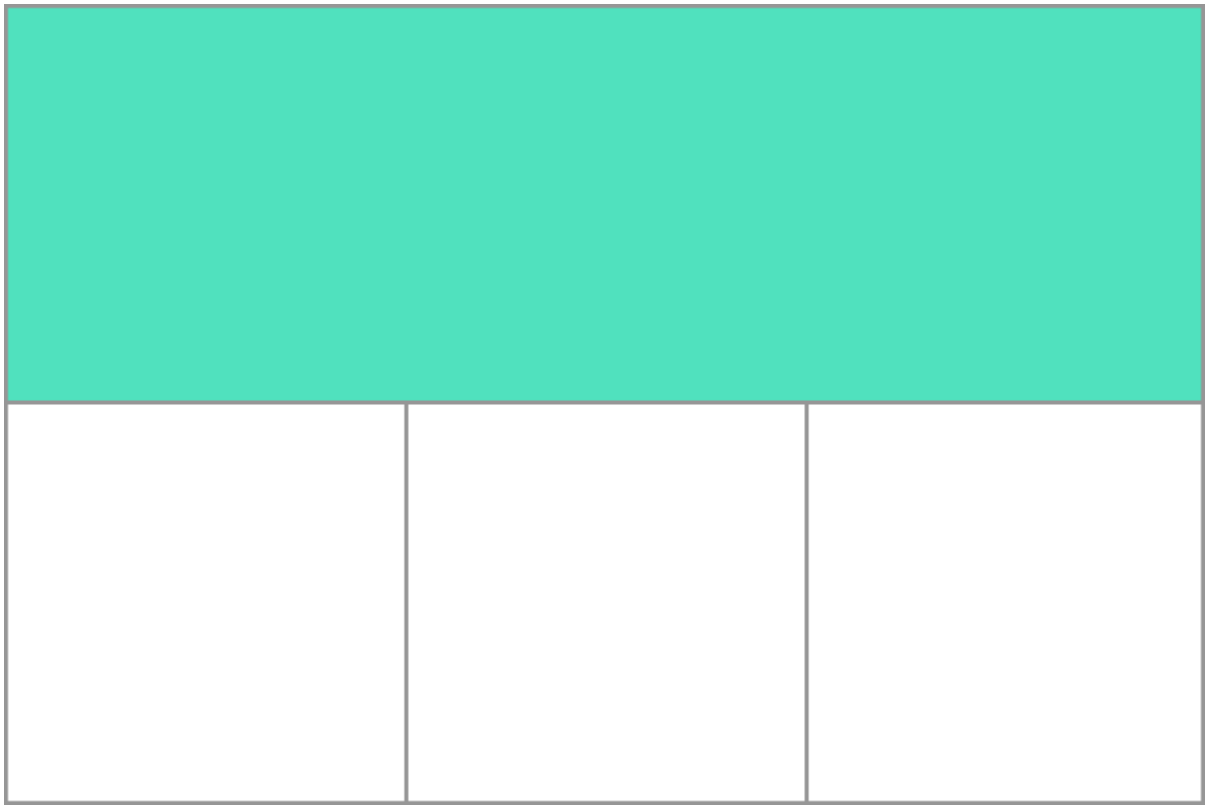


Conforme você aprende e então trabalha com o layout de grade CSS, esta ferramenta lhe dará uma ideia melhor do que está acontecendo com suas grades visualmente.

Se quisermos começar a tornar isso mais parecido com uma grade, precisamos adicionar trilhas de coluna.

Trilhas de grade

Definimos linhas e colunas em nossa grade com as propriedades `grid-template-rows` e `grid-template-columns`. Elas definem as trilhas de grade. Uma *trilha de grade* é o espaço entre quaisquer duas linhas adjacentes na grade. A imagem abaixo mostra uma trilha destacada – esta é a trilha da primeira linha em nossa grade.



As trilhas de grade são definidas na grade explícita usando as propriedades `grid-template-columns` e `grid-template-rows` ou as propriedades abreviadas `grid` ou `grid-template`. As trilhas também são criadas na grade implícita posicionando um item de grade fora das trilhas criadas na grade explícita.

Exemplo básico

Podemos incrementar o nosso exemplo anterior adicionando a propriedade `grid-template-columns` e, em seguida, definindo o tamanho das trilhas de coluna.

Agora criamos uma grade com três trilhas de coluna de 200 pixels de largura. Os itens filhos serão dispostos nesta grade, um em cada célula da grade.

```
<div class="wrapper">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: 200px 200px 200px;
}
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}
```

```
.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

A unidade <code>fr</code>

As trilhas podem ser definidas usando qualquer unidade de comprimento. A grade também introduz uma unidade de comprimento adicional para nos ajudar a criar trilhas de grade flexíveis. A nova unidade `fr` representa uma fração do espaço disponível no contêiner da grade. A definição de grade a seguir cria três trilhas de largura igual que aumentam e diminuem de acordo com o espaço disponível.

```
<div class="wrapper">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
}
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

Tamanhos desiguais

No próximo exemplo, criamos uma definição com uma trilha de `2fr` e depois duas trilhas de `1fr`. O espaço disponível é dividido em quatro. Duas partes são dadas à primeira trilha e uma parte para cada uma das duas trilhas seguintes.

```
<div class="wrapper">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: 2fr 1fr 1fr;
}
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
```

```
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

Misturando tamanhos flexíveis e absolutos

Neste último exemplo, misturamos trilhas de tamanho absoluto com unidades `fr`. A primeira trilha tem 500 pixels, então a largura fixa é retirada do espaço disponível. O espaço restante é dividido em três e atribuído em proporção às duas trilhas flexíveis.

```
<div class="wrapper">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: 500px 1fr 2fr;
}
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

Listagens de trilhas com a notação repeat()

Grades grandes com muitas trilhas podem usar a notação `repeat()` para repetir toda ou uma seção da listagem de trilhas. Por exemplo, a definição da grade:

```
.wrapper {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
}
```

Também pode ser escrita como:

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
}
```

A notação de repetição pode ser usada para uma parte da listagem de trilhas. No próximo exemplo, criamos uma grade com uma trilha inicial de 20 pixels, depois uma seção de repetição de 6 trilhas de `1fr`, e por fim uma trilha final de 20 pixels.

```
.wrapper {
  display: grid;
  grid-template-columns: 20px repeat(6, 1fr) 20px;
}
```

A notação de repetição recebe a listagem de trilhas e a usa para criar um padrão de repetição de trilhas. No próximo exemplo, nossa grade consistirá de 10 trilhas, uma trilha de `1fr` e, em seguida, uma trilha de `2fr`. Este padrão será repetido cinco vezes.

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(5, 1fr 2fr);
}
```

Grades implícitas e explícitas

Ao criar nossa grade de exemplo, definimos especificamente nossas trilhas de coluna com a propriedade `grid-template-columns`, mas a grade também criou linhas por conta própria. Essas linhas são parte da grade implícita. Enquanto a grade explícita consiste em quaisquer linhas e colunas definidas com `grid-template-columns` ou `grid-template-rows`.

Se você colocar algo fora da grade definida — ou devido à quantidade de conteúdo, mais trilhas de grade forem necessárias — então a grade cria linhas e colunas na grade implícita. Essas trilhas serão dimensionadas automaticamente por padrão, fazendo com que seu tamanho seja baseado no conteúdo que está dentro delas.

Você também pode definir um tamanho padrão para trilhas criadas na grade implícita com as propriedades `grid-auto-rows` e `grid-auto-columns`.

No exemplo abaixo, usamos `grid-auto-rows` para garantir que as trilhas criadas na grade implícita tenham 200 pixels de altura.

```
<div class="wrapper">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: 200px;
}
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

Dimensionamento de trilha e minmax

Ao configurar uma grade explícita ou definir o dimensionamento para linhas ou colunas criadas automaticamente, podemos querer dar às trilhas um tamanho mínimo, mas também garantir que elas se expandam para o tamanho necessário do conteúdo adicionado. Por exemplo, podemos querer que nossas linhas nunca colapsem para menos de 100 pixels, mas se o nosso conteúdo se estender até 300 pixels de altura, então gostaríamos que a linha se estendesse para essa altura.

A grade tem uma solução para isso com a função `minmax()`. No próximo exemplo, estamos usando `minmax()` no valor de `grid-auto-rows`. Isso significa que as linhas criadas automaticamente terão no mínimo 100 pixels de altura e no máximo `auto`. Usar `auto` significa que o tamanho observará o

tamanho do conteúdo e se estenderá para dar espaço para o item mais alto em uma célula, nesta linha.

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: minmax(100px, auto);
}
```

```
* {
  box-sizing: border-box;
}

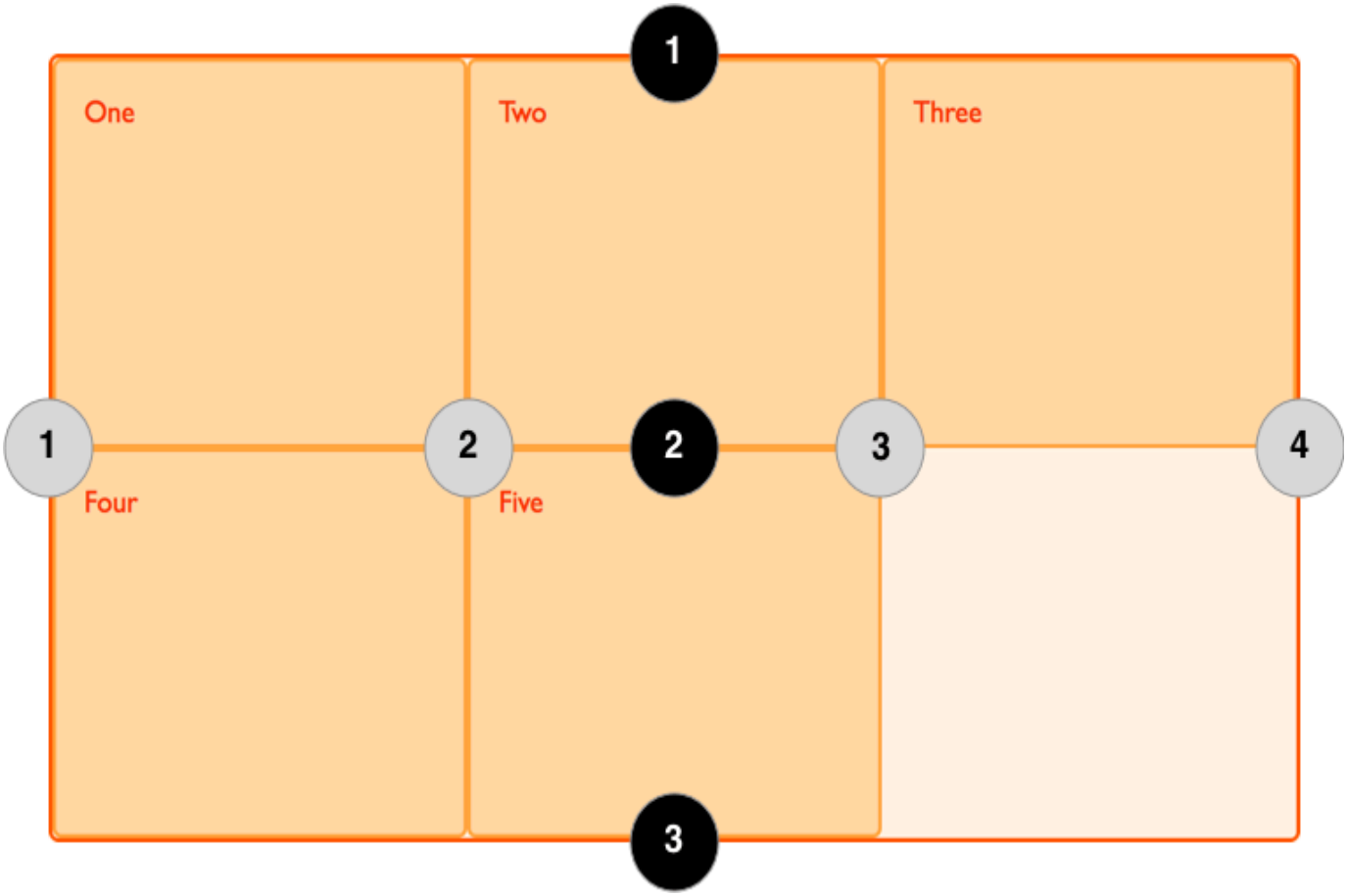
.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div>One</div>
  <div>
    Two
    <p>I have some more content in.</p>
    <p>This makes me taller than 100 pixels.</p>
  </div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
</div>
```

Linhas de grade

Devemos notar que quando definimos uma grade, definimos as trilhas da grade, não as linhas. A grade então nos dá linhas numeradas para usar ao posicionar itens. Em nossa grade de três colunas e duas linhas, temos quatro linhas de coluna.



As linhas são numeradas de acordo com a forma de escrita do documento. Em um idioma da esquerda para a direita, a linha 1 fica no lado esquerdo da grade. Em um idioma da direita para a esquerda, fica no lado direito da grade. As linhas também podem ser nomeadas, e veremos como fazer isso em um guia posterior desta série.

Posicionando itens em relação às linhas

Exploraremos o posicionamento baseado em linhas em detalhes em um artigo posterior. O exemplo a seguir demonstra como fazer isso de uma forma simples. Ao posicionar um item, focamos a linha — em vez da trilha.

No exemplo a seguir, estamos posicionando os dois primeiros itens em nossa grade de trilha de três colunas, usando as propriedades `grid-column-start`, `grid-column-end`, `grid-row-start` e `grid-row-end`. Trabalhando da esquerda para a direita, o primeiro item é posicionado em relação à linha da coluna 1 e se estende até a linha da coluna 4, que no nosso caso é a linha mais à direita na grade. Ele começa na linha da linha 1 e termina na linha da linha 3, portanto, abrangendo duas trilhas de linha.

O segundo item começa na linha da coluna 1 da grade e abrange uma trilha. Este é o padrão, então não precisamos especificar a linha final. Ele também abrange duas trilhas de linha da linha 3 até a linha 5. Os outros itens serão colocados em espaços vazios na grade.

```
<div class="wrapper">
  <div class="box1">One</div>
  <div class="box2">Two</div>
  <div class="box3">Three</div>
  <div class="box4">Four</div>
  <div class="box5">Five</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: 100px;
}

.box1 {
  grid-column-start: 1;
  grid-column-end: 4;
  grid-row-start: 1;
  grid-row-end: 3;
}
```

```
.box2 {
  grid-column-start: 1;
  grid-row-start: 3;
  grid-row-end: 5;
}
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

> **Nota:** Não se esqueça que você pode usar o Inspetor de Grade nas Ferramentas da Pessoa Desenvolvedora do Firefox para ver como os itens são posicionados em relação às linhas da grade.

Atalhos de posicionamento de linha

Os valores longos usados acima podem ser compactados em uma linha para colunas com `grid-column` e uma linha para linhas com `grid-row`. O exemplo a seguir resultaria no mesmo posicionamento do código anterior, mas com muito menos CSS. O valor antes do caractere de barra (`/`) é a linha inicial, o valor após é a linha final.

Você pode omitir o valor final se a área abranger apenas uma trilha.

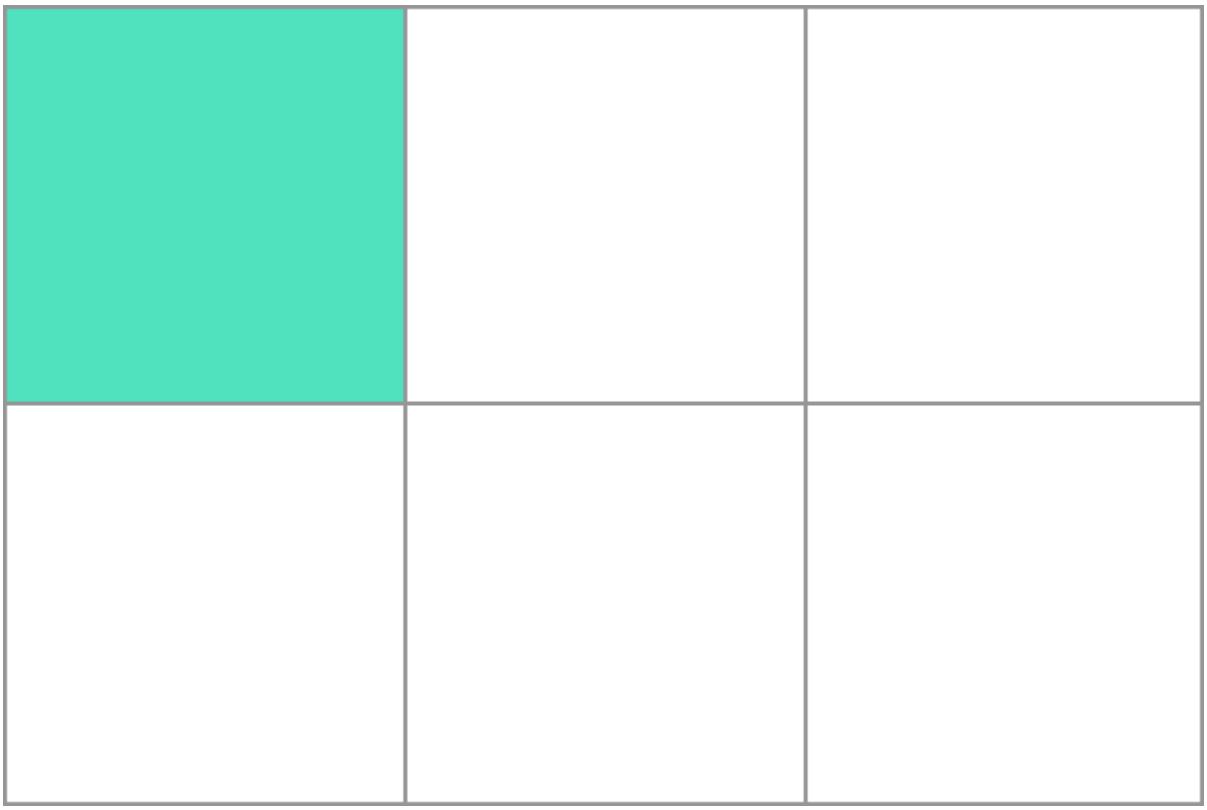
```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: 100px;
}

.box1 {
  grid-column: 1 / 4;
  grid-row: 1 / 3;
}

.box2 {
  grid-column: 1;
  grid-row: 3 / 5;
}
```

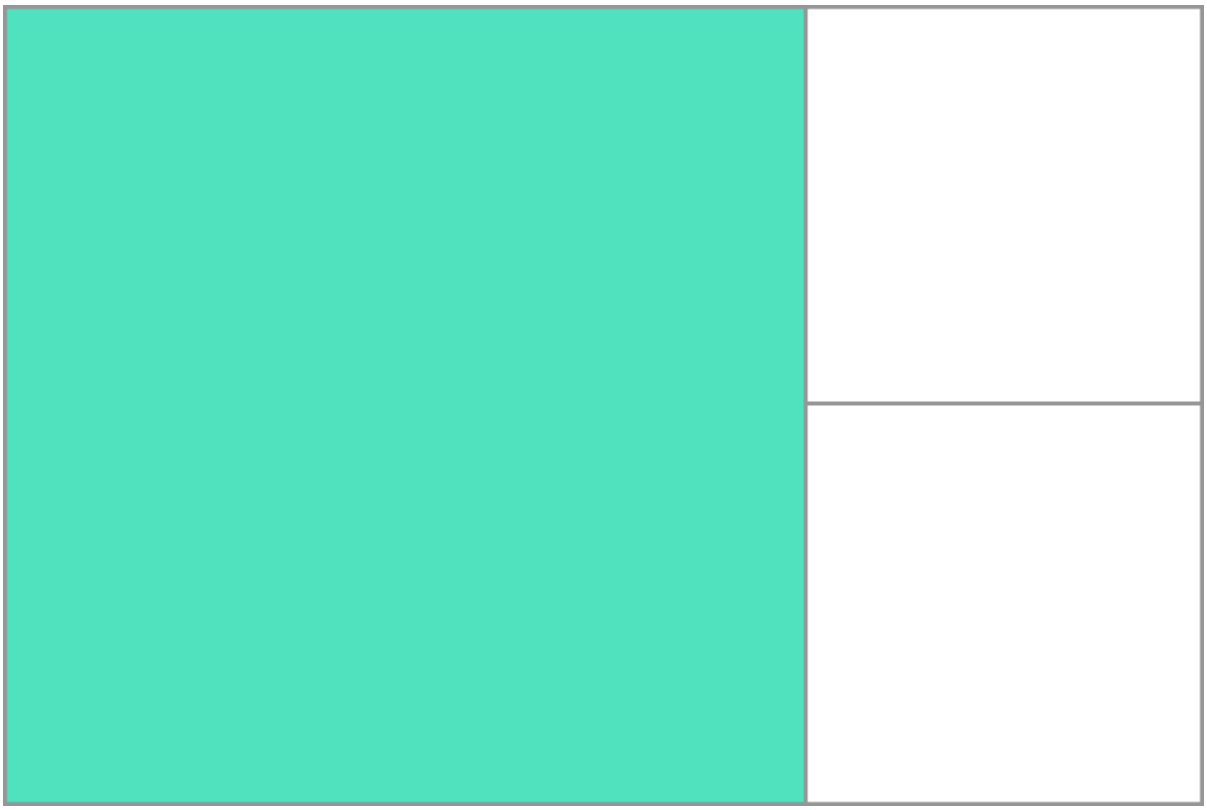
Células de grade

Uma *célula de grade* é a menor unidade em uma grade. Conceitualmente, é como uma célula de tabela. Como vimos em nossos exemplos anteriores, uma vez que uma grade é definida como pai, os itens filhos serão organizados cada um em uma célula da grade definida. Na imagem abaixo, temos destacada a primeira célula da grade.



Áreas de grade

Os itens podem abranger uma ou mais células, tanto por linha quanto por coluna, e isso cria uma *área de grade*. As áreas de grade devem ser retangulares – não é possível criar uma área em forma de L, por exemplo. A área de grade destacada abrange duas trilhas de linha e duas de coluna.



Calhas

Calhas ou *becos* entre células de grade podem ser criados usando as propriedades `column-gap` e `row-gap`, ou a forma resumida `gap`. No exemplo abaixo, estamos criando uma lacuna de 10 pixels entre colunas e uma lacuna de `1em` entre linhas.

```
.wrapper {
  display: grid;
```

```
  grid-template-columns: repeat(3, 1fr);
  column-gap: 10px;
  row-gap: 1em;
}
```

> **Nota:** Quando a grade foi lançada em navegadores, `column-gap`, `row-gap` e `gap` eram prefixados com o prefixo `grid-` como `grid-column-gap`, `grid-row-gap` e `grid-gap`, respectivamente.
>
> Todos os navegadores agora suportam valores não prefixados, no entanto, as versões prefixadas serão mantidas como apelidos, tornando-as seguras para uso.

```
<div class="wrapper">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
</div>
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  column-gap: 10px;
  row-gap: 1em;
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

Qualquer espaço usado por lacunas será contabilizado antes que o espaço seja atribuído às trilhas de comprimento flexível `fr`, e as lacunas agem para fins de dimensionamento como uma trilha de grade normal, no entanto, você não pode colocar nada em uma lacuna. Em termos de posicionamento baseado em linha, a lacuna age como uma linha grossa.

Grades aninhadas

Um item de grade pode se tornar um contêiner de grade. No exemplo a seguir, temos a grade de três colunas que criamos anteriormente, com nossos dois itens posicionados. Neste caso, o primeiro item tem alguns subitens. Como esses itens não são filhos diretos da grade, eles não participam do layout da grade e, portanto, são exibidos em um fluxo normal de documento.



Aninhamento sem subgrade

Se definirmos `box1` como `display: grid`, poderemos dar a ela uma definição de trilha e ela também se tornará uma grade. Os itens então são dispostos nessa nova grade.

```
.box1 {
  grid-column-start: 1;
  grid-column-end: 4;
  grid-row-start: 1;
  grid-row-end: 3;
  display: grid;
  grid-template-columns: repeat(3, 1fr);
}
```

```
<div class="wrapper">
  <div class="box box1">
    <div class="nested">a</div>
    <div class="nested">b</div>
    <div class="nested">c</div>
  </div>
  <div class="box box2">Two</div>
  <div class="box box3">Three</div>
  <div class="box box4">Four</div>
  <div class="box box5">Five</div>
</div>
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  gap: 3px;
  background-color: #fff4e6;
  display: grid;
  grid-template-columns: repeat(3, 1fr);
}
```

```
.box {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}

.box1 {
  grid-column: 1 / 4;
}

.nested {
  border: 2px solid #ffec99;
  border-radius: 5px;
  background-color: #fff9db;
  padding: 1em;
}
```

Neste caso, a grade aninhada não tem relação com o pai. Como você pode ver no exemplo, ela não herdou o `gap` do pai e as linhas na grade aninhada não estão alinhadas às linhas na grade pai.

Subgrade

Além das grades comuns, a *subgrade* nos permite criar grades aninhadas que usam a definição de trilha da grade pai.

Para usá-las, editamos o exemplo de grade aninhada acima para alterar a definição de trilha de `grid-template-columns: repeat(3, 1fr)`, para `grid-template-columns: subgrid`. A grade aninhada então usa as trilhas da grade pai para dispor os itens.

```
.box1 {
  grid-column-start: 1;
  grid-column-end: 4;
  grid-row-start: 1;
  grid-row-end: 3;
  display: grid;
  grid-template-columns: subgrid;
}
```

Sobreposição de itens com z-index

Itens de grade podem ocupar a mesma célula e, neste caso, podemos usar a propriedade `z-index` para controlar a ordem em que os itens sobrepostos são empilhados.

Sobreposição sem z-index

Se retornarmos ao nosso exemplo com itens posicionados por número de linha, podemos alterá-lo para fazer dois itens se sobreporem.

```
<div class="wrapper">
  <div class="box box1">One</div>
  <div class="box box2">Two</div>
  <div class="box box3">Three</div>
  <div class="box box4">Four</div>
  <div class="box box5">Five</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: 100px;
}

.box1 {
  grid-column-start: 1;
  grid-column-end: 4;
  grid-row-start: 1;
  grid-row-end: 3;
}

.box2 {
  grid-column-start: 1;
```

```
  grid-row-start: 2;
  grid-row-end: 4;
}
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.box {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

O item `box2` agora está sobrepondo `box1`, ele é exibido no topo, pois vem depois na ordem do código.

Controlando a ordem

Podemos controlar a ordem em que os itens são empilhados usando a propriedade `z-index` - assim como itens posicionados. Se dermos a `box2` um `z-index` menor que `box1`, ele será exibido abaixo de `box1` na pilha.

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: 100px;
}

.box1 {
  grid-column-start: 1;
  grid-column-end: 4;
  grid-row-start: 1;
  grid-row-end: 3;
  z-index: 2;
}

.box2 {
  grid-column-start: 1;
  grid-row-start: 2;
  grid-row-end: 4;
  z-index: 1;
}
```

```
<div class="wrapper">
  <div class="box box1">One</div>
  <div class="box box2">Two</div>
  <div class="box box3">Three</div>
  <div class="box box4">Four</div>
  <div class="box box5">Five</div>
</div>
```

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.box {
  border: 2px solid #ffa94d;
  border-radius: 5px;
```

```
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

Próximos passos

Neste artigo, demos uma olhada rápida nas possibilidades dos layouts de grade. Explore e brinque com os exemplos de código e, em seguida, passe para a próxima parte deste guia, onde realmente começaremos a nos aprofundar nos detalhes do layout de grade CSS.

Relationship of grid layout to other layout methods

O CSS Grid Layout foi projetado para funcionar junto com outras partes do CSS, como parte de um sistema completo para fazer o layout. Neste guia, explicarei como uma grade se encaixa com outras técnicas que você já pode estar usando.

Grid e flexbox

A diferença básica entre CSS Grid Layout e CSS Flexbox Layout é que o flexbox foi projetado para layout em uma dimensão - uma linha ou uma coluna. A grade foi projetada para layout bidimensional - linhas e colunas ao mesmo tempo. As duas especificações compartilham alguns recursos comuns, e se você já aprendeu a usar o flexbox, as semelhanças devem ajudá-lo a se familiarizar com o Grid.

Layout unidimensional versus bidimensional

Um exemplo simples pode demonstrar a diferença entre layouts unidimensionais e bidimensionais.

Neste primeiro exemplo, estou usando o flexbox para criar um conjunto de caixas. Eu tenho cinco itens filhos no meu contêiner e dei valores de propriedades flexíveis para que eles possam crescer e encolher de uma base flexível de 200 pixels.

Eu também defini a propriedade `flex-wrap` para agrupar, de modo que, se o espaço no contêiner ficar muito estreito para manter a base flexível, os itens serão agrupados em uma nova linha.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
</div>
```

```
.wrapper {
  display: flex;
  flex-wrap: wrap;
}
.wrapper > div {
  flex: 1 1 200px;
}
```

Na imagem, você pode ver que dois itens foram envolvidos em uma nova linha. Esses itens estão compartilhando o espaço disponível e não se alinhando abaixo dos itens acima. Isso ocorre porque quando você envolve itens flexíveis, cada nova linha (ou coluna ao trabalhar por coluna) se torna um novo contêiner flexível. Distribuição de espaço acontece em toda a linha.

Uma questão comum, então, é como alinhar esses itens. É aqui que você deseja um método de layout bidimensional: Você quer controlar o alinhamento por linha e coluna, e é aí que a grade entra.

The same layout with CSS grids

Neste próximo exemplo, eu crio o mesmo layout usando Grid. Desta vez, temos três faixas de coluna 1fr. Não precisamos definir nada nos itens em si; eles se estabelecerão em cada célula da grade criada. Como você pode ver, eles ficam em uma grade rígida, alinhando-se em linhas e colunas. Com cinco itens, temos uma lacuna no final da segunda linha.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
  <div>Four</div>
  <div>Five</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
}
```

A simple question to ask yourself when deciding between grid or flexbox is:

- do I only need to control the layout by row ***or*** column – use a flexbox
- do I need to control the layout by row ***and*** column – use a grid

Content out or layout in?

In addition to the one-dimensional versus two-dimensional distinction, there is another way to decide if you should use flexbox or grid for a layout. Flexbox works from the content out. An ideal use case for flexbox is when you have a set of items and want to space them out evenly in a container. You let the size of the content decide how much individual space each item takes up. If the items wrap onto a new line, they will work out their spacing based on their size and the available space *on that line*.

Grid works from the layout in. When you use CSS Grid Layout you create a layout and then you place items into it, or you allow the auto-placement rules to place the items into the grid cells according to that strict grid. It is possible to create tracks that respond to the size of the content, however, they will also change the entire track.

If you are using flexbox and find yourself disabling some of the flexibility, you probably need to use CSS Grid Layout. An example would be if you are setting a percentage width on a flex item to make it line up with other items in a row above. In that case, a grid is likely to be a better choice.

Box alignment

The feature of flexbox that was most exciting to many of us was that it gave us proper alignment control for the first time. It made it easy to center a box on the page. Flex items can stretch to the height of the flex container, meaning that equal height columns were possible. These were things we have wanted to do for a very long time, and have come up with all kinds of hacks to at least create the visual effect of.

The alignment properties from the flexbox specification have been added to a new specification called Box Alignment Level 3. This means that they can be used in other specifications, including Grid Layout. In the future, they may well apply to other layout methods as well.

In a later guide in this series, I'll be taking a proper look at Box Alignment and how it works in Grid Layout. For now, here is a comparison between simple examples of flexbox and grid.

In the first example, which uses flexbox, I have a container with three items inside. The wrapper `min-height` is set, so it defines the height of the flex container. I have set `align-items` on the flex container to `flex-end` so the items will line up at the end of the flex container. I have also set the `align-self` property on `box1` so it will override the default and stretch to the height of the container and on `box2` so it aligns to the start of the flex container.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div class="box1">One</div>
  <div class="box2">Two</div>
  <div class="box3">Three</div>
</div>
```

```
.wrapper {
  display: flex;
  align-items: flex-end;
  min-height: 200px;
}
.box1 {
  align-self: stretch;
}
.box2 {
  align-self: flex-start;
}
```

Alignment in CSS Grids

This second example uses a grid to create the same layout. This time we are using the box alignment properties as they apply to a grid layout. So we align to `start` and `end` rather than `flex-start` and `flex-end`. In the case of a grid layout, we are aligning the items inside their grid area. In this case that is a single grid cell, but it could be an area made up of several grid cells.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div class="box1">One</div>
  <div class="box2">Two</div>
  <div class="box3">Three</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  align-items: end;
  grid-auto-rows: 200px;
}
.box1 {
  align-self: stretch;
}
.box2 {
  align-self: start;
}
```

The <code>fr</code> unit and <code>flex-basis</code>

We have already seen how the `fr` unit works to assign a proportion of available space in the grid container to our grid tracks. The `fr` unit, when combined with the `minmax()` function can give us very similar behavior to the `flex` properties in flexbox while still enabling the creation of a layout in two dimensions.

If we look back at the example where I demonstrated the difference between one and two-dimensional layouts, you can see there is a difference between the way of the two layouts work responsively. With the flex layout, if we drag our window wider and smaller, the flexbox does a nice job of adjusting the number of items in each row according to the available space. If we have a lot of space all five items can fit on one row. If we have a very narrow container we may only have space for one.

In comparison, the grid version always has three column tracks. The tracks themselves will grow and shrink, but there are always three since we asked for three when defining our grid.

Auto-filling grid tracks

We can create a similar effect to flexbox, while still keeping the content arranged in strict rows and columns, by creating our track listing using repeat notation and the `auto-fill` and `auto-fit` properties.

In this next example, I have used the `auto-fill` keyword in place of an integer in the repeat notation and set the track listing to 200 pixels. This means that grid will create as many 200 pixels column tracks as will fit in the container.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(auto-fill, 200px);
}
```

A flexible number of tracks

This isn't quite the same as flexbox. In the flexbox example, the items are larger than the 200 pixel basis before wrapping. We can achieve the same in grid by combining `auto-fill` and the `minmax()` function. In this next example, I create auto filled tracks with `minmax`. I want my tracks to be a minimum

of 200 pixels, so I set the maximum to be `1fr`. Once the browser has worked out how many times 200 pixels will fit into the container–also taking account of grid gaps–it will treat the `1fr` maximum as an instruction to share out the remaining space between the items.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
}
```

We now have the ability to create a grid with a flexible number of flexible tracks, but see items laid out on the grid aligned by rows and columns at the same time.

Grid and absolutely positioned elements

Grid interacts with absolutely positioned elements, which can be useful if you want to position an item inside a grid or grid area. The specification defines the behavior when a grid container is a containing block and a parent of the absolutely positioned item.

A grid container as containing block

To make the grid container a containing block you need to add the position property to the container with a value of relative, just as you would make a containing block for any other absolutely positioned items. Once you have done this, if you give a grid item `position: absolute` it will take as its containing block the grid container or, if the item also has a grid position, the area of the grid it is placed into.

In the below example I have a wrapper containing four child items. Item three is absolutely positioned and also placed on the grid using line-based placement. The grid container has `position: relative` and so becomes the positioning context of this item.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div class="box1">One</div>
  <div class="box2">Two</div>
```

```
  <div class="box3">
    This block is absolutely positioned. In this example the grid container is
    the containing block and so the absolute positioning offset values are
    calculated in from the outer edges of the area it has been placed into.
  </div>
  <div class="box4">Four</div>
</div>
```

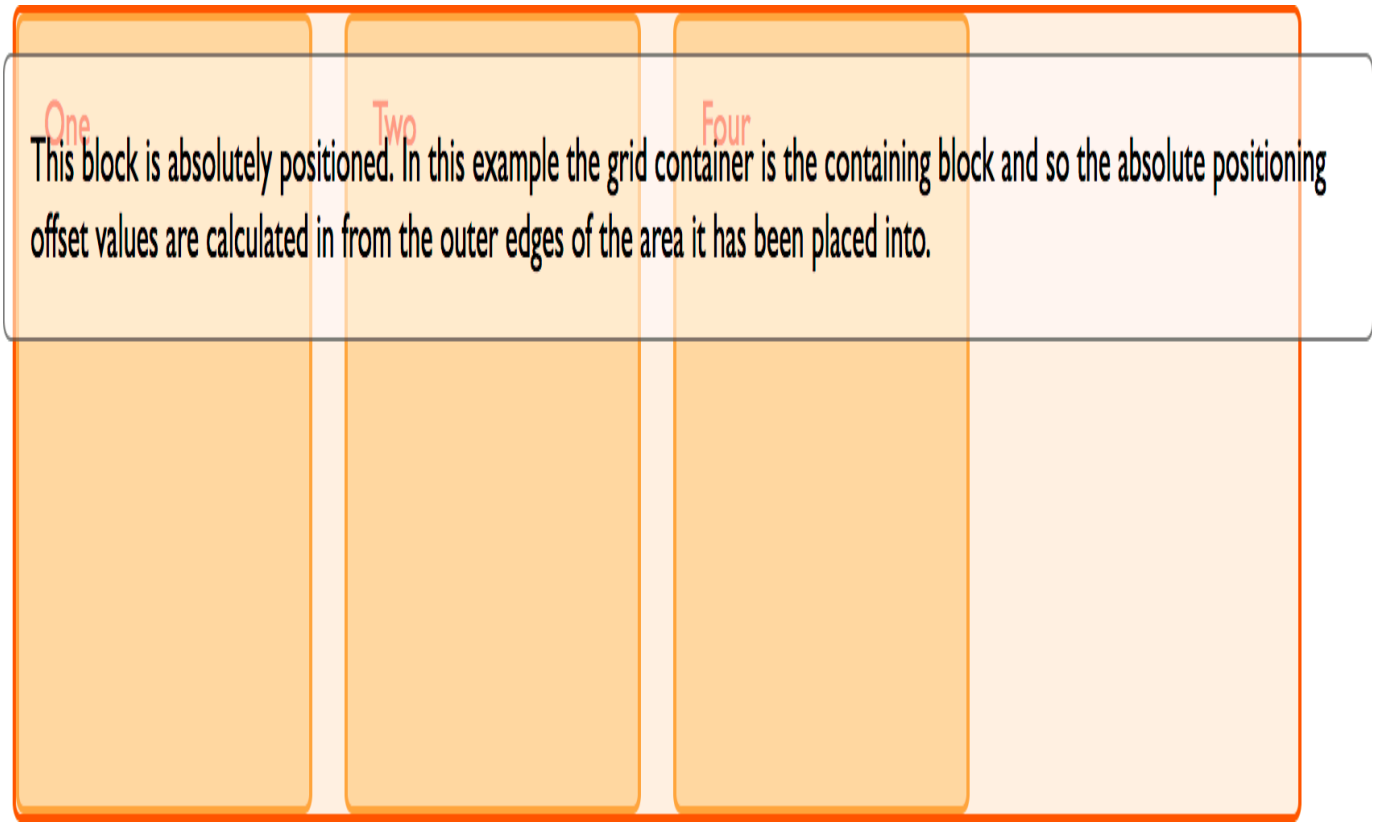
```
.wrapper {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  grid-auto-rows: 200px;
  grid-gap: 20px;
  position: relative;
}
.box3 {
  grid-column-start: 2;
  grid-column-end: 4;
  grid-row-start: 1;
  grid-row-end: 3;
  position: absolute;
  top: 40px;
  left: 40px;
}
```

You can see that the item is taking the area from grid row line 2 to 4, and starting after line 1. Then it is offset in that area using the top and left properties. However, it has been taken out of flow as is usually for absolutely positioned items and so the auto-placement rules now place items into the same space. The item also doesn't cause the additional row to be created to span to row line 3.

If we remove `position: absolute` from the rules for `.box3` you can see how it would display without the positioning.

A grid container as parent

If the absolutely positioned child has a grid container as a parent but that container does not create a new positioning context, then it is taken out of flow as in the previous example. The positioning context will be whatever element creates a positioning context as is common to other layout methods. In our case, if we remove `position: relative` from the wrapper above, positioning context is from the viewport, as shown in this image.



Once again the item no longer participates in the grid layout in terms of sizing or when other items are auto-placed.

With a grid area as the parent

If the absolutely positioned item is nested inside a grid area then you can create a positioning context on that area. In the below example we have our grid as before but this time I have nested an item inside `.box3` of the grid.

I have given `.box3` position relative and then positioned the sub-item with the offset properties. In this case, the positioning context is the grid area.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.wrapper > div {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
```

```
<div class="wrapper">
  <div class="box1">One</div>
  <div class="box2">Two</div>
  <div class="box3">
    Three
    <div class="abspos">
      This block is absolutely positioned. In this example the grid area is the
      containing block and so the absolute positioning offset values are
      calculated in from the outer edges of the grid area.
    </div>
  </div>
  <div class="box4">Four</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  grid-auto-rows: 200px;
  grid-gap: 20px;
}
.box3 {
  grid-column-start: 2;
  grid-column-end: 4;
  grid-row-start: 1;
  grid-row-end: 3;
  position: relative;
}
.abspos {
  position: absolute;
  top: 40px;
  left: 40px;
  background-color: rgba(255, 255, 255, 0.5);
  border: 1px solid rgba(0, 0, 0, 0.5);
  color: #000;
  padding: 10px;
}
```

Grid and <code>display: contents</code>

A final interaction with another layout specification that is worth noting is the interaction between CSS Grid Layout and `display: contents`. The `contents` value of the display property is a new value that is described in the Display specification as follows:

> "The element itself does not generate any boxes, but its children and pseudo-elements still generate boxes as normal. For the purposes of box generation and layout, the element must be treated as if it had been replaced with its children and pseudo-elements in the document tree."

If you set an item to `display: contents` the box it would normally create disappears, and the boxes of the child elements appear as if they have risen up a level. This means that children of a grid item can become grid items. Sound odd? Here is a simple example. In the following markup, I have a grid and the first item on the grid is set to span all three column tracks. It contains three nested items. As these items are not direct children, they don't become part of the grid layout and so display using regular block layout.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.box {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
}
.nested {
  border: 2px solid #ffec99;
  border-radius: 5px;
  background-color: #fff9db;
  padding: 1em;
}
```

```
<div class="wrapper">
  <div class="box box1">
    <div class="nested">a</div>
    <div class="nested">b</div>
    <div class="nested">c</div>
  </div>
  <div class="box box2">Two</div>
  <div class="box box3">Three</div>
  <div class="box box4">Four</div>
  <div class="box box5">Five</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: minmax(100px, auto);
}
.box1 {
  grid-column-start: 1;
  grid-column-end: 4;
}
```

If I now add `display: contents` to the rules for `box1`, the box for that item vanishes and the sub-items now become grid items and lay themselves out using the auto-placement rules.

```
* {
  box-sizing: border-box;
}

.wrapper {
  border: 2px solid #f76707;
  border-radius: 5px;
  background-color: #fff4e6;
}

.box {
  border: 2px solid #ffa94d;
  border-radius: 5px;
  background-color: #ffd8a8;
  padding: 1em;
  color: #d9480f;
```

```
}
.nested {
  border: 2px solid #ffec99;
  border-radius: 5px;
  background-color: #fff9db;
  padding: 1em;
}
```

```
<div class="wrapper">
  <div class="box box1">
    <div class="nested">a</div>
    <div class="nested">b</div>
    <div class="nested">c</div>
  </div>
  <div class="box box2">Two</div>
  <div class="box box3">Three</div>
  <div class="box box4">Four</div>
  <div class="box box5">Five</div>
</div>
```

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: minmax(100px, auto);
}
.box1 {
  grid-column-start: 1;
  grid-column-end: 4;
  display: contents;
}
```

This can be a way to get items nested into the grid to act as if they are part of the grid, and is a way around some of the issues that would be solved by subgrids once they are implemented. You can also use `display: contents` in a similar way with flexbox to enable nested items to become flex items.

As you can see from this guide, CSS Grid Layout is just one part of your toolkit. Don't be afraid to mix it with other methods of doing layout to get the different effects you need.

See Also

- Flexbox Guides
- Multiple-column Layout Guides

1. **CSS**
2. **CSS Reference**
3. CSS Grid Layout
4. **Guides**
    1. Basics concepts of grid layout
    2. Relationship to other layout methods
    3. Line-based placement
    4. Grid template areas
    5. Layout using named grid lines
    6. Auto-placement in grid layout
    7. Box alignment in grid layout
    8. Grids, logical values, and writing modes
    9. CSS Grid Layout and Accessibility
    10. CSS Grid Layout and Progressive Enhancement
    11. Realizing common layouts using grids
5. **Properties**
    1. grid
    2. grid-area
    3. grid-auto-columns
    4. grid-auto-flow

5. grid-auto-rows
6. grid-column
7. grid-column-end
8. grid-column-gap
9. grid-column-start
10. grid-gap
11. grid-row
12. grid-row-end
13. grid-row-gap
14. grid-row-start
15. grid-template
16. grid-template-areas
17. grid-template-columns
18. grid-template-rows

6. **Glossary**

    1. Grid
    2. Grid lines
    3. Grid tracks
    4. Grid cell
    5. Grid areas
    6. Gutters
    7. Grid Axis
    8. Grid row
    9. Grid column

CSS Images

CSS Images is a module of CSS that defines what types of images can be used (the `<image>` type, containing URLs, gradients and other types of images), how to resize them and how they, and other replaced content, interact with the different layout models.

Reference

Properties

- `image-orientation`
- `image-rendering`
- `object-fit`
- `object-position`

Functions

- `linear-gradient()`
- `radial-gradient()`
- `repeating-linear-gradient()`
- `repeating-radial-gradient()`
- `element()`

Data types

- `<gradient>`
- `<image>`

Guides

Using CSS gradients

> Presents a specific type of CSS images, *gradients*, and how to create and use these.

Implementing image sprites in CSS

> Describes the common technique grouping several images in one single document to save download requests and speed up the availability of a page.

Implementando sprites de imagens em CSS

Sprites de imagens são utilizados em diversas aplicações web onde várias imagens são usadas. Ao invés de incluir cada arquivo de imagem separadamente, é mais amigável com a memória e largura de banda enviar tudo como uma única imagem, diminuindo o número de pedidos em HTTP.

> **Nota:** Quando usando HTTP/2, é mais amigável com a largura de banda usar vários pequenos pedidos.

Implementação

Suponha que uma imagem é dada a cada item com a classe `toolbtn`:

```
.toolbtn {
  background: url(myfile.png);
  display: inline-block;
  height: 20px;
  width: 20px;
}
```

A posição do plano de fundo pode ser adicionada tanto com dois valores x, y depois de `url()` em background , ou como `background-position` . Por exemplo:

```
#btn1 {
  background-position: -20px 0px;
}

#btn2 {
  background-position: -40px 0px;
}
```

Isso vai mover o elemento com a ID 'btn1' 20 pixels para a esquerda e o elemento com a ID 'btn2' 40 pixels para a esquerda (presumindo que ambos tem a classe `toolbtn` atribuída e estão sendo afetados pela regra da imagem acima).

De forma similar, você pode criar um efeito de hover com:

```
#btn:hover {
  background-position: <pixels para direita>px <pixels para baixo>px;
}
```

Veja também

- Demo completa em CSS Tricks

Media queries

Media Queries (consultas de mídia) são um módulo de CSS que definem expressões, permitindo ajustar visualizações a uma variedade específica de dispositivos de saída sem mudar o conteúdo.

Referências

At-rules

- `@import`
- `@media`

Guides

Usando Media Queries

Apresenta o que fazem as media queries (consultas de mídia) e explica as possíveis expressões.

Testing media queries

Explains how to test a media query programmatically, from JavaScript.

Printing

Pode haver momentos em que seu site ou aplicação queira melhorar a experiência do usuário quando imprime um conteúdo. Existem diversos cenários possíveis:

- Você deseja ajustar o layout para tirar vantagem do tamanho e forma do papel.
- Você deseja usar diferentes estilos para melhorar a aparência do seu conteúdo no papel.
- Você deseja aumentar a resolução das imagens para um melhor resultado.
- Você quer ajustar a experiência do usuário de impressão, como apresentar uma versão especialmente formatada de seu conteúdo antes da impressão começar.

Podem haver outros casos onde você precisa gerenciar o processo de impressão, mas estes são alguns dos cenários mais comuns. Este artigo ensina dicas e técnicas para te ajudar a imprimir conteudo web de uma melhor forma.

Usando uma folha de estilos para impressão

Adicione o seguinte código dentro da tag `<head>`.

```
<link href="/path/to/print.css" media="print" rel="stylesheet" />
```

Usando media queries para melhorar o layout

Detectando requisições de impressão

Alguns navegadores (incluindo o Firefox 6 e versões mais antigas do Internet Explorer) enviam eventos `beforeprint` e `afterprint` para permitir que o conteúdo determine quando a impressão deve ocorrer. Você pode usar isto para ajustar a interface presente durante a impressão (como a exibição ou ocultação de elementos de interface do usuário durante o processo de impressão).

> **Nota:** Você também pode usar `window.onbeforeprint` e `window.onafterprint` para atribuir manipuladores para esses eventos, mas usando `EventTarget.addEventListener()` é preferível.

Exemplos

Aqui estão alguns exemplos comuns.

Abrir e fechar automaticamente uma janela popup quando finalizado

If you want to be able to automatically close a popup window (for example, the printer-friendly version of a document) after the user prints its contents, you can use code like this:

```
<!doctype html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
    <title>JavaScript Window Close Example</title>
    <script type="text/javascript">
      function popuponclick() {
        my_window = window.open(
          "",
          "mywindow",
          "status=1,width=350,height=150",
        );
        my_window.document.write("<html><head><title>Print Me</title></head>");
        my_window.document.write('<body onafterprint="self.close()">');
        my_window.document.write(
          "<p>When you print this window, it will close afterward.</p>",
        );
        my_window.document.write("</body></html>");
      }
    </script>
  </head>
  <body>
    <p>
      To try out the <code>afterprint</code> event, click the link below to open
      the window to print. You can also try changing the code to use
      <code>beforeprint</code> to see the difference.
    </p>
    <p><a href="javascript: popuponclick()">Open Popup Window</a></p>
```

```
  </body>
</html>
```

Ver Exemplo

Imprimir uma página externa sem abri-la

If you want to be able to print an external page without opening it, you can utilize a hidden `<iframe>` (see: HTMLIFrameElement), automatically removing it after the user prints its contents. The following is a possible example which will print a file named `externalPage.html`:

```
<!doctype html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
    <title>MDN Example</title>
    <script type="text/javascript">
      function closePrint() {
        document.body.removeChild(this.__container__);
      }

      function setPrint() {
        this.contentWindow.__container__ = this;
        this.contentWindow.onbeforeunload = closePrint;
        this.contentWindow.onafterprint = closePrint;
        this.contentWindow.focus(); // Required for IE
        this.contentWindow.print();
      }

      function printPage(sURL) {
        var oHiddFrame = document.createElement("iframe");
        oHiddFrame.onload = setPrint;
        oHiddFrame.style.position = "fixed";
        oHiddFrame.style.right = "0";
        oHiddFrame.style.bottom = "0";
        oHiddFrame.style.width = "0";
        oHiddFrame.style.height = "0";
        oHiddFrame.style.border = "0";
        oHiddFrame.src = sURL;
        document.body.appendChild(oHiddFrame);
      }
    </script>
  </head>

  <body>
    <p>
      <span
        onclick="printPage('externalPage.html');"
        style="cursor:pointer;text-decoration:underline;color:#0000ff;"
        >Print external page!</span
      >
    </p>
  </body>
</html>
```

> **Nota:** Older versions of Internet Explorer cannot print the contents of a hidden `<iframe>`.

Veja também

- `window.print`
- `window.onbeforeprint`
- `window.onafterprint`
- Media queries
- `@media`

Usando Media Queries

Uma **media query** consiste de um *media type* e pelo menos uma expressão que limita o escopo das folhas de estilo usando *media features*, tal como largura, altura e cor. *Media queries*, adicionadas no CSS3, deixam a apresentação do conteúdo adaptado a uma gama especifica de dispositivos não

precisando mudar o conteúdo em si.

Sintaxe

Media queries consistem de um *media type* e podem, a partir de uma especificação CSS3, contendo uma ou mais expressões, expressa em *media features*, que determinam ou verdadeiro ou falso. Os resultados da *query* são verdadeiros se o *media type* especificado na *media query* corresponde ao tipo do documento exibido no dispositivo e todas as expressões na *media query* são verdadeiras.

```
<!-- CSS media query em um elemento de link -->
<link rel="stylesheet" media="(max-width: 800px)" href="example.css" />
```

```
/* CSS media query dentro de um stylesheet */
@media (max-width: 600px) {
  .facet_sidebar {
    display: none;
  }
}
```

Quando uma *media query* é verdadeira, a camada de estilo ou as regras de estilos correspondentes são aplicadas, seguindo o padrão de regras de cascatas. Camadas de estilos com *media queries* ligadas a tag <link> vão fazer download mesmo se suas *medias queries* retornarem falso (eles não se aplicam, no entanto).

A menos que você use os operadores `not` ou `only`, o *media type* é opcional e o tipo `all` será implícito.

Operadores lógicos

Você pode compor *media queries* complexos usando operadores lógicos, incluindo `not`, `and`, e `only`. O operador `and` é usado para combinar múltiplas *media features* em uma mesma *media query*, requerendo que cada sequência de características, retorne verdadeiro na ordem para que a *query* seja verdadeiro. O operador `not` é usado para negar uma *media query* inteira. O operador `only` é usado para aplicar um estilo apenas se a *query* inteira for igual, útil para previnir que navegadores antigos apliquem os estilos selecionados. Se você usar os operadores `not` ou `only`, você tem que especificar um tipo de *media* explícito.

Você também pode combinar múltiplas *medias queries* em uma lista separadas por vírgulas, se qualquer uma das *media queries* na lista é verdadeira, toda a instrução retorna verdadeira. Isto é equivalente a um operador `or`.

and

A palavra-chave `and` é usada para combinar múltiplas *media features*, bem como combinar *media features* com *media types*. Uma *media query* básica, uma *media feature* simples com a *media type* `all`, pode parecer com isso:

```
@media (min-width: 700px) { ... }
```

Se, no entanto, você desejar que isso se aplique apenas para telas em landscape, você pode usar o operador `and` para deixar todas as *media features* juntas.

```
@media (min-width: 700px) and (orientation: landscape) { ... }
```

Agora, a *media query* acima vai apenas retorna verdadeira se o viewport for 700px, *wide* ou *wider* e a tela estiver em *landscape*. Se, no entanto, você deseja apenas que isso seja aplicado se a tela em questão for *media type* TV, você pode encadear essas *features* com a *media type* usando o operador `and`.

```
@media tv and (min-width: 700px) and (orientation: landscape) { ... }
```

Agora, a *media query* acima vai ser aplicada apenas se a *media type* for TV, o *viewport* for 700px *wide* ou *wider*, e a tela estiver em paisagem.

Listas separadas por vírgula

Listas separadas por vírgulas comportam-se como o operador `or` quando utilizadas em *media queries*. Quando utilizamos *media queries* com uma lista separada por vírgulas, se qualquer *media queries* retornar verdadeiro, os estilos ou folhas de estilos serão aplicadas. Cada *media query* em um lista separa por vírgulas é tratada como uma *query* individual, e qualquer operador aplica em uma *media query* não afeta os outros. Isto significa que *media queries* separadas por vírgulas podem ter objetivos diferentes de *media features*, *types* e *states*.

Por exemplo, se você quiser aplicar um conjunto de estilos se o dispositivo de visualização tiver um largura mínima de 700px ou estiver sendo segurando em paisagem, você pode escrever o seguinte:

```
@media (min-width: 700px), handheld and (orientation: landscape) { ... }
```

Acima, se eu estivesse em um dispositivo `screen` com um *viewport* largura de 800px, a afirmação retorna verdadeiro por que a primeira parte, interpretada como `@media all and (min-width: 700px)` será aplicada no meu dispositivo e portanto retorna verdadeiro, apesar do fato que meu dispositivo `screen` iria falhar no *media type* `handheld` na segunda *media query*. Do mesmo modo, se eu estivesse segurando um dispositivo em paisagem com um *viewport* de largura de 500px, enquanto a primeira media query falha devido a largura do *viewport*, a segunda *media query* teria sucesso e assim o *media statement* retorna verdadeiro.

not

A palavra chave `not` se aplica em toda a *media query* e retorna verdadeiro, caso contrário retorna falso (tal como monochrome como cor de tela ou uma tela de largura de 600px com um `min-width: 700px` recurso consultado). Um not vai apenas negar a *media query* que é aplicada, de modo não toda a *media query* que apresente uma *media querie* com uma lista separada por vírgulas. A palavra chave `not` não pode ser usada para negar uma característica individual da *query*, apenas uma *media query* inteira. Por exemplo, o `not` é avaliado por último na *query* seguinte:

```
@media not all and (monochrome) { ... }
```

Isto significa que a *query* é avaliada assim:

```
@media not (all and (monochrome)) { ... }
```

... em vez disso:

```
@media (not all) and (monochrome) { ... }
```

Um outro exemplo, veja a *media query* seguinte:

```
@media not screen and (color), print and (color) { ... }
```

É avalida desta forma:

```
@media (not (screen and (color))), print and (color) { ... }
```

only

A palavra chave *`only`* previne que navegadores antigos que não suportam media queries com media features de aplicar os estilos dados:

```
<link rel="stylesheet" media="only screen and (color)" href="example.css" />
```

Pseudo-BNF

```
media_query_list: <media_query> [, <media_query> ]*
media_query: [[only | not]? <media_type> [ and <expression> ]*]
  | <expression> [ and <expression> ]*
expression: ( <media_feature> [: <value>]? )
media_type: all | aural | braille | handheld | print |
  projection | screen | tty | tv | embossed
media_feature: width | min-width | max-width
  | height | min-height | max-height
  | device-width | min-device-width | max-device-width
  | device-height | min-device-height | max-device-height
  | aspect-ratio | min-aspect-ratio | max-aspect-ratio
  | device-aspect-ratio | min-device-aspect-ratio | max-device-aspect-ratio
  | color | min-color | max-color
  | color-index | min-color-index | max-color-index
  | monochrome | min-monochrome | max-monochrome
  | resolution | min-resolution | max-resolution
  | scan | grid
```

Media queries são *case insensitive*. *Media queries* envolvidas em *media types* desconhecidos serão sempre falsas.

> **Nota:** Parenteses são obrigatórios em volta de expressões; a falta deles é um erro.

Características de mídia

A maioria das *media features* podem ter prefixo "min-" ou "max-" para expressar as restrições "maior ou igual" ou "menor ou igual". Isto evita o uso dos símbolos "<" e ">" , que entrem em conflito com HTML e XML. Se você usar uma *media feature* sem especificar um valor, a expressão retorna

verdadeiro, se o valor da *feature* for diferente de zero.

> **Nota:** Se uma media feature não se aplicar ao dispositivo onde o navegador esta sendo executado, as expressões que envolvem essa media feature são sempre falsas. Por exemplo, consultar um aspecto de um dispositivo sonoro, sempre resulta em falso.

cor

Valor: `<color>` **Mídia:** `Visual` **Aceita prefixos min/max:** sim

Indica o número de bits por componente de cor no dispositivo de saída. Se o dispositivo não é um dispositivo de cor, o valor é zero.

> **Nota:** Se os componentes de cor têm diferentes números de bits por componente de cor, o menor valor é utilizado. Por exemplo, se o display usa 5 bits para azul e vermelho e 6 bits para verde, então o dispositivo considera 5 bits por componente de cor. Se o dispositivo usar cores indexadas, o menor número de bits por componente de cor na tabela de cores é usado.

Exemplos

Aplicar uma folha de estilo a todos dispositivos:

```
@media all and (color) { ... }
```

Aplicar uma folha de estilo a todos dispositivos com no mínimo 4 bits de color componente:

```
@media all and (min-color: 4) { ... }
```

color-index

Valor: `<integer>` **Mídia:** `Visual` **Aceita prefixos min/max:** Sim

Indica o número de entradas na tabela de consulta de cores para o dispositivo de saída.

Exemplos

Para indicar que uma folha de estilo deve ser aplicada para todos os dispositivos que usam cores indexadas, você pode fazer:

```
@media all and (color-index) { ... }
```

Para aplicar uma folha de estilo em um dispositivo com cores indexadas menor que 256 cores:

```
<link
  rel="stylesheet"
  media="all and (min-color-index: 256)"
  href="http://foo.bar.com/stylesheet.css" />
```

aspect-ratio

Valor: `<ratio>` **Mídia:** `Visual`, `Tactile` **Aceita prefixos min/max:** sim

Descreve o aspecto da relação da área do display do dispositivo de saída. Este valor consiste de dois inteiros positivos separados por um caractere barra ("/"). Isto representa a relação entre pixels horizontais (primeiro termo) para pixels verticais (segundo termo).

Exemplo

A seguir selecionamos uma folha de estilo especial para usarmos quando a área do display é pelo menos mais larga do que alta.

```
@media screen and (min-aspect-ratio: 1/1) { ... }
```

Isto seleciona o estilo quando a relação de aspecto seja 1:1 ou maior. Em outras palavras, estes estilos serão aplicados apenas quando a área de visualização for quadrada ou paisagem.

device-aspect-ratio

Valor: `<ratio>` **Mídia:** `Visual`, `Tactile` **Aceita prefixos min/max:** sim

Descreve a relação de aspecto do dispositivo de saída. Este valor consiste de dois inteiros positivos separados pelo carácter barra ("/"). Isto representa a relação de pixels horizontais (primeiro termo) por pixels verticais (segundo termo).

Exemplo

A seguir, selecionamos uma folha de estilo especial para usar em telas widescreen.

```
@media screen and (device-aspect-ratio: 16/9), screen and (device-aspect-ratio: 16/10) { ... }
```

Isso seleciona o estilo quando a relação de aspecto é 16:9 ou 16:10.

device-height

Valor: `<length>` **Mídia:** `Visual`, `Tactile` **Aceita prefixos min/max:** sim

Descreve a altura do dispositivo de saída(ou seja, toda a tela ou página, em vez de apenas a área de renderização, tal como a janela do documento).

Exemplo

Para aplicar uma folha de estilo a um documento quando exibido em uma tela menor que 800 pixels de altura, você pode usar isso:

```
<link rel="stylesheet" media="screen and (max-device-height: 799px)" />
```

device-width

Valor: `<length>` **Mídia:** `Visual`, `Tactile` **Aceita prefixos min/max:** sim

Descreve a largura do dispositivo e saída (ou seja, toda a tela ou página, em vez de apenas a área de renderização, tal como a janela do documento).

Exemplo

Para aplicar uma folha de estilo a um documento quando exibido em uma tela menor que 800px de largura, você pode usar isso:

```
<link rel="stylesheet" media="screen and (max-device-width: 799px)" />
```

grid

Valor: `<integer>` **Mídia:** todas **Aceita prefixos min/max:** não

Determina se o dispositivo de saída é um dispositivo grade ou um dispositivo bitmap. Se o dispositivo é baseado em grade(tal como um terminal TTY ou uma tela de telefone com apenas um tipo de letra), o valor é 1. De outro modo é zero.

Exemplo

Para aplicar um estilo a dispositivos postáteis com 15-carácteres ou uma tela mais estreita:

```
@media handheld and (grid) and (max-width: 15em) { ... }
```

> **Nota:** A unidade "em" tem um significado especial para dispositivos de grade, uma vez que a exata largura de um "em" não pode ser determinada, 1em é assumido para ser a largura de uma célula da grade horizontalmente, e a altura de uma célula verticalmente.

height

Valor: `<length>` **Mídia:** `Visual`, `Tactile` **Aceita prefixos min/max:** yes

A característica `height` descreve a altura da superfície de renderização do dispositivo de saída (tal como a altura do viewport ou da caixa de página em uma impressora).

> **Nota:** Como o usuário redimensiona a janela, o Firefox muda as folhas de estilo como apropriado, com base nas media queries, usando as media features `width` e `height`.

monochrome

Valor: `<integer>` **Mídia:** `Visual` **Aceita prefixos min/max:** sim

Indica o número de bits por pixel em um dispositivo monocromático (greyscale). Se o dispositivo não for monocromático, o valor é 0.

Exemplos

Para aplicar uma folha de estilo em todos os dispositivos monocromáticos:

```
@media all and (monochrome) { ... }
```

Para aplicar uma folha de estilo em dispositivos monocromáticos com pelo menos 8 bits por pixel:

```
@media all and (min-monochrome: 8) { ... }
```

orientation

Valor: `landscape` | `portrait` **Mídia:** `Visual` **Aceita prefixos min/max:** não

Indica se o *viewport* é modo *landscape* (o visor é mais largo do que mais alto) ou *portrait* (o visor é mais alto do que mais largo).

Exemplo

Para aplicar a folha de estilo apenas em orientação *portrait*:

```
@media all and (orientation: portrait) { ... }
```

> **Nota:** Este valor não corresponde com a orientação real do dispositivo. Abrindo o teclado virtual na maioria dos dispositivos na orientação retrato fará com que o viewport torne-se mais largo do que alto, fazendo assim que o navegador use estilos de paisagem em vez de retrato.

resolution

Valor: `<resolution>` **Mídia:** `bitmap` **Aceita prefixos min/max:** sim

Indica a resolução (densidade de pixel) da saída do dispositivo. A resolução pode ser especificada em pontos por inch(dpi) ou pontos por centímetro(dpcm).

Exemplos

Para aplicar a folha de estilo em dispositivos com resolução de pelo menos 300 pontos por inch:

```
@media print and (min-resolution: 300dpi) { ... }
```

Para substituir a antiga sintaxe (min-device-pixel-ratio: 2):

```
@media screen and (min-resolution: 2dppx) { ... }
```

scan

Valor: `progressive` | `interlace` **Mídia:** `TV` **Aceita prefixos min/max:** não

Descreve o processo de digitalização de dispositivos saída de televisão.

Exemplo

Para aplicar uma folha de estilo apenas para televisores de varredura progressiva:

```
@media tv and (scan: progressive) { ... }
```

width

Valor: `<length>` **Mídia:** `Visual`, `Tactile` **Aceita prefixos min/max:** sim

A *media feature* `width` descreve a largura da superficie de renderização do dispositivo de saída (tal como a largura da janela do documento, ou a largura da caixa de página em uma impressora).

Nota: Como o usuário redimensiona a janela, o Firefox muda as folhas de estilos como apropriado baseado em *media queries* usando media features `width` e `height`.

Exemplos

Se você quiser especificar uma folha de estilo para dispositivos portáteis, ou dispositivos screen com uma largura maior que 20em, você pode usar essa *query*:

```
@media handheld and (min-width: 20em), screen and (min-width: 20em) { ... }
```

Essa *media query* especifica uma folha de estilo que aplica-se para mídias impressas maiores que 8.5 inches.

```
<link
  rel="stylesheet"
  media="print and (min-width: 8.5in)"
  href="http://foo.com/mystyle.css" />
```

Essa *query* especifica uma folha de estilo que é usada quano o viewport está entre 500 e 800 pixels de largura:

```
@media screen and (min-width: 500px) and (max-width: 800px) { ... }
```

Especificação da Mozilla para mídias características

Mozilla oferece várias *media features* para específicos *Gecko* . Algumas dessas podem ser sugeridas como *media features* oficiais.

-moz-images-in-menus

Valor: `<integer>` **Mídia:** `Visual` **Aceita prefixos min/max:** não

Se o dispositivo permite aparecer imagens nos menus, o valor é 1; caso contrário, o valor é 0. Isto corresponde ao `:-moz-system-metric(images-in-menus)` CSS pseudo-class.

-moz-mac-graphite-theme

Valor: `<integer>` **Mídia:** `Visual` **Aceita prefixos min/max:**no

Se o usuário tenha configurado seu dispositivo para usar a aparência "*Graphite*" no *Mac OS X*, o valor é 1. Se o usuário está usando a aparência padrão *blue*, ou não está num *Mac OS X*, o valor é 0.

Isto corresponde ao `:-moz-system-metric(mac-graphite-theme)` CSS pseudo-class.

-moz-maemo-classic

Valor: `<integer>` **Mídia:** `Visual` **Aceita prefixos min/max:** não

Se o usuário está usando *Maemo* com o tema original, o valor é 1; Se está usando o mais novo tema *Fremantle*, o valor é 0.

Isto corresponde ao `:-moz-system-metric(maemo-classic)` CSS pseudo-class.

-moz-device-pixel-ratio

Valor: `<number>` **Mídia:** `Visual` **Aceita prefixos min/max:** sim

Dar o número de pixels do dispositivo por pixels do CSS.

Não use este recurso.

Em vez disso, use o recurso *`resolution`* com a unidade `dppx`.

`-moz-device-pixel-ratio` pode ser usada para compatibilidade com Firefox mais velho que a versão 16 e `-webkit-device-pixel-ratio` com navegadores baseados no WebKit que não suportam `dppx`.

Exemplo:

```
@media (-webkit-min-device-pixel-ratio: 2), /* Navegadores baseados no Webkit */
       (min--moz-device-pixel-ratio: 2),    /* Navegadores mais antigos do Firefox (antes do Firefox 16) */
       (min-resolution: 2dppx),             /* Forma padrão */
       (min-resolution: 192dpi)             /* dppx fallback */
```

Veja este artigo CSSWG para ccompatibilidade de boas práticas em relação a *`resolution`* e *`dppx`*.

> **Nota:** Esta *media feature* é também implementada pelo Webkit e pelo IE 11 para Windows Phone 8.1como -webkit-device-pixel-ratio. Os prefixos min e max implementados pelo Gecko são nomeados min--moz-device-pixel-ratio e max--moz-device-pixel-ratio; mas os mesmos prefixos implementados pelo Webkit são chamados -webkit-min-device-pixel-ratio e -webkit-max-device-pixel-ratio.

-moz-os-version

Valor: `windows-xp` | `windows-vista` | `windows-win7` | `windows-win8` **Mídia:** `Visual` **Aceita prefixos min/max:** não

Indica qual versão do sistema operacional está sendo usado atualmente. Atualmente apenas implementada no Windows. Possíveis valores são:

- `windows-xp`
- `windows-vista`
- `windows-win7`
- `windows-win8`

Isto é fornecido pelas *skins das aplicações* e outros códigos do chrome para serem capazes de se adaptar para funcionar bem com a versão atual do sistema operacional.

-moz-scrollbar-end-backward

Valor: `<integer>` **Mídia:** `Visual` **Aceita prefixos min/max:** não

Se a interface do usuário do dispositivo exibe uma seta para trás no final da barra de rolagem, o valor é 1. Caso contrário, é 0.

Isto corresponde ao `:-moz-system-metric(scrollbar-end-backward)` CSS pseudo-class.

-moz-scrollbar-end-forward

Valor: `<integer>` **Mídia:** `Visual` **Aceita prefixos min/max:** não

Se a interface do usuário do dispositivo a forward arrow button at the end of scrollbars, this is 1. Otherwise it's 0.

Isto corresponde ao `:-moz-system-metric(scrollbar-end-forward)` CSS pseudo-class.

-moz-scrollbar-start-backward

Valor: `<integer>` **Mídia:** `Visual` **Aceita prefixos min/max:** não

Se a interface do usuário do dispositivo a backward arrow button at the beginning of scrollbars, this is 1. Otherwise it's 0.

Isto corresponde ao `:-moz-system-metric(scrollbar-start-backward)` CSS pseudo-class.

-moz-scrollbar-start-forward

Valor: `<integer>` **Mídia:** `Visual` **Aceita prefixos min/max:** não

Se a interface do usuário do dispositivo a forward arrow button at the beginning of scrollbars, this is 1. Otherwise it's 0.

Isto corresponde ao `:-moz-system-metric(scrollbar-start-forward)` CSS pseudo-class.

-moz-scrollbar-thumb-proportional

Valor: `<integer>` **Mídia:** `Visual` **Aceita prefixos min/max:** não

Se a interface do usuário do dispositivo the thumb of scrollbars proportionally (that is, sized based on the percentage of the document that is visible), this is 1. Otherwise it's 0.

Isto corresponde ao `:-moz-system-metric(scrollbar-thumb-proportional)` CSS pseudo-class.

-moz-touch-enabled

Valor: `<integer>` **Mídia:** `Visual` **Aceita prefixos min/max:** não

If the device supports touch events (for a touch screen), this is 1. Otherwise it's 0.

Isto corresponde ao `:-moz-system-metric(touch-enabled)` CSS pseudo-class.

Exemplo

You might use this to render your buttons slightly larger, for example, if the user is on a touch-screen device, to make them more finger-friendly.

-moz-windows-classic

Valor: `<integer>` **Mídia:** `Visual` **Aceita prefixos min/max:** não

If the user is using Windows unthemed (in classic mode instead of using uxtheme), this is 1; otherwise it's 0.

Isto corresponde ao `:-moz-system-metric(windows-classic)` CSS pseudo-class.

-moz-windows-compositor

Valor: `<integer>` **Mídia:** `Visual` **Aceita prefixos min/max:** não

If the user is using Windows with the DWM compositor, this is 1; otherwise it's 0.

Isto corresponde ao `:-moz-system-metric(windows-compositor)` CSS pseudo-class.

-moz-windows-default-theme

Valor: `<integer>` **Mídia:** `Visual` **Aceita prefixos min/max:** não

If the user is currently using one of the default Windows themes (Luna, Royale, Zune, or Aero (including Vista Basic, Vista Advanced, and Aero Glass), this is 1. Otherwise it's 0.

Isto corresponde ao `:-moz-system-metric(windows-default-theme)` CSS pseudo-class.

-moz-windows-glass

Valor: `<integer>` **Mídia:** `Visual` **Aceita prefixos min/max:** não

If the user is using Windows Glass theme, this is 1; otherwise it's 0. Note that this only exists for Windows 7 and earlier.

-moz-windows-theme

Valor: `aero` | `luna-blue` | `luna-olive` | `luna-silver` | `royale` | `generic` | `zune` **Mídia:** `Visual` **Aceita prefixos min/max:** não

Indicates which Windows theme is currently being used. Only available on Windows. Possible values are:

- `aero`
- `luna-blue`
- `luna-olive`
- `luna-silver`
- `royale`
- `generic`
- `zune`

Isto é previsto para *skins* de aplicativo e outro código de aplicações de chrome a ser capaz de se adaptar a funcionar bem com o actual tema do Windows.

See also

- CSS 3 media query specification
- Media types
- Using media queries from code
- List of mobile and tablet viewport sizes with pixel ratios and physical sizes
- CSS Animations Between Media Queries by David Walsh

CSS Positioning

CSS Positioning é um módulo do CSS que define como ficam a posição dos elementos relativos e absolutos da página.

Referências

CSS Properties

- `bottom`
- `clear`
- `display`
- `float`
- `left`
- `position`

- `right`
- `top`
- `visibility`
- `z-index`

Roteiro

Entendendo o CSS z-index

> Apresenta o conceito de contexto de empilhamento e explica como funciona o atributo z-index, com diversos exemplos.

Understanding CSS z-index

Normalmente páginas HTML podem ser consideradas bi-dimensionais, pois texto, imagens e outros elementos podem ser dispostos na página sem sobreposição. Há apenas um fluxo de renderização e todos os elementos sabem do espaço ocupado por outros. O atributo `z-index` lhe permite ajustar a ordem de sobreposição dos objetos ao renderizar o conteúdo.

> *Em CSS 2.1, cada caixa tem uma posição nas três dimensões. Em adição às suas posições na horizontal e vertical, caixas ficam no "eixo-z" e são formatadas uma em cima da outra. Posições no Eixo-Z são particularmente relevantes quando caixas se sobrepõem visualmente.*

(from CSS 2.1 Section 9.9.1 - Layered presentation)

Isso significa que as regras de CSS te permitem posicionar caixas em camadas em adição ao *render* normal da camada (level 0). A posição Z de cada camada é expressa como um inteiro representando a ordem da pilha para renderização. Números maiores significam que são mais próximos do observador. A posição Z pode ser controlada pela propriedade CSS `z-index`.

Usar z-index aparenta ser extremamente fácil: uma única propriedade, endereçada a um único número inteiro, com um comportamento fácil-de-entender. No entanto, quando o z-index é aplicado para a hierarquia complexa dos elementos de HTML, seu comportamento pode ser difícil de entender ou até imprevisível. Isso é devido às complexas regras de *stacking*. Uma sessão dedicada foi reservada na especificação do CSS CSS-2.1 Appendix E para explicar melhor essas regras.

Esse artigo tentará explicar essas regras, com algumas simplificações e vários exemplos.

1. Stacking without z-index : Regras padrões de empilhamento
2. Stacking and float : Como lidar com elementos que usam *float*
3. Adding z-index : Usando index-z para mudar o empilhamento padrão
4. The stacking context : Notas sobre contexto do empilhamento
5. Stacking context example 1 : 2-level HTML hierarquia, z-index no último *level*
6. Stacking context example 2 : 2-level HTML hierarquia, z-index em todos os *levels*
7. Stacking context example 3 : 3-level HTML hierarquia, z-index no segundo *level*

Note of the author: Thanks to Wladimir Palant and Rod Whiteley for the review.

Original Document Information

- Author(s): Paolo Lombardi
- This article is the English translation of an article I wrote in Italian for YappY. I grant the right to share all the content under Creative Commons: Attribution-Sharealike license
- Last Updated Date: July 9th, 2005

Seletores CSS

Os Seletores definem quais elementos um conjunto de regras CSS se aplica.

Seletores Básicos

Seletor por tag

> Este seletor básico escolhe todos os elementos que correspondem ao nome fornecido. **Sintaxe:** `nome-da-tag` **Exemplo:** `input` corresponderá a qualquer elemento `<input>`.

Seletor por classe

> Este seletor básico escolhe elementos baseados no valor de seu atributo `classe`. **Sintaxe:** `.nome-da-classe` **Exemplo:** `.index` irá corresponder a qualquer elemento que tenha o índice de classe (provavelmente definido por um atributo class="index", ou similar).

Seletor por ID

Este seletor básico escolhe nós baseados no valor do atributo `id`. Deve existir apenas um elemento com o mesmo ID no mesmo documento. **Sintaxe:** `#nome-do-id` **Exemplo:** `#toc` irá corresponder ao elemento que possuir o id=toc (definido por um atributo id="toc", ou similar).

Seletores universais

Este seletor básico irá escolher todos os nós. Ele também existe em um namespace único e em uma variante de todo o namespace também. **Sintaxe:** `* ns|* *|*` **Exemplo:** `*` irá corresponder a todos os elementos do documento.

Seletores por atributo

Este seletor básico ira escolher nós baseados no valor de um de seus atributos, ou até mesmo pelo próprio atributo. **Sintaxe:** `[atrib] [atrib=valor] [atrib~=valor] [atrib|=valor] [atrib^=valor] [atrib$=valor] [atrib*=valor]` **Exemplo:** `[autoplay]` irá corresponder a todos os elementos que possuirem o atributo `autoplay` (para qualquer valor).

Combinadores

Seletores de irmãos adjacentes

O combinador + seleciona os nós que seguem imediatamente o elemento especificado anteriormente. **Sintaxe:** `A + B` **Exemplo:** `ul + li` irá corresponder a qualquer elemento `<li>` que segue imediatamente após um elemento `<ul>`.

Seletores gerais de irmãos

O combinador ~ seleciona os nós que seguem (não necessariamente imediatamente) o elemento especificado anteriormente, se ambos os elementos compartilham o mesmo pai. **Sintaxe:** `A ~ B` **Exemplo:** `p ~ span` irá corresponder a todo elemento `<span>` que seguir um elemento `<p>` dentro de um mesmo elemento pai.

Seletor de filhos

O combinador > seleciona nós que são filhos diretos do elemento especificado anteriormente. **Sintaxe:** `A > B` **Exemplo:** `ul > li` irá corresponder a todo elemento `<li>` que estiver diretamente dentro de um elemento `<ul>` especificado.

Seletor de descendentes

O combinador " " seleciona os nós que são filhos do elemento especificado anteriormente (não é necessário que seja um filho direto). **Sintaxe:** `A B`**Exemplo:**`div span` irá corresponder a todo e qualquer elemento `<span>` que estiver dentro do elemento `<div>`.

Pseudo-classes

Pseudo-classes permitem selecionar elementos baseados em informações que não estão contidas na árvore de documentos como um estado ou que é particularmente complexa de extrair. Por exemplo, eles correspondem se um link foi visitado anteriormente ou não.

Pseudo-elementos

Pseudo-elementos são asbtrações da árvore que representam entidades além do que o HTML faz. Por exemplo, o HTML não tem um elemento que descreva a primeira letra ou linha de um parágrafo, ou o marcador de uma lista. Os pseudo-elementos representam essas entidades e permitem que as regras CSS sejam associadas a elas. Desta forma, essas entidades podem ser denominadas independentemente.

Privacidade e o seletor :visited

Antes de 2010, o seletor CSS `:visited` permitia que websites descobrissem o histórico de navegação dos usuários e quais sites estes haviam visitado. Isto foi feito por meio do `window.getComputedStyle` e outras tecnicas. Este processo era fácil de ser feito, e tornou possível não somente determinar onde os usuários estiveram na internet, mas também poderia ser usado para descobrir um monte de informação sobre a identidade destes.

Para contornar este problema, Gecko 2 implementou atualizações de privacidade para limitar a quantidade de informações que poderiam ser obtidas através dos links visitados. Outros navegadores também fizeram mudanças similares.

Pequenas mentiras brancas

Para preservar a privacidade dos usuários, Firefox e outros navegadores irão mentir para aplicações webs sob certas circustâncias:

- O método `window.getComputedStyle`, e funções similares tal como `element.querySelector`, sempre retornarão valores indicando que os usuários nunca visitaram quaisquer links em uma página.
- Se você usa seletores como **seletores de irmãos** como `:visited + span`, o elemento imediato (`span` neste exemplo) será estilizado como se o link não houvesse sido visitado.
- Em raros casos, se você está utilizando elementos de links aninhados e o elemento que está sendo correspondido é diferente do link cuja presença no histórico está sendo testado, o elemento também será renderizado como se o link não houvesse sido visitado.

Limites para estilos de links visitados

Você pode estilizar links visitados, porém existem limites de quais estilos você pode usar. Somente os seguintes estilos podem ser aplicados para links visitados:

- `color`
- `background-color`
- `border-color` (e suas subpropriedades)
- `column-rule-color`
- `outline-color`
- As partes de cor dos atributos `fill` e `stroke`

Em adição, mesmo para os estilos acima, você não poderá alterar a transparência entre links visitados e não visitados, ou de outra maneira você seria capaz de usar `rgba()`, `hsla()`, ou a palavra chave `transparent`.

Aqui está um exemplo de como estilizar com as restrições acima mencionadas:

```
:link {
  outline: 1px dotted blue;
  background-color: white;
  /* O valor padrão de background-color é `transparent`. Você precisa
     especificar um valor diferente, caso contrário as mudanças
     feitas em :visited não se aplicam. */
}

:visited {
  outline-color: orange; /* Links visitados tem contornos laranja */
  background-color: green; /* Links visitados tem um fundo verde */
  color: yellow; /* Links visitados tem texto amarelo */
}
```

Impacto nos desenvolvedores web

No geral, essas restrições estas restrições não deveriam afetar desenvolvedores web tão significamente. Eles poderiam, como sempre, They may, no entanto, requerer essas seguintes mudanças nos sites existentes:

- Usar imagens de fundo para estilizar links baseados se eles foram visitados não irá funcionar, desde que apenas cores podem ser usadas para estilizar links visitados.
- As cores que são transparentes deixarão de aparecer se estilizados em seletores `:visited`.

Veja também

- privacy-related changes coming to CSS :visited em Mozilla Hacks (em inglês)
- Plugging the CSS History Leak em the Mozilla Security Blog (em inglês)
- Preventing attacks on a user's history through CSS :visited selectors (em inglês)

Regra At

Uma **regra at (at-rule)** é uma CSS statement iniciando com um caractere arroba, '`@`' (`U+0040 ARROBA COMERCIAL`), seguido por um indetificador e terminando no primeiro ponto e vírgula', `;`' (`U+003B SEMICOLON`), fora de um bloco, ou no fim do primeiro Bloco CSS.

Existem muitas regras at, criadas por seu identificador, cada um com uma sintaxe diferente:

- `@charset`, definindo o conjunto de caracteres (codificação) usado pela folha de estilos.
- `@import`, informando o motor CSS para incluir uma folha de estilos externa.
- `@namespace`, informando ao motor CSS que todo o conteúdo deve ser considerado prefixado com um namespace XML.
- Regras at aninhadas, um subconjunto de regras aninhadas que pode ser usado não somente como uma regra de uma folha de estilos, mas também dentro de um grupo de regras condicional:
  - `@media`, um grupo de regras condicional que é aplicado ao conteúdo caso o dispositivo de exibição cumpra com a condição de mídia definida.
  - `@font-face`, descrevendo o aspecto de uma fonte externa que deve ser baixada.
  - `@document`, um grupo de regras condicional que é aplicado ao conteúdo caso o documento cumpra com os critério definidos.
  - `@supports`, um grupo de regras condicional que é aplicado caso o navegador cumpra com os critérios definidos.

Grupos condicionais de regras

Assim como valores de propriedades, cada regra at possui uma sintaxe diferente; várias delas, entretanto, podem ser agrupadas em uma categoria especial, os **grupos condicionais de regras**. Essas regras compartilham uma sintaxe em comum. Cada uma delas inclui *regras aninhadas*, que são

conjuntos de regras ou *regras at aninhadas*. Todas elas transmitem um significado semântico em comum: todas indicam uma condição (de um tipo diferente) para que as regras sejam aplicadas.

Grupos condicionais de regras são definidos pelo CSS3 e são:

- `@media`,
- `@document`,
- `@supports`.

Cada uma delas sendo também uma regra aninhada, pode haver uma quantidade indeterminada de aninhamentos.

Sintaxe

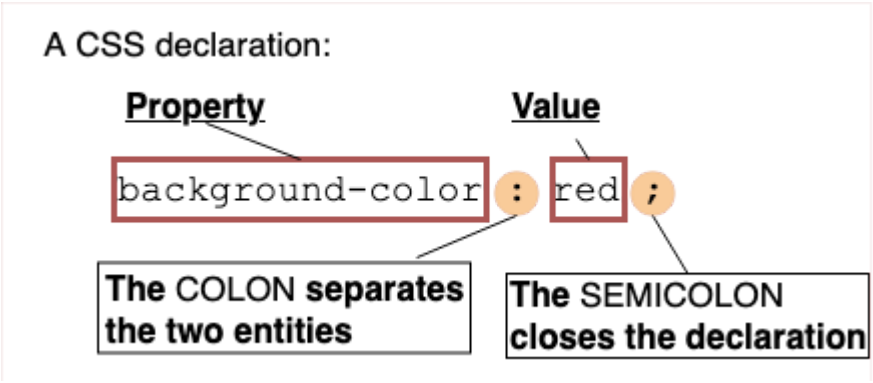
O Objetivo básico da linguagem de folhas de estilo em cascata (CSS) é permitir que um motor do navegador pinte elementos na página com características específicas como cores, posições, ou decorações. A sintaxe *CSS* reflete estes objetivos e seus blocos de contrução básicos são:

- A **propriedade** é um identificador que possui um nome legível, que define o que será considerado ou editado;
- O **valor** descreve como o recurso será tratado pelo motor. Cada propriedade possui um conjunto de valores válidos, definida por uma gramática formal, bem como um significado semântico, implementado para o motor do navegador.

Declarações do CSS

A definição de propriedades CSS para um valor específico é a função principal da linguagem CSS. A propriedade e valor são chamados de **declaração**, e qualquer motor do CSS calcula quais declarações serão aplicadas para todos um único elemento da página em ordem adequadamente, a fim de exibí-lo com o estilo correto.

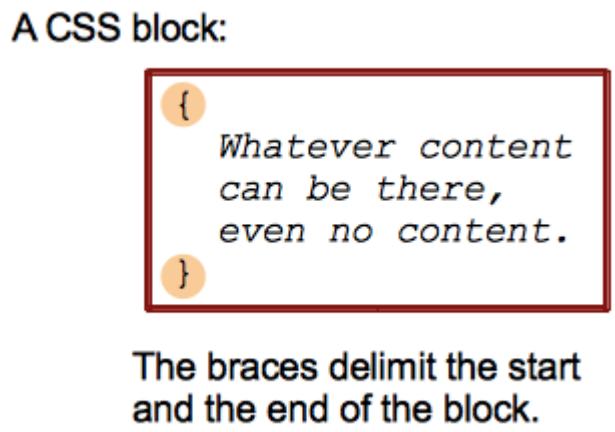
Tanto as propriedades como os valores são case-sensitive no CSS. Os pares se separam por dois pontos, '`:`' (`U+003A COLON`), e espaços em branco antes, entre e depois de propriedades e valores, porém os espaços dentro da declaração são ignorados.



Existem mais de 100 propriedades diferentes no CSS e um número quase infinito de diferentes valores. Nem todos os pares de propriedades e valores são permitidos em cada propriedade define o que são valores válidos. Quando um valor não é válido para uma determinada propriedade, a declaração é considerada inválida e é totalmente ignorada pelo motor do CSS.

Blocos de declaração CSS

Declarações são agrupadas em **blocos**, que estão delimitados na estrutura com uma chave de abertura, '`{`' (`U+007B LEFT CURLY BRACKET`), e fechadas com outra, '`}`' (`U+007D RIGHT CURLY BRACKET`). Os blocos as vezes podem estar aninhados, a abertura e fechamento de chaves no bloco CSS deve ser realizada.



Esses blocos são chamados de **blocos de declaração** e as declarações dentro deles são separadas por, '`;`' (`U+003B SEMICOLON`). Um bloco de declaração pode não conter nenhuma declaração. Espaços em branco em volta das declarações não são consideradas. Não é necessário que a última declaração possua ponto e vírgula, apesar de ser considerada uma boa prática pois previne o esquecimento de acrescê-la quando for necessário aumentar o bloco.

> **Nota:** O conteúdo de um bloco de declaração, que é uma lista separada por pontos e vírgulas, sem as chaves, pode ser posto dentro da tag HTML `style`.

Regras CSS

Se as folhas de estilo pudessem apenas aplicar uma declaração para cada elemento de uma página da web, eles seriam bem limitados. O principal objetivo é aplicar diferentes declarações a diferentes partes do documento.

O CSS associa as condições com os blocos de declaração. Cada bloco (válido) é precedido por um ou mais seletores, separados por vírgula, que são condições selecionando alguns elementos da página. O grupo de seletores é chamado de regra.

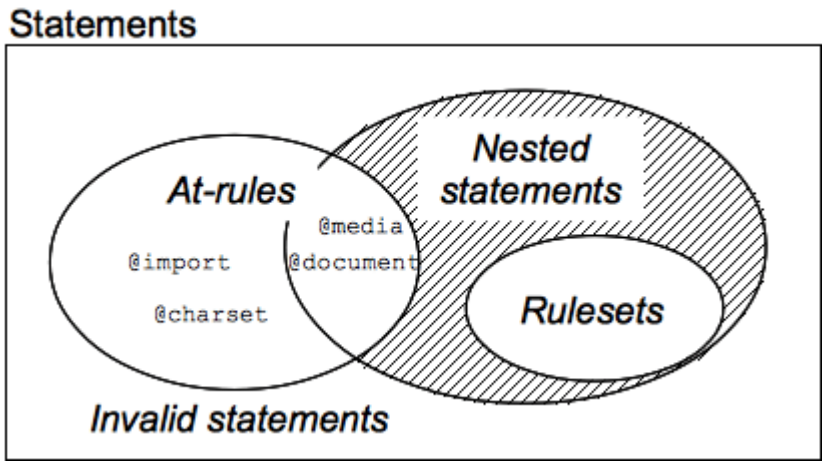
Um elemento pode ser modificado por diversos seletores, e por isso por diversas regras que potencialmente podem conter diversas propriedades, com diferentes valores, o CSS padrão define aquele que possui a precedência e que será aplicado: esse é o tal algoritmo em cascata.

> **Nota:** É importante perceber que quando uma regra é caracterizada por um grupo de seletores que são algum tipo de atalho com cada um sendo um simples seletor isso não se aplica a validade da regra por si só.
>
> Isso leva a uma importante consequência: se apenas um dos seletores for inválido, como usar uma pseudo-classe ou pseudo-elemento desconhecido, todo o *seletor* é inválido e por isso toda a regra é ignorada (invalidada também).

CSS statements

Rulesets are the main building blocks of a style sheet, which often consists of only a big list of them. But there is other information that a Web author wants to convey in the style sheet, like the character set, other external style sheets to import, font face or list counter descriptions and many more. It will use other and specific kinds of statements to do that.

A **statement** is a building block that begins with any non-space characters and ends at the first closing brace or semi-colon (outside a string, non-escaped and not included into another {}, () or [] pair).



There are different kinds of statements:

- **Rulesets** (or *rules*) that, as seen, associate a collection of CSS declarations to a condition described by a selector.
- **At-rules** that start with an at sign, '`@`' (`U+0040 COMMERCIAL AT`), followed by an identifier and then continuing up the end of the statement, that is up to the next semi-colon (;) outside of a block, or the end of the next block. Each type of at-rules, defined by the identifier, may have its own internal syntax, and semantics of course. They are used to convey meta-data information (like `@charset` or `@import`), conditional information (like `@media` or `@document`), or descriptive information (like `@font-face`).

Any statement which isn't a rule or an at-rule is invalid and ignored.

There is another group of statements, the **nested statements**, these are statements that can be used in a specific subset of at-rules, the *conditional group rules*. These statements only apply if a specific condition is matched: the `@media` at-rule content is applied only if the device on which runs the browser matches the expressed condition; the `@document` at-rule content is applied only if the current page matches some conditions, and so on. In CSS1 and CSS2.1, only *rulesets* could be used inside a conditional group rules. That was very restrictive and this restriction was lifted in *CSS*

Conditionals Level 3. Now, though it still is experimental and not support by every browser, a conditional group rules can contain a wider range of content, rulesets but also some, but not all, at-rules.

Whitespace no DOM

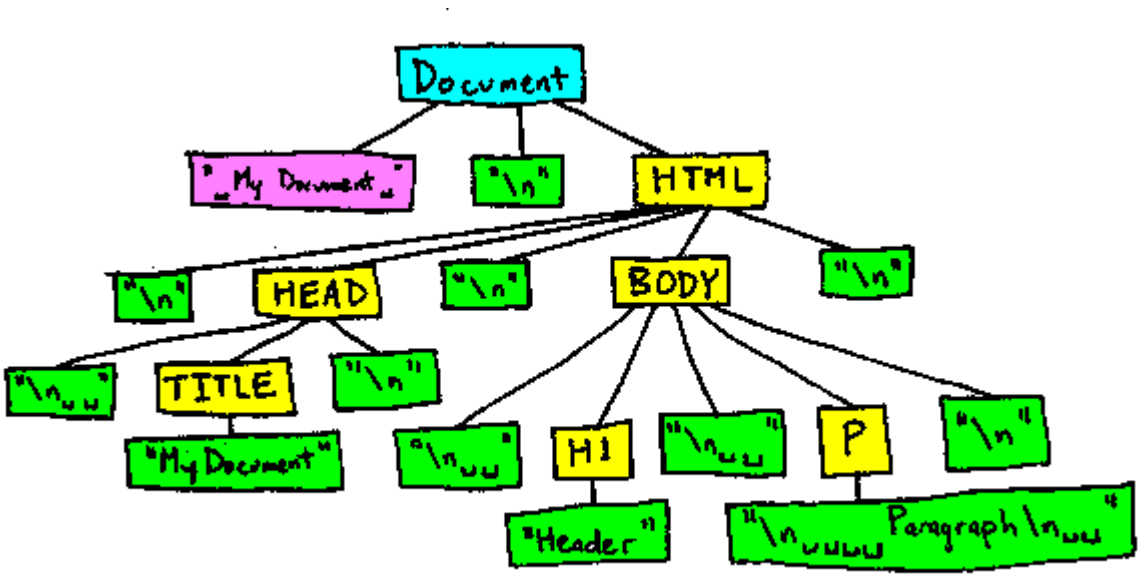
O problema

A presença de espaço branco no DOM pode dificultar a manipulação da árvore de conteúdo de formas imprevisíveis. No Mozilla, todo o espaço branco no conteúdo de texto do documento original é representado no DOM (isso não inclui whitespace entre tags). (Isso é necessário internamente para que o editor possa preservar a formatação de documentos e também que `white-space: pre` irá funcionar em CSS). Isso significa que:

- haverão alguns nós de texto que contêm somente whitespace, e
- alguns nós de texto terão whitespace no início ou no final.

Em outras palavras, a árvore do DOM para o documento seguinte irá parecer como a imagem abaixo (usando "\n" para representar novas linhas):

```
<!-- Meu documento -->
<html>
  <head>
    <title>Meu documento</title>
  </head>
  <body>
    <h1>Cabeçalho</h1>
    <p>Parágrafo</p>
  </body>
</html>
```



Isto pode fazer as coisas um pouco difíceis para qualquer usuário do DOM que quer iterar através do conteúdo, excluindo o whitespace.

Facilitando as coisas

É possível formatar o código como mostrado abaixo para contornar o problema:

```
<!-- Impressão bonita convencional
     com espaços brancos (whitespaces) entre as tags:
 -->
<div>
  <ul>
    <li>Posição 1</li>
    <li>Posição 2</li>
    <li>Posição 3</li>
  </ul>
</div>

<!-- Impressão bonita ajustada ao problema:
 -->
<div>
  <ul>
```

```
    <li>Posição 1</li>
    <li>Posição 2</li>
    <li>Posição 3</li>
  </ul>
</div>
```

O código Javascript abaixo define funções diversas que fazem a manipulação de whitespace no DOM mais fácil.

```
/**
 * Em todo, o whitespace é definido como um dos caracteres
 *  "\t" TAB \u0009
 *  "\n" LF  \u000A
 *  "\r" CR  \u000D
 *  " "  SPC \u0020
 *
 * Isto não usa o "\s" do Javascript porque inclui espaços
 * que não quebram (e alguns outros caracteres).
 */

/**
 * Determina se um conteúdo de texto do nó é inteiramente whitespace.
 *
 * @param nod  Um nó implementando a interface |CharacterData| (por exemplo:
 *             |Text|, |Comment|, ou nó |CDATASection|
 * @return     Verdadeiro se todo conteúdo de texto de |nod| é whitespace,
 *             de outra forma é falso.
 */
function is_all_ws(nod) {
  // Usa as características do ECMA-262 Edition 3 String e RegExp
  return !/[^\t\n\r ]/.test(nod.textContent);
}

/**
 * Determina se um nó deve ser ignorado pela função de iterador.
 *
 * @param nod  Um objeto implementando a interface DOM1 |Node|.
 * @return     verdadeiro se o nó é:
 *                1) Um nó |Text| que é todo whitespace
 *                2) Um nó |Comment|
 *             do contrário é falso.
 */

function is_ignorable(nod) {
  return (
    nod.nodeType == 8 || // Um nó de comentário
    (nod.nodeType == 3 && is_all_ws(nod))
  ); // um nó de texto, todo whitespace
}

/**
 * Versão de |previousSibling| que pula nós que são inteiramente
 * whitespace ou comentários.  (Normalmente |previousSibling| é uma propriedade
 * de todos os nós do DOM que dá o nó irmão, o nó que é
 * um filho do mesmo parente, que ocorre imediatamente antes do
 * nó de referência.)
 *
 * @param sib  O nó de referência.
 * @return     Ou:
 *               1) O irmão mais próximo do |sib| que não é
 *                  ignorável de acordo com |is_ignorable|, ou
 *               2) nulo se tal nó não existe.
 */
function node_before(sib) {
  while ((sib = sib.previousSibling)) {
    if (!is_ignorable(sib)) return sib;
  }
  return null;
}

/**
 * Versão de |nextSibling| que pula nós que são inteiramente
 * whitespace ou comentários.
```

```
 *
 * @param sib  O nó de referência.
 * @return     Ou:
 *               1) O irmão mais próximo do |sib| que não é
 *                  ignorável de acordo com |is_ignorable|, ou
 *               2) nulo se tal nó não existe.
 */
function node_after(sib) {
  while ((sib = sib.nextSibling)) {
    if (!is_ignorable(sib)) return sib;
  }
  return null;
}

/**
 * Versão de  |lastChild| que pula nós que são inteiramente
 * whitespace ou comentários.  (Normalmente |lastChild| é uma propriedade
 * de todos os nós do DOM que dá o último dos nós contidos
 * diretamente no nó de referência.)
 *
 * @param sib  O nó de referência.
 * @return     Ou:
 *               1) O último filho do |sib| que não é
 *                  ignorável de acordo com |is_ignorable|, ou
 *               2) nulo se tal nó não existe.
 */
function last_child(par) {
  var res = par.lastChild;
  while (res) {
    if (!is_ignorable(res)) return res;
    res = res.previousSibling;
  }
  return null;
}

/**
 * Versão de |firstChild| que pula nós que são inteiramente
 * whitespace ou comentários.
 *
 * @param sib  O nó de referência.
 * @return     Ou:
 *               1) O primeiro nó do |sib| que não é
 *                  ignorável de acordo com |is_ignorable|, ou
 *               2) nulo se tal nó não existe.
 */
function first_child(par) {
  var res = par.firstChild;
  while (res) {
    if (!is_ignorable(res)) return res;
    res = res.nextSibling;
  }
  return null;
}

/**
 * Versão de |data| que não inclui whitespace no início
 * e final e normaliza todos whitespaces para um espaço individual.  (Normalmente
 * |data| é uma propriedade de nós de texto que dá o texto do nó.)
 *
 * @param txt  O nó de texto do qual data deve ser retornado
 * @return     Uma string dando os conteúdos de um nó de texto com
 *             whitespace colapsado.
 */
function data_of(txt) {
  var data = txt.textContent;
  // Usa características do ECMA-262 Edition 3 String e RegExp
  data = data.replace(/[\t\n\r ]+/g, " ");
  if (data.charAt(0) == " ") data = data.substring(1, data.length);
  if (data.charAt(data.length - 1) == " ")
    data = data.substring(0, data.length - 1);
  return data;
}
```

Exemplo

O código seguinte demonstra o uso das funções acima. Ele itera através dos filhos de um elemento (dos quais filhos são todos os elementos) para encontrar aquele cujo o texto seja `"Este é o terceiro parágrafo"`, e então muda o atributo da classe e os conteúdos daquele parágrafo.

```
var cur = first_child(document.getElementById("teste"));
while (cur) {
  if (data_of(cur.firstChild) == "Este é o terceiro parágrafo.") {
    cur.className = "mágica";
    cur.firstChild.textContent = "Este é o parágrafo mágico";
  }
  cur = node_after(cur);
}
```

Usando CSS transforms

Mudando coordenadas no espaço da tela o **CSS transforms** permite que a posição do conteúdo afetado seja alterada sem afetar o fluxo de informação da página. Elas são implementadas usando um conjunto de propriedades CSS que permitem alterações lineares em elementos HTML. Essas alterações incluem rotação, inclinações, alteração da escala e tradução tanto em 2D quanto em 3D.

Propriedades de CSS transforms

Duas propriedades principais são usadas para definir o CSS transforms: `transform` and `transform-origin`

`transform-origin`

> Define a posição de origem do elemento. Por padrão esta posição é o canto superior esquerdo do elemento e pode ser movido. É utilizado para muitas alterações, como rotação, alteração de escala e inclinação, que precisam de um ponto especifico como parâmetro.

`transform`

> Define qual alteração será feita no elemento. Pode-se usar mais de um efeito separado-os por um espaço. Tais efeitos serão aplicados um após o outro, na ordem determinada no código.

Exemplo: Rotação

Este exemplo cria um iframe que permite que a página inicial do Google seja utilizada com uma rotação de 90 graus a partir do canto inferior esquerdo

Veja um exemplo funcionando Veja uma foto do exemplo

```
<div style="transform: rotate(90deg); transform-origin: bottom left;">
  <iframe src="http://www.google.com/" width="500" height="600"></iframe>
</div>
```

Example: Inclinando e Movendo

Este exemplo cria um iframe que permite que você use a página inicial do Google inclinada em 10 graus no eixo X.

Veja um exemplo funcionando View uma foto do exemplo

```
<div
  style="transform: skewx(10deg) translatex(150px);
            transform-origin: bottom left;">
  <iframe src="http://www.google.com/" width="600" height="500"></iframe>
</div>
```

Propriedades especificas 3D do CSS

Trabalhar com propriedades CSS em 3D é um pouco mais difícil. Você tem que começar configurando o espaço 3D dando perspectiva a ele, então você tem que configurar como os elementos 2D irão se comportar no espaço.

Configurando uma perspectiva

O primeiro elemento a ser configurado é a perspectiva. É ela a responsável pela sensação de 3D. Quanto mais longe o objeto estiver do usuário, menor ele é.

Quanto será a sensação de encolhimento é definida pela propriedade `perspective`. Quanto menor o valor, mais profundidade terá o objeto.

HTML

O HTML abaixo cria quatro cópias da mesma caixa, com a perspectiva definida em valores diferentes.

```
<table>
  <tbody>
    <tr>
      <th><code>perspective: 250px;</code></th>
      <th><code>perspective: 350px;</code></th>
    </tr>
    <tr>
      <td>
        <div class="container">
          <div class="cube pers250">
            <div class="face front">1</div>
            <div class="face back">2</div>
            <div class="face right">3</div>
            <div class="face left">4</div>
            <div class="face top">5</div>
            <div class="face bottom">6</div>
          </div>
        </div>
      </td>
      <td>
        <div class="container">
          <div class="cube pers350">
            <div class="face front">1</div>
            <div class="face back">2</div>
            <div class="face right">3</div>
            <div class="face left">4</div>
            <div class="face top">5</div>
            <div class="face bottom">6</div>
          </div>
        </div>
      </td>
    </tr>
    <tr>
      <th><code>perspective: 500px;</code></th>
      <th><code>perspective: 650px;</code></th>
    </tr>
    <tr>
      <td>
        <div class="container">
          <div class="cube pers500">
            <div class="face front">1</div>
            <div class="face back">2</div>
            <div class="face right">3</div>
            <div class="face left">4</div>
            <div class="face top">5</div>
            <div class="face bottom">6</div>
          </div>
        </div>
      </td>
      <td>
        <div class="container">
          <div class="cube pers650">
            <div class="face front">1</div>
            <div class="face back">2</div>
            <div class="face right">3</div>
            <div class="face left">4</div>
            <div class="face top">5</div>
            <div class="face bottom">6</div>
          </div>
        </div>
      </td>
    </tr>
  </tbody>
</table>
```

CSS

O CSS estabelece classes que podem ser usadas para definir a perspectiva a diferentes distâncias. Ele também inclui classes para a caixa do recipiente e o próprio cubo, bem como cada uma de suas faces.

```
/* Shorthand classes for different perspective values */
.pers250 {
  perspective: 250px;
}
.pers350 {
  perspective: 350px;
}
.pers500 {
  perspective: 500px;
}
.pers650 {
  perspective: 650px;
}
/* Define the container div, the cube div, and a generic face */
.container {
  width: 200px;
  height: 200px;
  margin: 75px 0 0 75px;
  border: none;
}
.cube {
  width: 100%;
  height: 100%;
  backface-visibility: visible;
  perspective-origin: 150% 150%;
  transform-style: preserve-3d;
}
.face {
  display: block;
  position: absolute;
  width: 100px;
  height: 100px;
  border: none;
  line-height: 100px;
  font-family: sans-serif;
  font-size: 60px;
  color: white;
  text-align: center;
}
/* Define each face based on direction */
.front {
  background: rgba(0, 0, 0, 0.3);
  transform: translateZ(50px);
}
.back {
  background: rgba(0, 255, 0, 1);
  color: black;
  transform: rotateY(180deg) translateZ(50px);
}
.right {
  background: rgba(196, 0, 0, 0.7);
  transform: rotateY(90deg) translateZ(50px);
}
.left {
  background: rgba(0, 0, 196, 0.7);
  transform: rotateY(-90deg) translateZ(50px);
}
.top {
  background: rgba(196, 196, 0, 0.7);
  transform: rotateX(90deg) translateZ(50px);
}
.bottom {
  background: rgba(196, 0, 196, 0.7);
  transform: rotateX(-90deg) translateZ(50px);
}
/* Make the table a little nicer */
th,
p,
```

```
td {
  background-color: #eeeeee;
  padding: 10px;
  font-family: sans-serif;
  text-align: left;
}
```

Resultado

O segundo elemento a ser configurado é a posição do espectador, com a propriedade `perspective-origin`. Por padrão, a perspectiva está centrada no telespectador, o que nem sempre é adequado.

Mudando a origem da perspectiva

Este exemplo mostra cubos com valores populares do `perspective-origin`.

HTML

```
<section>
  <figure>
    <figcaption><code>perspective-origin: top left;</code></figcaption>
    <div class="container">
      <div class="cube potl">
        <div class="face front">1</div>
        <div class="face back">2</div>
        <div class="face right">3</div>
        <div class="face left">4</div>
        <div class="face top">5</div>
        <div class="face bottom">6</div>
      </div>
    </div>
  </figure>

  <figure>
    <figcaption><code>perspective-origin: top;</code></figcaption>
    <div class="container">
      <div class="cube potm">
        <div class="face front">1</div>
        <div class="face back">2</div>
        <div class="face right">3</div>
        <div class="face left">4</div>
        <div class="face top">5</div>
        <div class="face bottom">6</div>
      </div>
    </div>
  </figure>

  <figure>
    <figcaption><code>perspective-origin: top right;</code></figcaption>
    <div class="container">
      <div class="cube potr">
        <div class="face front">1</div>
        <div class="face back">2</div>
        <div class="face right">3</div>
        <div class="face left">4</div>
        <div class="face top">5</div>
        <div class="face bottom">6</div>
      </div>
    </div>
  </figure>

  <figure>
    <figcaption><code>perspective-origin: left;</code></figcaption>
    <div class="container">
      <div class="cube poml">
        <div class="face front">1</div>
        <div class="face back">2</div>
        <div class="face right">3</div>
        <div class="face left">4</div>
        <div class="face top">5</div>
        <div class="face bottom">6</div>
      </div>
    </div>
```

```
</figure>

<figure>
  <figcaption><code>perspective-origin: 50% 50%;</code></figcaption>
  <div class="container">
    <div class="cube pomm">
      <div class="face front">1</div>
      <div class="face back">2</div>
      <div class="face right">3</div>
      <div class="face left">4</div>
      <div class="face top">5</div>
      <div class="face bottom">6</div>
    </div>
  </div>
</figure>

<figure>
  <figcaption><code>perspective-origin: right;</code></figcaption>
  <div class="container">
    <div class="cube pomr">
      <div class="face front">1</div>
      <div class="face back">2</div>
      <div class="face right">3</div>
      <div class="face left">4</div>
      <div class="face top">5</div>
      <div class="face bottom">6</div>
    </div>
  </div>
</figure>

<figure>
  <figcaption><code>perspective-origin: bottom left;</code></figcaption>
  <div class="container">
    <div class="cube pobl">
      <div class="face front">1</div>
      <div class="face back">2</div>
      <div class="face right">3</div>
      <div class="face left">4</div>
      <div class="face top">5</div>
      <div class="face bottom">6</div>
    </div>
  </div>
</figure>

<figure>
  <figcaption><code>perspective-origin: bottom;</code></figcaption>
  <div class="container">
    <div class="cube pobm">
      <div class="face front">1</div>
      <div class="face back">2</div>
      <div class="face right">3</div>
      <div class="face left">4</div>
      <div class="face top">5</div>
      <div class="face bottom">6</div>
    </div>
  </div>
</figure>

<figure>
  <figcaption><code>perspective-origin: bottom right;</code></figcaption>
  <div class="container">
    <div class="cube pobr">
      <div class="face front">1</div>
      <div class="face back">2</div>
      <div class="face right">3</div>
      <div class="face left">4</div>
      <div class="face top">5</div>
      <div class="face bottom">6</div>
    </div>
  </div>
</figure>

<figure>
```

```
    <figcaption><code>perspective-origin: -200% -200%;</code></figcaption>
    <div class="container">
      <div class="cube po200200neg">
        <div class="face front">1</div>
        <div class="face back">2</div>
        <div class="face right">3</div>
        <div class="face left">4</div>
        <div class="face top">5</div>
        <div class="face bottom">6</div>
      </div>
    </div>
  </figure>

  <figure>
    <figcaption><code>perspective-origin: 200% 200%;</code></figcaption>
    <div class="container">
      <div class="cube po200200pos">
        <div class="face front">1</div>
        <div class="face back">2</div>
        <div class="face right">3</div>
        <div class="face left">4</div>
        <div class="face top">5</div>
        <div class="face bottom">6</div>
      </div>
    </div>
  </figure>

  <figure>
    <figcaption><code>perspective-origin: 200% -200%;</code></figcaption>
    <div class="container">
      <div class="cube po200200">
        <div class="face front">1</div>
        <div class="face back">2</div>
        <div class="face right">3</div>
        <div class="face left">4</div>
        <div class="face top">5</div>
        <div class="face bottom">6</div>
      </div>
    </div>
  </figure>
</section>
```

CSS

```
/* perspective-origin values (unique per example) */
.potl {
  perspective-origin: top left;
}
.potm {
  perspective-origin: top;
}
.potr {
  perspective-origin: top right;
}
.poml {
  perspective-origin: left;
}
.pomm {
  perspective-origin: 50% 50%;
}
.pomr {
  perspective-origin: right;
}
.pobl {
  perspective-origin: bottom left;
}
.pobm {
  perspective-origin: bottom;
}
.pobr {
  perspective-origin: bottom right;
}
.po200200neg {
```

```
  perspective-origin: -200% -200%;
}
.po200200pos {
  perspective-origin: 200% 200%;
}
.po200200 {
  perspective-origin: 200% -200%;
}
/* Define the container div, the cube div, and a generic face */
.container {
  width: 100px;
  height: 100px;
  margin: 24px;
  border: none;
}
.cube {
  width: 100%;
  height: 100%;
  backface-visibility: visible;
  perspective: 300px;
  transform-style: preserve-3d;
}
.face {
  display: block;
  position: absolute;
  width: 100px;
  height: 100px;
  border: none;
  line-height: 100px;
  font-family: sans-serif;
  font-size: 60px;
  color: white;
  text-align: center;
}
/* Define each face based on direction */
.front {
  background: rgba(0, 0, 0, 0.3);
  transform: translateZ(50px);
}
.back {
  background: rgba(0, 255, 0, 1);
  color: black;
  transform: rotateY(180deg) translateZ(50px);
}
.right {
  background: rgba(196, 0, 0, 0.7);
  transform: rotateY(90deg) translateZ(50px);
}
.left {
  background: rgba(0, 0, 196, 0.7);
  transform: rotateY(-90deg) translateZ(50px);
}
.top {
  background: rgba(196, 196, 0, 0.7);
  transform: rotateX(90deg) translateZ(50px);
}
.bottom {
  background: rgba(196, 0, 196, 0.7);
  transform: rotateX(-90deg) translateZ(50px);
}
/* Make the layout a little nicer */
section {
  background-color: #eee;
  padding: 10px;
  font-family: sans-serif;
  text-align: left;
  display: grid;
  grid-template-columns: repeat(3, 1fr);
}
```

Resultado

Feito isso, você pode trabalhar com elementos 3D.

Elementos 2D em espaço 3D

Veja também

- Usando deviceorientation com 3D Transforms

Using CSS transitions

CSS transitions provê uma forma de controlar a velocidade de uma animação quando há mudanças de propriedades CSS. Ao invés de uma propriedade entrar em vigor imediatamente, você pode fazer com que as mudanças em uma propriedade ocorram durante um periodo de tempo. Por exemplo, se você mudar a cor de um elemento de branco para preto, a alteração será instantanea. Utilizando transições CSS, as alterações acontecem em uma intervalo de tempo que seguem uma curva de aceleração, e todas podem ser personalizadas

Animações que envolvem transições entre dois estados geralmente são chamadas de *transições implicitas,* como os estados entre o começo e o fim são implicitamente definidos pelo browser

Animações CSS permitem que você decida quais propriedades animar (listando-os explicitamente), quando a animação deve começar (definindo o delay), quão longa a transição será (definindo a duração), e como a transição irá ocorrer (definindo o tempo da função, linear ou rápido no início, lento no final).

Quais propriedade CSS podem transicionadas?

Você mesmo pode definir qual propriedade será transicionada e de qual maneira. E isso permite a criação de transições complexas. Como não faz sentido animar algumas propriedades, existe uma lista finita com propriedades que podem ser transicionadas.

> **Nota:** A lista de propriedades que podem ser animadas sofre alterações a medida que a especificação se desenvolve.

> **Nota:** O valor `automatico` , geralmente é complexo. A especificação recomenda não animar com valores automaticos. Alguns user agents, como aqueles baseados no Genko, implementam esse requisito, como aqueles baseados no WebKit, são menos rigosos. Animações CSS que utlizam o valor `automatico`, podem levar resultados imprevisiveis, dependendo do Browser e da versão, e isso pode ser evitado.

Definindo transições

Transições CSS são controladas usando a propriedade `transition`. Essa é a melhor maneira de configurar uma transição, assim evitamos parâmetros fora de sintonia, o que pode ser muito frustrante desperdiçar tanto tempo debuggando CSS.

Você pode controlar componentes individuais da transição com a seguinte sub propriedade:

(Observe que essas transições fazem um loop infinitamente apenas para o propósito de nossos exemplos; `transitions` apenas visualizam a mudança na propriedade de começo ao fim. Se você precisar de visualizações desse loop, procure na propriedade `animation`.)

`transition-property`

Especifica o nome ou nomes das propriedades CSS que as transições serão aplicadas. Somente as propriedades listadas aqui serão animadas durante a transição; alterações e todas outras propriedades ocorrem instantaneamente.

`transition-duration`

Especifica a duração em que as transições devem ocorrer. Você pode especificar uma única duração que se aplica a todas as propriedades durante a transição ou vários valores para permitir que cada propriedade faça a transição em um período de tempo diferente.

`transition-duration: 0.5s`

```
<div class="parent">
  <div class="box">Lorem</div>
</div>
```

```
.parent {
  width: 250px;
  height: 125px;
}
.box {
  width: 100px;
  height: 100px;
  background-color: red;
  font-size: 20px;
```

```
  left: 0px;
  top: 0px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left
    top transform -webkit-transform color;
  -webkit-transition-duration: 0.5s;
  -webkit-transition-timing-function: ease-in-out;
  transition-property: width height background-color font-size left top
    transform -webkit-transform color;
  transition-duration: 0.5s;
  transition-timing-function: ease-in-out;
}
.box1 {
  transform: rotate(270deg);
  -webkit-transform: rotate(270deg);
  width: 50px;
  height: 50px;
  background-color: blue;
  color: yellow;
  font-size: 18px;
  left: 150px;
  top: 25px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left
    top transform -webkit-transform color;
  -webkit-transition-duration: 0.5s;
  -webkit-transition-timing-function: ease-in-out;
  transition-property: width height background-color font-size left top
    transform -webkit-transformv color;
  transition-duration: 0.5s;
  transition-timing-function: ease-in-out;
}
```

```
function updateTransition() {
  var el = document.querySelector("div.box");

  if (el) {
    el.className = "box1";
  } else {
    el = document.querySelector("div.box1");
    el.className = "box";
  }

  return el;
}

var intervalID = window.setInterval(updateTransition, 7000);
```

`transition-duration: 1s`

```
<div class="parent">
  <div class="box">Lorem</div>
</div>
```

```
.parent {
  width: 250px;
  height: 125px;
}
.box {
  width: 100px;
  height: 100px;
  background-color: red;
  font-size: 20px;
  left: 0px;
  top: 0px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left
    top -webkit-transform color;
  -webkit-transition-duration: 1s;
  -webkit-transition-timing-function: ease-in-out;
  transition-property: width height background-color font-size left top
```

```
    transform color;
  transition-duration: 1s;
  transition-timing-function: ease-in-out;
}
.box1 {
  transform: rotate(270deg);
  -webkit-transform: rotate(270deg);
  width: 50px;
  height: 50px;
  background-color: blue;
  color: yellow;
  font-size: 18px;
  left: 150px;
  top: 25px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left
    top -webkit-transform transform color;
  -webkit-transition-duration: 1s;
  -webkit-transition-timing-function: ease-in-out;
  transition-property: width height background-color font-size left top
    transform -webkit-transform color;
  transition-duration: 1s;
  transition-timing-function: ease-in-out;
}
```

```
function updateTransition() {
  var el = document.querySelector("div.box");

  if (el) {
    el.className = "box1";
  } else {
    el = document.querySelector("div.box1");
    el.className = "box";
  }

  return el;
}

var intervalID = window.setInterval(updateTransition, 7000);
```

```
transition-duration: 2s
```

```
<div class="parent">
  <div class="box">Lorem</div>
</div>
```

```
.parent {
  width: 250px;
  height: 125px;
}
.box {
  width: 100px;
  height: 100px;
  background-color: red;
  font-size: 20px;
  left: 0px;
  top: 0px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left
    top transform -webkit-transform color;
  -webkit-transition-duration: 2s;
  -webkit-transition-timing-function: ease-in-out;
  transition-property: width height background-color font-size left top
    transform -webkit-transform color;
  transition-duration: 2s;
  transition-timing-function: ease-in-out;
}
.box1 {
  transform: rotate(270deg);
  -webkit-transform: rotate(270deg);
  width: 50px;
```

```
  height: 50px;
  background-color: blue;
  color: yellow;
  font-size: 18px;
  left: 150px;
  top: 25px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left
    top transform -webkit-transform color;
  -webkit-transition-duration: 2s;
  -webkit-transition-timing-function: ease-in-out;
  transition-property: width height background-color font-size left top
    transform -webkit-transform color;
  transition-duration: 2s;
  transition-timing-function: ease-in-out;
}
```

```
function updateTransition() {
  var el = document.querySelector("div.box");

  if (el) {
    el.className = "box1";
  } else {
    el = document.querySelector("div.box1");
    el.className = "box";
  }

  return el;
}

var intervalID = window.setInterval(updateTransition, 7000);
```

transition-duration: 4s

```
<div class="parent">
  <div class="box">Lorem</div>
</div>
```

```
.parent {
  width: 250px;
  height: 125px;
}
.box {
  width: 100px;
  height: 100px;
  background-color: red;
  font-size: 20px;
  left: 0px;
  top: 0px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left
    top transform -webkit-transform color;
  -webkit-transition-duration: 4s;
  -webkit-transition-timing-function: ease-in-out;
  transition-property: width height background-color font-size left top
    transform -webkit-transform color;
  transition-duration: 4s;
  transition-timing-function: ease-in-out;
}
.box1 {
  transform: rotate(270deg);
  -webkit-transform: rotate(270deg);
  width: 50px;
  height: 50px;
  background-color: blue;
  color: yellow;
  font-size: 18px;
  left: 150px;
  top: 25px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left
```

```
    top transform -webkit-transform color;
  -webkit-transition-duration: 4s;
  -webkit-transition-timing-function: ease-in-out;
  transition-property: width height background-color font-size left top
    transform -webkit-transform color;
  transition-duration: 4s;
  transition-timing-function: ease-in-out;
}
```

```
function updateTransition() {
  var el = document.querySelector("div.box");

  if (el) {
    el.className = "box1";
  } else {
    el = document.querySelector("div.box1");
    el.className = "box";
  }

  return el;
}

var intervalID = window.setInterval(updateTransition, 7000);
```

`transition-timing-function`

Especifica uma função para definir como os valores intermediários das propriedades são calculados. As timing functions determinam como os valores intermediários da transição são calculados. A maioria das funções de temporização pode ser especificada fornecendo o gráfico da função correspondente, conforme definido por quatro pontos definindo um bezier cúbico. Você também pode escolher o Easing Functions Cheat Sheet.

`transition-timing-function: ease`

```
<div class="parent">
  <div class="box">Lorem</div>
</div>
```

```
.parent {
  width: 250px;
  height: 125px;
}
.box {
  width: 100px;
  height: 100px;
  background-color: red;
  font-size: 20px;
  left: 0px;
  top: 0px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left
    top color;
  -webkit-transition-duration: 2s;
  -webkit-transition-timing-function: ease;
  transition-property: width height background-color font-size left top
    color;
  transition-duration: 2s;
  transition-timing-function: ease;
}
.box1 {
  width: 50px;
  height: 50px;
  background-color: blue;
  color: yellow;
  font-size: 18px;
  left: 150px;
  top: 25px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left
    top color;
  -webkit-transition-duration: 2s;
  -webkit-transition-timing-function: ease;
```

```
  transition-property: width height background-color font-size left top
    color;
  transition-duration: 2s;
  transition-timing-function: ease;
}
```

```
function updateTransition() {
  var el = document.querySelector("div.box");

  if (el) {
    el.className = "box1";
  } else {
    el = document.querySelector("div.box1");
    el.className = "box";
  }

  return el;
}

var intervalID = window.setInterval(updateTransition, 7000);
```

transition-timing-function: linear

```
<div class="parent">
  <div class="box">Lorem</div>
</div>
```

```
.parent {
  width: 250px;
  height: 125px;
}
.box {
  width: 100px;
  height: 100px;
  background-color: red;
  font-size: 20px;
  left: 0px;
  top: 0px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left
    top color;
  -webkit-transition-duration: 2s;
  -webkit-transition-timing-function: linear;
  transition-property: width height background-color font-size left top
    color;
  transition-duration: 2s;
  transition-timing-function: linear;
}
.box1 {
  width: 50px;
  height: 50px;
  background-color: blue;
  color: yellow;
  font-size: 18px;
  left: 150px;
  top: 25px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left
    top color;
  -webkit-transition-duration: 2s;
  -webkit-transition-timing-function: linear;
  transition-property: width height background-color font-size left top
    color;
  transition-duration: 2s;
  transition-timing-function: linear;
}
```

```
function updateTransition() {
  var el = document.querySelector("div.box");

  if (el) {
```

```
    el.className = "box1";
  } else {
    el = document.querySelector("div.box1");
    el.className = "box";
  }

  return el;
}

var intervalID = window.setInterval(updateTransition, 7000);
```

transition-timing-function: step-end

```
<div class="parent">
  <div class="box">Lorem</div>
</div>
```

```
.parent {
  width: 250px;
  height: 125px;
}
.box {
  width: 100px;
  height: 100px;
  background-color: red;
  font-size: 20px;
  left: 0px;
  top: 0px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left
    top color;
  -webkit-transition-duration: 2s;
  -webkit-transition-timing-function: step-end;
  transition-property: width height background-color font-size left top
    color;
  transition-duration: 2s;
  transition-timing-function: step-end;
}
.box1 {
  width: 50px;
  height: 50px;
  background-color: blue;
  color: yellow;
  font-size: 18px;
  left: 150px;
  top: 25px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left
    top color;
  -webkit-transition-duration: 2s;
  -webkit-transition-timing-function: step-end;
  transition-property: width height background-color font-size left top
    color;
  transition-duration: 2s;
  transition-timing-function: step-end;
}
```

```
function updateTransition() {
  var el = document.querySelector("div.box");

  if (el) {
    el.className = "box1";
  } else {
    el = document.querySelector("div.box1");
    el.className = "box";
  }

  return el;
}

var intervalID = window.setInterval(updateTransition, 7000);
```

```
transition-timing-function: steps(4, end)
```

```
<div class="parent">
  <div class="box">Lorem</div>
</div>
```

```
.parent {
  width: 250px;
  height: 125px;
}
.box {
  width: 100px;
  height: 100px;
  background-color: red;
  font-size: 20px;
  left: 0px;
  top: 0px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left
    top color;
  -webkit-transition-duration: 2s;
  -webkit-transition-timing-function: steps(4, end);
  transition-property: width height background-color font-size left top
    color;
  transition-duration: 2s;
  transition-timing-function: steps(4, end);
}
.box1 {
  width: 50px;
  height: 50px;
  background-color: blue;
  color: yellow;
  font-size: 18px;
  left: 150px;
  top: 25px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left
    top color;
  -webkit-transition-duration: 2s;
  -webkit-transition-timing-function: steps(4, end);
  transition-property: width height background-color font-size left top
    color;
  transition-duration: 2s;
  transition-timing-function: steps(4, end);
}
```

```
function updateTransition() {
  var el = document.querySelector("div.box");

  if (el) {
    el.className = "box1";
  } else {
    el = document.querySelector("div.box1");
    el.className = "box";
  }

  return el;
}

var intervalID = window.setInterval(updateTransition, 7000);
```

transition-delay

Define quanto tempo esperar entre o momento que uma propriedade é alterada e a transição começa.

```
transition-delay: 0.5s
```

```
<div class="parent">
  <div class="box">Lorem</div>
</div>
```

```
.parent {
  width: 250px;
  height: 125px;
}

.box {
  width: 100px;
  height: 100px;
  background-color: red;
  font-size: 20px;
  left: 0px;
  top: 0px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left
    top color;
  -webkit-transition-duration: 2s;
  -webkit-transition-delay: 0.5s;
  -webkit-transition-timing-function: linear;
  transition-property: width height background-color font-size left top
    color;
  transition-duration: 2s;
  transition-delay: 0.5s;
  transition-timing-function: linear;
}

.box1 {
  width: 50px;
  height: 50px;
  background-color: blue;
  color: yellow;
  font-size: 18px;
  left: 150px;
  top: 25px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left
    top color;
  -webkit-transition-duration: 2s;
  -webkit-transition-delay: 0.5s;
  -webkit-transition-timing-function: linear;
  transition-property: width height background-color font-size left top
    color;
  transition-duration: 2s;
  transition-delay: 0.5s;
  transition-timing-function: linear;
}
```

```
function updateTransition() {
  var el = document.querySelector("div.box");

  if (el) {
    el.className = "box1";
  } else {
    el = document.querySelector("div.box1");
    el.className = "box";
  }

  return el;
}

var intervalID = window.setInterval(updateTransition, 7000);
```

transition-delay: 1s

```
<div class="parent">
  <div class="box">Lorem</div>
</div>
```

```
.parent {
  width: 250px;
  height: 125px;
}
```

```
.box {
  width: 100px;
  height: 100px;
  background-color: red;
  font-size: 20px;
  left: 0px;
  top: 0px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left
    top color;
  -webkit-transition-duration: 2s;
  -webkit-transition-delay: 1s;
  -webkit-transition-timing-function: linear;
  transition-property: width height background-color font-size left top
    color;
  transition-duration: 2s;
  transition-delay: 1s;
  transition-timing-function: linear;
}

.box1 {
  width: 50px;
  height: 50px;
  background-color: blue;
  color: yellow;
  font-size: 18px;
  left: 150px;
  top: 25px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left
    top color;
  -webkit-transition-duration: 2s;
  -webkit-transition-delay: 1s;
  -webkit-transition-timing-function: linear;
  transition-property: width height background-color font-size left top
    color;
  transition-duration: 2s;
  transition-delay: 1s;
  transition-timing-function: linear;
}
```

```
function updateTransition() {
  var el = document.querySelector("div.box");

  if (el) {
    el.className = "box1";
  } else {
    el = document.querySelector("div.box1");
    el.className = "box";
  }

  return el;
}

var intervalID = window.setInterval(updateTransition, 7000);
```

transition-delay: 2s

```
<div class="parent">
  <div class="box">Lorem</div>
</div>
```

```
.parent {
  width: 250px;
  height: 125px;
}

.box {
  width: 100px;
  height: 100px;
```

```
  background-color: red;
  font-size: 20px;
  left: 0px;
  top: 0px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left
    top color;
  -webkit-transition-duration: 2s;
  -webkit-transition-delay: 2s;
  -webkit-transition-timing-function: linear;
  transition-property: width height background-color font-size left top
    color;
  transition-duration: 2s;
  transition-delay: 2s;
  transition-timing-function: linear;
}

.box1 {
  width: 50px;
  height: 50px;
  background-color: blue;
  color: yellow;
  font-size: 18px;
  left: 150px;
  top: 25px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left
    top color;
  -webkit-transition-duration: 2s;
  -webkit-transition-delay: 2s;
  -webkit-transition-timing-function: linear;
  transition-property: width height background-color font-size left top
    color;
  transition-duration: 2s;
  transition-delay: 2s;
  transition-timing-function: linear;
}
```

```
function updateTransition() {
  var el = document.querySelector("div.box");

  if (el) {
    el.className = "box1";
  } else {
    el = document.querySelector("div.box1");
    el.className = "box";
  }

  return el;
}

var intervalID = window.setInterval(updateTransition, 7000);
```

`transition-delay: 4s`

```
<div class="parent">
  <div class="box">Lorem</div>
</div>
```

```
.parent {
  width: 250px;
  height: 125px;
}

.box {
  width: 100px;
  height: 100px;
  background-color: red;
  font-size: 20px;
  left: 0px;
  top: 0px;
```

```
  position: absolute;
  -webkit-transition-property: width height background-color font-size left
    top color;
  -webkit-transition-duration: 2s;
  -webkit-transition-delay: 4s;
  -webkit-transition-timing-function: ease-in-out;
  transition-property: width height background-color font-size left top
    color;
  transition-duration: 2s;
  transition-delay: 4s;
  transition-timing-function: ease-in-out;
}

.box1 {
  width: 50px;
  height: 50px;
  background-color: blue;
  color: yellow;
  font-size: 18px;
  left: 150px;
  top: 25px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left
    top color;
  -webkit-transition-duration: 2s;
  -webkit-transition-delay: 4s;
  -webkit-transition-timing-function: ease-in-out;
  transition-property: width height background-color font-size left top
    color;
  transition-duration: 2s;
  transition-delay: 4s;
  transition-timing-function: ease-in-out;
}
```

```
function updateTransition() {
  var el = document.querySelector("div.box");

  if (el) {
    el.className = "box1";
  } else {
    el = document.querySelector("div.box1");
    el.className = "box";
  }

  return el;
}

var intervalID = window.setInterval(updateTransition, 7000);
```

A sintaxe do CSS é abreviada da seguinte forma:

```
div {
  transition: <property> <duration> <timing-function> <delay>;
}
```

Exemplos

Exemplo simples

Esse exemplo executa uma transição de com duração de 4 segundos de delay entre o tempo que o usuário passa o mouse sobre o elemento (evento mouseover), e o início do efeito da animação

```
#delay {
  font-size: 14px;
  transition-property: font-size;
  transition-duration: 4s;
  transition-delay: 2s;
}

#delay:hover {
```

```
  font-size: 36px;
}
```

Exemplo de multiplicas animações de propriedades

```
<body>
  <p>
    The box below combines transitions for: width, height, background-color,
    transform. Hover over the box to see these properties animated.
  </p>
  <div class="box">Sample</div>
</body>
```

Conteúdo CSS

```
.box {
  border-style: solid;
  border-width: 1px;
  display: block;
  width: 100px;
  height: 100px;
  background-color: #0000ff;
  -webkit-transition:
    width 2s,
    height 2s,
    background-color 2s,
    -webkit-transform 2s;
  transition:
    width 2s,
    height 2s,
    background-color 2s,
    transform 2s;
}

.box:hover {
  background-color: #ffcccc;
  width: 200px;
  height: 200px;
  -webkit-transform: rotate(180deg);
  transform: rotate(180deg);
}
```

Quando os tamanhos dos valores das listas das propriedades são de tamanhos diferentes

Se alguma propriedade da lista de valores for menor que os outros, esse valor será repetido para que eles correspondam. Por exemplo:

```
div {
  transition-property: opacity, left, top, height;
  transition-duration: 3s, 5s;
}
```

Isso é tratado como se fosse:

```
div {
  transition-property: opacity, left, top, height;
  transition-duration: 3s, 5s, 3s, 5s;
}
```

Da mesma forma, se um valor da propriedade da lista, for maior que a `transition-property`, ela será truncada, então você terá o seguinte CSS:

```
div {
  transition-property: opacity, left;
  transition-duration: 3s, 5s, 2s, 1s;
}
```

Isso pode ser interpretado como:

```
div {
  transition-property: opacity, left;
```

```
  transition-duration: 3s, 5s;
}
```

Usando transições ao destacar menus

Um uso comum de css é para destacar os itens do menu, como hover quando o usúario passa o mouse sobre um elemento. É facíl fácil usar transições CSS para tornar o efeito mais atraente;

Antes de olharmos para os códigos, você poderia dar uma olhada nesse live demo (assumindo que seu browser suporte transições).

Primeiro, nós criamos o menu utilizando apenas HTML

```
<nav>
  <a href="#">Home</a>
  <a href="#">About</a>
  <a href="#">Contact Us</a>
  <a href="#">Links</a>
</nav>
```

Em seguida, nós implementamos o CSS para implementar o visual e aparência do nosso menu. As partes relevantes são serão mostrada aqui:

```
a {
  color: #fff;
  background-color: #333;
  transition: all 1s ease-out;
}

a:hover,
a:focus {
  color: #333;
  background-color: #fff;
}
```

Este CSS estabelece a aparência do menu, com as cores de fundo e texto mudando quando o o ocorre o evento de `:hover` e estado de `:focus`.

Exemplos JavaScript

> **Nota:** Cuidados devem ser tomados ao usar uma transição imediatamente após:
>
> - adding the element to the DOM using `.appendChild()`
> - removing an element's `display: none;` property.
>
> This is treated as if the initial state had never occurred and the element was always in its final state. The easy way to overcome this limitation is to apply a `window.setTimeout()` of a handful of milliseconds before changing the CSS property you intend to transition to.

Using transitions to make JavaScript functionality smooth

Transitions are a great tool to make things look much smoother without having to do anything to your JavaScript functionality. Take the following example.

```
<p>Click anywhere to move the ball</p>
<div id="foo"></div>
```

Using JavaScript you can make the effect of moving the ball to a certain position happen:

```
var f = document.getElementById("foo");
document.addEventListener(
  "click",
  function (ev) {
    f.style.transform = "translateY(" + (ev.clientY - 25) + "px)";
    f.style.transform += "translateX(" + (ev.clientX - 25) + "px)";
  },
  false,
);
```

With CSS you can make it smooth without any extra effort. Simply add a transition to the element and any change will happen smoothly:

```
p {
  padding-left: 60px;
}

#foo {
  border-radius: 50px;
  width: 50px;
  height: 50px;
  background: #c00;
  position: absolute;
  top: 0;
  left: 0;
  transition: transform 1s;
}
```

Você pode brincar um pouco aqui: https://jsfiddle.net/9h261pzo/291/

Detectando o começo e a completude de uma transição

Você pode usar o evento `transitionend` para detectar a uma animação que estava sendo executada chegou ao fim. Esse é o objeto `TransitionEvent`, que possúi duas propriedades adicionadas além do objeto típico

`propertyName`

A string indicando o nome da propriedade CSS cuja transição será completada

`elapsedTime`

A float indicating the number of seconds the transition had been running at the time the event fired. This value isn't affected by the value of `transition-delay`.

As usual, you can use the `addEventListener()` method to monitor for this event:

```
el.addEventListener("transitionend", updateTransition, true);
```

Você detecta o começo de uma transição usando `transitionrun` (antes do delay) e `transitionstart` (depois do delay):

```
el.addEventListener("transitionrun", signalStart, true);
el.addEventListener("transitionstart", signalStart, true);
```

> **Nota:** O evento `transitionend` não dispara se a transição é abortada antes de a transição ser concluída, porque o efeito é feito com `display`: none

Veja Também

- The `TransitionEvent` interface and the `transitionend` event.
- Utilizando animações css

CSS Layout cookbook

O livro de receitas (Cookbook) de layout CSS visa reunir receitas para padrões de layout comuns, coisas que você pode precisar implementar em seus próprios sites. Além de fornecer código que você pode usar como ponto de partida em seus projetos, essas receitas destacam as diferentes maneiras pelas quais as especificações de layout podem ser usadas e as escolhas que você pode fazer como desenvolvedor.

> **Nota:** Se você é novo no layout CSS, então primeiro você pode dar uma olhada em nosso módulo de aprendizado de layout CSS, pois isso lhe dará o básico de que você precisa para usar as receitas aqui.

As Receitas

Recipe	Description	Layout Methods
Media objects	Uma caixa de duas colunas com uma imagem de um lado e um texto descritivo do outro, por ex. um post no facebook ou um tweet.	CSS Grid, `float` fallback, `fit-content` sizing

Recipe	Description	Layout Methods
Columns	Quando escolher layout de múltiplas colunas, flexbox ou grid para suas colunas.	CSS Grid, Multicol, Flexbox
Center an element	Como centralizar um item horizontalmente e verticalmente.	Flexbox, Box Alignment
Sticky footers	Criando um rodapé que fica na parte inferior do contêiner ou viewport quando o conteúdo é menor.	CSS Grid, Flexbox
Split navigation	Um padrão de navegação em que alguns links são visualmente separados dos outros.	Flexbox, `margin`
Breadcrumb navigation	Criando uma lista de links para permitir que o visitante navegue de volta pela hierarquia da página.	Flexbox
List group with badges	Uma lista de itens com um acessório para exibir uma contagem.	Flexbox, Box Alignment
Pagination	Links para páginas de conteúdo (como resultados de pesquisa).	Flexbox, Box Alignment
Card	Um componente de notas/cartão, exibido em uma grade de notas/cartão.	Grid Layout
Grid wrapper	Para alinhar o conteúdo da grade em um wrapper central, ao mesmo tempo em que permite que os itens sejam quebrados.	CSS Grid

Contribuir com uma receita

Como com todo o MDN, gostaríamos que você contribuísse com uma receita no mesmo formato que os mostrados acima. Veja esta página para um modelo e diretrizes para escrever seu próprio exemplo.

Navegação "Migalha de Pão"

A navegação "migalha de pão" ajuda o usuário a enteder sua localização no site, fornecendo uma trilha de navegação de volta à pagina inicial.



Exigências

Os itens tipicamente são exibidos em linha com um separador para indicar uma hierarquia entre páginas individuais.

Receita

> **Nota:** Baixe esse exemplo

Faça escolhas

Esse padrão é apresentado usando um layout flexível simples demonstrando como uma linha de CSS pode nos fornecer nossa navegação. Os separadores são adicionados usando o conteúdo gerado por CSS. Você pode alterá-las para qualquer separador que desejar.

Preocupações com acessibilidade

Eu utilizei os atributos `aria-label` e `aria-current` para ajudar os usuários a entender o que é essa navegação e onde a página atual está na estrutura. Veja os links relacionados para mais informações.

Compatibilidade com navegadores

Os vários métodos de layout têm diferente suporte para o navegador. Veja os gráficos abaixo para obter detalhes sobre o suporte básico para as propriedades usadas.

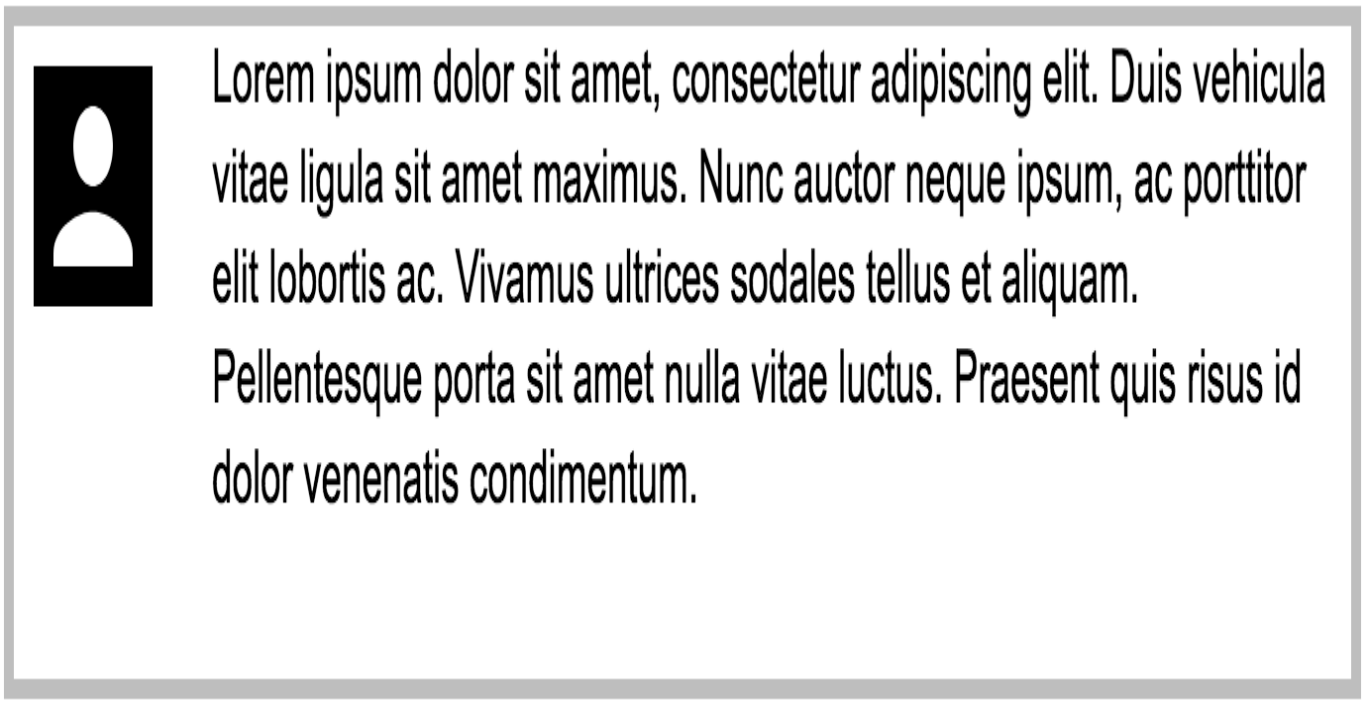
Flexbox

Veja também

- CSS Flexible Box Layout
- Fornecendo uma trilha de "migalha de pão"
- Usando o atributo aria-current

Recipe: Media objects

O *Media Object* (Objeto de Mídia) é um padrão que vemos em toda web. Nomeado por Nicole Sullivan refere-se a uma caixa de duas colunas com uma imagem de um lado e texto descritivo do outro, por ex. uma postagem no facebook ou tweet.



Exigências

O Padrão Media Object precisa de algumas ou todas as seguintes características:

- Empilhados no Celular, duas colunas no Computador.
- A imagem pode estar à esquerda ou à direita.
- A imagem pode ser pequena ou grande.
- Media Objects (Objetos de Mídia) podem ser aninhados.
- O Media Object deve limpar o conteúdo, não importando o tamanho.

A receita

> **Nota:** Download this example

Escolhas feitas

Eu escolhi usar o Layout de Grade para o Media Object (Objeto de Layout), pois ele permite controlar o layout em duas dimensões quando preciso. Isso significa que, quando temos um rodapé, com conteúdo curto acima, o rodapé pode ser empurrado para a parte inferior do Media Object (Objeto de Layout).

Outro motivo para usar o Layout de Gride é para que eu possa usar `fit-content` para o tamanho da faixa de imagem. Usando `fit-content` com um tamanho máximo de 200 pixels, quando temos uma imagem pequena como o ícone, a faixa fica tão grande quanto o tamanho dessa imagem — o tamanho `max-content`. Se a imagem for maior, a faixa para de crescer em 200 pixels e, como a imagem possui `max-width` de 100% aplicado, ele diminui para que continue a caber dentro da coluna.

Usando `grid-template-areas` para alcançar o layout, eu posso ver o padrão no CSS. Eu defino minha grade quando tivermos uma largura máxima de 500 pixels, em dispositivos menores o Media Object empilha.

Uma opção para o padrão é girá-lo para mudar a imagem para o outro lado — isso é feito adicionando a classe `media-flip`, que define um modelo de grade invertida, fazendo com que o layout seja espelhado.

Quando aninhamos um media object (Objeto de Layout) dentro do outro, precisamons colocá-lo na segunda faixa no layout regular, e na primeira faixa quando invertida.

Fallbacks (Alternativas, Plano B)

Existem várias alternativas possíveis para esse padrão, dependendo dos navegadores que você deseja oferecer suporte. Um bom exemplo seria flutuar a imagem para a esquerda e adicionar uma correção clara à caixa para garantir que ela contivesse as flutuações.

> **Nota:** Baixe este exemplo

Depois que os elementos flutuantes se tornam itens da grade, o flutuador não se aplica mais; portanto, você não precisa fazer nada de especial para limpá-lo.

O que você precisará fazer é remover as margens aplicadas ao item e as larguras que não precisamos em um contexto de grade (nós temos o `gap` propriedade para controlá-lo em grades, e a faixa assume o controle do dimensionamento).

Recursos relevantes no MDN

- Layout Grade CSS
- Aprimoramento progressivo e Layout de Grid
- Usando áreas de modelo de Grade
- Conteúdo Adequado
- Áreas de Modelo de Grade

Compatibilidade com navegadores

Os vários métodos de layout tem suporte diferente para os navegadores. Veja os gráficos abaixo para obter detalhes sobre o suporte básico para as propriedades usadas.

Áreas de Modelo de Grade

Flutuador

Referência de CSS

Use esta **referência de CSS** para navegar por um índice alfabético das propriedades padrão do CSS, pseudo-classes, pseudo-elementos, tipos de dados e @-rules.

Esta referência lista não somente as propriedades do CSS1 e CSS2.1, mas também referências para qualquer propriedade, conceito padronizado ou estabilizado do CSS3.

Veja também Extensões CSS Mozilla para propriedades específicas do Gecko prefixadas com `-moz` e Extensões CSS do WebKit para propriedades específicas do WebKit. Veja Visão geral de propriedades CSS prefixadas por distribuidor por Peter Beverloo para todas as propriedades prefixadas.

- -
- A
- B
- C
- D
- E
- F
- G
- H
- I
- J
- K
- L
- M
- N
- O
- P
- Q
- R

- S
- T
- U
- V
- W
- X
- Y
- Z

-

- `:-moz-broken` CSS pseudo-class
- `::-moz-color-swatch` CSS pseudo-element
- `:-moz-drag-over` CSS pseudo-class
- `:-moz-first-node` CSS pseudo-class
- `::-moz-focus-inner` CSS pseudo-element
- `:-moz-handler-blocked` CSS pseudo-class
- `:-moz-handler-crashed` CSS pseudo-class
- `:-moz-handler-disabled` CSS pseudo-class
- `:-moz-last-node` CSS pseudo-class
- `::-moz-list-bullet` CSS pseudo-element
- `::-moz-list-number` CSS pseudo-element
- `:-moz-loading` CSS pseudo-class
- `:-moz-locale-dir(ltr)` CSS pseudo-class
- `:-moz-locale-dir(rtl)` CSS pseudo-class
- `::-moz-meter-bar` CSS pseudo-element
- `:-moz-only-whitespace` CSS pseudo-class
- `::-moz-progress-bar` CSS pseudo-element
- `::-moz-range-progress` CSS pseudo-element
- `::-moz-range-thumb` CSS pseudo-element
- `::-moz-range-track` CSS pseudo-element
- `:-moz-submit-invalid` CSS pseudo-class
- `:-moz-suppressed` CSS pseudo-class
- `:-moz-user-disabled` CSS pseudo-class
- `:-moz-window-inactive` CSS pseudo-class
- `::-webkit-inner-spin-button` CSS pseudo-element
- `::-webkit-meter-bar` CSS pseudo-element
- `::-webkit-meter-even-less-good-value` CSS pseudo-element
- `::-webkit-meter-inner-element` CSS pseudo-element
- `::-webkit-meter-optimum-value` CSS pseudo-element
- `::-webkit-meter-suboptimum-value` CSS pseudo-element
- `::-webkit-progress-bar` CSS pseudo-element
- `::-webkit-progress-inner-element` CSS pseudo-element
- `::-webkit-progress-value` CSS pseudo-element
- `::-webkit-scrollbar` CSS pseudo-element
- `::-webkit-search-cancel-button` CSS pseudo-element
- `::-webkit-search-results-button` CSS pseudo-element
- `::-webkit-slider-runnable-track` CSS pseudo-element
- `::-webkit-slider-thumb` CSS pseudo-element

A

- Attribute selectors
- `abs()` CSS function
- `<absolute-size>` CSS type
- `accent-color` CSS property
- `acos()` CSS function
- `:active` CSS pseudo-class
- `:active-view-transition` CSS pseudo-class
- `:active-view-transition-type()` CSS pseudo-class
- `additive-symbols` CSS at-rule descriptor (`@counter-style`)
- `::after` CSS pseudo-element

- `align-content` CSS property
- `align-items` CSS property
- `align-self` CSS property
- `alignment-baseline` CSS property
- `all` CSS property
- `alpha()` CSS function
- `<alpha-value>` CSS type
- `anchor()` CSS function
- `anchor-name` CSS property
- `anchor-scope` CSS property
- `anchor-size()` CSS function
- `<angle-percentage>` CSS type
- `<angle>` CSS type
- `animation` CSS property
- <animation-action>
- `animation-composition` CSS property
- `animation-delay` CSS property
- `animation-direction` CSS property
- `animation-duration` CSS property
- `animation-fill-mode` CSS property
- `animation-iteration-count` CSS property
- `animation-name` CSS property
- `animation-play-state` CSS property
- `animation-range` CSS property
- `animation-range-end` CSS property
- `animation-range-start` CSS property
- `animation-timeline` CSS property
- `animation-timing-function` CSS property
- animation-trigger CSS property
- `:any-link` CSS pseudo-class
- `appearance` CSS property
- `ascent-override` CSS at-rule descriptor (`@font-face`)
- `asin()` CSS function
- `aspect-ratio` CSS property
- `atan()` CSS function
- `atan2()` CSS function
- `attr()` CSS function
- `:autofill` CSS pseudo-class
- `<axis>` CSS type

B

- `::backdrop` CSS pseudo-element
- `backdrop-filter` CSS property
- `backface-visibility` CSS property
- `background` CSS property
- `background-attachment` CSS property
- `background-blend-mode` CSS property
- `background-clip` CSS property
- `background-color` CSS property
- `background-image` CSS property
- `background-origin` CSS property
- `background-position` CSS property
- `background-position-x` CSS property
- `background-position-y` CSS property
- `background-repeat` CSS property
- `background-repeat-x` CSS property
- `background-repeat-y` CSS property
- `background-size` CSS property
- `base-palette` CSS at-rule descriptor (`@font-palette-values`)
- `<baseline-position>` CSS type

- `baseline-shift` CSS property
- `baseline-source` CSS property
- `<basic-shape>` CSS type
- `::before` CSS pseudo-element
- `:blank` CSS pseudo-class
- `<blend-mode>` CSS type
- `block-size` CSS property
- `blur()` CSS function
- `border` CSS property
- `-webkit-border-before` CSS property
- `border-block` CSS property
- `border-block-color` CSS property
- `border-block-end` CSS property
- `border-block-end-color` CSS property
- `border-block-end-style` CSS property
- `border-block-end-width` CSS property
- `border-block-start` CSS property
- `border-block-start-color` CSS property
- `border-block-start-style` CSS property
- `border-block-start-width` CSS property
- `border-block-style` CSS property
- `border-block-width` CSS property
- `border-bottom` CSS property
- `border-bottom-color` CSS property
- `border-bottom-left-radius` CSS property
- `border-bottom-right-radius` CSS property
- `border-bottom-style` CSS property
- `border-bottom-width` CSS property
- `border-collapse` CSS property
- `border-color` CSS property
- `border-end-end-radius` CSS property
- `border-end-start-radius` CSS property
- `border-image` CSS property
- `border-image-outset` CSS property
- `border-image-repeat` CSS property
- `border-image-slice` CSS property
- `border-image-source` CSS property
- `border-image-width` CSS property
- `border-inline` CSS property
- `border-inline-color` CSS property
- `border-inline-end` CSS property
- `border-inline-end-color` CSS property
- `border-inline-end-style` CSS property
- `border-inline-end-width` CSS property
- `border-inline-start` CSS property
- `border-inline-start-color` CSS property
- `border-inline-start-style` CSS property
- `border-inline-start-width` CSS property
- `border-inline-style` CSS property
- `border-inline-width` CSS property
- `border-left` CSS property
- `border-left-color` CSS property
- `border-left-style` CSS property
- `border-left-width` CSS property
- `border-radius` CSS property
- `border-right` CSS property
- `border-right-color` CSS property
- `border-right-style` CSS property
- `border-right-width` CSS property
- `border-shape` CSS property

- `border-spacing` CSS property
- `border-start-end-radius` CSS property
- `border-start-start-radius` CSS property
- `border-style` CSS property
- `border-top` CSS property
- `border-top-color` CSS property
- `border-top-left-radius` CSS property
- `border-top-right-radius` CSS property
- `border-top-style` CSS property
- `border-top-width` CSS property
- `border-width` CSS property
- `bottom` CSS property
- `box-align` CSS property
- `box-decoration-break` CSS property
- `box-direction` CSS property
- `<box-edge>` CSS type
- `box-flex` CSS property
- `box-flex-group` CSS property
- `box-lines` CSS property
- `box-ordinal-group` CSS property
- `box-orient` CSS property
- `box-pack` CSS property
- `-webkit-box-reflect` CSS property
- `box-shadow` CSS property
- `box-sizing` CSS property
- `break-after` CSS property
- `break-before` CSS property
- `break-inside` CSS property
- `brightness()` CSS function
- `:buffering` CSS pseudo-class

C

- Class selectors
- Custom properties (--*): CSS variables
- `calc()` CSS function
- `<calc-keyword>` CSS type
- `calc-size()` CSS function
- `<calc-sum>` CSS type
- `caption-side` CSS property
- `caret` CSS property
- `caret-animation` CSS property
- `caret-color` CSS property
- `caret-shape` CSS property
- `@charset` CSS at-rule
- `:checked` CSS pseudo-class
- `::checkmark` CSS pseudo-element
- `circle()` CSS function
- `clamp()` CSS function
- `clear` CSS property
- `clip` CSS property
- `clip-path` CSS property
- `clip-rule` CSS property
- `color` CSS property
- `color()` CSS function
- `color-interpolation` CSS property
- `color-interpolation-filters` CSS property
- `<color-interpolation-method>` CSS type
- `color-mix()` CSS function
- `@color-profile` CSS at-rule
- `color-scheme` CSS property

- `<color>` CSS type
- `::column` CSS pseudo-element
- `column-count` CSS property
- `column-fill` CSS property
- `column-gap` CSS property
- `column-height` CSS property
- `column-rule` CSS property
- `column-rule-break` CSS property
- `column-rule-color` CSS property
- `column-rule-inset-cap` CSS property
- `column-rule-inset-cap-end` CSS property
- `column-rule-inset-cap-start` CSS property
- `column-rule-inset-junction` CSS property
- `column-rule-inset-junction-end` CSS property
- `column-rule-style` CSS property
- `column-rule-visibility-items` CSS property
- `column-rule-width` CSS property
- `column-span` CSS property
- `column-width` CSS property
- `column-wrap` CSS property
- `columns` CSS property
- `conic-gradient()` CSS function
- `contain` CSS property
- `contain-intrinsic-block-size` CSS property
- `contain-intrinsic-height` CSS property
- `contain-intrinsic-inline-size` CSS property
- `contain-intrinsic-size` CSS property
- `contain-intrinsic-width` CSS property
- `@container` CSS at-rule
- `container` CSS property
- `container-name` CSS property
- `container-type` CSS property
- `content` CSS property
- `<content-distribution>` CSS type
- `<content-position>` CSS type
- `content-visibility` CSS property
- `contrast()` CSS function
- `contrast-color()` CSS function
- `corner-block-end-shape` CSS property
- `corner-block-start-shape` CSS property
- `corner-bottom-left-shape` CSS property
- `corner-bottom-right-shape` CSS property
- `corner-bottom-shape` CSS property
- `corner-end-end-shape` CSS property
- `corner-end-start-shape` CSS property
- `corner-inline-end-shape` CSS property
- `corner-inline-start-shape` CSS property
- `corner-left-shape` CSS property
- `corner-right-shape` CSS property
- `corner-shape` CSS property
- `<corner-shape-value>` CSS type
- `corner-start-end-shape` CSS property
- `corner-start-start-shape` CSS property
- `corner-top-left-shape` CSS property
- `corner-top-right-shape` CSS property
- `corner-top-shape` CSS property
- `cos()` CSS function
- `counter()` CSS function
- `counter-increment` CSS property
- `counter-reset` CSS property

- `counter-set` CSS property
- `@counter-style` CSS at-rule
- `counters()` CSS function
- `cross-fade()` CSS function
- `cubic-bezier()` CSS function
- `::cue` CSS pseudo-element
- `:current` CSS pseudo-class
- `cursor` CSS property
- `<custom-ident>` CSS type
- `@custom-media` CSS at-rule
- `cx` CSS property
- `cy` CSS property

D

- `d` CSS property
- `<dashed-function>` CSS type
- `<dashed-ident>` CSS type
- `:default` CSS pseudo-class
- `:defined` CSS pseudo-class
- `descent-override` CSS at-rule descriptor (`@font-face`)
- `::details-content` CSS pseudo-element
- `device-cmyk()` CSS function
- `<dimension>` CSS type
- `:dir()` CSS pseudo-class
- `direction` CSS property
- `:disabled` CSS pseudo-class
- `display` CSS property
- `<display-box>` CSS type
- `<display-inside>` CSS type
- `<display-internal>` CSS type
- `<display-legacy>` CSS type
- `<display-listitem>` CSS type
- `<display-outside>` CSS type
- `@document` CSS at-rule
- `dominant-baseline` CSS property
- `drop-shadow()` CSS function
- `dynamic-range-limit` CSS property
- `dynamic-range-limit-mix()` CSS function

E

- `<easing-function>` CSS type
- `element()` CSS function
- `ellipse()` CSS function
- `:empty` CSS pseudo-class
- `empty-cells` CSS property
- `:enabled` CSS pseudo-class
- `env()` CSS function
- `exp()` CSS function

F

- `fallback` CSS at-rule descriptor (`@counter-style`)
- `field-sizing` CSS property
- `::file-selector-button` CSS pseudo-element
- `fill` CSS property
- `fill-opacity` CSS property
- `fill-rule` CSS property
- `filter` CSS property
- `<filter-function>` CSS type
- `:first` CSS pseudo-class

- `:first-child` CSS pseudo-class
- `::first-letter` CSS pseudo-element
- `::first-line` CSS pseudo-element
- `:first-of-type` CSS pseudo-class
- `fit-content` CSS keyword
- `fit-content()` CSS function
- `flex` CSS property
- `flex-basis` CSS property
- `flex-direction` CSS property
- `flex-flow` CSS property
- `flex-grow` CSS property
- `flex-line-count` CSS property
- `flex-shrink` CSS property
- `flex-wrap` CSS property
- `<flex>` CSS type
- `float` CSS property
- `-moz-float-edge` CSS property
- `flood-color` CSS property
- `flood-opacity` CSS property
- `:focus` CSS pseudo-class
- `:focus-visible` CSS pseudo-class
- `:focus-within` CSS pseudo-class
- `font` CSS property
- `font-display` CSS at-rule descriptor (`@font-face`)
- `font-display` CSS at-rule descriptor (`@font-feature-values`)
- `@font-face` CSS at-rule
- `font-family` CSS at-rule descriptor (`@font-face`)
- `font-family` CSS at-rule descriptor (`@font-palette-values`)
- `font-family` CSS property
- `font-feature-settings` CSS at-rule descriptor (`@font-face`)
- `font-feature-settings` CSS property
- `@font-feature-values` CSS at-rule
- `font-kerning` CSS property
- `font-language-override` CSS property
- `font-optical-sizing` CSS property
- `font-palette` CSS property
- `@font-palette-values` CSS at-rule
- `font-size` CSS property
- `font-size-adjust` CSS property
- `font-smooth` CSS property
- `font-stretch` CSS at-rule descriptor (`@font-face`)
- `font-stretch` CSS property
- `font-style` CSS at-rule descriptor (`@font-face`)
- `font-style` CSS property
- `font-synthesis` CSS property
- `font-synthesis-position` CSS property
- `font-synthesis-small-caps` CSS property
- `font-synthesis-style` CSS property
- `font-synthesis-weight` CSS property
- `font-variant` CSS property
- `font-variant-alternates` CSS property
- `font-variant-caps` CSS property
- `font-variant-east-asian` CSS property
- `font-variant-emoji` CSS property
- `font-variant-ligatures` CSS property
- `font-variant-numeric` CSS property
- `font-variant-position` CSS property
- `font-variation-settings` CSS at-rule descriptor (`@font-face`)
- `font-variation-settings` CSS property
- `font-weight` CSS at-rule descriptor (`@font-face`)

- font-weight CSS property
- font-width CSS at-rule descriptor (@font-face)
- font-width CSS property
- -moz-force-broken-image-icon CSS property
- forced-color-adjust CSS property
- frame-sizing CSS property
- <frequency-percentage> CSS type
- <frequency> CSS type
- :fullscreen CSS pseudo-class
- @function CSS at-rule
- :future CSS pseudo-class

G

- gap CSS property
- <generic-family> CSS type
- <gradient> CSS type
- ::grammar-error CSS pseudo-element
- grayscale() CSS function
- grid CSS property
- grid-area CSS property
- grid-auto-columns CSS property
- grid-auto-flow CSS property
- grid-auto-rows CSS property
- grid-column CSS property
- grid-column-end CSS property
- grid-column-start CSS property
- grid-row CSS property
- grid-row-end CSS property
- grid-row-start CSS property
- grid-template CSS property
- grid-template-areas CSS property
- grid-template-columns CSS property
- grid-template-rows CSS property

H

- hanging-punctuation CSS property
- :has() CSS pseudo-class
- :has-slotted CSS pseudo-class
- :heading CSS pseudo-class
- :heading() CSS pseudo-class
- height CSS property
- <hex-color> CSS type
- ::highlight() CSS pseudo-element
- :host CSS pseudo-class
- :host() CSS pseudo-class
- :host-context() CSS pseudo-class
- :hover CSS pseudo-class
- hsl() CSS function
- <hue-interpolation-method> CSS type
- hue-rotate() CSS function
- <hue> CSS type
- hwb() CSS function
- hyphenate-character CSS property
- hyphenate-limit-chars CSS property
- hyphens CSS property
- hypot() CSS function

I

- ID selectors

- `<ident>` CSS type
- `if()` CSS function
- `image()` CSS function
- `image-orientation` CSS property
- `-moz-image-rect()` CSS function
- `image-rendering` CSS property
- `image-resolution` CSS property
- `image-set()` CSS function
- `<image>` CSS type
- `@import` CSS at-rule
- `!important` CSS keyword
- `:in-range` CSS pseudo-class
- `:indeterminate` CSS pseudo-class
- `inherit` CSS keyword
- `inherits` CSS at-rule descriptor (`@property`)
- `initial` CSS keyword
- `initial-letter` CSS property
- `initial-value` CSS at-rule descriptor (`@property`)
- `inline-size` CSS property
- `inset` CSS property
- `inset()` CSS function
- `inset-block` CSS property
- `inset-block-end` CSS property
- `inset-block-start` CSS property
- `inset-inline` CSS property
- `inset-inline-end` CSS property
- `inset-inline-start` CSS property
- `<integer>` CSS type
- `interactivity` CSS property
- `interest-delay` CSS property
- `interest-delay-end` CSS property
- `interest-delay-start` CSS property
- `:interest-source` CSS pseudo-class
- `:interest-target` CSS pseudo-class
- `interpolate-size` CSS property
- `:invalid` CSS pseudo-class
- `invert()` CSS function
- `:is()` CSS pseudo-class
- `isolation` CSS property

J

- `justify-content` CSS property
- `justify-items` CSS property
- `justify-self` CSS property

K

- Keyframe selectors
- `@keyframes` CSS at-rule

L

- `lab()` CSS function
- `:lang()` CSS pseudo-class
- `:last-child` CSS pseudo-class
- `:last-of-type` CSS pseudo-class
- `@layer` CSS at-rule
- `layer()` CSS function
- `lch()` CSS function
- `left` CSS property
- `:left` CSS pseudo-class

- `<length-percentage>` CSS type
- `<length>` CSS type
- `letter-spacing` CSS property
- `light-dark()` CSS function
- `lighting-color` CSS property
- `line-break` CSS property
- `line-clamp` CSS property
- `line-gap-override` CSS at-rule descriptor (`@font-face`)
- `line-height` CSS property
- `line-height-step` CSS property
- `<line-style>` CSS type
- `<line-width>` CSS type
- `linear()` CSS function
- `linear-gradient()` CSS function
- `:link` CSS pseudo-class
- `link-parameters` CSS property
- `list-style` CSS property
- `list-style-image` CSS property
- `list-style-position` CSS property
- `list-style-type` CSS property
- `:local-link` CSS pseudo-class
- `log()` CSS function

M

- `margin` CSS property
- `margin-block` CSS property
- `margin-block-end` CSS property
- `margin-block-start` CSS property
- `margin-bottom` CSS property
- `margin-inline` CSS property
- `margin-inline-end` CSS property
- `margin-inline-start` CSS property
- `margin-left` CSS property
- `margin-right` CSS property
- `margin-top` CSS property
- `margin-trim` CSS property
- `marker` CSS property
- `::marker` CSS pseudo-element
- `marker-end` CSS property
- `marker-mid` CSS property
- `marker-start` CSS property
- `mask` CSS property
- `mask-border` CSS property
- `mask-border-mode` CSS property
- `mask-border-outset` CSS property
- `mask-border-repeat` CSS property
- `mask-border-slice` CSS property
- `mask-border-source` CSS property
- `mask-border-width` CSS property
- `-webkit-mask-box-image` CSS property
- `mask-clip` CSS property
- `-webkit-mask-composite` CSS property
- `mask-composite` CSS property
- `mask-image` CSS property
- `mask-mode` CSS property
- `mask-origin` CSS property
- `mask-position` CSS property
- `-webkit-mask-position-x` CSS property
- `-webkit-mask-position-y` CSS property
- `mask-repeat` CSS property

- `-webkit-mask-repeat-x` CSS property
- `-webkit-mask-repeat-y` CSS property
- `mask-size` CSS property
- `mask-type` CSS property
- `math-depth` CSS property
- `math-shift` CSS property
- `math-style` CSS property
- `matrix()` CSS function
- `matrix3d()` CSS function
- `max()` CSS function
- `max-block-size` CSS property
- `max-content` CSS keyword
- `max-height` CSS property
- `max-inline-size` CSS property
- `max-width` CSS property
- `@media` CSS at-rule
- `min()` CSS function
- `min-block-size` CSS property
- `min-content` CSS keyword
- `min-height` CSS property
- `min-inline-size` CSS property
- `min-width` CSS property
- `minmax()` CSS function
- `mix-blend-mode` CSS property
- `mod()` CSS function
- `:modal` CSS pseudo-class
- `:muted` CSS pseudo-class

N

- Namespace separator
- `<named-color>` CSS type
- `@namespace` CSS at-rule
- `negative` CSS at-rule descriptor (`@counter-style`)
- & nesting selector
- `:not()` CSS pseudo-class
- `:nth-child()` CSS pseudo-class
- `:nth-last-child()` CSS pseudo-class
- `:nth-last-of-type()` CSS pseudo-class
- `:nth-of-type()` CSS pseudo-class
- `<number>` CSS type

O

- `object-fit` CSS property
- `object-position` CSS property
- `object-view-box` CSS property
- `offset` CSS property
- `offset-anchor` CSS property
- `offset-distance` CSS property
- `offset-path` CSS property
- `offset-position` CSS property
- `offset-rotate` CSS property
- `oklab()` CSS function
- `oklch()` CSS function
- `:only-child` CSS pseudo-class
- `:only-of-type` CSS pseudo-class
- `opacity` CSS property
- `opacity()` CSS function
- `:open` CSS pseudo-class
- `:optional` CSS pseudo-class

- order CSS property
- -moz-orient CSS property
- orphans CSS property
- :out-of-range CSS pseudo-class
- outline CSS property
- outline-color CSS property
- outline-offset CSS property
- outline-style CSS property
- outline-width CSS property
- overflow CSS property
- overflow-anchor CSS property
- overflow-block CSS property
- overflow-clip-margin CSS property
- overflow-inline CSS property
- <overflow-position> CSS type
- overflow-wrap CSS property
- overflow-x CSS property
- overflow-y CSS property
- <overflow> CSS type
- overlay CSS property
- override-colors CSS at-rule descriptor (@font-palette-values)
- overscroll-behavior CSS property
- overscroll-behavior-block CSS property
- overscroll-behavior-inline CSS property
- overscroll-behavior-x CSS property
- overscroll-behavior-y CSS property

P

- pad CSS at-rule descriptor (@counter-style)
- padding CSS property
- padding-block CSS property
- padding-block-end CSS property
- padding-block-start CSS property
- padding-bottom CSS property
- padding-inline CSS property
- padding-inline-end CSS property
- padding-inline-start CSS property
- padding-left CSS property
- padding-right CSS property
- padding-top CSS property
- @page CSS at-rule
- page CSS property
- page-break-after CSS property
- page-break-before CSS property
- page-break-inside CSS property
- page-orientation CSS at-rule descriptor (@page)
- paint() CSS function
- paint-order CSS property
- palette-mix() CSS function
- param() CSS function
- ::part() CSS pseudo-element
- :past CSS pseudo-class
- path() CSS function
- path-length CSS property
- :paused CSS pseudo-class
- <percentage> CSS type
- perspective CSS property
- perspective() CSS function
- perspective-origin CSS property
- ::picker() CSS pseudo-element

- `::picker-icon` CSS pseudo-element
- `:picture-in-picture` CSS pseudo-class
- `place-content` CSS property
- `place-items` CSS property
- `place-self` CSS property
- `::placeholder` CSS pseudo-element
- `:placeholder-shown` CSS pseudo-class
- `:playing` CSS pseudo-class
- `pointer-events` CSS property
- `polygon()` CSS function
- `:popover-open` CSS pseudo-class
- `position` CSS property
- `position-anchor` CSS property
- `position-area` CSS property
- `<position-area>` CSS type
- `@position-try` CSS at-rule
- `position-try` CSS property
- `position-try-fallbacks` CSS property
- `position-try-order` CSS property
- `position-visibility` CSS property
- `<position>` CSS type
- `pow()` CSS function
- `prefix` CSS at-rule descriptor (`@counter-style`)
- `print-color-adjust` CSS property
- `progress()` CSS function
- `@property` CSS at-rule

Q

- `quotes` CSS property

R

- `r` CSS property
- `radial-gradient()` CSS function
- `random()` CSS function
- `range` CSS at-rule descriptor (`@counter-style`)
- `<ratio>` CSS type
- `ray()` CSS function
- `:read-only` CSS pseudo-class
- `:read-write` CSS pseudo-class
- `reading-flow` CSS property
- `reading-order` CSS property
- `rect()` CSS function
- `<relative-size>` CSS type
- `rem()` CSS function
- `repeat()` CSS function
- `repeating-conic-gradient()` CSS function
- `repeating-linear-gradient()` CSS function
- `repeating-radial-gradient()` CSS function
- `:required` CSS pseudo-class
- `resize` CSS property
- `<resolution>` CSS type
- `revert` CSS keyword
- `revert-layer` CSS keyword
- `revert-rule` CSS keyword
- `rgb()` CSS function
- `right` CSS property
- `:right` CSS pseudo-class
- `:root` CSS pseudo-class
- `rotate` CSS property

- `rotate()` CSS function
- `rotate3d()` CSS function
- `rotateX()` CSS function
- `rotateY()` CSS function
- `rotateZ()` CSS function
- `round()` CSS function
- `row-gap` CSS property
- `row-rule` CSS property
- `row-rule-break` CSS property
- `row-rule-color` CSS property
- `row-rule-inset-cap` CSS property
- `row-rule-inset-cap-end` CSS property
- `row-rule-inset-cap-start` CSS property
- `row-rule-inset-junction` CSS property
- `row-rule-style` CSS property
- `row-rule-visibility-items` CSS property
- `row-rule-width` CSS property
- `ruby-align` CSS property
- `ruby-overhang` CSS property
- `ruby-position` CSS property
- `rule` CSS property
- `rule-break` CSS property
- `rule-color` CSS property
- `rule-inset-cap` CSS property
- `rule-inset-junction` CSS property
- `<rule-list>` CSS type
- `rule-overlap` CSS property
- `rule-style` CSS property
- `rule-visibility-items` CSS property
- `rule-width` CSS property
- `rx` CSS property
- `ry` CSS property

S

- Selector list
- `saturate()` CSS function
- `scale` CSS property
- `scale()` CSS function
- `scale3d()` CSS function
- `scaleX()` CSS function
- `scaleY()` CSS function
- `scaleZ()` CSS function
- `@scope` CSS at-rule
- `:scope` CSS pseudo-class
- `scroll()` CSS function
- `scroll-behavior` CSS property
- `::scroll-button()` CSS pseudo-element
- `scroll-initial-target` CSS property
- `scroll-margin` CSS property
- `scroll-margin-block` CSS property
- `scroll-margin-block-end` CSS property
- `scroll-margin-block-start` CSS property
- `scroll-margin-bottom` CSS property
- `scroll-margin-inline` CSS property
- `scroll-margin-inline-end` CSS property
- `scroll-margin-inline-start` CSS property
- `scroll-margin-left` CSS property
- `scroll-margin-right` CSS property
- `scroll-margin-top` CSS property
- `::scroll-marker` CSS pseudo-element

- `scroll-marker-group` CSS property
- `::scroll-marker-group` CSS pseudo-element
- `scroll-padding` CSS property
- `scroll-padding-block` CSS property
- `scroll-padding-block-end` CSS property
- `scroll-padding-block-start` CSS property
- `scroll-padding-bottom` CSS property
- `scroll-padding-inline` CSS property
- `scroll-padding-inline-end` CSS property
- `scroll-padding-inline-start` CSS property
- `scroll-padding-left` CSS property
- `scroll-padding-right` CSS property
- `scroll-padding-top` CSS property
- `scroll-snap-align` CSS property
- `scroll-snap-stop` CSS property
- `scroll-snap-type` CSS property
- `scroll-target-group` CSS property
- `scroll-timeline` CSS property
- `scroll-timeline-axis` CSS property
- `scroll-timeline-name` CSS property
- `scrollbar-color` CSS property
- `scrollbar-gutter` CSS property
- `scrollbar-width` CSS property
- `::search-text` CSS pseudo-element
- `:seeking` CSS pseudo-class
- `::selection` CSS pseudo-element
- `<self-position>` CSS type
- `sepia()` CSS function
- `shape()` CSS function
- `shape-image-threshold` CSS property
- `shape-margin` CSS property
- `shape-outside` CSS property
- `shape-rendering` CSS property
- `<shape>` CSS type
- `sibling-count()` CSS function
- `sibling-index()` CSS function
- `sign()` CSS function
- `sin()` CSS function
- `size` CSS at-rule descriptor (`@page`)
- `size-adjust` CSS at-rule descriptor (`@font-face`)
- `skew()` CSS function
- `skewX()` CSS function
- `skewY()` CSS function
- `::slotted()` CSS pseudo-element
- `speak-as` CSS at-rule descriptor (`@counter-style`)
- `speak-as` CSS property
- `::spelling-error` CSS pseudo-element
- `sqrt()` CSS function
- `src` CSS at-rule descriptor (`@font-face`)
- `:stalled` CSS pseudo-class
- `@starting-style` CSS at-rule
- `:state()` CSS pseudo-class
- `steps()` CSS function
- `stop-color` CSS property
- `stop-opacity` CSS property
- `<string>` CSS type
- `stroke` CSS property
- `stroke-dasharray` CSS property
- `stroke-dashoffset` CSS property
- `stroke-linecap` CSS property

- `stroke-linejoin` CSS property
- `stroke-miterlimit` CSS property
- `stroke-opacity` CSS property
- `stroke-width` CSS property
- `suffix` CSS at-rule descriptor (`@counter-style`)
- `superellipse()` CSS function
- `@supports` CSS at-rule
- `symbols` CSS at-rule descriptor (`@counter-style`)
- `symbols()` CSS function
- `syntax` CSS at-rule descriptor (`@property`)
- `system` CSS at-rule descriptor (`@counter-style`)
- `<system-color>` CSS type

T

- Type selectors
- `tab-size` CSS property
- `table-layout` CSS property
- `tan()` CSS function
- `-webkit-tap-highlight-color` CSS property
- `:target` CSS pseudo-class
- `:target-after` CSS pseudo-class
- `:target-before` CSS pseudo-class
- `:target-current` CSS pseudo-class
- `::target-text` CSS pseudo-element
- `text-align` CSS property
- `text-align-last` CSS property
- `text-anchor` CSS property
- `text-autospace` CSS property
- `text-box` CSS property
- `text-box-edge` CSS property
- `text-box-trim` CSS property
- `text-combine-upright` CSS property
- `text-decoration` CSS property
- `text-decoration-color` CSS property
- `text-decoration-inset` CSS property
- `text-decoration-line` CSS property
- `text-decoration-skip` CSS property
- `text-decoration-skip-ink` CSS property
- `text-decoration-style` CSS property
- `text-decoration-thickness` CSS property
- `<text-edge>` CSS type
- `text-emphasis` CSS property
- `text-emphasis-color` CSS property
- `text-emphasis-position` CSS property
- `text-emphasis-style` CSS property
- `-webkit-text-fill-color` CSS property
- `text-fit` CSS property
- `text-indent` CSS property
- `text-justify` CSS property
- `text-orientation` CSS property
- `text-overflow` CSS property
- `text-rendering` CSS property
- `-webkit-text-security` CSS property
- `text-shadow` CSS property
- `text-size-adjust` CSS property
- `text-spacing-trim` CSS property
- `-webkit-text-stroke` CSS property
- `-webkit-text-stroke-color` CSS property
- `-webkit-text-stroke-width` CSS property
- `text-transform` CSS property

- `text-underline-offset` CSS property
- `text-underline-position` CSS property
- `text-wrap` CSS property
- `text-wrap-mode` CSS property
- `text-wrap-style` CSS property
- `<time-percentage>` CSS type
- `<time>` CSS type
- `<timeline-range-name>` CSS type
- `timeline-scope` CSS property
- timeline-trigger CSS property
- timeline-trigger-activation-range CSS property
- timeline-trigger-activation-range-end CSS property
- timeline-trigger-activation-range-start CSS property
- timeline-trigger-active-range CSS property
- timeline-trigger-active-range-end CSS property
- timeline-trigger-active-range-start CSS property
- timeline-trigger-name CSS property
- timeline-trigger-source CSS property
- `top` CSS property
- `touch-action` CSS property
- `-webkit-touch-callout` CSS property
- `transform` CSS property
- `transform-box` CSS property
- `<transform-function>` CSS type
- `transform-origin` CSS property
- `transform-style` CSS property
- `transition` CSS property
- `transition-behavior` CSS property
- `transition-delay` CSS property
- `transition-duration` CSS property
- `transition-property` CSS property
- `transition-timing-function` CSS property
- `translate` CSS property
- `translate()` CSS function
- `translate3d()` CSS function
- `translateX()` CSS function
- `translateY()` CSS function
- `translateZ()` CSS function
- trigger-scope CSS property
- `type()` CSS function

U

- Universal selectors
- `unicode-bidi` CSS property
- `unicode-range` CSS at-rule descriptor (`@font-face`)
- `unset` CSS keyword
- `url()` CSS function
- `<url>` CSS type
- `-moz-user-focus` CSS property
- `-moz-user-input` CSS property
- `:user-invalid` CSS pseudo-class
- `user-modify` CSS property
- `user-select` CSS property
- `:user-valid` CSS pseudo-class

V

- `:valid` CSS pseudo-class
- `var()` CSS function
- `vector-effect` CSS property
- `vertical-align` CSS property

- `view()` CSS function
- `view-timeline` CSS property
- `view-timeline-axis` CSS property
- `view-timeline-inset` CSS property
- `view-timeline-name` CSS property
- `@view-transition` CSS at-rule
- `::view-transition` CSS pseudo-element
- `view-transition-class` CSS property
- `::view-transition-group()` CSS pseudo-element
- `::view-transition-image-pair()` CSS pseudo-element
- `view-transition-name` CSS property
- `::view-transition-new()` CSS pseudo-element
- `::view-transition-old()` CSS pseudo-element
- `view-transition-scope` CSS property
- `visibility` CSS property
- `:visited` CSS pseudo-class
- `:volume-locked` CSS pseudo-class

W

- `:where()` CSS pseudo-class
- `white-space` CSS property
- `white-space-collapse` CSS property
- `widows` CSS property
- `width` CSS property
- `will-change` CSS property
- `word-break` CSS property
- `word-spacing` CSS property
- `writing-mode` CSS property

X

- `x` CSS property
- `:xr-overlay` CSS pseudo-class
- `xywh()` CSS function

Y

- `y` CSS property

Z

- `z-index` CSS property
- `zoom` CSS property

Seletores

- Seletores básicos
    - Seletores de tipo `elementname`
    - Seletores de classe `.classname`
    - Seletores de ID `#idname`
    - Seletores universais `* ns|* *|*`
    - Seletores de atributo
        - `[attribute]`
        - `[attribute="value"]`
        - `[attribute~="value"]`
        - `[attribute|="value"]`
        - `[attribute^="value"]`
        - `[attribute$="value"]`
        - `[attribute*="value"]`
        - `[ns|attribute]`
- Combinadores

        - Combinadores equivalentes adjacentes `A + B`
        - Combinadores equivalentes gerais `A ~ B`
        - Combinadores filhos `A > B`
        - Combinadores descendentes `A B`

- Pseudo-elementos
- Pseudo-classes

Miscelânea

- `Comments`
- Consultas de mídias
- Contadores
- Gradientes
- Transformações

Conceitos

- Sintaxe do CSS
- Regra At
- Comentários
- Especificidade
- Valor inicial
- Herança
- Valor especificado
- Valor computado
- Valor usado
- Valor corrente
- Modelo de caixa

@charset

`<string>`

Resumo

A regra `@charset` especifica a codificação de caracteres utilizada na folha de estilo. Deve ser o primeiro elemento na folha de estilo e não ser precedida de qualquer caráter; uma vez que não é uma declaração aninhada, ele não pode ser usado dentro de um grupo condicional de regras. Se vários `@charset` são definidos, apenas o primeiro é usado, e não pode ser usado dentro de um atributo de estilo em um elemento HTML ou dentro do elemento `<style>`, onde o conjunto de caracteres da página HTML é relevante.

Esta regra at (at-rule) é útil quando se usa caracteres não ASCII em algumas propriedades CSS, como conteúdo.

Como existem várias maneiras de definir a codificação de caracteres em uma folha de estilo, o navegador vai tentar os seguintes métodos na seguinte ordem (e parar assim que um produzir um resultado):

1. O valor do carácter de ordem de byte Unicode colocado no início do arquivo.
2. O valor dado pelo atributo `charset` do `Content-Type:` cabeçalho HTTP ou o equivalente no protocolo usado para servir a folha de estilo.
3. O `@charset` CSS.
4. Use a codificação de caracteres definidos pelo documento referente: o atributo charset do elemento `<link>`. Este método é obsoleto em HTML5 e não deve ser utilizado.
5. Assume que o documento é UTF-8.

Sintaxe

```
@charset charset;
```

onde :

charset

> É uma `<string>` denotando a codificação de caracteres a ser usado. Ele deve ser o nome de uma codificação de caracteres segura para a Web definido no registro IANA. Se vários nomes estão associados com uma codificação, apenas um é marcado como *preferido* e assim ser utilizado.

Exemplos

```
@charset "UTF-8"; /* Define a codificação da folha de estilo para Unicode UTF-8 */
@charset 'iso-8859-15'; /* Define a codificação da folha de estilo para Latin-9 (línguas da Europa Ocidental, com sinal de euro) */
 @charset "UTF-8"; /* Inválido, há um caracter (espaço) antes da regra */
@charset UTF-8; /*  Inválido, sem ‘ ou ", o conjunto de caracteres não é uma <string> CSS */
```

missing-glyph

O conteúdo do `missing-glyph` é renderizado se, para um determinado caractere, não foi determinado um `<glyph>` apropriado.

Contexto de uso

invalid templ: No svginfo for @font-face

Exemplo

Atributos

Atributos globais

- Atributos principais »
- Atributos de apresentação »
- `class`
- `style`

Atributos específicos

- `d`
- `horiz-adv-x`
- `vert-origin-x`
- `vert-origin-y`
- `vert-adv-y`

Interface DOM

Este elemento implementa a interface do `SVGMissingGlyphElement`.

Relacionado

- `<font>`
- `<glyph>`
- SVG tutorial: SVG fonts

@import

Sumário

A Regra Atribuída (at-rule) CSS **`@import`** é usado para importar regras de estilo de outras folhas de estilo. Estas regras devem preceder todos os outros tipos de regras, exceto as regrasde @charset ; uma vez que não é um Declaração aninhada, `@import não pode ser usado dentro do` grupo condicional de regras atribuídas (at-rules).

Assim como os agentes do usuário podem evitar a recuperação de recursos para os tipos de mídia incompatíveis, os autores podem especificar regras @import dependentes de mídia. Estas importações condicionais devem ter media queries separadas por vírgulas após a URI. Na ausência de qualquer consulta de mídia (media queries), a importação é incondicional. Especificando todos para o médio tem o mesmo efeito.

Sintaxe

```
@import url;
@import url list-of-media-queries;
```

where:

url

> É algo como `<string>` ou como `<uri>` representaando o local de onde o recurso será importado. A URL pode ser absoluta ou relativa. Perceba que a URL não precisa, necessariamente, ser um arquivo específico; ela pode especificar o nome do pacotee parcialmente, e o arquivo apropriado será escolhido automaticamente (exemplo: **chrome://communicator/skin/**). Veja aqui para mais informações.

list-of-media-queries

É uma lista separada por vírgulas de consultas de mídia (media queries) conidicionando a aplicação das regras de CSS definidas na URL relacionada. Se o navegador não suportar quaisquer consultas de mídia, ele não carregará o recurso relacionado.

Sintaxe formal

```
@import =
  @import [ <url> | <string> ] [ [ layer | layer( <layer-name> ) ] || [ scope | scope( <scope-start> | <scope-boundaries> ) ] || <supports-
import-condition> ]? <media-import-condition> ;

<layer-name> =
  <ident> [ '.' <ident> ]*

<scope-start> =
  <selector-list>

<scope-boundaries> =
  [ [ ( <scope-start> ) ]? [ to ( <scope-end> ) ]? ]!

<supports-import-condition> =
  supports( [ <supports-condition> | <declaration> ] )

<media-import-condition> =
  <media-query-list>

<selector-list> =
  <complex-selector-list>

<scope-end> =
  <selector-list>

<supports-condition> =
  not <supports-in-parens>                          |
  <supports-in-parens> [ and <supports-in-parens> ]*  |
  <supports-in-parens> [ or <supports-in-parens> ]*

<complex-selector-list> =
  <complex-selector>#

<supports-in-parens> =
  ( <supports-condition> )  |
  <supports-feature>        |
  <general-enclosed>

<complex-selector> =
  <complex-selector-unit> [ <combinator>? <complex-selector-unit> ]*

<supports-feature> =
  <supports-selector-fn>       |
  <supports-font-tech-fn>      |
  <supports-font-format-fn>    |
  <supports-at-rule-fn>        |
  <supports-named-feature-fn>  |
  <supports-env-fn>            |
  <supports-decl>

<general-enclosed> =
  [ <function-token> <any-value>? ) ]  |
  [ ( <any-value>? ) ]

<complex-selector-unit> =
  [ <compound-selector>? <pseudo-compound-selector>* ]!

<combinator> =
  '>'                |
  '+'                |
  '~'                |
  [ '|' '|' ]        |
  [ / <wq-name> / ]

<supports-selector-fn> =
  selector( <complex-selector> )

<supports-font-tech-fn> =
  font-tech( <font-tech> )
```

```
<supports-font-format-fn> =
  font-format( <font-format> )

<supports-at-rule-fn> =
  at-rule( <at-keyword-token> )

<supports-named-feature-fn> =
  named-feature( <ident> )

<supports-env-fn> =
  env( <ident> )

<supports-decl> =
  ( [ <declaration> | <supports-condition-name> ] )

<compound-selector> =
  [ <type-selector>? <subclass-selector>* ]!

<pseudo-compound-selector> =
  <pseudo-element-selector> <pseudo-class-selector>*

<wq-name> =
  <ns-prefix>? <ident-token>

<font-tech> =
  <font-features-tech>  |
  <color-font-tech>     |
  variations            |
  palettes              |
  incremental

<font-format> =
  <string>           |
  collection         |
  embedded-opentype  |
  opentype           |
  svg                |
  truetype           |
  woff               |
  woff2

<type-selector> =
  <wq-name>         |
  <ns-prefix>? '*'

<subclass-selector> =
  <id-selector>            |
  <class-selector>         |
  <attribute-selector>     |
  <pseudo-class-selector>

<pseudo-element-selector> =
  : <pseudo-class-selector>         |
  <legacy-pseudo-element-selector>

<pseudo-class-selector> =
  : <ident-token>                   |
  : <function-token> <any-value> )

<ns-prefix> =
  [ <ident-token> | '*' ]? '|'

<font-features-tech> =
  features-opentype  |
  features-aat       |
  features-graphite

<color-font-tech> =
  color-COLRv0  |
  color-COLRv1  |
  color-SVG     |
  color-sbix    |
  color-CBDT

<id-selector> =
  <hash-token>
```

```
<class-selector> =
  '.' <ident-token>

<attribute-selector> =
  '[' <wq-name> ']'                                    |
  '[' <wq-name> <attr-matcher> [ <string-token> | <ident-token> ] <attr-modifier>? ']'

<legacy-pseudo-element-selector> =
  : [ before | after | first-line | first-letter ]

<attr-matcher> =
  [ '~' | '|' | '^' | '$' | '*' ]? '='

<attr-modifier> =
  i  |
  s
```

Esta sintaxe reflete o padrão mais recente conforme CSS Cascading and Inheritance Level 5, CSS Cascading and Inheritance Level 6, CSS Conditional Rules Module Level 3, CSS Conditional Rules Module Level 4, CSS Conditional Rules Module Level 5, CSS Fonts Module Level 5, Media Queries Level 5, Selectors Level 4, Selectors Level 5. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

```
@import url("fineprint.css") print;
@import url("bluish.css") projection, tv;
@import "custom.css";
@import url("chrome://communicator/skin/");
@import "common.css" screen, projection;
@import url("landscape.css") screen and (orientation: landscape);
```

@layer

A regra at (at-rule) **`@layer`** é utilizada para definir uma *camada de cascata*, também definindo a ordem de prioridade quando temos mais de uma camada.

Experimente

```
@layer module, state;

@layer state {
  .alert {
    background-color: brown;
  }
  p {
    border: medium solid limegreen;
  }
}

@layer module {
  .alert {
    border: medium solid violet;
    background-color: yellow;
    color: white;
  }
}
```

```
<p class="alert">Beware of the zombies</p>
```

Sintaxe

```
@layer nome-da-camada {regras}
@layer nome-da-camada;
@layer nome-da-camada, nome-da-camada, nome-da-camada;
@layer {regras}
```

Sendo que:

nome-da-camada

Define o nome da sua camada de cascata.

regras

São as regras CSS utilizadas dentro da camada.

Descrição

Adicionar regras CSS numa *camada de cascata* possibilita aos desenvolvedores web um controle maior sobre a cascata. Qualquer estilo fora de uma camada são agrupados numa camada anônima posicionada após todas as camadas declaradas, nomeadas ou anônimas. Isso significa que quaisquer estilos declarados fora de uma camada vão sobrescrever os estilos declarados em camadas, seja qual for seu grau de especificidade.

Podemos criar uma camada de cascata com a regra at `@layer` de três maneiras.

A primeira é criando uma camada de cascata nomeada, inserindo as regras CSS dentro deste bloco.

```
@layer utilitarios {
  .padding-sm {
    padding: 0.5rem;
  }

  .padding-lg {
    padding: 0.8rem;
  }
}
```

A segunda maneira é criando uma camada de cascata nomeada sem inserir nenhuma regra CSS, como no exemplo abaixo:

```
@layer utilitarios;
```

Múltiplas camadas podem ser definidas de uma só vez; como, por exemplo:

```
@layer temas, layout, utilitarios;
```

Isto pode ser útil pois a ordem inicial na qual as camadas são declaradas definem qual camada tem preferência. Assim como declarações, a última camada a ser definida terá preferência se declarações concorrentes forem encontradas em múltiplas camadas. Aplicando ao exemplo anterior, se uma regra conflitante estiver nas camadas `temas` e `utilitarios`, a regra encontrada no `utilitarios` é a vencedora e será aplicada.

A regra definida no `utilitarios` será aplicada *mesmo se tiver especificidade mais baixa* que a regra em `tema`. Isso ocorre, pois uma vez que a ordem das camadas é estabelecida, especificidade e ordem de aparição das regras são ignoradas. Isso nos permite criar seletores CSS mais simples, já que você não precisa se preocupar tanto se um seletor CSS terá especificidade alta o suficiente para sobrescrever regras concorrentes, você só precisa verificar se a sua regra está localizada em uma camada definida posteriormente.

> **Nota:** Ao declarar o nome de suas camadas e sua ordem, você pode adicionar regras CSS para a camada re-declarando seu nome. Os estilos serão incluídos na camada e a ordenação das camadas não será modificada.

A terceira opção de criação de camada são as camadas anônimas, como no exemplo abaixo:

```
@layer {
  p {
    margin-block: 1rem;
  }
}
```

Esta camada funciona da mesma maneira que as camadas nomeadas; porém, não poderão ser atribuídas novas regras a esta camada depois. O seu grau de preferência segue a ordem de declaração das camadas, anônimas ou não, sendo menor do que as regras definidas fora de camada.

Outra maneira de criar uma camada de cascata é utilizando o @import. Neste caso, as regras serão as da folha de estilo importada. Lembre-se que a regra at @import deve vir antes dos outros tipos de regras, com exceção das regras @charset e @layer.

```
@import "tema.css" layer(utilitarios);
```

Aninhando camadas

Camadas podem ser aninhadas (nested), como no exemplo abaixo:

```
@layer framework {
  @layer layout {
```

```
  }
}
```

Para juntas as regras da camada `layout` dentro de `framework`, junte os dois nomes com um `.`.

```
@layer framework.layout {
  p {
    margin-block: 1rem;
  }
}
```

Sintaxe formal

```
@layer =
  @layer <layer-name>? { <rule-list> }  |
  @layer <layer-name># ;

<layer-name> =
  <ident> [ '.' <ident> ]*
```

Esta sintaxe reflete o padrão mais recente conforme CSS Cascading and Inheritance Level 5. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

Exemplo simples

No exemplo seguinte, duas regras CSS são criadas. Uma para o elemento `<p>` fora de qualquer layer, e outra em uma camada chamada `tipo` com o seletor `.caixa p`.

Sem camadas, o seletor `.caixa p` teria a maior especificidade, então o texto 'Olá, mundo!' seria mostrado em verde. Mas como a camada `tipo` vem antes da camada anônima criada para agrupar todos as regras sem camada definida, o texto será mostrado roxo.

Perceba também a ordem. Mesmo se declararmos um estilo sem camada primeiro, ele ainda é aplicado *depois* dos estilos com camada.

HTML

```
<div class="caixa">
  <p>Olá, mundo!</p>
</div>
```

CSS

```
p {
  color: rebeccapurple;
}

@layer tipo {
  .caixa p {
    font-weight: bold;
    font-size: 1.3em;
    color: green;
  }
}
```

Resultado

Declarando regras para camadas pré-existentes

No exemplo seguinte, duas camadas são criadas sem regras definidas, e depois são aplicadas regras CSS para as duas camadas. A camada `base` define as propriedades `color`, `border`, `font-size` e `padding`. A camada `especial` define um valor de `color` diferente. Como o `especial` vem depois na definição das camadas, a cor que ele carrega é utilizada e o texto será mostrado com a cor `rebeccapurple`. Todas as outras regras da camada `base` ainda são aplicadas.

HTML

```
<div class="item">
  Eu estou sendo mostrado com <code>color: rebeccapurple</code> pois a camada
  <code>especial</code> vem depois da camada <code>base</code>. Minha borda
```

```
  verde, font-size, e padding vem da camada <code>base</code>.
</div>
```

CSS

```
@layer base, especial;

@layer especial {
  .item {
    color: rebeccapurple;
  }
}

@layer base {
  .item {
    color: green;
    border: 5px solid green;
    font-size: 1.3em;
    padding: 0.5em;
  }
}
```

Resultado

Veja também

- `@import`
- Cascata, especificidade e herança
- `CSSLayerBlockRule`
- `CSSLayerStatementRule`
- `!important`
- `revert-layer`
- Introducing the CSS cascade
- Cascade layers
- The future of CSS: Cascade layers em bram.us (2021)

aspect-ratio

A característica de mídia CSS **`aspect-ratio`** pode ser utilizada para testar a taxa de aspecto de sua viewport.

Sintaxe

A característica de `aspect-ratio` é especificada como um valor de `<ratio>` representando a relação de largura-para-altura em aspecto à sua viewport. É uma característica de diferencial, o que significa que você pode fazer uso de variações pré-fixadas de **`min-aspect-ratio`** e **`max-aspect-ratio`** para definir os os seus valores mínimos e máximos, respectivamente.

Exemplo

O exemplo abaixo está contido em um `<iframe>`, que cria o seu próprio viewport. Redimensione o `<iframe>` para ver o `aspect-ratio` em ação !

HTML

```
<div id="inner">
  Observe este elemento, conforme voc&ecirc; altera a largura e a altura da sua
  viewport.
</div>
```

CSS

```
/* Taxa de aspecto mínima */
@media (min-aspect-ratio: 8/5) {
  div {
    background: #9af; /* azul */
  }
}

/* Taxa de aspecto máxima */
@media (max-aspect-ratio: 3/2) {
  div {
```

```
    background: #9ff; /* ciano */
  }
}

/* Taxa de aspecto exata, coloque-a no final para evitar sobrescrita*/
@media (aspect-ratio: 1/1) {
  div {
    background: #f9a; /* vermelho */
  }
}
```

```
<label id="wf" for="w">width:165</label>
<input id="w" name="w" type="range" min="100" max="250" step="5" value="165" />
<label id="hf" for="w">height:165</label>
<input id="h" name="h" type="range" min="100" max="250" step="5" value="165" />

<iframe
  id="outer"
  src="data:text/html,<style> @media (min-aspect-ratio: 8/5) { div { background: %239af; } } @media (max-aspect-ratio: 3/2) { div {
background: %239ff; } } @media (aspect-ratio: 1/1) { div { background: %23f9a; } }</style><div id='inner'> Observe este elemento,
conforme voc&ecirc; altera a largura e a altura da sua viewport.</div>">
</iframe>
```

```
iframe {
  display: block;
}
```

```
outer.style.width = outer.style.height = "165px";

w.onchange = w.oninput = function () {
  outer.style.width = w.value + "px";
  wf.textContent = "width:" + w.value;
};
h.onchange = h.oninput = function () {
  outer.style.height = h.value + "px";
  hf.textContent = "height:" + h.value;
};
```

Result

Modo de exibição (display-mode)

O recurso de mídia CSS `@media` do modo de exibição pode ser usado para aplicar estilos com base no modo de exibição de um aplicativo. Você pode usar para prover uma consistente experiência de usuário entre abrir um site a partir de uma URL e um ícone desktop.

Esse recurso corresponde ao membro da exibição do manifesto do aplicativo da Web. Ambos aplicam-se ao contexto de navegação de nível superior e a qualquer contexto afilhado de navegação. O recurso de consulta aplica-se independentemente de onde um manifesto de aplicativo web está presente.

Sintaxe

O recurso `display-mode` é especificado como um valor de uma palavra chave escolhido da lista abaixo.

Modo de exibição	Descrição	Modo de exibição secundário
`fullscreen`	Toda a área disponível da tela será usada e nada do agente-usuário (navegador) chrome é mostrado.	`standalone`
`standalone`	A aplicação irá parecer uma aplicação independente. Isso pode incluir a aplicação ter uma janela diferente, seu próprio ícone no menu de aplicações, etc. Neste modo, o navegador não irá mostrar os controles de navegação (como barra de endereço, botão voltar, botão página inicial, etc), mas pode incluir outros elementos gráficos como a barra de status.	`minimal-ui`
`minimal-ui`	A aplicação irá parecer uma aplicação independente, mas terá um conjunto mínimo de elementos gráficos (UI) para controlar a navegação. Esses elementos podem variar de acordo com o navegador.	`browser`

Modo de exibição	Descrição	Modo de exibição secundário
`browser`	A aplicação abre numa aba ou nova janela convencional do navegador, dependendo do navegador e da plataforma.	(nenhum)

Exemplo

```
@media all and (display-mode: fullscreen) {
  body {
    margin: 0;
    border: 5px solid black;
  }
}
```

prefers-color-scheme

> **Nota:** Se você definiu `privacy.resistFingerprinting` como **true**, `prefers-color-scheme` será substituída pela `light`.

O **`prefers-color-scheme`** CSS media feature é usado para detectar se o usuário solicitou que o sistema use um tema de cores claras ou escuras.

Syntax

`no-preference`

Indica que o usuário não fez nenhuma preferência conhecida pelo sistema. Este valor da palavra-chave é avaliado como `false` no contexo booleano.

`light`

Indica que o usuário notificou o sistema de que prefere uma interface com um tema claro.

`dark`

Indica que o usuário notificou o sistema de que prefere uma interface com um tema escuro.

Exemplos

Os elementos abaixo têm um tema de cores inicial. Eles podem ser mais temáticos de acordo com a preferência do esquema de cores do usuário.

HTML

```
<div class="day">Dia (inicial)</div>
<div class="day light-scheme">Dia (modificar e utilizar um tema claro)</div>
<div class="day dark-scheme">Dia (modificar e utilizar um tema escuro)</div>
<br />

<div class="night">Noite (inicial)</div>
<div class="night light-scheme">Noite (modificar e utilizar um tema claro)</div>
<div class="night dark-scheme">Noite (modificar e utilizar um tema escuro)</div>
```

CSS

```
.day {
  background: #eee;
  color: black;
}
.night {
  background: #333;
  color: white;
}

@media (prefers-color-scheme: dark) {
  .day.dark-scheme {
    background: #333;
    color: white;
  }
  .night.dark-scheme {
```

```
    background: black;
    color: #ddd;
  }
}

@media (prefers-color-scheme: light) {
  .day.light-scheme {
    background: white;
    color: #555;
  }
  .night.light-scheme {
    background: #eee;
    color: black;
  }
}

.day,
.night {
  display: inline-block;
  padding: 1em;
  width: 7em;
  height: 2em;
  vertical-align: middle;
}
```

Resultado

Veja também

- Video tutorial: Coding a Dark Mode for your Website
- Redesigning your product and website for dark mode
- Changing color schemes in Windows, macOS and Android.

page(Doc) not found /pt-BR/docs/Web/CSS/@media/

@page

Resumo

A regra CSS `@page` é utilizada para modificar algumas propriedades CSS quando o documento for impresso. Você não pode mudar todas as propriedades CSS com @page. Você poderá somente mudar as margens, orphans, widows, e page breaks do documento. Na tentativa de mudar outra propriedade CSS, elas serão ignoradas.

A regra CSS `@page` pode ser acessada via interface do modelo de objeto `CSSPageRule`.

> **Nota:** A W3C está analisando como lidar com unidades viewport-related `<length>`, `vh`, `vw`, `vmin`, and `vmax`. Enquanto isso, não use eles junto com a regra @page.

Sintaxe

```
@page :pseudo-class {
  margin:2in;
}
```

Exemplos

Podemos fazer referência a vários pseudo-classes de `@page` por exemplo.

- `:first`
- `:left`
- `:right`

Propriedades personalizadas (--*)

> **Experimental: Esta é uma tecnologia experimental**
> Verifique a tabela de compatibilidade entre Navegadores cuidadosamente antes de usar essa funcionalidade em produção.

Nomes de propriedades com o prefixo `--`, como `--example-name`, representam *Propriedade personalizadas* que contém um valor que pode ser reutilizado por todo o documento usando a função (`var()`).

Propriedades personalizadas participam na cascata: cada uma delas pode aparecer várias vezes e o valor da variável corresponderá ao valor definido na propriedade personalizada, decidido pelo algoritmo de cascata.

Initial value	see prose
Aplica-se a	all elements
Inherited	yes
Computed value	as specified with variables substituted
Animation type	discrete

Sintaxe

```
--somekeyword: left;
--somecolor: #0000ff;
--somecomplexvalue: 3px 6px rgb(20, 32, 54);
```

`<declaration-value>`

> Este valor corresponde a qualquer sequência de um ou mais tokens, desde que a sequência não contenha tokens não permitidos. Representa a totalidade do que uma declaração válida pode ter como valor.

Sintaxe formal

Error: could not find syntax for this item

Exemplo

HTML

```
<p id="firstParagraph">
  Este parágrafo deve ter um fundo azul e um texto amarelo.
</p>
<p id="secondParagraph">
  Este parágrafo deve ter um fundo amarelo e um texto azul.
</p>
<div id="container">
  <p id="thirdParagraph">
    Este parágrafo deve ter um fundo verde e um texto amarelo.
  </p>
</div>
```

CSS

```
:root {
  --first-color: #488cff;
  --second-color: #ffff8c;
}

#firstParagraph {
  background-color: var(--first-color);
  color: var(--second-color);
}

#secondParagraph {
  background-color: var(--second-color);
  color: var(--first-color);
}

#container {
  --first-color: #48ff32;
}

#thirdParagraph {
  background-color: var(--first-color);
  color: var(--second-color);
}
```

Resultado

Veja também

- Utilizando variáveis CSS

-webkit-text-security

> **Não padrão:** Este recurso não é padronizado. Não recomendamos o uso de recursos não padronizados em produção, pois eles têm suporte limitado nos navegadores e podem mudar ou ser removidos. No entanto, podem ser uma alternativa viável em casos específicos onde não exista uma opção padrão.

`-webkit-text-security` é uma propriedade CSS não padronizada que ofusca caracteres em um campo `<form>` (como `<input>` ou `<textarea>`) substituindo eles com uma forma. Afeta apenas campos que não são `type=password`.

Sintaxe

```
-webkit-text-security: circle;
-webkit-text-security: disc;
-webkit-text-security: square;
-webkit-text-security: none;
```

Exemplo

Tente digitar no campo abaixo. Se o seu navegador suportar essa propriedade, os caracteres deverão ser visualmente substituídos por quadrados.

HTML

```
<label for="name">Name:</label> <input type="text" name="name" id="name" />
```

CSS

```
input {
  -webkit-text-security: square;
}
```

Result

Especificação

Não faz parte de nenhuma especificação.

Compatibilidade com navegadores

Suportado em navegadores baseados no WebKit e Blink.

Veja também

- WebKit CSS extensions

align-content

A propriedade CSS **`align-content`** define a distribuição entre e ao redor dos items do conteúdo do eixo transversal de uma flexbox ou do eixo de bloco de uma grid.

O exemplo interativo abaixo use Grid Layout para demonstrar alguns dos valores dessa propriedade.

Experimente

```
align-content: start;
```

```
align-content: center;
```

```
align-content: space-between;
```

```
align-content: space-around;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">
      <div>One</div>
      <div>Two</div>
      <div>Three</div>
    </div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  display: grid;
  grid-template-columns: 60px 60px;
  grid-auto-rows: 40px;
  column-gap: 10px;
  height: 180px;
}

#example-element > div {
  background-color: rgba(0, 0, 255, 0.2);
  border: 3px solid blue;
}
```

This property has no effect on single line flex containers (i.e. ones with `flex-wrap: nowrap`).

Syntax

```
/* Basic positional alignment */
/* align-content does not take left and right values */
align-content: center; /* Pack items around the center */
align-content: start; /* Pack items from the start */
align-content: end; /* Pack items from the end */
align-content: flex-start; /* Pack flex items from the start */
align-content: flex-end; /* Pack flex items from the end */

/* Normal alignment */
align-content: normal;

/* Baseline alignment */
align-content: baseline;
align-content: first baseline;
align-content: last baseline;

/* Distributed alignment */
align-content: space-between; /* Distribute items evenly
                                 The first item is flush with the start,
                                 the last is flush with the end */
align-content: space-around; /* Distribute items evenly
                                 Items have a half-size space
                                 on either end */
align-content: space-evenly; /* Distribute items evenly
                                 Items have equal space around them */
align-content: stretch; /* Distribute items evenly
                                 Stretch 'auto'-sized items to fit
                                 the container */

/* Overflow alignment */
align-content: safe center;
align-content: unsafe center;

/* Global values */
align-content: inherit;
align-content: initial;
align-content: revert;
align-content: unset;
```

Values

`start`

The items are packed flush to each other against the start edge of the alignment container in the cross axis.

`end`

The items are packed flush to each other against the end edge of the alignment container in the cross axis.

`flex-start`

The items are packed flush to each other against the edge of the alignment container depending on the flex container's cross-start side. This only applies to flex layout items. For items that are not children of a flex container, this value is treated like `start`.

`flex-end`

The items are packed flush to each other against the edge of the alignment container depending on the flex container's cross-end side. This only applies to flex layout items. For items that are not children of a flex container, this value is treated like `end`.

`center`

The items are packed flush to each other in the center of the alignment container along the cross axis.

`normal`

The items are packed in their default position as if no `align-content` value was set.

`baseline`, `first baseline`, `last baseline`

Specifies participation in first- or last-baseline alignment: aligns the alignment baseline of the box's first or last baseline set with the corresponding baseline in the shared first or last baseline set of all the boxes in its baseline-sharing group.



The fallback alignment for `first baseline` is `start`, the one for `last baseline` is `end`.

`space-between`

The items are evenly distributed within the alignment container along the cross axis. The spacing between each pair of adjacent items is the same. The first item is flush with the start edge of the alignment container in the cross axis, and the last item is flush with the end edge of the alignment container in the cross axis.

`space-around`

The items are evenly distributed within the alignment container along the cross axis. The spacing between each pair of adjacent items is the same. The empty space before the first and after the last item equals half of the space between each pair of adjacent items.

`space-evenly`

The items are evenly distributed within the alignment container along the cross axis. The spacing between each pair of adjacent items, the start edge and the first item, and the end edge and the last item, are all exactly the same.

`stretch`

If the combined size of the items along the cross axis is less than the size of the alignment container, any `auto`-sized items have their size increased equally (not proportionally), while still respecting the constraints imposed by `max-height`/`max-width` (or equivalent functionality), so that the combined size exactly fills the alignment container along the cross axis.

`safe`

Used alongside an alignment keyword. If the chosen keyword means that the item overflows the alignment container causing data loss, the item is instead aligned as if the alignment mode were `start`.

`unsafe`

Used alongside an alignment keyword. Regardless of the relative sizes of the item and alignment container and whether overflow which causes data loss might happen, the given alignment value is honored.

Formal definition

Initial value	`normal`
Aplica-se a	Block-containers, multi-column containers, flex containers
Inherited	não
Computed value	as specified
Animation type	discrete

Formal syntax

```
align-content =
  normal                                |
  <baseline-position>                   |
  <content-distribution>                |
  <overflow-position>? <content-position>

<baseline-position> =
  [ first | last ]?  &&
  baseline

<content-distribution> =
  space-between  |
  space-around   |
  space-evenly   |
  stretch

<overflow-position> =
  unsafe  |
  safe

<content-position> =
  center      |
  start       |
  end         |
  flex-start  |
  flex-end
```

Esta sintaxe reflete o padrão mais recente conforme CSS Box Alignment Module Level 3. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Examples

CSS

```
#container {
  height: 200px;
  width: 240px;
  align-content: center; /* Can be changed in the live sample */
  background-color: #8c8c8c;
}

.flex {
  display: flex;
  flex-wrap: wrap;
}

.grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, 50px);
}

div > div {
  box-sizing: border-box;
  border: 2px solid #8c8c8c;
  width: 50px;
  display: flex;
  align-items: center;
  justify-content: center;
}

#item1 {
```

```
  background-color: #8cffa0;
  min-height: 30px;
}

#item2 {
  background-color: #a0c8ff;
  min-height: 50px;
}

#item3 {
  background-color: #ffa08c;
  min-height: 40px;
}

#item4 {
  background-color: #ffff8c;
  min-height: 60px;
}

#item5 {
  background-color: #ff8cff;
  min-height: 70px;
}

#item6 {
  background-color: #8cffff;
  min-height: 50px;
  font-size: 30px;
}

select {
  font-size: 16px;
}

.row {
  margin-top: 10px;
}
```

HTML

```
<div id="container" class="flex">
  <div id="item1">1</div>
  <div id="item2">2</div>
  <div id="item3">3</div>
  <div id="item4">4</div>
  <div id="item5">5</div>
  <div id="item6">6</div>
</div>

<div class="row">
  <label for="display">display: </label>
  <select id="display">
    <option value="flex">flex</option>
    <option value="grid">grid</option>
  </select>
</div>

<div class="row">
  <label for="values">align-content: </label>
  <select id="values">
    <option value="normal">normal</option>
    <option value="stretch">stretch</option>
    <option value="flex-start">flex-start</option>
    <option value="flex-end">flex-end</option>
    <option value="center" selected>center</option>
    <option value="space-between">space-between</option>
    <option value="space-around">space-around</option>
    <option value="space-evenly">space-evenly</option>

    <option value="start">start</option>
    <option value="end">end</option>
    <option value="left">left</option>
    <option value="right">right</option>
```

```
    <option value="baseline">baseline</option>
    <option value="first baseline">first baseline</option>
    <option value="last baseline">last baseline</option>

    <option value="safe center">safe center</option>
    <option value="unsafe center">unsafe center</option>
    <option value="safe right">safe right</option>
    <option value="unsafe right">unsafe right</option>
    <option value="safe end">safe end</option>
    <option value="unsafe end">unsafe end</option>
    <option value="safe flex-end">safe flex-end</option>
    <option value="unsafe flex-end">unsafe flex-end</option>
  </select>
</div>
```

```
var values = document.getElementById("values");
var display = document.getElementById("display");
var container = document.getElementById("container");

values.addEventListener("change", function (evt) {
  container.style.alignContent = evt.target.value;
});

display.addEventListener("change", function (evt) {
  container.className = evt.target.value;
});
```

Result

See also

- CSS Flexbox Guide: *Basic Concepts of Flexbox*
- CSS Flexbox Guide: *Aligning items in a flex container*
- CSS Grid Guide: *Box alignment in CSS Grid layouts*
- CSS Box Alignment

align-items

A propriedade CSS **`align-items`** estabelece o valor `align-self` em todos filhos diretos como um grupo. A propriedade align-self estabelece o alinhamento de um certo item dentro do bloco que o contém. Em Flexbox ele controla o alinhamento dos itens em Cross Axis, enquanto que no Grid Layout, controla o alinhamento dos itens no Eixo de Bloco dentro de sua grid area.

O exemplo interativo abaixo demonstra alguns dos valores para `align-items` usando grid layout.

Experimente

```
align-items: stretch;
```

```
align-items: center;
```

```
align-items: start;
```

```
align-items: end;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">
      <div>One</div>
      <div>Two</div>
      <div>Three</div>
    </div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  display: grid;
  width: 200px;
```

```
  grid-template-columns: 1fr 1fr;
  grid-auto-rows: 80px;
  grid-gap: 10px;
}

#example-element > div {
  background-color: rgba(0, 0, 255, 0.2);
  border: 3px solid blue;
}
```

Sintaxe

```
/* Palavras-chave básicas */
align-items: normal;
align-items: stretch;

/* Posicionamento do alinhamento */
/* align-items não recebe valores left e right */
align-items: center; /* Itens posicionados ao redor do centro */
align-items: start; /* Posiciona itens a partir do início */
align-items: end; /* Posiciona itens a partir do fim */
align-items: flex-start; /* Posiciona itens-flex a partir do início */
align-items: flex-end; /* Posiciona itens-flex a partir do fim */
align-items: self-start;
align-items: self-end;

/* Alinhamento da baseline */
align-items: baseline;
align-items: first baseline;
align-items: last baseline; /* Overflow alinhamento (apenas para alinhamento de posição) */
align-items: safe center;
align-items: unsafe center;

/* Valores globais */
align-items: inherit;
align-items: initial;
align-items: unset;
```

Valores

`normal`

O efeito desta palavra-chave depende do modo de layout em que nos encontramos:

- Em layouts absolutamente posicionados, a palavra-chave se comporta como `start` em *replaced* de caixas absolutamente posicionadas, e como `stretch` em *all others* de caixas absolutamente posicionadas.
- Em posição estática de layouts absolutamente posicionados, a palavra-chave se comporta como `stretch`.
- Para itens-flex, a palavra chave se comporto como `stretch`.
- Para itens-grid, esta palavra-chave leva a um comportamento similiar ao `stretch`, exceto para caixas com um aspect ratio ou tamanhos intrínsecos, onde se comporta como `start`.
- A propriedade não se aplica para caixas block-level, e para células de tabela.

`flex-start`

As bordas da margem superior dos itens-flex são juntados com a borda superior da linha.

`flex-end`

As bordas da margem inferior dos itens flex são juntados com a borda inferior da linha.

`center`

As caixas de margem dos itens flex são centralizados ao longo da linha do eixo. Caso o tamanho de um item seja superior ao container flex, irá transbordar de maneira igual em ambas direções.

`start`

Os itens são posicionados, para cada um, em direção a borda início do container de alinhamento no eixo apropriado.

`end`

Os itens são posicionados, para cada um, em direção a borda fim do container de alinhamento no eixo apropriado.

`self-start`

Os itens são posicionados para a borda do container de alinhamento do lado inicial do item, no eixo apropriado.

`self-end`

Os itens são posicionados para a borda do container de alinhamento do lado final do item, no eixo apropriado.

`baselinefirst baselinelast baseline`

Todos itens-flex são alinhados de tal forma que suas baselines de container flex estejam alinhados. O item com a maior distância entre o eixo de início de sua margem e sua baseline é juntado com eixo de início da linha.

`stretch`

Itens-flex são esticados de tal maneira que o tamanho da caixa de margem do item seja o mesmo da linha, respeitando as restrições de comprimento e altura.

`safe`

Usado em conjunto com uma palavra-chave de alinhamento. Se a palavra-chave escolhida signifique que o item transborde o alinhamento do container, causando assim perda de dados, o item é alinhado como estivesse no modo `start`.

`unsafe`

Usado em conjunto com uma palavra-chave de alinhamento. Independente dos tamanhos relativos do item, do alinhamento do container e caso overflow possa causar perda de dados, o valor de alinhamento estabelecido é mantido.

Sintaxe formal

```
align-items =
  normal                                |
  stretch                               |
  <baseline-position>                   |
  <overflow-position>? <self-position>

<baseline-position> =
  [ first | last ]?  &&
  baseline

<overflow-position> =
  unsafe  |
  safe

<self-position> =
  center      |
  start       |
  end         |
  self-start  |
  self-end    |
  flex-start  |
  flex-end
```

Esta sintaxe reflete o padrão mais recente conforme CSS Box Alignment Module Level 3. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplo

CSS

```
#container {
  height: 200px;
  width: 240px;
  align-items: center; /* Pode ser mudado na amostra ao vivo */
  background-color: #8c8c8c;
}

.flex {
  display: flex;
  flex-wrap: wrap;
}

.grid {
  display: grid;
```

```
  grid-template-columns: repeat(auto-fill, 50px);
}

div > div {
  box-sizing: border-box;
  border: 2px solid #8c8c8c;
  width: 50px;
  display: flex;
  align-items: center;
  justify-content: center;
}

#item1 {
  background-color: #8cffa0;
  min-height: 30px;
}

#item2 {
  background-color: #a0c8ff;
  min-height: 50px;
}

#item3 {
  background-color: #ffa08c;
  min-height: 40px;
}

#item4 {
  background-color: #ffff8c;
  min-height: 60px;
}

#item5 {
  background-color: #ff8cff;
  min-height: 70px;
}

#item6 {
  background-color: #8cffff;
  min-height: 50px;
  font-size: 30px;
}

select {
  font-size: 16px;
}

.row {
  margin-top: 10px;
}
```

HTML

```
<div id="container" class="flex">
  <div id="item1">1</div>
  <div id="item2">2</div>
  <div id="item3">3</div>
  <div id="item4">4</div>
  <div id="item5">5</div>
  <div id="item6">6</div>
</div>

<div class="row">
  <label for="display">display: </label>
  <select id="display">
    <option value="flex">flex</option>
    <option value="grid">grid</option>
  </select>
</div>

<div class="row">
  <label for="values">align-items: </label>
  <select id="values">
```

```
    <option value="normal">normal</option>
    <option value="flex-start">flex-start</option>
    <option value="flex-end">flex-end</option>
    <option value="center" selected>center</option>
    <option value="baseline">baseline</option>
    <option value="stretch">stretch</option>

    <option value="start">start</option>
    <option value="end">end</option>
    <option value="self-start">self-start</option>
    <option value="self-end">self-end</option>
    <option value="left">left</option>
    <option value="right">right</option>

    <option value="first baseline">first baseline</option>
    <option value="last baseline">last baseline</option>

    <option value="safe center">safe center</option>
    <option value="unsafe center">unsafe center</option>
    <option value="safe right">safe right</option>
    <option value="unsafe right">unsafe right</option>
    <option value="safe end">safe end</option>
    <option value="unsafe end">unsafe end</option>
    <option value="safe self-end">safe self-end</option>
    <option value="unsafe self-end">unsafe self-end</option>
    <option value="safe flex-end">safe flex-end</option>
    <option value="unsafe flex-end">unsafe flex-end</option>
  </select>
</div>
```

```
var values = document.getElementById("values");
var display = document.getElementById("display");
var container = document.getElementById("container");

values.addEventListener("change", function (evt) {
  container.style.alignItems = evt.target.value;
});

display.addEventListener("change", function (evt) {
  container.className = evt.target.value;
});
```

Resultado

Compatibilidade com navegadores

Veja também

- CSS Flexbox Guide: *Basic Concepts of Flexbox*
- CSS Flexbox Guide: *Aligning items in a flex container*
- CSS Grid Guide: *Box alignment in CSS Grid layouts*
- CSS Box Alignment
- The `align-self` property

align-self

A propriedade CSS **`align-self`** alinha itens-flex da linha flex alvo, sobreescrevendo o valor `align-items`. Se alguma dos eixos das margens do dado item está estabelecido como `auto`, então `align-self` é ignorado.

Experimente

```
align-self: stretch;
```

```
align-self: center;
```

```
align-self: start;
```

```
align-self: end;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">One</div>
    <div>Two</div>
    <div>Three</div>
  </div>
</section>
```

```
.example-container {
  border: 1px solid #c5c5c5;
  display: grid;
  width: 200px;
  grid-template-columns: 1fr 1fr;
  grid-auto-rows: 80px;
  grid-gap: 10px;
}

.example-container > div {
  background-color: rgba(0, 0, 255, 0.2);
  border: 3px solid blue;
}
```

A propriedade não se aplica a caixas *block-level*, ou células de tabela.

Sintaxe

```
/* valores de palavras-chave */
align-self: auto;
align-self: normal;

/* Alinhamento de posicionamento */
/* align-self não recebe valores left e right */
align-self: center; /* Coloca o item em torno do centro */
align-self: start; /* Coloca o item no início */
align-self: end; /* Coloca o item no fim */
align-self: self-start; /* Alinha o item flush no início */
align-self: self-end; /* Alinha o item flush no fim */
align-self: flex-start; /* Coloca o item flex no início */
align-self: flex-end; /* Coloca o item flex no fim */

/* Alinhamento a partir da baseline */
align-self: baseline;
align-self: first baseline;
align-self: last baseline;
align-self: stretch; /* Estica itens de tamanho 'auto' para encaixar no container */

/* Alinhamento com Oveflow */
align-self: safe center;
align-self: unsafe center;

/* Valores globais */
align-self: inherit;
align-self: initial;
align-self: unset;
```

Valores

`auto`

O valor `align-items` é originado do valor de seu pai.

`normal`

O efeito dessa palavra-chave depende do modo de layout em que nos encontramos:

- Em layouts absolutamente posicionados, a palavra-chave se comporta como `start` em caixas absolutamente posicionadas *replaced*, e como `stretch` em caixas absolutamente posicionadas *all other*.
- Em posição estática de layouts absolutamente posicionados, a palavra-chave se comporta como `stretch`.
- Para itens-flex, a palavra-chave se comporta como `stretch`.

- Para itens-grid, essa palavra-chave leva ao comportamento similar ao de `stretch`, com exceção de caixas com aspect ratio ou tamanhos intrínsecos onde se comportam como `start`.
- A propriedade não se aplica para caixas *block-level*, e para células de tabela.

`self-start`

Alinha os itens de maneira a mesclar a borda do alinhamento do container correspondente com o lado inicial do item no eixo.

`self-end`

Alinha os itens de maneira a mesclar a borda do alinhamento do container correspondente com o lado final do item no eixo.

`flex-start`

A borda da margem inicial do item flex é mesclada com a borda inicial da linha.

`flex-end`

A borda da margem fim do item flex é mesclada com a borda fim da linha.

`center`

A caixa-margem do item flex é centralizada ao longo da linha no eixo transversal. Se o tamanho do item é superior to container flex, então ele irá transbordar de maneira equivalente em ambas direções.

`baseline first baseline` `last baseline`

Specifies participation in first- or last-baseline alignment: aligns the alignment baseline of the box's first or last baseline set with the corresponding baseline in the shared first or last baseline set of all the boxes in its baseline-sharing group. The fallback alignment for `first baseline` is `start`, the one for `last baseline` is `end`.

`stretch`

Caso o tamanho combinado dos itens ao longo do eixo transversal seja menor do que o tamanho do container de alinhamento, e caso o tamanho do item segue o estilo `auto`, seu tamanho cresce igualmente (mas não proporcionalmente), enquanto que ainda respeitando as restrições impostas por `max-height`/`max-width` (ou a funcionalidade equivalente), de maneira que o tamanho combinado de todos itens tamanho-`auto` preencha exatamente o container de alinhamento ao longo do eixo transversal.

`safe`

Caso o tamanho do item supere o alinhamento do container, o item é alinhado como se o modo de alinhamento fosse `start`.

`unsafe`

Independente dos tamanhos relativos do item e do alinhamento do container, o valor do alinhmento dado é mantido.

Sintaxe formal

```
align-self =
  auto                                          |
  <overflow-position>? [ normal | <self-position> ]  |
  stretch                                       |
  <baseline-position>                           |
  anchor-center

<overflow-position> =
  unsafe  |
  safe

<self-position> =
  center      |
  start       |
  end         |
  self-start  |
  self-end    |
  flex-start  |
  flex-end

<baseline-position> =
  [ first | last ]?  &&
  baseline
```

Esta sintaxe reflete o padrão mais recente conforme CSS Box Alignment Module Level 3, CSS Specification. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplo

HTML

```
<section>
  <div>Item #1</div>
  <div>Item #2</div>
  <div>Item #3</div>
</section>
```

CSS

```
section {
  display: flex;
  align-items: center;
  height: 120px;
  background: beige;
}

div {
  height: 60px;
  background: cyan;
  margin: 5px;
}

div:nth-child(3) {
  align-self: flex-end;
  background: pink;
}
```

Resultado

Veja também

- CSS Flexbox Guide: *Basic Concepts of Flexbox*
- CSS Flexbox Guide: *Aligning items in a flex container*
- CSS Grid Guide: *Box alignment in CSS Grid layouts*
- CSS Box Alignment
- The `align-items` property

animation

A propriedade CSS abreviada **`animation`**, aplica uma animação entre estilos. É uma abreviação de `animation-name`, `animation-duration`, `animation-timing-function`, `animation-delay`, `animation-iteration-count`, `animation-direction`, `animation-fill-mode`, e `animation-play-state`.

Experimente

```
animation: 3s ease-in 1s infinite reverse both running slidein;
```

```
animation: 3s linear 1s infinite running slidein;
```

```
animation: 3s linear 1s infinite alternate slidein;
```

```
animation: 0.5s linear 1s infinite alternate slidein;
```

```
<section class="flex-column" id="default-example">
  <div id="example-element"></div>
</section>
```

```
#example-element {
  background-color: #1766aa;
  margin: 20px;
  border: 5px solid #333;
  width: 150px;
  height: 150px;
  border-radius: 50%;
}
```

```
@keyframes slidein {
  from {
    margin-left: -20%;
  }
  to {
    margin-left: 100%;
  }
}
```

Propriedades constituintes

Esta propriedade é uma abreviação para as seguintes propriedades CSS:

- `animation-delay`
- `animation-direction`
- `animation-duration`
- `animation-fill-mode`
- `animation-iteration-count`
- `animation-name`
- `animation-play-state`
- `animation-timing-function`

Sintaxe

```
/* @keyframes duration | easing-function | delay |
iteration-count | direction | fill-mode | play-state | name */
animation: 3s ease-in 1s 2 reverse both paused slidein;

/* @keyframes duration | easing-function | delay | name */
animation: 3s linear 1s slidein;

/* two animations */
animation:
  3s linear slidein,
  3s ease-out 5s slideout;
```

A propriedade `animation` é especificada como uma ou mais animações separadas por vírgulas.

Cada animação individual é especificada como:

- zero, uma ou duas ocorrências do valor `<time>`
- zero ou uma ocorrência dos seguintes valores:
    - `<single-easing-function>`
    - `<single-animation-iteration-count>`
    - `<single-animation-direction>`
    - `<single-animation-fill-mode>`
    - `<single-animation-play-state>`
- um nome opcional para a animação que pode ser `none`, um `<custom-ident>` ou uma `<string>`

Valores

`<single-easing-function>`

Determina o tipo de transição. O valor deve ser um dos disponíveis em `<easing-function>`.

`<single-animation-iteration-count>`

O número de vezes que a animação é reproduzida. O valor deve ser um dos disponíveis em `animation-iteration-count`.

`<single-animation-direction>`

A direção em que a animação é reproduzida. O valor deve ser um dos disponíveis em `animation-direction`.

`<single-animation-fill-mode>`

Determina como os estilos devem ser aplicados no alvo da animação antes e depois de sua execução. O valor deve ser um dos disponíveis em `animation-fill-mode`.

`<single-animation-play-state>`

Determina se a animação está sendo reproduzida ou não. O valor deve ser um dos disponíveis em `animation-play-state`.

Descrição

A ordem dos valores de tempo dentro de cada definição de animação é importante: o primeiro valor que pode ser analisado como `<time>` é atribuído ao `animation-duration` , e o segundo é atribuído a `animation-delay`.

A ordem de outros valores dentro de cada definição de animação também é importante para distinguir um valor `animation-name` de outros valores. Se um valor na abreviação `animation` puder ser analisado como um valor para uma propriedade de animação diferente de `animation-name`, o valor será aplicado a essa propriedade primeiro e não a `animation-name`. Por esta razão, a prática recomendada é especificar um valor para `animation-name` como o último valor em uma lista de valores ao usar a abreviação `animation`; isso é válido mesmo quando você especifica várias animações separadas por vírgula usando a abreviação `animation`.

Um valor `animation-name` não precisa ser declarado na propriedade abreviada `animation`. Se nenhum nome existir, não haverá animação para aplicar em nenhuma das propriedades.

Quando o valor `animation-duration` é omitido da propriedade abreviada `animation`, o valor padrão dessa propriedade é `0s`. Nesse caso, a animação ainda ocorrerá (o `animationStart` e [`animationEnd`](/pt-BR/docs/Web/API/ Element/animationend_event) eventos serão acionados), mas nenhuma animação será visível.

Preocupações com acessibilidade

A animação piscando e piscando pode ser problemática para pessoas com problemas cognitivos, como o Transtorno de Déficit de Atenção e Hiperatividade (TDAH). Além disso, certos tipos de movimento podem ser um gatilho para distúrbios vestibulares, epilepsia e enxaqueca e sensibilidade escotópica.

Considere fornecer um mecanismo para pausar ou desabilitar a animação, bem como usar a Consulta de mídia de movimento reduzido para criar uma experiência complementar para usuários que têm expressou uma preferência por experiências animadas reduzidas.

- Projetando uma animação na Web mais segura para sensibilidade ao movimento · Um artigo separado
- Uma introdução à consulta de mídia de movimento reduzido | CSS-Tricks
- Design responsivo para movimento | WebKit
- MDN Understanding WCAG, Diretriz 2.2 explicações
- Entendendo o Critério de Sucesso 2.2.2 | W3C Understanding WCAG 2.0

Definição formal

Initial value	as each of the properties of the shorthand: • `animation-name`: `none` • `animation-duration`: `0s` • `animation-timing-function`: `ease` • `animation-delay`: `0s` • `animation-iteration-count`: `1` • `animation-direction`: `normal` • `animation-fill-mode`: `none` • `animation-play-state`: `running` • `animation-timeline`: `auto`
Aplica-se a	all elements
Inherited	não
Computed value	as each of the properties of the shorthand: • `animation-name`: as specified • `animation-duration`: as specified • `animation-timing-function`: as specified • `animation-delay`: as specified • `animation-direction`: as specified • `animation-iteration-count`: as specified • `animation-fill-mode`: as specified • `animation-play-state`: as specified • `animation-timeline`: a list, each item either a case-sensitive CSS identifier or the keywords `none`, `auto`

Animation type	Not animatable

Sintaxe formal

```
animation =
  <single-animation>#

<single-animation> =
  <'animation-duration'>              ||
  <easing-function>                   ||
  <'animation-delay-start'>           ||
  <single-animation-iteration-count>  ||
  <single-animation-direction>        ||
  <single-animation-fill-mode>        ||
  <single-animation-play-state>       ||
  [ none | <keyframes-name> ]         ||
  <single-animation-timeline>

<animation-duration> =
  [ auto | <time [0s,∞]> ]#

<easing-function> =
  <linear-easing-function>        |
  <cubic-bezier-easing-function>  |
  <step-easing-function>

<animation-delay-start> =
  <time>#

<single-animation-iteration-count> =
  infinite        |
  <number [0,∞]>

<single-animation-direction> =
  normal             |
  reverse            |
  alternate          |
  alternate-reverse

<single-animation-fill-mode> =
  none       |
  forwards   |
  backwards  |
  both

<single-animation-play-state> =
  running  |
  paused

<keyframes-name> =
  <custom-ident>  |
  <string>

<single-animation-timeline> =
  auto            |
  none            |
  <dashed-ident>  |
  <scroll()>      |
  <view()>

<linear-easing-function> =
  linear      |
  <linear()>

<cubic-bezier-easing-function> =
  ease              |
  ease-in           |
  ease-out          |
  ease-in-out       |
  <cubic-bezier()>

<step-easing-function> =
  step-start  |
  step-end    |
  <steps()>

<scroll()> =
```

```
  scroll( [ <scroller> || <axis> ]? )

<view()> =
  view( [ <axis> || <'view-timeline-inset'> ]? )

<linear()> =
  linear( [ <number> && <percentage>{0,2} ]# )

<cubic-bezier()> =
  cubic-bezier( [ <number [0,1]> , <number> ]#{2} )

<steps()> =
  steps( <integer> , <step-position>? )

<scroller> =
  root     |
  nearest  |
  self

<axis> =
  block   |
  inline  |
  x       |
  y

<view-timeline-inset> =
  [ [ auto | <length-percentage> ]{1,2} ]#

<integer> =
  <number-token>

<step-position> =
  jump-start  |
  jump-end    |
  jump-none   |
  jump-both   |
  start       |
  end

<length-percentage> =
  <length>      |
  <percentage>
```

Esta sintaxe reflete o padrão mais recente conforme CSS Animations Level 1, CSS Animations Level 2, CSS Easing Functions Level 2, CSS Values and Units Module Level 4, Scroll-driven Animations. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

> **Nota:** A animação das propriedades do CSS Box Model não é recomendada. A animação de qualquer propriedade de box model é inerentemente para a CPU; considere animar a propriedade transform.

Nascer do Sol

Aqui animamos um sol amarelo em um céu azul claro. O sol nasce para o centro da viewport e depois desaparece de vista.

```
<div class="sun"></div>
```

```
:root {
  overflow: hidden; /* esconde qualquer parte do sol abaixo do horizonte */
  background-color: lightblue;
  display: flex;
  justify-content: center; /* centraliza o sol ao fundo */
}

.sun {
  background-color: yellow;
  border-radius: 50%; /* cria um fundo circular */
  height: 100vh; /* torna o sol do tamanho da viewport */
  aspect-ratio: 1 / 1;
  animation: 4s linear 0s infinite alternate sun-rise;
}
```

```
@keyframes sun-rise {
  from {
    /* empurra o sol para além da janela de visualização */
    transform: translateY(110vh);
  }
  to {
    /* retorna o sol à sua posição padrão */
    transform: translateY(0);
  }
}
```

Animando Múltiplas Propriedades

Adicionando à animação do sol no exemplo anterior, adicionamos uma segunda animação mudando a cor do sol conforme ele nasce e se põe. O sol começa vermelho escuro quando está abaixo do horizonte e muda para laranja brilhante quando atinge o topo.

```
<div class="sun"></div>
```

```
:root {
  overflow: hidden;
  background-color: lightblue;
  display: flex;
  justify-content: center;
}

.sun {
  background-color: yellow;
  border-radius: 50%;
  height: 100vh;
  aspect-ratio: 1 / 1;
  animation: 4s linear 0s infinite alternate animating-multiple-properties;
}

/* é possível animar várias propriedades em uma única animação */
@keyframes animating-multiple-properties {
  from {
    transform: translateY(110vh);
    background-color: red;
    filter: brightness(75%);
  }
  to {
    transform: translateY(0);
    background-color: orange;
    /* propriedades não definidas, ou seja, 'filtro' reverterá para os valores padrão */
  }
}
```

Aplicando Múltiplas Animações

Aqui está um sol que nasce e se põe sobre um fundo azul claro. O sol gira gradualmente através de um arco-íris de cores. O tempo do sol, posição e cor são independentes.

```
<div class="sun"></div>
```

```
:root {
  overflow: hidden;
  background-color: lightblue;
  display: flex;
  justify-content: center;
}

.sun {
  background-color: yellow;
  border-radius: 50%;
  height: 100vh;
  aspect-ratio: 1 / 1;
  /* várias animações são separadas por vírgulas, os parâmetros de cada animação são definidos independentemente */
  animation:
    4s linear 0s infinite alternate rise,
    24s linear 0s infinite psychedelic;
```

```
}

@keyframes rise {
  from {
    transform: translateY(110vh);
  }
  to {
    transform: translateY(0);
  }
}

@keyframes psychedelic {
  from {
    filter: hue-rotate(0deg);
  }
  to {
    filter: hue-rotate(360deg);
  }
}
```

Animações Múltiplas em Cascata

Aqui está um sol amarelo sobre um fundo azul claro. O sol salta entre as lados esquerdo e direito da viewport. O sol permanece na janela de visualização mesmo embora uma animação de ascensão seja definida. A propriedade de transformação da animação de subida é 'substituído' pela animação de salto.

```
<div class="sun"></div>
```

```
:root {
  overflow: hidden;
  background-color: lightblue;
  display: flex;
  justify-content: center;
}

.sun {
  background-color: yellow;
  border-radius: 50%;
  height: 100vh;
  aspect-ratio: 1 / 1;
  /*
    as animações declaradas posteriormente na cascata substituirão as
    propriedades de animações declaradas anteriormente
  */
  /* bounce 'substitui' a transformação definida por ascensão, portanto, o sol só se move horizontalmente */
  animation:
    4s linear 0s infinite alternate rise,
    4s linear 0s infinite alternate bounce;
}

@keyframes rise {
  from {
    transform: translateY(110vh);
  }
  to {
    transform: translateY(0);
  }
}

@keyframes bounce {
  from {
    transform: translateX(-50vw);
  }
  to {
    transform: translateX(50vw);
  }
}
```

Veja Usando animações CSS para exemplos adicionais.

Veja também

- Usando animações CSS
- API JavaScript `AnimationEvent`

animation-delay

A propriedade CSS **`animation-delay`** especifica quando uma animação deve começar. Você pode começar a animação no futuro, imediatamente ou desde o início, ou imediatamente e parcialmente através do ciclo de animação.

```
/* Uma animação */
animation-delay: 3s;
animation-delay: 0s;
animation-delay: -1500ms;

/* Várias animações */
animation-delay: 2.1s, 480ms;
```

Muitas vezes é conveniente usar a propriedade abreviada `animation` para definir todas as propriedades de animação de uma só vez.

Initial value	`0s`
Aplica-se a	all elements, `::before` and `::after` pseudo-elements
Inherited	não
Computed value	as specified
Animation type	Not animatable

Sintaxe

Valores

`<time>`

O deslocamento do tempo, a partir do momento em que a animação é aplicada ao elemento, no qual a animação deve começar. Isso pode ser especificado em segundos (`s`) ou milesegundos or milliseconds (`ms`). A unidade é necessaria.

Um valor positivo indica que a animação deve começar após o período de especificação ter decorrido. O valor de `0s`, que é o padrão, incida que a animação deve começar desde que foi aplicada.

Um valor negativo faz com que a animação comece imediatamente, mas parcialmente através do seu ciclo. Por exemplo, se você específicar `-1s` como tempo da animation-delay , a animação vai começar imediatamente, mas começará 1 segundo na sequência de animação. Se você específicar um valor negativo para a animation-delay, mas o valor inicial é implícito, o valor inicial é retirado do momento em que a animação é aplicada ao elemento.

> **Nota:** Quando você especifica vários valores separados por vírgula em uma propriedade `animation-*` , eles serão atribuídos às animações especificadas na propriedade `animation-name` em diferentes modos, dependendo de quantos existam. Para mais informações veja Setting multiple animation property values.

Sintaxe formal

```
animation-delay =
  [ <'animation-delay-start'> <'animation-delay-end'>? ]#

<animation-delay-start> =
  <time>#

<animation-delay-end> =
  <time [0s,∞]>#
```

Esta sintaxe reflete o padrão mais recente conforme CSS Animations Level 2. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

Veja CSS animations para exemplos.

Veja também

- Using CSS animations
- JavaScript `AnimationEvent` API

backdrop-filter

A propriedade CSS **`backdrop-filter`** permite que você aplique efeitos gráficos, como desfoque ou mudança de cores, na área ao fundo de um elemento. Como ela se aplica a tudo que fica *atrás* do elemento, é preciso deixá-lo (ou o seu plando de fundo) ao menos parcialmente transparente para poder ver o efeito em ação.

```
/* Valor de palavra-chave */
backdrop-filter: none;

/* Filtro com URL para SVG */
backdrop-filter: url(commonfilters.svg#filter);

/* Valores <filter-function> */
backdrop-filter: blur(2px);
backdrop-filter: brightness(60%);
backdrop-filter: contrast(40%);
backdrop-filter: drop-shadow(4px 4px 10px blue);
backdrop-filter: grayscale(30%);
backdrop-filter: hue-rotate(120deg);
backdrop-filter: invert(70%);
backdrop-filter: opacity(20%);
backdrop-filter: sepia(90%);
backdrop-filter: saturate(80%);

/* Vários filtros */
backdrop-filter: url(filters.svg#filter) blur(4px) saturate(150%);

/* Valores globais */
backdrop-filter: inherit;
backdrop-filter: initial;
backdrop-filter: unset;
```

Sintaxe

Valores

`none`

Nenhum filtro é aplicado ao plano de fundo.

`<filter-function-list>`

Uma lista de `<filter-function>`s separadas por espaços ou um filtro SVG que será aplicado ao plano de fundo.

Definição formal

Initial value	none
Aplica-se a	all elements; In SVG, it applies to container elements excluding the `<defs>` element and all graphics elements
Inherited	não
Computed value	as specified
Animation type	a filter function list

Sintaxe formal

```
backdrop-filter =
  none                |
  <filter-value-list>

<filter-value-list> =
  [ <filter-function> | <url> ]+

<filter-function> =
  <blur()>          |
```

```
  <brightness()>   |
  <contrast()>     |
  <drop-shadow()>  |
  <grayscale()>    |
  <hue-rotate()>   |
  <invert()>       |
  <opacity()>      |
  <sepia()>        |
  <saturate()>

<blur()> =
  blur( <length>? )

<brightness()> =
  brightness( [ <number> | <percentage> ]? )

<contrast()> =
  contrast( [ <number> | <percentage> ]? )

<drop-shadow()> =
  drop-shadow( [ <color>? && <length>{2,3} ] )

<grayscale()> =
  grayscale( [ <number> | <percentage> ]? )

<hue-rotate()> =
  hue-rotate( [ <angle> | <zero> ]? )

<invert()> =
  invert( [ <number> | <percentage> ]? )

<opacity()> =
  opacity( [ <number> | <percentage> ]? )

<sepia()> =
  sepia( [ <number> | <percentage> ]? )

<saturate()> =
  saturate( [ <number> | <percentage> ]? )
```

Esta sintaxe reflete o padrão mais recente conforme Filter Effects Module Level 1, Filter Effects Module Level 2. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplo

CSS

```
.box {
  background-color: rgba(255, 255, 255, 0.3);
  border-radius: 5px;
  font-family: sans-serif;
  text-align: center;
  line-height: 1;
  -webkit-backdrop-filter: blur(10px);
  backdrop-filter: blur(10px);
  max-width: 50%;
  max-height: 50%;
  padding: 20px 40px;
}

html,
body {
  height: 100%;
  width: 100%;
}

body {
  background-image: url(https://picsum.photos/id/1080/6858/4574);
  background-position: center center;
  background-repeat: no-repeat;
  background-size: cover;
}

.container {
  align-items: center;
```

```
  display: flex;
  justify-content: center;
  height: 100%;
  width: 100%;
}
```

HTML

```
<div class="container">
  <div class="box">
    <p>backdrop-filter: blur(10px)</p>
  </div>
</div>
```

Resultado

Veja também

- `filter`

backface-visibility

> **Experimental: Esta é uma tecnologia experimental**
> Verifique a tabela de compatibilidade entre Navegadores cuidadosamente antes de usar essa funcionalidade em produção.

Sumário

A propriedade CSS **`backface-visibility`** determina se deve ser exibido ou não a parte de trás do elemento para o usuário. A parte de trás de um elemento é sempre um fundo transparente, deixando, quando visível, uma imagem espelho da parte da frente a ser exibida.

Há casos quando nós não queremos que a parte da frente de um elemento seja visível pela parte de trás, quando estamos fazendo um efeito de giro de cartões (colocando dois elementos lado a lado).

Esta propriedade não possui nenhum efeito em transformações 2D quando não há nenhum tipo de perspectiva.

Initial value	`visible`
Aplica-se a	qualquer elemento transformavel
Inherited	não
Computed value	as specified
Animation type	discrete

Sintaxe

```
backface-visibility: visible;
backface-visibility: hidden;
```

Valores

Esta propriedade tem dois valores (exclusivos):

- `visible` significa que a parte de trás estará visível, permitindo que a parte da frente seja exibida como espelho
- `hidden` significa que a parte de trás não é visível, sendo então escondida pela parte da frente do elemento

Sintaxe formal

```
backface-visibility =
  visible  |
  hidden
```

Esta sintaxe reflete o padrão mais recente conforme CSS Transforms Module Level 2. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

Cubos com faces transparentes

Conteúdo HTML

```
<table>
  <tbody>
    <tr>
      <th><code>backface-visibility: visible;</code></th>
      <th><code>backface-visibility: hidden;</code></th>
    </tr>
    <tr>
      <td>
        <div class="container">
          <div class="cube showbf">
            <div class="face front">1</div>
            <div class="face back">2</div>
            <div class="face right">3</div>
            <div class="face left">4</div>
            <div class="face top">5</div>
            <div class="face bottom">6</div>
          </div>
        </div>
        <p>
          Todas as faces são transparentes e há três faces atrás que estão
          visíveis através das faces da frente.
        </p>
      </td>
      <td>
        <div class="container">
          <div class="cube hidebf">
            <div class="face front">1</div>
            <div class="face back">2</div>
            <div class="face right">3</div>
            <div class="face left">4</div>
            <div class="face top">5</div>
            <div class="face bottom">6</div>
          </div>
        </div>
        <p>
          Nenhuma face é opaca, mas as três faces de trás estarão sempre
          escondidas agora.
        </p>
      </td>
    </tr>
  </tbody>
</table>
```

Conteúdo CSS

```
/* Shorthand classes that will show or hide the three back faces of the "cube" */
.hidebf div {
  backface-visibility: hidden;
  -webkit-backface-visibility: hidden;
}

.showbf div {
  backface-visibility: visible;
  -webkit-backface-visibility: visible;
}

/* Define the container div, the cube div, and a generic face */
.container {
  width: 150px;
  height: 150px;
  margin: 75px 0 0 75px;
  border: none;
}

.cube {
  width: 100%;
  height: 100%;
```

```
  perspective: 550px;
  perspective-origin: 150% 150%;
  transform-style: preserve-3d;
  -webkit-perspective: 300px;
  -webkit-perspective-origin: 150% 150%;
  -webkit-transform-style: preserve-3d;
}

.face {
  display: block;
  position: absolute;
  width: 100px;
  height: 100px;
  border: none;
  line-height: 100px;
  font-family: sans-serif;
  font-size: 60px;
  color: white;
  text-align: center;
}

/* Define each face based on direction */
.front {
  background: rgba(0, 0, 0, 0.3);
  transform: translateZ(50px);
  -webkit-transform: translateZ(50px);
}

.back {
  background: rgba(0, 255, 0, 1);
  color: black;
  transform: rotateY(180deg) translateZ(50px);
  -webkit-transform: rotateY(180deg) translateZ(50px);
}

.right {
  background: rgba(196, 0, 0, 0.7);
  transform: rotateY(90deg) translateZ(50px);
  -webkit-transform: rotateY(90deg) translateZ(50px);
}

.left {
  background: rgba(0, 0, 196, 0.7);
  transform: rotateY(-90deg) translateZ(50px);
  -webkit-transform: rotateY(-90deg) translateZ(50px);
}

.top {
  background: rgba(196, 196, 0, 0.7);
  transform: rotateX(90deg) translateZ(50px);
  -webkit-transform: rotateX(90deg) translateZ(50px);
}

.bottom {
  background: rgba(196, 0, 196, 0.7);
  transform: rotateX(-90deg) translateZ(50px);
  -webkit-transform: rotateX(-90deg) translateZ(50px);
}

/* Make the table a little nicer */
th,
p,
td {
  background-color: #eeeeee;
  margin: 0px;
  padding: 6px;
  font-family: sans-serif;
  text-align: left;
}
```

Resultado

Veja também

- Usando transformações CSS

background

Resumo

A propriedade CSS **`background`** é um atalho para definir os valores de fundo individuais em um único lugar na folha de estilo. Background pode ser usado para definir os valores para um ou mais dos seguintes: `background-clip`, `background-color`, `background-image`, `background-origin`, `background-position` , `background-repeat`, `background-size` e `background-attachment`.

O `fundo` CSS propriedade estenográfica atribui valores dados explícitos e conjuntos de propriedades para seus valores iniciais em falta.

Initial value	as each of the properties of the shorthand: • `background-image`: `none` • `background-position`: `0% 0%` • `background-size`: `auto auto` • `background-repeat`: `repeat` • `background-origin`: `padding-box` • `background-clip`: `border-box` • `background-attachment`: `scroll` • `background-color`: `transparent`
Aplica-se a	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	não
Percentages	as each of the properties of the shorthand: • `background-position`: refer to the size of the background positioning area minus size of background image; size refers to the width for horizontal offsets and to the height for vertical offsets • `background-size`: relative to the background positioning area
Computed value	as each of the properties of the shorthand: • `background-image`: as specified, but with `<url>` values made absolute • `background-position`: as each of the properties of the shorthand: ◦ `background-position-x`: A list, each item consisting of: an offset given as a combination of an absolute length and a percentage, plus an origin keyword ◦ `background-position-y`: A list, each item consisting of: an offset given as a combination of an absolute length and a percentage, plus an origin keyword • `background-size`: as specified, but with relative lengths converted into absolute lengths • `background-repeat`: a list, each item consisting of two keywords, one per dimension • `background-origin`: as specified • `background-clip`: as specified • `background-attachment`: as specified • `background-color`: computed color
Animation type	as each of the properties of the shorthand: • `background-color`: a color • `background-image`: discrete • `background-clip`: a repeatable list • `background-position`: a repeatable list • `background-size`: a repeatable list • `background-repeat`: discrete • `background-attachment`: discrete

Sintaxe

```
/ * Usando uma <cor-de-fundo> * /
background: green;
```

```
/ * Usando uma <imagem-de-fundo> e <estilo-de-repetição> * /
background: url("test.jpg") repeat-y;

/ * Usando uma <box> e <cor-de-fundo> * /
background: border-box red;

/ * Uma única imagem, centrado e escalado * /
background: no-repeat center/80% url( "../img/image.png");
```

> **Nota:** O `background-color` só pode ser definido no último fundo, como há apenas uma cor de fundo para todo o elemento.

Valores

Um ou mais dos seguintes, por qualquer ordem:

`<anexo>`

Veja `background-attachment`

`<box>`

Veja `background-clip`

`<cor-de-fundo>`

Veja `background-color`

`<imagem-de-fundo>`

Veja `background-image`

`<posição>`

Veja `background-position`

`<estilo-de-repetição>`

Veja `background-repeat`

`<tamanho-do-fundo>`

Veja `background-size`. Esta propriedade deve ser especificado após **<posição>** , separados com o caractere '/'.

Sintaxe formal

```
background =
  <bg-layer>#? , <final-bg-layer>

<bg-layer> =
  <bg-image>                      ||
  <bg-position> [ / <bg-size> ]?  ||
  <repeat-style>                  ||
  <attachment>                    ||
  <visual-box>                    ||
  <bg-clip>

<final-bg-layer> =
  <bg-image>                      ||
  <bg-position> [ / <bg-size> ]?  ||
  <repeat-style>                  ||
  <attachment>                    ||
  <visual-box>                    ||
  <bg-clip>                       ||
  <'background-color'>

<bg-image> =
  <image>  |
  none

<bg-position> =
  <position>        |
  <position-three>
```

```
<bg-size> =
  [ <length-percentage [0,∞]> | auto ]{1,2}  |
  cover                                      |
  contain

<repeat-style> =
  repeat-x           |
  repeat-y           |
  repeat-block       |
  repeat-inline      |
  <repetition>{1,2}

<attachment> =
  scroll  |
  fixed   |
  local

<visual-box> =
  content-box  |
  padding-box  |
  border-box

<bg-clip> =
  <visual-box>             |
  [ border-area || text ]

<background-color> =
  <color>

<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
  <gradient>

<position> =
  <position-one>   |
  <position-two>   |
  <position-four>

<position-three> =
  [ left | center | right ] && [ [ top | bottom ] <length-percentage> ]  |
  [ [ left | right ] <length-percentage> ] && [ top | center | bottom ]

<length-percentage> =
  <length>      |
  <percentage>

<repetition> =
  repeat     |
  space      |
  round      |
  no-repeat

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-set()> =
  image-set( <image-set-option># )

<cross-fade()> =
  cross-fade( <cf-image># )

<element()> =
  element( <id-selector> )

<position-one> =
  left                |
  center              |
  right               |
  top                 |
  bottom              |
  x-start             |
  x-end               |
```

```
  y-start             |
  y-end               |
  block-start         |
  block-end           |
  inline-start        |
  inline-end          |
  <length-percentage>

<position-two> =
  [ left | center | right | x-start | x-end ] && [ top | center | bottom | y-start | y-end ]  |
  [ left | center | right | x-start | x-end | <length-percentage> ] [ top | center | bottom | y-start | y-end | <length-percentage> ]  |
  [ block-start | center | block-end ] && [ inline-start | center | inline-end ]  |
  [ start | center | end ]{2}

<position-four> =
  [ [ left | right | x-start | x-end ] <length-percentage> ] && [ [ top | bottom | y-start | y-end ] <length-percentage> ]  |
  [ [ block-start | block-end ] <length-percentage> ] && [ [ inline-start | inline-end ] <length-percentage> ]  |
  [ [ start | end ] <length-percentage> ]{2}

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<id-selector> =
  <hash-token>
```

Esta sintaxe reflete o padrão mais recente conforme CSS Backgrounds Module Level 4, CSS Box Model Module Level 4, CSS Images Module Level 4, CSS Images Module Level 5, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5, Selectors Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

HTML

```
<p class="Topbanner">
  céu estrelado<br />
  Cintilando Cintilando<br />
  Céu estrelado
</p>
<p class="atencao">Este é um parágrafo</p>
<p></p>
```

CSS

```
.atencao {
  background: red;
}

.Topbanner {
  background: url("star-solid.gif") #00d repeat-y fixed;
}
```

Resultado

Veja também

- - moz-fundo-inline-política
- Usando gradientes
- Vários fundos

background-attachment

Resumo

Se um `background-image` é especificado, a propriedade CSS **`background-attachment`** determina se aquela posição da imagem é fixa com o viewport, ou se irá rolar juntamente com o seu bloco.

Initial value	`scroll`
Aplica-se a	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	não
Computed value	as specified
Animation type	discrete

Sintaxe

```
background-attachment: scroll;
background-attachment: fixed;
background-attachment: local;

background-attachment: inherit;
```

Valores

`fixed`

> Essa chave significa que o background é fixo em relação ao viewport. Mesmo que um elemento tenha um mecanismo de rolar, o background 'fixed' não movimenta com o elemento.

`local`

> Essa chave significa que o background é fixo em relação ao conteúdo do elemento: ise ele tem um mecanismo de rolar, o background rola com o conteúdo do elemento, e a área pintada e o posicionamento do background são relativos à área de rolagem do elemento ao invés da borda de fronteira deles.

`scroll`

> Essa chave significa que o background é fixo em relação ao elemento em si e não rola com seu conteúdo. (É efetivamente ligado à borda do elemento.)

Formal syntax

```
background-attachment =
  <attachment>#

<attachment> =
  scroll  |
  fixed   |
  local
```

Esta sintaxe reflete o padrão mais recente conforme CSS Backgrounds Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

Exemplo simples

CSS

```
p {
  background-image: url("star-solid.gif");
  background-attachment: fixed;
}
```

HTML

```
<p>
  There were doors all round the hall, but they were all locked; and when Alice
  had been all the way down one side and up the other, trying every door, she
```

```
  walked sadly down the middle, wondering how she was ever to get out again.
</p>
```

Resultado

Suporte de múltiplas imagem de background

Essa propriedade suporta múltiplas imagens de background. Você pode especificar `<attachment>` para cada background, separado por vírgulas. Cada imagem é combinada com o correspondente tipo de anexo, do primeiro especificado ao último.

CSS

```
p {
  background-image: url("star-solid.gif"), url("star-transparent.gif");
  background-attachment: fixed, scroll;
  background-repeat: no-repeat, repeat-y;
}
```

HTML

```
<p>
  There were doors all round the hall, but they were all locked; and when Alice
  had been all the way down one side and up the other, trying every door, she
  walked sadly down the middle, wondering how she was ever to get out again.
  Suddenly she came upon a little three-legged table, all made of solid glass;
  there was nothing on it except a tiny golden key, and Alice's first thought
  was that it might belong to one of the doors of the hall; but, alas! either
  the locks were too large, or the key was too small, but at any rate it would
  not open any of them. However, on the second time round, she came upon a low
  curtain she had not noticed before, and behind it was a little door about
  fifteen inches high: she tried the little golden key in the lock, and to her
  great delight it fitted!
</p>
```

Resultado

Veja também

- Multiple backgrounds

background-blend-mode

A propriedade CSS **`background-blend-mode`** descreve como as imagens de fundo do elemento devem se misturar entre si e a cor de fundo do elemento.

```
/* Um valor */
background-blend-mode: normal;

/* Dois valores, um por fundo */
background-blend-mode: darken, luminosity;

/* Valores globais */
background-blend-mode: initial;
background-blend-mode: inherit;
background-blend-mode: unset;
```

Os modos de mistura devem ser definidos na mesma ordem que a propriedade CSS `background-image`. Se os comprimentos da lista dos modos de mistura e das imagens de fundo não forem iguais, ele será repetido e/ou truncado até que os comprimentos correspondam.

Initial value	`normal`
Aplica-se a	All elements. In SVG, it applies to container elements, graphics elements, and graphics referencing elements.. It also applies to `::first-letter` and `::first-line`.
Inherited	não
Computed value	as specified
Animation type	Not animatable

Sintaxe

Valores

`<blend-mode>`

É um `<blend-mode>` denotando o modo de mesclagem a ser aplicado. Pode haver vários valores, separados por vírgulas.

Sintaxe formal

```
background-blend-mode =
  <'mix-blend-mode'>#

<mix-blend-mode> =
  <blend-mode>  |
  plus-lighter

<blend-mode> =
  normal       |
  darken       |
  multiply     |
  color-burn   |
  lighten      |
  screen       |
  color-dodge  |
  overlay      |
  soft-light   |
  hard-light   |
  difference   |
  exclusion    |
  hue          |
  saturation   |
  color        |
  luminosity
```

Esta sintaxe reflete o padrão mais recente conforme Compositing and Blending Module Level 2. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

```
<div id="div"></div>
<select id="select">
  <option>normal</option>
  <option>multiply</option>
  <option selected>screen</option>
  <option>overlay</option>
  <option>darken</option>
  <option>lighten</option>
  <option>color-dodge</option>
  <option>color-burn</option>
  <option>hard-light</option>
  <option>soft-light</option>
  <option>difference</option>
  <option>exclusion</option>
  <option>hue</option>
  <option>saturation</option>
  <option>color</option>
  <option>luminosity</option>
</select>
```

```
#div {
  width: 300px;
  height: 300px;
  background: url("br.png"), url("tr.png");
  background-blend-mode: screen;
}
```

```
document.getElementById("select").onchange = function (event) {
  document.getElementById("div").style.backgroundBlendMode =
    document.getElementById("select").selectedOptions[0].innerHTML;
};
console.log(document.getElementById("div"));
```

Veja também

- `<blend-mode>`
- `mix-blend-mode`

background-clip

A propriedade CSS **`background-clip`** especifica se o fundo de um elemento, seja cor ou imagem, se extende debaixo de sua área de borda, preenchimento ou conteúdo.

Experimente

```
background-clip: border-box;
```

```
background-clip: padding-box;
```

```
background-clip: content-box;
```

```
background-clip: text;
color: transparent;
```

```
<section id="default-example">
  <div id="example-element">This is the content of the element.</div>
</section>
```

```
#example-element {
  background-image: url("/shared-assets/images/examples/leopard.jpg");
  color: #d73611;
  text-shadow: 2px 2px black;
  padding: 20px;
  border: 10px dashed #333;
  font-size: 2em;
  font-weight: bold;
}
```

Se o elemento não possuir as propriedades `background-image` ou `background-color` definidas, esta propriedade só terá um efeito visual quando a borda tiver regiões transparentes ou regiões parcialmente opacas (devido ao `border-style` ou `border-image`); caso contrário a borda encobrirá a diferença.

Initial value	`border-box`
Aplica-se a	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	não
Computed value	as specified
Animation type	a repeatable list

Sintaxe

```
/* Valores de palavra-chave */
background-clip: border-box;
background-clip: padding-box;
background-clip: content-box;
background-clip: text;

/* Valores globais */
background-clip: inherit;
background-clip: initial;
background-clip: unset;
```

Valores

`border-box`

O fundo se extende até fora da fronteira da borda (mas por baixo da borda na ordenação-z).

`padding-box`

Nenhum fundo é desenhado abaixo da borda (o fundo se extende até a borda externa do preenchimento).

`content-box`

O fundo é desenhado dentro (cortado) a caixa de conteúdo.

`text`

O fundo é desenhado dentro (cortado) do texto aparente.

Sintaxe formal

```
background-clip =
  <bg-clip>#

<bg-clip> =
  <visual-box>             |
  [ border-area || text ]

<visual-box> =
  content-box  |
  padding-box  |
  border-box
```

Esta sintaxe reflete o padrão mais recente conforme CSS Backgrounds Module Level 4, CSS Box Model Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

Usando <code>border-box</code>

Conteúdo HTML

```
<p>O fundo se extende atrás da borda.</p>
```

Conteúdo CSS

```
p {
  border: 0.8em darkviolet;
  border-style: dotted double;
  margin: 1em 0;
  padding: 1.4em;
  background: linear-gradient(60deg, red, yellow, red, yellow, red);
  font: 900 1.2em sans-serif;
  text-decoration: underline;
  background-clip: border-box;
}
```

Resultado

Usando <code>padding-box</code>

Conteúdo HTML

```
<p>O fundo se extende até dentro da fronteira da borda.</p>
```

Conteúdo CSS

```
p {
  border: 0.8em darkviolet;
  border-style: dotted double;
  margin: 1em 0;
  padding: 1.4em;
  background: linear-gradient(60deg, red, yellow, red, yellow, red);
  font: 900 1.2em sans-serif;
  text-decoration: underline;
  background-clip: padding-box;
}
```

Resultado

Usando <code>content-box</code>

Conteúdo HTML

```
<p>O fundo se extende somente até o limite da caixa de conteúdo.</p>
```

Conteúdo CSS

```
p {
  border: 0.8em darkviolet;
  border-style: dotted double;
  margin: 1em 0;
  padding: 1.4em;
  background: linear-gradient(60deg, red, yellow, red, yellow, red);
  font: 900 1.2em sans-serif;
  text-decoration: underline;
  background-clip: content-box;
}
```

Resultado

Usando <code>text</code>

Conteúdo HTML

```
<p>O fundo se extende dentro do texto.</p>
```

Conteúdo CSS

```
p {
  border: 0.8em darkviolet;
  border-style: dotted double;
  margin: 1em 0;
  padding: 1.4em;
  background: linear-gradient(60deg, red, yellow, red, yellow, red);
  font: 900 1.2em sans-serif;
  text-decoration: underline;

  /* Note a necessidade de adicionar trânsparência ao texto*/
  background-clip: text;
  -webkit-background-clip: text;
  color: rgba(0, 0, 0, 0.2);
}
```

Resultado

Veja também

- `clip`
- CSS Box Model

background-color

A propriedade CSS **`background-color`** define a cor de fundo de um elemento.

Experimente

```
background-color: brown;
```

```
background-color: #74992e;
```

```
background-color: rgb(255, 255, 128);
```

```
background-color: rgba(255, 255, 128, 0.5);
```

```
background-color: hsl(50, 33%, 25%);
```

```
background-color: hsla(50, 33%, 25%, 0.75);
```

```
<section id="default-example">
  <div class="transition-all" id="example-element"></div>
</section>
```

```
#example-element {
  min-width: 100%;
  min-height: 100%;
  padding: 10%;
}
```

Sintaxe

```
/* Valores em chaves */
background-color: red;
background-color: indigo;

/* Valores em Hexadecimais */
background-color: #bbff00; /* Totalmente opaco */
background-color: #bf0; /* Forma abreviada totalmente opaco */
background-color: #11ffee00; /* Totalmente transparente */
background-color: #1fe0; /* Forma abreviada totalmente transparente */
background-color: #11ffeeff; /* Totalmente opaco */
background-color: #1fef; /* Forma abreviada totalmente opaco */

/* Valores em RGB */
background-color: rgb(255, 255, 128); /* Totalmente opaco */
background-color: rgba(117, 190, 218, 0.5); /* 50% transparente */

/* Valores em HSL */
background-color: hsl(50, 33%, 25%); /* Totalmente opaco */
background-color: hsla(50, 33%, 25%, 0.75); /* 75% transparente */

/* Valores em chaves especiais */
background-color: currentcolor;
background-color: transparent;

/* Valores globais */
background-color: inherit;
background-color: initial;
background-color: unset;
```

A propriedade `background-color` recebe um único valor `<color>`.

Valores

`<color>`

> A cor uniforme de fundo. É renderizado atrás de qualquer `background-image` especificado, embora a cora ainda continue visible através de qualquer transparência da imagem.

Acessibilidade

É importante assegurar que o contraste entre a cor de fundo e a cor do texto colocado sobre ele é alta o suficiente para pessoas com condições de baixa visão possam ler o conteúdo da página.

A cor do contrast é determinada comparando a luminância da cor do texto e da cor de fundo. Para atender às atuais Diretrizes de Acessibilidade de Conteúdo da Web (WCAG), uma proporção de 4.5:1 é necessária para conteúdo de texto e 3:1 para textos maiores como títulos. Texto grande é definido como 18.66px e negrito ou maior, ou 24px ou maior.

- WebAIM: Color Contrast Checker
- MDN Understanding WCAG, Guideline 1.4 explanations
- Understanding Success Criterion 1.4.3 | W3C Understanding WCAG 2.0

Definição formal

Initial value	`transparent`
Aplica-se a	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	não

Computed value	computed color
Animation type	a color

Sintaxe formal

```
background-color =
  <color>
```

Esta sintaxe reflete o padrão mais recente conforme CSS Backgrounds Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

HTML

```
<div class="exampleone">Lorem ipsum dolor sit amet, consectetuer</div>

<div class="exampletwo">Lorem ipsum dolor sit amet, consectetuer</div>

<div class="exemplothree">Lorem ipsum dolor sit amet, consectetuer</div>
```

CSS

```
.exampleone {
  background-color: teal;
  color: white;
}

.exampletwo {
  background-color: rgb(153, 102, 153);
  color: rgb(255, 255, 204);
}

.examplethree {
  background-color: #777799;
  color: #ffffff;
}
```

Result

Veja também

- Multiple backgrounds
- The `<color>` data type
- Other color-related properties: `color`, `border-color`, `outline-color`, `text-decoration-color`, `text-emphasis-color`, `text-shadow`, `caret-color`, and `column-rule-color`
- Applying color to HTML elements using CSS

background-origin

A propriedade CSS **`background-origin`** define *a área de posicionamento do plano de fundo*, isto é, a ponto de origem de uma imagem específica usando a propriedade `background-image`.

Experimente

```
background-origin: border-box;
background-repeat: no-repeat;
```

```
background-origin: padding-box;
background-repeat: no-repeat;
```

```
background-origin: content-box;
background-repeat: no-repeat;
```

```
<section id="default-example">
  <div id="example-element">This is the content of the element.</div>
</section>
```

```
#example-element {
  background-image: url("/shared-assets/images/examples/leopard.jpg");
  color: #d73611;
  text-shadow: 2px 2px black;
  padding: 20px;
  border: 10px dashed #333;
  font-size: 2em;
  font-weight: bold;
}
```

Note que `background-origin` é ignorado quando `background-attachment` é `fixed`.

> **Nota:** A taquigrafia `background` redefine o valor desta propriedade para seu valor inicial se esta não foi especificada.

Sintaxe

```
/* Valores chave */
background-origin: border-box;
background-origin: padding-box;
background-origin: content-box;

/* Valores globais */
background-origin: inherit;
background-origin: initial;
background-origin: unset;
```

A propriedade `background-origin` é especificada por uma chave de valores listadas abaixo.

Valores

`border-box`

O plano de fundo é posicionado relativamente nas bordas do elemento.

`padding-box`

O plano de fundo é posicionado relativamente no espaçamento do elemento.

`content-box`

O plano de fundo é posicionado relativamente para o conteúdo do elemento.

Sintaxe formal

```
background-origin =
  <visual-box>#

<visual-box> =
  content-box  |
  padding-box  |
  border-box
```

Esta sintaxe reflete o padrão mais recente conforme CSS Backgrounds Module Level 4, CSS Box Model Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

```
.exemplo {
  border: 10px double;
  padding: 10px;
  background: url("image.jpg");
  background-position: center left;
  background-origin: content-box;
}
```

```
#exemplo2 {
  border: 4px solid black;
  padding: 10px;
  background: url("image.gif");
  background-repeat: no-repeat;
```

```
  background-origin: border-box;
}
```

```
div {
  background-image:
    url("logo.jpg"), url("mainback.png"); /* Applies two images to the background */
  background-position:
    top right,
    0px 0px;
  background-origin: content-box, padding-box;
}
```

Veja também

- background-clip

background-size

A propriedade **`background-size`** do CSS define o tamanho da imagem de fundo do elemento. A imagem pode ser deixada no seu tamanho natural, estendida ou comprimida para caber no espaço disponível.

Experimente

```
background-size: contain;
```

```
background-size: contain;
background-repeat: no-repeat;
```

```
background-size: cover;
```

```
background-size: 30%;
```

```
background-size: 200px 100px;
```

```
<section id="default-example">
  <div class="transition-all" id="example-element"></div>
</section>
```

```
#example-element {
  background-image: url("/shared-assets/images/examples/hand.jpg");
  min-width: 100%;
  min-height: 100%;
}
```

Espaços não cobertos por uma imagem de fundo são preenchidos com a propriedade `background-color`, e a cor do fundo ficará visível atrás das imagens de fundo que têm transparência/translucidez.

Sintaxe

```
/* Valores de palavras-chave */
background-size: cover;
background-size: contain;

/* Sintaxe de um valor */
/* largura da imagem (altura torna-se 'auto') */
background-size: 50%;
background-size: 3.2em;
background-size: 12px;
background-size: auto;

/* Sintaxe de dois valores */
/* Primeiro valor: largura da imagem, segundo valor: altura */
background-size: 50% auto;
background-size: 3em 25%;
background-size: auto 6px;
background-size: auto auto;

/* Vários planos de fundo */
```

```
background-size: auto, auto; /* Não confunda com `auto auto` */
background-size: 50%, 25%, 25%;
background-size: 6px, auto, contain;

/* Valores globais */
background-size: inherit;
background-size: initial;
background-size: revert;
background-size: revert-layer;
background-size: unset;
```

A propriedade `background-size` é especificada de uma das seguintes maneiras:

- Usando a palavra-chave `contain` ou `cover`.
- Usando somente o valor da largura, caso em que o padrão de altura é `auto`.
- Usando ambos os valores da largura e altura, nesse caso o primeiro define a largura e o segundo define a altura. Cada valor pode ser `<length>`, `<percentage>` ou `auto`.

Para especificar o tamanho de várias imagens de fundo, separe o valor de cada uma com uma vírgula.

Valores

`contain`

> Dimensiona a imagem o máximo possível dentro do seu contêiner sem cortar ou estender a imagem. Se o contêiner for maior do que a imagem, resultará em uma imagem lado a lado, a menos que a propriedade `background-repeat` esteja definida como `no-repeat`.

`cover`

> Dimensiona a imagem (enquanto preserva sua proporção) para o menor tamanho possível para caber no contêiner (ou seja: tanto a altura quanto o comprimento *cobrem* completamente o contêiner), sem deixar espaço vazio. Se as proporções do plano de fundo diferirem do elemento, a imagem será cortada verticalmente ou horizontalmente.

`auto`

> Dimensiona a imagem de fundo na direção correspondente de forma que as proporções intrínsecas sejam mantidas.

`<length>`

> Estende a imagem na dimensão correspondente ao comprimento especificado. Valores negativos não são permitidos.

`<percentage>`

> Estende a imagem na dimensão correspondente à porcentagem especificada da *área de posicionamento do plano de fundo*. A área de posicionamento do plano de fundo é determinada pelo valor de `background-origin` (por padrão, o padding box). No entanto, se o valor `background-attachment` do plano de fundo for `fixed`, a área de posicionamento será o viewport inteiro. Valores negativos não são permitidos.

Dimensões e proporções intrínsecas

O cálculo dos valores depende das dimensões intrínsecas da imagem (largura e altura) e das proporções intrínsecas (relação largura/altura). Esses atributos são os seguintes:

- Uma imagem bitmap (como a JPG) sempre possui dimensões e proporções intrínsecas.
- Uma imagem vetorial (como a SVG) não tem necessariamente dimensões intrínsecas. Se tiver dimensões intrínsecas horizontais e verticais, também terá proporções intrínsecas. Se não tiver dimensões ou tiver apenas uma dimensão, pode ou não ter proporções.
- O `<gradient>` do CSS não têm dimensões intrínsecas ou proporções intrínsecas.
- Imagens de plano de fundo criados com a função `element()` usam as dimensões e proporções intrínsecas do elemento gerador.

> **Nota:** No Gecko, imagens de fundo criadas usando a função `elemen()` são atualmente tratadas como imagens com as dimensões do elemento, ou da área de posicionamento do fundo se o elemento é SVG, com a proporção intrínseca correspondente. Este é um comportamento fora do padrão.

Com base nas dimensões e proporções intrínsecas, o tamanho renderizado da imagem de fundo é calculado da seguinte forma:

- **Se ambos os componentes de `background-size` forem especificados e não forem `auto`:** A imagem de fundo será renderizada no tamanho especificado.

- **Se o `background-size` for `contain` ou `cover`:** Embora preservando suas proporções intrínsecas, a imagem é renderizada no maior tamanho contido ou cobrindo a área de posicionamento do plano de fundo. Se a imagem não possui proporções intrínsecas, então ela é renderizada no tamanho da área de posicionamento do plano de fundo.
- **Se o `background-size` for `auto` ou `auto auto`:**
  - Se a imagem tiver dimensões intrínsecas horizontais e verticais, ela será renderizada nesse tamanho.
  - Se a imagem não tiver dimensões intrínsecas e proporções intrínsecas, ela será renderizada no tamanho da área de posicionamento do plano de fundo.
  - Se a imagem não tiver dimensões intrínsecas, mas tiver proporções intrínsecas, ela será renderizada como se `contain` tivesse sido especificado.
  - Se a imagem tiver apenas uma dimensão intrínseca e tiver proporções intrínsecas, ela será renderizada no tamanho correspondente a essa dimensão. A outra dimensão é calculada usando a dimensão especificada e as proporções intrínsecas.
  - Se a imagem tiver apenas uma dimensão intrínseca, mas não tiver proporções intrínsecas, ela será renderizada usando a dimensão especificada e a outra dimensão da área de posicionamento do plano de fundo.

  > **Nota:** Imagens SVG têm um atributo preserveAspectRatio que é equivalente ao `contain`; um `background-size` explícito faz com que `preserveAspectRatio` seja ignorado.

- **Se o `background-size` tem um componente `auto` e um componente não `auto`:**
  - Se a imagem tiver proporções intrínsecas, ela será ampliada para a dimensão especificada. A dimensão não especificada é calculada usando a dimensão especificada e as proporções intrínsecas.
  - Se a imagem não tiver proporções intrínsecas, ela será ampliada para a dimensão especificada. A dimensão não especificada é calculada usando a dimensão intrínseca correspondente da imagem, se houver. Se não houver essa dimensão intrínseca, ela se tornará a dimensão correspondente da área de posicionamento do plano de fundo

> **Nota:** O dimensionamento do plano de fundo para imagens vetoriais que não possuem dimensões ou proporções intrínsecas ainda não foi totalmente implementado em todos os navegadores. Tenha cuidado ao confiar no comportamento descrito acima e teste em vários navegadores para garantir que os resultados sejam aceitáveis.

Definição formal

Initial value	`auto auto`
Aplica-se a	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	não
Percentages	relative to the background positioning area
Computed value	as specified, but with relative lengths converted into absolute lengths
Animation type	a repeatable list

Sintaxe formal

```
background-size =
  <bg-size>#

<bg-size> =
  [ <length-percentage [0,∞]> | auto ]{1,2}  |
  cover                                      |
  contain

<length-percentage> =
  <length>      |
  <percentage>
```

Esta sintaxe reflete o padrão mais recente conforme CSS Backgrounds Module Level 4, CSS Values and Units Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

Colocar lado a lado uma imagem grande

Vamos considerar uma imagem grande, uma imagem do logotipo do Firefox de 2982x2808. Queremos agrupar quatro cópias dessa imagem em um elemento de 300x300 pixels. Para fazer isso, podemos usar um valor fixo de `background-size` de 150 pixels.

HTML

```
<div class="tiledBackground"></div>
```

CSS

```
.tiledBackground {
  background-image: url(https://www.mozilla.org/media/img/logos/firefox/logo-quantum.9c5e96634f92.png);
  background-size: 150px;
  width: 300px;
  height: 300px;
  border: 2px solid;
  color: pink;
}
```

Resultado

Veja Redimensionando as imagens de fundo para mais exemplos.

Veja também

- Redimensionando as imagens de fundo
- Scaling of SVG backgrounds
- `object-fit`

block-size

> **Experimental: Esta é uma tecnologia experimental**
> Verifique a tabela de compatibilidade entre Navegadores cuidadosamente antes de usar essa funcionalidade em produção.

A propriedade CSS **`block-size`** define o tamanho horizontal ou vertical de um elemento bloco. Corresponde à propriedade `width` ou `height`, dependendo do valor do `writing-mode`.

Experimente

```
block-size: 150px;
writing-mode: horizontal-tb;
```

```
block-size: 150px;
writing-mode: vertical-rl;
```

```
block-size: auto;
writing-mode: horizontal-tb;
```

```
block-size: auto;
writing-mode: vertical-lr;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box where you can change the block-size.
  </div>
</section>
```

```
#example-element {
  display: flex;
  flex-direction: column;
  background-color: #5b6dcd;
  justify-content: center;
  color: #ffffff;
}
```

Sintaxe

```
/* valores <length> */
block-size: 300px;
block-size: 25em;
```

```
/* valores <percentage> */
block-size: 75%;

/* valores de Keyword */
block-size: 25em border-box;
block-size: 75% content-box;
block-size: max-content;
block-size: min-content;
block-size: available;
block-size: fit-content;
block-size: auto;

/* valores globais */
block-size: inherit;
block-size: initial;
block-size: unset;
```

Se o modo de escrita está orientado verticalmente, o valor do `block-size` refere-se à largura do elemento; senão, refere-se à altura do elemento. Uma propriedade relacionada é `inline-size`, que define as outras dimensões do elemento.

Valores

A propriedade `block-size` leva os mesmos valores que as propriedades `width` e `height`.

Definição formal

Initial value	`auto`
Aplica-se a	same as `width` and `height`
Inherited	não
Percentages	block-size of containing block
Computed value	same as `width` and `height`
Animation type	a length, percentage or calc();

Sintaxe formal

```
block-size =
  auto             |
  <box-size>       |
  <anchor-size()>

<box-size> =
  <length-percentage [0,∞]>                 |
  stretch                                   |
  contain                                   |
  min-content                               |
  max-content                               |
  fit-content                               |
  stretch( <length-percentage [0,∞]> )      |
  fit-content( <length-percentage [0,∞]> )  |
  <calc-size()>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<length-percentage> =
  <length>      |
  <percentage>

<calc-size()> =
  calc-size( <calc-size-basis> , <calc-sum> )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width        |
  height       |
  block        |
  inline       |
  self-block   |
```

```
  self-inline

<calc-size-basis> =
  <size-keyword>  |
  <calc-sum>      |
  <calc-size()>   |
  any

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

Esta sintaxe reflete o padrão mais recente conforme CSS Anchor Positioning Module Level 1, CSS Box Sizing Module Level 3, CSS Box Sizing Module Level 4, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplo

HTML

```
<p class="exampleText">Example text</p>
```

CSS

```
.exampleText {
  writing-mode: vertical-rl;
  background-color: yellow;
  block-size: 200px;
}
```

See also

- As propriedades: `width` and `height`
- `writing-mode`

border-left

Resumo

A **`border-left`** é uma propriedade shorthand em CSS que define todas as propriedades da `border` esquerda de um elemento.

Experimente

```
border-left: solid;
```

```
border-left: dashed red;
```

```
border-left: 1rem solid;
```

```
border-left: thick double #32a1ce;
```

```
border-left: 4mm ridge rgba(211, 220, 50, 0.6);
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    This is a box with a border around it.
  </div>
</section>
```

```
#example-element {
  background-color: #eee;
  color: #8b008b;
  padding: 0.75em;
  width: 80%;
  height: 100px;
}
```

Tal como acontece com todas as propriedades abreviadas, `border-left` sempre define os valores de todas as propriedades que possam ser definidas, ainda que estas não tenham sido especificadas. Ela define aquelas que não forem especificadas para os valores padrão. Considere o código a seguir:

```
border-left-style: dotted;
border-left: thick green;
```

É na verdade o mesmo que:

```
border-left-style: dotted;
border-left: none thick green;
```

O valor de `border-left-style` dado antes de `border-left` é ignorado. Uma vez que o valor padrão de `border-left-style` é `none`, não especificar a parte `border-style` resulta em nenhuma borda.

Propriedades constituintes

Essa propriedade é uma abreviação das seguintes propriedades CSS:

- `border-left-color`
- `border-left-style`
- `border-left-width`

Sintaxe

```
border-left: 1px;
border-left: 2px dotted;
border-left: medium dashed blue;

/* Global values */
border-left: inherit;
border-left: initial;
border-left: revert;
border-left: revert-layer;
border-left: unset;
```

Os três valores da propriedade abreviada podem ser especificados em qualquer ordem, e um ou dois deles podem ser omitidos.

Valores

`<br-width>`

Veja `border-left-width`.

`<br-style>`

Veja `border-left-style`.

`<color>`

Veja `border-left-color`.

Definição formal

Initial value	as each of the properties of the shorthand:

	• `border-left-width`: `medium` • `border-left-style`: `none` • `border-left-color`: `currentcolor`
Aplica-se a	all elements. It also applies to `::first-letter`.
Inherited	não
Computed value	as each of the properties of the shorthand: • `border-left-width`: the absolute `<length>`, snapped as a line width • `border-left-style`: as specified • `border-left-color`: computed color
Animation type	as each of the properties of the shorthand: • `border-left-width`: a length • `border-left-style`: discrete • `border-left-color`: a color

Sintaxe formal

```
border-left =
  <line-width>  ||
  <line-style>  ||
  <color>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick

<line-style> =
  none    |
  hidden  |
  dotted  |
  dashed  |
  solid   |
  double  |
  groove  |
  ridge   |
  inset   |
  outset
```

Esta sintaxe reflete o padrão mais recente conforme CSS Borders and Box Decorations Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

Aplicando uma borda esquerda

HTML

```
<div>Essa caixa tem uma borda no seu lado esquerdo.</div>
```

CSS

```
div {
  border-left: 4px dashed blue;
  background-color: gold;
  height: 100px;
  width: 100px;
  font-weight: bold;
  text-align: center;
}
```

Resultado

Veja também

- `border`
- `border-block`
- `outline`
- Planos de fundo e bordas
- Aprenda CSS: planos de fundo e bordas

-moz-box-ordinal-group

> **Aviso:** Essa propriedade é parte do projeto do módulo original do CSS Flexible Box Layout, e foi substituído em projetos recentes.

Veja Flexbox para mais informações do que você deveria usar ao invés dessa propriedade.

Sumário

Indica o grupo ordinal ao qual o elemento percente. Elementos com um grupo ordinal menor são mostrados antes daqueles com grupo ordinal maior.

Valores

Os valores devem ser inteiros maiores que zero. O valor padrão para essa propriedade é 1.

Exemplos

```
#Flexbox {
  display: -ms-box;
  display: -moz-box;
  display: -webkit-box;
}

#text1 {
  background: red;
  -ms-box-ordinal-group: 4;
  -moz-box-ordinal-group: 4;
  -webkit-box-ordinal-group: 4;
}

#text2 {
  background: green;
  -ms-box-ordinal-group: 3;
  -moz-box-ordinal-group: 3;
  -webkit-box-ordinal-group: 3;
}

#text3 {
  background: blue;
  -ms-box-ordinal-group: 2;
  -moz-box-ordinal-group: 2;
  -webkit-box-ordinal-group: 2;
}

#text4 {
  background: orange;
}
```

```
<div id="Flexbox">
  <div id="text1">text 1</div>
  <div id="text2">text 2</div>
  <div id="text3">text 3</div>
  <div id="text4">text 4</div>
</div>
```

box-shadow

O **`box-shadow`** é uma propriedade do CSS, é utilizado para adicionar efeitos de sombra em volta de um elemento. Você pode especificar mais de um efeito, os separando com virgulas.

Uma box-shadow é descrita pelo deslocamentos (offset) X e Y em relação ao elemento, desfoco e propagação do raio e cor.

```
/* offset-x | offset-y | color */
box-shadow: 60px -16px teal;

/* offset-x | offset-y | blur-radius | color */
box-shadow: 10px 5px 5px black;

/* offset-x | offset-y | blur-radius | spread-radius | color */
box-shadow: 2px 2px 2px 1px rgba(0, 0, 0, 0.2);

/* inset | offset-x | offset-y | color */
box-shadow: inset 5em 1em gold;

/* Any number of shadows, separated by commas */
box-shadow:
  3px 3px red,
  -1em 0 0.4em olive;

/* Global keywords */
box-shadow: inherit;
box-shadow: initial;
box-shadow: unset;
```

```
box-shadow: 60px -16px teal;
```

```
box-shadow: 10px 5px 5px black;
```

```
box-shadow:
  3px 3px red,
  -1em 0 0.4em olive;
```

```
box-shadow: inset 5em 1em gold;
```

```
box-shadow: 0 0 1em gold;
```

```
box-shadow: inset 0 0 1em gold;
```

```
box-shadow:
  inset 0 0 1em gold,
  0 0 1em red;
```

A propriedade box-shadow permite que você adicione uma sombra em quase qualquer elemento. Se um `border-radius` for especificado no elemento com uma box-shadow, a sombra assume os mesmos cantos arredondados. O comando z-ordering de múltiplas box-shadows é o mesmo que múltiplas text shadows (a primeira sombra especificada está na parte superior).

Box-shadow generator é uma ferrament interativa qu permite a criação de `box-shadow`.

Initial value	none
Aplica-se a	all elements. It also applies to `::first-letter`.
Inherited	não
Computed value	any length made absolute; any specified color computed; otherwise as specified
Animation type	a shadow list

Sintaxe

Especificando uma única box-shadow usando:

- Dois, três ou quatro valores para `<length>`.
    - Se apenas dois valores forem definidos, eles serão interpretados como valores de `<offset-x><offset-y>`.
    - Se o terceiro valor for dados, é interpretado como `<blur-radius>`.
    - Se o quarto valor for dados, é interpretado como `<spread-radius>`.
- Opcionalmente, o atributo `inset`.

- Opcionalmente, o valor `<color>`.

Para especificar mais de uma sombra, separe-as com virgula.

Valores

`inset`

Se não for especificado, a sombra ganha o efeito de como se a caixa fosse aumentada acima do conteúdo). A presença da palavra-chave `inset` muda a sombra para dentro da moldura (como se o conteúdo estivesse pressionado dentro da caixa). As sombras de inserção são desenhadas dentro da borda (mesmo as transparentes), acima do plano de fundo, mas abaixo do conteúdo.

`<offset-x>` `<offset-y>`

Existem dois valores `<length>` para configurar o desvio (offset) da sombra:

- `<offset-x>` especifica a distância horizontal. Valores negativos colocarão a sombra à esquerda do elemento.
- `<offset-y>` especifca a distância vertical. Valores negativos colocam a sombra acima do elemento.

Consulte `<length>` para as unidades disponíveis.

Se ambos os valores forem `0`, a sombra será posicionada atrás do elemento (e poderá gerar um efeito de desfocagem caso `<blur-radius>` e/ou `<spread-radius>` estiverem configurados).

`<blur-radius>`

Este é um terceiro valor para `<length>`. Quanto maior for este valor, maior o efeito de desfocagem, desta forma a sombra se tornará maior e mais clara. Valores negativos não são permitidos. Se não for especificado, o valor padrão é `0` (os limites da sombra serão retos). A especificação não inclui um algoritmo exato de como o raio de esmaecimento deve ser calculado, no entanto, descreve o seguinte:

> ...para uma sombra com bordas longas e retas, deverá ser criada uma transição de cor do comprimento da distância de desfocagem que é perpendicular e centralizada com as bordas da sombra, e isto deve variar da cor total da sombra do seu ponto central até totalmente transparente no ponto final fora dela.

`<spread-radius>`

Este é um quarto valor de `<length>`. Valores positivos farão com que a sombra expanda e cresça maior, valores negativos farão com que a sombra encolha. Se não for especificado, o valor padrão é `0` (a sombra terá o mesmo tamanho do elemento)

`<color>`

Consulte `<color>` para possiveis palavras-chave e notações. Se não for especificada, a cor que será utilizada vai depender do navegador - geralmente é o valor da propriedade `color`, mas tenha em mente que o Safari atualmente imprime uma sombra transparente neste caso.

Interpolação

Each shadow in the list (treating `none` as a 0-length list) is interpolated via the color (as color) component, and x, y, blur, and (when appropriate) spread (as length) components. For each shadow, if both input shadows are or are not `inset`, then the interpolated shadow must match the input shadows in that regard. If any pair of input shadows has one `inset` and the other not `inset`, the entire shadow list is uninterpolable. If the lists of shadows have different lengths, then the shorter list is padded at the end with shadows whose color is `transparent`, all lengths are `0`, and whose `inset` (or not) matches the longer list.

Formal syntax

```
box-shadow =
  <spread-shadow>#

<spread-shadow> =
  <'box-shadow-color'>?                              &&
  [ [ none | <length>{2} ] [ <'box-shadow-blur'> <'box-shadow-spread'>? ]? ]  &&
  <'box-shadow-position'>?

<box-shadow-color> =
  <color>#

<box-shadow-blur> =
  <length [0,∞]>#

<box-shadow-spread> =
  <length>#
```

```
<box-shadow-position> =
  [ outset | inset ]#
```

Esta sintaxe reflete o padrão mais recente conforme CSS Borders and Box Decorations Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Live examples

- box-shadow test
- box-shadow tutorial and examples
- Box Shadow CSS Generator

See also

- The `<color>` data type
- Other color-related properties: `color`, `background-color`, `border-color`, `outline-color`, `text-decoration-color`, `text-emphasis-color`, `caret-color`, and `column-rule-color`
- Applying color to HTML elements using CSS

box-sizing

A propriedade CSS box-sizing é utilizada para alterar a propriedade padrão da box model, usada para calcular larguras (widths) e alturas (heights) dos elementos. É possível usar essa propriedade para emular o comportamento dos navegadores (browsers) que não suportam corretamente a especificação da propriedade CSS box model.

Sintaxe

Sintaxe formal:

```
box-sizing =
  content-box  |
  border-box
```

Esta sintaxe reflete o padrão mais recente conforme CSS Box Sizing Module Level 3. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

```
box-sizing: content-box
box-sizing: border-box

box-sizing: inherit
```

Valores

`content-box`

> Essa é o estilo padrão, conforme especificado pela norma CSS. As propriedades `width` (largura) e `height` (altura) são medidas incluindo só o conteúdo, mas não o padding, border ou margin. Nota: Padding, border e margin serão fora da box. Exemplo.: Se .box {width: 350px} então se você aplicar uma propriedade {border: 10px solid black;} o resultado renderizado no navegador (browser) será .box {width: 370px;}

`padding-box`

> As propriedades de largura (`width`) e de altura (`height`) incluem o tamanho padding size, mas não incluem a propriedade border ou margem.

`border-box`

> As propriedades de largura (`width`) e de altura (`height`) incluem o tamanho padding size e a propriedade border, mas não incluem a propriedade margin.

Exemplos

```
/* funciona em Firefox, WebKit, Opera and IE8+ */
/* exemplo de uso no navegador Firefox */

.exemplo {
  -moz-box-sizing: border-box;
  box-sizing: border-box;
}
```

Veja também

- CSS box model [Eng]

clear

A propriedade **`clear` do** CSS especifica se um elemento pode ter elementos flutuantes ao seu lado ou se devem ser movidos para abaixo dele (clear). Essa propriedade se aplica à elementos flutuantes ou não flutuantes.

Experimente

```
clear: none;
```

```
clear: left;
```

```
clear: right;
```

```
clear: both;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="floated-left">Left</div>
    <div class="floated-right">Right</div>
    <div class="transition-all" id="example-element">
      As much mud in the streets as if the waters had but newly retired from the
      face of the earth, and it would not be wonderful to meet a Megalosaurus,
      forty feet long or so, waddling like an elephantine lizard up Holborn
      Hill.
    </div>
  </div>
</section>
```

```
.example-container {
  border: 1px solid #c5c5c5;
  padding: 0.75em;
  text-align: left;
  line-height: normal;
}

.floated-left {
  border: solid 10px #ffc129;
  background-color: rgba(81, 81, 81, 0.6);
  padding: 1em;
  float: left;
}

.floated-right {
  border: solid 10px #ffc129;
  background-color: rgba(81, 81, 81, 0.6);
  padding: 1em;
  float: right;
  height: 150px;
}
```

Quando essa propriedade é aplicada em elementos não flutuantes, ele move a borda (border edge) desse elemento para a borda da margem (margin edge) de todos os elementos flutuantes relevantes. Ocorre um colapso das margens verticais dos elementos não flutuantes.

As margens verticais entre dois elementos flutuantes não irão sofrer esse colapso. Quando aplicada a elementos flutuantes, a borda de margem (margin edge) do elemento inferior é movida abaixo da borda de margem (border edge) de todos os elementos flutuantes relevantes. Isso afeta a posição dos elementos flutuantes posteriores, sendo que os elementos flutuantes posteriores não podem ser posicionados acima dos anteriores.

Os elementos flutuantes que devem ser limpos (clear) são os elementos anteriores dentro do mesmo contexto de bloco (block formatting context).

> **Nota:** se um elemento possuir apenas elementos flutuantes, sua altura é zerada. Se você quiser que o mesmo seja redimensionado, de modo que contenha elementos flutuantes dentro dele, você precisa limpar(clear) seus filhos automaticamente. Isso é chamado clearfix, e uma maneira de fazê-lo é adicionando um `::after` pseudo-elemento com a propriedade `clear`.

```
#container::after {
  content: "";
  display: block;
  clear: both;
}
```

Sintaxe

```
/* Valores chaves */
clear: none;
clear: left;
clear: right;
clear: both;
clear: inline-start;
clear: inline-end;

/* Valores globais */
clear: inherit;
clear: initial;
clear: unset;
```

Valores

`none`

É uma palavra-chave que indica que o elemento não foi movido para baixo para limpar os elementos flutuantes anteriores.

`left`

É uma palavra-chave que indica que o elemento foi movido para baixo para limpar os últimos flutuantes da esquerda.

`right`

É uma palavra-chave que indica que o elemento foi movido para baixo para liberar os últimos elementos flutuantes.

`both`

É uma palavra-chave que indica que o elemento será movido para baixo para limpar os elementos flutuantes do lado esquerdo e direito.

`inline-start`

É uma palavra-chave que indica que o elemento será movido para baixo para limpar os elementos flutuantes no lado inicial de seu bloco, ou seja, os da esquerda flutuam nos scripts de ltr e os da direita nos scripts de rtl.

`inline-end`

É uma palavra-chave que indica que o elemento será movido para baixo para limpar os flutuantes no final de seu bloco, os da direita nos scripts de ltr e os flutuantes da esquerda nos scripts de rtl.

Sintaxe

```
clear =
  inline-start  |
  inline-end    |
  block-start   |
  block-end     |
  left          |
  right         |
  top           |
  bottom        |
  both-inline   |
  both-block    |
  both          |
  none
```

Esta sintaxe reflete o padrão mais recente conforme CSS Page Floats. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

clear: left

HTML

```
<div class="wrapper">
  <p class="black">
    Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Phasellus sit amet
    diam. Duis mattis varius dui. Suspendisse eget dolor.
  </p>
  <p class="red">Lorem ipsum dolor sit amet, consectetuer adipiscing elit.</p>
  <p class="left">This paragraph clears left.</p>
</div>
```

CSS

```
.wrapper {
  border: 1px solid black;
  padding: 10px;
}
.left {
  border: 1px solid black;
  clear: left;
}
.black {
  float: left;
  margin: 0;
  background-color: black;
  color: #fff;
  width: 20%;
}
.red {
  float: left;
  margin: 0;
  background-color: pink;
  width: 20%;
}
p {
  width: 50%;
}
```

clear: right

HTML

```
<div class="wrapper">
  <p class="black">
    Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Phasellus sit amet
    diam. Duis mattis varius dui. Suspendisse eget dolor.
  </p>
  <p class="red">Lorem ipsum dolor sit amet, consectetuer adipiscing elit.</p>
  <p class="right">This paragraph clears right.</p>
</div>
```

CSS

```
.wrapper {
  border: 1px solid black;
  padding: 10px;
}
.right {
  border: 1px solid black;
  clear: right;
}
.black {
  float: right;
  margin: 0;
  background-color: black;
  color: #fff;
  width: 20%;
}
```

```
.red {
  float: right;
  margin: 0;
  background-color: pink;
  width: 20%;
}
p {
  width: 50%;
}
```

clear: both

HTML

```
<div class="wrapper">
  <p class="black">
    Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Phasellus sit amet
    diam. Duis mattis varius dui. Suspendisse eget dolor. Fusce pulvinar lacus
    ac dui.
  </p>
  <p class="red">
    Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Phasellus sit amet
    diam. Duis mattis varius dui. Suspendisse eget dolor.
  </p>
  <p class="both">This paragraph clears both.</p>
</div>
```

CSS

```
.wrapper {
  border: 1px solid black;
  padding: 10px;
}
.both {
  border: 1px solid black;
  clear: both;
}
.black {
  float: left;
  margin: 0;
  background-color: black;
  color: #fff;
  width: 20%;
}
.red {
  float: right;
  margin: 0;
  background-color: pink;
  width: 20%;
}
p {
  width: 45%;
}
```

Veja também

- Box model

Propriedade color do CSS

A propriedade **`color`** do CSS definine o valor da cor de um elemento com seu conteúdo em texto e seus decorações (estilos). Também define o valorer da `<currentcolor>`, no qual pode ser usado um valor indireto de outra propriedade, e tem como padrão as cores de outras propriedades (enquanto não definida), tal como a propriedade `border-color`.

Experimente

```
color: rebeccapurple;
```

```
color: #00a400;
```

```
color: rgb(214, 122, 127);
```

```
color: hsl(30deg 82% 43%);
```

```
color: hsla(237deg 74% 33% / 61%);
```

```
color: hwb(152deg 0% 58% / 70%);
```

```
<section id="default-example">
  <div class="example-container">
    <p id="example-element">
      London. Michaelmas term lately over, and the Lord Chancellor sitting in
      Lincoln's Inn Hall. Implacable November weather.
    </p>
  </div>
</section>
```

```
#example-element {
  font-size: 1.5em;
}

.example-container {
  background-color: white;
  padding: 10px;
}
```

Para uma visão geral de como usar a propriedade color e semelhantes no HTML, veja Cor - CSS, como começar.

Syntaxe

```
/* Valores com Palavras-chave */
color: white; /* Aqui será definida a cor atual*/

/* Valores com Palavras-chave */
/* color define a cor do texto no elemento*/
color: red; /* Define como 'vermelho'*/
color: blue; /* Define como 'laranja'*/
color: cyan; /* Define como 'verde-água'*/

/* Obs: Utilizando "light" antes da cor, ela ficará clara.
   ex: lightblue;
   E utilizando "dark", ela ficará mais escura.
   ex: darkblue;
*/

/* Cores com valores Hexa-decimais,
ou seja de 1 à 15, sendo de 0 à 9 normais,mas de 10 à 15
são as letras do alfabeto , logo de "a" à "f". */

color: #090;
color: #0099fg;
color: #090a;
color: #0099bcaa;

/* Valores "rgb" ou "rgba",
ou seja, o primeiro é vermelho, o segundo é verde e
o terceiro é azul, o "a" é o nível de transparência da cor,
e vai de 0 à 1;
Ex: preto meio transparênte = rgba(255,255,255,0.5); */

color: rgb(34, 12, 64, 0.6);
color: rgba(34, 12, 64, 0.6);
color: rgb(34 12 64 / 0.6);
color: rgba(34 12 64 / 0.3);
color: rgb(34 12 64 / 60%);
color: rgba(34.6 12 64 / 30%);

/* Valores "hsl()", sendo
hsl(num da cor"1-369", saturação"0-100", luminosidade"0-100")*/
color: hsl(30, 100%, 50%, 0.6);
```

```
color: hsla(30, 100%, 50%, 0.6);
color: hsl(30 100% 50% / 0.6);
color: hsla(30 100% 50% / 0.6);
color: hsl(30 100% 50% / 60%);
color: hsla(30.2 100% 50% / 60%);

/* Valores globais, pega algun valor já definido,
para todo o <body>, por exemplo. */
color: inherit;
color: initial;
color: unset;
```

A propriedade `color` é especificada somente com o valor `<color>` .

Note que o valor sempre será uniforme para `color`. Não podendo ter `<gradient>`(gradiente), que é atualmente um tipo para `<image>`.

Valores

`<color>`

Define a cor dos caracteres presentes no elemento a ser estilizado.

Syntaxe formal

```
p { color: gray; }
```

Exemplos

A seguir, você verá várias maneira de tornar a cor de um texto num parágrafo vermelha:

```
p {
  color: red;
}
p {
  color: #f00;
}
p {
  color: #ff0000;
}
p {
  color: rgb(255, 0, 0);
}
p {
  color: rgb(100%, 0%, 0%);
}
p {
  color: hsl(0, 100%, 50%);
}

/* 50% translúcido(transparênte) */
p {
  color: rgba(255, 0, 0, 0.5);
}
p {
  color: hsla(0, 100%, 50%, 0.5);
}
```

Preocupações com acessibilidade

É extremamente crucial que todo site, contenha um contraste de cores adequado, para que pessoas com níveis baixos de visão tenha um aboa experiência ao utilizar o site.

Veja também

- Tipo de dado <color> no CSS.

color-scheme

A Propriedade CSS **`color-scheme`** possibilita que um elemento possa indicar em qual esquema de cores é mais amigável de ser renderizado.

Opções comuns em esquemas de cores de sistemas operacionais são "claro" e "escuro", ou "modo diurno" e "modo noturno". Quando um usuário seleciona um desses esquemas de cores, o sistema operacional faz ajustes na interface do usuário. Isso inclui controles de formulários, barra de scroll, e os valores utilizados no sistema de cores do CSS.

Syntax

```
color-scheme: normal;
color-scheme: light;
color-scheme: dark;
color-scheme: light dark;

/* Valores globais */
color-scheme: inherit;
color-scheme: initial;
color-scheme: revert;
color-scheme: unset;
```

O valor da propriedade `color-scheme` precisa ser uma das palavras-chaves abaixo.

Valores

`normal`

Indica que o elemento não possui nenhum esquema de cores, e deve ser renderizado utilizando o esquema de cores padrão do navegador.

`light`

Indica que o elemento pode ser renderizado utilizando o esquema de cores claro do sistema operacional.

`dark`

Indica que o elemento pode ser renderizado utilizando o esquema de cores escuro do sistema operacional.

Definição formal

Initial value	`normal`
Aplica-se a	all elements and text
Inherited	yes
Computed value	as specified
Animation type	discrete

Sintaxe formal

```
color-scheme =
  normal                                  |
  [ light | dark | <custom-ident> ]+ && only?
```

Esta sintaxe reflete o padrão mais recente conforme CSS Color Adjustment Module Level 1. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

Adaptando a esquemas de cores

Para fazer com que toda a página tenha o esquema de cores de preferência do usuário declare `color-scheme` no elemento `:root`.

```
:root {
  color-scheme: light dark;
}
```

Veja também

- Aplicando cores em elementos HTML com CSS
- Outras propriedades de cores relacionadas: `color`, `background-color`, `border-color`, `outline-color`, `text-decoration-color`, `text-emphasis-color`, `text-shadow`, `caret-color`, and `column-rule-color`
- `background-image`
- `print-color-adjust`

contain

> **Experimental: Esta é uma tecnologia experimental**
> Verifique a tabela de compatibilidade entre Navegadores cuidadosamente antes de usar essa funcionalidade em produção.

A propriedade **`contain`** permite que um autor indique que elemento e seus conteúdos são, na medida do possível, *independente* do resto da árvore do documento. Isso permite que o navegador recalcule o layout, estilo, pintura, tamanho ou alguma combinação deles para uma área limitada do DOM e não da página inteira.

```
/* Sem contenção de layout. */
contain: none;

/* Ativa a contenção para layout, estilo, pintura, e tamanho. */
contain: strict;

/* Ativa a contenção para layout, estilo, e pintura. */
contain: content;

/* Ativa a contenção para um elemento. */
contain: size;

/* Ativa a contenção de layout para um elemento. */
contain: layout;

/* Ativa a contenção de estilo para um elemento. */
contain: style;

/* Ativa a contenção de pintura para um elemento. */
contain: paint;
```

Esta propriedade é útil em páginas que contêm um monte de widgets que são todos independentes, uma vez que pode ser usado para impedir que as regras CSS de um widget mude outras coisas na página.

Initial value	none
Aplica-se a	all elements
Inherited	não
Computed value	as specified
Animation type	Not animatable

Sintaxe

Valores

none

Indica que o elemento processa como normal, sem contenção aplicada.

strict

Indica que todas as regras de contenção são aplicadas ao elemento. Isso equivale a`contain: size layout style paint`.

content

Indica que todas as regras de contenção, exceto *size* são aplicadas ao elemento. Isso equivale a `contain: layout style paint`.

size

Indica que o elemento pode ser dimensionado sem a necessidade de examinar seus dependentes para alterações de tamanho.

layout

Indica que nada fora do elemento pode afetar seu layout interno e vice-versa.

style

Indica que, para propriedades que podem ter efeitos em mais do que apenas um elemento e seus descendentes, esses efeitos não escape o elemento que contém.

paint

> Indica que os descendentes do elemento não são exibidos fora de seus limites. Se um elemento estiver fora da tela ou de outra forma não visível, seus descendentes também são garantidos para não ser visível.

Sintaxe Formal

```
contain =
  none                                          |
  strict                                        |
  content                                       |
  [ [ size | inline-size ] || layout || style || paint ]
```

Esta sintaxe reflete o padrão mais recente conforme CSS Containment Module Level 2. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

content

A propriedade CSS **`content`** é usada com os pseudoelementos `::before` e `::after` para gerar conteúdo em um elemento. Objetos inseridos usando a propriedade `content` são *elementos substituídos anônimos*.

```
/* Palavras-chave que não podem ser combinadas com outros valores */
content: normal;
content: none;

/* Valores <string> com caracteres especiais devem ser escritos */
/* com escape Unicode, por exemplo \00A0 para   */
content: "prefixo";

/* Valores <url> */
content: url("http://www.example.com/test.png");

/* Valores <counter> */
content: counter(contador_capitulo);
content: counters(contador_secao, ".");

/* Valores attr() associados ao valor do atributo HTML */
content: attr(valor string);

/* Palavras-chave que variam com o idioma e posição */
content: open-quote;
content: close-quote;
content: no-open-quote;
content: no-close-quote;

/* Com exceção de normal e none, vários valores */
/* podem ser usados simultaneamente */
content: open-quote chapter_counter;

/* Valores globais */
content: inherit;
content: initial;
content: unset;
```

Initial value	`normal`
Aplica-se a	All elements, tree-abiding pseudo-elements, and page margin boxes
Inherited	não
Computed value	On elements, always computes to `normal`. On `::before` and `::after`, if `normal` is specified, computes to `none`. Otherwise, for URI values, the absolute URI; for `attr()` values, the resulting string; for other keywords, as specified.
Animation type	discrete

Sintaxe

Valores

`none`

O pseudoelemento não será gerado.

`normal`

Equivalente a `none` para os pseudoelementos `::before` e `::after`.

`<string>`

Um ou mais caracteres de texto.

`<url>`

Uma URL que designa um recurso externo (como uma imagem). Se o recurso não puder ser exibido, ele será ignorado ou um *placeholder* será exibido no seu lugar.

`<counter>()`

O valor de um contador CSS, geralmente um número. Ele pode ser exibido usando a função `counter()` ou `counters()`.

A função `counter()` tem duas formas: 'counter(*nome*)' ou 'counter(*nome*, *estilo*)'. O texto gerado será o valor do contador mais próximo do nome fornecido no pseudoelemento. Ele será formatado com o estilo especificado (o padrão é `decimal`).

A função `counters()` também tem duas formas: 'counters(*nome*, *string*)' ou 'counters(*nome*, *separador*, *estilo*)'. O texto gerado será o valor de todos os contadores com o mesmo nome no escopo do pseudoelemento, do mais afastado ao mais próximo, separados pelo separador especificado. Os contadores serão exibidos no estilo indicado (o padrão é `decimal`).

`attr(x)`

O valor do atributo `x` do elemento. Se não houver um atributo `x`, uma *string* vazia será retornada. A diferenciação de maiúsculas e minúsculas dependerá da linguagem do documento.

`open-quote` | `close-quote`

Esses valores serão substituídos pela *string* apropriada da propriedade `quotes`.

`no-open-quote` | `no-close-quote`

Não introduzem nenhum conteúdo, mas aumentam (ou diminuem) o nível de aninhamento de aspas.

Sintaxe formal

```
content =
  normal                                                  |
  none                                                    |
  [ <content-replacement> | <content-list> ] [ / <alt-text> ]?  |
  <element()>

<content-replacement> =
  <image>

<content-list> =
  [ <string> | <image> | <attr()> | contents | <quote> | <leader()> | <target> | <string()> | <content()> | <counter> | <symbolic-counter-style>
]+

<alt-text> =
  [ <string> | <counter> | <attr()> ]+

<element()> =
  element( <id-selector> )

<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
  <gradient>
```

```
<attr()> =
  attr( <attr-name> <attr-type>? , <declaration-value>? )

<quote> =
  open-quote      |
  close-quote     |
  no-open-quote   |
  no-close-quote

<leader()> =
  leader( <leader-type> )

<target> =
  <target-counter()>   |
  <target-counters()>  |
  <target-text()>

<string()> =
  string( <custom-ident> , [ first | start | last | first-except ]? )

<content()> =
  content( [ text | before | after | first-letter | marker ]? )

<counter> =
  <counter()>   |
  <counters()>

<id-selector> =
  <hash-token>

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-set()> =
  image-set( <image-set-option># )

<cross-fade()> =
  cross-fade( <cf-image># )

<attr-name> =
  [ <ident-token>? '|' ]? <ident-token>

<attr-type> =
  type( <syntax> )  |
  raw-string        |
  number            |
  <attr-unit>

<leader-type> =
  dotted    |
  solid     |
  space     |
  <string>

<target-counter()> =
  target-counter( [ <string> | <url> ] , <custom-ident> , <counter-style>? )

<target-counters()> =
  target-counters( [ <string> | <url> ] , <custom-ident> , <string> , <counter-style>? )

<target-text()> =
  target-text( [ <string> | <url> ] , [ content | before | after | first-letter ]? )

<counter()> =
  counter( <counter-name> , <counter-style>? )

<counters()> =
  counters( <counter-name> , <string> , <counter-style>? )

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>
```

```
<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<syntax> =
  '*'                                                 |
  <syntax-component> [ <syntax-combinator> <syntax-component> ]*  |
  <syntax-string>

<attr-unit> =
  <custom-ident>

<counter-style> =
  <counter-style-name>  |
  <symbols()>

<counter-name> =
  <custom-ident>

<syntax-component> =
  <syntax-single-component> <syntax-multiplier>?  |
  '<' transform-list '>'

<syntax-combinator> =
  '|'

<syntax-string> =
  <string>

<counter-style-name> =
  <custom-ident>

<symbols()> =
  symbols( <symbols-type>? [ <string> | <image> ]+ )

<syntax-single-component> =
  '<' <syntax-type-name> '>'  |
  <ident>

<syntax-multiplier> =
  '#'  |
  '+'

<symbols-type> =
  cyclic      |
  numeric     |
  alphabetic  |
  symbolic    |
  fixed

<syntax-type-name> =
  angle               |
  color               |
  custom-ident        |
  image               |
  integer             |
  length              |
  length-percentage   |
  number              |
  percentage          |
  resolution          |
  string              |
  time                |
  url                 |
  transform-function
```

Esta sintaxe reflete o padrão mais recente conforme CSS Counter Styles Level 3, CSS Generated Content Module Level 3, CSS Generated Content for Paged Media Module, CSS Images Module Level 4, CSS Images Module Level 5, CSS Lists and Counters Module Level 3, CSS Values and Units Module Level 5, Selectors Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

Cabeçalhos e citações

Este exemplo insere aspas ao redor de citações e adiciona a palavra "Capítulo" antes dos cabeçalhos.

HTML

```
<h1>5</h1>
<p>
  De acordo com o Sr. Tim Berners-Lee,
  <q cite="https://www.w3.org/People/Berners-Lee/FAQ.html#Internet"
    >I was lucky enough to invent the Web at the time when the Internet already
    existed - and had for a decade and a half.</q
  >
  Devemos entender que não há nada fundamentalmente errado em criar algo com
  base nas contribuições de outras pessoas.
</p>

<h1>6</h1>
<p>
  De acordo com o Manifesto Mozilla,
  <q cite="https://www.mozilla.org/about/manifesto/"
    >As pessoas precisam ter a capacidade de moldar a Internet e suas
    experiências com ela.</q
  >
  Portanto, podemos concluir que contribuir para a Web aberta pode proteger
  nossas próprias experiências individuais nela.
</p>
```

CSS

```
q {
  color: blue;
}

q::before {
  content: open-quote;
}

q::after {
  content: close-quote;
}

h1::before {
  content: "Cap\00EDtulo "; /* O espaço no final cria uma separação
                               entre o conteúdo adicionado e o
                               resto do conteúdo */
}
```

Resultado

Imagem combinada com texto

Este exemplo insere uma imagem antes do *link*. Se a imagem não for encontrada, o texto será exibido no seu lugar.

HTML

```
<a href="https://www.mozilla.org/">Mozilla Home Page</a>
```

CSS

```
a::before {
  content: url("https://mozorg.cdn.mozilla.net/media/img/favicon.ico")
    " MOZILLA: ";
  font:
    x-small Arial,
    sans-serif;
  color: gray;
}
```

Resultado

Alterando classes específicas

Este exemplo insere texto adicional no final de itens especiais em uma lista.

HTML

```
<h2>Categoriass mais vendidas</h2>
<ol>
  <li>Suspenses políticos</li>
  <li class="new-entry">Histórias de terror</li>
  <li>Biografias</li>
  <li class="new-entry">Romances de Vampiros</li>
</ol>
```

CSS

```
.new-entry::after {
  content: " Novo!"; /* O espaço no final cria uma separação
                        entre o conteúdo adicionado e o
                        resto do conteúdo */
  color: red;
}
```

Resultado

Imagens e atributos de elementos

Este exemplo insere uma imagem antes de cada *link* e adiciona o conteúdo do seu atributo `id` depois.

HTML

```
<ul>
  <li><a id="moz" href="http://www.mozilla.org/"> Mozilla Home Page</a></li>
  <li>
    <a id="mdn" href="https://developer.mozilla.org/">
      Mozilla Developer Network</a
    >
  </li>
</ul>
```

CSS

```
a {
  text-decoration: none;
  border-bottom: 3px dotted navy;
}

a::after {
  content: " (" attr(id) ")";
}

#moz::before {
  content: url("https://mozorg.cdn.mozilla.net/media/img/favicon.ico");
}

#mdn::before {
  content: url("mdn-favicon16.png");
}

li {
  margin: 1em;
}
```

Resultado

Veja também

- `::after`
- `::before`

- quotes

cursor

Sumário

A propriedade CSS **cursor** especifica o cursor do mouse mostrado quando o ponteiro do mouse está sobre um elemento.

Experimente

```
cursor: help;
```

```
cursor: wait;
```

```
cursor: crosshair;
```

```
cursor: not-allowed;
```

```
cursor: zoom-in;
```

```
cursor: grab;
```

```
<section class="default-example container" id="default-example">
  <div id="example-element">
    Move over this element to see the cursor style.
  </div>
</section>
```

```
#example-element {
  display: flex;
  background-color: #1766aa;
  color: white;
  height: 180px;
  width: 360px;
  justify-content: center;
  align-items: center;
  font-size: 14pt;
  padding: 5px;
}
```

Sintaxe

```
/* Keywords */
cursor: pointer;
cursor: auto;

/* Usando URL e cordenadas */
cursor:
  url(cursor1.png) 4 12,
  auto;
cursor:
  url(cursor2.png) 2 2,
  pointer;

/* Valores globais */
cursor: inherit;
cursor: initial;
cursor: unset;
```

Valores

`<uri>`

> Uma url(...) ou uma lista de url(...) separada por vírgula, apontando para uma arquivo de imagem. Mais que uma `<uri>` pode ser fornecida como fallback, em caso de alguns tipos de imagem não ser suportado. Um fallback não-URL (um ou mais dos outros valores) **deve** estar no final da lista de fallbacks.

`<x>` `<y>`

Coordenadas x e y opcionais. Dois números não-negativos, sem unidade e menores que 32.

`Keyword`

Mova o mouse sobre os valores para testar:

Categoria	Valor CSS	Actual	Descrição
General	`auto`		O brower determina o cursor para ser exibido baseado no contexto atual.
	`default`		Cursor padrão, tipicamente uma seta.
	`none`		Nenhum cursor é mostrado.
Links & status	`context-menu`		Um menu de contexto está disponível sob o cursor. Somente IE 10 e superior implementou esta propriedade no Windows: [Erro do Firefox 258960](https://bugzil.la/258960).
	`help`		Indicando que ajuda está disponível.
	`pointer`		E.g. Usado quando pairando o cursor sobre links, tipicamente uma mão.
	`progress`		O programa está processando em segundo plano, mas o o usuário ainda pode interagir com a interface (diferente de `wait`).
	`wait`		O programa está ocupado (às vezes uma ampulheta ou relógio).
Seleção	`cell`		Indica que a célula pode ser selecionada.
	`crosshair`		Cursor de cruz, normalmente usado para indicar seleção em uma imagem bitmap.
	`text`		Indica que o texto pode ser selecionado, normalmente um I maiúsculo.
	`vertical-text`		Indica que o texto vertical pode ser selecionado, normalmente um I maiúsculo virado de lado.
Arrastar e soltar	`alias`		Indicando que um alias ou atalho deve ser criado.
	`copy`		Indica que algo será copiado.
	`move`		O item sob o mouse pode ser movido.
	`no-drop`		Cursor mostrando que não é permitido largar um elemento no local atual. [Erro do Firefox 275173](https://bugzil.la/275173)no Windows e no Mac OS X, "sem soltar é o mesmo que não permitido".
	`not-allowed`		Cursor mostrando que uma ação não pode ser efetuada.
Resize & scrolling	`all-scroll`		Cursor showing that something can be scrolled in any direction (panned). [Erro do Firefox 275174](https://bugzil.la/275174) on Windows, "*all-scroll* is the same as *move*".
	`col-resize`		The item/column can be resized horizontally. Often rendered as arrows pointing left and right with a vertical bar separating.
	`row-resize`		The item/row can be resized vertically. Often rendered as arrows pointing up and down with a horizontal bar separating them.
	`n-resize`		Some edge is to be moved. For example, the `se-resize` cursor is used when the movement starts from the *south-east* corner of the box.
	`e-resize`		
	`s-resize`		

	`w-resize`		
	`ne-resize`		
	`nw-resize`		
	`se-resize`		
	`sw-resize`		
	`ew-resize`		Indicates a bidirectional resize cursor.
	`ns-resize`		
	`nesw-resize`		
	`nwse-resize`		
Zoom	`zoom-in`		Indicates that something can be zoomed (magnified) in or out.
	`zoom-out`		
Grab	`grab`		Indicates that something can be grabbed (dragged to be moved).
	`grabbing`		

Sintaxe formal

```
cursor =
  [ <cursor-image> , ]* <cursor-predefined>

<cursor-image> =
  [ <url> | <url-set> ] <number>{2}?

<cursor-predefined> =
  auto           |
  default        |
  none           |
  context-menu   |
  help           |
  pointer        |
  progress       |
  wait           |
  cell           |
  crosshair      |
  text           |
  vertical-text  |
  alias          |
  copy           |
  move           |
  no-drop        |
  not-allowed    |
  grab           |
  grabbing       |
  e-resize       |
  n-resize       |
  ne-resize      |
  nw-resize      |
  s-resize       |
  se-resize      |
  sw-resize      |
  w-resize       |
  ew-resize      |
  ns-resize      |
  nesw-resize    |
```

```
  nwse-resize    |
  col-resize     |
  row-resize     |
  all-scroll     |
  zoom-in        |
  zoom-out
```

Esta sintaxe reflete o padrão mais recente conforme CSS Basic User Interface Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

```
.foo {
  cursor: crosshair;
}

/* use velor prefixado se "zoom-in" não for suportado */
.bar {
  cursor: -webkit-zoom-in;
  cursor: zoom-in;
}

/* valor padrão como fallback para url() deve ser fornecido (não funciona sem) */
.baz {
  cursor: url(hyper.cur), auto;
}
```

Definição formal

Initial value	`auto`
Aplica-se a	all elements
Inherited	yes
Computed value	as specified, but with `<url>` values made absolute
Animation type	discrete

Veja também

- `<uri>`
- `pointer-events`

Display

A propriedade **`display`** CSS define se um elemento é tratado como um bloco ou elemento inline e o layout usado para seus filhos, como layout de fluxo, grid ou flex.

Formalmente, a propriedade **`display`** define os *tipos de exibição* internos e externos de um elemento. O tipo externo define a participação de um elemento no layout de fluxo; o tipo interno define o layout dos filhos. Alguns valores de `display` são totalmente definidos em suas próprias especificações individuais; por exemplo, o detalhe do que acontece quando `display: flex` é declarado é definido na especificação CSS Flexible Box Model.

Sintaxe

A propriedade CSS `display` é especificada usando valores de palavra-chave.

```
/* valores pré-compostos */
display: block;
display: inline;
display: inline-block;
display: flex;
display: inline-flex;
display: grid;
display: inline-grid;
display: flow-root;

/* geração de caixas */
display: none;
display: contents;
```

```
/* sintaxe de dois valores */
display: block flow;
display: inline flow;
display: inline flow-root;
display: block flex;
display: inline flex;
display: block grid;
display: inline grid;
display: block flow-root;

/* outros valores */
display: table;
display: table-row; /* todos os elementos da tabela têm um valor de exibição CSS equivalente */
display: list-item;

/* Valores globais */
display: inherit;
display: initial;
display: revert;
display: revert-layer;
display: unset;
```

Valores agrupados

Os valores de palavra-chave podem ser agrupados em seis categorias de valor.

Fora

`<display-outside>`

essas palavras-chave especificam o tipo de exibição externa do elemento, que é essencialmente sua função no layout de fluxo:

`block`

O elemento gera uma caixa de elemento de bloco, gerando quebras de linha antes e depois do elemento quando no fluxo normal.

`inline`

O elemento gera uma ou mais caixas de elemento em linha que não geram quebras de linha antes ou depois de si mesmas. No fluxo normal, o próximo elemento estará na mesma linha se houver espaço.

> **Nota:** Navegadores que suportam a sintaxe de dois valores, ao localizar apenas o valor externo, como quando `display: block` ou `display: inline` é especificado, definirão o valor interno como `flow`. Isso resultará no comportamento esperado; por exemplo, se você especificar um elemento para ser bloco, você esperaria que os filhos desse elemento participassem do bloco e do layout de fluxo normal embutido.

Lado de dentro

`<display-inside>`

Estas palavras-chave especificam o tipo de exibição interna do elemento, que define o tipo de contexto de formatação em que seu conteúdo é apresentado (supondo que seja um elemento não substituído):

`flow`

O elemento apresenta seu conteúdo usando o layout de fluxo (layout de bloco e linha).

Se seu tipo de exibição externa for `inline` ou `run-in`, e estiver participando de um bloco ou contexto de formatação inline, ele gerará uma caixa inline. Caso contrário, gera uma caixa de contêiner de bloco.

Dependendo do valor de outras propriedades (como `position`, `float` ou `overflow`) e se ele próprio está participando em um contexto de formatação em bloco ou em linha, ele estabelece um novo contexto de formatação de bloco (BFC) para seu conteúdo ou integra seu conteúdo em seu contexto de formatação pai.

`flow-root`

O elemento gera uma caixa de elemento de bloco que estabelece um novo contexto de formatação de bloco, definindo onde está a raiz de formatação.

`table`

Esses elementos se comportam como elementos HTML `<table>`. Ele define uma caixa de nível de bloco.

`flex`

O elemento se comporta como um elemento de bloco e apresenta seu conteúdo de acordo com o modelo flexbox.

`grid`

O elemento se comporta como um elemento de bloco e apresenta seu conteúdo de acordo com o modelo de grade.

`ruby`

O elemento se comporta como um elemento inline e apresenta seu conteúdo de acordo com o modelo de formatação ruby. Ele se comporta como os elementos HTML `<ruby>` correspondentes.

> **Nota:** Navegadores que suportam a sintaxe de dois valores, ao encontrar apenas o valor interno, como quando `display: flex` ou `display: grid` é especificado, definirão seu valor externo como `block`. Isso resultará no comportamento esperado; por exemplo, se você especificar um elemento como `display: grid`, você esperaria que a caixa criada no contêiner da grade fosse uma caixa de nível de bloco.

Item de lista

`<display-listitem>`

O elemento gera uma caixa de bloco para o conteúdo e uma caixa inline de item de lista separada.

Um único valor de `list-item` fará com que o elemento se comporte como um item de lista. Isso pode ser usado junto com `list-style-type` e `list-style-position`.

`list-item` também pode ser combinado com qualquer palavra-chave `<display-outside>` e `flow` ou `flow-root` `<display-inside>` palavras-chave.

> **Nota:** Em navegadores que suportam a sintaxe de dois valores, se nenhum valor interno for especificado, o padrão será `flow`. Se nenhum valor externo for especificado, a caixa principal terá um tipo de exibição externa de `bloco`.

Interno

`<display-internal>`

Alguns modelos de layout como `table` e `ruby` possuem uma estrutura interna complexa, com vários papéis diferentes que seus filhos e descendentes podem preencher. Esta seção define os valores de exibição "internos", que só têm significado dentro desse modo de layout específico.

`table-row-group`

Esses elementos se comportam como `<tbody>` elementos HTML.

`table-header-group`

Esses elementos se comportam como `<thead>` elementos HTML.

`table-footer-group`

Esses elementos se comportam como `<tfoot>` elementos HTML.

`table-row`

Esses elementos se comportam como `<tr>` elementos HTML.

`table-cell`

Esses elementos se comportam como `<td>` elementos HTML.

`table-column-group`

Esses elementos se comportam como `<colgroup>` elementos HTML.

`table-column`

Esses elementos se comportam como `<col>` elementos HTML.

`table-caption`

Esses elementos se comportam como `<caption>` elementos HTML.

`ruby-base`

Esses elementos se comportam como `<rb>` elementos HTML.

`ruby-text`

Esses elementos se comportam como `<rt>` elementos HTML.

`ruby-base-container`

Esses elementos são gerados como caixas anônimas.

`ruby-text-container`

Esses elementos se comportam como `<rtc>` elementos HTML.

Caixa

`<display-box>`

Esses valores definem se um elemento gera caixas de exibição.

`contents`

Esses elementos não produzem uma caixa específica por si mesmos. Eles são substituídos por sua pseudocaixa e suas caixas filhas. Observe que a especificação CSS Display Level 3 define como o valor `contents` deve afetar os "elementos incomuns" — elementos que não são renderizados puramente por conceitos de caixa CSS, como elementos substituídos. Consulte Apêndice B: Efeitos da exibição: conteúdo sobre elementos incomuns para obter mais detalhes.

Devido a um bug nos navegadores, isso removerá o elemento da árvore de acessibilidade - os leitores de tela não verão o que está dentro. Consulte a seção Preocupações de acessibilidade abaixo para obter mais detalhes.

`none`

Desativa a exibição de um elemento para que não tenha efeito no layout (o documento é renderizado como se o elemento não existisse). Todos os elementos descendentes também têm sua exibição desligada. Para que um elemento ocupe o espaço que normalmente ocuparia, mas sem realmente renderizar nada, use a propriedade `visibility`.

Pré-composto

`<display-legacy>`

CSS 2 usava uma palavra-chave única, sintaxe pré-composta para a propriedade `display`, exigindo palavras-chave separadas para variantes de nível de bloco e nível inline do mesmo modo de layout.

`inline-block`

O elemento gera uma caixa de elemento de bloco que fluirá com o conteúdo ao redor como se fosse uma única caixa em linha (comportando-se como um elemento substituído faria).

É equivalente a `inline flow-root`.

`inline-table`

O valor `inline-table` não possui um mapeamento direto em HTML. Ele se comporta como um elemento HTML `<table>`, mas como uma caixa embutida, em vez de uma caixa em nível de bloco. Dentro da caixa da tabela há um contexto de nível de bloco.

É equivalente a `tabela inline`.

`inline-flex`

O elemento se comporta como um elemento inline e apresenta seu conteúdo de acordo com o modelo flexbox.

É equivalente a `inline flex`.

`inline-grid`

O elemento se comporta como um elemento inline e apresenta seu conteúdo de acordo com o modelo de grade.

É equivalente a `inline grid`.

Qual sintaxe você deve usar agora?

A especificação de Nível 3 detalha dois valores para a propriedade `display` — habilitando explicitamente a especificação do tipo de exibição externa e interna — mas isso ainda não é bem suportado pelos navegadores.

Os métodos `<display-legacy>` pré-compostos permitem os mesmos resultados com valores de palavra-chave única e devem ser favorecidos pelos desenvolvedores até que os dois valores de palavra-chave sejam melhor suportados. Por exemplo, usando dois valores, você pode especificar um contêiner flexível em linha da seguinte forma:

```
.container {
  display: inline flex;
}
```

Atualmente, isso pode ser especificado usando um único valor.

```
.container {
  display: inline-flex;
}
```

Para obter mais informações sobre essas alterações na especificação, consulte o artigo Adapting to the new two-value sintax of display.

Global

```
/* Valores globais */
display: inherit;
display: initial;
display: unset;
```

Descrição

As páginas individuais para os diferentes tipos de valor que o `display` pode ter definido apresentam vários exemplos desses valores em ação — consulte a seção Sintaxe. Além disso, consulte o material a seguir, que abrange os vários valores de exibição em profundidade.

- Adaptando-se à nova sintaxe de exibição de dois valores

Layout de fluxo CSS (exibir: bloquear, exibir: embutido)

- Block and Inline Layout in Normal Flow
- Flow Layout and Overflow
- Layout de fluxo e modos de escrita
- Contextos de formatação explicados
- Em fluxo e fora de fluxo

display: flex

- Conceitos básicos do flexbox
- Alinhamento de itens em um contêiner flexível
- Controle de proporções de itens flexíveis ao longo do eixo principal
- Mastering Wrapping of Flex Items
- Ordering Flex Items
- Relação do flexbox com outros métodos de layout
- Casos de uso típicos do Flexbox

display: grid

- Conceitos básicos de layout de grade
- Relação com outros métodos de layout
- Posicionamento baseado em linha
- Áreas de modelo de grade
- Layout usando linhas de grade nomeadas
- Colocação automática no layout da grade
- Alinhamento da caixa no layout da grade
- Grades, valores lógicos e modos de escrita
- Layout e acessibilidade da grade CSS
- Layout de grade CSS e aprimoramento progressivo

- Realizando layouts comuns usando grades

Preocupações de acessibilidade

display: none

Usar um valor `display` de `none` em um elemento irá removê-lo da árvore de acessibilidade. Isso fará com que o elemento e todos os seus descendentes não sejam mais anunciados pela tecnologia de leitura de tela.

Se você deseja ocultar visualmente o elemento, uma alternativa mais acessível é usar uma combinação de propriedades para remover visualmente da tela, mas mantenha-o analisável por tecnologia assistiva, como leitores de tela.

display: contents

As implementações atuais na maioria dos navegadores removerão da árvore de acessibilidade qualquer elemento com um valor `display` de `contents` (mas os descendentes permanecerão). Isso fará com que o próprio elemento não seja mais anunciado pela tecnologia de leitura de tela. Este é um comportamento incorreto de acordo com a especificação CSS.

- Marcação mais acessível com exibição: conteúdo | Esconda de Vries
- Exibir: o conteúdo não é uma redefinição de CSS | Adrian Roselli

Tabelas

Alterar o valor `display` de um elemento `<table>` para `block`, `grid` ou `flex` alterará sua representação na árvore de acessibilidade. Isso fará com que a mesa não seja mais anunciada corretamente pela tecnologia de leitura de tela.

- Pequena nota sobre o que as propriedades de exibição CSS fazem com a semântica da tabela — The Paciello Group
- Conteúdo oculto para melhor a11y | Vá fazer coisas
- MDN Understanding WCAG, Diretriz 1.3 explicações
- Entendendo o Critério de Sucesso 1.3.1 | W3C Understanding WCAG 2.0

Definição formal

Initial value	`inline`
Aplica-se a	all elements
Inherited	não
Computed value	as the specified value, except for positioned and floating elements and the root element. In both cases the computed value may be a keyword other than the one specified.
Animation type	Discrete behavior except when animating to or from `none` is visible for the entire duration

Sintaxe formal

```
display =
  [ <display-outside> || <display-inside> ]         |
  <display-listitem>                                |
  <display-internal>                                |
  <display-box>                                     |
  <display-legacy>                                  |
  grid-lanes                                        |
  inline-grid-lanes                                 |
  <display-outside> || [ <display-inside> | math ]

<display-outside> =
  block   |
  inline  |
  run-in

<display-inside> =
  flow       |
  flow-root  |
  table      |
  flex       |
  grid       |
  ruby

<display-listitem> =
  <display-outside>?     &&
  [ flow | flow-root ]?  &&
  list-item
```

```
<display-internal> =
  table-row-group      |
  table-header-group   |
  table-footer-group   |
  table-row            |
  table-cell           |
  table-column-group   |
  table-column         |
  table-caption        |
  ruby-base            |
  ruby-text            |
  ruby-base-container  |
  ruby-text-container

<display-box> =
  contents  |
  none

<display-legacy> =
  inline-block  |
  inline-table  |
  inline-flex   |
  inline-grid
```

Esta sintaxe reflete o padrão mais recente conforme CSS Display Module Level 4, CSS Grid Layout Module Level 3, MathML Core. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

exibir comparação de valores

Neste exemplo, temos dois elementos de contêiner em nível de bloco, cada um com três filhos embutidos. Abaixo disso, temos um menu de seleção que permite aplicar diferentes valores `display` aos contêineres, permitindo comparar e contrastar como os diferentes valores afetam o layout do elemento e de seus filhos.

Incluímos `padding` e `background-color` nos contêineres e seus filhos, para que seja mais fácil ver o efeito que os valores de exibição estão tendo.

> **Nota:** não incluímos nenhuma sintaxe moderna de dois valores, pois o suporte a ela ainda é bastante limitado.

HTML

```
<article class="container">
  <span>Primeiro</span>
  <span>Segundo</span>
  <span>Terceiro</span>
</article>

<article class="container">
  <span>Primeiro</span>
  <span>Segundo</span>
  <span>Terceiro</span>
</article>

<div>
  <label for="display">Escolha um valor de exibição:</label>
  <select id="display">
    <option selected>block</option>
    <option>inline</option>
    <option>inline-block</option>
    <option>none</option>
    <option>flex</option>
    <option>inline-flex</option>
    <option>grid</option>
    <option>inline-grid</option>
    <option>table</option>
    <option>list-item</option>
  </select>
</div>
```

CSS

```
html {
  font-family: helvetica, arial, sans-serif;
  letter-spacing: 1px;
  padding-top: 10px;
}

article {
  background-color: red;
}

article span {
  background-color: black;
  color: white;
  margin: 1px;
}

article,
span {
  padding: 10px;
  border-radius: 7px;
}

article,
div {
  margin: 20px;
}
```

JavaScript

```
const articles = document.querySelectorAll(".container");
const select = document.querySelector("select");

function updateDisplay() {
  articles.forEach((article) => {
    article.style.display = select.value;
  });
}

select.addEventListener("change", updateDisplay);

updateDisplay();
```

Resultado

> **Nota:** você pode encontrar mais exemplos nas páginas para cada tipo de dados de exibição separado, linkado acima.

Veja também

- Layout de bloco e inline no fluxo normal
- Introdução aos contextos de formatação
- `visibility`, `float`, `position`
- `grid`, `flex`

flex

A propriedade flex do CSS, define como um ítem será posicionado para no espaço disponível dentro de seu container.

Experimente

```
flex: 1;
```

```
flex: 2;
```

```
flex: 1 30px;
```

```
flex: 1 1 100px;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">Change me</div>
  <div>flex: 1</div>
  <div>flex: 1</div>
</section>
```

```
.default-example {
  border: 1px solid #c5c5c5;
  width: auto;
  max-height: 300px;
  display: flex;
}

.default-example > div {
  background-color: rgba(0, 0, 255, 0.2);
  border: 3px solid blue;
  margin: 10px;
  flex-grow: 1;
  flex-shrink: 1;
  flex-basis: 0;
}

#example-element {
  background-color: rgba(255, 0, 200, 0.2);
  border: 3px solid rebeccapurple;
}
```

Propriedades

Esta propriedade é uma abreviação das seguintes propriedades CSS:

- `flex-grow`
- `flex-shrink`
- `flex-basis`

```
<div class="flex-container">
  <div class="item auto">auto</div>
  <div class="item auto">auto</div>
  <div class="item auto">auto</div>
</div>

<div class="flex-container">
  <div class="item auto">auto</div>
  <div class="item initial">initial</div>
  <div class="item initial">initial</div>
</div>

<div class="flex-container">
  <div class="item auto">auto</div>
  <div class="item auto">auto</div>
  <div class="item none">none</div>
</div>

<div class="flex-container">
  <div class="item initial">initial</div>
  <div class="item none">none</div>
  <div class="item none">none</div>
</div>

<div class="flex-container">
  <div class="item four">4</div>
  <div class="item two">2</div>
  <div class="item one">1</div>
</div>
```

```
* {
  box-sizing: border-box;
}

.flex-container {
  background-color: #f4f7f8;
```

```
  resize: horizontal;
  overflow: hidden;
  display: flex;
  margin: 1em;
}

.item {
  margin: 1em;
  padding: 0.5em;
  width: 110px;
  min-width: 0;
  background-color: #1b5385;
  color: white;
  font-family: monospace;
  font-size: 13px;
}

.initial {
  flex: initial;
}

.auto {
  flex: auto;
}

.none {
  flex: none;
}

.four {
  flex: 4;
}

.two {
  flex: 2;
}

.one {
  flex: 1;
}
```

By default flex items don't shrink below their minimum content size. To change this, set the item's `min-width` or `min-height`.

Sintaxe

```
/* Propriedades principais */
flex: auto;
flex: initial;
flex: none;

/* Valor único, sem unidade: flex-grow */
flex: 2;

/* Valor único, unidade width/height: flex-basis */
flex: 10em;
flex: 30%;
flex: min-content;

/* Dois valores: flex-grow | flex-basis */
flex: 1 30px;

/* Dois valores: flex-grow | flex-shrink */
flex: 2 2;

/* Três valores: flex-grow | flex-shrink | flex-basis */
flex: 2 2 10%;

/* Valores globais */
flex: inherit;
flex: initial;
flex: unset;
```

The `flex` property may be specified using one, two, or three values.

- **One-value syntax**: the value must be one of:
  - a `<number>`: In this case it is interpreted as `flex: <number> 1 0`; the `<flex-shrink>` value is assumed to be 1 and the `<flex-basis>` value is assumed to be `0`.
  - one of the keywords: `none`, `auto`, or `initial`.
- **Two-value syntax**: the first value must be a `<number>` and it is interpreted as `<flex-grow>`. The second value must be one of:
  - a `<number>`: then it is interpreted as `<flex-shrink>`.
  - a valid value for `width`: then it is interpreted as `<flex-basis>`.
- **Three-value syntax:** the values must be in the following order:
  1. a `<number>` for `<flex-grow>`.
  2. a `<number>` for `<flex-shrink>`.
  3. a valid value for `width` for `<flex-basis>`.

Values

`initial`

The item is sized according to its `width` and `height` properties. It shrinks to its minimum size to fit the container, but does not grow to absorb any extra free space in the flex container. This is equivalent to setting "`flex: 0 1 auto`".

`auto`

The item is sized according to its `width` and `height` properties, but grows to absorb any extra free space in the flex container, and shrinks to its minimum size to fit the container. This is equivalent to setting "`flex: 1 1 auto`".

`none`

The item is sized according to its `width` and `height` properties. It is fully inflexible: it neither shrinks nor grows in relation to the flex container. This is equivalent to setting "`flex: 0 0 auto`".

`<'flex-grow'>`

Defines the `flex-grow` of the flex item. Negative values are considered invalid. Defaults to `1` when omitted.

`<'flex-shrink'>`

Defines the `flex-shrink` of the flex item. Negative values are considered invalid. Defaults to `1` when omitted.

`<'flex-basis'>`

Defines the `flex-basis` of the flex item. A preferred size of `0` must have a unit to avoid being interpreted as a flexibility. Defaults to `0` when omitted.

Formal syntax

```
flex =
  none                                                |
  [ <'flex-grow'> <'flex-shrink'>? || <'flex-basis'> ]

<flex-grow> =
  <number [0,∞]>

<flex-shrink> =
  <number [0,∞]>

<flex-basis> =
  content    |
  <'width'>

<width> =
  auto             |
  <box-size>       |
  <anchor-size()>

<box-size> =
  <length-percentage [0,∞]>                |
  stretch                                  |
```

```
  contain                                 |
  min-content                             |
  max-content                             |
  fit-content                             |
  stretch( <length-percentage [0,∞]> )    |
  fit-content( <length-percentage [0,∞]> ) |
  <calc-size()>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<length-percentage> =
  <length>      |
  <percentage>

<calc-size()> =
  calc-size( <calc-size-basis> , <calc-sum> )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width       |
  height      |
  block       |
  inline      |
  self-block  |
  self-inline

<calc-size-basis> =
  <size-keyword>  |
  <calc-sum>      |
  <calc-size()>   |
  any

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

Esta sintaxe reflete o padrão mais recente conforme CSS Anchor Positioning Module Level 1, CSS Box Sizing Module Level 3, CSS Box Sizing Module Level 4, CSS Flexible Box Layout Module Level 2, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Example

```
#flex-container {
  display: flex;
  flex-direction: row;
}

#flex-container > .flex-item {
  flex: auto;
}

#flex-container > .raw-item {
  width: 5rem;
}
```

```
<div id="flex-container">
  <div class="flex-item" id="flex">Flex box (click to toggle raw box)</div>
  <div class="raw-item" id="raw">Raw box</div>
</div>
```

```
var flex = document.getElementById("flex");
var raw = document.getElementById("raw");
flex.addEventListener("click", function () {
  raw.style.display = raw.style.display == "none" ? "block" : "none";
});
```

```
#flex-container {
  width: 100%;
  font-family: Consolas, Arial, sans-serif;
}

#flex-container > div {
  border: 1px solid #f00;
  padding: 1rem;
}

#flex-container > .raw-item {
  border: 1px solid #000;
}
```

Result

Initial value	as each of the properties of the shorthand: • `flex-grow`: `0` • `flex-shrink`: `1` • `flex-basis`: `auto`
Aplica-se a	flex items, including in-flow pseudo-elements
Inherited	não
Computed value	as each of the properties of the shorthand: • `flex-grow`: as specified • `flex-shrink`: as specified • `flex-basis`: as specified, but with relative lengths converted into absolute lengths
Animation type	as each of the properties of the shorthand: • `flex-grow`: a number • `flex-shrink`: a number • `flex-basis`: a length, percentage or calc();

See also

- CSS Flexbox Guide: *Basic Concepts of Flexbox*
- CSS Flexbox Guide: *Controlling Ratios of flex items along the main axis*

flex-direction

A propriedade CSS **`flex-direction`** define como os itens flexíveis são colocados no contêiner flexível, definindo o eixo principal e a direção (normal ou invertido).

Experimente

```
flex-direction: row;
```

```
flex-direction: row-reverse;
```

```
flex-direction: column;
```

```
flex-direction: column-reverse;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    <div>Item One</div>
    <div>Item Two</div>
    <div>Item Three</div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  width: 80%;
  display: flex;
}

#example-element > div {
  background-color: rgba(0, 0, 255, 0.2);
  border: 3px solid blue;
  width: 60px;
  margin: 10px;
}
```

Observe que os valores `row` e `row-reverse` são afetados pela direcionalidade do contêiner flexível. Se seu atributo `dir` for **`ltr`**, **`row`** representa o eixo horizontal orientado da esquerda para a direita, e `row-reverse` está orientado da direita para esquerda; Se seu atributo **`rtl`**, **`row`** representa o eixo orientado da direita para a esquerda e `row-reverse` da esquerda para a direita.

Syntax

```
/* The direction text is laid out in a line */
flex-direction: row;

/* Like <row>, but reversed */
flex-direction: row-reverse;

/* The direction in which lines of text are stacked */
flex-direction: column;

/* Like <column>, but reversed */
flex-direction: column-reverse;

/* Global values */
flex-direction: inherit;
flex-direction: initial;
flex-direction: unset;
```

Valores

Os seguintes valores são aceitos:

`row`

> O eixo principal do flex container é definido para ser o mesmo que a direção do texto. Os **pontos de início** e **término** principais são iguais à direção do conteúdo.

`row-reverse`

> Se comporta da mesma forma que `row` porém os **pontos de início** e **término** serão invertidos (permutados).

`column`

> O eixo principal do flex container é igual ao eixo do bloco. Os **pontos de início** e **término** principais são iguais aos pontos de antes e depois do modo de escrita.

`column-reverse`

> Se comporta da mesma forma que `column` porém os **pontos de início** e **término** serão invertidos (permutados).

Formal syntax

```
flex-direction =
  row             |
  row-reverse     |
  column          |
  column-reverse
```

Esta sintaxe reflete o padrão mais recente conforme CSS Flexible Box Layout Module Level 2. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Example

HTML

```
<h4>This is a Column-Reverse</h4>
<div id="content">
  <div class="box" style="background-color:red;">A</div>
  <div class="box" style="background-color:lightblue;">B</div>
  <div class="box" style="background-color:yellow;">C</div>
</div>
<h4>This is a Row-Reverse</h4>
<div id="content1">
  <div class="box1" style="background-color:red;">A</div>
  <div class="box1" style="background-color:lightblue;">B</div>
  <div class="box1" style="background-color:yellow;">C</div>
</div>
```

CSS

```
#content {
  width: 200px;
  height: 200px;
  border: 1px solid #c3c3c3;
  display: flex;
  flex-direction: column-reverse;
}

.box {
  width: 50px;
  height: 50px;
}

#content1 {
  width: 200px;
  height: 200px;
  border: 1px solid #c3c3c3;
  display: flex;
  flex-direction: row-reverse;
}

.box1 {
  width: 50px;
  height: 50px;
}
```

Result

Accessibility Concerns

Usar a propriedade `flex-direction` com o valores de `row-reverse` or `column-reverse` vai criar uma desconexão entre a apresentação visual do conteúdo e a ordem do DOM. Isso afetará adversamente os usuários com problemas de visão navegando com o auxílio de tecnologia assistiva, como um leitor de tela. Se a oredem visual (CSS) for importante, os usuários de leitores de tela não terão acesso à ordem de leitura correta.

- Flexbox & the keyboard navigation disconnect — Tink
- Source Order Matters | Adrian Roselli
- MDN Understanding WCAG, Guideline 1.3 explanations
- Understanding Success Criterion 1.3.2 | W3C Understanding WCAG 2.0

Initial value	`row`

Aplica-se a	flex containers
Inherited	não
Computed value	as specified
Animation type	discrete

See also

- CSS Flexbox Guide: *Basic Concepts of Flexbox*
- CSS Flexbox Guide: *Ordering flex items*

flex-flow

A propriedade CSS `flex-flow` é uma junção das propriedades `flex-direction` e `flex-wrap`.

Experimente

```
flex-flow: row wrap;
```

```
flex-flow: row-reverse nowrap;
```

```
flex-flow: column wrap-reverse;
```

```
flex-flow: column wrap;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    <div>Item One</div>
    <div>Item Two</div>
    <div>Item Three</div>
    <div>Item Four</div>
    <div>Item Five</div>
    <div>Item Six</div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  width: 80%;
  max-height: 300px;
  display: flex;
}

#example-element > div {
  background-color: rgba(0, 0, 255, 0.2);
  border: 3px solid blue;
  width: 60px;
  margin: 10px;
}
```

Sintaxe

```
/* flex-flow: <'flex-direction'> */
flex-flow: row;
flex-flow: row-reverse;
flex-flow: column;
flex-flow: column-reverse;

/* flex-flow: <'flex-wrap'> */
flex-flow: nowrap;
flex-flow: wrap;
flex-flow: wrap-reverse;

/* flex-flow: <'flex-direction'> and <'flex-wrap'> */
flex-flow: row nowrap;
flex-flow: column wrap;
flex-flow: column-reverse wrap-reverse;
```

```
/* Global values */
flex-flow: inherit;
flex-flow: initial;
flex-flow: unset;
```

Valores

Consulte `flex-direction` e `flex-wrap` para ver os detalhes sobre os valores.

Sintaxe formal

```
flex-flow =
  <'flex-direction'>  ||
  <'flex-wrap'>

<flex-direction> =
  row             |
  row-reverse     |
  column          |
  column-reverse

<flex-wrap> =
  nowrap                              |
  [ wrap | wrap-reverse ] || balance
```

Esta sintaxe reflete o padrão mais recente conforme CSS Flexible Box Layout Module Level 2. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

```
elemento {
  /* O eixo principal é a direção do bloco com main-start e main-end invertidos.
   * Os itens flex estão dispostos em múltiplas linhas */
  flex-flow: column-reverse wrap;
}
```

Initial value	as each of the properties of the shorthand: • `flex-direction`: `row` • `flex-wrap`: `nowrap`
Aplica-se a	flex containers
Inherited	não
Computed value	as each of the properties of the shorthand: • `flex-direction`: as specified • `flex-wrap`: as specified
Animation type	as each of the properties of the shorthand: • `flex-direction`: discrete • `flex-wrap`: discrete

Veja também

- Guia de Flexbox CSS: *Conceitos básicos de flexbox*
- Guia de Flexbox CSS: *Ordenando itens flex*

flex-wrap

A propriedade CSS **`flex-wrap`** define se os itens flexíveis são forçados a ficarem na mesma linha ou se podem ser quebradas em varias linhas. Se o argumento for valido, ele define a direção em que as linhas são empilhadas.

Experimente

```
flex-wrap: nowrap;
```

```
flex-wrap: wrap;
```

```
flex-wrap: wrap-reverse;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    <div>Item One</div>
    <div>Item Two</div>
    <div>Item Three</div>
    <div>Item Four</div>
    <div>Item Five</div>
    <div>Item Six</div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  width: 80%;
  display: flex;
}

#example-element > div {
  background-color: rgba(0, 0, 255, 0.2);
  border: 3px solid blue;
  width: 60px;
  margin: 10px;
}
```

Veja Using CSS flexible boxes para mais propriedades e informação.

Syntax

```
flex-wrap: nowrap; /* Default value */
flex-wrap: wrap;
flex-wrap: wrap-reverse;

/* Global values */
flex-wrap: inherit;
flex-wrap: initial;
flex-wrap: unset;
```

A propriedade `flex-wrap` é especificada como uma unica palavra-chave escolhida na lista de valores abaixo.

Valores

Os seguintes valores são aceitos :

`nowrap`

> Os itens flexíveis são agrupados em uma unica linha, o que pode fazer com que o flex container transborde. O **cross-start** é equivalente ao **início** ou **antes**, dependendo do valor da `flex-direction`. Este é o valor padrão.

`wrap`

> Os itens flexíveis são quebrados em multiplas linhas. O **cross-start** é equivalente a iniciar ou antes dependendo do valor do `flex-direction` e **cross-end** é o oposto do especificado **cross-start**.

`wrap-reverse`

> Se comporta da mesma maneira que o `wrap` mas a de linha ocorre na direção contrária, ou seja, para a linha acima.

Formal definição

Initial value	`nowrap`
Aplica-se a	flex containers
Inherited	não
Computed value	as specified

Animation type	discrete

Formal syntax

```
flex-wrap =
  nowrap                            |
  [ wrap | wrap-reverse ] || balance
```

Esta sintaxe reflete o padrão mais recente conforme CSS Flexible Box Layout Module Level 2. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

Definindo valores em um flex container wrap

HTML

```
<h4>This is an example for flex-wrap:wrap</h4>
<div class="content">
  <div class="red">1</div>
  <div class="green">2</div>
  <div class="blue">3</div>
</div>
<h4>This is an example for flex-wrap:nowrap</h4>
<div class="content1">
  <div class="red">1</div>
  <div class="green">2</div>
  <div class="blue">3</div>
</div>
<h4>This is an example for flex-wrap:wrap-reverse</h4>
<div class="content2">
  <div class="red">1</div>
  <div class="green">2</div>
  <div class="blue">3</div>
</div>
```

CSS

```
/* Common Styles */
.content,
.content1,
.content2 {
  color: #fff;
  font: 100 24px/100px sans-serif;
  height: 150px;
  text-align: center;
}

.content div,
.content1 div,
.content2 div {
  height: 50%;
  width: 300px;
}
.red {
  background: orangered;
}
.green {
  background: yellowgreen;
}
.blue {
  background: steelblue;
}

/* Flexbox Styles */
.content {
  display: flex;
  flex-wrap: wrap;
}
.content1 {
  display: flex;
  flex-wrap: nowrap;
```

```
}
.content2 {
  display: flex;
  flex-wrap: wrap-reverse;
}
```

Resultados

Veja também

- Guia CSS Flexbox: *Basic Concepts of Flexbox*
- Guia CSS Flexbox: *Mastering wrapping of flex items*

float

Resumo

A propriedade **float** do CSS determina que um elemento deve ser retirado do seu fluxo normal e colocado ao longo do lado direito ou esquerdo do seu containêr, onde textos e elementos em linha irão se posicionar ao seu redor.

Um elemento flutuante é um tipo de elemento cujo valor de `float` é diferente de `none`.

Initial value	none
Aplica-se a	all elements, but has no effect if the value of `display` is `none`.
Inherited	não
Computed value	as specified
Animation type	discrete

Como `float` implica o uso the *block layout,* ele modifica o valor computado de `display` em alguns casos:

Valor Especificado	Valor Computado
`inline`	`block`
`inline-block`	`block`
`inline-table`	`table`
`table-row`	`block`
`table-row-group`	`block`
`table-column`	`block`
`table-column-group`	`block`
`table-cell`	`block`
`table-caption`	`block`
`table-header-group`	`block`
`table-footer-group`	`block`
`flex`	`flex`, porém `float` não produz efeito em tais elementos
`inline-flex`	`inline-flex`, porém `float` não produz efeito em tais elementos
other	*unchanged*

> **Nota:** Se você está se referindo a essa propriedade do Javascript como um membro do objeto `element.style`, você deve referir-se a `cssFloat`. Observe também que as versões 8 e posteriores referem-se a `styleFloat`. Esta é uma exceção à regra de que o nome do membro DOM é o nome em estilo *camel-case* do nome CSS separado por traço (e se deve ao fato de que "float" é uma palavra reservada em Javascript, tal qual a necessidade de especificar "class" como "className" e "for" como "htmlFor").

Sintaxe

```
float: left;
float: right;
```

```
float: none;
float: inline-start;
float: inline-end;

/* Global values */
float: inherit;
float: initial;
float: unset;
```

Valores

`left`

É uma palavra-chave que indica que o elemento deve flutuar à esquerda do bloco.

`right`

É uma palavra-chave que indica que o elemento deve flutuar à direita do bloco.

`none`

É uma palavra-chave que indica que o elemtno não deve flutuar.

`inline-start`

É uma palavra-chave que indica que o elemento deve flutuar no lado inicial do seu bloco, ou seja, o lado esquerdo em scripts `ltr` e no direito em scripts `rtl`.

`inline-end`

É uma palavra-chave que indica que o elemento deve flutuar no lado final do seu bloco, ou seja, o lado direito em scripts `ltr` e no esquerdo em scripts `rtl`.

Sintaxe Formal

```
float =
  block-start      |
  block-end        |
  inline-start     |
  inline-end       |
  snap-block       |
  <snap-block()>   |
  snap-inline      |
  <snap-inline()>  |
  left             |
  right            |
  top              |
  bottom           |
  none             |
  footnote

<snap-block()> =
  snap-block( <length> , [ start | end | near ]? )

<snap-inline()> =
  snap-inline( <length> , [ left | right | near ]? )
```

Esta sintaxe reflete o padrão mais recente conforme CSS Generated Content for Paged Media Module, CSS Page Floats. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

CSS

```
div {
  border: solid red;
  max-width: 70ex;
}
h4 {
  float: left;
  margin: 0;
}
```

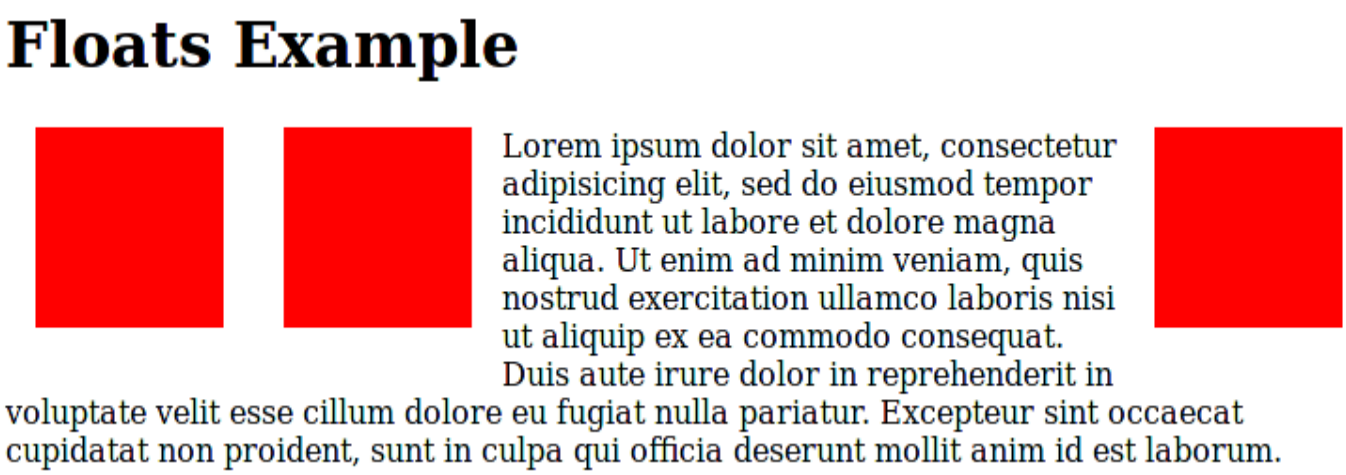
HTML

```
<div>
  <h4>HELLO!</h4>
  This is some text. This is some text. This is some text. This is some text.
  This is some text. This is some text. This is some text. This is some text.
  This is some text. This is some text. This is some text. This is some text.
</div>
```

Como <code>floats</code> são posicionados

Como mencionado acima, quando um elemento é flutuado ele é retirado do seu fluxo normal no documento. Ele é reposicionado à esquerda ou à direita até tocar a borda do seu próprio *box* ou outro elemento flutuante.

Na imagem abaixo, existem 3 quadrados vermelho. Dois flutuando à esquerda e um flutuando à direita. Observe que o segundo quadrado vermelho à esquerda é posicionado à direita do primeiro. Quadrados adicionais poderiam continuar sendo empilhados à direita até que eles completassem o box, após isso eles iriam para próxima linha.



Limpando floats

No exemplo abaixo, os elementos flutuantes são menores verticalmente que o bloco de texto com os quais estão flutuando. Apesar disso, se o texto não fosse comprido o bastante para passar abaixo do limite inferior de todos os *floats,* nos veríamos efeitos não previstos. Se o parágrafo abaixo, por enquanto, somente lê "Lorem ipsum dolor sit amet,", e fosse seguido por outro cabeçalho do mesmo estilo de cabeçaljo "Floats Example", o segundo cabeçalho iria aparecer entre os dois *boxes* vermelhos. Muito provavelmente, queremos que o próximo título seja alinhado à esquerda. Para conseguir isso, precisamos limpar os *floats*.

O jeito mais simples de limpar os floats neste exemplo é adicionar a propriedade `clear` para o próximo cabeçalho que queremos assegurar que esteja alinhado à esquerda.

```
h2.secondHeading {
  clear: both;
}
```

Porém, este método apenas funciona se não existirem outros elementos com o mesmo block formatting context aos quais nós queiramos que o cabeçalho continue aparecendo ao lado horizontalmente. Se o nosso `H2` possui irmãos os quais são barras laterais flutuadas à esquerda e à direita, usar `clear` irá forçá-lo a aparecer abaixo das duas barras laterais, o que provavelmente não é o que queríamos.

Se limpar `floats` em um elemento não é uma opção, uma outra abordagem é limitar o contexto de formatação do bloco do containêr flutuante. Referindo-se ao exemplo acima novamente, aparentemente todos os três quadrados vermelhos possuem um elemento paragrafo `P` . Nós podemos atribuir a propiedade `overflow` neste paragrafo para `hidden` ou `auto` para obrigá-lo a expandir para contê-los, porém não permitirá que os mesmos parágrafos pulem fora do limite inferior.

```
p.withRedBoxes {
  overflow: hidden;
  height: auto;
}
```

> **Nota:** Atribuir `overflow` para `scroll` irá conter também qualquer elemento filho flutuante, mas mostrará as barras de rolagem não importando o comprimento vertical do elemento. Aqui nós estamos atribuindo `height` como `auto` mesmo sendo este o padrão para indicar que o containêr

> deve crescer para acomodar seu conteúdo.

Veja também

- Block formatting context

font-family

Resumo

A propriedade `font-family do` CSS permite que se faça uma lista de prioridades de familias de fontes e/ou nomes genéricos de famílias a serem especificados para um elemento selecionado. Ao contrário da maioria das demais propriedades CSS, os valores são separados por vírgula para indicar quais são as alternativas. O Browser irá utilizar a primeira fonte da lista que for encontrada no computador, ou poderá fazer o dowload utilizando a informação contida na regra `@font-face`.

Programadores WEB devem sempre adicionar pelo menos uma família genérica para a lista de `font-family`, já que não há garantia que aquela fonte específica está instalada no computador, ou possa ser baixada pela regra `@font-face`. A família genérica permite o browser selecionar uma fonte aceita pelo computador, quando necessário.

É também conveniente usar de antemão a propriedade `font` para definir a `font-size` e outras propriedades relacionadas a fonte todas de uma só vez.

> **Nota:** A propriedade `font-family` especifica a lista de fontes, da prioridade mais alta para a mais baixa.``` A seleção de fontes não para simplesmente na primeira fonte nomeada na lista que está no sistema do usuário. Em vez disso, a seleção de fontes é feita um caractere de cada vez, para que, se uma fonte disponível não tiver um glifo que possa exibir um caracter necessário, as fontes disponíveis mais tarde sejam tentadas. No entanto, isso não funciona no Internet Explorer 6 ou anterior.
>
> Quando uma fonte está disponível apenas em alguns estilos, variantes ou tamanhos, essas propriedades também podem influenciar qual família de fontes é escolhida.

Initial value	depends on user agent
Aplica-se a	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	as specified
Animation type	discrete

Syntax

Formal syntax:

```
font-family =
  [ <font-family-name> | <generic-font-family> ]#

<font-family-name> =
  <string>         |
  <custom-ident>+

<generic-font-family> =
  <generic-font-script-specific>  |
  <generic-font-complete>         |
  <generic-font-incomplete>

<generic-font-script-specific> =
  generic( fangsong )   |
  generic( kai )        |
  generic( khmer-mul )  |
  generic( nastaliq )

<generic-font-complete> =
  serif       |
  sans-serif  |
  system-ui   |
  cursive     |
  fantasy     |
  math        |
  monospace
```

```
<generic-font-incomplete> =
  ui-serif       |
  ui-sans-serif  |
  ui-monospace   |
  ui-rounded
```

Esta sintaxe reflete o padrão mais recente conforme CSS Fonts Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

```
font-family: Gill Sans Extrabold, sans-serif
font-family: "Goudy Bookletter 1911", sans-serif

font-family: sans-serif
font-family: serif
font-family: fantasy
font-family: cursive
font-family: monospace

font-family: inherit
```

Valores

`<family-name>`	O nome de uma fonte. Por exemplo, "Times" e "Helvetica" são font families. Nomes de fonte-família contém espaços em branco para serem citados..

`<generic-name>`	Generic font families are a fallback mechanism, a means of preserving some of the style sheet author's intent in case when none of the specified fonts are available. Generic family names are keywords and must not be quoted. A generic font family should be the a last alternative in the list of font family names.
	
	`serif`
	Glyphs have finishing strokes, flared or tapering ends, or have actual serifed endings.
	E.g. Palatino, "Palatino Linotype", Palladio, "URW Palladio", serif
	
	`sans-serif`
	Glyphs have stroke endings that are plain.
	E.g. 'Trebuchet MS', 'Liberation Sans', 'Nimbus Sans L', sans-serif
	
	`monospace`
	All glyphs have the same fixed width.
	E.g. "DejaVu Sans Mono", Menlo, Consolas, "Liberation Mono", Monaco, "Lucida Console", monospace
	
	`cursive`
	Glyphs in cursive fonts generally have either joining strokes or other cursive characteristics beyond those of italic typefaces. The glyphs are partially or completely connected, and the result looks more like handwritten pen or brush writing than printed letterwork.
	
	`fantasy`
	Fantasy fonts are primarily decorative fonts that contain playful representations of characters.

Valid family names

Font family names must either be given quoted as strings, or unquoted as a sequence of one or more identifiers. This means that punctuation characters and digits at the start of each token must be escaped in unquoted font family names.

For example, the following declarations are valid:

```
font-family: Gill Sans Extrabold, sans-serif;
font-family: "Goudy Bookletter 1911", sans-serif;
```

The following declarations are **invalid**:

```
font-family: Goudy Bookletter 1911, sans-serif;
font-family: Red/Black, sans-serif;
font-family: "Lucida" Grande, sans-serif;
font-family: Ahem!, sans-serif;
font-family: test@foo, sans-serif;
```

```
font-family: #POUND, sans-serif;
font-family: Hawaii 5-0, sans-serif;
```

Examples

Example 1

```
body {
  font-family: "Gill Sans Extrabold", Helvetica, sans-serif;
}

.receipt {
  font-family: Courier, "Lucida Console", monospace;
}
```

Example 2

```
.exampleserif {
  font-family: Times, "Times New Roman", Georgia, serif;
}

.examplesansserif {
  font-family: Verdana, Arial, Helvetica, sans-serif;
}

.examplemonospace {
  font-family: "Lucida Console", Courier, monospace;
}

.examplecursive {
  font-family: cursive;
}

.examplefantasy {
  font-family: fantasy;
}
```

```
<div class="exampleserif">This is an example of a serif font.</div>

<div class="examplesansserif">This is an example of a sans-serif font.</div>

<div class="examplemonospace">This is an example of a monospace font.</div>

<div class="examplecursive">This is an example of a cursive font.</div>

<div class="examplefantasy">This is an example of a fantasy font.</div>
```

Live Sample

font-feature-settings

Sumário

A propriedade **`font-feature-settings`** do CSS te dá controle sobre tipografia avançada nas fontes do tipo OpenType.

> **Nota:** Sempre que possível, deve usar o `font-variant` propriedade abreviada ou uma propriedade extensa associada, `font-variant-ligatures`, `font-variant-caps`, `font-variant-east-asian`, `font-variant-alternates`, `font-variant-numeric` ou `font-variant-position`.
>
> Esta propriedade é um recurso de baixo nível projetado para lidar com casos especiais onde não existe outra maneira de habilitar ou acessar um recurso de fonte OpenType.
>
> Em particular, esta propriedade CSS não deve ser usada para habilitar small caps.

Initial value	`normal`
Aplica-se a	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes

Computed value	as specified
Animation type	discrete

Sintaxe

```
/* Use the default settings */
font-feature-settings: normal;

/* Set values for OpenType feature tags */
font-feature-settings: "smcp";
font-feature-settings: "smcp" on;
font-feature-settings: "swsh" 2;
font-feature-settings:
  "smcp",
  "swsh" 2;

/* Global values */
font-feature-settings: inherit;
font-feature-settings: initial;
font-feature-settings: unset;
```

Valores

`normal`

Text is laid out using default settings.

`<feature-tag-value>`

Ao renderizar texto, a lista de características OpenType é passada para o mecanismo de layout de texto para ativar ou desativar recursos do tipo de letra. A tag é sempre uma `<string>` de 4 caracteres ASCII. Se possuir mais ou menos caracteres ou conter caracteres fora da faixa U+20 - U+7E, toda a propriedade é invalida. O valor é um inteiro positivo. As duas palavras-chaves `on` e `off` são sinônimos de `1` e `0` respectivamente. Se nenhum valor é definido, o padrão é `1`. Para características OpenType não-booleanas (ex. stylistic alternates), o valor implica em um glifo particular para ser selecionado; para valores booleanos, é um interruptor.

Sintaxe formal

```
font-feature-settings =
  normal                |
  <feature-tag-value>#

<feature-tag-value> =
  <opentype-tag> [ <integer [0,∞]> | on | off ]?

<opentype-tag> =
  <string>

<integer> =
  <number-token>
```

Esta sintaxe reflete o padrão mais recente conforme CSS Fonts Module Level 4, CSS Values and Units Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

```
/* use small-cap alternate glyphs */
.smallcaps {
  font-feature-settings: "smcp" on;
}

/* convert both upper and lowercase to small caps (affects punctuation also) */
.allsmallcaps {
  font-feature-settings: "c2sc", "smcp";
}

/* enable historical forms */
.hist {
  font-feature-settings: "hist";
}

/* disable common ligatures, usually on by default */
```

```
.noligs {
  font-feature-settings: "liga" 0;
}

/* enable tabular (monospaced) figures */
td.tabular {
  font-feature-settings: "tnum";
}

/* enable automatic fractions */
.fractions {
  font-feature-settings: "frac";
}

/* use the second available swash character */
.swash {
  font-feature-settings: "swsh" 2;
}

/* enable stylistic set 7 */
.fancystyle {
  font-family: Gabriola; /* available on Windows 7, and on Mac OS */
  font-feature-settings: "ss07";
}
```

Veja Também

- FontFont OpenType User Guide (pdf)
- OpenType Feature Tags list
- Using the whole font (The -moz syntax is the old one. On Gecko, use the -ms syntax but with -moz).

font-kerning

A propriedade CSS **`font-kerning`** controla o o uso da informação de kerning, que é, o controle de como as letras serão espaçadas. A informação de kerning é armazenada na fonte, e se a fonte é *well-kerned*, essa caracteristica permite que o espaçamento entre caracteres seja muito parecido, independente dos caracteres.




```
font-kerning: auto;
font-kerning: normal;
font-kerning: none;

/* Global values */
font-kerning: inherit;
font-kerning: initial;
font-kerning: unset;
```

Initial value	`auto`
Aplica-se a	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	as specified

Animation type	discrete

Syntax

Values

`auto`

This keyword defers to the browser regarding whether to use kerning. When the font size is small, font kerning may look strange and browsers will disable it. This is the default value.

`normal`

This keyword requires kerning to be applied.

`none`

This keyword prevents the browser from using the kerning information stored in the font.

Formal syntax

```
font-kerning =
  auto    |
  normal  |
  none
```

Esta sintaxe reflete o padrão mais recente conforme CSS Fonts Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Examples

```
p {
  font-kerning: none;
}
```

Kerning Demo

HTML Content

```
<div id="kern"></div>
<div id="nokern"></div>
<textarea id="input">AV T. ij</textarea>
```

CSS Content

```
#nokern,
#kern {
  font-size: 2rem;
  font-family: serif;
}
#nokern {
  font-kerning: none;
}
#kern {
  font-kerning: normal;
}
```

JS Content

```
var input = document.getElementById("input"),
  kern = document.getElementById("kern"),
  nokern = document.getElementById("nokern");
input.addEventListener("keyup", function () {
  kern.textContent = input.value; /* Update content */
  nokern.textContent = input.value;
});
kern.textContent = input.value; /* Initialize content */
nokern.textContent = input.value;
```

See also

- font-variant, font-variant-position, font-variant-east-asian, font-variant-caps, font-variant-ligatures, font-variant-numeric, font-variant-alternates, font-synthesis, letter-spacing.

font-size

A propriedade **font-size** no CSS estabelece o tamanho da fonte.

Esta propriedade também é usada para computaro tamanho de em, ex, e outras unidades <length> relativas.

Experimente

```
font-size: 1.2rem;
```

```
font-size: x-small;
```

```
font-size: smaller;
```

```
font-size: 12px;
```

```
font-size: 80%;
```

```
<section id="default-example">
  <p id="example-element">
    London. Michaelmas term lately over, and the Lord Chancellor sitting in
    Lincoln's Inn Hall. Implacable November weather. As much mud in the streets
    as if the waters had but newly retired from the face of the earth, and it
    would not be wonderful to meet a Megalosaurus, forty feet long or so,
    waddling like an elephantine lizard up Holborn Hill.
  </p>
</section>
```

Sintaxe

```
/* <absolute-size> values */
font-size: xx-small;
font-size: x-small;
font-size: small;
font-size: medium;
font-size: large;
font-size: x-large;
font-size: xx-large;

/* <relative-size> values */
font-size: smaller;
font-size: larger;

/* <length> values */
font-size: 12px;
font-size: 0.8em;

/* <percentage> values */
font-size: 80%;

/* Global values */
font-size: inherit;
font-size: initial;
font-size: unset;
```

The font-size property is specified in one of the following ways:

- As one of the absolute-size or relative-size keywords
- As a <length> or a <percentage>, relative to the parent element's font size

Valores

xx-small, x-small, small, medium, large, x-large, xx-large

Absolute-size keywords, based on the user's default font size (which is `medium`).

`larger`, `smaller`

Relative-size keywords. The font will be larger or smaller relative to the parent element's font size, roughly by the ratio used to separate the absolute-size keywords above.

`<length>`

A positive `<length>` value. For most font-relative units (such as `em` and `ex`), the font size is relative to the parent element's font size.

For font-relative units that are root-based (such as `rem`), the font size is relative to the size of the font used by the `<html>` (root) element.

`<percentage>`

A positive `<percentage>` value, relative to the parent element's font size.

> **Nota:** To maximize accessibility, it is generally best to use values that are relative to the user's default font size.

Formal syntax

```
font-size =
  <absolute-size>            |
  <relative-size>            |
  <length-percentage [0,∞]>  |
  math

<absolute-size> =
  xx-small   |
  x-small    |
  small      |
  medium     |
  large      |
  x-large    |
  xx-large   |
  xxx-large

<relative-size> =
  larger   |
  smaller

<length-percentage> =
  <length>      |
  <percentage>
```

Esta sintaxe reflete o padrão mais recente conforme CSS Fonts Module Level 4, CSS Values and Units Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Possible approaches

There are several ways to specify the font size, including with keywords or numerical values for pixels or ems. Choose the appropriate method based on the needs for the particular web page.

Keywords

Keywords are a good way to set the size of fonts on the web. By setting a keyword font size on the `<body>` element, you can set relative font-sizing everywhere else on the page, giving you the ability to easily scale the font up or down on the entire page accordingly.

Pixels

Setting the font size in pixel values (`px`) is a good choice when you need pixel accuracy. A `px` value is static. This is an OS-independent and cross-browser way of literally telling the browsers to render the letters at exactly the number of pixels in height that you specified. The results may vary slightly across browsers, as they may use different algorithms to achieve a similar effect.

Font sizing settings can also be used in combination. For example, if a parent element is set to `16px` and its child element is set to `larger`, the child element displays larger than the parent element in the page.

> **Nota:** Defining font sizes in `px` is *not accessible*, because the user cannot change the font size from the browser. For example, users with limited vision may wish to set the font size much larger than the size chosen by a web designer. Avoid using them for font sizes if you wish to create an

> inclusive design.

Ems

Another way of setting the font size is with `em` values. The size of an `em` value is dynamic. When defining the `font-size` property, an em is equal to the size of the font that applies to the parent of the element in question. If you haven't set the font size anywhere on the page, then it is the browser default, which is often 16px. So, by default 1em = 16px, and 2em = 32px. If you set a `font-size` of 20px on the body element, then 1em = 20px and 2em = 40px. Note that the value 2 is essentially a multiplier of the current em size.

In order to calculate the em equivalent for any pixel value required, you can use this formula:

```
em = desired element pixel value / parent element font-size in pixels
```

For example, suppose the font-size of the body of the page is set to 16px. If the font-size you want is 12px, then you should specify 0.75em (because 12/16 = 0.75). Similarly, if you want a font size of 10px, then specify 0.625em (10/16 = 0.625); for 22px, specify 1.375em (22/16).

The em is a very useful unit in CSS, since it automatically adapts its length relative to the font that the reader chooses to use.

One important fact to keep in mind: em values compound. Take the following HTML and CSS:

```
html {
  font-size: 62.5%; /* font-size 1em = 10px on default browser settings */
}
span {
  font-size: 1.6em;
}
```

```
<div>
  <span>Outer <span>inner</span> outer</span>
</div>
```

The result is:

Assuming that the browser's default `font-size` is 16px, the words "outer" would be rendered at 16px, but the word "inner" would be rendered at 25.6px. This is because the inner `<span>`'s `font-size` is 1.6em which is relative to its parent's `font-size`, which is in turn relative to its parent's `font-size`. This is often called **compounding**.

Rems

`rem` values were invented in order to sidestep the compounding problem. `rem` values are relative to the root `html` element, not the parent element. In other words, it lets you specify a font size in a relative fashion without being affected by the size of the parent, thereby eliminating compounding.

The CSS below is nearly identical to the previous example. The only exception is that the unit has been changed to `rem`.

```
html {
  font-size: 62.5%; /* font-size 1em = 10px on default browser settings */
}
span {
  font-size: 1.6rem;
}
```

Then we apply this CSS to the same HTML, which looks like this:

```
<span>Outer <span>inner</span> outer</span>
```

In this example, the words "outer inner outer" are all displayed at 16px (assuming that the browser's `font-size` has been left at the default value of 16px).

Examples

Example 1

```
/* Sets paragraph text to be very large */
p {
  font-size: xx-large;
}
```

```
/* Sets <h1> text to be 2.5 times the size
   of the text around it */
h1 {
  font-size: 250%;
}

/* Sets text enclosed within span tag to be 16px */
span {
  font-size: 16px;
}
```

Example 2

CSS

```
.small {
  font-size: xx-small;
}
.larger {
  font-size: larger;
}
.point {
  font-size: 24pt;
}
.percent {
  font-size: 200%;
}
```

HTML

```
<h1 class="small">Small H1</h1>
<h1 class="larger">Larger H1</h1>
<h1 class="point">24 point H1</h1>
<h1 class="percent">200% H1</h1>
```

Result

Notes

`em` and `ex` units on the `font-size` property are relative to the parent element's font size (unlike all other properties, where they're relative to the font size on the element). This means `em` units and percentages do the same thing for `font-size`.

Initial value	`medium`
Aplica-se a	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Percentages	refer to the parent element's font size
Computed value	absolute `<length>`
Animation type	by computed value type

font-variation-settings

A propriedade CSS **`font-variation-settings`** fornece controle de baixo nível sobre características de variable font, especificando os nomes dos quatro eixos das características que você deseja variar, juntamente com seus valores.

Experimente

```
font-variation-settings: "wght" 50;
```

```
font-variation-settings: "wght" 850;
```

```
font-variation-settings: "wdth" 25;
```

```
font-variation-settings: "wdth" 75;
```

```
<section id="default-example">
  <p id="example-element">
    ...it would not be wonderful to meet a Megalosaurus, forty feet long or so,
    waddling like an elephantine lizard up Holborn Hill.
  </p>
</section>
```

```
@font-face {
  src: url("/shared-assets/fonts/variable-fonts/AmstelvarAlpha-VF.ttf");
  font-family: Amstelvar;
  font-style: normal;
}

p {
  font-size: 1.5rem;
  font-family: Amstelvar;
}
```

> **Nota:** This property is a low-level mechanism designed to set variable font features where no other way to enable or access those features exist. You should only use it when no basic properties exist to set those features (e.g. `font-weight`, `font-style`).

> **Nota:** font characteristics set using `font-variation-settings` will always override those set using the corresponding basic font properties, e.g. `font-weight`, no matter where they appear in the cascade. In some browsers, this is currently only true when the `@font-face` declaration includes a `font-weight` range.

Sintaxe

```
/* Usar as configurações padrão */
font-variation-settings: normal;

/* Definir valores para os nomes dos eixos de fontes variáveis */
font-variation-settings: "XHGT" 0.7;

/* Valores globais */
font-variation-settings: inherit;
font-variation-settings: initial;
font-variation-settings: unset;
```

Values

O valor desta propriedade pode assumir uma de duas formas:

`normal`

O texto é colocado usando as configurações padrão.

`<string> <number>`

Ao renderizar texto, a lista de nomes dos eixos de fontes variáveis é passada para o mecanismo de layout de texto para ativar ou desativar os recursos de fonte. Cada configuração é sempre um ou mais pares que consistem em um `<string>` de 4 caracteres ASCII seguidos por um `<number>` indicando o valor do eixo a ser definido. Se a `<string>` tiver mais ou menos caracteres ou contiver caracteres fora do intervalo de códigos U+20 - U+7E, toda a propriedade será inválida. O `<number>` pode ser fracionário ou negativo, dependendo do intervalo de valores disponível em sua fonte, conforme definido pelo designer da fonte.

```
font-variation-settings =
  normal                       |
  [ <opentype-tag> <number> ]#

<opentype-tag> =
  <string>
```

Esta sintaxe reflete o padrão mais recente conforme CSS Fonts Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Eixos registrados e personalizados

Os eixos de fontes variáveis vêm em dois tipos: **registrados** e **personalizados**.

Os eixos registrados são os mais comumente encontrados - comuns o suficiente para que os autores das especificações considerassem que valia a pena padronizar. Observe que isso não significa que o autor precise incluir tudo isso em sua fonte.

Aqui estão os eixos registrados, juntamente com suas propriedades CSS correspondentes:

Tag do Eixo	Propriedade CSS
"wght"	`font-weight`
"wdth"	`font-stretch`
"slnt" (slant)	`font-style`: `oblique + angle`
"ital"	`font-style`: `italic`
"opsz"	`font-optical-sizing`

Os eixos personalizados podem ser qualquer coisa que o designer da fonte deseje variar em sua fonte, por exemplo, alturas ascendentes ou descendentes, o tamanho das serifas ou qualquer outra coisa que possam imaginar. Qualquer eixo pode ser usado desde que seja dado um eixo exclusivo de 4 caracteres. Alguns acabam se tornando mais comuns e podem até se registrar com o tempo.

> **Nota:** As tags de eixo registrados são identificadas usando tags minúsculas, enquanto os eixos personalizados devem receber tags maiúsculas. Observe que os designers de fontes não são forçados a seguir essa prática de maneira alguma, e alguns não o fazem. O ponto importante aqui é que as tags de eixo diferenciam maiúsculas de minúsculas.

Exemplos

Veja abaixo exemplos interativos que demonstram os eixos registrados. Você pode encontrar vários outros exemplos de fontes variáveis em nosso Variable fonts guide, v-fonts.com, e axis-praxis.org.

> **Warning: Aviso**: Para usar fontes variáveis em seu sistema operacional, você precisa verificar se está atualizado. Por exemplo, os sistemas operacionais Linux precisam da versão mais recente do Linux Freetype, e o macOS anterior à 10.13 não suportam fontes variáveis. Se o seu sistema operacional não estiver atualizado, você não poderá usar fontes variáveis em páginas da Web ou no Firefox Developer Tools.

Peso (wght)

O CSS do exemplo a seguir pode ser editado para permitir que você mude os valores de peso da fonte.

Inclinação (slnt)

O CSS do exemplo a seguir pode ser editado para permitir que você mude os valores oblíquos / oblíquos da fonte.

Initial value	`normal`
Aplica-se a	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	as specified
Animation type	a transform

Veja também

- Variable fonts guide
- OpenType Font Variations Overview
- OpenType Design-Variation Axis Tag Registry

font-weight

Resumo

A propriedade CSS `font-weight` especifica o peso ou a intensidade da fonte (ex.: negrito). Algumas fontes oferecem apenas as opções `normal` e negrito.

Initial value	`normal`
Aplica-se a	all elements and text. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	the keyword or the numerical value as specified, with `bolder` and `lighter` transformed to the real value
Animation type	by computed value type

Sintaxe

```
font-weight: normal;
font-weight: bold;

/* Relativo ao elemento pai */
font-weight: lighter;
font-weight: bolder;

font-weight: 100;
font-weight: 200;
font-weight: 300;
font-weight: 400;
font-weight: 500;
font-weight: 600;
font-weight: 700;
font-weight: 800;
font-weight: 900;

/* Valores globais */
font-weight: inherit;
font-weight: initial;
font-weight: unset;
```

Valores

`normal`

Peso normal. O mesmo que `400`.

`bold`

Peso negrito. O mesmo que `700`.

`lighter`

Um peso de fonte mais claro que o elemento pai (dentre os pesos disponíveis da fonte).

`bolder`

Um peso de fonte mais escuro que o elemento pai (dentre os pesos disponíveis da fonte).

`100`, `200`, `300`, `400`, `500`, `600`, `700`, `800`, `900`

Pesos numéricos para fontes que oferecem mais que normal e negrito.

Contingência

Caso o dado peso solicitado não esteja disponível, a seguinte heurística é usada para determinar o peso efetivo a ser usado:

- Se solicitado um peso maior que `500`, usa-se o próximo peso mais escuro (ou, se não houver, usa-se o próximo mais claro).
- Se solicitado um peso menor que `400`, usa-se o próximo peso mais claro (ou, se não houver, usa-se o próximo mais escuro).
- Se solicitado o peso `400`, usa-se o peso `500`. Se `500` não estiver disponível, usa-se a heurística para peso menor do que 500.
- Se solicitado o peso `500`, usa-se o peso `400`. Se `400` não estiver disponível, usa-se a heurística para peso menor do que 400.

Isto significa que para fontes que oferecem apenas `normal` e negrito, `100`-`500` significa normal, e `600`-`900` significa negrito.

Significado de pesos relativos

Quando for solicitado `lighter` ou `bolder`, a seguinte tabela é usada para calcular o peso absoluto do elemento:

Valor herdado	`bolder`	`lighter`
100	400	100
200	400	100
300	400	100
400	700	100
500	700	100
600	900	400
700	900	400
800	900	700
900	900	700

Mapeamento de nomes comuns de pesos

Os valores de 100 a 900 correspondem grosseiramente aos seguintes nomes comuns de pesos:

`100`

Thin (Hairline)

`200`

Extra Light (Ultra Light, Ultra Fino)

`300`

Light (Fino)

`400`

Normal

`500`

Medium (Médio)

`600`

Semi Bold (Demi Bold, Semi Negrito)

`700`

Bold (Negrito)

`800`

Extra Bold (Ultra Bold, Super Negrito)

`900`

Black (Heavy)

Interpolação

Um valor de `font-weight` é interpolado com passos discretos (múltiplos de 100). A interpolação acontece no espaço de números reais e depois é feita uma conversão para números inteiros por arredondamento para o múltiplo de 100 mais próximo. Valores exatamente entre dois múltiplos de 100 são arredondados para mais.

Sintaxe formal

```
font-weight =
  <font-weight-absolute>  |
  bolder                  |
  lighter

<font-weight-absolute> =
  normal             |
```

```
  bold              |
  <number [1,1000]>
```

Esta sintaxe reflete o padrão mais recente conforme CSS Fonts Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

HTML

```
<p>
  Alice was beginning to get very tired of sitting by her sister on the bank,
  and of having nothing to do: once or twice she had peeped into the book her
  sister was reading, but it had no pictures or conversations in it, 'and what
  is the use of a book,' thought Alice 'without pictures or conversations?'
</p>

<div>
  I'm heavy<br />
  <span>I'm lighter</span>
</div>
```

CSS

```
/* Set paragraph text to be bold. */
p {
  font-weight: bold;
}

/* Set div text to two steps darker than
   normal but less than a standard bold. */
div {
  font-weight: 600;
}

/* Sets text enclosed within span tag
   to be one step lighter than the parent. */
span {
  font-weight: lighter;
}
```

Result

gap

A propriedade CSS **gap** define os espaços (gutters) entre as linhas e colunas. É uma propriedade shorthand para row-gap e column-gap.

Experimente

```
gap: 0;
```

```
gap: 10%;
```

```
gap: 1em;
```

```
gap: 10px 20px;
```

```
gap: calc(20px + 10%);
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">
      <div>One</div>
      <div>Two</div>
      <div>Three</div>
      <div>Four</div>
      <div>Five</div>
    </div>
```

```
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  display: grid;
  grid-template-columns: 1fr 1fr;
  width: 200px;
}

#example-element > div {
  background-color: rgba(0, 0, 255, 0.2);
  border: 3px solid blue;
}
```

É importante notar que `grid-gap` é um pseudônimo para esta propriedade.

Sintaxe

```
/* Um valor de comprimento */
/* Tipo: <length> */
gap: 20px;
gap: 1em;
gap: 3vmin;
gap: 0.5cm;

/* Um valor de porcentagem */
/* Tipo: <percentage> */
gap: 16%;
gap: 100%;

/* Dois valores de comprimento */
gap: 20px 10px;
gap: 1em 0.5em;
gap: 3vmin 2vmax;
gap: 0.5cm 2mm;

/* Um ou dois valores de porcentagem */
gap: 16% 100%;
gap: 21px 82%;

/* Valores de calc() */
gap: calc(10% + 20px);
gap: calc(20px + 10%) calc(10% - 5px);

/* Valores globais */
gap: inherit;
gap: initial;
gap: revert;
gap: revert-layer;
gap: unset;
```

Esta propriedade é especificada como um valor para `<'row-gap'>` seguido opcionalmente por um valor para `<'column-gap'>`. Se o valor para `<'column-gap'>` for omitido, este será definido como o mesmo valor de `<'row-gap'>`.

Os valores de `<'row-gap'>` e `<'column-gap'>` são ambos especificados como `<length>` ou `<percentage>`.

Valores

`<length>`

É o comprimento do espaçamento (gutter) que separa as linhas do grid.

`<percentage>`

É o comprimento do espaçamento (gutter) que separa as linhas do grid, relativo à dimensão do elemento.

Definição formal

Initial value	as each of the properties of the shorthand: • `row-gap`: normal

	• `column-gap`: `normal`
Aplica-se a	multi-column elements, flex containers, grid containers
Inherited	não
Computed value	as each of the properties of the shorthand: • `row-gap`: as specified, with <length>s made absolute, and normal computing to zero except on multi-column elements • `column-gap`: as specified, with <length>s made absolute, and normal computing to zero except on multi-column elements
Animation type	as each of the properties of the shorthand: • `row-gap`: a length, percentage or calc(); • `column-gap`: a length, percentage or calc();

Sintaxe formal

```
gap =
  <'row-gap'> <'column-gap'>?

<row-gap> =
  normal                      |
  <length-percentage [0,∞]>   |
  <line-width>

<column-gap> =
  normal                      |
  <length-percentage [0,∞]>   |
  <line-width>

<length-percentage> =
  <length>      |
  <percentage>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick
```

Esta sintaxe reflete o padrão mais recente conforme CSS Borders and Box Decorations Module Level 4, CSS Gap Decorations Module Level 1, CSS Values and Units Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

Flex layout

HTML

```
<div id="flexbox">
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
</div>
```

CSS

```
#flexbox {
  display: flex;
  flex-wrap: wrap;
  width: 300px;
  gap: 20px 5px;
}
```

```
#flexbox > div {
  border: 1px solid green;
  background-color: lime;
  flex: 1 1 auto;
  width: 100px;
  height: 50px;
}
```

Resultado

Grid layout

HTML

```
<div id="grid">
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
</div>
```

CSS

```
#grid {
  display: grid;
  height: 200px;
  grid-template: repeat(3, 1fr) / repeat(3, 1fr);
  gap: 20px 5px;
}

#grid > div {
  border: 1px solid green;
  background-color: lime;
}
```

Resultado

Layout de múltiplas colunas

HTML

```
<p class="content-box">
  This is some multi-column text with a 40px column gap created with the CSS
  <code>gap</code> property. Don't you think that's fun and exciting? I sure do!
</p>
```

CSS

```
.content-box {
  column-count: 3;
  gap: 40px;
}
```

Resultado

Veja também

- Propriedades CSS relacionadas: `row-gap`, `column-gap`
- Guia para layout Grid: *`Conceitos básicos de Grid Layout - Gutters`*

grid

A propriedade **`grid`** do CSS é uma abreviação das propriedades que definem todas as propriedades da grade explícita (`grid-template-rows`, `grid-template-columns`, e `grid-template-areas`), e todas as propriedades implícitas da grade (`grid-auto-rows`, `grid-auto-columns`, e `grid-auto-flow`), em uma única declaração.

Experimente

```
grid: auto-flow / 1fr 1fr 1fr;
```

```
grid: auto-flow dense / 40px 40px 1fr;
```

```
grid: repeat(3, 80px) / auto-flow;
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">
      <div>One</div>
      <div>Two</div>
      <div>Three</div>
    </div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  display: grid;
  grid-gap: 10px;
  width: 200px;
}

#example-element :nth-child(1) {
  background-color: rgba(0, 0, 255, 0.2);
  border: 3px solid blue;
}

#example-element :nth-child(2) {
  background-color: rgba(255, 0, 200, 0.2);
  border: 3px solid rebeccapurple;
  grid-column: auto / span 3;
  grid-row: auto / span 2;
}

#example-element :nth-child(3) {
  background-color: rgba(94, 255, 0, 0.2);
  border: 3px solid green;
  grid-column: auto / span 2;
}
```

> **Nota:** Você pode especificar apenas as propriedades explícitas *ou* implícitas da grade em uma única declaração `grid`. As subpropriedades que você não especifica são definidas como seu valor inicial, como é normal para abreviações. Além disso, as propriedades da medianiz NÃO são redefinidas por essa abreviação.

Sintaxe

```
/* <'grid-template'> values */
grid: none;
grid: "a" 100px "b" 1fr;
grid: [linename1] "a" 100px [linename2];
grid: "a" 200px "b" min-content;
grid: "a" minmax(100px, max-content) "b" 20%;
grid: 100px / 200px;
grid: minmax(400px, min-content) / repeat(auto-fill, 50px);

/* <'grid-template-rows'> /
   [ auto-flow && dense? ] <'grid-auto-columns'>? values */
grid: 200px / auto-flow;
grid: 30% / auto-flow dense;
grid: repeat(3, [line1 line2 line3] 200px) / auto-flow 300px;
grid: [line1] minmax(20em, max-content) / auto-flow dense 40%;

/* [ auto-flow && dense? ] <'grid-auto-rows'>? /
   <'grid-template-columns'> values */
grid: auto-flow / 200px;
grid: auto-flow dense / 30%;
```

```
grid: auto-flow 300px / repeat(3, [line1 line2 line3] 200px);
grid: auto-flow dense 40% / [line1] minmax(20em, max-content);

/* Global values */
grid: inherit;
grid: initial;
grid: unset;
```

Valores

`<'grid-template'>`

Define o `grid-template` incluindo `grid-template-columns`, `grid-template-rows` e `grid-template-areas`.

`<'grid-template-rows'> / [ auto-flow && dense? ] <'grid-auto-columns'>?`

Configura um fluxo automático definindo as faixas de linha explicitamente por meio da propriedade `grid-template-rows` (e a propriedade `grid-template-columns` para `none`) e especificando como repetir automaticamente as faixas da coluna via `grid-auto-columns` (e setando `grid-auto-rows` para `auto`). `grid-auto-flow` também é definido para a `column`, de acordo com `dense`, se especificado.

Todas as outras subpropriedades `grid` são redefinidas para seus valores iniciais.

`[ auto-flow && dense? ] <'grid-auto-rows'>? / <'grid-template-columns'>`

Configura um fluxo automático definindo as faixas da coluna explicitamente por meio da propriedade `grid-template-columns` (e a propriedade `grid-template-rows` para `none`) e especificando como repetir automaticamente as faixas de linha via `grid-auto-rows` (e setando `grid-auto-columns` para `auto`). `grid-auto-flow` também é definido para `row` de acordo com `dense`, se especificado.

Todas as outras subpropriedades `grid` são redefinidas para seus valores iniciais.

Sintaxe formal

```
grid =
  <'grid-template'>                                  |
  <'grid-template-rows'> / [ auto-flow && dense? ] <'grid-auto-columns'>?  |
  [ auto-flow && dense? ] <'grid-auto-rows'>? / <'grid-template-columns'>

<grid-template> =
  none                                               |
  [ <'grid-template-rows'> / <'grid-template-columns'> ]  |
  [ <line-names>? <string> <track-size>? <line-names>? ]+ [ / <explicit-track-list> ]?

<grid-template-rows> =
  none                        |
  <track-list>                |
  <auto-track-list>           |
  subgrid <line-name-list>?

<grid-auto-columns> =
  <track-size>+

<grid-auto-rows> =
  <track-size>+

<grid-template-columns> =
  none                        |
  <track-list>                |
  <auto-track-list>           |
  subgrid <line-name-list>?

<line-names> =
  '[' <custom-ident>* ']'

<track-size> =
  <track-breadth>                                  |
  minmax( <inflexible-breadth> , <track-breadth> )  |
  fit-content( <length-percentage [0,∞]> )

<explicit-track-list> =
  [ <line-names>? <track-size> ]+ <line-names>?

<track-list> =
  [ <line-names>? [ <track-size> | <track-repeat> ] ]+ <line-names>?
```

```
<auto-track-list> =
  [ <line-names>? [ <fixed-size> | <fixed-repeat> ] ]* <line-names>? <auto-repeat> [ <line-names>? [ <fixed-size> | <fixed-repeat> ] ]* <line-
names>?

<line-name-list> =
  [ <line-names> | <name-repeat> ]+

<track-breadth> =
  <length-percentage [0,∞]>  |
  <flex [0,∞]>               |
  min-content                |
  max-content                |
  auto

<inflexible-breadth> =
  <length-percentage [0,∞]>  |
  min-content                |
  max-content                |
  auto

<length-percentage> =
  <length>      |
  <percentage>

<track-repeat> =
  repeat( [ <integer [1,∞]> ] , [ <line-names>? <track-size> ]+ <line-names>? )

<fixed-size> =
  <fixed-breadth>                              |
  minmax( <fixed-breadth> , <track-breadth> )  |
  minmax( <inflexible-breadth> , <fixed-breadth> )

<fixed-repeat> =
  repeat( [ <integer [1,∞]> ] , [ <line-names>? <fixed-size> ]+ <line-names>? )

<auto-repeat> =
  repeat( [ auto-fill | auto-fit ] , [ <line-names>? <track-size> ]+ <line-names>? )

<name-repeat> =
  repeat( [ <integer [1,∞]> | auto-fill ] , <line-names>+ )

<integer> =
  <number-token>

<fixed-breadth> =
  <length-percentage [0,∞]>
```

Esta sintaxe reflete o padrão mais recente conforme CSS Grid Layout Module Level 2, CSS Grid Layout Module Level 3, CSS Values and Units Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplo

Conteúdo HTML

```
<div id="container">
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
</div>
```

Conteúdo CSS

```
#container {
  display: grid;
  grid: repeat(2, 60px) / auto-flow 80px;
}

#container > div {
  background-color: #8ca0ff;
  width: 50px;
```

```
  height: 50px;
}
```

Resultado

Initial value	as each of the properties of the shorthand: • `grid-template-rows`: `none` • `grid-template-columns`: `none` • `grid-template-areas`: `none` • `grid-auto-rows`: `auto` • `grid-auto-columns`: `auto` • `grid-auto-flow`: `row` • `grid-column-gap`: `0` • `grid-row-gap`: `0` • `column-gap`: `normal` • `row-gap`: `normal`
Aplica-se a	grid containers
Inherited	não
Percentages	as each of the properties of the shorthand: • `grid-template-rows`: refer to corresponding dimension of the content area • `grid-template-columns`: refer to corresponding dimension of the content area • `grid-auto-rows`: refer to corresponding dimension of the content area • `grid-auto-columns`: refer to corresponding dimension of the content area
Computed value	as each of the properties of the shorthand: • `grid-template-rows`: as specified, but with relative lengths converted into absolute lengths • `grid-template-columns`: as specified, but with relative lengths converted into absolute lengths • `grid-template-areas`: as specified • `grid-auto-rows`: the percentage as specified or the absolute length • `grid-auto-columns`: the percentage as specified or the absolute length • `grid-auto-flow`: as specified • `grid-column-gap`: the percentage as specified or the absolute length • `grid-row-gap`: the percentage as specified or the absolute length • `column-gap`: as specified, with <length>s made absolute, and normal computing to zero except on multi-column elements • `row-gap`: as specified, with <length>s made absolute, and normal computing to zero except on multi-column elements
Animation type	as each of the properties of the shorthand: • `grid-template-rows`: simple list of length, percentage, or calc, provided the only differences are in the values of the length, percentage, or calc components in the list • `grid-template-columns`: simple list of length, percentage, or calc, provided the only differences are in the values of the length, percentage, or calc components in the list • `grid-template-areas`: discrete • `grid-auto-rows`: by computed value type • `grid-auto-columns`: by computed value type • `grid-auto-flow`: discrete • `grid-column-gap`: a length • `grid-row-gap`: a length • `column-gap`: a length, percentage or calc(); • `row-gap`: a length, percentage or calc();

Veja também

- Propriedades CSS relacionadas: `grid-template`, `grid-template-rows`, `grid-template-columns`, `grid-template-areas`, `grid-auto-columns`, `grid-auto-rows`, `grid-auto-flow`

- Guia de Layout de Grade: *Line-based placement with CSS Grid*
- Guia de Layout de Grade: *Grid template areas - Grid definition shorthands*

1. **CSS**
2. **CSS Reference**
3. CSS Grid Layout
4. **Guias**
    1. Basics concepts of grid layout
    2. Relationship to other layout methods
    3. Line-based placement
    4. Grid template areas
    5. Layout using named grid lines
    6. Auto-placement in grid layout
    7. Box alignment in grid layout
    8. Grids, logical values and writing modes
    9. CSS Grid Layout and Accessibility
    10. CSS Grid Layout and Progressive Enhancement
    11. Realizing common layouts using grids
5. **Propriedades**
    1. grid
    2. grid-area
    3. grid-auto-columns
    4. grid-auto-flow
    5. grid-auto-rows
    6. grid-column
    7. grid-column-end
    8. grid-column-gap
    9. grid-column-start
    10. grid-gap
    11. grid-row
    12. grid-row-end
    13. grid-row-gap
    14. grid-row-start
    15. grid-template
    16. grid-template-areas
    17. grid-template-columns
    18. grid-template-rows
6. **Glossário**
    1. Grid lines
    2. Grid tracks
    3. Grid cell
    4. Grid areas
    5. Gutters
    6. Grid row
    7. Grid column

grid-auto-flow

A propriedade de CSS **`grid-auto-flow`** controla como o algoritmo de arrumação automática funciona, especificando exatamente como os itens fluirão no grid.

```
/* Keywords Chaves */
grid-auto-flow: row;
grid-auto-flow: column;
grid-auto-flow: dense;
grid-auto-flow: row dense;
grid-auto-flow: column dense;

/* Valores globais */
```

```
grid-auto-flow: inherit;
grid-auto-flow: initial;
grid-auto-flow: unset;
```

Initial value	`row`
Aplica-se a	grid containers
Inherited	não
Computed value	as specified
Animation type	discrete

Syntaxe

Esta propriedade pode ser de duas maneiras:

- uma única keyword (palavra-chave): `row(linha)`, `column`(coluna), ou `dense(denso)`.
- duas keywords (palavras-chave): `row dense(linha densa)` ou `column dense(coluna densa)`.

Valores

`row` (linha)

> Keyword (palavra-chave) que determina que o algoritmo de arrumação automática coloca os itens preenchendo cada linha de uma vez, adicionando novas linhas se necessário. Se nem `row` (linha) ou `column` (coluna) for especificado, `row` (linha) é assumido.

`column` (coluna)

> É uma keyword (palavra-chave) que determina que o algoritmo de arrumação automática coloque os itens preenchendo colunas, adicionando novas se necessário.

`dense` (denso)

> Keyword (palavra-chave) que determina que o algoritmo de arrumação automática use um formato de compactação "denso", que tenta preencher os espaços anteriores do grid caso receba itens menores após. Isto talvez deixe os itens fora de ordem, preenchendo espaços deixados por itens maiores.
>
> Se omitido, o algoritmo "esparso" é utilizado, onde o algoritmo de arrumação somente "avança" no grid ao colocar os itens, nunca voltando para preencher os espaços. Isso garante que todos os itens colocados automaticamente apareçam "em ordem", mesmo que deixem espaçs que poderiam ser preenchidos por itens posteriores.

Sintaxe Formal

```
grid-auto-flow =
  [ row | column ]  ||
  dense
```

Esta sintaxe reflete o padrão mais recente conforme CSS Grid Layout Module Level 2. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplo

Conteúdo HTML

```
<div id="grid">
  <div id="item1"></div>
  <div id="item2"></div>
  <div id="item3"></div>
  <div id="item4"></div>
  <div id="item5"></div>
</div>
<select id="direction" onchange="changeGridAutoFlow()">
  <option value="column">column</option>
  <option value="row">row</option>
</select>
<input id="dense" type="checkbox" onchange="changeGridAutoFlow()" />
<label for="dense">dense</label>
```

Conteúdo CSS

```
#grid {
  height: 200px;
  width: 200px;
  display: grid;
  grid-gap: 10px;
  grid-template: repeat(4, 1fr) / repeat(2, 1fr);
  grid-auto-flow: column; /* or 'row', 'row dense', 'column dense' */
}

#item1 {
  background-color: lime;
  grid-row-start: 3;
}

#item2 {
  background-color: yellow;
}

#item3 {
  background-color: blue;
}

#item4 {
  grid-column-start: 2;
  background-color: red;
}

#item5 {
  background-color: aqua;
}
```

```
function changeGridAutoFlow() {
  var grid = document.getElementById("grid");
  var direction = document.getElementById("direction");
  var dense = document.getElementById("dense");
  var gridAutoFlow = direction.value === "row" ? "row" : "column";

  if (dense.checked) {
    gridAutoFlow += " dense";
  }

  grid.style.gridAutoFlow = gridAutoFlow;
}
```

Veja Também

- Propriedades CSS relacionadas: `grid-auto-rows`, `grid-auto-columns`, `grid`
- Guia de Grid Layout: *Arrumação automática no grid layout*
- Video tutorial: *Introduzindo arrumação automática no grid e ordem*

1. **CSS**
2. **CSS Reference**
3. CSS Grid Layout
4. **Guias**
    1. Conceitos básicos do grid layout
    2. Relacionamento com outros tipos de layout
    3. Arrumação baseada em linha
    4. Grid template areas
    5. Layout usando named grid lines
    6. Arrumação automática no grid layout
    7. Alinhamento de Box no grid layout
    8. Grids, logical values e writing modes
    9. CSS Grid Layout e Acessibilidade
    10. CSS Grid Layout e Progressive Enhancement

11. Identificando layouts comuns usando grids

5. **Propriedades**

    1. grid
    2. grid-area
    3. grid-auto-columns
    4. grid-auto-flow
    5. grid-auto-rows
    6. grid-column
    7. grid-column-end
    8. grid-column-gap
    9. grid-column-start
    10. grid-gap
    11. grid-row
    12. grid-row-end
    13. grid-row-gap
    14. grid-row-start
    15. grid-template
    16. grid-template-areas
    17. grid-template-columns
    18. grid-template-rows

6. **Glossário**

    1. Grid lines
    2. Grid tracks
    3. Grid cell
    4. Grid areas
    5. Gutters
    6. Grid row
    7. Grid column

grid-template-columns

A propriedade `grid-template-columns` do CSS define os nomes das linhas e funções de dimensionamento (track sizing) do grid columns.

```
/* valores padrão */
grid-template-columns: none;

/* exemplos de <track-list> */
grid-template-columns: 100px 1fr;
grid-template-columns: [linename] 100px;
grid-template-columns: [linename1] 100px [linename2 linename3];
grid-template-columns: minmax(100px, 1fr);
grid-template-columns: fit-content(40%);
grid-template-columns: repeat(3, 200px);

/* exemplos de <auto-track-list> */
grid-template-columns: 200px repeat(auto-fill, 100px) 300px;
grid-template-columns:
  minmax(100px, max-content)
  repeat(auto-fill, 200px) 20%;
grid-template-columns:
  [linename1] 100px [linename2]
  repeat(auto-fit, [linename3 linename4] 300px)
  100px;
grid-template-columns:
  [linename1 linename2] 100px
  repeat(auto-fit, [linename1] 300px) [linename3];

/* definições de herança */
grid-template-columns: inherit;
grid-template-columns: initial;
grid-template-columns: unset;
```

Initial value	none

Aplica-se a	grid containers
Inherited	não
Percentages	refer to corresponding dimension of the content area
Computed value	as specified, but with relative lengths converted into absolute lengths
Animation type	simple list of length, percentage, or calc, provided the only differences are in the values of the length, percentage, or calc components in the list

Sintaxe

Alternativas

`none`

Indica que não há uma grade explícita. Colunas serão automaticamente geradas e seu tamanho será determinado pela propriedade `grid-auto-columns`.

`<length>`

Tamanho não negativo.

`<percentage>`

Is a non-negative `<percentage>` value relative to the inline size of the grid container. If the size of the grid container depends on the size of its tracks, then the percentage must be treated as `auto`. The intrinsic size contributions of the track may be adjusted to the size of the grid container and increase the final size of the track by the minimum amount that would result in honoring the percentage.

`<flex>`

Is a non-negative dimension with the unit `fr` specifying the track's flex factor. Each `<flex>`-sized track takes a share of the remaining space in proportion to its flex factor.

When appearing outside a `minmax()` notation, it implies an automatic minimum (i.e. `minmax(auto, <flex>)`).

`max-content`

Is a keyword representing the largest maximal content contribution of the grid items occupying the grid track.

`min-content`

Is a keyword representing the largest minimal content contribution of the grid items occupying the grid track.

`minmax(min, max)`

Is a functional notation that defines a size range greater than or equal to *min* and less than or equal to *max*. If *max* is smaller than *min*, then *max* is ignored and the function is treated as *min*. As a maximum, a `<flex>` value sets the track's flex factor. It is invalid as a minimum.

`auto`

Is a keyword that is identical to maximal content if it's a maximum. As a minimum it represents the largest minimum size (as specified by `min-width`/`min-height`) of the grid items occupying the grid track.

> **Nota:** `auto` track sizes (and only `auto` track sizes) can be stretched by the `align-content` and `justify-content` properties.

`fit-content( [ <length> | <percentage> ] )`

Represents the formula `min(max-content, max(auto, argument))`, which is calculated similar to `auto` (i.e. `minmax(auto, max-content)`), except that the track size is clamped at *argument* if it is greater than the `auto` minimum.

`repeat( [ <positive-integer> | auto-fill | auto-fit ] , <track-list> )`

Represents a repeated fragment of the track list, allowing a large number of columns that exhibit a recurring pattern to be written in a more compact form.

Sintaxe formal

```
grid-template-columns =
  none                        |
  <track-list>                |
  <auto-track-list>           |
  subgrid <line-name-list>?

<track-list> =
  [ <line-names>? [ <track-size> | <track-repeat> ] ]+ <line-names>?

<auto-track-list> =
  [ <line-names>? [ <fixed-size> | <fixed-repeat> ] ]* <line-names>? <auto-repeat> [ <line-names>? [ <fixed-size> | <fixed-repeat> ] ]* <line-
names>?

<line-name-list> =
  [ <line-names> | <name-repeat> ]+

<line-names> =
  '[' <custom-ident>* ']'

<track-size> =
  <track-breadth>                                 |
  minmax( <inflexible-breadth> , <track-breadth> )  |
  fit-content( <length-percentage [0,∞]> )

<track-repeat> =
  repeat( [ <integer [1,∞]> ] , [ <line-names>? <track-size> ]+ <line-names>? )

<fixed-size> =
  <fixed-breadth>                                 |
  minmax( <fixed-breadth> , <track-breadth> )       |
  minmax( <inflexible-breadth> , <fixed-breadth> )

<fixed-repeat> =
  repeat( [ <integer [1,∞]> ] , [ <line-names>? <fixed-size> ]+ <line-names>? )

<auto-repeat> =
  repeat( [ auto-fill | auto-fit ] , [ <line-names>? <track-size> ]+ <line-names>? )

<name-repeat> =
  repeat( [ <integer [1,∞]> | auto-fill ] , <line-names>+ )

<track-breadth> =
  <length-percentage [0,∞]>  |
  <flex [0,∞]>               |
  min-content                |
  max-content                |
  auto

<inflexible-breadth> =
  <length-percentage [0,∞]>  |
  min-content                |
  max-content                |
  auto

<length-percentage> =
  <length>      |
  <percentage>

<integer> =
  <number-token>

<fixed-breadth> =
  <length-percentage [0,∞]>
```

Esta sintaxe reflete o padrão mais recente conforme CSS Grid Layout Module Level 2, CSS Grid Layout Module Level 3, CSS Values and Units Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

CSS

```
#grid {
  display: grid;
  width: 100%;
```

```
  grid-template-columns: 50px 1fr;
}

#areaA {
  background-color: lime;
}

#areaB {
  background-color: yellow;
}
```

HTML

```
<div id="grid">
  <div id="areaA">A</div>
  <div id="areaB">B</div>
</div>
```

Resultado

Leia Mais

- Propriedades CSS relacionadas: `grid-template-rows`, `grid-template-areas`, `grid-template`
- Guia do *layout* em grade: *Basic concepts of grid layout - Grid Tracks*
- Tutorial em vídeo: *Defining a Grid*

1. **CSS**
2. **Referência do CSS**
3. CSS: Layout em grid
4. **Guia**
    1. Basics concepts of grid layout
    2. Relationship to other layout methods
    3. Line-based placement
    4. Grid template areas
    5. Layout using named grid lines
    6. Auto-placement in grid layout
    7. Box alignment in grid layout
    8. Grids, logical values and writing modes
    9. CSS Grid Layout and Accessibility
    10. CSS Grid Layout and Progressive Enhancement
    11. Realizing common layouts using grids
5. **Propriedades**
    1. grid
    2. grid-area
    3. grid-auto-columns
    4. grid-auto-flow
    5. grid-auto-rows
    6. grid-column
    7. grid-column-end
    8. grid-column-gap
    9. grid-column-start
    10. grid-gap
    11. grid-row
    12. grid-row-end
    13. grid-row-gap
    14. grid-row-start
    15. grid-template
    16. grid-template-areas
    17. grid-template-columns
    18. grid-template-rows
6. **Glossário**

1. Grid lines
2. Grid tracks
3. Grid cell
4. Grid areas
5. Gutters
6. Grid row
7. Grid column

grid-template-rows

A propriedade **`grid-template-rows`** do CSS define o nome das linhas e funções de dimensionamento (track sizing) do grid rows.

Experimente

```
grid-template-rows: auto;
```

```
grid-template-rows: 40px 4em 40px;
```

```
grid-template-rows: 1fr 2fr 1fr;
```

```
grid-template-rows: 3ch auto minmax(10px, 60px);
```

```
<section class="default-example" id="default-example">
  <div class="example-container">
    <div class="transition-all" id="example-element">
      <div>One</div>
      <div>Two</div>
      <div>Three</div>
      <div>Four</div>
      <div>Five</div>
    </div>
  </div>
</section>
```

```
#example-element {
  border: 1px solid #c5c5c5;
  display: grid;
  grid-template-columns: 1fr 1fr;
  grid-gap: 10px;
  width: 200px;
}

#example-element > div {
  background-color: rgba(0, 0, 255, 0.2);
  border: 3px solid blue;
}
```

Syntax

```
/* Keyword value */
grid-template-rows: none;

/* <track-list> values */
grid-template-rows: 100px 1fr;
grid-template-rows: [linename] 100px;
grid-template-rows: [linename1] 100px [linename2 linename3];
grid-template-rows: minmax(100px, 1fr);
grid-template-rows: fit-content(40%);
grid-template-rows: repeat(3, 200px);

/* <auto-track-list> values */
grid-template-rows: 200px repeat(auto-fill, 100px) 300px;
grid-template-rows:
  minmax(100px, max-content)
  repeat(auto-fill, 200px) 20%;
grid-template-rows:
  [linename1] 100px [linename2]
  repeat(auto-fit, [linename3 linename4] 300px)
  100px;
```

```
grid-template-rows:
  [linename1 linename2] 100px
  repeat(auto-fit, [linename1] 300px) [linename3];

/* Valores globais */
grid-template-rows: inherit;
grid-template-rows: initial;
grid-template-rows: unset;
```

Esta propriedade pode ser especificada como:

- com a palavra-chave `none`
- ou um valor `<track-list>`
- ou um valor `<auto-track-list>`

Valores

`none`

É uma palavra-chave que indica que não há uma grid explícita. Qualquer linha irá ser implicitamente gerada e seu tamanho será determinado pela a propriedade `grid-auto-rows` .

`<length>`

É um comprimento não negativo.

`<percentagem>`

É um valor de `<percentagem>` não negativo, relativo ao tamanho do bloco do grid contêiner. Se o tamanho do grid contêiner depender do tamanho de suas tracks, então a percentagem deve ser tratada como `auto`. As contribuições de tamanho inerentes da track podem ser ajustados ao tamanho do grid contêiner, e aumentar o tamanho final da track pelo o valor mínimo que respeitará a percentagem.

`<flex>`

Is a non-negative dimension with the unit `fr` specifying the track's flex factor. Each `<flex>`-sized track takes a share of the remaining space in proportion to its flex factor. When appearing outside a `minmax()` notation, it implies an automatic minimum (i.e. `minmax(auto, <flex>)`).

`max-content`

Is a keyword representing the largest maximal content contribution of the grid items occupying the grid track.

`min-content`

Is a keyword representing the largest minimal content contribution of the grid items occupying the grid track.

`minmax(min, max)`

Is a functional notation that defines a size range, greater than or equal to *min*, and less than or equal to *max*. If *max* is smaller than *min*, then *max* is ignored and the function is treated as *min*. As a maximum, a `<flex>` value sets the track's flex factor. It is invalid as a minimum.

`auto`

Is a keyword that is identical to maximal content if it's a maximum. As a minimum it represents the largest minimum size (as specified by `min-width`/`min-height`) of the grid items occupying the grid track.

> **Nota:** `auto` track sizes (and only `auto` track sizes) can be stretched by the `align-content` and `justify-content` properties.

`fit-content( [ <length> | <percentage> ] )`

Represents the formula `min(max-content, max(auto, argument))`, which is calculated similar to `auto` (i.e. `minmax(auto, max-content)`), except that the track size is clamped at *argument* if it is greater than the `auto` minimum.

`repeat( [ <positive-integer> | auto-fill | auto-fit ] , <track-list> )`

Represents a repeated fragment of the track list, allowing a large number of rows that exhibit a recurring pattern to be written in a more compact form.

Formal syntax

```
grid-template-rows =
  none                        |
  <track-list>                |
  <auto-track-list>           |
  subgrid <line-name-list>?

<track-list> =
  [ <line-names>? [ <track-size> | <track-repeat> ] ]+ <line-names>?

<auto-track-list> =
  [ <line-names>? [ <fixed-size> | <fixed-repeat> ] ]* <line-names>? <auto-repeat> [ <line-names>? [ <fixed-size> | <fixed-repeat> ] ]* <line-
names>?

<line-name-list> =
  [ <line-names> | <name-repeat> ]+

<line-names> =
  '[' <custom-ident>* ']'

<track-size> =
  <track-breadth>                                 |
  minmax( <inflexible-breadth> , <track-breadth> )  |
  fit-content( <length-percentage [0,∞]> )

<track-repeat> =
  repeat( [ <integer [1,∞]> ] , [ <line-names>? <track-size> ]+ <line-names>? )

<fixed-size> =
  <fixed-breadth>                                 |
  minmax( <fixed-breadth> , <track-breadth> )       |
  minmax( <inflexible-breadth> , <fixed-breadth> )

<fixed-repeat> =
  repeat( [ <integer [1,∞]> ] , [ <line-names>? <fixed-size> ]+ <line-names>? )

<auto-repeat> =
  repeat( [ auto-fill | auto-fit ] , [ <line-names>? <track-size> ]+ <line-names>? )

<name-repeat> =
  repeat( [ <integer [1,∞]> | auto-fill ] , <line-names>+ )

<track-breadth> =
  <length-percentage [0,∞]>  |
  <flex [0,∞]>               |
  min-content                |
  max-content                |
  auto

<inflexible-breadth> =
  <length-percentage [0,∞]>  |
  min-content                |
  max-content                |
  auto

<length-percentage> =
  <length>      |
  <percentage>

<integer> =
  <number-token>

<fixed-breadth> =
  <length-percentage [0,∞]>
```

Esta sintaxe reflete o padrão mais recente conforme CSS Grid Layout Module Level 2, CSS Grid Layout Module Level 3, CSS Values and Units Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Examples

CSS

```
#grid {
  display: grid;
  height: 100px;
```

```
  grid-template-rows: 30px 1fr;
}

#areaA {
  background-color: lime;
}

#areaB {
  background-color: yellow;
}
```

HTML

```
<div id="grid">
  <div id="areaA">A</div>
  <div id="areaB">B</div>
</div>
```

Result

Initial value	none
Aplica-se a	grid containers
Inherited	não
Percentages	refer to corresponding dimension of the content area
Computed value	as specified, but with relative lengths converted into absolute lengths
Animation type	simple list of length, percentage, or calc, provided the only differences are in the values of the length, percentage, or calc components in the list

See also

- Related CSS properties: `grid-template-columns`, `grid-template-areas`, `grid-template`
- Grid Layout Guide: *Basic concepts of grid layout - Grid Tracks*
- Video tutorial: *Defining a Grid*

1. **CSS**
2. **CSS Reference**
3. CSS Grid Layout
4. **Guides**
    1. Basics concepts of grid layout
    2. Relationship to other layout methods
    3. Line-based placement
    4. Grid template areas
    5. Layout using named grid lines
    6. Auto-placement in grid layout
    7. Box alignment in grid layout
    8. Grids, logical values and writing modes
    9. CSS Grid Layout and Accessibility
    10. CSS Grid Layout and Progressive Enhancement
    11. Realizing common layouts using grids
5. **Properties**
    1. grid
    2. grid-area
    3. grid-auto-columns
    4. grid-auto-flow
    5. grid-auto-rows
    6. grid-column
    7. grid-column-end
    8. grid-column-gap

9. grid-column-start
10. grid-gap
11. grid-row
12. grid-row-end
13. grid-row-gap
14. grid-row-start
15. grid-template
16. grid-template-areas
17. grid-template-columns
18. grid-template-rows

6. **Glossary**

    1. Grid lines
    2. Grid tracks
    3. Grid cell
    4. Grid areas
    5. Gutters
    6. Grid row
    7. Grid column

height

Sumário

A propriedade `height` do CSS determina a altura da área do conteúdo de um elemento. A área de conteúdo consiste no `padding`, `margin` e `border` do elemento.

As propriedades `min-height` e `max-height` sobrepõem a `height`.

Initial value	`auto`
Aplica-se a	all elements but non-replaced inline elements, table columns, and column groups
Inherited	não
Percentages	The percentage is calculated with respect to the height of the generated box's containing block. If the height of the containing block is not specified explicitly (i.e., it depends on content height), and this element is not absolutely positioned, the value computes to `auto`. A percentage height on the root element is relative to the initial containing block.
Computed value	a percentage or `auto` or the absolute length
Animation type	a length, percentage or calc();

Sintaxe

```
Sintaxe formal: [<length> | <percentage>] && [border-box | content-box]? | available | min-content | max-content | fit-content | complex
| auto
```

```
height: auto     /* auto keyword */

height: 120px    /* <length> values */
height: 10em

height: 75%      /* <percentage> values */

height: inherit
```

Valores

`<length>`

Veja `<length>` para unidades disponíveis.

`<percentage>`

Especificada como uma `<percentage>` da altura do conteúdo do bloco.

`border-box`

Se presente, o último `<length>` ou `<percentage>` é aplicado a caixa de borda do elemento.

`content-box`

Se presente, o último `<length>` ou `<percentage>` é aplicado a caixa de conteúdo do elemento.

`auto`

O navegador calculará a altura do conteúdo especificado.

`max-content`

A altura máxima intrínseca.

`min-content`

A altura mínima intrínseca.

`available`

Altura da caixa menos margem vertical, borda e padding.

`fit-content`

O maior de:

- a altura mínima intrínseca
- a menor entre a altura intrínseca preferida e a altura disponível

Exemplos

HTML

```
<div id="red">
  <span>I'm 50 pixels tall.</span>
</div>
<div id="green">
  <span>I'm 25 pixels tall.</span>
</div>
<div id="parent">
  <div id="child">
    <span>I'm half the height of my parent.</span>
  </div>
</div>
```

CSS

```
div {
  width: 250px;
  margin-bottom: 5px;
  border: 3px solid #999999;
}

#red {
  height: 50px;
}

#green {
  height: 25px;
}

#parent {
  height: 100px;
}

#child {
  height: 50%;
  width: 75%;
}
```

Veja também

- box model, `width`, `box-sizing`, `min-height`, `max-height`

hifens

A propriedade CSS **`hyphens`** especifica como palavras devem ser hifenizadas quando o há quebra de texto em múltiplas linhas. Você pode prevenir a hifenização completamente, especificar pontos manualmente, ou permitir que o navegador insira automaticamente quando apropriado.

```
hyphens: none;
hyphens: manual;
hyphens: auto;

/* Valores globais */
hyphens: inherit;
hyphens: initial;
hyphens: unset;
```

Regras de hifenização são específicas para cada idioma. Em HTML, o idioma é determinado pelo atributo `lang`, e os navegadores irão utilizar hífen apenas caso este atributo esteja presente e se houver um dicionário de hifenização disponível. Em XML, deve ser usado o atributo `xml:lang`.

> **Nota:** As regras que definem como a hifenização é realizada não são explicitamente definidas pela especificação, então a hifenização exata pode variar de navegador para navegador.

Initial value	`manual`
Aplica-se a	all elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Sintaxe

A propriedade `hyphens` é especificada como uma única palavra-chave escolhida da lista abaixo.

Valores

`none`

Palavras não são separadas em quebras de linha, mesmo se seus caracteres sugiram pontos de quebra de linha. Linhas são quebradas apenas em espaços em branco.

`manual`

Palavras são separadas apenas onde caracteres dentro de uma palavras sugiram oportunidades de quebra de linha. Veja abaixo Oportunidades sugeridas de quebra de linha para mais detalhes.

`auto`

O navegador é livre para quebrar palavras automaticamente nos pontos apropriados de hifenização, seguindo quaisquer regras que ele escolher. Entretanto, oportunidades sugeridas de quebras de linha (veja Oportunidades sugeridas de quebra de linha abaixo) irão sobrepor a seleção automática de pontos de quebra quando presentes.

> **Nota:** O comportamento da configuração `auto` requer que a propriedade idioma seja indicada corretamente para que as regras de hifenização sejam selecionadas. Você deve especificar o idioma utilizando o atributo HTML `lang` para garantir que a hifenização automática seja aplicada na linguagem de sua escolha.

Oportunidades sugeridas de quebra de linha

Existem dois caracteres Unicode que podem ser utilizados para especificar manualmente potenciais pontos de quebra de linha no texto:

U+2010 (HYPHEN)

O caractere hífen "duro" indica uma oportunidade de quebra de linha visível. Mesmo que não haja quebra de linha naquele ponto, o hífen ainda será renderizado.

U+00AD (SHY)

Um hífen invisivel, "suave". Este caractere não é renderizado visivelmente; ao invés, ele marca um local onde o browser deve quebrar a palavra se um hífen for necessário. Em HTML você pode usar `` para inserir um hífen suave.

Sintaxe formal

```
hyphens =
  none    |
  manual  |
  auto
```

Esta sintaxe reflete o padrão mais recente conforme CSS Text Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplo

Este exemplo usa três classes, uma para cada configuração possível da propriedade `hyphens`.

```
<ul>
  <li>
    <code>none</code>: sem hífen; transbordar se necessário
    <p lang="en" class="none">An extreme&shy;ly long English word</p>
  </li>
  <li>
    <code>manual</code>: hífen apenas em &amp;hyphen; ou &amp;shy; (se
    necessário)
    <p lang="en" class="manual">An extreme&shy;ly long English word</p>
  </li>
  <li>
    <code>auto</code>: hífen onde o algorítmo decidir (se necessário)
    <p lang="en" class="auto">An extreme&shy;ly long English word</p>
  </li>
</ul>
```

```
p {
  width: 55px;
  border: 1px solid black;
}
p.none {
  -webkit-hyphens: none;
  -ms-hyphens: none;
  hyphens: none;
}
p.manual {
  -webkit-hyphens: manual;
  -ms-hyphens: manual;
  hyphens: manual;
}
p.auto {
  -webkit-hyphens: auto;
  -ms-hyphens: auto;
  hyphens: auto;
}
```

Veja também

- `content`

inline-size

> **Experimental: Esta é uma tecnologia experimental**
> Verifique a tabela de compatibilidade entre Navegadores cuidadosamente antes de usar essa funcionalidade em produção.

A propriedade do CSS **`inline-size`** define o tamanho horizontal e vertical de um elemento bloco. Ele é correspondente a cada propriedade `width` ou a `height`, dependendo do valor de `writing-mode`.

```
/* <length> values */
inline-size: 300px;
inline-size: 25em;

/* <percentage> values */
inline-size: 75%;

/* Keyword values */
inline-size: 25em border-box;
inline-size: 75% content-box;
inline-size: max-content;
inline-size: min-content;
inline-size: available;
inline-size: fit-content;
inline-size: auto;

/* Global values */
inline-size: inherit;
inline-size: initial;
inline-size: unset;
```

Se o writing mode é orientado verticalmente, o valor de `inline-size` relaciona a altura do elemento; ao contrário, está relacionado a largura do elemento. A propriedade relacionada é a `block-size`, no qual define a outra dimensão do elemento.

Initial value	`auto`
Aplica-se a	same as `width` and `height`
Inherited	não
Percentages	inline-size of containing block
Computed value	same as `width` and `height`
Animation type	a length, percentage or calc();

Sintaxe

Valores

A propriedade `inline-size` aproveita os mesmos valores que as propriedades `width` e `height`.

Sintaxe formal

```
inline-size =
  auto             |
  <box-size>       |
  <anchor-size()>

<box-size> =
  <length-percentage [0,∞]>                |
  stretch                                  |
  contain                                  |
  min-content                              |
  max-content                              |
  fit-content                              |
  stretch( <length-percentage [0,∞]> )     |
  fit-content( <length-percentage [0,∞]> ) |
  <calc-size()>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<length-percentage> =
  <length>      |
  <percentage>

<calc-size()> =
  calc-size( <calc-size-basis> , <calc-sum> )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width        |
```

```
  height       |
  block        |
  inline       |
  self-block   |
  self-inline

<calc-size-basis> =
  <size-keyword>  |
  <calc-sum>      |
  <calc-size()>   |
  any

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

Esta sintaxe reflete o padrão mais recente conforme CSS Anchor Positioning Module Level 1, CSS Box Sizing Module Level 3, CSS Box Sizing Module Level 4, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplo

HTML

```
<p class="exampleText">Texto de Exemplo</p>
```

CSS

```
.exampleText {
  writing-mode: vertical-rl;
  background-color: yellow;
  inline-size: 110px;
}
```

Veja também

- As propriedades físicas mapeadas: `width` e `height`
- `writing-mode`

isolation

A propriedade CSS **`isolation`** define se o elemento deve criar um novo stacking context.

```
/* Keyword values */
isolation: auto;
isolation: isolate;

/* Global values */
isolation: inherit;
isolation: initial;
isolation: unset;
```

É especialmente útil em conjunto com `background-blend-mode` que mistura os fundos em um dado stacking context: ela permite isolar grupos de elementos dos seus fundos e misturar as suas cores.

Initial value	`auto`
Aplica-se a	All elements. In SVG, it applies to container elements, graphics elements, and graphics referencing elements.
Inherited	não
Computed value	as specified
Animation type	Not animatable

Sintaxe

Uma das palavras-chave listadas abaixo.

Valores

`auto`

> É uma palavra-chave indicando que um novo stacking context deve ser criado somente se uma das propriedades aplicadas ao elemento requererem.

`isolate`

> É uma palavra-chave indicando que um novo stacking context deve ser criado.

Sintaxe formal

```
isolation =
  <isolation-mode>

<isolation-mode> =
  auto     |
  isolate
```

Esta sintaxe reflete o padrão mais recente conforme Compositing and Blending Module Level 2. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

```
<div id="b" class="a">
  <div id="d">
    <div class="a c">auto</div>
  </div>
  <div id="e">
    <div class="a c">isolate</div>
  </div>
</div>
```

```
.a {
  background-color: rgb(0, 255, 0);
}
#b {
  width: 200px;
  height: 210px;
}
.c {
  width: 100px;
  height: 100px;
  border: 1px solid black;
  padding: 2px;
  mix-blend-mode: difference;
}
#d {
  isolation: auto;
}
#e {
  isolation: isolate;
}
```

Veja também

- `<blend-mode>`
- `mix-blend-mode`, `background-blend-mode`

letter-spacing

A propriedade CSS **`letter-spacing`** especifica o comportamento do espaçamento entre caracteres de texto.

Experimente

```
letter-spacing: normal;
```

```
letter-spacing: 0.2rem;
```

```
letter-spacing: 1px;
```

```
letter-spacing: -1px;
```

```
<section id="default-example">
  <p id="example-element">
    As much mud in the streets as if the waters had but newly retired from the
    face of the earth, and it would not be wonderful to meet a Megalosaurus,
    forty feet long or so, waddling like an elephantine lizard up Holborn Hill.
  </p>
</section>
```

```
@font-face {
  src: url("/shared-assets/fonts/variable-fonts/AmstelvarAlpha-VF.ttf");
  font-family: Amstelvar;
  font-style: normal;
}

section {
  font-size: 1.2em;
  font-family: Amstelvar;
}
```

Syntax

```
/* Keyword value */
letter-spacing: normal;

/* <length> values */
letter-spacing: 0.3em;
letter-spacing: 3px;
letter-spacing: 0.3px;

/* Global values */
letter-spacing: inherit;
letter-spacing: initial;
letter-spacing: unset;
```

Values

`normal`

> O espaçamento de letras normal para a fonte atual. Ao contrário de um valor de 0, essa palavra-chave permite que o agente do usuário altere o espaço entre os caracteres para justificar o texto.

`<length>`

> Especifica o espaço adicional entre caracteres, além do espaço padrão entre os caracteres. Os valores podem ser negativos, mas pode haver limites específicos da implementação. Os agentes de usuário podem não aumentar ou diminuir o espaço inter-personagem para justificar o texto.

Formal syntax

```
letter-spacing =
  normal               |
  <length-percentage>

<length-percentage> =
```

```
  <length>      |
  <percentage>
```

Esta sintaxe reflete o padrão mais recente conforme CSS Text Module Level 4, CSS Values and Units Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Examples

HTML

```
<p class="normal">letter spacing</p>
<p class="em-wide">letter spacing</p>
<p class="em-wider">letter spacing</p>
<p class="em-tight">letter spacing</p>
<p class="px-wide">letter spacing</p>
```

CSS

```
.normal {
  letter-spacing: normal;
}
.em-wide {
  letter-spacing: 0.4em;
}
.em-wider {
  letter-spacing: 1em;
}
.em-tight {
  letter-spacing: -0.05em;
}
.px-wide {
  letter-spacing: 6px;
}
```

Result

Initial value	`normal`
Aplica-se a	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	an optimum value consisting of either an absolute length or the keyword `normal`
Animation type	a length

See also

- `font-kerning`

line-break

> **Experimental: Esta é uma tecnologia experimental**
> Verifique a tabela de compatibilidade entre Navegadores cuidadosamente antes de usar essa funcionalidade em produção.

A propriedade CSS **`line-break`** é usada para especificar como (ou se) quebrar linhas.

Sintaxe

```
/* Keyword values */
line-break: auto;
line-break: loose;
line-break: normal;
line-break: strict;

/* Valores Globais */
line-break: inherit;
line-break: initial;
line-break: unset;
```

Valores

`auto`

quebra o texto usando a regra padrão de quebra de linha.

`loose`

quebra o texto usando a regra de quebra de linha que é mais curta, como um jornal.

`normal`

quebra o texto usando a regra padrão de quebra de linha.

`strict`

quebra o texto usando uma regra rigorosa para a quebra de linha.

Sintaxe formal

```
line-break =
  auto      |
  loose     |
  normal    |
  strict    |
  anywhere
```

Esta sintaxe reflete o padrão mais recente conforme CSS Text Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Initial value	`auto`
Aplica-se a	all elements
Inherited	yes
Computed value	as specified
Animation type	discrete

margin

A propriedade **`margin`** do CSS define a área de margem nos quatro lados do elemento. É uma abreviação que define todas as margens individuais de uma só vez: `margin-top`, `margin-right`, `margin-bottom`, e `margin-left`.

Experimente

```
margin: 1em;
```

```
margin: 5% 0;
```

```
margin: 10px 50px 20px;
```

```
margin: 10px 50px 20px 0;
```

```
margin: 0;
```

```
<section id="default-example">
  <div id="container">
    <div class="row"></div>
    <div class="row transition-all" id="example-element"></div>
    <div class="row"></div>
  </div>
</section>
```

```
#container {
  width: 300px;
  height: 200px;
  display: flex;
  align-content: flex-start;
  flex-direction: column;
```

```
  justify-content: flex-start;
}

.row {
  height: 33.33%;
  display: inline-block;
  border: solid #ce7777 10px;
  background-color: #2b3a55;
  flex-shrink: 0;
}

#example-element {
  border: solid 10px #ffbf00;
  background-color: #2b3a55;
}
```

Sintaxe

```
/* Aplica para todos os quatro lados */
margin: 1em;

/* vertical | horizontal */
margin: 5% auto;

/* topo | horizontal | inferior */
margin: 1em auto 2em;

/* topo | direita | inferior | esquerda */
margin: 2px 1em 0 auto;

/* Valores globais */
margin: inherit;
margin: initial;
margin: unset;
```

A propriedade `margin` pode ser especificada usando um, dois, três ou quatro valores. Cada valor deve ser um `<length>`, uma `<percentage>`, ou a palavra-chave `auto`. Cada valor pode ser positivo, zero ou negativo.

- Quando **um** valor é especificado, a mesma margem é aplicada para **todos os quatro lados**.
- Quando **dois** valores são especificados, a primeira margem é aplicada aos **lados superior e inferior,** e a segunda aos **lados esquerdo e direito**.
- Quando **três** valores são especificados, a primeira margem é apliacada ao **topo**, a segunda aos **lados esquerdo e direito**, e a terceira ao **lado inferior**.
- Quando **quatro** valores são especificados, as margens são aplicadas aos lados **superior**, **direito**, **inferior** e **esquerdo**, nesta ordem (sentido horário).

Valores

`<length>`

O tamanho da margem como um valor fixo.

`<percentage>`

O tamanho da margem como um percetual, relativo à *largura* do bloco em que o elemento está contido.

`auto`

O navegador seleciona uma margem adequada para utilizar. Por exemplo, em alguns casos este valor pode ser utilizado para centralizar o elemento.

Sintaxe formal

```
margin =
  <'margin-top'>{1,4}

<margin-top> =
  <length-percentage>  |
  auto                 |
  <anchor-size()>

<length-percentage> =
```

```
  <length>      |
  <percentage>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width        |
  height       |
  block        |
  inline       |
  self-block   |
  self-inline
```

Esta sintaxe reflete o padrão mais recente conforme CSS Anchor Positioning Module Level 1, CSS Box Model Module Level 4, CSS Values and Units Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

Exemplo simples

HTML

```
<div class="center">Este elemento está centralizado.</div>

<div class="outside">
  Este elemento está posicionado fora de seu bloco recipiente.
</div>
```

CSS

```
.center {
  margin: auto;
  background: lime;
  width: 66%;
}

.outside {
  margin: 3rem 0 0 -3rem;
  background: cyan;
  width: 66%;
}
```

Mais exemplos

```
margin: 5%; /* todos os lados: margem de 5% */

margin: 10px; /* todos os lados: margem de 10px */

margin: 1.6em 20px; /* topo e inferior:    margem de 1.6em */
/* esquerda e direita: margem de 20px  */

margin: 10px 3% 1em; /* topo:               margem de 10px */
/* esquerda e direita: margem de 3%   */
/* inferior:           margem de 1em  */

margin: 10px 3px 30px 5px; /* topo:     margem de 10px */
/* direita:  margem de 3px  */
/* inferior: margem de 30px */
/* esquerda: margem de 5px  */

margin: 2em auto; /* topo e inferior: margem de 2em          */
/* caixa está horizontalmente centralizada */

margin: auto; /* topo e inferior: margem de 0            */
/* caixa está horizontalmente centralizada */
```

Notas

Centralização horizontal

Para centralizar algo horizontalmente em navegadores modernos, você pode utilizar `display: flex; justify-content: center;`

Contudo, em navegadores antigos, como IE8-9 que não suporta layout flexbox, estes não estão disponíveis. Para centralizar um elemento dentro de seu pai, use `margin: 0 auto;` .

Colapso de margens

Às vezes, as margens superior e inferior de elementos são colapsadas em uma única margem que é igual à maior das duas margens. Veja Dominando margin collapsing para mais informações.

Initial value	as each of the properties of the shorthand: • `margin-bottom`: `0` • `margin-left`: `0` • `margin-right`: `0` • `margin-top`: `0`
Aplica-se a	all elements, except elements with table `display` types other than `table-caption`, `table` and `inline-table`. It also applies to `::first-letter`.
Inherited	não
Percentages	refer to the width of the containing block
Computed value	as each of the properties of the shorthand: • `margin-bottom`: the percentage as specified or the absolute length • `margin-left`: the percentage as specified or the absolute length • `margin-right`: the percentage as specified or the absolute length • `margin-top`: the percentage as specified or the absolute length
Animation type	a length

Veja também

- CSS Box Model
- Margin collapsing

margin-bottom

A propriedade CSS **`margin-bottom`** define a margin area na parte inferior de um elemento. Um valor positivo coloca o valor mais longe de outros elementos vizinhos, enquanto um valor negativo, aproxima.

Experimente

```
margin-bottom: 1em;
```

```
margin-bottom: 10%;
```

```
margin-bottom: 10px;
```

```
margin-bottom: 0;
```

```
<section id="default-example">
  <div id="container">
    <div class="row"></div>
    <div class="row transition-all" id="example-element"></div>
    <div class="row"></div>
  </div>
</section>
```

```
#container {
  width: 300px;
  height: 200px;
  display: flex;
  align-content: flex-start;
  flex-direction: column;
  justify-content: flex-start;
}

.row {
  height: 33.33%;
  display: inline-block;
  border: solid #ce7777 10px;
  background-color: #2b3a55;
  flex-shrink: 0;
}

#example-element {
  border: solid 10px #ffbf00;
  background-color: #2b3a55;
}
```

A propriedade não tem efeito em non-replaced elementos inline, como `<span>` ou `<code>`.

Sintaxe

```
/* <length> values */
margin-bottom: 10px; /* Um tamanho absoluto */
margin-bottom: 1em; /* relativo ao tamanho do texto */
margin-bottom: 5%; /* relativo à largura do container do bloco mais próximo */

/* Keyword values */
margin-bottom: auto;

/* Global values */
margin-bottom: inherit;
margin-bottom: initial;
margin-bottom: unset;
```

A propriedade `margin-bottom` é especificada como a palavra chave `auto`, ou a `<length>`, ou a`<percentage>`. Esse valor pode ser positivo, zero ou negativo

Values

`<length>`

O tamanho de uma margin com um valor fixo

`<percentage>`

O tamanho de uma margin como porcentagem, é relativa com o *tamanho* do bloco pai.

`auto`

O navegador seleciona um valor adequado para usar. Veja `margin`.

Sintaxe formal

```
margin-bottom =
  <length-percentage>  |
  auto                 |
  <anchor-size()>

<length-percentage> =
  <length>      |
  <percentage>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<anchor-name> =
  <dashed-ident>
```

```
<anchor-size> =
  width       |
  height      |
  block       |
  inline      |
  self-block  |
  self-inline
```

Esta sintaxe reflete o padrão mais recente conforme CSS Anchor Positioning Module Level 1, CSS Box Model Module Level 4, CSS Values and Units Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplo

HTML

```
<div class="container">
  <div class="box0">Box 0</div>
  <div class="box1">Box 1</div>
  <div class="box2">Box one's negative margin pulls me up</div>
</div>
```

CSS

CSS para definir margin-bottom e height

```
.box0 {
  margin-bottom: 1em;
  height: 3em;
}
.box1 {
  margin-bottom: -1.5em;
  height: 4em;
}
.box2 {
  border: 1px dashed black;
  border-width: 1px 0;
  margin-bottom: 2em;
}
```

Algumas definições para container e divs, para que o os efeitos de margin possam ser vistos com mais clareza

```
.container {
  background-color: orange;
  width: 320px;
  border: 1px solid black;
}
div {
  width: 320px;
  background-color: gold;
}
```

Initial value	`0`
Aplica-se a	all elements, except elements with table `display` types other than `table-caption`, `table` and `inline-table`. It also applies to `::first-letter`.
Inherited	não
Percentages	refer to the width of the containing block
Computed value	the percentage as specified or the absolute length
Animation type	a length

margin-top

Sumário

A propriedade CSS **`margin-top`** de um elemento indica o espaço acima do elemento. Valores negativos são aceitos.

Essa propriedade não tem efeito em *elementos non-replaced* inline elements, como `<tt>` ou `<span>`.

Initial value	`0`
Aplica-se a	all elements, except elements with table `display` types other than `table-caption`, `table` and `inline-table`. It also applies to `::first-letter`.
Inherited	não
Percentages	refer to the width of the containing block
Computed value	the percentage as specified or the absolute length
Animation type	a length

Sintaxe

```
/* Valores <length> */
margin-top: 10px; /* An absolute length */
margin-top: 1em; /* A length relative to the text size */
margin-top: 5%; /* A margin relative to the nearest block container's width */

/* Valores padrão */
margin-top: auto;

/* Valores globais */
margin-top: inherit;
margin-top: initial;
margin-top: unset;
```

Valores

`<length>`

Especifica um tamanho fixo. Veja `<length>` para possíveis valores.

`<porcentagem>`

Um `<percentage>` sempre relativo ao **width** do bloco.

`auto`

Veja `margin`.

Sintaxe formal

```
margin-top =
  <length-percentage>  |
  auto                 |
  <anchor-size()>

<length-percentage> =
  <length>      |
  <percentage>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width        |
  height       |
  block        |
  inline       |
```

```
  self-block   |
  self-inline
```

Esta sintaxe reflete o padrão mais recente conforme CSS Anchor Positioning Module Level 1, CSS Box Model Module Level 4, CSS Values and Units Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

```
.content {
  margin-top: 5%;
}
.sidebox {
  margin-top: 10px;
}
.logo {
  margin-top: -5px;
}
#footer {
  margin-top: 1em;
}
```

mask

Resumo

A propriedade máscara no CSS permite aos usuários alterarem a visibilidade de um item parcialmente ou totalmente escondendo o item. Isso é obtido por qualquer mascaramento ou cortes na imagem em pontos específicos.

Initial value	as each of the properties of the shorthand: • `mask-image`: `none` • `mask-mode`: `match-source` • `mask-repeat`: `repeat` • `mask-position`: `0% 0%` • `mask-clip`: `border-box` • `mask-origin`: `border-box` • `mask-size`: `auto` • `mask-composite`: `add`
Aplica-se a	all elements; In SVG, it applies to container elements excluding the `<defs>` element and all graphics elements
Inherited	não
Percentages	as each of the properties of the shorthand: • `mask-position`: refer to size of mask painting area minus size of mask layer image (see the text for `background-position`)
Computed value	as each of the properties of the shorthand: • `mask-image`: as specified, but with `<url>` values made absolute • `mask-mode`: as specified • `mask-repeat`: Consists of two keywords, one per dimension • `mask-position`: Consists of two keywords representing the origin and two offsets from that origin, each given as an absolute length (if given a <length>), otherwise as a percentage. • `mask-clip`: as specified • `mask-origin`: as specified • `mask-size`: as specified, but with relative lengths converted into absolute lengths • `mask-composite`: as specified
Animation type	as each of the properties of the shorthand: • `mask-image`: discrete • `mask-mode`: discrete • `mask-repeat`: discrete

	• `mask-position`: a repeatable list • `mask-clip`: discrete • `mask-origin`: discrete • `mask-size`: a repeatable list • `mask-composite`: discrete
Creates stacking context	yes

Sintaxe

```
/* Palavra-Chave */
mask: none;

/* Valor das Imagens */
mask: url(mask.png); /* Imagem bitmap usada da máscara */
mask: url(masks.svg#star); /* Elemento dentro do SVG usado como máscara */

/* Valores Combinados */
mask: url(masks.svg#star) luminance; /* Elemento dentro do SVG usado como máscara de luminância */
mask: url(masks.svg#star) 40px 20px; /* Elemento dentro do SVG usado como máscara posicionada 40px do topo e 20px da esquerda */
mask: url(masks.svg#star) 0 0/50px 50px; /* Elemento dentro do SVG usado como máscara com a largura e altura de 50px */
mask: url(masks.svg#star) repeat-x; /* Elemento dentro do SVG usado como máscara repedida horizontalmente */
mask: url(masks.svg#star) stroke-box; /* Elemento dentro do SVG usado como máscara extendendo-se até a caixa delimitada pela linha */
mask: url(masks.svg#star) exclude; /* Elemento dentro do SVG usado como máscara e combinado com o fundo usando partes que não se
sobrepõem */

/* Valores Globais */
mask: inherit;
mask: initial;
mask: unset;
```

Valores

Se o valor é um valor URI, o elemento apontado pelo URI é usado como uma máscara SVG.

```
mask =
  <mask-layer>#

<mask-layer> =
  <mask-reference>             ||
  <position> [ / <bg-size> ]?  ||
  <repeat-style>               ||
  <geometry-box>               ||
  [ <geometry-box> | no-clip ] ||
  <compositing-operator>       ||
  <masking-mode>

<mask-reference> =
  none          |
  <image>       |
  <mask-source>

<position> =
  <position-one>   |
  <position-two>   |
  <position-four>

<bg-size> =
  [ <length-percentage [0,∞]> | auto ]{1,2}  |
  cover                                      |
  contain

<repeat-style> =
  repeat-x          |
  repeat-y          |
  repeat-block      |
  repeat-inline     |
  <repetition>{1,2}

<geometry-box> =
  <shape-box>  |
  fill-box     |
```

```
  stroke-box   |
  view-box

<compositing-operator> =
  add        |
  subtract   |
  intersect  |
  exclude

<masking-mode> =
  alpha         |
  luminance     |
  match-source

<image> =
  <url>           |
  <image()>       |
  <image-set()>   |
  <cross-fade()>  |
  <element()>     |
  <gradient>

<mask-source> =
  <url>

<position-one> =
  left                 |
  center               |
  right                |
  top                  |
  bottom               |
  x-start              |
  x-end                |
  y-start              |
  y-end                |
  block-start          |
  block-end            |
  inline-start         |
  inline-end           |
  <length-percentage>

<position-two> =
  [ left | center | right | x-start | x-end ] && [ top | center | bottom | y-start | y-end ]  |
  [ left | center | right | x-start | x-end | <length-percentage> ] [ top | center | bottom | y-start | y-end | <length-percentage> ]  |
  [ block-start | center | block-end ] && [ inline-start | center | inline-end ]  |
  [ start | center | end ]{2}

<position-four> =
  [ [ left | right | x-start | x-end ] <length-percentage> ] && [ [ top | bottom | y-start | y-end ] <length-percentage> ]  |
  [ [ block-start | block-end ] <length-percentage> ] && [ [ inline-start | inline-end ] <length-percentage> ]  |
  [ [ start | end ] <length-percentage> ]{2}

<length-percentage> =
  <length>      |
  <percentage>

<repetition> =
  repeat     |
  space      |
  round      |
  no-repeat

<shape-box> =
  <visual-box>     |
  margin-box       |
  half-border-box

<image()> =
  image( <image-tags>? [ <image-src>? , <color>? ]! )

<image-set()> =
  image-set( <image-set-option># )

<cross-fade()> =
  cross-fade( <cf-image># )

<element()> =
```

```
  element( <id-selector> )

<visual-box> =
  content-box  |
  padding-box  |
  border-box

<image-tags> =
  ltr  |
  rtl

<image-src> =
  <url>     |
  <string>

<image-set-option> =
  [ <image> | <string> ] [ <resolution> || type( <string> ) ]?

<cf-image> =
  [ <image> | <color> ]  &&
  <percentage [0,100]>?

<id-selector> =
  <hash-token>
```

Esta sintaxe reflete o padrão mais recente conforme CSS Backgrounds Module Level 4, CSS Box Model Module Level 4, CSS Images Module Level 4, CSS Images Module Level 5, CSS Masking Module Level 1, CSS Shapes Module Level 1, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5, Selectors Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

```
.target {
  mask: url(#c1);
}

.anothertarget {
  mask: url(resources.svg#c1);
}
```

Veja também

- `clip-path`, `filter`
- Aplicanto efeitos SVG no conteúdo HTML
- SVG

max-width

A propriedade **`max-width`** do CSS estabelece a largura máxima de um elemento. Ele evita que o valor usado da propriedade `width` se torne maior que o valor especificado por `max-width`.

Experimente

```
max-width: 150px;
```

```
max-width: 20em;
```

```
max-width: 75%;
```

```
max-width: 20ch;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    Change the maximum width.
  </div>
</section>
```

```
#example-element {
  display: flex;
  flex-direction: column;
```

```
  background-color: #5b6dcd;
  height: 80%;
  justify-content: center;
  color: #ffffff;
}
```

`max-width` substitui `width`, mas `min-width` substitui `max-width`.

Sintaxe

```
/* <length> valor */
max-width: 3.5em;

/* <porcentagem> valor */
max-width: 75%;

/* Valores de keyword */
max-width: none;
max-width: max-content;
max-width: min-content;
max-width: fit-content(20em);

/* Valores Globais */
max-width: inherit;
max-width: initial;
max-width: revert;
max-width: revert-layer;
max-width: unset;
```

Valores

`<length>`

Define a `max-width` como um valor absoluto.

`<percentage>`

Define a `max-width` como uma porcentagem da largura do bloco que o contém.

`none`

A largura não possui valor máximo. (padrão)

`max-content`

A `max-width` intrínseca preferida.

`min-content`

O mínimo `max-width` intrínseco.

`fit-content(<length-percentage>)`

Use a fórmula `fit-content` com o espaço disponível substituído pelo argumento especificado, i.e. `min(max-content, max(min-content, argumento))`.

Preocupações de Acessibilidade

Certifique-se que os elementos definidos com `max-width` não sejam truncados e/ou não obscureçam outros conteúdos quando a página for ampliada para aumentar o tamanho do texto.

- MDN Understanding WCAG, explicações da Diretriz 1.4
- Compreender o Critério de Sucesso 1.4.4 | W3C Entendendo as WCAG 2.0

Definição formal

Initial value	none
Aplica-se a	all elements but non-replaced inline elements, table rows, and row groups
Inherited	não

Percentages	refer to the width of the containing block
Computed value	the percentage as specified or the absolute length or none
Animation type	a length, percentage or calc();

Sintaxe formal

```
max-width =
  none             |
  <box-size>       |
  <anchor-size()>

<box-size> =
  <length-percentage [0,∞]>                 |
  stretch                                   |
  contain                                   |
  min-content                               |
  max-content                               |
  fit-content                               |
  stretch( <length-percentage [0,∞]> )      |
  fit-content( <length-percentage [0,∞]> )  |
  <calc-size()>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<length-percentage> =
  <length>      |
  <percentage>

<calc-size()> =
  calc-size( <calc-size-basis> , <calc-sum> )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width        |
  height       |
  block        |
  inline       |
  self-block   |
  self-inline

<calc-size-basis> =
  <size-keyword>  |
  <calc-sum>      |
  <calc-size()>   |
  any

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

Esta sintaxe reflete o padrão mais recente conforme CSS Anchor Positioning Module Level 1, CSS Box Sizing Module Level 3, CSS Box Sizing Module Level 4, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

Definindo a largura máxima em pixels

Neste exemplo, o "child" terá 150 pixels de largura ou a largura do "parent", o que for menor:

HTML

```
<div id="parent">
  <div id="child">
    Fusce pulvinar vestibulum eros, sed luctus ex lobortis quis.
  </div>
</div>
```

CSS

```
#parent {
  background: lightblue;
  width: 300px;
}

#child {
  background: gold;
  width: 100%;
  max-width: 150px;
}
```

Resultado

Veja também

- O box model, `box-sizing`
- `width`, `min-width`
- The mapped logical properties: `max-inline-size`, `max-block-size`

min-height

Sumário

A propriedade `min-height` do CSS é usado para definir a altura mínima de um determinado elemento. Ele impede que o valor usado da propriedade `height` se torne menor que o valor especificado para min-height. O valor de `min-height` substitui os dois `max-height` e `height`.

Initial value	`auto`
Aplica-se a	all elements but non-replaced inline elements, table columns, and column groups
Inherited	não
Percentages	The percentage is calculated with respect to the height of the generated box's containing block. If the height of the containing block is not specified explicitly (i.e., it depends on content height), and this element is not absolutely positioned, the percentage value is treated as `0`.
Computed value	the percentage as specified or the absolute length
Animation type	a length, percentage or calc();

Sintaxe

```
sintaxe forma:

  min-height =
    auto             |
    <box-size>       |
    <anchor-size()>

  <box-size> =
    <length-percentage [0,∞]>               |
    stretch                                 |
    contain                                 |
    min-content                             |
```

```
  max-content                              |
  fit-content                              |
  stretch( <length-percentage [0,∞]> )     |
  fit-content( <length-percentage [0,∞]> ) |
  <calc-size()>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<length-percentage> =
  <length>     |
  <percentage>

<calc-size()> =
  calc-size( <calc-size-basis> , <calc-sum> )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width       |
  height      |
  block       |
  inline      |
  self-block  |
  self-inline

<calc-size-basis> =
  <size-keyword>  |
  <calc-sum>      |
  <calc-size()>   |
  any

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

Esta sintaxe reflete o padrão mais recente conforme CSS Anchor Positioning Module Level 1, CSS Box Sizing Module Level 3, CSS Box Sizing Module Level 4, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

```
min-height: 3.5em
min-height: 10%
min-height: max-content
min-height: min-content
min-height: fit-content
min-height: fill-available

min-height: inherit
```

Valores

`<length>`

O valor fixo mínimo da altura. Veja `<length>` para possíveis unidades. Valores negativos fazem da declaração inválida.

`<percentage>`

O valor fixo mínimo da altura expressado como uma `<percentage>` do conteúdo da altura do bloco. Valores negativos fazem da declaração inválida.

`max-content`

The intrinsic preferred height.

`min-content`

The intrinsic minimum height.

`fill-available`

The containing block height minus horizontal margin, border and padding. Some browsers implement an ancient name for this keyword, `available`.

`fit-content`

According CSS3 Box, this is a synonym of `min-content`. CSS3 Sizing defines a more complex algorithm, but no browser implements it, even in an experimental way.

Exemplos

```
table {
  min-height: 75%;
}

form {
  min-height: 0;
}
```

Ver também

- `width`, `height`, `max-height`
- The box model, `min-width`, `box-sizing`, `height`, `max-height`

mix-blend-mode

Resumo

A propriedade **`mix-blend-mode`** descreve como um elemento de conteúdo deve ser mesclado com os elementos abaixo dele no background.

Initial value	`normal`
Aplica-se a	all elements
Inherited	não
Computed value	as specified
Animation type	Not animatable
Creates stacking context	yes

Sintaxe

```
/* Valores para esta propriedade */
mix-blend-mode: normal;
mix-blend-mode: multiply;
mix-blend-mode: screen;
mix-blend-mode: overlay;
mix-blend-mode: darken;
mix-blend-mode: lighten;
mix-blend-mode: color-dodge;
mix-blend-mode: color-burn;
mix-blend-mode: hard-light;
mix-blend-mode: soft-light;
mix-blend-mode: difference;
mix-blend-mode: exclusion;
mix-blend-mode: hue;
mix-blend-mode: saturation;
mix-blend-mode: color;
mix-blend-mode: luminosity;
```

```
/* Valores globais */
mix-blend-mode: initial;
mix-blend-mode: inherit;
mix-blend-mode: unset;
```

Valores

`<blend-mode>`

É uma `<blend-mode>` que descreve qual mesclagem deve ser aplicada. Podem ser descritos vários valores, separados por vírgula.

Sintaxe formal

```
mix-blend-mode =
  <blend-mode>  |
  plus-lighter

<blend-mode> =
  normal       |
  darken       |
  multiply     |
  color-burn   |
  lighten      |
  screen       |
  color-dodge  |
  overlay      |
  soft-light   |
  hard-light   |
  difference   |
  exclusion    |
  hue          |
  saturation   |
  color        |
  luminosity
```

Esta sintaxe reflete o padrão mais recente conforme Compositing and Blending Module Level 2. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

```
<svg>
  <circle cx="40" cy="40" r="40" fill="red" />
  <circle cx="80" cy="40" r="40" fill="lightgreen" />
  <circle cx="60" cy="80" r="40" fill="blue" />
</svg>
```

```
circle {
  mix-blend-mode: screen;
}
```

Resultado

Veja também

- `<blend-mode>`
- `background-blend-mode`

object-fit

Resumo

A propriedade CSS `object-fit` especifica como o conteúdo de um elemento substituído deve ser ajustado à caixa estabelecida pela altura e largura usadas.

Initial value	`fill`
Aplica-se a	replaced elements
Inherited	não
Computed value	as specified

Animation type	discrete

Syntax

```
object-fit =
  fill                              |
  none                              |
  [ contain | cover ] || scale-down
```

Esta sintaxe reflete o padrão mais recente conforme CSS Images Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

```
object-fit: fill
object-fit: contain
object-fit: cover
object-fit: none
object-fit: scale-down
```

Valores

`fill`

O conteúdo substituído é dimensionado para preencher a caixa de conteúdo do elemento: o tamanho concreto do objeto é a largura e a altura usadas do elemento.

`contain`

O conteúdo substituído é dimensionado para manter sua proporção enquanto se encaixa na caixa de conteúdo do elemento: seu tamanho de objeto concreto é resolvido como uma restrição de restrição em relação à largura e altura usadas do elemento.

`cover`

O conteúdo substituído é dimensionado para manter sua proporção e preencher toda a caixa de conteúdo do elemento: seu tamanho de objeto concreto é resolvido como uma restrição de capa contra a largura e a altura usadas do elemento.

`none`

O conteúdo substituído não é redimensionado para caber dentro da caixa de conteúdo do elemento: o tamanho do objeto concreto do objeto é determinado usando o algoritmo de tamanho padrão sem tamanho especificado e um tamanho de objeto padrão igual à largura e altura usadas do elemento substituído.

`scale-down`

O conteúdo é dimensionado se `none` ou `contain` for especificado, o que resultaria em um tamanho menor de objeto concreto.

Exemplos

HTML Content

```
<div>
  <h2>object-fit: fill</h2>
  <img src="mdn_logo_only_color.png" alt="MDN Logo" class="fill" />

  <img src="mdn_logo_only_color.png" alt="MDN Logo" class="fill narrow" />

  <h2>object-fit: contain</h2>
  <img src="mdn_logo_only_color.png" alt="MDN Logo" class="contain" />

  <img src="mdn_logo_only_color.png" alt="MDN Logo" class="contain narrow" />

  <h2>object-fit: cover</h2>
  <img src="mdn_logo_only_color.png" alt="MDN Logo" class="cover" />

  <img src="mdn_logo_only_color.png" alt="MDN Logo" class="cover narrow" />

  <h2>object-fit: none</h2>
  <img src="mdn_logo_only_color.png" alt="MDN Logo" class="none" />

  <img src="mdn_logo_only_color.png" alt="MDN Logo" class="none narrow" />

  <h2>object-fit: scale-down</h2>
  <img src="mdn_logo_only_color.png" alt="MDN Logo" class="scale-down" />
```

```
  <img src="mdn_logo_only_color.png" alt="MDN Logo" class="scale-down narrow" />
</div>
```

CSS Content

```
h2 {
  font-family:
    Courier New,
    monospace;
  font-size: 1em;
  margin: 1em 0 0.3em;
}

div {
  display: flex;
  flex-direction: column;
  flex-wrap: wrap;
  align-items: flex-start;
  height: 940px;
}

img {
  width: 150px;
  height: 100px;
  border: 1px solid #000;
}

.narrow {
  width: 100px;
  height: 150px;
  margin-top: 10px;
}

.fill {
  object-fit: fill;
}

.contain {
  object-fit: contain;
}

.cover {
  object-fit: cover;
}

.none {
  object-fit: none;
}

.scale-down {
  object-fit: scale-down;
}
```

Output

See also

- Other image-related CSS properties: `object-position`, `image-orientation`, `image-rendering`, `image-resolution`.
- An object-fit polyfill for browsers not supporting it.

offset

> **Experimental: Esta é uma tecnologia experimental**
> Verifique a tabela de compatibilidade entre Navegadores cuidadosamente antes de usar essa funcionalidade em produção.

A propriedade CSS **`offset`** é uma propriedade abreviada para animar um elemento ao longo de um caminho definido.

> **Nota:** As primeiras versões da especificação chamam essa propriedade de `motion`.

Initial value	as each of the properties of the shorthand: • `offset-position`: `normal` • `offset-path`: `none` • `offset-distance`: `0` • `offset-anchor`: `auto` • `offset-rotate`: `auto`
Aplica-se a	qualquer elemento transformavel
Inherited	não
Percentages	as each of the properties of the shorthand: • `offset-position`: refer to the size of containing block • `offset-distance`: refer to the total path length • `offset-anchor`: relative to the width and the height of the element's reference box
Computed value	as each of the properties of the shorthand: • `offset-position`: for `<length>` the absolute value, otherwise a percentage • `offset-path`: as specified • `offset-distance`: for `<length>` the absolute value, otherwise a percentage • `offset-anchor`: for `<length>` the absolute value, otherwise a percentage • `offset-rotate`: as specified
Animation type	as each of the properties of the shorthand: • `offset-position`: a position • `offset-path`: by computed value type • `offset-distance`: a length, percentage or calc(); • `offset-anchor`: a position • `offset-rotate`: as <angle>, <basic-shape> or <path()>
Creates stacking context	yes

Sintaxe

```
/* Posição do offset */
offset: auto
offset: 10px 30px;
offset: none;

/* Caminho do offset */
offset: ray(45deg closest-side);
offset: path(M 100 100 L 300 100 L 200 300 z);
offset: url(arc.svg);

/* Caminho do offset com distância e/ou rotação */
offset: url(circle.svg) 100px;
offset: url(circle.svg) 40%;
offset: url(circle.svg) 30deg;
offset: url(circle.svg) 50px 20deg;

/* Incluindo âncora no offset */
offset: ray(45deg closest-side) / 40px 20px;
offset: url(arc.svg) 2cm / 0.5cm 3cm;
offset: url(arc.svg) 30deg / 50px 100px;
```

Sintaxe formal

```
offset =
  [ <'offset-position'>? [ <'offset-path'> [ <'offset-distance'> || <'offset-rotate'> ]? ]? ]! [ / <'offset-anchor'> ]?
```

```
<offset-position> =
  normal      |
  auto        |
  <position>

<offset-path> =
  none                          |
  <offset-path> || <coord-box>

<offset-distance> =
  <length-percentage>

<offset-rotate> =
  [ auto | reverse ]  ||
  <angle>

<offset-anchor> =
  auto        |
  <position>

<position> =
  <position-one>   |
  <position-two>   |
  <position-four>

<offset-path> =
  <ray()>        |
  <url>          |
  <basic-shape>

<coord-box> =
  <paint-box>  |
  view-box

<length-percentage> =
  <length>      |
  <percentage>

<position-one> =
  left                |
  center              |
  right               |
  top                 |
  bottom              |
  x-start             |
  x-end               |
  y-start             |
  y-end               |
  block-start         |
  block-end           |
  inline-start        |
  inline-end          |
  <length-percentage>

<position-two> =
  [ left | center | right | x-start | x-end ] && [ top | center | bottom | y-start | y-end ]  |
  [ left | center | right | x-start | x-end | <length-percentage> ] [ top | center | bottom | y-start | y-end | <length-percentage> ]  |
  [ block-start | center | block-end ] && [ inline-start | center | inline-end ]  |
  [ start | center | end ]{2}

<position-four> =
  [ [ left | right | x-start | x-end ] <length-percentage> ] && [ [ top | bottom | y-start | y-end ] <length-percentage> ]  |
  [ [ block-start | block-end ] <length-percentage> ] && [ [ inline-start | inline-end ] <length-percentage> ]  |
  [ [ start | end ] <length-percentage> ]{2}

<ray()> =
  ray( <angle>          &&
  <ray-size>?           &&
  contain?              &&
  [ at <position> ]? )

<basic-shape> =
  <basic-shape-rect>  |
  <circle()>          |
  <ellipse()>         |
  <polygon()>         |
  <path()>            |
```

```
  <shape()>

<paint-box> =
  <visual-box>  |
  fill-box      |
  stroke-box

<ray-size> =
  <radial-extent>  |
  sides

<basic-shape-rect> =
  <inset()>  |
  <rect()>   |
  <xywh()>

<circle()> =
  circle( <radial-size>? [ at <position> ]? )

<ellipse()> =
  ellipse( <radial-size>? [ at <position> ]? )

<polygon()> =
  polygon( <'fill-rule'>? [ round <length> ]? , [ <length-percentage> <length-percentage> ]# )

<path()> =
  path( <'fill-rule'>? , <string> )

<shape()> =
  shape( <'fill-rule'>? from <position> , <shape-command># )

<visual-box> =
  content-box  |
  padding-box  |
  border-box

<radial-extent> =
  closest-corner   |
  closest-side     |
  farthest-corner  |
  farthest-side

<inset()> =
  inset( <length-percentage>{1,4} [ round <'border-radius'> ]? )

<rect()> =
  rect( <top> , <right> , <bottom> , <left> )

<xywh()> =
  xywh( <length-percentage>{2} <length-percentage [0,∞]>{2} [ round <'border-radius'> ]? )

<radial-size> =
  <radial-extent>               |
  <length [0,∞]>                |
  <length-percentage [0,∞]>{2}

<fill-rule> =
  nonzero  |
  evenodd

<shape-command> =
  <move-command>             |
  <line-command>             |
  close                      |
  <horizontal-line-command>  |
  <vertical-line-command>    |
  <curve-command>            |
  <smooth-command>           |
  <arc-command>

<border-radius> =
  <length-percentage [0,∞]>{1,4} [ / <length-percentage [0,∞]>{1,4} ]?

<move-command> =
  move <command-end-point>

<line-command> =
```

```
  line <command-end-point>

<horizontal-line-command> =
  hline [ to [ <length-percentage> | left | center | right | x-start | x-end ] | by <length-percentage> ]

<vertical-line-command> =
  vline [ to [ <length-percentage> | top | center | bottom | y-start | y-end ] | by <length-percentage> ]

<curve-command> =
  curve [ [ to <position> with <control-point> [ / <control-point> ]? ] | [ by <coordinate-pair> with <relative-control-point> [ / <relative-
control-point> ]? ] ]

<smooth-command> =
  smooth [ [ to <position> [ with <control-point> ]? ] | [ by <coordinate-pair> [ with <relative-control-point> ]? ] ]

<arc-command> =
  arc <command-end-point> [ [ of <length-percentage>{1,2} ] && <arc-sweep>? && <arc-size>? && [ rotate <angle> ]? ]

<command-end-point> =
  to <position>         |
  by <coordinate-pair>

<control-point> =
  <position>               |
  <relative-control-point>

<coordinate-pair> =
  <length-percentage>{2}

<relative-control-point> =
  <coordinate-pair> [ from [ start | end | origin ] ]?

<arc-sweep> =
  cw   |
  ccw

<arc-size> =
  large  |
  small
```

Esta sintaxe reflete o padrão mais recente conforme CSS Borders and Box Decorations Module Level 4, CSS Box Model Module Level 4, CSS Fill and Stroke Module Level 3, CSS Images Module Level 3, CSS Masking Module Level 1, CSS Shapes Module Level 1, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5, Motion Path Module Level 1. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplo

HTML

```
<div id="offsetElement"></div>
```

CSS

```
@keyframes move {
  from {
    offset-distance: 0%;
  }

  to {
    offset-distance: 100%;
  }
}

#offsetElement {
  width: 50px;
  height: 50px;
  background-color: blue;
  offset: path("M 100 100 L 300 100 L 200 300 z") auto;
  animation: move 3s linear infinite;
}
```

Resultado

opacity

Sumário

A propriedade CSS **opacity** especifica a transparência de um elemento, isto é, o grau no qual o background atrás do elemento é sobreposto.

O valor aplica-se ao elemento como um todo, incluindo seu conteúdo, apesar de o valor não ser herdado por elementos filhos. Assim, um elemento e seus elementos filhos têm todos a mesma opacidade relativa ao background do elemento, mesmo se o elemento e seus elementos filhos tiverem opacidades diferentes entre si.

Se você não quer aplicar a opacidade ao elemento filho - use isto:

```
background: rgba(0, 0, 0, 0.4);
```

Usando essa propriedade com um valor diferente de 1, o elemento é colocado em um novo contexto de empilhamento.

Initial value	`1`
Aplica-se a	all elements
Inherited	não
Percentages	map to the range `[0,1]`
Computed value	The same as the specified value after clipping the `<number>` to the range [0.0, 1.0].
Animation type	by computed value type

Sintaxe

```
/* Totalmente opaco */
opacity: 1;
opacity: 1;

/* Translúcido */
opacity: 0.6;

/* Totalmente transparente */
opacity: 0;
opacity: 0;

/* Valores globais */
opacity: inherit;
opacity: initial;
opacity: unset;
```

Valores

`<number>`

> É um `<number>` no intervalo de 0.0 a 1.0, sendo estes incluídos, representando a opacidade do canal, que é o valor de seu canal alfa. Qualquer valor fora do intervalo, apesar de válido, é aproximado ao valor mais próximo dentro do intervalo.

Valor	Significado
`0`	O elemento é totalmente transparente (isto é, invisível).
Qualquer `<number>` entre 0 e 1	O elemento é translúcido (isto é, o background é visível).
`1`	O elemento é totalmente opaco (sólido).

Sintaxe formal

```
opacity =
  <opacity-value>

<opacity-value> =
  <number>      |
  <percentage>
```

Esta sintaxe reflete o padrão mais recente conforme CSS Color Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

Exemplo básico

```
div {
  background-color: yellow;
}
.light {
  opacity: 0.2; /* Mal consegue ver o texto acima do background */
}
.medium {
  opacity: 0.5; /* Vê o texto mais claramente acima do background */
}
.heavy {
  opacity: 0.9; /* Vê o texto muito claramente acima do background */
}
```

```
<div class="light">Você mal vê isso.</div>
<div class="medium">Isso é mais fácil de ver.</div>
<div class="heavy">Isso é muito fácil de ver.</div>
```

Opacidade diferente com <code>:hover</code>

```
img.opacity {
  opacity: 1;
  filter: alpha(opacity=100); /* IE8 e anteriores */
  zoom: 1; /* Ativa "hasLayout" no IE 7 e anteriores */
}

img.opacity:hover {
  opacity: 0.5;
  filter: alpha(opacity=50);
  zoom: 1;
}
```

```
<img
  src="//developer.mozilla.org/media/img/mdn-logo.png"
  alt="MDN logo"
  width="128"
  height="146"
  class="opacity" />
```

Veja também

- MSDN Microsoft's filter:alpha(opacity=xx)

outline

Sumário

A propriedade CSS **`outline`** é uma propriedade abreviada para configurar uma ou mais das propriedades de contorno `outline-style`, `outline-width` e `outline-color` em uma única declaração. Na maioria dos casos o uso abreviado é preferível e mais conveniente.

Contornos se diferenciam de bordas das seguintes maneiras:

- Contornos não ocupam espaços, eles são desenhados acima do conteúdo.
- Contornos podem não ser retangulares. Eles são retangulares no Gecko/Firefox. Mas por exemplo, Opera desenha uma forma não retangular em torno de uma construção como este: **TEXTTEXTTEXT**

Initial value	as each of the properties of the shorthand: • `outline-width`: `medium` • `outline-style`: `none` • `outline-color`: `auto`
Aplica-se a	all elements
Inherited	não

Computed value	as each of the properties of the shorthand: • `outline-width`: the absolute `<length>`, snapped as a line width • `outline-style`: as specified • `outline-color`: For the keyword `auto`, the computed value is `currentcolor`. For the color value, if the value is translucent, the computed value will be the `rgba()` corresponding one. If it isn't, it will be the `rgb()` corresponding one. The `transparent` keyword maps to `rgba(0,0,0,0)`.
Animation type	as each of the properties of the shorthand: • `outline-width`: a length • `outline-style`: by computed value type • `outline-color`: a color

Sintaxe

```
/* largura | estilo | cor */
outline: 1px solid white;

/* Valores globais */
outline: inherit;
outline: initial;
outline: unset;
```

Valores

Um, dois ou três valores, dada em ordem arbitrária:

`<'outline-width'>`

Veja `outline-width`.

`<'outline-style'>`

Veja `outline-style`.

`<'outline-color'>`

Desde Gecko 1.9 (Firefox 3), o valor da propriedade `color` (cor de primeiro plano) é usado. Veja `outline-color`.

Sintaxe completa

```
outline =
  <'outline-width'>  ||
  <'outline-style'>  ||
  <'outline-color'>

<outline-width> =
  <line-width>

<outline-style> =
  auto                  |
  <outline-line-style>

<outline-color> =
  auto                  |
  <'border-top-color'>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick

<outline-line-style> =
  none    |
  auto    |
  dotted  |
  dashed  |
  solid   |
```

```
  double  |
  groove  |
  ridge   |
  inset   |
  outset

<border-top-color> =
  <color>     |
  <image-1D>

<image-1D> =
  <stripes()>

<stripes()> =
  stripes( <color-stripe># )

<color-stripe> =
  <color>                          &&
  [ <length-percentage> | <flex> ]?

<length-percentage> =
  <length>      |
  <percentage>
```

Esta sintaxe reflete o padrão mais recente conforme CSS Basic User Interface Module Level 4, CSS Borders and Box Decorations Module Level 4, CSS Images Module Level 4, CSS Values and Units Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

```
/* duas declarações identicas */

:link:hover {
  outline: 1px solid #000;
}
:link:hover {
  outline: solid black 1px;
}
```

overflow

Resumo

A propriedade `overflow` especifica quando o conteúdo de um elemento de nível de bloco deve ser cortado, exibido com barras de rolagem ou se transborda do elemento.

O uso da propriedade `overflow` com valor diferente de `visible` (seu valor padrão), criará um novo contexto de formatação de bloco. Isto é tecnicamente necessário para evitar que um conteúdo flutuante que entre em contato com o objeto dentro da área de rolamento e quebre as linhas do conteúdo para ajustar a disposição do texto. A quebra das linhas ocorre sempre que a barra de rolagem é utilizada, tornando a experiência de rolagem lenta.

> **Nota:** Ao definir a propredade `scrollTop` para o `elemento` HTML relevante, mesmo que o valor de `overflow` seja `hidden`, o conteúdo ainda pode precisar rolar.

Initial value	`visible`
Aplica-se a	Block-containers, flex containers, and grid containers
Inherited	não
Computed value	as each of the properties of the shorthand: • `overflow-x`: as specified, except with `visible`/`clip` computing to `auto`/`hidden` respectively if one of `overflow-x` or `overflow-y` is neither `visible` nor clip • `overflow-y`: as specified, except with `visible`/`clip` computing to `auto`/`hidden` respectively if one of `overflow-x` or `overflow-y` is neither `visible` nor clip
Animation type	discrete

Sintaxe

```
overflow:  visible | hidden | scroll | auto | inherit
```

Valores

`visible`

Valor padrão. O conteúdo não é cortado e pode ser renderizado para fora da caixa de conteúdo.

`hidden`

O conteúdo é cortado e nenhuma barra de rolagem é exibida.

`scroll`

O conteúdo é acessível através de barras de rolagem que são exibidas mesmo que o conteúdo não precise. Isso evita o problema de barras de rolagem aparecendo e desaparecendo quando o conteúdo é dinâmico. Impressoras podem imprimir o conteúdo vazado.

`auto`

Depende do navegador utilizado. Navegadores desktop como o Firefox geralmente exibem barras de rolagem apenas quando necessário.

Extensões da Mozilla

`-moz-scrollbars-none`

Use `overflow:hidden` no lugar.

`-moz-scrollbars-horizontal`

O uso de `overflow-x` e `overflow-y` é preferido.

`-moz-scrollbars-vertical`

O uso de `overflow-x` e `overflow-y` é preferido.

-moz-hidden-unscrollable

É mantido principalmente para uso interno e por temas. Desabilita o rolamento de elementos raiz de XML e `<html>`, `<body>` mediante o uso das setas do teclado ou scroll do mouse.

Exemplos

```
p {
  width: 12em;
  border: dotted;
  overflow: visible; /* Exibe o conteúdo de forma vazada, fora da caixa de conteúdo */
}
```

`visible` (default) Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantium.

```
p { overflow: hidden; /* Esconde o conteúdo excedente */ }
```

`overflow: hidden` Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantium.

```
p { overflow: scroll; /* Sempre exibe barras de rolagem */ }
```

`overflow: scroll` Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantium.

```
p { overflow: auto; /* Exibe barras de rolagem apenas se necessário */ }
```

`overflow: auto` Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantium.

Veja também

- Propriedades CSS relacionadas: `text-overflow`, `white-space`, `overflow-x`, `overflow-y`, `clip`, `display`

word-wrap

> **Experimental: Esta é uma tecnologia experimental**
> Verifique a tabela de compatibilidade entre Navegadores cuidadosamente antes de usar essa funcionalidade em produção.

Resumo

A propriedade CSS `word-wrap` é utilizada para especificar se o navegador pode ou não quebrar linhas dentro das palavras, afim de prevenir o vazamento quando do contrário uma sequencia de caracteres é muito longa para caber na caixa que o contém.

> **Nota:** Originalmente uma extensão proprietária da Microsoft (não prefixada), a propriedade `word-wrap` foi renomeada para `overflow-wrap` no rascunho atual do texto de especificações do CSS3. Compilações estáveis do Google Chrome e do Opera têm suporte a nova sintaxe.

Initial value	`normal`
Aplica-se a	text elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Sintaxe

```
word-wrap:  normal | break-word
```

Valores

`normal`

Indica que as linhas só podem quebrar em pontos de quebra normais de palavras.

`break-word`

Indica que as palavras normalmente inquebráveis podem ser quebrados em pontos arbitrários se não houver pontos de quebra de outra forma aceitáveis na linha.

Exemplos

```
p {
  width: 13em;
  background: gold;
}
```

FStrPrivFinÄndG (Gesetz zur Änderung des Fernstraßenbauprivatfinanzierungsgesetzes und straßenverkehrsrechtlicher Vorschriften)

```
p {
  width: 13em;
  background: gold;
  word-wrap: break-word;
}
```

FStrPrivFinÄndG (Gesetz zur Änderung des Fernstraßenbauprivatfinanzierungsgesetzes und straßenverkehrsrechtlicher Vorschriften)

Veja também

- Microsoft's original documentation: MSDN Microsoft library word-wrap

padding

A propriedade **`padding`** define uma a distância entre o conteúdo de um elemento e suas bordas. É um atalho que evita definir uma distância para cada lado separadamente (`padding-top`, `padding-right`, `padding-bottom`, `padding-left`).

```
/* Aplica-se à todas as bordas */
padding: 1em;
```

```
/* vertical | horizontal */
padding: 5% 10%;

/* superior | horizontal | inferior */
padding: 1em 2em 2em;

/* superior | direita | inferior | esquerda */
padding: 2px 1em 0 1em;

/* Valores globais */
padding: inherit;
padding: initial;
padding: unset;
```

Initial value	as each of the properties of the shorthand: • `padding-bottom`: `0` • `padding-left`: `0` • `padding-right`: `0` • `padding-top`: `0`
Aplica-se a	all elements, except `table-row-group`, `table-header-group`, `table-footer-group`, `table-row`, `table-column-group` and `table-column`. It also applies to `::first-letter` and `::first-line`.
Inherited	não
Percentages	refer to the width of the containing block
Computed value	as each of the properties of the shorthand: • `padding-bottom`: the percentage as specified or the absolute length • `padding-left`: the percentage as specified or the absolute length • `padding-right`: the percentage as specified or the absolute length • `padding-top`: the percentage as specified or the absolute length
Animation type	a length

Sintaxe

A propriedade `padding` pode ser utilizada usando um, dois, três ou todos os quatro valores. Cada valor é um `<comprimento>` ou uma `<porcentagem>`.

- Quando **um** único é valor utilizado, ele se aplica a todos os **quatro lados**.
- Se **Dois** valores forem utilizados o primeiro se aplica às bordas verticais (**superior e inferior**) e o segundo se aplica às bordas horizontais (**esquerda e direita**).
- Quando **Três** valores são utilizados o primeiro se aplica à borda **superior**, o segundo será utilizado tanto para a borda **esquerda quanto para a borda direita**. O terceiro será aplicado à borda **inferior**.
- Com **Quatro** valores a ordem de aplicação é: **superior, direita, inferior e esquerda** (de acordo com o movimento dos ponteiros do relógio).

Valores

`<comprimento>`

Media do comprimento. Não pode ser negativo. Veja `<length>` para maiores detalhes.

`<porcentagem>`

Valor percentual que é aplicado em relação à **largura** do bloco contido.

Sintaxe formal

```
padding =
  <'padding-top'>{1,4}

<padding-top> =
  <length-percentage [0,∞]>

<length-percentage> =
  <length>      |
  <percentage>
```

Esta sintaxe reflete o padrão mais recente conforme CSS Box Model Module Level 4, CSS Values and Units Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

```
padding: 5%;
/* padding 5% em todos os lados */
```

```
padding: 10px;
/* padding 10px em todos os lados */
```

```
padding: 10px 20px;
/*  10px acima e abaixo  */
/*  20px esquerda e direita  */
```

```
padding: 10px 3% 20px;
/*  10px acima          */
/*  3% esquerda e direita */
/*  20px abaixo       */
```

```
padding: 1em 3px 30px 5px;
/*  acima    1em  padding  */
/*  direita  3px  padding  */
/*  abaixo   30px padding  */
/*  esquerda 5px  padding  */
```

border:outset; padding:5% 1em;

Amostra

HTML

```
<h4>Hello world!</h4>
<h3>O padding é diferente nesta linha.</h3>
```

CSS

```
h4 {
  background-color: green;
  padding: 50px 20px 20px 50px;
}

h3 {
  background-color: blue;
  padding: 400px 50px 50px 400px;
}
```

Veja também

- CSS Box Model

padding-left

Resumo

`A propriedade CSS padding-left` é um elemento que define o espaço de preenchimento obrigatório no lado esquerdo de um elemento. padding area é o espaço entre o conteúdo do elemento e a borda. Um valor negativo não é permitido.

Initial value	`0`
Aplica-se a	all elements, except `table-row-group`, `table-header-group`, `table-footer-group`, `table-row`, `table-column-group` and `table-column`. It also applies to `::first-letter` and `::first-line`.
Inherited	não
Percentages	refer to the width of the containing block
Computed value	the percentage as specified or the absolute length

Animation type	a length

Syntax

Syntax formal:

```
padding-left =
  <length-percentage [0,∞]>

<length-percentage> =
  <length>      |
  <percentage>
```

Esta sintaxe reflete o padrão mais recente conforme CSS Box Model Module Level 4, CSS Values and Units Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

```
padding-left: 0.5em;
padding-left: 0;
padding-left: 2cm;
padding-left: 10%

padding-left: inherit
```

Valores

`<length>`

Especifica uma largura fixa positiva. Consulte `<length>` para obter detalhes.

`<percentage>`

Um percentual em relação a largura do bloco.

Exemplos

```
.content {
  padding-left: 5%;
}
.sidebox {
  padding-left: 10px;
}
```

page-break-before

> **Aviso:** Esta propriedade foi substituída pela propriedade `break-before`.

A propriedade CSS **`page-break-before`** ajusta as "quebras de páginas" *antes* do elemento atual.

Esta propriedade aplica-se para elementos em bloco (block) que geram uma caixa. Não será aplicada em uma `<div>` vazia que não irá gerar uma caixa (box).

```
/* Keyword values */
page-break-before: auto;
page-break-before: always;
page-break-before: avoid;
page-break-before: left;
page-break-before: right;
page-break-before: recto;
page-break-before: verso;

/* Global values */
page-break-before: inherit;
page-break-before: initial;
page-break-before: unset;
```

Initial value	`auto`

Aplica-se a	block-level elements in the normal flow of the root element. User agents may also apply it to other elements like `table-row` elements.
Inherited	não
Computed value	as specified
Animation type	discrete

Page break aliases

A propriedade `page-break-before` agora é uma propriedade legada, substituída por `break-before`.

Por motivos de compatibilidade, `page-break-before` é tratada pelos navegadores como um alias de `break-before`. Isso garante que sites usando `page-break-before` continuem a funcionar. Um subconjunto de valores deve manter aliases como a seguir:

page-break-before	break-before
`auto`	`auto`
`left`	`left`
`right`	`right`
`avoid`	`avoid`
`always`	`page`

Syntax

Values

`auto`

: Initial value. Automatic page breaks (neither forced nor forbidden).

`always`

: Always force page breaks before the element.

`avoid`

: Avoid page breaks before the element.

`left`

: Force page breaks before the element so that the next page is formatted as a left page.

`right`

: Force page breaks before the element so that the next page is formatted as a right page.

`recto`

: If pages progress left-to-right, then this acts like `right`. If pages progress right-to-left, then this acts like `left`.

`verso`

: If pages progress left-to-right, then this acts like `left`. If pages progress right-to-left, then this acts like `right`.

Formal syntax

```
page-break-before =
  auto     |
  always   |
  avoid    |
  left     |
  right    |
  inherit
```

Esta sintaxe reflete o padrão mais recente conforme Cascading Style Sheets Level 2. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Examples

```
/* avoid page break before the div */
div.note {
  page-break-before: avoid;
}
```

See also

- break-before, break-after, break-inside
- page-break-after, page-break-inside
- orphans, widows

pointer-events

Resumo

A propriedade CSS `pointer-events` permite autores controlarem sob qualquer circustancia(se houver) um elemento gráfico particular podendo ser o alvo do evento do mouse. Quando essa propriedade não é especificada, as mesmas características do valor `visiblePainted` é aplicada no conteúdo SVG.``

Além de indicar que este elemento não é o alvo do evento do mouse, o valor `none` instrui o evento do mouse a "passar" o elemento e tudo que está "abaixo" deste elemento.

Initial value	`auto`
Aplica-se a	all elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Sintaxe

```
pointer-events: auto;
pointer-events: none;
pointer-events: visiblePainted;
pointer-events: visibleFill;
pointer-events: visibleStroke;
pointer-events: visible;
pointer-events: painted;
pointer-events: fill;
pointer-events: stroke;
pointer-events: all;

pointer-events: inherit;
```

Valores

`auto`

> O elemento se comporta como se a propriedade `pointer-events` não fosse especificada. Em conteúdo SVG, esse valor e o valor `visiblePainted` tem o mesmo efeito.

`none`

> O elemento nunca é o alvo de eventos do mouse; contudo, o evento do mouse pode direcionar seus elementos descendentes se esses descendentes tiverem algum outro valor configurado no `pointer-events`. Nessas circunstâncias, os eventos do mouse vão acionar eventos de escuta em seus elementos pai conforme apropriado no seu caminho de/para o descendente, durante as fases do evento captura/borbulha.

`visiblePainted`

> Apenas SVG. O elemento pode apenas ser alvo de um evento de mouse quando a propriedade `visibility` é definida como `visible` e quando o cursor do mouse estiver sobre o interior (exemplo: "preenchimento") do elemento e a propriedade `fill` tiver um valor exceto `none`, ou quando o cursor do mouse estiver sobre o perímetro(exemplo: "linha") do elemento e a propriedade `stroke` estiver definida com um valor exceto `none`.

`visibleFill`

Apenas SVG. O elemento apenas pode ser o alvo de um evento mouse quando a propriedade `visibility` é definida como `visible` e quando o cursor do mouse estiver sobre o interior (exemplo: "preenchimento") do elemento. O valor da propriedade `fill` não afeta o processamento dos eventos.

`visibleStroke`

Apenas SVG. O elemento apenas pode ser o alvo de um evento do mouse quando a propriedade `visibility` é definida como `visible` e quando o cursor do mouse estiver sobre o perímetro (exemplo: "linha") do elemento. O valor da propriedade `stroke` não afeta o processamento do evento.

`visible`

Apenas SVG. O elemento pode ser o alvo de um evento de mouse quando a propriedade `visibility` é definida como `visible` e o cursor do mouse estiver sobre ou no interior (exemplo: preenchimento) ou perímetro (exemplo: linha) do elemento. O valor do `fill` e `stroke` não afeta o processamento do evento.

`painted`

Apenas SVG. O elemento apenas pode ser o alvo de um evento do mouse quando o cursor do mouse estiver sobre o interior (exemplo: "preenchimento") do elemento e a propriedade `fill` definida com valor diferente de `none`, ou quando o cursor do mouse estiver no perímetro(exemplo: "linha") do elemento e a propriedade `stroke` definida com valor diferente de `none`. O valor da propriedade `visibility` não afeta o processamento do evento.

`fill`

Apenas SVG. O elemento apenas pode ser o alvo de um evento do mouse quando o ponteiro estiver sobre o interior (exemplo: preenchimento) do elemento. Os valores das propriedades `fill` e `visibility` não afetam o processamento do evento.

`stroke`

Apenas SVG. O elemento apenas pode ser o alvo de um evento do mouse quando o ponteiro estiver sobre o perímetro (exemplo: linha) do elemento. Os valores das propriedades `stroke` e `visibility` não afetam o processamento do evento.

`all`

Apenas SVG. O elemento apenas pode ser alvo do evento do mouse quando o ponteiro estiver sobre o interior (exemplo: preenchimento) ou perímetro (exemplo: linha) do elemento. Os valores das propriedades `fill`, `stroke` e `visibility` não afetam o processamento do evento.

Sintaxe formal

```
pointer-events =
  auto            |
  bounding-box    |
  visiblePainted  |
  visibleFill     |
  visibleStroke   |
  visible         |
  painted         |
  fill            |
  stroke          |
  all             |
  none
```

Esta sintaxe reflete o padrão mais recente conforme CSS Specification. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

Exemplo 1

```
/* Exemplo 1: Fazer todas img não reagir a qualquer evento de mouse tal como arrastar, passar sobre, clique etc */
img {
  pointer-events: none;
}
```

Exemplo 2

Makes the link to `http://example.com` non-reactive.

```
<ul>
  <li><a href="https://developer.mozilla.org">MDN</a></li>
```

```
  <li><a href="http://example.com">example.com</a></li>
</ul>
```

```
a[href="http://example.com"] {
  pointer-events: none;
}
```

Notas

Note que previnir um elemento de ser o alvo de eventos de mouse usando `pointer-events` não necessariamente quer dizer que o evento ouvinte do mouse desde elemento *não pode* ou *não vai se* desencadeado. Se um dos elementos filhos tem `pointer-events` explicitamente definidos para *permitir* que o filho seja o alvo do evento do mouse, então qualquer evento direcionado para este filho vai passar através do pai como o evento se deslocando ao longo da cadeia pai, e os ouvintes do evento gatilho do pai conforme o caso. Claro que qualquer atividade do mouse em um ponto da tela que é coberto pelo pai, mas não pelo filho, não será capturado por qualquer filho ou pai (isso vai "através" do pai e qualquer alvo que está abaixo).

Nós gostaríamos de fornecer um controle de textura mais refinado (do que apenas `auto` e `none`) em HTML para que partes de um elemento fizessem com que os eventos mouse "capturem" , e quando. Para nos ajudar a decidir como `pointer-events` devem ser estendido para HTML, se você tem qualquer coisa particular que você gostaria de fazer com essa propriedade, então por favor adicione na seção Caso de Uso dessa wiki page (não se preocupe sobre ficar organizado).

Esta propriedade pode também ser usada para alcançar melhores taxas de quadros durante a rolagem. De fato, enquanto rola a página, o mouse passa sobre alguns elementos, então efeitos hover se aplicam. Contudo, esses efeitos são muitas vezes despercebidas pelo usuário e principalmente resultam em uma rolagem ruim. Aplicar `pointer-events: none` no `body` desabilita eventos de mouse incluindo hover que resultam em uma performance melhor de rolagem.

Veja também

- The SVG attribute `pointer-events`
- WebKit Specs PointerEventsProperty extended for use in (X)HTML content
- 60fps scrolling using pointer-events: none

position

A propriedade **`position`**, encontrada no CSS, define como um elemento pode ser posicionado (renderizado) no documento (página). Essa propriedade (**`position`**) pode ser acompanhada de outras, tais como, `top`, `right`, `bottom`, and `left`, que determinam como ficará a localização final do objeto, permitindo seu deslocamento, como será apresentado adiante.

Experimente

```
position: static;
```

```
position: relative;
top: 40px;
left: 40px;
```

```
position: absolute;
top: 40px;
left: 40px;
```

```
position: sticky;
top: 20px;
```

```
<section class="default-example" id="default-example">
  <div id="example-element-container">
    <p>
      In this demo you can control the <code>position</code> property for the
      yellow box.
    </p>
    <div class="box"></div>
    <div class="box" id="example-element"></div>
    <div class="box"></div>
    <p class="clear">
      To see the effect of <code>sticky</code> positioning, select the
      <code>position: sticky</code> option and scroll this container.
    </p>
    <p>
```

```
      The element will scroll along with its container, until it is at the top
      of the container (or reaches the offset specified in <code>top</code>),
      and will then stop scrolling, so it stays visible.
    </p>
    <p>
      The rest of this text is only supplied to make sure the container
      overflows, so as to enable you to scroll it and see the effect.
    </p>
    <hr />
    <p>
      Far out in the uncharted backwaters of the unfashionable end of the
      western spiral arm of the Galaxy lies a small unregarded yellow sun.
      Orbiting this at a distance of roughly ninety-two million miles is an
      utterly insignificant little blue green planet whose ape-descended life
      forms are so amazingly primitive that they still think digital watches are
      a pretty neat idea.
    </p>
  </div>
</section>
```

```
section {
  align-items: flex-start;
  overflow: auto;
}

.box {
  background-color: rgba(0, 0, 255, 0.2);
  border: 3px solid blue;
  float: left;
  width: 65px;
  height: 65px;
}

.box + .box {
  margin-left: 10px;
}

.clear {
  clear: both;
  padding-top: 1em;
}

#example-element-container {
  position: relative;
  text-align: left;
}

#example-element {
  background-color: yellow;
  border: 3px solid red;
  z-index: 1;
}
```

Tipos de posicionamentos

- Um **elemento posicionado** é um elemento cujo valor de sua posição pode ser computado (calculado) como `relative`, `absolute`, `fixed`, or `sticky`. (Em outras palavras, são todos esses, com exceção do `static`, sendo estático o valor como padrão do elemento.)
- Um elemento denominado com **posicionamento relativo**, ou **relatively positioned element**, é um elemento cuja função é calculada ao ser definida a `position` como valor `relative`. Com isso, a propriedade `top` e `bottom` determinam o deslocamento ou projeção vertical a posição padrão do elemento que estava definida por `static`; Não sendo outro, o `left` e o `right`, por sua vez, determinam o deslocamento horizontal.
- Um elemento denominado com **posicionamento absoluto**, ou **absolutely positioned element**, é um elemento determinado e calculado ao atribuir ao `position` o valor `absolute` ou `fixed`. Com `top`, `right`, `bottom`, and `left` é possível especificar o deslocamento das laterais (bordas) dos blocos que contêm (containing block) os elementos. (O containing block, ou blocos que envolvem um elemento é definido como um antecessor relativo ao qual o elemento está posicionado, acomplado ou englobado. Se por ventura o elemento tiver definido margens (margin), ele são adicionados ao deslocamento (offset). O elemento estabelecerá um novo contexto de formatação do bloco (BFC) para seus conteúdos.
- Um elemento denominado com **posicionamento adesivo** (!), ou **stickily positioned element**, é um elemento determinado e calculado ao atribuir ao `position` como valor `sticky`. Sendo tratado como um posicionamento relativo até que ultrapasse (atinja) os limites do bloco no qual está contido. (Se assemelha a configurar a propriedade `top` com um valor diferente de automático (auto)) Dentro do seu fluxo principal

(fluxo raiz, ou o contêiner no qual ele desloca-se), No momento que há o deslocamento da página o elemento é tratado como "preso" ou "stuck" até encontrar-se com as paredes opostas do bloco no qual esteja contido.

Maior parte do tempo, um **elemento de posicionamento absoluto** que tem `height` e `width` configurados como `auto` são dimensionados de acordo com seu conteúdo interno. However, non-replaced, absolutely positioned elements can be made to fill the available vertical space by specifying both `top` and `bottom` and leaving `height` unspecified (that is, `auto`). They can likewise be made to fill the available horizontal space by specifying both `left` and `right` and leaving `width` as `auto`.

Exceto para esses casos descritos abaixo (para elementos de posicionamento absoluto que preenchem o espaço disponível):

- Se ambos `top` e `bottom` são definidos (tecnicamento não sendo `auto`), `top` prevalece.
- Se ambos `left` e `right` são definidos, `left` prevalece quando `direction` é `ltr` (Direção da escrita Portuguesa, Japonês na horizontal, etc.) e `right` prevalece quando `direction` for configurado `rtl` (Padrão de escrita em idiomas como Persa, Arabe, Hebraico, etc.).

Sintaxe

A propriedade `position` é definida com apenas uma palavra-chave da lista abaixo.

Valores

`static`

O elemento é posicionado de acordo com o fluxo normal do documento. O `top`, `right`, `bottom`, `left`, e `z-index` propriedades *não têm efeito*.. Este é o valor padrão.

`relative`

O elemento é posicionado de acordo com o fluxo normal do documento e, em seguida, deslocado *em relação a si mesmo* com base nos valores de `top`, `right`, `bottom`, e `left`. O deslocamento não afeta a posição de nenhum outro elemento; assim, o espaço dado para o elemento no layout da página é o mesmo que se a posição fosse `static`.

Este valor cria um novo stacking context (contexto de empilhamento) quando o valor de `z-index` não for `auto`. Seu efeito em `table-*-group`, `table-row`, `table-column`, `table-cell` e os elementos `table-caption` são indefinidos.

`absolute`

O elemento é removido do fluxo normal de documentos e nenhum espaço é criado para o elemento no layout da página. Ele é posicionado em relação ao seu ancestral posicionado mais próximo, se houver; caso contrário, ele é colocado em relação ao inicial containing block. Sua posição final é determinada pelos valores de `top`, `right`, `bottom`, e `left`.

Este valor cria um novo stacking context quando o valor de `z-index` não é `auto`. As margens das caixas posicionadas absolutamente não collapse com outras margens.

`fixed`

O elemento é removido do fluxo normal de documentos e nenhum espaço é criado para o elemento no layout da página. Ele está posicionado em relação ao inicial containing block estabelecido pelo viewport, exceto quando um de seus ancestrais tiver uma propriedade `transform`, `perspective` ou `filter` definida como algo diferente de `none` (see the CSS Transforms Spec), nesse caso, esse ancestral se comporta como o bloco que o contém. (Observe que há inconsistências no navegador com `perspective` e `filter` contribuindo para conter a formação do bloco.) Sua posição final é determinada pelos valores de `top`, `right`, `bottom` e `left`.

Este valor sempre cria um novo stacking context. Em documentos impressos, o elemento é colocado na mesma posição em *todas as páginas*.

`sticky`

O elemento é posicionado de acordo com o fluxo normal do documento e, em seguida, deslocado em relação ao *ancestral de rolagem mais próximo* e containing block (ancestral de nível de bloco mais próximo), incluindo elementos relacionados à tabela, com base nos valores de `top`, `right`, `bottom` e `left`. O deslocamento não afeta a posição de nenhum outro elemento.

Este valor sempre cria um novo stacking context. Observe que um elemento sticky "gruda" em seu ancestral mais próximo que possui um "mecanismo de rolagem" (criado quando `overflow` é `hidden`, `scroll`, `auto` ou `overlay`), mesmo que esse ancestral não seja o ancestral de rolagem mais próximo. Isso inibe efetivamente qualquer comportamento "pegajoso" (see the Github issue on W3C CSSWG).

Sintáxe formal

```
position =
  static      |
  relative    |
  absolute    |
  sticky      |
```

```
  fixed        |
  <running()>

<running()> =
  running( <custom-ident> )
```

Esta sintaxe reflete o padrão mais recente conforme CSS Generated Content for Paged Media Module, CSS Positioned Layout Module Level 3. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

Posicionamento relativo

Relatively positioned elements are offset a given amount from their normal position within the document, but without the offset affecting other elements. In the example below, note how the other elements are placed as if "Two" were taking up the space of its normal location.

HTML

```
<div class="box" id="one">One</div>
<div class="box" id="two">Two</div>
<div class="box" id="three">Three</div>
<div class="box" id="four">Four</div>
```

CSS

```
.box {
  display: inline-block;
  width: 100px;
  height: 100px;
  background: red;
  color: white;
}

#two {
  position: relative;
  top: 20px;
  left: 20px;
  background: blue;
}
```

Posicionamento absoluto

Elements that are relatively positioned remain in the normal flow of the document. In contrast, an element that is absolutely positioned is taken out of the flow; thus, other elements are positioned as if it did not exist. The absolutely positioned element is positioned relative to its *nearest positioned ancestor* (i.e., the nearest ancestor that is not `static`). If a positioned ancestor doesn't exist, it is positioned relative to the ICB (initial containing block — see also the W3C definition), which is the containing block of the document's root element.

A simple example follows:

```
<h1>Absolute positioning</h1>

<p>
  I am a basic block level element. My adjacent block level elements sit on new
  lines below me.
</p>

<p class="positioned">
  By default we span 100% of the width of our parent element, and we are as tall
  as our child content. Our total width and height is our content + padding +
  border width/height.
</p>

<p>
  We are separated by our margins. Because of margin collapsing, we are
  separated by the width of one of our margins, not both.
</p>

<p>
  inline elements <span>like this one</span> and <span>this one</span> sit on
  the same line as one another, and adjacent text nodes, if there is space on
  the same line. Overflowing inline elements
  <span>wrap onto a new line if possible — like this one containing text</span>,
```

```
  or just go on to a new line if not, much like this image will do:
  <img src="long.jpg" />
</p>
```

```
body {
  width: 500px;
  margin: 0 auto;
}

p {
  background: aqua;
  border: 3px solid blue;
  padding: 10px;
  margin: 10px;
}

span {
  background: red;
  border: 1px solid black;
}

.positioned {
  position: absolute;
  background: yellow;
  top: 30px;
  left: 30px;
}
```

Posicionamento fixo

Fixed positioning is similar to absolute positioning, with the exception that the element's containing block is the initial containing block established by the *viewport*, unless any ancestor has `transform`, `perspective`, or `filter` property set to something other than `none` (see CSS Transforms Spec), which then causes that ancestor to take the place of the elements containing block. This can be used to create a "floating" element that stays in the same position regardless of scrolling. In the example below, box "One" is fixed at 80 pixels from the top of the page and 10 pixels from the left. Even after scrolling, it remains in the same place relative to the viewport.

HTML

```
<div class="outer">
  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nam congue tortor
    eget pulvinar lobortis. Vestibulum ante ipsum primis in faucibus orci luctus
    et ultrices posuere cubilia Curae; Nam ac dolor augue. Pellentesque mi mi,
    laoreet et dolor sit amet, ultrices varius risus. Nam vitae iaculis elit.
    Aliquam mollis interdum libero. Sed sodales placerat egestas. Vestibulum ut
    arcu aliquam purus viverra dictum vel sit amet mi. Duis nisl mauris, aliquam
    sit amet luctus eget, dapibus in enim. Sed velit augue, pretium a sem
    aliquam, congue porttitor tortor. Sed tempor nisl a lorem consequat, id
    maximus erat aliquet. Sed sagittis porta libero sed condimentum. Aliquam
    finibus lectus nec ante congue rutrum. Curabitur quam quam, accumsan id
    ultrices ultrices, tempor et tellus.
  </p>
  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nam congue tortor
    eget pulvinar lobortis. Vestibulum ante ipsum primis in faucibus orci luctus
    et ultrices posuere cubilia Curae; Nam ac dolor augue. Pellentesque mi mi,
    laoreet et dolor sit amet, ultrices varius risus. Nam vitae iaculis elit.
    Aliquam mollis interdum libero. Sed sodales placerat egestas. Vestibulum ut
    arcu aliquam purus viverra dictum vel sit amet mi. Duis nisl mauris, aliquam
    sit amet luctus eget, dapibus in enim. Sed velit augue, pretium a sem
    aliquam, congue porttitor tortor. Sed tempor nisl a lorem consequat, id
    maximus erat aliquet. Sed sagittis porta libero sed condimentum. Aliquam
    finibus lectus nec ante congue rutrum. Curabitur quam quam, accumsan id
    ultrices ultrices, tempor et tellus.
  </p>
  <div class="box" id="one">One</div>
</div>
```

CSS

```
.box {
  width: 100px;
  height: 100px;
  background: red;
  color: white;
}

#one {
  position: fixed;
  top: 80px;
  left: 10px;
  background: blue;
}

.outer {
  width: 500px;
  height: 300px;
  overflow: scroll;
  padding-left: 150px;
}
```

Posicionamento Adesivo

Sticky positioning can be thought of as a hybrid of relative and fixed positioning. A stickily positioned element is treated as relatively positioned until it crosses a specified threshold, at which point it is treated as fixed until it reaches the boundary of its parent. For instance...

```
#one {
  position: sticky;
  top: 10px;
}
```

...would position the element with id *one* relatively until the viewport were scrolled such that the element would be less than 10 pixels from the top. Beyond that threshold, the element would be fixed to 10 pixels from the top.

A common use for sticky positioning is for the headings in an alphabetized list. The "B" heading will appear just below the items that begin with "A" until they are scrolled offscreen. Rather than sliding offscreen with the rest of the content, the "B" heading will then remain fixed to the top of the viewport until all the "B" items have scrolled offscreen, at which point it will be covered up by the "C" heading, and so on.

You must specify a threshold with at least one of `top`, `right`, `bottom`, or `left` for sticky positioning to behave as expected. Otherwise, it will be indistinguishable from relative positioning.

HTML

```
<dl>
  <div>
    <dt>A</dt>
    <dd>Andrew W.K.</dd>
    <dd>Apparat</dd>
    <dd>Arcade Fire</dd>
    <dd>At The Drive-In</dd>
    <dd>Aziz Ansari</dd>
  </div>
  <div>
    <dt>C</dt>
    <dd>Chromeo</dd>
    <dd>Common</dd>
    <dd>Converge</dd>
    <dd>Crystal Castles</dd>
    <dd>Cursive</dd>
  </div>
  <div>
    <dt>E</dt>
    <dd>Explosions In The Sky</dd>
  </div>
  <div>
    <dt>T</dt>
    <dd>Ted Leo &amp;amp; The Pharmacists</dd>
    <dd>T-Pain</dd>
    <dd>Thrice</dd>
```

```
    <dd>TV On The Radio</dd>
    <dd>Two Gallants</dd>
  </div>
</dl>
```

CSS

```
* {
  box-sizing: border-box;
}

dl > div {
  background: #fff;
  padding: 24px 0 0 0;
}

dt {
  background: #b8c1c8;
  border-bottom: 1px solid #989ea4;
  border-top: 1px solid #717d85;
  color: #fff;
  font:
    bold 18px/21px Helvetica,
    Arial,
    sans-serif;
  margin: 0;
  padding: 2px 0 0 12px;
  position: -webkit-sticky;
  position: sticky;
  top: -1px;
}

dd {
  font:
    bold 20px/45px Helvetica,
    Arial,
    sans-serif;
  margin: 0;
  padding: 0 0 0 12px;
  white-space: nowrap;
}

dd + dd {
  border-top: 1px solid #ccc;
}
```

Accessibility concerns

Ensure that elements positioned with an `absolute` or `fixed` value do not obscure other content when the page is zoomed to increase text size.

- MDN Understanding WCAG, Guideline 1.4 explanations
- Visual Presentation: Understanding SC 1.4.8 | Understanding WCAG 2.0

Performance & Accessibility

Scrolling elements containing `fixed` or `sticky` content can cause performance and accessibility issues. As a user scrolls, the browser must repaint the sticky or fixed content in a new location. Depending on the content needing to be repainted, the browser performance, and the device's processing speed, the browser may not be able to manage repaints at 60 fps, causing accessibility concerns for people with sensitivities and jank for everyone. One solution is to add `will-change: transform` to the positioned elements to render the element in its own layer, improving repaint speed and therefore improving performance and accessibility.

Initial value	`static`
Aplica-se a	all elements
Inherited	não
Computed value	as specified
Animation type	discrete
Creates stacking context	yes

ruby-align

> **Experimental: Esta é uma tecnologia experimental**
> Verifique a tabela de compatibilidade entre Navegadores cuidadosamente antes de usar essa funcionalidade em produção.

Summary

A propriedade **`ruby-align`** do CSS define a distribuição dos direfentes elementos do ruby sobre a base.

Initial value	`space-around`
Aplica-se a	ruby bases, ruby annotations, ruby base containers, ruby annotation containers
Inherited	yes
Computed value	as specified
Animation type	by computed value type

Sintaxe

```
/* Valores chave */
ruby-align: start;
ruby-align: center;
ruby-align: space-between;
ruby-align: space-around;

/* Valores globais */
ruby-align: inherit;
ruby-align: initial;
ruby-align: unset;
```

Values

`start`

É uma palavra-chave indicando que o ruby será alinhado com o início do texto base.

`center`

É uma palavra-chave indicando que o ruby será alinhado no meio do texto base.

`space-between`

É uma palavra-chave indicando que o espaço extra será distruibuído entre os elementos do ruby.

`space-around`

É uma palavra-chave indicando que o espaço extra será distruibuído entre os elementos do ruby e em torno deles.

Sintaxe formal

```
ruby-align =
  start          |
  center         |
  space-between  |
  space-around
```

Esta sintaxe reflete o padrão mais recente conforme CSS Ruby Annotation Layout Module Level 1. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

Esse HTML irá renderizar diferentemente com cada valor do `ruby-align`:

```
<ruby>
  <rb>Esse é um grande texto para verificar</rb>
  <rp> (</rp><rt>ruby curto</rt><rp>) </rp>
</ruby>
```

Ruby alinhado no ínicio do texto base

```
<ruby>
  <rb>This is a long text to check</rb>
  <rp> (</rp><rt>short ruby</rt><rp>) </rp>
</ruby>
```

```
ruby {
  ruby-align: start;
}
```

Isso dará o seguinte resultado:

Ruby alinhado no centro do texto base

```
<ruby>
  <rb>This is a long text to check</rb>
  <rp> (</rp><rt>short ruby</rt><rp>) </rp>
</ruby>
```

```
ruby {
  ruby-align: center;
}
```

Isso dará o seguinte resultado:

Espaço extra distribuído entre os elementos do ruby

```
<ruby>
  <rb>This is a long text to check</rb>
  <rp> (</rp><rt>short ruby</rt><rp>) </rp>
</ruby>
```

```
ruby {
  ruby-align: space-between;
}
```

Isso dará o seguinte resultado:

Espaço extra distribuído entre os elementos do ruby e em torno deles

```
<ruby>
  <rb>This is a long text to check</rb>
  <rp> (</rp><rt>short ruby</rt><rp>) </rp>
</ruby>
```

```
ruby {
  ruby-align: space-around;
}
```

Isso dará o seguinte resultado:

Veja também

- Elementos Ruby do HTML: `<ruby>`, `<rt>`, `<rp>`, and `<rtc>`.
- Propriedades Ruby do CSS: `ruby-position`, `ruby-merge`.

scroll-behavior

A propriedade css **`scroll-behavior`** especifica o comportamento de rolagem para uma caixa de scrolling, quando a rolagem acontece devido às APIs do CSSOM. Qualquer outro scroll, aqueles que são executados pelo usuário, não são afetados por esta propriedade. Quando esta propriedade é especificada no elemento raiz, ela se aplica à viewport.

User agents podem ignorar essa propriedade.

```
/* Valores das palavras-chave */
scroll-behavior: auto;
scroll-behavior: smooth;
```

```
/* Valores globais */
scroll-behavior: inherit;
scroll-behavior: initial;
scroll-behavior: unset;
```

Note que qualquer outro scroll, tem essa tal performace para o usuario, essa propriedade não é afetada. Quando esta propriedade é especificada na raiz do elemento então aplicamos na instancia do viewport.

Os usuarios que agem com está propriedade pode ignora-lá.

Initial value	`auto`
Aplica-se a	scrolling boxes
Inherited	não
Computed value	as specified
Animation type	Not animatable

Sintaxe

O `scroll-behavior` é uma propriedade específica, com um valor de uma palavra-chave listada abaixo.

Valores

`auto`

A caixa de rolagem rola instantaneamente.

`smooth`

A caixa de scroll rola de forma suave usando uma função de temporização definida pelo usuário em um período de tempo. Os agentes do usuário devem seguir as convenções da plataforma, se houver.

Sintaxe formal

```
scroll-behavior =
  auto    |
  smooth
```

Esta sintaxe reflete o padrão mais recente conforme CSS Overflow Module Level 3. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplo

HTML

```
<nav>
  <a href="#page-1">1</a>
  <a href="#page-2">2</a>
  <a href="#page-3">3</a>
</nav>
<scroll-container>
  <scroll-page id="page-1">1</scroll-page>
  <scroll-page id="page-2">2</scroll-page>
  <scroll-page id="page-3">3</scroll-page>
</scroll-container>
```

CSS

```
a {
  display: inline-block;
  width: 50px;
  text-decoration: none;
}
nav,
scroll-container {
  display: block;
  margin: 0 auto;
  text-align: center;
}
```

```
nav {
  width: 339px;
  padding: 5px;
  border: 1px solid black;
}
scroll-container {
  display: block;
  width: 350px;
  height: 200px;
  overflow-y: scroll;
  scroll-behavior: smooth;
}
scroll-page {
  display: flex;
  align-items: center;
  justify-content: center;
  height: 100%;
  font-size: 5em;
}
```

Saída

scrollbar-color

> **Experimental: Esta é uma tecnologia experimental**
> Verifique a tabela de compatibilidade entre Navegadores cuidadosamente antes de usar essa funcionalidade em produção.

A propriedade CSS **`scrollbar-color`** define as cores da barra de rolagem do navegador.

Track refere-se ao fundo da barra de rolagem, que normalmente é fixa, independente da posição da página.

Thumb refere-se a parte móvel da barra de rolagem, um botão auxiliar que flutua acima da **Track**.

Initial value	`auto`
Aplica-se a	scrolling boxes
Inherited	yes
Computed value	as specified
Animation type	by computed value type

Sintaxe

```
/* Palavras reservadas */
scrollbar-color: auto;
scrollbar-color: dark;
scrollbar-color: light;

/* Nomes de <color> */
scrollbar-color: rebeccapurple green; /* Duas cores válidas.
O primeiro valor aplica-se ao Thumb, o segundo ao Track */

/* Valores globais */
scrollbar-color: inherit;
scrollbar-color: initial;
scrollbar-color: unset;
```

Valores

`<scrollbar-color>`

Define a cor da scrollbar.

`auto`	**Valor padrão de renderização do background da scrollbar (Track), caso não sejam definidos outros valores.**
`dark`	Mostra uma scrollbar mais escura, podendo ser um estilo padrão definido pelo navegador, ou personalizado com cores escuras.

`auto`	**Valor padrão de renderização do background da scrollbar (Track), caso não sejam definidos outros valores.**
`light`	Mostra uma scrollbar mais clara, podendo ser um estilo padrão definido pelo navegador, ou personalizado com cores escuras.
`<color>` `<color>`	A primeira cor é aplicada à **thumb**, a segunda cor ao **track**

> **Nota:** User Agents must apply any `scrollbar-color` value set on the root element to the viewport.

Sintaxe formal

```
scrollbar-color =
  auto        |
  <color>{2}
```

Esta sintaxe reflete o padrão mais recente conforme CSS Scrollbars Styling Module Level 1. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplo

CSS

```
.scroller {
  width: 300px;
  height: 100px;
  overflow-y: scroll;
  scrollbar-color: rebeccapurple green;
}
```

HTML

```
<div class="scroller">
  Veggies es bonus vobis, proinde vos postulo essum magis kohlrabi welsh onion
  daikon amaranth tatsoi tomatillo melon azuki bean garlic. Gumbo beet greens
  corn soko endive gumbo gourd. Parsley shallot courgette tatsoi pea sprouts
  fava bean collard greens dandelion okra wakame tomato. Dandelion cucumber
  earthnut pea peanut soko zucchini.
</div>
```

Result

Cuidados de acessibilidade

When using `scrollbar-color` property with specific color values, authors should ensure the specified colors have enough contrast between them. For keyword values, UAs should ensure the colors they use have enough contrast. See Techniques for WCAG 2.0: G183: Using a contrast ratio of 3:1.

text-align

Sumário

A propriedade CSS **`text-align`** descreve como conteúdo inline, como texto, é alinhado no elemento pai em bloco. `text-align` não controla o alinhamento de elementos em bloco, apenas o seu conteúdo inline.

Initial value	`start`, or a nameless value that acts as `left` if `direction` is `ltr`, `right` if `direction` is `rtl` if `start` is not supported by the browser.
Aplica-se a	block containers
Inherited	yes
Computed value	as specified, except for the `match-parent` value which is calculated against its parent's `direction` value and results in a computed value of either `left` or `right`
Animation type	discrete

Sintaxe

```
/* Keyword values */
text-align: left;
text-align: right;
text-align: center;
text-align: justify;
text-align: justify-all;
text-align: start;
text-align: end;
text-align: match-parent;

/* Block alignment values (Non-standard syntax) */
text-align: -moz-center;
text-align: -webkit-center;

/* Global values */
text-align: inherit;
text-align: initial;
text-align: unset;
```

Valores

`start`

Igual `left` se a direção é esquerda-para-direita e `right` se a direção é direita-para-esquerda.

`end`

Igual `right` se a direção é esquerda-para-direita e `left` se a direção é direita-para-esquerda.

`left`

O conteúdo é alinhado na borda esquerda do box.

`right`

O conteúdo é alinhado na borda direita do box.

`center`

Os conteúdos inline são centralizados entre a linha da caixa.

`justify`

Os conteúdos inline são justificados. Text should be spaced to line up its left and right edges to the left and right edges of the line box, except for the last line.

`justify-all`

Semelhante a `justify`, porém força a última linha a ser justificada.

`match-parent`

Similar to `inherit`, but the values `start` and `end` are calculated according the parent's `direction` and are replaced by the adequate `left` or `right` value.

Sintaxe Formal

```
text-align =
  start         |
  end           |
  left          |
  right         |
  center        |
  <string>      |
  justify       |
  match-parent  |
  justify-all
```

Esta sintaxe reflete o padrão mais recente conforme CSS Text Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

Alinhamento Esquerdo

HTML

```
<p class="example">
  Integer elementum massa at nulla placerat varius. Suspendisse in libero risus,
  in interdum massa. Vestibulum ac leo vitae metus faucibus gravida ac in neque.
  Nullam est eros, suscipit sed dictum quis, accumsan a ligula.
</p>
```

CSS

```
.example {
  text-align: left;
  border: solid;
}
```

Resultado

Texto centralizado

HTML

```
<p class="example">
  Integer elementum massa at nulla placerat varius. Suspendisse in libero risus,
  in interdum massa. Vestibulum ac leo vitae metus faucibus gravida ac in neque.
  Nullam est eros, suscipit sed dictum quis, accumsan a ligula.
</p>
```

CSS

```
.example {
  text-align: center;
  border: solid;
}
```

Resultado

Justificado

HTML

```
<p class="example">
  Integer elementum massa at nulla placerat varius. Suspendisse in libero risus,
  in interdum massa. Vestibulum ac leo vitae metus faucibus gravida ac in neque.
  Nullam est eros, suscipit sed dictum quis, accumsan a ligula.
</p>
```

CSS

```
.example {
  text-align: justify;
  border: solid;
}
```

Resultado

Notas

O modo compátivel-padrão de centralizar um bloco em si sem centralizar seu conteúdo inline é setando a `margin` left e right para `auto`, e.g.:

```
.something {
  margin: auto;
}
```

```
.something {
  margin: 0 auto;
}
```

```
.something {
  margin-left: auto;
  margin-right: auto;
}
```

Veja também

- `margin:auto`, `margin-left:auto`, `vertical-align`

text-decoration

Resumo

A propriedade **`text-decoration`** do CSS é usada para definir a formatação de `underline`, `overline`, `line-through` ou `blink`. As decorações underline e overline são posicionadas abaixo e acima do texto (respectivamente), e line-through cortando-o.

A propriedade text-decoration desenha em elementos descendentes. Isso significa que não é possivel desabilitar em um descendente uma decoração que foi especificada em um de seus antecessores. Por exemplo, no markup `<p>Esse texto tem <em>alguns elementos enfatizados</em> nele.</p>`, a regra de estilo `p { text-decoration: underline; }` faria com que todo o parágrafo fosse sublinhado. A regra `em { text-decoration: none; }` não causaria qualquer mudanca; o parágrafo inteiro ainda estaria sublinhado. Entretanto, a regra `em { text-decoration: overline; }` iria decorar o trecho "alguns elementos enfatizados".

> **Nota:** CSS 3 transformou a propriedade Text-Decoration para forma reduzida, usando as seguintes propriedades: `text-decoration-color`, `text-decoration-line`, e `text-decoration-style`. Como em outras propriedades reduzidas, isso significa que ele reseta o valor de cada uma para padrão se não for explicitamente especificada na forma reduzida.

Initial value	as each of the properties of the shorthand: • `text-decoration-color`: `currentcolor` • `text-decoration-style`: `solid` • `text-decoration-line`: `none`
Aplica-se a	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	não
Computed value	as each of the properties of the shorthand: • `text-decoration-line`: as specified • `text-decoration-style`: as specified • `text-decoration-color`: computed color • `text-decoration-thickness`: as specified
Animation type	as each of the properties of the shorthand: • `text-decoration-color`: a color • `text-decoration-style`: discrete • `text-decoration-line`: discrete • `text-decoration-thickness`: by computed value type

Sintaxe

```
/* Valores Chave */
text-decoration: none; /* Sem decoração */
text-decoration: underline red; /* Sublinhado vermelho */
text-decoration: underline wavy red; /* Sublinhado ondulado vermelho */

/* Valores Globais */
text-decoration: inherit;
text-decoration: initial;
text-decoration: unset;
```

Valores

A propriedade `text-decoration` é escrita de forma reduzida e pode usar os valores de cada uma das propriedades: `text-decoration-line`, `text-decoration-color`, e `text-decoration-style`

Sintaxe Formal

```
text-decoration =
  <'text-decoration-line'>       ||
  <'text-decoration-thickness'>  ||
  <'text-decoration-style'>      ||
  <'text-decoration-color'>

<text-decoration-line> =
  none                                          |
  [ underline || overline || line-through || blink ]  |
  spelling-error                                |
  grammar-error

<text-decoration-thickness> =
  auto                 |
  from-font            |
  <length-percentage>  |
  <line-width>

<text-decoration-style> =
  solid   |
  double  |
  dotted  |
  dashed  |
  wavy

<text-decoration-color> =
  <color>

<length-percentage> =
  <length>      |
  <percentage>

<line-width> =
  <length [0,∞]>  |
  hairline        |
  thin            |
  medium          |
  thick
```

Esta sintaxe reflete o padrão mais recente conforme CSS Borders and Box Decorations Module Level 4, CSS Text Decoration Module Level 4, CSS Values and Units Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

```
h1.under {
  text-decoration: underline;
}
h1.over {
  text-decoration: overline;
}
p.line {
  text-decoration: line-through;
}
p.blink {
  text-decoration: blink;
}
a.none {
  text-decoration: none;
}
p.underover {
  text-decoration: underline overline;
}
```

```
<h1 class="under">Underlined Header</h1>
<p class="line">If you want to strike out text, use line-through.</p>
<p class="blink">
```

```
  This text might blink for you - if you use Explorer, it will not.
</p>
<h1 class="over">and now an overlined header.</h1>
<p>
  This
  <a class="none" href="textdecoration.html">link will not be underlined</a>, as
  most links are by default. Be careful with removing the text decoration on
  links since most users depend on the underline to signify a hyperlink.
</p>
<p class="underover">This text has both underline and overline.</p>
```

Veja também

- O atributo `list-style` controla a aparência dos itens em listas `<ol>` e `<ul>` no HTML.

text-decoration-style

Resumo

A propriedade CSS **`text-decoration-style`** CSS define o estilo das linhas especificadas por `text-decoration-line`. O modelo aplica-se a todas as linhas, não há nenhuma forma de definir estilos diferentes para cada uma das linhas utilizando `text-decoration-line`.

Se a decoração especificada tem um significado semântico específico, como uma linha de line-through o que significa que um texto tenha sido excluído, autores são encorajados a denotar este significado usando uma tag HTML, como `<del>` ou `<s>`. Como os navegadores podem ser deficientes em alguns casos, o significado semântico não vai desaparecer em tal situação.

Initial value	`solid`
Aplica-se a	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	não
Computed value	as specified
Animation type	discrete

Syntax

```
/* Keyword values */
text-decoration-style: solid;
text-decoration-style: double;
text-decoration-style: dotted;
text-decoration-style: dashed;
text-decoration-style: wavy;

/* Global values */
text-decoration-style: inherit;
text-decoration-style: initial;
text-decoration-style: unset;
```

Valores

`solid double dotted dashed wavy`

É uma das seguintes palavras-chave:

Keyword	Description	Comment
`solid`	Draws a single line	
`double`	Draws a double line	
`dotted`	Draws a dotted line	
`dashed`	Draws a dashed line	
`wavy`	Draws a wavy line	
`-moz-none`	Do not draw a line	**Do not use :** use `text-decoration-line: none` instead

`inherit`

É uma palavra-chave indicando a reutilizar o valor calculado sobre o elemento pai.

Syntax Formal

```
text-decoration-style =
  solid   |
  double  |
  dotted  |
  dashed  |
  wavy
```

Esta sintaxe reflete o padrão mais recente conforme CSS Text Decoration Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplo

```
.example {
  -moz-text-decoration-line: underline;
  -moz-text-decoration-style: wavy;
  -moz-text-decoration-color: red;
  -webkit-text-decoration-line: underline;
  -webkit-text-decoration-style: wavy;
  -webkit-text-decoration-color: red;
}
```

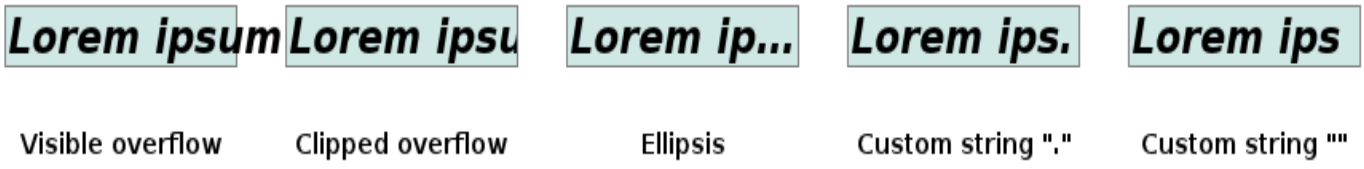
```
.example {
  text-decoration-line: underline;
  text-decoration-style: wavy;
  text-decoration-color: red;
}
```

```
<p class="example">This is how it looks.</p>
```

text-overflow

Sumário

A propriedade `text-overflow` do CSS determina como o conteúdo que ultrapassou a sua div e que não é mostrado ao usuário deve ser exibido. Ele pode ser cortado, mostrar reticências ou até mesmo exibir qualquer string definida pelo autor.



O corte acontece na borda da caixa; para cortar no limite de caracteres de uma string personalizada, pode-se usar (`''`).

Essa propriedade afeta apenas o conteúdo que está ultrapassando o bloco ou a div no sentido da linha horizontal (não corta o texto na parte de baixo de um bloco ou div, por exemplo). O texto pode ultrapassar ou transbordar quando utilizado '`white-space:nowrap`' ou quando uma palavra for muito grande.

Essa propriedade do CSS não força um estouro para que ele ocorra. Para que isso aconteça, é necessário que o autor utilize algumas propriedades adicionais no elemento, como setar a propriedade overflow para hidden.

Initial value	`clip`
Aplica-se a	block container elements
Inherited	não
Computed value	as specified
Animation type	discrete

Syntaxe

```
Formal syntax:
```

```
text-overflow =
  [ clip | ellipsis | <string> | fade | <fade()> ]{1,2}
```

```
<fade()> =
  fade( [ <length-percentage> ] )

<length-percentage> =
  <length>      |
  <percentage>
```

Esta sintaxe reflete o padrão mais recente conforme CSS Overflow Module Level 4, CSS Values and Units Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

```
/* One-value syntax: the value describe the overflow performed at the end of the line (right end if ltr, left end if rtl) */
text-overflow: clip
text-overflow: ellipsis
text-overflow: "…"

/* Two-value syntax: the first value describes the overflow at the left end of the line,
                     the second at the right end. Directionality has no influence */

text-overflow: clip ellipsis
text-overflow: "…" "…"

text-overflow: inherit
```

Values

`clip`

> This keyword value indicates to truncate the text at the limit of the content area, therefore the truncation can happen in the middle of a character. To truncate at the transition between two characters, the empty string value (`''`) must be used. The value `clip` is the default for this property.

`ellipsis`

> This keyword value indicates to display ellipses (`'…'`, `U+2026 Horizontal Ellipsis`) to represent clipped text. The ellipsis is displayed inside the content area, shortening more the size of the displayed text. If there is not enough place to display ellipsis, they are clipped.

`<string>`

> The `<string>` to be used to represent clipped text. The string is displayed inside the content area, shortening more the size of the displayed text. If there is not enough place to display the string itself, it is clipped.

Examples

```
p {
  white-space: nowrap;
  width: 100%;
  overflow: hidden; /* "overflow" value must be different from "visible" */

  text-overflow: ellipsis;
}
```

CSS value	direction: ltr		direction: rtl	
	Expected Result	Live result	Expected Result	Live result
visible overflow	1234567890	1234567890	0987654321	1234567890
`text-overflow: clip`	123456	123456	654321	567890
`text-overflow: ''`	12345	123456	54321	567890
`text-overflow: ellipsis`	1234…	12345…	…4321	…67890
`text-overflow: '.'`	1234.	123456	.4321	567890
`text-overflow: clip clip`	123456	123456	654321	567890
`text-overflow: clip ellipsis`	1234…	123456	6543…	567890
`text-overflow: clip '.'`	1234.	123456	6543.	567890
`text-overflow: ellipsis clip`	…3456	123456	…4321	567890

CSS value	direction: ltr		direction: rtl	
	Expected Result	Live result	Expected Result	Live result
text-overflow: ellipsis ellipsis	…34…	123456	…43…	567890
text-overflow: ellipsis '.'	…34.	123456	…43.	567890
text-overflow: ',' clip	,3456	123456	,4321	567890
text-overflow: ',' ellipsis	,34…	123456	,43…	567890
text-overflow: ',' '.'	,34.	123456	,53.	567890

Using the abbr tag of HTML5 along with text-overflow

We have the example of the use of abbr tag inside a div with width 250px;

```
div {
  width: 250px;
  display: inline-block;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
  direction: ltr;
}
```

```
<div>
  <abbr title="https://developer.mozilla.org/pt-BR/docs/Web/CSS/text-overflow"
    >https://developer.mozilla.org/pt-BR/docs/Web/CSS/text-overflow</abbr
  >
</div>
```

The full path of the URL will be displayed over the mouse, and the screen will be printed as follows: `https://developer.mozilla.org/pt-BR/...`

See also

- CSS Reference index
- Related CSS properties: `overflow`, `white-space`

text-rendering

Sumário

A propriedade CSS `text-rendering` provê informações para o mecanismo de renderização sobre o que otimizar no momento em que ele renderiza texto. O navegador faz escolhas entre velocidade, legibilidade e precisão geométrica. A propriedade `text-rendering` é uma propriedade SVG que não é definida em nenhum padrão CSS. Porém, navegadores Gecko e WebKit permitem o uso da propriedade em conteúdos HTML e XML nas plataformas Windows, Mac OS X e Linux.

Um efeito bastante visível é o `optimizeLegibility`, que habilita ligaduras (ff, fi, fl etc.) para algumas fontes com o texto menores que 20px como, por exemplo, fontes da Microsoft (*Calibri, Candara, Constantia* e *Corbel* ou a família de fontes *DejaVu*).

Initial value	`auto`
Aplica-se a	text elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Syntax

```
text-rendering:  auto | optimizeSpeed | optimizeLegibility | geometricPrecision | inherit
```

Valores

`auto`

> O navegador faz sugestões de quando otimizar velocidade, legibilidade e precisão geométrica enquando formata o texto. Para diferenças em como esse valor é interpretado pelo navegador, veja a tabela de compatibilidades.

`optimizeSpeed`

O navegador prioriza a velocidade de renderização sobre a legibilidade e precisão geométrica no momento de rederização do texto. Essa opção desabilita o kerning1 e ligatures2.

`optimizeLegibility`

O navegador prioriza legibilidade sobre a velocidade de renderização e precisão geométrica. Essa propriedade habilita o kerning e ligatures opcionais.

`geometricPrecision`

O browser prioriza a precisão geométrica sobre a velocidade de renderização e legibilidade. Alguns aspectos de fontes - tais como kerning - não se escala linearmente e, dessa forma, `geometricPrecision` pode tornar essas fontes visualmente boas.

Em SVG, quando o texto é escalado para cima ou para baixo, os navegadores calculam o tamanho final do texto (que é o tamanho de fonte específicado e a escala aplicada) e requer uma fonte de tamanho computado à partir das fontes do sistema. Mas se você requer um tamanho de fonte de, digamos, 9 com a escala de 140%, o resultado do tamanho da fonte de 12.6 não explicitamente existe no sistema, então o browser ira arredondar o tamanho da fonte para 12.

Mas a propriedade `geometricPrecision` - quando totalmente surpotada pelo mecanismo de renderização - deixa você escalar seu texto fluidamente. Para fatores de larga escala, você pode ver uma renderização de texto não tão bonita, mas com o tamanho esperado - nem arredondado para cima ou para baixo para o mais próximo tamanho suportado pelo sistema Windows ou Linux.

Navegadores WebKit precisamente aplicam o valor específico, mas os navegadores Gecko tratam o esse valor como `optimizeLegibility`.

Exemplos

```
/* make sure all fonts in the HTML document display in all its glory,
   but avoid inadequate ligatures in class foo elements */

body {
  text-rendering: optimizeLegibility;
}
.foo {
  text-rendering: optimizeSpeed;
}
```

Live Example

CSS code	Kerning	Ligatures
font: 19.9px 'DejaVu Serif',Constantia;	LYoWAT	ff fi fl ffl
font: 20px 'DejaVu Serif',Constantia;	LYoWAT	ff fi fl ffl
font: 3em 'DejaVu Serif',Constantia; text-rendering: optimizeSpeed;	LYoWAT	ff fi fl ffl
font: 3em 'Dejavu Serif',Constantia; text-rendering: optimizeLegibility;	LYoWAT	ff fi fl ffl

Gecko Notes

O valor padrão de 20px para `auto` pode ser alterado na propriedade `browser.display.auto_quality_min_font_size` do navegador.

A opção optimizeSpeed não tem efeito na versão 2.0 do Gecko, devido ao código padrão de renderização de texto que já é muito rápido e não existe até o momento um código mais rápido para esse trabalho. Veja detalhes em Firefox bug 595688.

text-shadow

Sumário

A propriedade `text-shadow` acrescenta sombras ao texto. Ela aceita uma lista de sombras separadas por vírgula que serão aplicados ao texto e ao `text-decorations` do elemento.

Cada sombra é especificada como um deslocamento do texto, juntamente com valores opcionais de cor e raio de desfoque.

Multiplas sombras são aplicadas de frente-para-trás, com a primeira sombra especificada no topo.

Esta propriedade se aplica a ambos `::first-line` e `::first-letter` pseudo-elements.

Initial value	none

Aplica-se a	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	a color plus three absolute lengths
Animation type	a shadow list

Sintaxe

```
/* deslocamento-x | deslocamento-y | raio-de-desfoque | cor */
text-shadow: 1px 1px 2px black;

/* cor | deslocamento-x | deslocamento-y | raio-de-desfoque */
text-shadow: #ccc 1px 0 10px;

/* deslocamento-x | deslogamento-y | cor */
text-shadow: 5px 5px #558abb;

/* cor | deslocamento-x | deslocamento-y */
text-shadow: white 2px 5px;

/* deslocamento-x | deslocamento-y
/* Usa o padrão para cor e raio-de-desfoque */
text-shadow: 5px 10px;

/* Valores globais */
text-shadow: inherit;
text-shadow: initial;
text-shadow: unset;
```

Valores

<cor>

Opcional. Pode ser especificado tanto antes quanto depois dos valores de deslocamento. Se a cor não é especificada, uma cor UA-chosen será usada.

> **Nota:** Se voce quer garantir a consistência entre os navegadores, especifique explicitamente uma cor.

<offset-x> <offset-y>

Obrigatório. These `<length>` values specify the shadow's offset from the text. `<offset-x>` specifies the horizontal distance; a negative value places the shadow to the left of the text. `<offset-y>` specifies the vertical distance; a negative value places the shadow above the text. If both values are `0`, then the shadow is placed behind the text (and may generate a blur effect when `<blur-radius>` is set). To find out what units you can use, see `<length>`.

<blur-radius>

Opcional. This is a `<length>` value. If not specified, it defaults to `0`. The higher this value, the bigger the blur; the shadow becomes wider and lighter.

Formal syntax

```
text-shadow =
  none       |
  <shadow>#

<shadow> =
  <color>?                                  &&
  [ <length>{2} [ <length [0,∞]> <length>? ]? ]  &&
  inset?
```

Esta sintaxe reflete o padrão mais recente conforme CSS Backgrounds and Borders Module Level 3, CSS Text Decoration Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

```
.red-text-shadow {
  text-shadow: red 0 -2px;
```

```
}
```

```
<p class="red-text-shadow">
  Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium
  doloremque laudantium, totam rem aperiam, eaque ipsa quae ab illo inventore
  veritatis et quasi architecto beatae vitae dicta sunt explicabo.
</p>
```

```
.white-with-blue-shadow {
  text-shadow:
    1px 1px 2px black,
    0 0 1em blue,
    0 0 0.2em blue;
  color: white;
  font:
    1.5em Georgia,
    "Bitstream Charter",
    "URW Bookman L",
    "Century Schoolbook L",
    serif;
}
```

```
<p class="white-with-blue-shadow">
  Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium
  doloremque laudantium, totam rem aperiam, eaque ipsa quae ab illo inventore
  veritatis et quasi architecto beatae vitae dicta sunt explicabo.
</p>
```

```
.gold-on-gold {
  text-shadow:
    rgba(0, 0, 0, 0.1) -1px 0,
    rgba(0, 0, 0, 0.1) 0 -1px,
    rgba(255, 255, 255, 0.1) 1px 0,
    rgba(255, 255, 255, 0.1) 0 1px,
    rgba(0, 0, 0, 0.1) -1px -1px,
    rgba(255, 255, 255, 0.1) 1px 1px;
  color: gold;
  background: gold;
}
```

```
<p class="gold-on-gold">
  Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium
  doloremque laudantium, totam rem aperiam, eaque ipsa quae ab illo inventore
  veritatis et quasi architecto beatae vitae dicta sunt explicabo.
</p>
```

Veja também

- `box-shadow`

text-transform

A propriedade de CSS **`text-transform`** especifica como capitalizar um texto de um elemento. Pode ser usado para que o texto apareça com todas as letras maiúsculas ou todas minúsculas, ou com cada palavra em maiúscula.

```
/* Keyword values */
text-transform: capitalize;
text-transform: uppercase;
text-transform: lowercase;
text-transform: none;
text-transform: full-width;

/* Global values */
text-transform: inherit;
text-transform: initial;
text-transform: unset;
```

The `text-transform` property takes into account language-specific case mapping rules, like:

- in Turkic languages, like Turkish (`tr`), Azerbaijani (`az`), Crimean Tatar (`crh`), Volga Tatar (`tt`), and Bashkir (`ba`), there are two kinds of i, with and without the dot, and two case pairings: `i`/`İ` and `ı`/`I`.
- In German (`de`), the ß becomes SS in uppercase.
- In Dutch (`nl`), the `ij` digraph becomes `IJ`, even with `text-transform: capitalize`, which only put the first letter of a word in uppercase.
- In Greek (`el`), vowels lose their accent when the whole word is in uppercase (ά/Α), except for the disjunctive eta (ή/Η). Also, diphthongs with an accent on the first vowel lose the accent and gain a diaeresis on the second vowel (άι/Αΐ).
- In Greek (`el`), the lowercase sigma character has two forms: σ and ς. ς is used only when sigma terminates a word. When applying `text-transform: lowercase` to an uppercase sigma (Σ), the browser needs to choose the right lowercase form based on context.
- in Irish (`ga`), certain prefixed letters remain in lowercase when the base initial is capitalised, so for example `text-transform: uppercase` will change `ar aon tslí` to `AR AON tSLÍ` and not, as one might expect, `AR AON TSLÍ` (Firefox only). In some cases, a hyphen is also removed upon uppercasing: `an t-uisce` transforms to `AN tUISCE` (and the hyphen is correctly reinserted by `text-transform: lowercase`)

The language is defined by the `lang` HTML attribute or the `xml:lang` XML attribute.

Support for these specific cases vary from one browser to the other, so check the browser compatibility table.

Initial value	none
Aplica-se a	all elements. It also applies to `::first-letter` and `::first-line`.
Inherited	yes
Computed value	as specified
Animation type	discrete

Syntax

`capitalize`

Is a keyword forcing the first *letter* of each word to be converted to uppercase. Other characters are unchanged; that is, they retain their original case as written in the element's text. A letter is any Unicode character part of the Letter or Number general categories : it excludes any punctuation marks or symbols at the beginning of the word.

> **Nota:** Authors should not expect `capitalize` to follow language-specific title casing conventions (such as skipping articles in English).

> **Nota:** The `capitalize` keyword was under-specified in CSS 1 and CSS 2.1. There were differences between browsers in the way the first letter was calculated (Firefox considered - and _ as letters, but not the others. Both Webkit and Gecko incorrectly considered letter-based symbols like ⓐ to be real letters. Internet Explorer 9 was the closest to the CSS 2 definition, but with some weird cases). By precisely defining the correct behavior, CSS Text Level 3 cleans this mess up. The `capitalize` line in the browser compatibility table contains the version the different engines started to support this now precisely defined behavior.

`uppercase`

Is a keyword forcing all characters to be converted to uppercase.

`lowercase`

Is a keyword forcing all characters to be converted to lowercase.

`none`

Is a keyword preventing the case of all characters to be changed.

`full-width`

Is a keyword forcing the writing of a character — mainly ideograms and latin scripts — inside a square, allowing them to be aligned in the usual East Asian scripts (like Chinese or Japanese).

Formal syntax

```
text-transform =
  none                                                |
  [ capitalize | uppercase | lowercase ] || full-width || full-size-kana  |
  math-auto
```

Esta sintaxe reflete o padrão mais recente conforme CSS Text Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Examples

Example using "none"

```
<p>
  Initial String
  <strong>Lorem ipsum dolor sit amet, consectetur adipisicing elit, ...</strong>
</p>
<p>
  text-transform: none
  <strong
    ><span
      >Lorem ipsum dolor sit amet, consectetur adipisicing elit, ...</span
    ></strong
  >
</p>
```

```
span {
  text-transform: none;
}
strong {
  float: right;
}
```

This demonstrates no text transformation.

<code>capitalize</code> (General)

```
<p>
  Initial String
  <strong>Lorem ipsum dolor sit amet, consectetur adipisicing elit, ...</strong>
</p>
<p>
  text-transform: capitalize
  <strong
    ><span
      >Lorem ipsum dolor sit amet, consectetur adipisicing elit, ...</span
    ></strong
  >
</p>
```

```
span {
  text-transform: capitalize;
}
strong {
  float: right;
}
```

This demonstrates text capitalization.

<code>capitalize</code> (Punctuation)

```
<p>
  Initial String
  <strong
    >(this) “is” [a] –short– -test- «for» *the* _css_ ¿capitalize?
    ?¡transform!</strong
  >
</p>
<p>
  text-transform: capitalize
  <strong
    ><span
      >(this) “is” [a] –short– -test- «for» *the* _css_ ¿capitalize?
      ?¡transform!</span
    ></strong
  >
</p>
```

```
span {
  text-transform: capitalize;
}
strong {
  float: right;
}
```

This demostrates how initial punctuations of a word are ignored. The keyword target the first letter, that is the first Unicode character part of the Letter or Number general category.

<code>capitalize</code> (Symbols)

```
<p>
  Initial String
  <strong>ⓐⓑⓒ (ⓓⓔⓕ) –ⓖⓗⓘ– ⓙkl</strong>
</p>
<p>
  text-transform: capitalize
  <strong><span>ⓐⓑⓒ (ⓓⓔⓕ) –ⓖⓗⓘ– ⓙkl</span></strong>
</p>
```

```
span {
  text-transform: capitalize;
}
strong {
  float: right;
}
```

This demonstrates how initial symbols are ignored. The keyword target the first letter, that is the first Unicode character part of the Letter or Number general category.

<code>capitalize</code> (Dutch ij digraph)

```
<p>
  Initial String
  <strong lang="nl">The Dutch word: "ijsland" starts with a digraph.</strong>
</p>
<p>
  text-transform: capitalize
  <strong
    ><span lang="nl"
      >The Dutch word: "ijsland" starts with a digraph.</span
    ></strong
  >
</p>
```

```
span {
  text-transform: capitalize;
}
strong {
  float: right;
}
```

This demonstrates how the Dutch *ij* digraph must be handled like one single letter.

<code>uppercase</code> (General)

```
<p>
  Initial String
  <strong>Lorem ipsum dolor sit amet, consectetur adipisicing elit, ...</strong>
</p>
<p>
  text-transform: uppercase
  <strong
    ><span
      >Lorem ipsum dolor sit amet, consectetur adipisicing elit, ...</span
    ></strong
  >
</p>
```

```
span {
  text-transform: uppercase;
}
strong {
  float: right;
}
```

This demonstrates transforming the text to uppercase.

<code>uppercase</code> (Greek Vowels)

```
<p>
  Initial String
  <strong>Θα πάμε στο "Θεϊκό φαΐ" ή στη "Νεράιδα"</strong>
</p>
<p>
  text-transform: uppercase
  <strong><span>Θα πάμε στο "Θεϊκό φαΐ" ή στη "Νεράιδα"</span></strong>
</p>
```

```
span {
  text-transform: uppercase;
}
strong {
  float: right;
}
```

This demonstrates how Greek vowels except disjunctive *eta* should have no accent, and the accent on the first vowel of a vowel pair becomes a diaeresis on the second vowel.

<code>lowercase</code> (General)

```
<p>
  Initial String
  <strong>Lorem ipsum dolor sit amet, consectetur adipisicing elit, ...</strong>
</p>
<p>
  text-transform: lowercase
  <strong
    ><span
      >Lorem ipsum dolor sit amet, consectetur adipisicing elit, ...</span
    ></strong
  >
</p>
```

```
span {
  text-transform: lowercase;
}
strong {
  float: right;
}
```

This demonstrates transforming the text to lowercase.

<code>lowercase</code> (Greek Σ)

```
<p>
  Initial String
  <strong>Σ IS A greek LETTER that appears SEVERAL TIMES IN ΟΔΥΣΣΕΥΣ.</strong>
</p>
<p>
  text-transform: lowercase
  <strong
    ><span
      >Σ IS A greek LETTER that appears SEVERAL TIMES IN ΟΔΥΣΣΕΥΣ.</span
    ></strong
  >
</p>
```

```
span {
  text-transform: lowercase;
}
strong {
  float: right;
}
```

This demonstrates how the Greek character sigma (Σ) is transformed into the regular lowercase sigma (σ) or the word-final variant (ς), according the context.

<code>full-width</code> (General)

```
<p>
  Initial String
  <strong
    >0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ!"#$%&()*+,-./:;<=>?@{|}~</strong
  >
</p>
<p>
  text-transform: full-width
  <strong
    ><span
      >0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ!"#$%&()*+,-./:;<=>?@{|}~</span
    ></strong
  >
</p>
```

```
span {
  text-transform: full-width;
}
strong {
  width: 100%;
  float: right;
}
```

Some characters exists in two formats, normal width and a full-width, with different Unicode code points. The full-width version is used to mix them smoothly with Asian ideographic characters.

See also

- `font-variant`

transform

Resumo

A propriedade CSS `transform` permite modificar o espaço coordenado do modelo de formatação CSS. Usando-a, elementos podem ser traduzidos, rotacionados, ter seu tamanho ajustado e inclinados de acordo com os valores definidos.

Experimente

```
transform: matrix(1, 2, 3, 4, 5, 6);
```

```
transform: translate(120px, 50%);
```

```
transform: scale(2, 0.5);
```

```
transform: rotate(0.5turn);
```

```
transform: skew(30deg, 20deg);
```

```
transform: scale(0.5) translate(-100%, -100%);
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="example-element"
```

```
    src="/shared-assets/images/examples/firefox-logo.svg"
    width="200" />
</section>
```

Se a propriedade tem um valor diferente de `none`, um contexto de empilhamento será criado. Neste caso, o objeto atuará como um bloco recipiente para `position: fixed` para os elementos que estão contidos.

Sintaxe

```
Gramática Formal: <transform-função> [<transform-função>]* | none
```

```
transform: none
transform: matrix(1.0, 2.0, 3.0, 4.0, 5.0, 6.0)
transform: translate(12px, 50%)
transform: translateX(2em)
transform: translateY(3in)
transform: scale(2, 0.5)
transform: scaleX(2)
transform: scaleY(0.5)
transform: rotate(0.5turn)
transform: skewX(30deg)
transform: skewY(1.07rad)
transform: matrix3d(1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0, 11.0, 12.0, 13.0, 14.0, 15.0, 16.0)
transform: translate3d(12px, 50%, 3em)
transform: translateZ(2px)
transform: scale3d(2.5, 1.2, 0.3)
transform: scaleZ(0.3)
transform: rotate3d(1, 2.0, 3.0, 10deg)
transform: rotateX(10deg)
transform: rotateY(10deg)
transform: rotateZ(10deg)
transform: perspective(17px)

transform: translateX(10px) rotate(10deg) translateY(5px)
```

A propriedade transform pode ser especificada com a palavra reservada `none` ou com uma das várias funções disponíveis.

Se `perspective()` é uma das múltiplas funções atribuídas à transform, então ela obrigatoriamente deve ser a primeira.

Valores

`<transform-função>`

Uma ou mais das funções CSS transform para serem aplicadas, veja abaixo.

`none`

Especifica que nenhuma função transform deve ser aplicada.

Exemplos

Veja Using CSS transforms.

Exemplo ao vivo

```
pre {
  width: 33em;
  border: solid red;

  -webkit-transform: translate(100px) rotate(20deg);
  -webkit-transform-origin: 60% 100%;

  -o-transform: translate(100px) rotate(20deg);
  -o-transform-origin: 60% 100%;

  transform: translate(100px) rotate(20deg);
  transform-origin: 60% 100%;
}
```

CSS transform functions

The `transform` CSS property allows the coordinate system used by an element to be manipulated using **transform functions**. These functions are described below.

matrix

```
transform:  matrix(a, c, b, d, tx, ty)

/* Where a, b, c, d build the transformation matrix
   ┌     ┐
   │ a b │
   │ c d │
   └     ┘
   and tx, ty are the translate values.  */
```

Specifies a 2D transformation matrix comprised of the specified six values. This is the equivalent to applying the transformation **matrix [a b c d tx ty]**.

> **Nota:** Gecko (Firefox) accepts a `<length>` value for **tx** and **ty**. Webkit (Safari, Chrome) and Opera currently support a unitless `<number>` for **tx** and **ty**.

Live examples

```
 background: gold;  width: 30em;

 -webkit-transform: matrix(1, -0.2, 0, 1, 0, 0);
      -o-transform: matrix(1, -0.2, 0, 1, 0, 0);
         transform: matrix(1, -0.2, 0, 1, 0, 0);
```

```
 background: wheat;
 max-width: intrinsic;

 -webkit-transform: matrix(1, 0, 0.6, 1,  250, 0);
      -o-transform: matrix(1, 0, 0.6, 1,  250, 0);
         transform: matrix(1, 0, 0.6, 1,  250, 0);
```

See also

- Examples of linear transformation matrices Wikipedia
- Coordinate transformation matrices mathamazement.com
- Microsoft's matrix filter MSDN

rotate

```
transform:  rotate(angle);       /* an <angle>, e.g.  rotate(30deg) */
```

Rotates the element clockwise around its origin (as specified by the `transform-origin` property) by the specified `angle`. The operation corresponds to the matrix **[cos(angle) sin(angle) -sin(angle) cos(angle) 0 0]**.

scale

```
transform:  scale(sx[, sy]);     /* one or two unitless <number>s, e.g.  scale(2.1,4) */
```

Specifies a 2D scaling operation described by **[sx, sy]**. If `sy` isn't specified, it is assumed to be equal to `sx`.

scaleX

```
transform:  scaleX(sx);          /* a unitless <number>, e.g.  scaleX(2.7) */
```

Specifies a scale operation using the vector **[sx, 1]**.

scaleY

```
transform:  scaleY(sy)           /* a unitless <number>, e.g.  scaleY(0.3) */
```

Specifies a scale operation using the vector **[1, sy]**.

skew

> **Não padrão:** Este recurso não é padronizado. Não recomendamos o uso de recursos não padronizados em produção, pois eles têm suporte limitado nos navegadores e podem mudar ou ser removidos. No entanto, podem ser uma alternativa viável em casos específicos onde não exista uma opção padrão.

```
transform:  skew(ax[, ay])      /* one or two <angle>s, e.g.  skew(30deg,-10deg) */
```

Skews the element around the X and Y axes by the specified angles. If `ay` isn't provided, no skew is performed on the Y axis.

> **Nota:** The `skew()` function was present in early drafts. It has been removed but is still present in some implementations. Do not use it.
>
> To achieve the same effect, use `skewX()` if you were using `skew()` with one parameter or `matrix(1, tan(ax), tan(ay), 1, 0, 0)` for the general way. Note that *tan()* isn't a CSS function and you have to precalculate it yourself.

skewX

```
transform:  skewX(angle)        /* an <angle>, e.g.  skewX(-30deg) */
```

Skews the element around the X axis by the given `angle`.

skewY

```
transform:  skewY(angle)        /* an <angle>, e.g.  skewY(4deg) */
```

Skews the element around the Y axis by the given `angle`.

translate

```
transform:  translate(tx[, ty])  /* one or two <length> values */
```

Specifies a 2D translation by the vector **[tx, ty]**. If `ty` isn't specified, its value is assumed to be zero.

translateX

```
transform:  translateX(tx)      /* see <length> for possible values */
```

Translates the element by the given amount along the X axis.

translateY

```
transform:  translateY(ty)      /* see <length> for possible values */
```

Translates the element by the given amount along the Y axis.

See also

- Using CSS Transforms
- More info on CSS3 Rotation / Matrix Filter issues in the comments on Paul Irish's blog.
- A cross-browser 2D transform plugin for jQuery

transform-origin

A propriedade **`transform-origin`** define a origem das transformações de um elemento no CSS.

Experimente

```
transform-origin: center;
```

```
transform-origin: top left;
```

```
transform-origin: 50px 50px;
```

```
/* 3D rotation with z-axis origin */
transform-origin: bottom right 60px;
```

```
<section id="default-example">
  <div id="example-container">
    <div id="example-element">Rotate me!</div>
    <img
      alt=""
      id="crosshair"
      src="/shared-assets/images/examples/crosshair.svg"
      width="24px" />
    <div id="static-element"></div>
  </div>
</section>
```

```
@keyframes rotate {
  from {
    transform: rotate(0);
  }

  to {
    transform: rotate(30deg);
  }
}

@keyframes rotate3d {
  from {
    transform: rotate3d(0);
  }

  to {
    transform: rotate3d(1, 2, 0, 60deg);
  }
}

#example-container {
  width: 160px;
  height: 160px;
  position: relative;
}

#example-element {
  width: 100%;
  height: 100%;
  display: flex;
  position: absolute;
  align-items: center;
  justify-content: center;
  background: #f7ebee;
  color: #000000;
  font-size: 1.2rem;
  text-transform: uppercase;
}

#example-element.rotate {
  animation: rotate 1s forwards;
}

#example-element.rotate3d {
  animation: rotate3d 1s forwards;
}

#crosshair {
  width: 24px;
  height: 24px;
  opacity: 0;
  position: absolute;
}

#static-element {
  width: 100%;
  height: 100%;
```

```
  position: absolute;
  border: dotted 3px #ff1100;
}
```

```
"use strict";

window.addEventListener("load", () => {
  function update() {
    const selected = document.querySelector(".selected");

    /* Restart the animation
           https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Animations/Tips */
    el.className = "";
    window.requestAnimationFrame(() => {
      window.requestAnimationFrame(() => {
        el.className =
          el.style.transformOrigin.split(" ")[2] === "60px"
            ? "rotate3d"
            : "rotate";
      });
    });

    const transformOrigin = getComputedStyle(el).transformOrigin;
    const pos = transformOrigin.split(/\s+/);
    crosshair.style.left = `calc(${pos[0]} - 12px)`;
    crosshair.style.top = `calc(${pos[1]} - 12px)`;
  }

  const crosshair = document.getElementById("crosshair");
  const el = document.getElementById("example-element");

  const observer = new MutationObserver(() => {
    update();
  });

  observer.observe(el, {
    attributes: true,
    attributeFilter: ["style"],
  });

  update();
  crosshair.style.opacity = "1";
});
```

A origem da transformação é o ponto em torno do qual uma transformação é aplicada. Por exemplo, a origem da transformação da função `rotate()` é o centro de rotação.

Na prática, essa propriedade envolve um par de translações em torno das outras transformações do elemento. A primeira translação move a origem da transformação para a origem real em $(0, 0)$. Em seguida, as outras transformações são aplicadas e, porque a origem da transformação está em $(0, 0)$, essas transformações atuam em relação à origem da transformação. Por fim, a translação oposta é aplicada, movendo a origem da transformação de volta à sua localização original. Consequentemente, essa definição

```
transform-origin: -100% 50%;
transform: rotate(45deg);
```

resulta na mesma transformação que

```
transform-origin: 0 0;
transform: translate(-100%, 50%) rotate(45deg) translate(100%, -50%);
```

Lendo da direita para a esquerda, `translate(100%, -50%)` é a tradução para trazer a origem da transformação para a origem real, `rotate(45deg)` é a transformação original e `translate(-100%, 50%)` é a tradução para restaurar a origem da transformação à sua localização original.

Por padrão, a origem de uma transformação é `center` (centro).

Sintaxe

```
/* One-value syntax */
transform-origin: 2px;
transform-origin: bottom;
```

```
/* x-offset | y-offset */
transform-origin: 3cm 2px;

/* x-offset-keyword | y-offset */
transform-origin: left 2px;

/* x-offset-keyword | y-offset-keyword */
transform-origin: right top;

/* y-offset-keyword | x-offset-keyword */
transform-origin: top right;

/* x-offset | y-offset | z-offset */
transform-origin: 2px 30% 10px;

/* x-offset-keyword | y-offset | z-offset */
transform-origin: left 5px -3px;

/* x-offset-keyword | y-offset-keyword | z-offset */
transform-origin: right bottom 2cm;

/* y-offset-keyword | x-offset-keyword | z-offset */
transform-origin: bottom right 2cm;

/* Global values */
transform-origin: inherit;
transform-origin: initial;
transform-origin: revert;
transform-origin: revert-layer;
transform-origin: unset;
```

A propriedade `transform-origin` pode ser especificada usando um, dois ou três valores, onde cada valor representa um deslocamento. Deslocamentos que não são definidos explicitamente são redefinidos para seus respectivos valores iniciais.

Se um único valor `<length>` ou `<percentage>` for definido, ele representa o deslocamento horizontal.

Se dois ou mais valores forem definidos e nenhum deles for uma palavra-chave, ou a única palavra-chave utilizada for `center`, então o primeiro valor representa o deslocamento horizontal e o segundo representa o deslocamento vertical.

- Sintaxe de um valor:
    - O valor deve ser um `<length>`, um `<percentage>`, ou uma das palavras-chave `left`, `center`, `right`, `top` e `bottom`.
- Sintaxe de dois valores:
    - Um valor deve ser um `<length>`, um `<percentage>`, ou uma das palavras-chave `left`, `center` e `right`.
    - O outro valor deve ser um `<length>`, um `<percentage>`, ou uma das palavras-chave `top`, `center` e `bottom`.
- Sintaxe de três valores:
    - Os dois primeiros valores são os mesmos da sintaxe de dois valores.
    - O terceiro valor deve ser um `<length>`. Ele representa sempre o deslocamento Z.

Valores

x-offset

É um `<length>` ou um `<percentage>` que descreve a distância a partir da borda esquerda da caixa onde a origem da transformação é definida.

offset-keyword

É uma das palavras-chave `left`, `right`, `top`, `bottom` ou `center` que descreve o deslocamento correspondente.

y-offset

É um `<length>` ou um `<percentage>` que descreve a distância a partir da borda superior da caixa onde a origem da transformação é definida.

x-offset-keyword

É uma das palavras-chave `left`, `right` ou `center` que descreve a distância a partir da borda esquerda da caixa onde a origem da transformação é definida.

y-offset-keyword

É uma das palavras-chave `top`, `bottom` ou `center` que descreve a distância a partir da borda superior da caixa onde a origem da transformação é definida.

z-offset

É um `<length>` (e nunca um `<percentage>` que tornaria a afirmação inválida) que descreve a distância do olho do usuário onde a origem z=0 é definida.

As palavras-chave são atalhos convenientes e correspondem aos seguintes valores de `<percentage>`:

Keyword	Value
`left`	`0%`
`center`	`50%`
`right`	`100%`
`top`	`0%`
`bottom`	`100%`

Definição formal

Initial value	`50% 50% 0`
Aplica-se a	qualquer elemento transformavel
Inherited	não
Percentages	refer to the size of bounding box
Computed value	for `<length>` the absolute value, otherwise a percentage
Animation type	simple list of length, percentage, or calc

> **Nota:** O valor inicial de `transform-origin` é `0 0` para todos os elementos SVG, exceto para elementos `<svg>` raiz e elementos `<svg>` que são filhos diretos de um elemento foreignObject, cujo `transform-origin` é `50% 50%`, assim como outros elementos CSS. Consulte o atributo transform-origin do SVG para obter mais informações.

Sintaxe formal

```
transform-origin =
  [ left | center | right | top | bottom | <length-percentage> ]  |
  [ left | center | right | <length-percentage> ] [ top | center | bottom | <length-percentage> ] <length>?  |
  [ [ center | left | right ] && [ center | top | bottom ] ] <length>?

<length-percentage> =
  <length>      |
  <percentage>
```

Esta sintaxe reflete o padrão mais recente conforme CSS Transforms Module Level 1, CSS Values and Units Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

Uma demonstração de vários valores de transformação

Este exemplo mostra o efeito de escolher diferentes valores de `transform-origin` para uma variedade de funções de transformação.

```
<div class="container">
  <div class="example">
    <div class="box box1">&amp;nbsp;</div>
    <div class="box original">&amp;nbsp;</div>
  </div>

  <pre>
transform: none;
  </pre>
```

```
  <div class="example">
    <div class="box box2"> </div>
    <div class="box original"> </div>
  </div>

  <pre>
transform: rotate(30deg);
  </pre>

  <div class="example">
    <div class="box box3"> </div>
    <div class="box original"> </div>
  </div>

  <pre>
transform: rotate(30deg);
transform-origin: 0 0;
  </pre>

  <div class="example">
    <div class="box box4"> </div>
    <div class="box original"> </div>
  </div>

  <pre>
transform: rotate(30deg);
transform-origin: 100% 100%;
  </pre>

  <div class="example">
    <div class="box box5"> </div>
    <div class="box original"> </div>
  </div>

  <pre>
transform: rotate(30deg);
transform-origin: -1em -3em;
  </pre>

  <div class="example">
    <div class="box box6"> </div>
    <div class="box original"> </div>
  </div>

  <pre>
transform: scale(1.7);
  </pre>

  <div class="example">
    <div class="box box7"> </div>
    <div class="box original"> </div>
  </div>

  <pre>
transform: scale(1.7);
transform-origin: 0 0;
  </pre>

  <div class="example">
    <div class="box box8"> </div>
    <div class="box original"> </div>
  </div>

  <pre>
transform: scale(1.7);
transform-origin: 100% -30%;
  </pre>

  <div class="example">
    <div class="box box9"> </div>
    <div class="box original"> </div>
  </div>
```

```
  <pre>
transform: skewX(50deg);
transform-origin: 100% -30%;
  </pre>

  <div class="example">
    <div class="box box10">&amp;nbsp;</div>
    <div class="box original">&amp;nbsp;</div>
  </div>

  <pre>
transform: skewY(50deg);
transform-origin: 100% -30%;
  </pre>
</div>
```

```
.container {
  display: grid;
  grid-template-columns: 200px 100px;
  gap: 20px;
}

.example {
  position: relative;
  margin: 0 2em 4em 5em;
}

.box {
  display: inline-block;
  width: 3em;
  height: 3em;
  border: solid 1px;
  background-color: palegreen;
}

.original {
  position: absolute;
  left: 0;
  opacity: 20%;
}

.box1 {
  transform: none;
}

.box2 {
  transform: rotate(30deg);
}

.box3 {
  transform: rotate(30deg);
  transform-origin: 0 0;
}

.box4 {
  transform: rotate(30deg);
  transform-origin: 100% 100%;
}

.box5 {
  transform: rotate(30deg);
  transform-origin: -1em -3em;
}

.box6 {
  transform: scale(1.7);
}

.box7 {
  transform: scale(1.7);
  transform-origin: 0 0;
}
```

```
.box8 {
  transform: scale(1.7);
  transform-origin: 100% -30%;
}

.box9 {
  transform: skewX(50deg);
  transform-origin: 100% -30%;
}

.box10 {
  transform: skewY(50deg);
  transform-origin: 100% -30%;
}
```

Veja também

- Usando transformações CSS
- https://css-tricks.com/almanac/properties/t/transform-origin/

transform-style

A propriedade **`transform-style`** CSS define se os elemento filhos são posicionados em um espaco 3D ou serão achatados (flattened) no plano do elemento

Experimente

```
transform-style: flat;
```

```
transform-style: preserve-3d;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all layer" id="example-element">
    <p>Parent</p>
    <div class="numeral"><code>rotate3d(1, 1, 1, 45deg)</code></div>
  </div>
</section>
```

```
.layer {
  background: #623e3f;
  border-radius: 0.75rem;
  color: white;
  transform: perspective(200px) rotateY(30deg);
}

.numeral {
  background-color: #ffba08;
  border-radius: 0.2rem;
  color: #000;
  margin: 1rem;
  padding: 0.2rem;
  transform: rotate3d(1, 1, 1, 45deg);
}
```

Se achatados, os elementos filho não existirão por conta própria ou sozinhos no espaço 3D.

Como essa propriedade não é herdada, ela deve ser definida para todos os descendentes que não sejam folhas do elemento.

Syntax

```
/* Keyword values */
transform-style: flat;
transform-style: preserve-3d;

/* Valores globais */
transform-style: inherit;
transform-style: initial;
transform-style: unset;
```

Values

`flat`

Indica que os filhos do elemento estão no plano do próprio elemento.

`preserve-3d`

Indica que os filhos do elemento devem ser posicionados no espaço 3D.

Formal syntax

```
transform-style =
  flat         |
  preserve-3d
```

Esta sintaxe reflete o padrão mais recente conforme CSS Transforms Module Level 2. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Initial value	`flat`
Aplica-se a	qualquer elemento transformavel
Inherited	não
Computed value	as specified
Animation type	discrete
Creates stacking context	yes

Veja também

- Using CSS transforms

transition

> **Experimental: Esta é uma tecnologia experimental**
> Verifique a tabela de compatibilidade entre Navegadores cuidadosamente antes de usar essa funcionalidade em produção.

Summary

A propriedade de CSS**transition** é uma propriedade abreviada para `transition-property`, `transition-duration`, `transition-timing-function` e `transition-delay`. Ela permite definir a transição entre dois estados de um elemento. Estados diferentes podem ser definidos usando pseudo-classes tais como `:hover` ou `:active`; ou dinamicamente, usando javascript.

Initial value	as each of the properties of the shorthand: • `transition-delay`: `0s` • `transition-duration`: `0s` • `transition-property`: `all` • `transition-timing-function`: `ease` • `transition-behavior`: `normal`
Aplica-se a	all elements, `::before` and `::after` pseudo-elements
Inherited	não
Computed value	as each of the properties of the shorthand: • `transition-delay`: as specified • `transition-duration`: as specified • `transition-property`: as specified • `transition-timing-function`: as specified • `transition-behavior`: as specified
Animation type	Not animatable

Syntax

```
/* Apply to 1 property */
/* property name | duration */
transition: margin-left 4s;

/* property name | duration | delay */
transition: margin-left 4s 1s;

/* property name | duration | timing function | delay */
transition: margin-left 4s ease-in-out 1s;

/* Apply to 2 properties */
transition:
  margin-left 4s,
  color 1s;

/* Apply to all changed properties */
transition: all 0.5s ease-out;

/* Global values */
transition: inherit;
transition: initial;
transition: unset;
```

Observe que a ordem é importante dentro dos itens nesta propriedade: o primeiro valor que pode ser analisado como tempo é atribuído ao `transition-duration`, duration e o segundo valor que pode ser analisado como tempo é atribuído ao `transition-delay`.

Veja como as coisas são manipuladas quando as listas de valores de propriedades não têm o mesmo comprimento. Em resumo, descrições de transição extras além do número de propriedades realmente sendo animadas são ignoradas.

Formal syntax

```
transition =
  <single-transition>#

<single-transition> =
  [ none | <single-transition-property> ]  ||
  <time>                                   ||
  <easing-function>                        ||
  <time>                                   ||
  <transition-behavior-value>

<single-transition-property> =
  all             |
  <custom-ident>

<easing-function> =
  <linear-easing-function>        |
  <cubic-bezier-easing-function>  |
  <step-easing-function>

<transition-behavior-value> =
  normal          |
  allow-discrete

<linear-easing-function> =
  linear      |
  <linear()>

<cubic-bezier-easing-function> =
  ease              |
  ease-in           |
  ease-out          |
  ease-in-out       |
  <cubic-bezier()>

<step-easing-function> =
  step-start  |
  step-end    |
  <steps()>

<linear()> =
  linear( [ <number> && <percentage>{0,2} ]# )
```

```
<cubic-bezier()> =
  cubic-bezier( [ <number [0,1]> , <number> ]#{2} )

<steps()> =
  steps( <integer> , <step-position>? )

<integer> =
  <number-token>

<step-position> =
  jump-start  |
  jump-end    |
  jump-none   |
  jump-both   |
  start       |
  end
```

Esta sintaxe reflete o padrão mais recente conforme CSS Easing Functions Level 2, CSS Transitions Module Level 1, CSS Transitions Module Level 2, CSS Values and Units Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Examples

Existem vários outros exemplos de transições CSS incluídas no artigo principal sobre CSS transitions.

See also

- Using CSS transitions
- `TransitionEvent`

transition-delay

A propriedade CSS **`transition-delay`** (atraso de transição) determina o tempo de duração da espera antes de iniciar um efeito de transição(transition effect) quando seu valor muda.

Experimente

```
transition-delay: 250ms;
transition-property: margin-right;
```

```
transition-delay: 1s;
transition-property: background-color;
```

```
transition-delay: 1s;
transition-property: margin-right, color;
```

```
transition-delay: 1s, 250ms;
transition-property: margin-right, color;
```

```
<section id="default-example">
  <div id="example-element">Hover to see<br />the transition.</div>
</section>
```

```
#example-element {
  background-color: #e4f0f5;
  color: #000;
  padding: 1rem;
  border-radius: 0.5rem;
  font: 1em monospace;
  width: 100%;
  transition: margin-right 2s;
}

#default-example:hover > #example-element {
  background-color: #909;
  color: #fff;
  margin-right: 40%;
}
```

Essa espera, ou atraso de transição, pode ser zero, positiva ou negativa:

- Um valor de `0s`(ou `0ms`) iniciará o efeito de transição imediatamente.
- Um valor positivo atrasará o inicio do efeito de transição de acordo com o tempo estabelecido.
- Um valor negativo iniciará o efeito de transição imediatamente e parcialmente durante o efeito. Em outras palavras, o efeito será animado como se já tivesse sido executado pelo tempo estipulado.

Você pode especificar vários atrasos, o que é útil quando é necessário realizar a transição de várias propriedades. Cada atraso pode ser aplicado à propriedade correspondente como especificado pela propriedade de transição(`transition-property`), que atua como uma lista mestre. Se existem menos atrasos do que o determinado pela lista mestre, a lista de valores de atraso será repetida até que seja suficiente. Se existem mais atrasos, a lista de valores de atraso será trucanda(limitada) ao combinar o número de propriedades. Em ambos os casos, a declaração CSS permacene válida.

Syntax

```
/* Valores de Tempo <time>  */
transition-delay: 3s;
transition-delay: 2s, 4ms;

/* Valores Globais */
transition-delay: inherit;
transition-delay: initial;
transition-delay: unset;
```

Valores

`<time>`

Indica a quantidade de tempo de espera entre um valor de propriedade e o inicio do efeito de transição(transition effect).

Sintaxe Formal

```
transition-delay =
  <time>#
```

Esta sintaxe reflete o padrão mais recente conforme CSS Transitions Module Level 1. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

`transition-delay: 0.5s`

```
<div class="parent">
  <div class="box">Lorem</div>
</div>
```

```
.parent {
  width: 250px;
  height: 125px;
}
.box {
  width: 100px;
  height: 100px;
  background-color: red;
  font-size: 20px;
  left: 0px;
  top: 0px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left top
    color;
  -webkit-transition-duration: 2s;
  -webkit-transition-delay: 0.5s;
  -webkit-transition-timing-function: linear;
  transition-property: width height background-color font-size left top color;
  transition-duration: 2s;
  transition-delay: 0.5s;
  transition-timing-function: linear;
}
.box1 {
  width: 50px;
  height: 50px;
  background-color: blue;
  color: yellow;
  font-size: 18px;
```

```
  left: 150px;
  top: 25px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left top
    color;
  -webkit-transition-duration: 2s;
  -webkit-transition-delay: 0.5s;
  -webkit-transition-timing-function: linear;
  transition-property: width height background-color font-size left top color;
  transition-duration: 2s;
  transition-delay: 0.5s;
  transition-timing-function: linear;
}
```

```
function updateTransition() {
  var el = document.querySelector("div.box");

  if (el) {
    el.className = "box1";
  } else {
    el = document.querySelector("div.box1");
    el.className = "box";
  }

  return el;
}

var intervalID = window.setInterval(updateTransition, 7000);
```

transition-delay: 1s

```
<div class="parent">
  <div class="box">Lorem</div>
</div>
```

```
.parent {
  width: 250px;
  height: 125px;
}
.box {
  width: 100px;
  height: 100px;
  background-color: red;
  font-size: 20px;
  left: 0px;
  top: 0px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left top
    color;
  -webkit-transition-duration: 2s;
  -webkit-transition-delay: 1s;
  -webkit-transition-timing-function: linear;
  transition-property: width height background-color font-size left top color;
  transition-duration: 2s;
  transition-delay: 1s;
  transition-timing-function: linear;
}
.box1 {
  width: 50px;
  height: 50px;
  background-color: blue;
  color: yellow;
  font-size: 18px;
  left: 150px;
  top: 25px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left top
    color;
  -webkit-transition-duration: 2s;
  -webkit-transition-delay: 1s;
  -webkit-transition-timing-function: linear;
```

```
  transition-property: width height background-color font-size left top color;
  transition-duration: 2s;
  transition-delay: 1s;
  transition-timing-function: linear;
}
```

```
function updateTransition() {
  var el = document.querySelector("div.box");

  if (el) {
    el.className = "box1";
  } else {
    el = document.querySelector("div.box1");
    el.className = "box";
  }

  return el;
}

var intervalID = window.setInterval(updateTransition, 7000);
```

transition-delay: 2s

```
<div class="parent">
  <div class="box">Lorem</div>
</div>
```

```
.parent {
  width: 250px;
  height: 125px;
}
.box {
  width: 100px;
  height: 100px;
  background-color: red;
  font-size: 20px;
  left: 0px;
  top: 0px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left top
    color;
  -webkit-transition-duration: 2s;
  -webkit-transition-delay: 2s;
  -webkit-transition-timing-function: linear;
  transition-property: width height background-color font-size left top color;
  transition-duration: 2s;
  transition-delay: 2s;
  transition-timing-function: linear;
}
.box1 {
  width: 50px;
  height: 50px;
  background-color: blue;
  color: yellow;
  font-size: 18px;
  left: 150px;
  top: 25px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left top
    color;
  -webkit-transition-duration: 2s;
  -webkit-transition-delay: 2s;
  -webkit-transition-timing-function: linear;
  transition-property: width height background-color font-size left top color;
  transition-duration: 2s;
  transition-delay: 2s;
  transition-timing-function: linear;
}
```

```
function updateTransition() {
  var el = document.querySelector("div.box");
```

```
  if (el) {
    el.className = "box1";
  } else {
    el = document.querySelector("div.box1");
    el.className = "box";
  }

  return el;
}

var intervalID = window.setInterval(updateTransition, 7000);
```

transition-delay: 4s

```
<div class="parent">
  <div class="box">Lorem</div>
</div>
```

```
.parent {
  width: 250px;
  height: 125px;
}
.box {
  width: 100px;
  height: 100px;
  background-color: red;
  font-size: 20px;
  left: 0px;
  top: 0px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left top
    color;
  -webkit-transition-duration: 2s;
  -webkit-transition-delay: 4s;
  -webkit-transition-timing-function: ease-in-out;
  transition-property: width height background-color font-size left top color;
  transition-duration: 2s;
  transition-delay: 4s;
  transition-timing-function: ease-in-out;
}
.box1 {
  width: 50px;
  height: 50px;
  background-color: blue;
  color: yellow;
  font-size: 18px;
  left: 150px;
  top: 25px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left top
    color;
  -webkit-transition-duration: 2s;
  -webkit-transition-delay: 4s;
  -webkit-transition-timing-function: ease-in-out;
  transition-property: width height background-color font-size left top color;
  transition-duration: 2s;
  transition-delay: 4s;
  transition-timing-function: ease-in-out;
}
```

```
function updateTransition() {
  var el = document.querySelector("div.box");

  if (el) {
    el.className = "box1";
  } else {
    el = document.querySelector("div.box1");
    el.className = "box";
  }
```

```
  return el;
}

var intervalID = window.setInterval(updateTransition, 7000);
```

Initial value	`0s`
Aplica-se a	all elements, `::before` and `::after` pseudo-elements
Inherited	não
Computed value	as specified
Animation type	Not animatable

Veja também

- Usando Transições CSS
- `TransitionEvent` API

transition-timing-function

A propriedade CSS **`transition-timing-function`** é usada para descrever como os valores intermediários das propriedades CSS sendo afetados por um efeito de transição são calculados. Em essência isso permite você estabelecer uma curva de aceleração, para que então a velocidade da transição possa variar durante sua duração.

Essa curva de aceleração é definida usando `<timing-function>` para cada propriedade a ser transicionada.

```
/* Keyword values */
transition-timing-function: ease;
transition-timing-function: ease-in;
transition-timing-function: ease-out;
transition-timing-function: ease-in-out;
transition-timing-function: linear;
transition-timing-function: step-start;
transition-timing-function: step-end;

/* Function values */
transition-timing-function: steps(4, end);
transition-timing-function: cubic-bezier(0.1, 0.7, 1, 0.1);
transition-timing-function: frames(10);

/* Multiple timing functions */
transition-timing-function: ease, step-start, cubic-bezier(0.1, 0.7, 1, 0.1);

/* Global values */
transition-timing-function: inherit;
transition-timing-function: initial;
transition-timing-function: unset;
```

You may specify multiple timing functions; each one will be applied to the corresponding property as specified by the `transition-property` property, which acts as a master list. If there are fewer functions specified than in the master list, missing values are set to the initial value (`ease`). If there are more timing functions, the list is simply truncated to the right size. In both case the CSS declaration stays valid.

Initial value	`ease`
Aplica-se a	all elements, `::before` and `::after` pseudo-elements
Inherited	não
Computed value	as specified
Animation type	Not animatable

Syntax

Values

`<timing-function>`

Each `<timing-function>` represents the timing function to link to the corresponding property to transition, as defined in `transition-property`.

Formal syntax

```
transition-timing-function =
  <easing-function>#

<easing-function> =
  <linear-easing-function>        |
  <cubic-bezier-easing-function>  |
  <step-easing-function>

<linear-easing-function> =
  linear      |
  <linear()>

<cubic-bezier-easing-function> =
  ease              |
  ease-in           |
  ease-out          |
  ease-in-out       |
  <cubic-bezier()>

<step-easing-function> =
  step-start  |
  step-end    |
  <steps()>

<linear()> =
  linear( [ <number> && <percentage>{0,2} ]# )

<cubic-bezier()> =
  cubic-bezier( [ <number [0,1]> , <number> ]#{2} )

<steps()> =
  steps( <integer> , <step-position>? )

<integer> =
  <number-token>

<step-position> =
  jump-start  |
  jump-end    |
  jump-none   |
  jump-both   |
  start       |
  end
```

Esta sintaxe reflete o padrão mais recente conforme CSS Easing Functions Level 2, CSS Transitions Module Level 1, CSS Values and Units Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Examples

`transition-timing-function: ease`

```
<div class="parent">
  <div class="box">Lorem</div>
</div>
```

```
.parent {
  width: 250px;
  height: 125px;
}
.box {
  width: 100px;
  height: 100px;
  background-color: red;
  font-size: 20px;
  left: 0px;
  top: 0px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left top
    color;
  -webkit-transition-duration: 2s;
  -webkit-transition-timing-function: ease;
  transition-property: width height background-color font-size left top color;
```

```
  transition-duration: 2s;
  transition-timing-function: ease;
}
.box1 {
  width: 50px;
  height: 50px;
  background-color: blue;
  color: yellow;
  font-size: 18px;
  left: 150px;
  top: 25px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left top
    color;
  -webkit-transition-duration: 2s;
  -webkit-transition-timing-function: ease;
  transition-property: width height background-color font-size left top color;
  transition-duration: 2s;
  transition-timing-function: ease;
}
```

```
function updateTransition() {
  var el = document.querySelector("div.box");

  if (el) {
    el.className = "box1";
  } else {
    el = document.querySelector("div.box1");
    el.className = "box";
  }

  return el;
}

var intervalID = window.setInterval(updateTransition, 7000);
```

transition-timing-function: ease-in

```
<div class="parent">
  <div class="box">Lorem</div>
</div>
```

```
.parent {
  width: 250px;
  height: 125px;
}
.box {
  width: 100px;
  height: 100px;
  background-color: red;
  font-size: 20px;
  left: 0px;
  top: 0px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left top
    color;
  -webkit-transition-duration: 2s;
  -webkit-transition-timing-function: ease-in;
  transition-property: width height background-color font-size left top color;
  transition-duration: 2s;
  transition-timing-function: ease-in;
}
.box1 {
  width: 50px;
  height: 50px;
  background-color: blue;
  color: yellow;
  font-size: 18px;
  left: 150px;
  top: 25px;
  position: absolute;
```

```
  -webkit-transition-property: width height background-color font-size left top
    color;
  -webkit-transition-duration: 2s;
  -webkit-transition-timing-function: ease-in;
  transition-property: width height background-color font-size left top color;
  transition-duration: 2s;
  transition-timing-function: ease-in;
}
```

```
function updateTransition() {
  var el = document.querySelector("div.box");

  if (el) {
    el.className = "box1";
  } else {
    el = document.querySelector("div.box1");
    el.className = "box";
  }

  return el;
}

var intervalID = window.setInterval(updateTransition, 7000);
```

transition-timing-function: ease-out

```
<div class="parent">
  <div class="box">Lorem</div>
</div>
```

```
.parent {
  width: 250px;
  height: 125px;
}
.box {
  width: 100px;
  height: 100px;
  background-color: red;
  font-size: 20px;
  left: 0px;
  top: 0px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left top
    color;
  -webkit-transition-duration: 2s;
  -webkit-transition-timing-function: ease-out;
  transition-property: width height background-color font-size left top color;
  transition-duration: 2s;
  transition-timing-function: ease-out;
}
.box1 {
  width: 50px;
  height: 50px;
  background-color: blue;
  color: yellow;
  font-size: 18px;
  left: 150px;
  top: 25px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left top
    color;
  -webkit-transition-duration: 2s;
  -webkit-transition-timing-function: ease-out;
  transition-property: width height background-color font-size left top color;
  transition-duration: 2s;
  transition-timing-function: ease-out;
}
```

```
function updateTransition() {
  var el = document.querySelector("div.box");
```

```
  if (el) {
    el.className = "box1";
  } else {
    el = document.querySelector("div.box1");
    el.className = "box";
  }

  return el;
}

var intervalID = window.setInterval(updateTransition, 7000);
```

transition-timing-function: ease-in-out

```
<div class="parent">
  <div class="box">Lorem</div>
</div>
```

```
.parent {
  width: 250px;
  height: 125px;
}
.box {
  width: 100px;
  height: 100px;
  background-color: red;
  font-size: 20px;
  left: 0px;
  top: 0px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left top
    color;
  -webkit-transition-duration: 2s;
  -webkit-transition-timing-function: ease-in-out;
  transition-property: width height background-color font-size left top color;
  transition-duration: 2s;
  transition-timing-function: ease-in-out;
}
.box1 {
  width: 50px;
  height: 50px;
  background-color: blue;
  color: yellow;
  font-size: 18px;
  left: 150px;
  top: 25px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left top
    color;
  -webkit-transition-duration: 2s;
  -webkit-transition-timing-function: ease-in-out;
  transition-property: width height background-color font-size left top color;
  transition-duration: 2s;
  transition-timing-function: ease-in-out;
}
```

```
function updateTransition() {
  var el = document.querySelector("div.box");

  if (el) {
    el.className = "box1";
  } else {
    el = document.querySelector("div.box1");
    el.className = "box";
  }

  return el;
}

var intervalID = window.setInterval(updateTransition, 7000);
```

transition-timing-function: linear

```
<div class="parent">
  <div class="box">Lorem</div>
</div>
```

```
.parent {
  width: 250px;
  height: 125px;
}
.box {
  width: 100px;
  height: 100px;
  background-color: red;
  font-size: 20px;
  left: 0px;
  top: 0px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left top
    color;
  -webkit-transition-duration: 2s;
  -webkit-transition-timing-function: linear;
  transition-property: width height background-color font-size left top color;
  transition-duration: 2s;
  transition-timing-function: linear;
}
.box1 {
  width: 50px;
  height: 50px;
  background-color: blue;
  color: yellow;
  font-size: 18px;
  left: 150px;
  top: 25px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left top
    color;
  -webkit-transition-duration: 2s;
  -webkit-transition-timing-function: linear;
  transition-property: width height background-color font-size left top color;
  transition-duration: 2s;
  transition-timing-function: linear;
}
```

```
function updateTransition() {
  var el = document.querySelector("div.box");

  if (el) {
    el.className = "box1";
  } else {
    el = document.querySelector("div.box1");
    el.className = "box";
  }

  return el;
}

var intervalID = window.setInterval(updateTransition, 7000);
```

transition-timing-function: step-start

```
<div class="parent">
  <div class="box">Lorem</div>
</div>
```

```
.parent {
  width: 250px;
  height: 125px;
}
.box {
  width: 100px;
```

```
  height: 100px;
  background-color: red;
  font-size: 20px;
  left: 0px;
  top: 0px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left top
    color;
  -webkit-transition-duration: 2s;
  -webkit-transition-timing-function: step-start;
  transition-property: width height background-color font-size left top color;
  transition-duration: 2s;
  transition-timing-function: step-start;
}
.box1 {
  width: 50px;
  height: 50px;
  background-color: blue;
  color: yellow;
  font-size: 18px;
  left: 150px;
  top: 25px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left top
    color;
  -webkit-transition-duration: 2s;
  -webkit-transition-timing-function: step-start;
  transition-property: width height background-color font-size left top color;
  transition-duration: 2s;
  transition-timing-function: step-start;
}
```

```
function updateTransition() {
  var el = document.querySelector("div.box");

  if (el) {
    el.className = "box1";
  } else {
    el = document.querySelector("div.box1");
    el.className = "box";
  }

  return el;
}

var intervalID = window.setInterval(updateTransition, 7000);
```

transition-timing-function: step-end

```
<div class="parent">
  <div class="box">Lorem</div>
</div>
```

```
.parent {
  width: 250px;
  height: 125px;
}
.box {
  width: 100px;
  height: 100px;
  background-color: red;
  font-size: 20px;
  left: 0px;
  top: 0px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left top
    color;
  -webkit-transition-duration: 2s;
  -webkit-transition-timing-function: step-end;
  transition-property: width height background-color font-size left top color;
  transition-duration: 2s;
```

```
  transition-timing-function: step-end;
}
.box1 {
  width: 50px;
  height: 50px;
  background-color: blue;
  color: yellow;
  font-size: 18px;
  left: 150px;
  top: 25px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left top
    color;
  -webkit-transition-duration: 2s;
  -webkit-transition-timing-function: step-end;
  transition-property: width height background-color font-size left top color;
  transition-duration: 2s;
  transition-timing-function: step-end;
}
```

```
function updateTransition() {
  var el = document.querySelector("div.box");

  if (el) {
    el.className = "box1";
  } else {
    el = document.querySelector("div.box1");
    el.className = "box";
  }

  return el;
}

var intervalID = window.setInterval(updateTransition, 7000);
```

transition-timing-function: steps(4, end)

```
<div class="parent">
  <div class="box">Lorem</div>
</div>
```

```
.parent {
  width: 250px;
  height: 125px;
}
.box {
  width: 100px;
  height: 100px;
  background-color: red;
  font-size: 20px;
  left: 0px;
  top: 0px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left top
    color;
  -webkit-transition-duration: 2s;
  -webkit-transition-timing-function: steps(4, end);
  transition-property: width height background-color font-size left top color;
  transition-duration: 2s;
  transition-timing-function: steps(4, end);
}
.box1 {
  width: 50px;
  height: 50px;
  background-color: blue;
  color: yellow;
  font-size: 18px;
  left: 150px;
  top: 25px;
  position: absolute;
  -webkit-transition-property: width height background-color font-size left top
```

```
    color;
  -webkit-transition-duration: 2s;
  -webkit-transition-timing-function: steps(4, end);
  transition-property: width height background-color font-size left top color;
  transition-duration: 2s;
  transition-timing-function: steps(4, end);
}
```

```
function updateTransition() {
  var el = document.querySelector("div.box");

  if (el) {
    el.className = "box1";
  } else {
    el = document.querySelector("div.box1");
    el.className = "box";
  }

  return el;
}

var intervalID = window.setInterval(updateTransition, 7000);
```

See also

- Using CSS transitions
- `TransitionEvent`

translate

A propriedade **`translate`** CSS do CSS, permite que você especifique transformações `translate` individualmente e independentemente da propriedade `transform`. É mais adequado para o uso de interface de usuário, e nos poupa quando se trata de lembrar a ordem exata que devemos especificar nos valores das funções de `transform`.

Syntax

```
/* Propriedades principais */
translate: none;

/* Valores únicos */
translate: 100px;
translate: 50%;

/* Dois valores */
translate: 100px 200px;
translate: 50% 105px;

/* Três valores */
translate: 50% 105px 5rem;

/* Valores Globais */
translate: inherit;
translate: initial;
translate: revert;
translate: revert-layer;
translate: unset;
```

Valores

Único `<length-percentage>` valor

> Um `<length>` ou `<percentage>` que especifica o deslocamento `translate` ao longo do eixo-X. Equivalente a função `translate()` (2D translation) com um único valor especificado.

Dois `<length-percentage>` valores

> Dois `<length>` ou `<percentage>` que especifica os deslocametos `translate` ao longo dos eixos X e Y (respectivamente) de uma 2D translation. Equivalente a função `translate()` (2D translation) com dois valores especificados.

Três valores

Dois `<length-percentage>` e um único `<length>` valores que especifica os deslocamentos dos eixos X, Y, e Z (respectivamente) de uma 3D translation. Equivalente a função `translate3d()` (3D translation).

`none`

Especifica que nenhum deslocamento deva ser aplicado.

Definição formal

Initial value	none
Aplica-se a	qualquer elemento transformavel
Inherited	não
Percentages	refer to the size of bounding box
Computed value	as specified, but with relative lengths converted into absolute lengths
Animation type	a transform
Creates stacking context	yes

Sintaxe formal

```
translate =
  none                                              |
  <length-percentage> [ <length-percentage> <length>? ]?

<length-percentage> =
  <length>      |
  <percentage>
```

Esta sintaxe reflete o padrão mais recente conforme CSS Transforms Module Level 2, CSS Values and Units Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

HTML

```
<div>
  <p class="translate">Translation</p>
</div>
```

CSS

```
* {
  box-sizing: border-box;
}

html {
  font-family: sans-serif;
}

div {
  width: 150px;
  margin: 0 auto;
}

p {
  padding: 10px 5px;
  border: 3px solid black;
  border-radius: 20px;
  width: 150px;
  font-size: 1.2rem;
  text-align: center;
}

.translate {
  transition: translate 1s;
}

div:hover .translate {
```

```
  translate: 200px 50px;
}
```

Resultado

Veja também

- `scale`
- `rotate`
- `transform`

Nota: `skew` não é um valor independente de `transform`

vertical-align

A propriedade **`vertical-align`** do CSS define o alinhamento vertical de uma caixa inline, inline-block ou table-cell.

Experimente

```
vertical-align: baseline;
```

```
vertical-align: top;
```

```
vertical-align: middle;
```

```
vertical-align: bottom;
```

```
vertical-align: sub;
```

```
vertical-align: text-top;
```

```
<section class="default-example" id="default-example">
  <p>
    Align the star:
    <img id="example-element" src="/shared-assets/images/examples/star2.png" />
  </p>
</section>
```

```
#default-example > p {
  line-height: 3em;
  font-family: monospace;
  font-size: 1.2em;
  text-decoration: underline overline;
}
```

A propriedade vertical-align pode ser usada em dois contextos:

- Para alinhar verticalmente a caixa de um elemento inline dentro da caixa de linha que a contém. Por exemplo, pode ser usado para posicionar verticalmente uma imagem em uma linha de texto.
- Para alinhar verticalmente o conteúdo de uma célula em uma tabela.

Observe que `vertical-align` só se aplica a elementos inline, inline-block e table-cell: você não pode usá-lo para alinhar verticalmente elementos em nível de bloco.

Sintaxe

```
/* Valores de palavra-chave */
vertical-align: baseline;
vertical-align: sub;
vertical-align: super;
vertical-align: text-top;
vertical-align: text-bottom;
vertical-align: middle;
vertical-align: top;
vertical-align: bottom;

/* valores <comprimento> */
vertical-align: 10em;
```

```
vertical-align: 4px;

/* valores <porcentagem> */
vertical-align: 20%;

/* Valores globais */
vertical-align: inherit;
vertical-align: initial;
vertical-align: revert;
vertical-align: revert-layer;
vertical-align: unset;
```

A propriedade `vertical-align` é especificada como um dos valores listados abaixo.

Valores para elementos inline

Valores relativos aos pais

Esses valores alinham verticalmente o elemento em relação ao seu elemento pai:

`baseline`

: Alinha a linha de base do elemento com a linha de base de seu pai. A linha de base de alguns elementos substituídos, como `<textarea>`, não é especificada pela especificação HTML, o que significa que seu comportamento com esta palavra-chave pode variar entre os navegadores.

`sub`

: Alinha a linha de base do elemento com a linha de base subscrita de seu pai.

`super`

: Alinha a linha de base do elemento com a linha de base sobrescrita de seu pai.

`text-top`

: Alinha a parte superior do elemento com a parte superior da fonte do elemento pai.

`text-bottom`

: Alinha a parte inferior do elemento com a parte inferior da fonte do elemento pai.

`middle`

: Alinha o meio do elemento com a linha de base mais metade da altura x do pai.

`<length>`

: Alinha a linha de base do elemento ao comprimento especificado acima da linha de base de seu pai. Um valor negativo é permitido.

`<percentage>`

: Alinha a linha de base do elemento à porcentagem fornecida acima da linha de base de seu pai, com o valor sendo uma porcentagem da propriedade `line-height`. Um valor negativo é permitido.

Valores relativos à linha

Os seguintes valores alinham verticalmente o elemento em relação à linha inteira:

`top`

: Alinha o topo do elemento e seus descendentes com o topo de toda a linha.

`bottom`

: Alinha a parte inferior do elemento e seus descendentes com a parte inferior de toda a linha.

Para elementos que não possuem uma linha de base, a borda da margem inferior é usada.

Valores para células da tabela

`baseline` (e `sub`, `super`, `text-top`, `text-bottom`, `<length>` e `<percentage>`)

: Alinha a linha de base da célula com a linha de base de todas as outras células na linha que estão alinhadas com a linha de base.

`top`

Alinha a borda de preenchimento superior da célula com a parte superior da linha.

`middle`

Centraliza a caixa de preenchimento da célula dentro da linha.

`bottom`

Alinha a borda de preenchimento inferior da célula com a parte inferior da linha.

Valores negativos são permitidos.

Definição formal

Initial value	`baseline`
Aplica-se a	as each of the properties of the shorthand:. It also applies to `::first-letter` and `::first-line`.
Inherited	não
Computed value	as each of the properties of the shorthand:
Animation type	discrete

Sintaxe formal

```
vertical-align =
  [ first | last ]        ||
  <'alignment-baseline'>  ||
  <'baseline-shift'>

<alignment-baseline> =
  baseline           |
  <baseline-metric>

<baseline-shift> =
  <length-percentage>  |
  sub                  |
  super                |
  top                  |
  center               |
  bottom

<baseline-metric> =
  text-bottom   |
  alphabetic    |
  ideographic   |
  middle        |
  central       |
  mathematical  |
  hanging       |
  text-top

<length-percentage> =
  <length>      |
  <percentage>
```

Esta sintaxe reflete o padrão mais recente conforme CSS Inline Layout Module Level 3, CSS Values and Units Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

Exemplo básico

HTML

```
<div>
  Uma imagem <img src="frame_image.svg" alt="link" width="32" height="32" /> com
  um alinhamento padrão.
</div>
<div>
  Uma imagem
  <img class="top" src="frame_image.svg" alt="link" width="32" height="32" />
  com um alinhamento de texto superior.
```

```
</div>
<div>
  Uma imagem
  <img class="bottom" src="frame_image.svg" alt="link" width="32" height="32" />
  com um alinhamento de texto inferior.
</div>
<div>
  Uma imagem
  <img class="middle" src="frame_image.svg" alt="link" width="32" height="32" />
  com um alinhamento médio.
</div>
```

CSS

```
img.top {
  vertical-align: text-top;
}
img.bottom {
  vertical-align: text-bottom;
}
img.middle {
  vertical-align: middle;
}
```

Resultado

Alinhamento vertical em uma caixa de linha

HTML

```
<p>
  top: <img style="vertical-align: top" src="star.png" /><br />
  middle: <img style="vertical-align: middle" src="star.png" /><br />
  bottom: <img style="vertical-align: bottom" src="star.png" /><br />
  super: <img style="vertical-align: super" src="star.png" /><br />
  sub: <img style="vertical-align: sub" src="star.png" /><br />
</p>

<p>
  text-top: <img style="vertical-align: text-top" src="star.png" /><br />
  text-bottom: <img style="vertical-align: text-bottom" src="star.png" /><br />
  0.2em: <img style="vertical-align: 0.2em" src="star.png" /><br />
  -1em: <img style="vertical-align: -1em" src="star.png" /><br />
  20%: <img style="vertical-align: 20%" src="star.png" /><br />
  -100%: <img style="vertical-align: -100%" src="star.png" /><br />
</p>
```

```
#* {
  box-sizing: border-box;
}

img {
  margin-right: 0.5em;
}

p {
  height: 3em;
  padding: 0 0.5em;
  font-family: monospace;
  text-decoration: underline overline;
  margin-left: auto;
  margin-right: auto;
  width: 80%;
}
```

Resultado

Alinhamento vertical em uma célula da tabela

HTML

```
<table>
  <tr>
    <td style="vertical-align: baseline">baseline</td>
    <td style="vertical-align: top">top</td>
    <td style="vertical-align: middle">middle</td>
    <td style="vertical-align: bottom">bottom</td>
    <td>
      <p>
        Existe uma teoria que afirma que se alguém descobrir exatamente para que
        serve o Universo e por que ele está aqui, ele desaparecerá
        instantaneamente e será substituído por algo ainda mais bizarro e
        inexplicável.
      </p>
      <p>Há outra teoria que afirma que isso já aconteceu.</p>
    </td>
  </tr>
</table>
```

CSS

```
table {
  margin-left: auto;
  margin-right: auto;
  width: 80%;
}

table,
th,
td {
  border: 1px solid black;
}

td {
  padding: 0.5em;
  font-family: monospace;
}
```

Resultado

Veja também

- Casos de uso típicos do Flexbox, seção "Item central"
- `line-height`, `text-align`, `margin`
- Entendendo `vertical-align`, ou "Como (não) centralizar o conteúdo verticalmente"
- Vertical-Align: Tudo o que você precisa saber

visibility

A propriedade CSS de visibilidade pode mostrar ou ocultar um elemento sem afetar o layout de um documento (ou seja, o espaço é criado para elementos independentemente de serem visíveis ou não). A propriedade também pode ocultar linhas ou colunas em um `<table>`.

```
/* palavra-chave valores */
visibility: visible;
visibility: hidden;
visibility: collapse;

/* Global values */
visibility: inherit;
visibility: initial;
visibility: unset;
```

> **Nota:** Para ocultar um elemento ou removê-lo do layout do documento, defina a propriedade `display` como `none` em vez de usar `visibility`.

Initial value	`visible`

Aplica-se a	all elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Sintaxe

A propriedade é especificada como um dos valores das palavras-chave listados abaixo.

Valores

`visible`

The element box is visible.

`hidden`

The element box is invisible (not drawn), but still affects layout as normal. Descendants of the element will be visible if they have `visibility` set to `visible`. The element cannot receive focus (such as when navigating through tab indexes).

`collapse`

The `collapse` keyword has different effects for different elements:

- For `<table>` rows, columns, column groups, and row groups, the row(s) or column(s) are hidden and the space they would have occupied is removed (as if `display: none` were applied to the column/row of the table). However, the size of other rows and columns is still calculated as though the cells in the collapsed row(s) or column(s) are present. This value allows for the fast removal of a row or column from a table without forcing the recalculation of widths and heights for the entire table.
- Collapsed flex items and ruby annotations are hidden, and the space they would have occupied is removed.
- For other elements, `collapse` is treated the same as `hidden`.

Formal syntax

```
visibility =
  visible       |
  hidden        |
  force-hidden  |
  collapse
```

Esta sintaxe reflete o padrão mais recente conforme CSS Display Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Interpolação

Visibility values are interpolable between *visible* and *not-visible*. One of the start or ending values must therefore be `visible` or no interpolation can happen. The value is interpolated as a discrete step, where values of the timing function between `0` and `1` map to `visible` and other values of the timing function (which occur only at the start/end of the transition or as a result of `cubic-bezier()` functions with y values outside of [0, 1]) map to the closer endpoint.

Exemplos

Exemplo básico

HTML

```
<p class="visible">The first paragraph is visible.</p>
<p class="not-visible">The second paragraph is NOT visible.</p>
<p class="visible">
  The third paragraph is visible. Notice the second paragraph is still occupying
  space.
</p>
```

CSS

```
.visible {
  visibility: visible;
}

.not-visible {
```

```
  visibility: hidden;
}
```

Table example

HTML

```
<table>
  <tr>
    <td>1.1</td>
    <td class="collapse">1.2</td>
    <td>1.3</td>
  </tr>
  <tr class="collapse">
    <td>2.1</td>
    <td>2.2</td>
    <td>2.3</td>
  </tr>
  <tr>
    <td>3.1</td>
    <td>3.2</td>
    <td>3.3</td>
  </tr>
</table>
```

CSS

```
.collapse {
  visibility: collapse;
}

table {
  border: 1px solid red;
}

td {
  border: 1px solid gray;
}
```

Notas

- Support for `visibility: collapse` is missing or partially incorrect in some modern browsers. It may not be correctly treated like `visibility: hidden` on elements other than table rows and columns.
- `visibility: collapse` may change the layout of a table if the table has nested tables within the cells that are collapsed, unless `visibility: visible` is specified explicitly on nested tables.

white-space

Resumo

A propriedade CSS white-space é usada para descrever como os espaços em branco dentro dos elementos são manipulados.

Initial value	`normal`
Aplica-se a	all elements
Inherited	yes
Computed value	as specified
Animation type	discrete

Sintaxe

```
Formal syntax:

  white-space =
    normal                                          |
    pre                                             |
    pre-wrap                                        |
    pre-line                                        |
    <'white-space-collapse'> || <'text-wrap-mode'> || <'white-space-trim'>
```

```
<white-space-collapse> =
  collapse          |
  discard           |
  preserve          |
  preserve-breaks   |
  preserve-spaces   |
  break-spaces

<text-wrap-mode> =
  wrap    |
  nowrap

<white-space-trim> =
  none                                          |
  discard-before || discard-after || discard-inner
```

Esta sintaxe reflete o padrão mais recente conforme CSS Text Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

```
white-space: normal
white-space: nowrap
white-space: pre
white-space: pre-wrap
white-space: pre-line

white-space: inherit
```

Valores

`normal`

Sequências de espaços em branco são recolhidas. Caracteres de nova linha no código são tratados como outros espaços em branco. As linhas quebram conforme a necessidade para preencher as linhas das caixas (boxes).

`nowrap`

Recolhe os espaços em branco como no normal, mas suprime as quebras de linha (quebra de texto) no texto.

`pre`

Sequências de espaço em branco são preservadas, linhas são quebradas apenas em carateres de nova linha no código e em elementos `<br>`.

`pre-wrap`

Sequências de espaço em branco são preservadas. Linhas são quebradas em caracteres de nova linha, em `<br>` e quando necessário para preencher as linhas das caixas (boxes).

`pre-line`

Sequências de espaços em branco são recolhidas. Linhas são quebradas em caracteres de nova linha, em `<br>` e quando necessário para preencher as linhas das caixas (boxes).

A tabela a seguir resume o comportamento dos valores para white-space:

	Novas linhas	**Espaços e Tabulações (tabs)**	**Quebra de texto**
`normal`	Recolhido	Recolhido	Quebra
`nowrap`	Recolhido	Recolhido	Sem quebra
`pre`	Preservado	Preservado	Sem quebra
`pre-wrap`	Preservado	Preservado	Quebra
`pre-line`	Preservado	Recolhido	Quebra

Exemplos

```
code {
  white-space: pre;
}
```

[1] Internet Explorer 5.5+ suporta `word-wrap: break-word;` O código a seguir permite quebras de linhas dentro de elementos pre:

```
pre {
  word-wrap: break-word; /* IE 5.5-7 */
  white-space: -moz-pre-wrap; /* Firefox 1.0-2.0 */
  white-space: pre-wrap; /* current browsers */
}
```

width

Resumo

A propriedade CSS **width** determina a largura da área de conteúdo de um elemento. A área de conteúdo fica dentro do preenchimento, da borda, e da margem de um elemento.

As propriedades `min-width` e `max-width` sobrescrevem o `width`.

Initial value	`auto`
Aplica-se a	all elements but non-replaced inline elements, table rows, and row groups
Inherited	não
Percentages	refer to the width of the containing block
Computed value	a percentage or `auto` or the absolute length
Animation type	a length, percentage or calc();

Sintaxe

```
/* Valores de largura - <length> */
width: 300px;
width: 25em;

/* Valores percentuais - <percentage> */
width: 75%;

/* Valores com palavras-chave */
width: 25em border-box;
width: 75% content-box;
width: max-content;
width: min-content;
width: available;
width: fit-content;
width: auto;

/* Valores Globais */
width: inherit;
width: initial;
width: unset;
```

Valores

`<length>`

Veja `<length>` para possíveis unidades.

`<percentage>`

Especificado como `<percentage>` da largura do bloco contido. Se a largura do bloco contido depender da largura do elemento, o layout resultante é indefinido.

`border-box`

Se presente, o precedente `<length>` ou `<percentage>` é aplicado para o border box do elemento.

`content-box`

Se presente, o precedente `<length>` ou `<percentage>` é aplicado para o content box do elemento.

`auto`

O navegador irá calcular e selecionar a largura para o elemento específicado.

fill

Use o fill-available inline size ou fill-available block size, como um modo apropriado de escrita.

`max-content`

Da largura interna preferível.

`min-content`

Da largura interna mínina.

`available`

Do bloco contendo a largura menos a margin horizontal, borda ou preenchimento.

`fit-content`

A largura:

- do comprimento interno mínimo.
- do menor comprimento interno preferível e da largura disponível.

Sintaxe Formal

```
width =
  auto             |
  <box-size>       |
  <anchor-size()>

<box-size> =
  <length-percentage [0,∞]>                |
  stretch                                  |
  contain                                  |
  min-content                              |
  max-content                              |
  fit-content                              |
  stretch( <length-percentage [0,∞]> )     |
  fit-content( <length-percentage [0,∞]> ) |
  <calc-size()>

<anchor-size()> =
  anchor-size( [ <anchor-name> || <anchor-size> ]? , <length-percentage>? )

<length-percentage> =
  <length>      |
  <percentage>

<calc-size()> =
  calc-size( <calc-size-basis> , <calc-sum> )

<anchor-name> =
  <dashed-ident>

<anchor-size> =
  width        |
  height       |
  block        |
  inline       |
  self-block   |
  self-inline

<calc-size-basis> =
  <size-keyword>  |
  <calc-sum>      |
  <calc-size()>   |
  any

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*
```

```
<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

Esta sintaxe reflete o padrão mais recente conforme CSS Anchor Positioning Module Level 1, CSS Box Sizing Module Level 3, CSS Box Sizing Module Level 4, CSS Values and Units Module Level 4, CSS Values and Units Module Level 5. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

Largura padrão

```
p.douradinho {
  background: gold;
}
```

```
<p class="douradinho">
  A comunidade Mozilla produz diversos softwares incríveis.
</p>
```

Pixels e ems

```
.largura_px {
  width: 200px;
  background-color: red;
  color: white;
  border: 1px solid black;
}

.largura_em {
  width: 20em;
  background-color: white;
  color: red;
  border: 1px solid black;
}
```

```
<div class="largura_px">Largura medida com px</div>
<div class="largura_em">Largura medida com em</div>
```

Porcentagem

```
.porcentagem {
  width: 20%;
  background-color: silver;
  border: 1px solid red;
}
```

```
<div class="porcentagem">Largura em porcentagem</div>
```

Exemplo utilizando "max-content"

```
p.maxgreen {
  background: lightgreen;
  width: intrinsic; /* Safari/WebKit uses a non-standard name */
  width: -moz-max-content; /* Firefox/Gecko */
  width: -webkit-max-content; /* Chrome */
}
```

```
<p class="maxgreen">
  A comunidade Mozilla produz diversos softwares incríveis.
```

```
</p>
```

Exemplo utilizando "min-content"

```
p.minblue {
  background: lightblue;
  width: -moz-min-content; /* Firefox */
  width: -webkit-min-content; /* Chrome */
}
```

```
<p class="minblue">A comunidade Mozilla produz diversos softwares incríveis.</p>
```

Veja também

- box model, `height`, `box-sizing`, `min-width`, `max-width`

will-change

Sumário

A propriedade **`will-change`** do CSS fornece um modo dos autores dizerem aos navegadores sobre que tipo de mudanças devem ser esperadas no elemento, então o navegador pode fazer as otimizações apropriadas com antecedência, antes do elemento ser efetivamente modificado.

Esses tipos de otimizações podem aumentar a capacidade de resposta de uma página, fazendo o trabalho potencialmente caro antes do tempo antes que eles são realmente necessários.

O uso adequado dessa propriedade pode ser um pouco complicado:

- *Não aplique will-change em muitos elementos.* O navegador já tenta ao máximo otimizar tudo. Algumas das otimizações mais fortes que podem utilizar o `will-change` acabam usando muitos recursos da máquina, e quando utilizados em demasia como neste caso, pode fazer a página para retardar o carregamento ou consumir uma grande quantidade de recursos.
- *Use com moderação.* O comportamento normal do navegador para otimizações é fazê-las e depois voltar ao normal. Mas adicionando `will-change` diretamente no estilo, implica que o elemento alvo está sempre alguns momentos antes da mudança, e o navegador irá manter as otimizações por muito mais tempo do que teria de outra forma. Então é uma boa prática colocar e retirar o `will-change` usando script antes e depois da mudança ocorrer.
- *Não aplicar will-change em elementos para realizar a otimização prematura.* Se sua página está performando bem, não adicione a propriedade `will-change` nos elementos apenas para trazer mais velocidade. `will-change` é entendido como algo para ser usado como último recurso, com o intuito de tentar solucionar problemas de performance . Ele não deve ser usado para antecipar esses possíveis problemas. O uso excessivo do `will-change` poderá resultar em excessivo uso de memória e causar uma renderização mais complexa enquanto o browser tenta se preparar para uma possível mudança. Isso levará a um pior desempenho.
- *Dê tempo suficiente para trabalhar.* Essa propriedade destina-se como um método para os autores dizerem ao agente do browser sobre como mudarão as properiedades antes disso acontecer. Então o navegador pode escolher como aplicar qualquer otimização requisitada na propriedade antes do tempo da ação acontecer. Isso é importante para dar ao navegador algum tempo para fazer as otimizações. Identifique um jeito de prever as mudanças, e use a proiedade `will-change` pra isso.

Initial value	`auto`
Aplica-se a	all elements
Inherited	não
Computed value	as specified
Animation type	discrete

Sintaxe

```
/* Valores chave */
will-change: auto;
will-change: scroll-position;
will-change: contents;
will-change: transform; /* Exemplo de <custom-ident> */
will-change: opacity; /* Exemplo de <custom-ident> */
will-change: left, top; /* Exemplo de dois <animateable-feature> */

/* Valores globais */
will-change: inherit;
```

```
will-change: initial;
will-change: unset;
```

Valores

`auto`

Essa palavra chave não expressa nenhuma intenção em particular; o *user agent* deve aplicar qualquer otimização como normalmente ocorre.

O `<animateable-feature>` pode ser um dos seguintes valores:

`scroll-position`

Indica que o autor espera que haja uma animação ou mudança na posição do scroll do elemento no futuro.

`contents`

Indica que o autor espera que haja uma animação ou mudança no conteúdo do elemento no futuro.

`<custom-ident>`

Indica que o autor espera que haja uma animação ou mudança na propriedade com o nome dado no elemento no futuro. Isso não pode ser um dos seguintes valores: `unset`, `initial`, `inherit`, `will-change`, `auto`, `scroll-position`, or `contents`. A especificação não define o comportamento de um valor em particular, mas é comum pro `transform` ser uma camada de composição. O Chrome atualmente toma duas ações, dadas as propriedades particulares do CSS: estabelece uma nova camada de composição ou um novo stacking context.

Sintaxe formal

```
will-change =
  auto                    |
  <animateable-feature>#

<animateable-feature> =
  scroll-position  |
  contents         |
  <custom-ident>
```

Esta sintaxe reflete o padrão mais recente conforme CSS Will Change Module Level 1. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

```
.sidebar {
  will-change: transform;
}
```

O exemplo acima adiciona a propriedade `will-change` diretamente no estilo, o que irá fazer com que o navegador mantenha a otimização em memória por muito mais tempo que o necessário e nós já vimos que isso deve ser evitado. Abaixo outro exemplo mostrando como aplicar o `will-change` através de script, o que provavelmente é o que você deve fazer na maioria dos casos.

```
var el = document.getElementById("element");

// Set will-change quando ocorre hover no elemento
el.addEventListener("mouseenter", hintBrowser);
el.addEventListener("animationEnd", removeHint);

function hintBrowser() {
  // As propriedades otimizáveis que vão mudar no bloco de animações
  this.style.willChange = "transform, opacity";
}

function removeHint() {
  this.style.willChange = "auto";
}
```

Entretanto pode ser apropriado incluir `will-change` no seu estilo css para uma aplicação que faz *page flips* quando uma tecla é pressionada como um álbum ou uma apresentação de slides, onde a página é grande e complexa. Isso irá dizer ao navegador para preparar a transição à frente do tempo e aceitar transições entre as páginas assim que a tecla for pressionada.

```
.slide {
  will-change: transform;
}
```

word-break

A propriedade CSS **`word-break`** é usada para especificar se o navegador deve inserir ou não quebras de linha onde, normalmente, o texto vazaria de seu container.

Experimente

```
word-break: normal;
```

```
word-break: break-all;
```

```
word-break: keep-all;
```

```
word-break: break-word;
```

```
<section class="default-example" id="default-example">
  <div class="transition-all" id="example-element">
    Honorificabilitudinitatibus califragilisticexpialidocious
    Taumatawhakatangihangakoauauotamateaturipukakapikimaungahoronukupokaiwhenuakitanatahu
    グレートブリテンおよび北アイルランド連合王国という言葉は本当に長い言葉
  </div>
</section>
```

```
#example-element {
  width: 80%;
  padding: 20px;
  text-align: start;
  border: solid 1px darkgray;
}
```

> **Nota:** comparando com `overflow-wrap`, `word-break` criará uma quebra de linha no ponto exato em que o texto vazaria, mesmo que uma palavra pudesse ser colocada por completo em uma nova linha sem a necessidade de quebra da palavra.

Sintaxe

```
/* Valores específicos */
word-break: normal;
word-break: break-all;
word-break: keep-all;

/* Valores globais */
word-break: inherit;
word-break: initial;
word-break: unset;
```

Valores

`normal`

Usa a regra de quebra de linha padrão.

`break-all`

Quebras de linha podem ser inseridas entre quaisquer caracteres de texto não-CJC (Chinês/Japonês/Coreano).

`keep-all`

Não permite quebra de linha para texto CJC. Texto não-CJC se comporta como `normal`.

`break-word`

Para evitar o vazamento, palavras que normalmente não seriam quebradas podem ser quebradas em pontos arbitrários se não houver pontos de quebra válidos na linha.

Sintaxe formal

```
word-break =
  normal       |
  break-all    |
  keep-all     |
  manual       |
  auto-phrase  |
  break-word
```

Esta sintaxe reflete o padrão mais recente conforme CSS Text Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

Conteúdo HTML

```
<p>1. <code>word-break: normal</code></p>
<p class="normal narrow">
  This is a long and Honorificabilitudinitatibus califragilisticexpialidocious
  Taumatawhakatangihangakoauauotamateaturipukakapikimaungahoronukupokaiwhenuakitanatahu
  次の単語グレートブリテンおよび北アイルランド連合王国で本当に大きな言葉
</p>

<p>2. <code>word-break: break-all</code></p>
<p class="breakAll narrow">
  This is a long and Honorificabilitudinitatibus califragilisticexpialidocious
  Taumatawhakatangihangakoauauotamateaturipukakapikimaungahoronukupokaiwhenuakitanatahu
  次の単語グレートブリテンおよび北アイルランド連合王国で本当に大きな言葉
</p>

<p>3. <code>word-break: keep-all</code></p>
<p class="keepAll narrow">
  This is a long and Honorificabilitudinitatibus califragilisticexpialidocious
  Taumatawhakatangihangakoauauotamateaturipukakapikimaungahoronukupokaiwhenuakitanatahu
  次の単語グレートブリテンおよび北アイルランド連合王国で本当に大きな言葉
</p>

<p>4. <code>word-break: break-word</code></p>
<p class="breakWord narrow">
  This is a long and Honorificabilitudinitatibus califragilisticexpialidocious
  Taumatawhakatangihangakoauauotamateaturipukakapikimaungahoronukupokaiwhenuakitanatahu
  次の単語グレートブリテンおよび北アイルランド連合王国で本当に大きな言葉
</p>
```

Conteúdo CSS

```
.narrow {
  padding: 5px;
  border: 1px solid;
  display: table;
  max-width: 100%;
}

.normal {
  word-break: normal;
}

.breakAll {
  word-break: break-all;
}

.keepAll {
  word-break: keep-all;
}

.breakWord {
  word-break: break-word;
}
```

Veja também

- `word-wrap`

writing-mode

A propriedade CSS **`writing-mode`** define se as linhas do texto são dispostas horizontalmente ou verticalmente e a direção que os blocos seguem

```
/* Valores das palavras-chave */
writing-mode: horizontal-tb;
writing-mode: vertical-rl;
writing-mode: vertical-lr;

/* Valores globais */
writing-mode: inherit;
writing-mode: initial;
writing-mode: unset;
```

Sintaxe

A propriedade especifica o *comportamento de fluxo do bloco,* que é a maneira em que os containers são empilhados ou distribuídos, e a direção em que o conteudo é alinhado ou disposto dentro dos containers. Assim a propriedade `writing-mode` também determina a distribuição do conteúdo no nível de bloco

Initial value	`horizontal-tb`
Aplica-se a	all elements except table row groups, table column groups, table rows, and table columns
Inherited	yes
Computed value	as specified
Animation type	Not animatable

Values

`horizontal-tb`

> Conteúdo é disposto horizontalmente da esquerda para a direita, verticalmente de cima para baixo. A próxima linha horizontal é posicionada abaixo da linha anterior.

`vertical-rl`

> Conteúdo é disposto verticalmente de cima para baixo, horizontalmente da direita para a esquerda. A proxima linha vertical é posicionada à esquerda da linha anterior.

`vertical-lr`

> Conteúdo é disposto verticalmente de cima para baixo, horizontalmente da esquerda para a direita. A proxima linha vertical é posicionada à direita da linha anterior.

`sideways-rl`

> Conteúdo é disposto verticalmente de cima para baixo e todos os glifos, até mesmo os que estão em uma escrita vertical, são colocados lado a lado para a direita.

`sideways-lr`

> Conteúdo é disposto verticalmente de cima para baixo e todos os glifos, até mesmo os que estão em uma escrita vertical, são colocados lado a lado para a esquerda.

`lr`

> Descontinuado exceto para docmentos SVG1. Para CSS, use `horizontal-tb`.

`lr-tb`

> Descontinuado exceto para docmentos SVG1. Para CSS, use `horizontal-tb`.

`rl`

> Descontinuado exceto para docmentos SVG1. Para CSS, use `horizontal-tb`.

`tb`

Descontinuado exceto para docmentos SVG1. Para CSS, use `vertical-lr`.

`tb-rl`

Descontinuado exceto para docmentos SVG1. Para CSS, use `vertical-rl`.

Sintaxe formal

```
writing-mode =
  horizontal-tb  |
  vertical-rl    |
  vertical-lr    |
  sideways-rl    |
  sideways-lr
```

Esta sintaxe reflete o padrão mais recente conforme CSS Writing Modes Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplo

Esse exemplo demonstra todos os modos de escrita, mostrando cada um com texto e várias línguas.

HTML

O HTML é simplesmente um `<table>` com cada modo de escrita em uma linha com uma coluna mostrando texto em diversos scripts usando esse modo de escrita.

```
<table>
  <tr>
    <th>value</th>
    <th>Vertical script</th>
    <th>Horizontal script</th>
    <th>Mixed script</th>
  </tr>
  <tr>
    <td>horizontal-tb</td>
    <td class="example Text1"><span>我家没有电脑。</span></td>
    <td class="example Text1"><span>Example text</span></td>
    <td class="example Text1"><span>1994年に至っては</span></td>
  </tr>
  <tr>
    <td>vertical-lr</td>
    <td class="example Text2"><span>我家没有电脑。</span></td>
    <td class="example Text2"><span>Example text</span></td>
    <td class="example Text2"><span>1994年に至っては</span></td>
  </tr>
  <tr>
    <td>vertical-rl</td>
    <td class="example Text3"><span>我家没有电脑。</span></td>
    <td class="example Text3"><span>Example text</span></td>
    <td class="example Text3"><span>1994年に至っては</span></td>
  </tr>
  <tr>
    <td>sideways-lr</td>
    <td class="example Text4"><span>我家没有电脑。</span></td>
    <td class="example Text4"><span>Example text</span></td>
    <td class="example Text4"><span>1994年に至っては</span></td>
  </tr>
  <tr>
    <td>sideways-rl</td>
    <td class="example Text5"><span>我家没有电脑。</span></td>
    <td class="example Text5"><span>Example text</span></td>
    <td class="example Text5"><span>1994年に至っては</span></td>
  </tr>
</table>
```

CSS

```
table {
  border-collapse: collapse;
```

```
}
td,
th {
  border: 1px black solid;
  padding: 3px;
}
th {
  background-color: lightgray;
}
.example {
  height: 75px;
  width: 75px;
}
```

O CSS que ajusta a direcionalidade do conteúdo é algo como esse:

```
.example.Text1 span,
.example.Text1 {
  writing-mode: horizontal-tb;
  -webkit-writing-mode: horizontal-tb;
  -ms-writing-mode: horizontal-tb;
}

.example.Text2 span,
.example.Text2 {
  writing-mode: vertical-lr;
  -webkit-writing-mode: vertical-lr;
  -ms-writing-mode: vertical-lr;
}

.example.Text3 span,
.example.Text3 {
  writing-mode: vertical-rl;
  -webkit-writing-mode: vertical-rl;
  -ms-writing-mode: vertical-rl;
}

.example.Text4 span,
.example.Text4 {
  writing-mode: sideways-lr;
  -webkit-writing-mode: sideways-lr;
  -ms-writing-mode: sideways-lr;
}

.example.Text5 span,
.example.Text5 {
  writing-mode: sideways-rl;
  -webkit-writing-mode: sideways-rl;
  -ms-writing-mode: sideways-rl;
}
```

Resultado atual

Essa é uma imagem mostrando o que o resultado deve parecer, no caso do suporte do seu navegador para `writing-mode` estiver incompleto.

value	Vertical script	Horizontal script	Mixed script
horizontal-tb	我家没有电脑。	Example text	1994年に至っては
vertical-lr	我家没有电脑。	Example text	1994年に至っては
vertical-rl	我家没有电脑。	Example text	1994年に至っては
sideways-lr	我家没有电脑。	Example text	1994年に至っては
sideways-rl	我家没有电脑。	Example text	1994年に至っては

See also

- SVG `writing-mode` attribute
- `direction`
- `unicode-bidi`
- `text-orientation`
- `text-combine-upright`

::-webkit-scrollbar

> **Não padrão:** Este recurso não é padronizado. Não recomendamos o uso de recursos não padronizados em produção, pois eles têm suporte limitado nos navegadores e podem mudar ou ser removidos. No entanto, podem ser uma alternativa viável em casos específicos onde não exista uma opção padrão.

O pseudo-elemento CSS `:-webkit-scrollbar` afeta o estilo de um elemento referente a barra de rolagem quando se tem por definido `overflow:scroll;`.

> **Nota:** Se o `overflow:scroll;` não estiver definido nenhuma barra de rolagem será exibida.

> **Nota:** `::-webkit-scrollbar` está disponível somente em navegadores baseados em Blink - e WebKit - (Chrome, Edge, Opera, Safari, todos os navegadores para iOS, e outros). Um método padronizado para estilização de barras de rolagem está disponível com `scrollbar-color` e `scrollbar-width`.

Seletores CSS da Barra de Rolagem

Você pode usar os seguintes pseudo-elementos para customizar diversas partes da barra de rolagem para navegadores baseados em webkit:

- `::-webkit-scrollbar` — a barra de rolagem inteira.
- `::-webkit-scrollbar-button` — os botões na barra (setas para cima e para baixo no qual rolam uma linha de cada vez).
- `::-webkit-scrollbar-thumb` — a barra arrastável de rolagem.
- `::-webkit-scrollbar-track` — o caminho (barra de progresso) da barra de rolagem onde há uma barra cinza encima de uma barra branca.
- `::-webkit-scrollbar-track-piece` — a parte do caminho (barra de progresso) não coberto por uma barra.

- `::-webkit-scrollbar-corner` — o canto inferior da barra de rolagem, onde ambas barras horizontal e vertical se encontram. Muitas das vezes o canto inferior direito da janela do navegador.
- `::-webkit-resizer` — a barra redimensionável que aparece no canto inferior de alguns elementos.

Exemplos

CSS

```
.visible-scrollbar, .invisible-scrollbar, .mostly-customized-scrollbar {
  display: block;
  width: 10em;
  overflow: auto;
  height: 2em;
}

.invisible-scrollbar::-webkit-scrollbar {
  display: none;
}

/* Demonstração de uma barra de rolagem personalizada
  * (não será visível ao menos que a largura/altura seja especificada) */
.mostly-customized-scrollbar::-webkit-scrollbar {
  width: 5px;
  height: 8px;
  background-color: #aaa; /* ou que isso seja adicionado ao "caminho" da barra */
}

/* Adiciona a barra arrastável */
.mostly-customized-scrollbar::-webkit-scrollbar-thumb {
    background: #000;
}
```

HTML

```
<div class="visible-scrollbar">
  Etiam sagittis sem sed lacus laoreet, eu fermentum eros auctor.
  Proin at nulla elementum, consectetur ex eget, commodo ante.
  Sed eros mi, bibendum ut dignissim et, maximus eget nibh. Phasellus
  blandit quam turpis, at mollis velit pretium ut. Nunc consequat
  efficitur ultrices. Nullam hendrerit posuere est. Nulla libero
  sapien, egestas ac felis porta, cursus ultricies quam. Vestibulum
  tincidunt accumsan sapien, a fringilla dui semper in. Vivamus
  consectetur ipsum a ornare blandit. Aenean tempus at lorem sit
  amet faucibus. Curabitur nibh justo, faucibus sed velit cursus,
  mattis cursus dolor. Pellentesque id pretium est. Quisque
  convallis nisi a diam malesuada mollis. Aliquam at enim ligula.
</div>

<div class="invisible-scrollbar">
Estaéumapalavramuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuitolonga
</div>

<div class="mostly-customized-scrollbar">
Estaéumapalavramuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuitolonga<br>
E bem alta<br>
com uma barra de rolagem estranha.<br>
Quem diria que barras de rolagem poderiam ser estraaaanhas?
</div>
```

Resultado

Especificações

Não é parte de nenhum padrão.

Compatibilidade entre navegadores

<code>::-webkit-scrollbar</code>

<code>::-webkit-scrollbar-button</code>

<code>::-webkit-scrollbar-thumb</code>

<code>::-webkit-scrollbar-track</code>

<code>::-webkit-scrollbar-track-piece</code>

<code>::-webkit-scrollbar-corner</code>

<code>::-webkit-resizer</code>

Veja Também

- WebKit blog em Estilizando barras de rolagem
- Testando WebKit para os estilos da barra de rolagem mencionados acima
- `scrollbar-width`
- `scrollbar-color`

::after (:after)

Em CSS, **`::after`** cria um pseudo-elemento que é o último filho do elemento selecionado. Muitas vezes é usado para adicionar e melhorar o conteúdo de um elemento como a propriedade `content`. É inline por padrão.

```
/* Adiciona uma seta após os links */
a::after {
  content: "→";
}
```

Sintaxe

```
::after {
  content: /* value */;
  /* properties */
}
```

> **Nota:** O CSS3 introduziu a notação `::after` (com dois sinais de dois pontos) para distinguir pseudo-classes dos pseudo-elementos. Os navegadores também aceitam `:after`, introduzido no CSS2.

Exemplos

Uso simples

Vamos criar duas classes: uma para parágrafos tediosos e uma para parágrafos excitantes. Podemos então marcar cada parágrafo adicionando um pseudo-elemento ao final dele.

```
<p class="boring-text">Here is some plain old boring text.</p>
<p>Here is some normal text that is neither boring nor exciting.</p>
<p class="exciting-text">
  Contributing to MDN is easy and fun. Just hit the edit button to add new live
  samples, or improve existing samples.
</p>
```

```
.exciting-text::after {
  content: "<- now this *is* exciting!";
  color: green;
}

.boring-text::after {
  content: "<- BORING!";
  color: red;
}
```

Visualização

Exemplos decorativos

Podemos estilizar textos ou imagens na propriedade `content` praticamente de qualquer forma que quisermos.

```
<span class="ribbon">Observe onde a caixa de laranja está.</span>
```

```
.ribbon {
  background-color: #5bc8f7;
}

.ribbon::after {
  content: "Observe esta caixa laranja";
  background-color: #ffba10;
  border-color: black;
  border-style: dotted;
}
```

Visualização

Dicas

O exemplo a seguir mostra o uso do `::after` pseudo-elemento em conjunto com a expressão CSS `attr()` e um atributo data personalizado `data-descr` para criar uma *dica* em forma de glossário feito em CSS puro. Verifique a visualização abaixo, ou veja este exemplo em página separada.

```
<p>
  Aqui está o exemplo ao vivo do código acima.<br />
  Temos um pouco de
  <span data-descr="collection of words and punctuation">texto</span> aqui com
  algumas
  <span data-descr="small popups which also hide again">dicas</span>.<br />
  Não seja tímido, passe o mouse por cima para dar uma
  <span data-descr="not to be taken literally">olhada</span>.
</p>
```

```
span[data-descr] {
  position: relative;
  text-decoration: underline;
  color: #00f;
  cursor: help;
}

span[data-descr]:hover::after {
  content: attr(data-descr);
  position: absolute;
  left: 0;
  top: 24px;
  min-width: 200px;
  border: 1px #aaaaaa solid;
  border-radius: 10px;
  background-color: #ffffcc;
  padding: 12px;
  color: #000000;
  font-size: 14px;
  z-index: 1;
}
```

Visualização

Veja também

- `::before`, `content`

::backdrop

> **Experimental: Esta é uma tecnologia experimental**
> Verifique a tabela de compatibilidade entre Navegadores cuidadosamente antes de usar essa funcionalidade em produção.

Cada elemento na camada superior de uma pilha de elementos possui um `pseudo-elemento` *`::backdrop`*. Esse pseudo-elemento é uma caixa renderizada imediatamente abaixo do elemento superior (e acima do elemento logo abaixo desse elemento, caso exista algum), dentro da camada superior.

> **Nota:** O pseudo-elemento `::backdrop` pode ser utilizado para criar um pano de fundo que esconde o conteúdo por trás de um elemento posicionado no topo de uma pilha de elementos. Por exemplo, para o elemento que é mostrado em tela cheia conforme descrito nessa especificação.

Ele não herda de nenhum elemento e também não é herdado. Não há restrição de propriedades que podem ser aplicadas a esse pseudo-elemento.

Veja também

- `:fullscreen`
- `<dialog>`

::before (:before)

Sumário

`::before` cria um pseudo-elemento que é o primeiro filho do elemento atingido. É frequentemente utilizado para adicionar conteúdo decorativo à um element utilizando a propriedade `content`. Este elemento é inline por padrão.

Sintaxe

```
::before {
  content: /* value */;
  /* properties */
}
```

A notação `::before` (com dois dois-pontos) foi introduzida no CSS3 afim de diferenciar pseudo-classes de pseudo-elementos. Navegadores também aceitam a notação `:before` introduzida no CSS 2.

Exemplos

Adicionando aspas

Um exemplo simples do uso de pseudo-elementos `::before` pseudo-elementos é a exibição de aspas. Aqui usamos `::before` e `::after` para inserir caracteres de aspas.

Conteúdo HTML

```
<q>Algumas citações</q>, ele disse, <q>são melhor do que nenhuma.</q>
```

Conteúdo CSS

```
q::before {
  content: "«";
  color: blue;
}
q::after {
  content: "»";
  color: red;
}
```

Saída

Exemplo decorativo

Podemos estilizar textos e imagens na propriedade `content` praticamente da forma que desejarmos.

Conteúdo HTML

```
<span class="ribbon">Observe onde a caixa laranja está.</span>
```

Conteúdo CSS

```
.ribbon {
  background-color: #5bc8f7;
```

```
}

.ribbon::before {
  content: "Olhe para esta caixa laranja.";
  background-color: #ffba10;
  border-color: black;
  border-style: dotted;
}
```

Saída

Lista To-do

Neste exemplo vamos criar uma simples lista de tarefas, usando pseudo-elementos. Este método pode frequentemente ser utilizado para dar pequenos toques à UI (User Interface) e melhorar a experiência do usuário.

Conteúdo HTML

```
<ul>
  <li>Comprar Leite</li>
  <li>Levar o cachorro para passear</li>
  <li>Exercitar-se</li>
  <li>Escrever codigo</li>
  <li>Tocar musica</li>
  <li>Relaxar</li>
</ul>
```

Conteúdo CSS

```
li {
  list-style-type: none;
  position: relative;
  margin: 2px;
  padding: 0.5em 0.5em 0.5em 2em;
  background: lightgrey;
  font-family: sans-serif;
}

li.done {
  background: #ccff99;
}

li.done::before {
  content: "";
  position: absolute;
  border-color: #009933;
  border-style: solid;
  border-width: 0 0.3em 0.25em 0;
  height: 1em;
  top: 1.3em;
  left: 0.6em;
  margin-top: -1em;
  transform: rotate(45deg);
  width: 0.5em;
}
```

Conteúdo JavaScript

```
var list = document.querySelector("ul");
list.addEventListener(
  "click",
  function (ev) {
    if (ev.target.tagName === "LI") {
      ev.target.classList.toggle("done");
    }
  },
  false,
);
```

Aqui está rodando o código acima. Perceba que nenhum ícone é utilizado e o check-mark é o `::before` que foi estilizado no CSS.

Saída

Notas

Embora as correções posicionadas e fixadas no Firefox 3.5 não permitam que o conteúdo seja gerado como um irmão anterior ao elemento (conforme a especificação para o CSS indicam "Os Pseudo-elementos :before e :after interagem com outros elementos como se fosse elementos reais inseridos dentro do elemento associado."), eles podem ser usados para proporcionar uma ligeira melhora em layouts sem tables (por exemplo, para alcançar a centralização), a medida que o conteúdo a ser centrado está envolvido com um elemento filho adicional, uma coluna antes e depois do conteúdo pode ser introduzida sem adicionar elementos filhos reais (isto é, talvez mais semanticamente correto para adicionar uma pequena distância, do que seria adicionar um div vazio antes e / ou depois). (E sempre lembre-se de adicionar uma largura a um flutuador, pois, de outra forma, não flutuará!)

HTML content

```
<div class="example">
  <span id="floatme"
    >"Floated Before" should be generated on the left of the viewport and not
    allow overflow in this line to flow under it. Likewise should "Floated
    After" appear on the right of the viewport and not allow this line to flow
    under it.</span
  >
</div>
```

CSS content

```
#floatme {
  float: left;
  width: 50%;
}

/* To get an empty column, just indicate a hex code for a non-breaking space: \a0 as the content (use \0000a0 when following such a
space with other characters) */
.example::before {
  content: "Floated Before";
  float: left;
  width: 25%;
}
.example::after {
  content: "Floated After";
  float: right;
  width: 25%;
}

/* For styling */
.example::before,
.example::after,
.first {
  background: yellow;
  color: red;
}
```

Output

See also

- `::after`, `content`

::first-letter (:first-letter)

Sumário

O pseudo-elemento CSS `::first-letter` seleciona a primeira letra da primeira linha de um bloco, desde que não seja precedida por outro conteúdo (como imagens ou tabelas embutidas/inline) na mesma linha.

A primeira letra de um elemento não é necessariamente uma identificação trivial:

- Pontuação, que é qualquer caractere definido em Unicode de abertura (Ps), de fechamento (Pe), aspa inicial (Pi), aspa final (Pf) e outras pontuações (Po), precedendo ou imediatamente após a primeira letra também é correspondida por esse pseudo-elemento.
- Da mesma forma, alguns idiomas têm dígitos que são sempre escritos em maiúsculas, como o IJ em holandês. Nesses casos raros, as duas letras do dígrafo devem corresponder ao pseudo-elemento `::first-letter`. (Isso é pouco suportado pelos navegadores, verifique a tabela

de compatibilidade).
- Finalmente, uma combinação do pseudoelemento `::before` e a propriedade `content` podem injetar algum texto no início do elemento. Nesse caso, `::first-letter` corresponderá à primeira letra deste conteúdo gerado.

Uma primeira linha tem apenas significado em uma caixa de contêiner de bloco; portanto, o pseudo-elemento `::first-letter` apenas afeta os elementos com um valor de `display` de `block`, `inline-block`, `table-cell`, `list-item` ou `table-caption`. Em todos os outros casos, `::first-letter` não tem efeito.

Somente um pequeno subconjunto de todas as propriedades CSS pode ser usado dentro de um bloco de declaração de um conjunto de regras CSS que contém um seletor usando o pseudoelemento `::first-letter`:

- Todas as propriedades de fonts : `font`, `font-style`, `font-feature-settings`, `font-kerning`, `font-language-override`, `font-stretch`, `font-synthesis`, `font-variant`, `font-variant-alternates`, `font-variant-caps`, `font-variant-east-asian`, `font-variant-ligatures`, `font-variant-numeric`, `font-variant-position`, `font-weight`, `font-size`, `font-size-adjust`, `line-height` and `font-family`.
- Todas as propriededas de background : `background-color`, `background-image`, `background-clip`, `background-origin`, `background-position`, `background-repeat`, `background-size`, `background-attachment`, and `background-blend-mode`.
- Todas as propriedades de margem: `margin`, `margin-top`, `margin-right`, `margin-bottom`, `margin-left`.
- Todas as propriedades de "padding" (preenchimento): `padding`, `padding-top`, `padding-right`, `padding-bottom`, `padding-left`.
- Todas as bordas: the taquigraficas `border`, `border-style`, `border-color`, `border-width`, `border-radius`, `border-image`, e as propriedades manuscritas.
- A propriedade `color`.
- As propriedades CSS: `text-decoration`, `text-shadow`, `text-transform`, `letter-spacing`, `word-spacing` (when appropriate), `line-height`, `text-decoration-color`, `text-decoration-line`, `text-decoration-style`, `box-shadow`, `float`, `vertical-align` (somente se `float` for `none`).

Como toda essa lista será estendida no futuro, é recomendado que você não use qualquer propriedade dentro da declaração do bloco, afim de manter o CSS a qualquer prova.

> **Nota:** No CSS 2, os pseudoelementos eram prefixados com um caractere de dois pontos (`:`). Como as pseudo-classes também estavam seguindo a mesma convenção, elas eram indistinguíveis. Para resolver isso, o CSS 2.1 mudou a convenção para pseudoelementos. Agora, um pseudoelemento é prefixado com dois caracteres de dois pontos (`::`) e uma pseudo-classe ainda é prefixado com um único dois pontos (`:`).
>
> Exemplo de **pseudoclasse**: `.classedoelemento:hover { ... }`
>
> Exemplo de **pesudoelemento:** `> .classedoelemento::first-letter { ... }`
>
> Como vários navegadores já implementaram a versão CSS 2 em uma versão de lançamento, todos os navegadores que suportam a sintaxe de dois pontos também suportam a antiga sintaxe de dois pontos.
>
> > **Aviso:** Mas isso pode mudar, portanto use SEMPRE `::` para pseudolementos
> >
> > ...a não se que você **precise muito** que seu código seja compatível com Internet Explore 8, então use um carectere de dois pontos.

Exemplo

```
/* Fazendo odas as letras dos paragrados maior e vermelha */

p::first-letter {
  /* Use :first-letter if você precise muito de compatibilidade com IE 8 */
  color: red;
  font-size: 130%;
}
```

Veja também

- O peseudo-elemento `::first-line` .

::first-line (:first-line)

O pseudoelemento **`::first-line`** aplica estilos à primeira linha de um elemento no nível do bloco. Observe que o comprimento da primeira linha depende de muitos fatores, incluindo a largura do elemento, a largura do documento e o tamanho da fonte do texto.

```
/* Seleciona a primeira linha dentro da tag <p> */
p::first-line {
```

```
  color: red;
}
```

> **Nota:** O CSS3 introduziu a notação `::first-line`(com dois pontos) para distinguir pseudo-classes de pseudo-elementos. Os navegadores também aceitam `:first-line` introduzida no CSS2.

Propriedades permitidas

Somente um pequeno subconjunto de propriedades CSS pode ser usado com o`::first-line` pseudo-elemento:

- Todas as propriedades relacionadas à fonte: `font`, `font-kerning`, `font-style`, `font-variant`, `font-variant-numeric`, `font-variant-position`, `font-variant-east-asian`, `font-variant-caps`, `font-variant-alternates`, `font-variant-ligatures`, `font-synthesis`, `font-feature-settings`, `font-language-override`, `font-weight`, `font-size`, `font-size-adjust`, `font-stretch`, e `font-family`
- Todas as propriedades relacionadas ao background: `background-color`, `background-clip`, `background-image`, `background-origin`, `background-position`, `background-repeat`, `background-size`, `background-attachment`, e `background-blend-mode`
- A propriedade `color`
- `word-spacing`, `letter-spacing`, `text-decoration`, `text-transform`, e `line-height`
- `text-shadow`, `text-decoration`, `text-decoration-color`, `text-decoration-line`, `text-decoration-style`, e `vertical-align`.

Sintaxe

```
::first-line {
  /* ... */
}
```

Exemplos

HTML

```
<p>
  Os estilos serão aplicados apenas à primeira linha deste parágrafo. Depois
  disso, todo o texto será estilizado normalmente. Entendeu o que eu quis dizer?
</p>

<span
  >A primeira linha deste texto não receberá um estilo especial porque não é um
  elemento no nível do bloco.</span
>
```

CSS

```
::first-line {
  color: blue;
  text-transform: uppercase;

  /* AVISO: NÃO UTILIZE: */
  /* Muitas propriedades são inválidas em pseudo-elementos ::first-line */
  margin-left: 20px;
  text-indent: 20px;
}
```

Resultado

Veja também

- `::first-letter`

::selection

O pseudo-elemento CSS **`::selection`** aplica estilos para partes do documento que foram selecionadas pelo usuário (tal como clicar e arrastar o ponteiro do mouse através de um texto.

```
::selection {
  background-color: cyan;
}
```

Propriedades permitidas

Apenas certas propriedades podem ser usadas com o `::selection`:

- `color`
- `background-color`
- `cursor`
- `caret-color`
- `outline` and its longhands
- `text-decoration` and its associated properties
- `text-emphasis-color`
- `text-shadow`

> **Aviso:** Em particular, `background-image` é ignorada.

Sintaxe

```
/* Sintaxe legado do Firefox (até a versão 61) */
::-moz-selection
```

```
::selection {
  /* ... */
}
```

Exemplo

HTML

```
Esse texto tem estilos especiais quando selecionado.
<p>Tente selecionando também o texto nesse parágrafo.</p>
```

CSS

```
::-moz-selection {
  color: gold;
  background-color: red;
}

p::-moz-selection {
  color: white;
  background-color: blue;
}
```

```
/* Texto selecionado dourado com o fundo vermelho */
::selection {
  color: gold;
  background-color: red;
}

/* Texto selecionado branco com o fundo azul */
p::selection {
  color: white;
  background-color: blue;
}
```

Resultado

Preocupações com acessibilidade

Não substitua os estilos de texto selecionados por razões puramente estéticas — os usuários podem personalizá-los para atender às suas necessidades. Para pessoas com problemas cognitivos ou com menos conhecimento tecnológico, alterações inesperadas nos estilos de seleção podem prejudicar sua compreensão da funcionalidade.

Se substituído, é importante garantir que a **taxa de contraste** entre o texto e as cores de plano de fundo da seleção seja alta o suficiente para que pessoas com condições de baixa visão possam lê-lo.

A taxa de contraste da cor é encontrada comparando a luminosidade do texto selecionado e as cores de fundo do texto selecionado. Para atender às atuais Diretrizes de acessibilidade de conteúdo da Web (WCAG), o conteúdo de texto deve ter uma taxa de contraste de 4.5:1 ou 3:1 para texto maior, como títulos. (WCAG define texto grande entre 18,66 px e 24 px e negrito, ou 24 px ou maior.)

Facilite para os usuários ver e ouvir o conteúdo, incluindo a separação do primeiro plano do segundo plano

- WebAIM: Verificador de Constraste de Cor
- MDN Understanding WCAG,1.4: Facilite para os usuários vizualizarem o conteúdo, incluindo a separação do primeiro plano do segundo plano (em inglês).
- Understanding Success Criterion 1.4.3 | W3C Understanding WCAG 2.0

Veja também

- `pointer-events` - permite autores controlarem sob qualquer circustancia(se houver) um elemento gráfico particular podendo ser o alvo do evento do mouse.

:active

A pseudo-classe css :active

Esse estilo pode ser substituído por qualquer outra pseudo-classe relacionada ao link, ou seja, `:link`, `:hover` e `:visited`, aparecendo nas regras subseqüentes. Para estilizar apropriadamente os links, você precisa colocar a regra `:active` depois de todas as outras regras relacionadas ao link, conforme definido pela ordem LVHA. `:link` — `:visited` — `:hover` — `:active`.

> **Nota:** Em sistemas com mouses com vários botões, o CSS 3 especifica que a pseudo-classe :active só deve ser aplicada ao botão principal em mouses destros, este é tipicamente o botão mais à esquerda.

Exemplo

HTML

```
<body>
  <h1>:active CSS selector example</h1>
  <p>
    The following link will turn lime during the time you click it and release
    the click: <a href="#">Mozilla Developer Network</a>.
  </p>
</body>
```

CSS

```
body {
  background-color: #ffffc9;
}
a:link {
  color: blue;
} /* links não visitados */
a:visited {
  color: purple;
} /* links visitados */
a:hover {
  font-weight: bold;
} /* user hovers */
a:active {
  color: lime;
} /* links ativos */
```

Veja também

- Link-related pseudo-classes: `:link`, `:visited`, and `:hover`.

:blank

> **Experimental: Esta é uma tecnologia experimental**
> Verifique a tabela de compatibilidade entre Navegadores cuidadosamente antes de usar essa funcionalidade em produção.

> **Nota:** The `:blank` selector is considered at risk, as the CSSWG keeps changing it.
>
> See CSSWG issue #1967.

A pseudo-classe CSS **`:blank`** seleciona elementos de entrada do usuário vazios (ex. `<input>` ou `<textarea>`).

Sintaxe

```
:blank {
  /* ... */
}
```

Compatibilidade com navegadores

Veja também

- `:empty`
- `:-moz-only-whitespace` - Definição anterior de `:blank`.

:checked

A pseudo-classe **`:checked`** de um seletor CSS representa um elemento **radio** (`<input type="radio">`), **checkbox** (`<input type="checkbox">`) ou **option** (`<option>` em um `<select>`) que está marcada ou alternado para um estado ligado. O usuário pode mudar seu estado clicando no elemento, ou selecionando um valor diferente, nesse caso a pseudo-classe **:checked** não se aplica a esse elemento, mas vai para um elemento relevante.

Sintaxe

```
input:checked {
  margin-left: 25px;
  border: 1px solid blue;
}
```

Sintaxe formal

```
elemento:checked { estilos de propriedades }
```

Exemplos

Exemplo de seletores

```
/* qualquer elemento "checkable" */
:checked {
  width: 50px;
  height: 50px;
}

/* qualquer elemento radio */
input[type="radio"]:checked {
  margin-left: 25px;
}

/* apenas elementos checkbox */
input[type="checkbox"]:checked {
  display: none;
}

/* apenas elementos option */
option:checked {
  color: red;
}
```

`input[type="radio"]:checked`

Representa todos os botões de rádio na página que estão ativos

`input[type="checkbox"]:checked`

Representa todos os checkboxes na página que estão ativos

`option:checked`

Representa todos os selects na página que estão selecionados

Usando checkboxes escodindos para armazenar algum valor booleano

A pseudo-classe `:checked` aplicada em checkboxes escondidos, adicionado ao início de sua página pode ser aplicada de forma a armazenar algum booleano dinâmico para ser usado por uma regra CSS. O exemplo a seguir mostra como exibir/esconder um elemento expansível com um simples clique em um botão (download esse demo).

```
#expand-btn {
  margin: 0 3px;
  display: inline-block;
  font:
    12px / 13px "Lucida Grande",
    sans-serif;
  text-shadow: rgba(255, 255, 255, 0.4) 0 1px;
  padding: 3px 6px;
  border: 1px solid rgba(0, 0, 0, 0.6);
  background-color: #969696;
  cursor: default;
  -moz-border-radius: 3px;
  -webkit-border-radius: 3px;
  border-radius: 3px;
  -moz-box-shadow:
    rgba(255, 255, 255, 0.4) 0 1px,
    inset 0 20px 20px -10px white;
  -webkit-box-shadow:
    rgba(255, 255, 255, 0.4) 0 1px,
    inset 0 20px 20px -10px white;
  box-shadow:
    rgba(255, 255, 255, 0.4) 0 1px,
    inset 0 20px 20px -10px white;
}

#isexpanded:checked ~ #expand-btn,
#isexpanded:checked ~ * #expand-btn {
  background: #b5b5b5;
  -moz-box-shadow:
    inset rgba(0, 0, 0, 0.4) 0 -5px 12px,
    inset rgba(0, 0, 0, 1) 0 1px 3px,
    rgba(255, 255, 255, 0.4) 0 1px;
  -webkit-box-shadow:
    inset rgba(0, 0, 0, 0.4) 0 -5px 12px,
    inset rgba(0, 0, 0, 1) 0 1px 3px,
    rgba(255, 255, 255, 0.4) 0 1px;
  box-shadow:
    inset rgba(0, 0, 0, 0.4) 0 -5px 12px,
    inset rgba(0, 0, 0, 1) 0 1px 3px,
    rgba(255, 255, 255, 0.4) 0 1px;
}

#isexpanded,
.expandable {
  display: none;
}

#isexpanded:checked ~ * tr.expandable {
  display: table-row;
  background: #cccccc;
}

#isexpanded:checked ~ p.expandable,
#isexpanded:checked ~ * p.expandable {
  display: block;
  background: #cccccc;
}
```

```
<input type="checkbox" id="isexpanded" />

<h1>Elementos expansíveis</h1>
<table>
```

```
    <thead>
        <tr><th>Coluna #1</th><th>Coluna #2</th><th>Coluna #3</th></tr>
    </thead>
    <tbody>
        <tr class="expandable"><td>[cell text]</td><td>[cell text]</td><td>[cell text]</td></tr>
        <tr><td>[cell text]</td><td>[cell text]</td><td>[cell text]</td></tr>
        <tr><td>[cell text]</td><td>[cell text]</td><td>[cell text]</td></tr>
        <tr class="expandable"><td>[cell text]</td><td>[cell text]</td><td>[cell text]</td></tr>
        <tr class="expandable"><td>[cell text]</td><td>[cell text]</td><td>[cell text]</td></tr>
    </tbody>
</table>

<p>[some sample text]</p>
<p><label for="isexpanded" id="expand-btn">Mostra ou esconder elementos</label></p>
<p class="expandable">[outra amostra de texto]</p>
<p>[alguma amostra de texto]</p>
</body>
</html>
```

Usando radioboxes escondidos para armazenar algum valor CSS booleano

Você também pode usar a pseudo-classe `:checked`aplicada a um radioboxe escondido afim de construir, por exemplo, uma **galeria de imagem onde as imagens apenas sejam exibidas no tamanho cheio com o clique do mouse em visualizar.** Veja essa demonstração como uma dica.

> **Nota:** Para um efeito análogo, mas baseada na pseudo-classe `:hover` sem radioboxes escondidos, veja essa demostração, tomadas a partir de uma página `:hover`.

:disabled

A pseudo-classe CSS **`:disabled`** representa qualquer elemento desativado. Um elemento é desativado se não puder ser ativado (selecionado, clicado, digitado etc.) ou aceitar o foco. O elemento também possui um estado habilitado, no qual ele pode ser ativado ou aceitar o foco.

```
/* Selects any disabled <input> */
input:disabled {
  background: #ccc;
}
```

Sintaxe

```
:disabled {
  /* ... */
}
```

Exemplo

Este exemplo mostra um formulário básico de envio. Ele usa o evento JavaScript `change` para permitir que o usuário ative / desative os campos de faturamento.

HTML

```
<form action="#">
  <fieldset id="shipping">
    <legend>Shipping address</legend>
    <input type="text" placeholder="Name" />
    <input type="text" placeholder="Address" />
    <input type="text" placeholder="Zip Code" />
  </fieldset>
  <br />
  <fieldset id="billing">
    <legend>Billing address</legend>
    <label for="billing-checkbox">Same as shipping address:</label>
    <input type="checkbox" id="billing-checkbox" checked />
    <br />
    <input type="text" placeholder="Name" disabled />
    <input type="text" placeholder="Address" disabled />
    <input type="text" placeholder="Zip Code" disabled />
  </fieldset>
</form>
```

CSS

```
input[type="text"]:disabled {
  background: #ccc;
}
```

JavaScript

```
// Wait for the page to finish loading
document.addEventListener(
  "DOMContentLoaded",
  function () {
    // Attach `change` event listener to checkbox
    document.getElementById("billing-checkbox").onchange = toggleBilling;
  },
  false,
);

function toggleBilling() {
  // Select the billing text fields
  var billingItems = document.querySelectorAll('#billing input[type="text"]');

  // Toggle the billing text fields
  for (var i = 0; i < billingItems.length; i++) {
    billingItems[i].disabled = !billingItems[i].disabled;
  }
}
```

Resultado

Veja também

- `:enabled`

:empty

A pseudo-classe CSS **`:empty`** representa qualquer elemento que não tenha filhos. Aqui, filhos podem ser outros elementos ou blocos de texto (incluindo espaços em branco). Comentários e instruções de processamento não são considerados ao se determinar se um elemento é vazio.

```
/* Seleciona qualquer <div> que não tenha conteúdo */
div:empty {
  background: lime;
}
```

Sintaxe

```
:empty {
  /* ... */
}
```

Exemplos

HTML

```
<div class="box"><!-- Eu serei verde. --></div>
<div class="box">Eu serei rosa.</div>
<div class="box">
  <!-- Eu serei rosa por causa do espaço em branco em volta desse comantário -->
</div>
```

CSS

```
body {
  display: flex;
  justify-content: space-around;
}
```

```
.box {
  background: pink;
```

```
  height: 80px;
  width: 80px;
}

.box:empty {
  background: lime;
}
```

Resultado

Questões de acessibilidade

Tecnologias assistivas como leitores de tela não conseguem identificar e processar conteúdos interativos que estejam vazios. Todo conteúdo interativo deve ter um nome acessível, que é criado ao fornecer um valor de texto para o elemento pai do controle interativo (âncoras, botões, etc.). Nomes acessíveis expõem o controle interativo à árvore de acessibilidade, uma API que comunica informações importantes úteis para tecnologias assistivas.

The text that provides the interactive control's accessible name can be hidden using a combination of properties that remove it visually from the screen but keep it parseable by assistive technology. This is commonly used for buttons that rely solely on an icon to convey purpose.

- What is an accessible name? | The Paciello Group
- Hidden content for better a11y | Go Make Things
- MDN Understanding WCAG, Guideline 2.4 explanations
- Understanding Success Criterion 2.4.4 | W3C Understanding WCAG 2.0

:enabled

A pseudo-classe CSS **`:enabled`** representa qualquer elemento ativado. Um elemento é ativado se puder ser ativado (selecionado, clicado, digitado etc.) ou aceitar o foco. O elemento também possui um estado desativado, no qual não pode ser ativado nem aceitar foco.

```
/* Seleciona qualquer <input> ativado */
input:enabled {
  color: blue;
}
```

Sintaxe

```
:enabled {
  /* ... */
}
```

Exemplo

O exemplo a seguir torna a cor do texto e do botão `<input>` verde quando ativada e cinza quando desativada. Isso ajuda o usuário a entender quais elementos podem ser interagidos..

HTML

```
<form action="url_of_form">
  <label for="FirstField">First field (enabled):</label>
  <input type="text" id="FirstField" value="Lorem" /><br />

  <label for="SecondField">Second field (disabled):</label>
  <input type="text" id="SecondField" value="Ipsum" disabled="disabled" /><br />

  <input type="button" value="Submit" />
</form>
```

CSS

```
input:enabled {
  color: #2b2;
}

input:disabled {
  color: #aaa;
}
```

Result

See also

- `:disabled`

:first-child

Sumário

A pseudo-classe de CSS **`:first-child`** representa qualquer elemento que seja o primeiro filho de seus pais.

Sintaxe

```
elemento:first-child { estilos }
```

Exemplos

Exemplo 1

HTML

```
<div>
  <span>Este span é verde limão!</span>
  <span>Este span não é verde limão. :(</span>
</div>
```

CSS

```
span:first-child {
  background-color: lime;
}
```

... resultado ...

Exemplo 2 - Usando UL

HTML

```
<ul>
  <li>Elemento de Lista 1</li>
  <li>Elemento de Lista 2</li>
  <li>Elemento de Lista 3</li>
</ul>
```

CSS

```
li {
  color: red;
}
li:first-child {
  color: green;
}
```

... resultado ...

:first-of-type

A CSS pseudo-classe **`:first-of-type`** representa o primeiro elemento de seu tipo entre os filhos de seu elemento pai.

```
/* Seleciona qualquer <p> que seja o primeiro elemento
   desse tipo entre seus irmãos */
p:first-of-type {
  color: red;
}
```

> **Nota:** Originalmente definido, o elemento selecionado tinha que ter um pai. Desde o Seletores Nível 4, isso não é mais necessário.

Sintaxe

```
:first-of-type {
  /* ... */
}
```

Exemplos

Estilizando o primeiro parágrafo

HTML

```
<h2>Título</h2>
<p>Parágrafo 1</p>
<p>Parágrafo 2</p>
```

CSS

```
p:first-of-type {
  color: red;
  font-style: italic;
}
```

Resultado

Elementos aninhados

Este exemplo mostra como os elementos aninhados também podem ser estilizados. Note que o seletor universal (*) está implícito quando nenhum seletor simples está escrito.

HTML

```
<article>
  <div>Esta `div` é a primeira!</div>
  <div>Esta <span>`span` aninhada é a primeira</span>!</div>
  <div>
    Este <em>`em` aninhado é o primeiro</em>, mas este
    <em>`em` aninhado é o último</em>!
  </div>
  <div>Este <span>`span` aninhado pegou o estilo</span>!</div>
  <b>Este `b` qualifica!</b>
  <div>Esta é a `div` final.</div>
</article>
```

CSS

```
article :first-of-type {
  background-color: pink;
}
```

Resultado

Veja também

- `:first-child`, `:last-of-type`, `:nth-of-type`

:focus

Resumo

A pseudo-classe `:focus` do CSS é aplicada quando um elemento recebe foco, o que pode ocorrer quando o usuário seleciona o elemento utilizando o teclado ou ativando o mesmo com o mouse (ex: um campo de um formulário).

A pseudo classe é aplicada apenas ao elemento focado, e não aos seus elementos ascendentes, como ocorre com `:checked` e `:enabled` e diferentemente de `:active` ou `:hover`.

Sintaxe

```
elemento:focus { ... }
```

Exemplos

```
.nome:focus {
  color: red;
}

.sobrenome:focus {
  color: lime;
}
```

```
<input class="nome" value="Esse elemento ficará vermelho quando focado" />
<input
  class="sobrenome"
  value="Esse elemento ficará verde-limão quando focado" />
```

:focus-within

A pseudo-classe CSS **`:focus-within`** representa um elemento que recebeu o foco ou contém um elemento que recebeu o foco. Em outras palavras, isto representa um elemento que é correspondido por si só pela pseudo-classe `:focus` ou tem um descendente que é correspondido por `:focus`. (Isto inclui descendentes em shadow trees.)

```
/* Seleciona uma <div> quando um de seus descendentes é focado */
div:focus-within {
  background: cyan;
}
```

Este seletor é útil, pegando um exemplo comum, para destacar um todo `<form>` container quando o usuário focar em um de seus campos `<input>`.

Sintaxe

```
:focus-within {
  /* ... */
}
```

Exemplo

Neste exemplo, o formulário receberá estilos de coloração especiais quando o input de texto recebe o foco

HTML

```
<p>Tente digitar neste formulário.</p>

<form>
  <label for="given_name">Given Name:</label>
  <input id="given_name" type="text" />
  <br />
  <label for="family_name">Family Name:</label>
  <input id="family_name" type="text" />
</form>
```

CSS

```
form {
  border: 1px solid;
  color: gray;
  padding: 4px;
}

form:focus-within {
  background: #ff8;
  color: black;
}

input {
  margin: 4px;
}
```

Resultado

Veja também

- `:focus`

:hover

Resumo

A pseudo-classe `:hover` corresponde quando o usuário designa um elemento com um dispositivo apontador, mas não necessariamente o ativa. Este estilo pode ser substituído por qualquer outra pseudo-classe de link-relacionados, isto é `:link`, `:visited`, e `:active`, aparecendo em regras subsequentes. Na ordem para estilizar apropriadamente links, você precisa colocar a regra`:hover`depois das regras `:link` e `:visited` mas antes da regra `:active`, como definido pela ordem *LVHA*: `:link – :visited – :hover – :active.`

A pseudo-class `:hover` pode ser aplicada a qualquer pseudo-elemento. .

User agents visuais como Firefox, Internet Explorer, Safari, Opera ou Chrome, aplicam o estilo associado quando o cursor(mouse pointer) passa sobre um elemento.

> **Nota:** Em telas de toque (touch screens) :hover é impossível ou problemático. Dependendo do navegador a pseudo-classe :hover pode nunca funcionar, ou funcionar apenas por um curto momento depois de "tocar" um elemento, ou podem continuar a funcionar mesmo depois que o usuário pare de "tocar" o elemento até que o usuário toque outro elemento. Como dispositivos touchscreen são comuns, é importante para desenvolvedores web não terem conteúdos acessíveis apenas quando passamos sobre ele, uma vez que esse conteúdo é mais complicado ou impossível de acessar para usuários desses dispositivos.

Exemplos

```
:link:hover {
  outline: dotted red;
}

.foo:hover {
  background: gold;
}
```

Menu dropdown

Com a pseudo-classe`:hover`você pode criar algoritmos complexos de cascata. Esta é uma técnica comumente usada, por exemplo, para criar **menus dropdown com CSS** puro (apenas utilizando CSS, sem usar JavaScript). A essência desta técnica é criação de uma regra como a seguinte:

```
div.menu-bar ul ul {
  display: none;
}

div.menu-bar li:hover > ul {
  display: block;
}
```

para ser aplicada a uma estrutura HTML como a seguinte:

```
<div class="menu-bar">
  <ul>
    <li>
      <a href="example.html">Menu</a>
      <ul>
        <li>
          <a href="example.html">Link</a>
        </li>
        <li>
          <a class="menu-nav" href="example.html">Submenu</a>
          <ul>
            <li>
              <a class="menu-nav" href="example.html">Submenu</a>
              <ul>
                <li><a href="example.html">Link</a></li>
                <li><a href="example.html">Link</a></li>
                <li><a href="example.html">Link</a></li>
                <li><a href="example.html">Link</a></li>
```

```
            </ul>
          </li>
          <li><a href="example.html">Link</a></li>
        </ul>
      </li>
    </ul>
  </li>
</ul>
</div>
```

Veja nosso exemplo de menu dropdown baseado em CSS para entender melhor.

Galeria com imagens full-size e previews

Você pode usar a pseudo-classe `:hover` para construir uma galeria de imagem, exibindo imagens full-size apenas quando o mouse passa sobre as imagens. Veja esse exemplo para uma possível dica.

> **Nota:** Para efeitos análogos, mas baseado na pseudo-classe `:checked` (aplicada para esconder radioboxes), veja esse demo, tirado desta página En/CSS/:checked.

:invalid

A pseudo-classe CSS **`:invalid`** representa qualquer elemento `<input>` ou `<form>` cujo conteúdo não esteja válido.

```
/* Seleciona todos os inputs inválidos */
input:invalid {
  background-color: pink;
}
```

Essa pseudo-classe é útil para usuário identificar quais campos foram preenchidos incorretamente.

Sintaxe

```
:invalid {
  /* ... */
}
```

Exemplo

Esse exemplo apresenta um simples formulário onde a cor verde realça um input válido e a cor vermelha realça um input inválido.

HTML

```
<form>
  <label for="url_input">Adicione uma URL:</label>
  <input type="url" id="url_input" />
  <br />
  <br />
  <label for="email_input">Adicione um endereço de email:</label>
  <input type="email" id="email_input" required />
</form>
```

CSS

```
input:invalid {
  background-color: #ffdddd;
}

form:invalid {
  border: 5px solid #ffdddd;
}

input:valid {
  background-color: #ddffdd;
}

form:valid {
  border: 5px solid #ddffdd;
```

```
}

input:required {
  border-color: #800000;
  border-width: 3px;
}

input:required:invalid {
  border-color: #c00000;
}
```

Resultado

Notas

Radio buttons

Se algum radio button de um grupo possuir o atributo `required`, a pseudo-classe `:invalid` é aplicada a todos se nenhum deles for selecionado. (Radio buttons agrupados compartilham o mesmo valor pelo atributo `name`.)

Gecko defaults

Por padrão, Gecko não aplica o estilo para a pseudo-classe `:invalid`. Entretanto, pode-se aplicar um estilo (um "brilho" vermelho usando a propriedade `box-shadow`) à pseudo-classe `:-moz-ui-invalid`, que são um conjunto que se aplicam como o `:invalid`.

Você pode desabilitar o "brilho" usando o CSS a seguir, ou substituir completamente para alterar a aparência dos campos inválidos:

```
:invalid {
  box-shadow: none;
}

:-moz-submit-invalid {
  box-shadow: none;
}

:-moz-ui-invalid {
  box-shadow: none;
}
```

Veja também

- Outras pseudo-classes relacionadas com validações: `:required`, `:optional`, `:valid`
- Mozilla pseudo-classes relacionadas: `:-moz-ui-invalid`, `:-moz-submit-invalid`
- Form data validation

:last-child

Resumo

A CSS pseudo-classe `:last-child` representa qualquer elemento que é o último filho de seu elemento pai.

Sintaxe

```
elemento:last-child { propriedades de estilo }
```

Exemplo

Conteúdo HTML

```
<ul>
  <li>Esse item não é afetado pelo estilo.</li>
  <li>Esse também não.</li>
  <li>Esse item é! :)</li>
</ul>
```

Conteúdo CSS

```
li:last-child {
  background-color: lime;
```

```
}
```

Veja também

- `:first-child`
- `:nth-child`
- `:last-of-type`

:last-of-type

Sumário

A CSS pseudo-class `:last-of-type` representa o último irmão com o nome de elemento dado na lista de filhos de seu elemento pai.

Syntax

```
element:last-of-type { style properties }
```

Exemplo

Para selecionar o último elemento *em* dentro de um elemento `<p>`, você pode usar o seguinte:

```
p em:last-of-type {
  color: lime;
}
```

```
<p>
  <em>I'm not lime :(</em>
  <strong>I'm not lime :(</strong>
  <em>I'm lime :D</em>
  <strong>I'm also not lime :(</strong>
</p>

<p>
  <em>I'm not lime :(</em>
  <span><em>I am lime!</em></span>
  <strong>I'm not lime :(</strong>
  <em>I'm lime :D</em>
  <span><em>I am also lime!</em> <strike> I'm not lime </strike></span>
  <strong>I'm also not lime :(</strong>
</p>
```

...irá resultar em:

See also

- `:nth-last-of-type`
- `:first-of-type`
- `:nth-of-type`

:link

Resumo

A pseudo-class CSS :link permite que você selecione os links dentro de um elemento. Ela seleciona todos os links, até mesmo os que não foram visitados, incluindo os links ja estilizados em outras classes ou ids com o `:hover`, `:active` ou `:visited`. Para um funcionamento adequado é essencial que ela venha antes das regras: `:visited` — `:hover` — `:active`. O `:focus` é uma pseudo-class geralmente usada antes de a:hover ou depois, dependendo do resultado esperado.

Exemplos

```
a:link {
  color: slategray;
}
.external:link {
  background-color: lightblue;
}
```

Veja também

- `:visited`, `:hover`, `:active`

:not()

Resumo

A pseudo-classe CSS de **negação**, `:not(X)`, é uma notação funcional que recebe um seletor simples X como argumento. Ela seleciona um elemento que não é representado por seu argumento. *X* não pode conter outro seletor de negação.

> **Nota:** Seletores inúteis podem ser escritos usando esta pseudo-classe. Por exemplo, `:not(*)` seleciona qualquer elemento que não é algum elemento, então a regra nunca é aplicada.
>
> - É possível sobreescrever outras regras. E.g. `foo:not(bar)` vai selecionar os mesmos elementos do que o mais simples foo. No entanto a especificação é maior no exemplo com `:not`.
> - `:not(foo){} vai selecionar tudo que não for foo`, **incluindo os elementos `<html>` e `<body>`.**
> - Este seletor apenas se aplica a um elemento; não se pode usa-lo para excluir todos os seus ancestores. Por exemplo, `body :not(table) a` ainda vai ser aplicado a links dentro de tabelas, já que `<tr>` vai ser selecionado pelo `:not()` do seletor.

Sintaxe

```
:not(selector) { style properties }
```

Exemplos

```
p:not(.classico) {
  color: red;
}
body *:not(p) {
  color: green;
}
```

Dado o CSS acima e o HTML abaixo...

```
<p>Um pouco de texto.</p>
<p class="classico">Um pouco mais de texto.</p>
<span>Mais um texto</span>
```

Se obtém resultados como este:

:nth-child()

A pseudo-classe CSS `:nth-child()` seleciona elementos com base em suas posições em um grupo de elementos irmãos.

```
/* Seleciona um a cada quatro elementos
   de qualquer grupo de elementos irmãos,
   começando do quarto elemento (4, 8 12, etc.). */
:nth-child(4n) {
  color: lime;
}
```

Sintaxe

A pseudo-classe **`nth-child`** é usada com apenas um argumento, que representa o padrão usado para selecionar os elementos.

Valores

`odd`

Representa elementos cuja posição numérica em uma série de irmãos seja ímpar: 1, 3, 5, etc.

`even`

Representa elementos cuja posição numérica em uma série de irmãos seja par: 2, 4, 6, etc.

Notação funcional

`<An+B>`

Representa elementos cuja posição numérica em uma série de irmãos corresponda ao padrão `An+B`, o qual será aplicado a todo número maior ou igual a zero (ou seja, começando do zero). O índice do primeiro elemento é `1`. Ambos os valores `A` e `B` devem ser `<integer>` (inteiros).

Sintaxe formal

```
:nth-child([ <An+B> | even | odd ] [of <complex-selector-list>]?) {
  /* ... */
}
```

Exemplos

Exemplos de seletores

`tr:nth-child(odd)` or `tr:nth-child(2n+1)`

Representa as linhas ímpares de uma tabela HTML: 1, 3, 5, etc.

`tr:nth-child(even)` or `tr:nth-child(2n)`

Representa as linhas pares de uma tabela HTML: 2, 4, 6, etc.

`:nth-child(7)`

Representa o sétimo elemento.

`:nth-child(5n)`

Representa os elementos de número **5** [=5×1], **10** [=5×2], **15** [=5×3], **etc.**

`:nth-child(3n+4)`

Representa os elementos de número **4** [=(3×0)+4], **7** [=(3×1)+4], **10** [=(3×2)+4], **13** [=(3×3)+4], **etc.**

`:nth-child(-n+3)`

Representa os primeiros três elementos. [=-0+3, -1+3, -2+3]

`p:nth-child(n)`

Representa todos os elementos `<p>` em um grupo de irmãos. Isso seleciona os mesmos elementos que um simples seletor `p` faria (só que com um nível maior de especificidade).

`p:nth-child(1)` or `p:nth-child(0n+1)`

Representa todo `<p>` que seja o primeiro de seu grupo de irmãos. Ele é idêntico ao seletor `:first-child` (e tem o mesmo nível de especificidade).

`p:nth-child(n+8):nth-child(-n+15)`

Representa do 8º até o 15º elementos `<p>` de um grupo de irmãos.

Exemplo detalhado

HTML

```
<h3>
  <code>span:nth-child(2n+1)</code>, SEM um <code>&lt;em&gt;</code> entre os
  elementos filhos.
</h3>
<p>Os filhos 1, 3, 5 e 7 são selecionados.</p>
<div class="primeiro">
  <span>Span 1!</span>
  <span>Span 2</span>
  <span>Span 3!</span>
  <span>Span 4</span>
  <span>Span 5!</span>
  <span>Span 6</span>
  <span>Span 7!</span>
</div>
```

```
<br />

<h3>
  <code>span:nth-child(2n+1)</code>, COM um <code>&lt;em&gt;</code> entre os
  elementos filhos.
</h3>
<p>
  Os filhos 1, 5 e 7 são selecionados.<br />
  O 3 está incluído na contagem por ser um filho, mas não é selecionado porque
  ele não é um <code>&lt;span&gt;</code>.
</p>
<div class="segundo">
  <span>Span!</span>
  <span>Span</span>
  <em>Este é um `em`.</em>
  <span>Span</span>
  <span>Span!</span>
  <span>Span</span>
  <span>Span!</span>
  <span>Span</span>
</div>

<br />

<h3>
  <code>span:nth-of-type(2n+1)</code>, COM um <code>&lt;em&gt;</code> entre os
  elementos filhos.
</h3>
<p>
  Os filhos 1, 4, 6 e 8 são selecionados.<br />
  O 3 não está incluso na contagem nem é selecionado porque ele é um
  <code>&lt;em&gt;</code>, não um <code>&lt;span&gt;</code>, e
  <code>nth-of-type</code> seleciona apenas os filhos desse último tipo. O
  <code>&lt;em&gt;</code> é completamente pulado e ignorado.
</p>
<div class="terceiro">
  <span>Span!</span>
  <span>Span</span>
  <em>Este é um `em`.</em>
  <span>Span!</span>
  <span>Span</span>
  <span>Span!</span>
  <span>Span</span>
  <span>Span!</span>
</div>
```

CSS

```
html {
  font-family: sans-serif;
}

span,
div em {
  padding: 5px;
  border: 1px solid green;
  display: inline-block;
  margin-bottom: 3px;
}

.primeiro span:nth-child(2n + 1),
.segundo span:nth-child(2n + 1),
.terceiro span:nth-of-type(2n + 1) {
  background-color: lime;
}
```

Resultado

Veja também

- `:nth-of-type`, `:nth-last-child`

:nth-last-child()

A pseudo-classe CSS **`:nth-last-child()`** seleciona um ou mais elementos com base em sua posição entre um grupo de elementos-irmãos, contando a partir do último.

```
/* Seleciona todo quarto elemento
   em qualquer grupo de irmãos,
   contando do último para o primeiro */
:nth-last-child(4n) {
  color: lime;
}
```

> **Nota:** Essa pseudo-classe é essencialmente o mesmo que usar `:nth-child`, porém conta os itens de trás para a frente, e não do início para o final.

Sintaxe

A pseudo-classe `nth-last-child` é especificada com um único argumento, que representa o padrão para elementos correspondentes, contando do final.

Valores chave

`odd`

Representa elementos cuja posição numérica em uma série de irmãos é ímpar: 1, 3, 5, etc., contando do final.

`even`

Representa elementos cuja posição numérica em uma série de irmãos é par: 2, 4, 6, etc., contando do final.

Notação funcional

`<An+B>`

Representa elementos cuja posição numérica em uma série de irmãos corresponde ao padrão `An+B`, para cada valor de `n` que seja um número inteiro positivo ou zero. O índice do primeiro elemento, contando do final, é `1`. Os valores `A` e `B` devem ambos ser `<integer>`s (inteiros).

Sintaxe formal

```
:nth-last-child(<nth> [of <complex-selector-list>]?) {
  /* ... */
}
```

Exemplos

Seletores de exemplo

`tr:nth-last-child(odd)` ou `tr:nth-last-child(2n+1)`

Representa as linhas ímpares de uma tabela HTML: 1, 3, 5, etc., contando do final.

`tr:nth-last-child(even)` ou `tr:nth-last-child(2n)`

Representa as linhas pares de uma tabela HTML: 2, 4, 6, etc., contando do final.

`:nth-last-child(7)`

Representa o sétimo elemento, contando do final.

`:nth-last-child(5n)`

Representa elementos 5, 10, 15, etc., contando do final.

`:nth-last-child(3n+4)`

Representa elementos 4, 7, 10, 13, etc., contando do final.

`:nth-last-child(-n+3)`

Representa os últimos três elementos em um grupo de irmãos.

`p:nth-last-child(n)`

Representa cada elemento `<p>` em um grupo de irmãos. Isso é o mesmo que um simples seletor `p`.

`p:nth-last-child(1)` ou `p:nth-last-child(0n+1)`

Representa cada `<p>` que seja o primeiro elemento em um grupo de irmãos, contando do final. Isso é o mesmo que o seletor `:last-child`.

Exemplo de tabela

HTML

```
<table>
  <tbody>
    <tr>
      <td>Primeira linha</td>
    </tr>
    <tr>
      <td>Segunda linha</td>
    </tr>
    <tr>
      <td>Terceira linha</td>
    </tr>
    <tr>
      <td>Quarta linha</td>
    </tr>
    <tr>
      <td>Quinta linha</td>
    </tr>
  </tbody>
</table>
```

CSS

```
table {
  border: 1px solid blue;
}

/* Seleciona os últimos três elementos */
tr:nth-last-child(-n + 3) {
  background-color: pink;
}
```

Resultado

Ver também

- `:nth-child`, `:nth-last-of-type`

:nth-of-type()

A pseudo-classe CSS **`:nth-of-type()`** corresponde a um ou mais elementos de um dado tipo, baseado em sua posição entre um grupo de irmãos.

```
/* Seleciona cada quarto elemento <p>
   entre qualquer grupo de irmãos */
p:nth-of-type(4n) {
  color: lime;
}
```

Sintaxe

A pseudo-classe `nth-of-type` é especificada com um único argumento, o qual representa o padrão para a correspondência dos elementos.

Veja `:nth-child` para uma explicação mais detalhada de sua sintaxe.

Sintaxe formal

```
:nth-of-type(<An+B> | even | odd) {
  /* ... */
}
```

Exemplos

Exemplo básico

HTML

```
<div>
  <div>Este elemento não é contado.</div>
  <p>1º parágrafo.</p>
  <p>2º parágrafo.</p>
  <div>Este elemento não é contado.</div>
  <p>3º parágrafo.</p>
  <p>4º parágrafo.</p>
</div>
```

CSS

```
/* Parágrafos ímpares */
p:nth-of-type(2n + 1) {
  color: red;
}

/* Parágrafos pares */
p:nth-of-type(2n) {
  color: blue;
}

/* Primeiro parágrafo */
p:nth-of-type(1) {
  font-weight: bold;
}
```

Resultado

Veja também

- `:nth-child`, `:nth-last-of-type`

:only-child

A pseudo-classe de CSS **`:only-child`** representa um elemento sem nenhum elemento-irmão. É o mesmo que `:first-child:last-child` ou `:nth-child(1):nth-last-child(1)`, mas com uma especificidade mais baixa.

```
/* Seleciona cada <p>, mas apenas se for o */
/* único filho de seu elemento-pai */
p:only-child {
  background-color: lime;
}
```

> **Nota:** Originalmente, foi definido que o elemento selecionado deveria ter um elemento-pai. A partir de Selectors Level 4, isso não é mais necessário.

Sintaxe

```
:only-child {
  /* ... */
}
```

Exemplos

Exemplo básico

HTML

```
<main>
  <div>
    <i>Eu sou um filho único solitário.</i>
  </div>
```

```
  <div>
    <i>Eu tenho irmãos.</i><br />
    <b>Eu também!</b><br />
    <span>Eu também tenho irmãos, <span>mas este é um filho único.</span></span>
  </div>
</main>
```

CSS

```
main :only-child {
  color: red;
}
```

Resultado

Exemplo de lista

HTML

```
<ol>
  <li>
    Primeiro
    <ul>
      <li>Essa lista tem apenas um elemento.</li>
    </ul>
  </li>
  <li>
    Segundo
    <ul>
      <li>Essa lista tem três elementos.</li>
      <li>Essa lista tem três elementos.</li>
      <li>Essa lista tem três elementos.</li>
    </ul>
  </li>
</ol>
```

CSS

```
li li {
  list-style-type: disc;
}
li:only-child {
  color: red;
  list-style-type: square;
}
```

Resultado

Ver também

- `:only-of-type`
- `:first-child`
- `:last-child`
- `:nth-child`

:only-of-type

Descrição

A pseudo-classe CSS `:only-of-type` representa qualquer elemento que não possui irmãos de um determinado tipo.

Sintaxe

```
:only-of-type { propriedades}
```

Ver também

- `:nth-of-type`, `:first-of-type`, `:last-of-type`

:optional

A pseudo-classe CSS **`:optional`** representa qualquer elemento `<input>`, `<select>`, ou `<textarea>` que não contenha o atributo `required`.

```
/* Selects qualquer <input> opcional */
input:optional {
  border: 1px dashed black;
}
```

Esta pseudo-classe é utilizada para estilizar campos do formulário que não são requeridos para sumissão.

> **Nota:** A pseudo-classe `:required` seleciona campos do fumulário *requeridos*.

Sintaxe

```
:optional {
  /* ... */
}
```

Exemplos

Veja `:invalid` como exemplo.

Referências de acessibilidade

Se um formulário contém `<input>` opcionais, inputs requeridos devem ser indicados utilizando o atriuto `required`. Isto garantirá que usuários que navegam com tecnologia de acessibilidade, como um leitor de tela, possam entender quais entradas devem ter o conteúdo validado para submeter o formulário com êxito.

Entradas requeridas devem ser indicadas visualmente, utilizando um tratamento que não seja apenas a cor para passar o significado. Usualmente, textos descritivos e/ou um ícone são utilizados.

- MDN Understanding WCAG, Guideline 3.3 explanations
- Understanding Success Criterion 3.3.2 | W3C Understanding WCAG 2.0

Ver também

- Outra pseudo-classes de validação relacionadas: `:required`, `:invalid`, `:valid`
- Validação de dados de formulário

:out-of-range

A pseudo classe de CSS **`:out-of-range`** representa um elemento de `<entrada>` cujo valor atual esta fora dos limites especificados pelo `min` e `max` atributos

```
/* Selecione qualquer <entrada>, mas somente quando estiver dentros dos
 limites especificados, e quando este valor estiver fora deste limite */
input:out-of-range {
  background-color: rgba(255, 0, 0, 0.25);
}
```

Essa pseudo classe é muito útil por dar ao usuario uma indicacão visual de que o valor atual esta fora dos limites permitidos

> **Nota:** Esta pseudo classe se aplica somente a elementos que tem (ou podem ter) uma limitacao. Na falta dessa de algo como essa limitacao, o elemento nao tera como ficar "in-range"(dentro dos limites) ou "out-of-range" (fora dos limites).

Sintaxe

```
:out-of-range {
  /* ... */
}
```

Exemplo

HTML

```
<form action="" id="form1">
  <p>Valores entre 1 e 10 são validos.</p>
  <ul>
    <li>
      <input
        id="value1"
        name="value1"
        type="number"
        placeholder="1 to 10"
        min="1"
        max="10"
        value="12" />
      <label for="value1">Seu valor está </label>
    </li>
  </ul>
</form>
```

CSS

```
li {
  list-style: none;
  margin-bottom: 1em;
}

input {
  border: 1px solid black;
}

input:in-range {
  background-color: rgba(0, 255, 0, 0.25);
}

input:out-of-range {
  background-color: rgba(255, 0, 0, 0.25);
  border: 2px solid red;
}

input:in-range + label::after {
  content: "dentro dos limites.";
}

input:out-of-range + label::after {
  content: "Fora dos limites!";
}
```

Resultado

See also

- `:in-range`
- Form data validation

:read-write

A pseudo-classe CSS **`:read-write`** representa um elemento (como o `input` ou `textarea`) que é editável pelo usuário.

```
input:read-write,
textarea:read-write {
  background-color: #bbf;
}

p:read-write {
  background-color: #bbf;
}
```

Sintaxe

```
:read-write {
  /* ... */
}
```

Exemplos

Confirmando informações do formulário em controles read-only/read-write

Ao utilizar os controles de formulário `readonly` permite que o usuário verifique e verifique as informações que eles podem ter inserido em um formulário anterior (por exemplo: Detalhes de envio), enquanto ainda pode enviar as informações junto com o resto do formulário. Nós fazemos exatamente isso no exemplo abaixo.

A pseudo-classe `:read-only` *(:somente-leitura)* é usada para remover todo o estilo que faz com que as entradas pareçam campos clicáveis, fazendo com que pareçam outro parágrafo *somente-leitura*. A pseudo-classe `:read-write` *(:leitura-escrita)* por outro lado, é usado para fornecer um estilo mais agradável para a `<textarea>`.

```
input:-moz-read-only,
textarea:-moz-read-only,
input:read-only,
textarea:read-only {
  border: 0;
  box-shadow: none;
  background-color: white;
}

textarea:-moz-read-write,
textarea:read-write {
  box-shadow: inset 1px 1px 3px #ccc;
  border-radius: 5px;
}
```

Você pode encontrar o código fonte completo em readonly-confirmation.html (disponível em inglês); isso torna assim:

Estilizar controle não formal de read-write

Este seletor não seleciona apenas `<input>`/`<textarea>` elementos - ele selecionará qualquer elemento que possa ser editado pelo usuário, como um `<p>` elemento com `contenteditable` definido sobre ele.

```
<p contenteditable>Este parágrafo é editável; é read-write.</p>

<p>Este parágrafo não é editável; é read-only.</p>
```

```
p {
  font-size: 150%;
  padding: 5px;
  border-radius: 5px;
}

p:read-only {
  background-color: red;
  color: white;
}

p:read-write {
  background-color: lime;
}
```

Veja também

- `:read-only`
- Atributo HTML `contenteditable`.

:required

A pseudo-classe CSS **`:required`** representa qualquer `<input>`, `<select>`, ou `<textarea>` contendo o atributo `required`.

```
/* Seleciona qualquer <input> requerido */
input:required {
  border: 1px dashed red;
}
```

Esta pseudo-classe é utilizada para destacar campos que devem ter dados válidos antes do formulário ser submetido.

> **Nota:** A pseudo-classe `:optional` seleciona campos do formulário que são *opcionais*.

Sintaxe

```
:required {
  /* ... */
}
```

Exemplos

Veja `:invalid` como exemplo.

Preocupações com acessibilidade

Elementos `<input>` obrigatórios devem ter o atributo `required` atribuídos. Isso garante que pessoas que navegam com auxílio de recursos de acessibilidade, como o leitor de tela, possam entender quais campos precisam ter conteúdos válidos para garantir a submissão dos dados .

Se o formulário também possui campos opcionais, entradas requeridas devem ser indicadas visualmente usando um tratamento que não seja apenas a cor para passar o significado. Usualmente, textos descritivos e/ou um ícone são utilizados.

- MDN Understanding WCAG, Guideline 3.3 explanations
- Understanding Success Criterion 3.3.2 | W3C Understanding WCAG 2.0

Veja também

- Outras pseudo-classe relacionadas a validação: `:optional`, `:invalid`, `:valid`
- Validação de dados de formulário

:root

A pseudo-classe CSS **`:root`** se equipara à raiz de uma árvore, que por sua vez representa o documento. Aplicado ao HTML, `:root` representa o elemento `<html>` e é idêntico ao seletor `html`, exceto que sua especificidade é mais alta.

```
/* Seleciona o elemento raiz do documento:
   <html> no caso do HTML */
:root {
  background: yellow;
}
```

Sintaxe

```
:root
```

Exemplos

Declarando variáveis CSS globais

O `:root` pode ser útil para declarar uma variável CSS global:

```
:root {
  --main-color: hotpink;
  --pane-padding: 5px 42px;
}
```

Veja também

- `<html>`
- `Document.rootElement`
- `Node.getRootNode()`

- `Element.shadowRoot`
- `ShadowRoot`

:target

A pseudo-classe CSS **`:target`** representa um único elemento (o elemento alvo) com uma `id` correspondente ao fragmento da URL.

```
/* Seleciona um elemento com a ID correspondente ao fragmento da URL */
:target {
  border: 2px solid black;
}
```

Por exemplo, a seguinte URL tem um fragmento (denotado pelo sinal de #) que aponta para o elemento chamado `section2`:

```
http://www.example.com/index.html#section2
```

O seguinte elemento será selecionado pelo seletor `:target` quando a URL for igual acima:

```
<section id="section2">Exemplo</section>
```

Sintaxe

```
:target {
  /* ... */
}
```

Exemplos

Índice

A pseudo-classe `:target` pode ser usada para destacar uma parte da página que foi vinculada a partir de um índice.

HTML

```
<h3>Índice</h3>
<ol>
  <li><a href="#p1">Ir para o primeiro parágrafo!</a></li>
  <li><a href="#p2">Ir para o segundo parágrafo!</a></li>
  <li>
    <a href="#vazio"
      >Esse link não vai pra lugar nenhum, pois, o alvo não existe</a
    >
  </li>
</ol>

<h3>Meu artigo divertido</h3>
<p id="p1">
  Você pode definir <i>este parágrafo</i> como alvo usando um fragmento de URL.
  Clique no link acima para experimentar!
</p>
<p id="p2">
  Esse é <i>outro parágrafo</i>, também acessável pelos links acima. Não é
  incrível?
</p>
```

CSS

```
p:target {
  background-color: gold;
}

/* Adicione o pseudo-elemento dentro do elemento alvo */
p:target::before {
  font: 70% sans-serif;
  content: "►";
  color: limegreen;
  margin-right: 0.25em;
}
```

```
/* Estilize nos elementos em itálico dentro do elemento alvo  */
p:target i {
  color: red;
}
```

Resultado

Lightbox com CSS puro

Você pode usar a pseudo-classe `:target` para criar uma lightbox sem usar JavaScript. Essa técnica requer que os links apontem para os elementos que inicialmente estavam escondidas na página. Uma vez designado, o CSS muda o `display` então, assim o conteúdo pode ser mostrado.

> **Nota:** Uma lightbox com CSS puro mais completo usando a pseudo-classe `:target` está disponível no GitHub (demo).

HTML

```
<ul>
  <li><a href="#example1">Abrir exemplo #1</a></li>
  <li><a href="#example2">Abrir exemplo #2</a></li>
</ul>

<div class="lightbox" id="example1">
  <figure>
    <a href="#" class="close"></a>
    <figcaption>
      Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec felis enim,
      placerat id eleifend eu, semper vel sem.
    </figcaption>
  </figure>
</div>

<div class="lightbox" id="example2">
  <figure>
    <a href="#" class="close"></a>
    <figcaption>
      Cras risus odio, pharetra nec ultricies et, mollis ac augue. Nunc et diam
      quis sapien dignissim auctor. Quisque quis neque arcu, nec gravida magna.
    </figcaption>
  </figure>
</div>
```

CSS

```
/* Lightbox fechado */
.lightbox {
  display: none;
}

/* Lightbox aberto */
.lightbox:target {
  position: absolute;
  left: 0;
  top: 0;
  width: 100%;
  height: 100%;
  display: flex;
  align-items: center;
  justify-content: center;
}

/* Conteúdo da lightbox */
.lightbox figcaption {
  width: 25rem;
  position: relative;
  padding: 1.5em;
  background-color: lightpink;
}

/* Botão fechar */
.lightbox .close {
```

```
  position: relative;
  display: block;
}

.lightbox .close::after {
  right: -1rem;
  top: -1rem;
  width: 2rem;
  height: 2rem;
  position: absolute;
  display: flex;
  z-index: 1;
  align-items: center;
  justify-content: center;
  background-color: black;
  border-radius: 50%;
  color: white;
  content: "x";
  cursor: pointer;
}

/* Lightbox overlay */
.lightbox .close::before {
  left: 0;
  top: 0;
  width: 100%;
  height: 100%;
  position: fixed;
  background-color: rgba(0, 0, 0, 0.7);
  content: "";
  cursor: default;
}
```

Resultado

Veja também

- Usando a pseudo-classe :target em seletores.

:valid

A pseudo-classe CSS **`:valid`** representa qualquer `<input>` ou outro elemento do `<form>` cujo conteúdo foi validado com sucesso. Isso permite, facilmente, adicionar uma aparência que ajude o usuário a identificar os campos validados.

```
/* Seleciona qualquer input válido */
input:valid {
  background-color: powderblue;
}
```

Essa pseudo-classe é útil para realçar os campos válidos para o usuário.

Sintaxe

```
:valid {
  /* ... */
}
```

Exemplos

Veja `:invalid` para um exemplo.

Veja também

- Outras pseudo-classe relacionada com validação: `:required`, `:optional`, `:invalid`
- Form data validation

:visited

O **`:visited`** é uma Pseudo-classe do CSS para indicar se o link em questão já foi visitado pelo usuário. Por questões de privacidade, os styles que podem ser modificados usando este seletor são muito limitados.

```
/* Seleciona qualquer <a> que poderá ser visitado */
a:visited {
  color: green;
}
```

Styles definidos pela pseudo-classe `:visited` vão ser substituidos por qualquer pseudo-classe relacionada ao link subsequente (`:link`, `:hover`, ou `:active`) que tenha ao menos uma especificação igual. Para estilizar links apropriadamente, garantindo assim seu funcionamento adequado, ponha a regra `:visited` depois da regra `:link` mas antes das regras `:hover` e `:active` , Pode-se definir pela ordem *LVHA*: `:link` — `:visited` — `:hover` — `:active`.

Restrições de Estilo

Por razões de privacidade, navegadores limitam estritamente quais estilos você pode aplicar usando esta pseudo-class, como elas podem ser usadas:

- Propriedades admicíveis no CSS são:
  - `color`;
  - `background-color`;
  - `border-color`;
  - `border-bottom-color`;
  - `border-left-color`;
  - `border-right-color`;
  - `border-top-color`;
  - `column-rule-color`;
  - `outline-color`.
- Atributos SVG admissíveis são `fill` e `stroke`.
- Os componentes alpha permitidos de estilos seram ignorados. Ao invés disso, o componente alpha do elemento de estado non-`:visited` será usado, exceto quando o componente é `0`, no qual o estilo definido em `:visited` será inteiramente ignorado.
- Embora esses estilos podem mudar como as cores serão mostradas ao usuário, o método `window.getComputedStyle` vai mentir e sempre retornar o valor da cor em non-`:visited`.

> **Nota:** Para mais informações quanto a essas limitações e as razões por trás delas, veja Privacy and the :visited selector. Obs: No momento estará apenas disponível em inglês.

Syntaxe

```
a {
  background-color: white;
  border: 1px solid white;
}
a:visited {
  background-color: lightblue;
  border-color: cyan;
  color: darkblue;
}
```

Exemplo

Propriedades que de alguma forma não tenha cor ("color") definida, ou seja transparente, não podem ser modificadas como o `:visited`. Das propriedades que podem ser definidas por essa pseudo-classe, seu navegador provavelmente terá por padão valores para `color` e `column-rule-color` somente. Portanto, se você quiser modificar outras propriedades, você necessitará pré-definir os valores fora do seletor `:visited`.

HTML

```
<a href="#">Você já visitou este link?</a>
<br />
<a href="">Você já visitou este link.</a>
```

CSS

```
a {
  /* Specifica non-transparent por padrão para algumas
     propriedades, permitindo-os a serem estilizados pelo
```

```
     estado :visited */
  background-color: white;
  border: 1px solid white;
}

a:visited {
  background-color: yellow;
  border-color: hotpink;
  color: hotpink;
}
```

Resultado

Veja também (inglês)

- Privacy and the :visited selector
- Link-related pseudo-classes: `:link`, `:active`, `:hover`

Seletor de atributos

O **seletor de atributos** combina elementos baseado no valor de um atributo dado.

```
/* <a> elementos com um atributo de título  */
a[title] {
  color: purple;
}

/* <a> elementos combinando com um href "https://example.org" */
a[href="https://example.org"] {
  color: green;
}

/* <a> elementos com um href contendo "example" */
a[href*="example"] {
  font-size: 2em;
}

/* <a> elementos com href terminando em ".org" */
a[href$=".org"] {
  font-style: italic;
}
```

`[attr]`

Representa um elemento com atributo de nome *attr*.

`[attr=value]`

Representa um elemento com um atributo de nome *attr*, o qual o valor é exatamente *value*.

`[attr~=value]`

Representa um elemento com um atributo de nome *attr,* o qual *value* é uma lista de palavras separadas por espaços, e uma dessas é exatamente *value*.

`[attr|=value]`

Representa um elemento com um atributo de nome *attr* o qual o valor pode ser exatamente *value* ou pode começar com *value* imediatamente seguido por hífen - (U+002D). Isso somente é usado para linguagens que combinam sub-códigos.

`[attr^=value]`

Representa um elemento com um atributo com nome *attr* que tem um valor prefixado (precedido) por *value*.

`[attr$=value]`

Representa um elemento com um atributo de nome *attr* que tem como sufixo (seguido) por *value*.

`[attr*=value]`

Representa um elemento com um atributo de nome *attr* o qual valor contém ao menos uma ocorrência de *value* contido na string.

`[attr operator value i]`

Adiciona um `i` (ou `I`) antes do fechamento das chaves {}, faz com que o valor seja comparado sem levar em conta caixa alta ou caixa baixa(para caracteres dentro da faixa ASCII).

Exemplos

Links

CSS

```
a {
  color: blue;
}

/* Links internos, começando com "#" */
a[href^="#"] {
  background-color: gold;
}

/* Links com "example" em qualquer lugar da URL */
a[href*="example"] {
  background-color: silver;
}

/* Links com "insensitive" em qualquer lugar da URL,
   independentemente da capitalização */
a[href*="insensitive" i] {
  color: cyan;
}

/* Links com final ".org" */
a[href$=".org"] {
  color: red;
}
```

HTML

```
<ul>
  <li><a href="#internal">Internal link</a></li>
  <li><a href="http://example.com">Example link</a></li>
  <li><a href="#InSensitive">Insensitive internal link</a></li>
  <li><a href="http://example.org">Example org link</a></li>
</ul>
```

Resultado

Languages

CSS

```
/* Todas divs com um atributo `lang` em negrito. */
div[lang] {
  font-weight: bold;
}
/* Todas divs com US English em azul (blue). */
div[lang~="en-us"] {
  color: blue;
}
/* Todas divs onde Portuguese esta em verde (green). */
div[lang="pt"] {
  color: green;
}
/* Todas divs onde Chinese esta em vermelho (red), Simplificado para (zh-CN) ou tradicional (zh-TW). */
div[lang|="zh"] {
  color: red;
}
/* Todas divs com Traditional Chinese `data-lang` que é purple. */
/* Nota: Você também poderia usar atributos separados por hífen com aspas duplas */
div[data-lang="zh-TW"] {
  color: purple;
}
```

HTML

```
<div lang="en-us en-gb en-au en-nz">Hello World!</div>
<div lang="pt">Olá Mundo!</div>
<div lang="zh-CN">世界您好! </div>
<div lang="zh-TW">世界您好! </div>
<div data-lang="zh-TW">?世界您好! </div>
```

Resultado

Seletores de classe

O **seletor de classes** CSS corresponde aos elementos com base no conteúdo de seus atributos `class`.

```
/* Todos os elementos com class="espaçoso" */
.espaçoso {
  margin: 2em;
}

/* Todos os elementos <li> com class="espaçoso" */
li.espaçoso {
  margin: 2em;
}

/* Todos os elementos <li> cujo conteúdo do atributo class possua "espaçoso" e "elegante" */
/* Por exemplo, class="elegante antigo espaçoso" */
li.espaçoso.elegante {
  margin: 2em;
}
```

Sintaxe

```
.classe_nome { propriedades de estilo }
```

Observe que isso é o mesmo que `seletor de atributo`:

```
[classe~=classe_nome] { propriedades de estilo  }
```

Exemplo

CSS

```
.vermelho {
  color: #f33;
}

.amarelo-bg {
  background: #ffa;
}

.elegante {
  font-weight: bold;
  text-shadow: 4px 4px 3px #77f;
}
```

HTML

```
<p class="vermelho">Este parágrafo tem o texto vermelho.</p>
<p class="vermelho amarelo-bg">
  Este parágrafo possui texto em vermelho e fundo amarelo.
</p>
<p class="vermelho elegante">
  Este parágrafo possui texto em vermelho e uma propriedade de estilo
  "elegante".
</p>
<p>Este é um parágrafo comum.</p>
```

Resultado

Descendant combinator

O **combinador descendente** - normalmente representado por um caractere de espaço simples (" ") - combina dois seletores para que os elementos correspondidos pelo segundo seletor sejam selecionados se eles tiverem um elemento ancestral (pai, pai ou mãe, pai ou mãe, etc.) correspondente ao primeiro seletor. Os seletores que utilizam um combinador descendente são chamados de seletores descendentes.

```
/* Listar itens descendentes da lista "my-things" */
ul.my-things li {
  margin: 2em;
}
```

O combinador descendente é tecnicamente um ou mais caracteres de espaço em branco do CSS - o caractere de espaço e/ou um dos quatro caracteres de controle: retorno de carro (carriage return), Quebra de página (form feed), nova linha e caracteres de tabulação — entre dois seletores na ausência de outro combinador. Além disso, os caracteres de espaço em branco dos quais o combinador é composto podem conter qualquer número de comentários CSS.

Sintaxe

```
selector1 selector2 {
  /*  declarações de propriedade  */
}
```

Exemplo

CSS

```
li {
  list-style-type: disc;
}

li li {
  list-style-type: circle;
}
```

HTML

```
<ul>
  <li>
    <div>Item 1</div>
    <ul>
      <li>Subitem A</li>
      <li>Subitem B</li>
    </ul>
  </li>
  <li>
    <div>Item 2</div>
    <ul>
      <li>Subitem A</li>
      <li>Subitem B</li>
    </ul>
  </li>
</ul>
```

Resultado

Veja também

- Child combinator

Seletores de ID

Em um documento HTML, os seletores de ID do CSS selecionam um elemento baseado no conteúdo de seu atributo `ID`, o qual deve ser exatamente igual ao valor dado ao seletor.

Sintaxe

```
#valor_id { propriedades de estilo }
```

Note que isso é equivalente aos seguintes `seletores de atributo`:

```
[id=valor_id] { propriedades de estilo }
```

Exemplo

```
span#identificado {
  background-color: DodgerBlue;
}
```

```
<span id="identificado">Aqui um span com um pouco de texto.</span>
<span>Aqui outro.</span>
```

Pseudo-classes

Uma ***pseudo-classe*** CSS é uma palavra-chave adicionada a seletores que especifica um estado especial do elemento selecionado. Por exemplo, `:hover` pode ser usado para alterar a cor de um botão quando o usuário passar o cursor sobre ele.

```
/*Qualquer botão sobre o qual o ponteiro do usuário esteja passando sobre*/
button:hover {
  color: blue;
}
```

Pseudo-classes permitem que você aplique um estilo a um elemento não apenas em relação ao conteúdo da árvore do documento, mas também em relação a fatores externos como o histórico de navegação (`:visited`, por exemplo), o status do seu conteúdo (como `:checked` em certos elementos de um formulário), ou a posição do mouse (como `:hover`, que permite saber se o mouse está sobre um elemento ou não).

> **Nota:** Diferentemente das pseudo-classes, pseudo-elementos podem ser usados para estilizar uma *parte específica* de um elemento.

Sintaxe

```
seletor:pseudo-classe {
  propriedade: valor;
}
```

Assim como nas classes regulares, você pode encadear juntas quantas pseudo-classes você desejar em um seletor.

Índice de pseudo-classes padrão

- `:active`
- `:checked`
- `:default`
- `:dir()`
- `:disabled`
- `:empty`
- `:enabled`
- `:first`
- `:first-child`
- `:first-of-type`
- `:fullscreen`
- `:focus`
- `:hover`
- `:indeterminate`
- `:in-range`
- `:invalid`
- `:lang()`
- `:last-child`
- `:last-of-type`
- `:left`
- `:link`
- `:not()`
- `:nth-child()`
- `:nth-last-child()`
- `:nth-last-of-type()`

- `:nth-of-type()`
- `:only-child`
- `:only-of-type`
- `:optional`
- `:out-of-range`
- `:read-only`
- `:read-write`
- `:required`
- `:right`
- `:root`
- `:scope`
- `:target`
- `:valid`
- `:visited`

Veja também

- `pseudo-elements`

Pseudo-elementos

Um **pseudo-elemento** CSS é uma palavra-chave adicionada a um seletor que permite que você estilize uma parte específica do elemento selecionado. Por exemplo, o pseudo-elemento `::first-line` aplica o estilo apenas na primeira linha de um parágrafo.

```
/* A primeira linha de todo elemento <p>. */
p::first-line {
  color: blue;
  text-transform: uppercase;
}
```

> **Nota:** Diferentemente dos pseudo-elementos, `pseudo-classes` podem ser utilizadas para estilizar um elemento baseado em seu *estado*.

Sintaxe

```
seletor::pseudo-elemento {
  propriedade: valor;
}
```

Você pode utilizar apenas um pseudo-elemento em um seletor. Ele deve aparecer depois da declaração de um elemento simples.

> **Nota:** Como regra, os dois pontos devem ser usados duas vezes (`::`) ao invés de uma única vez (`:`). Isso distingue pseudo-classes de pseudo-elementos. Apesar disso, devido a essa distinção não estar presente em versões mais antigas da especificação da W3C, a maioria dos navegadores suportam ambas as sintaxes para os pseudo-elementos originais.

Índice de pseudo-elementos comuns

- `::after`
- `::before`
- `::cue`
- `::first-letter`
- `::first-line`
- `::selection`
- `::slotted`
- `::backdrop`
- `::placeholder`
- `::marker`
- `::spelling-error`
- `::grammar-error`

Navegador	Versão mais baixa	Suporte de

Internet Explorer	8.0	`:pseudo-element`
	9.0	`:pseudo-element ::pseudo-element`
Firefox (Gecko)	1.0 (1.0)	`:pseudo-element`
	1.0 (1.5)	`:pseudo-element ::pseudo-element`
Opera	4.0	`:pseudo-element`
	7.0	`:pseudo-element ::pseudo-element`
Safari (WebKit)	1.0 (85)	`:pseudo-element ::pseudo-element`

Ver também

- pseudo-classes

General sibling selectors

Resumo

O **combinador geral de irmãos** (~) separa dois seletores e corresponde ao segundo elemento somente se ele for precedido pelo primeiro, e ambos compartilham um pai comum.

Sintaxe

```
elemento_anterior ~ elemento_alvo { propriedades de estilo }
```

Exemplo

```
p ~ span {
  color: red;
}
```

```
<span>Este não é vermelho.</span>
<p>Aqui temos um parágrafo.</p>
<code>Aqui temos um pouco de código.</code>
<span>E aqui há um span vermelho!</span>
<code>Mais código...</code>
<span>E este também é um span vermelho!</span>
```

Veja mais

- Adjacent sibling selectors

Type selectors

O **seletor de tipo** CSS corresponde aos elementos pelo nome do nó. Em outras palavras, ele seleciona todos os elementos do tipo fornecido em um documento.

```
/* Todos os elementos <a> */
a {
  color: red;
}
```

Sintaxe

```
element { style properties }
```

Exemplos

CSS

```
span {
  background-color: skyblue;
}
```

HTML

```
<span>Here's a span with some text.</span>
<p>Here's a p with some text.</p>
<span>Here's a span with more text.</span>
```

Result

Seletor universal

O **seletor universal** do CSS (`*`) aplica estilos a elementos de qualquer tipo.

```
/* Seleciona todos os elementos */
* {
  color: green;
}
```

A partir do CSS3, o asterisco pode ser combinado com `namespaces`:

- `ns|*` - aplica a todos os elementos no namespace *ns*
- `*|*` - aplica a todos elementos
- `|*` - aplica a todos os elementos que não tem namespaces declarados

Sintaxe

```
* { propriedades de estilo }
```

O asterisco é opcional para seletores simples. Por exemplo, `*.atencao` e `.atencao` são equivalentes.

Exemplos

CSS

```
* [lang^="pt"] {
  color: green;
}

*.atencao {
  color: red;
}

*#conteudoprincipal {
  border: 1px solid blue;
}

.flutuando {
  float: left;
}

/* automaticamente aplica clear ao próximo irmão após o elemento com a classe .flutuando */
.flutuando + * {
  clear: left;
}
```

HTML

```
<p class="atencao">
  <span lang="pt-br">Um span verde</span> em um parágrafo vermelho.
</p>
<p id="conteudoprincipal" lang="pt-pt">
  <span class="atencao">Um span vermelho</span> em um parágrafo verde.
</p>
```

Resultado

attr

A função **`attr()`** do CSS é utilizada para se obter o valor de um determinado atributo do elemento selecionado e usá-lo na folha de estilo. Também pode ser utilizado em pseudo-elementos, nesse caso o valor do atributo no pseudo-elemento do elemento original é retornado.

```
/* Uso Simples */
attr(data-count);
attr(title);

/* Com o tipo */
attr(src url);
attr(data-count number);
attr(data-width px);

/* Com fallback */
attr(data-count number, 0);
attr(src url, '');
attr(data-width px, inherit);
attr(data-something, 'default');
```

> **Nota:** A função `attr()` pode ser utilizada com qualquer propriedade CSS, mas o suporte para propriedades que não sejam `content` é experimental.

Sintaxe

Valores

`nome-do-atributo`

É o nome do atributo HTML do elemento referenciado no CSS.

`<tipo-ou-unidade>`

É uma palavra-chave representando tanto o tipo quanto o valor do atributo, ou sua unidade, assim como no HTML alguns atributos tem unidades implícitas. Se o uso de `<tipo-ou-unidade>` como um valor para determinado atributo for inválido, a expressão `attr()` será inválida também. Se omitida, o padrão será `string`. A lista de valores válidos é:

Palavra-chave	Tipo Associado	Comentário	Valor Padrão
`string`	`<string>`	O valor do atributo é tratado como um CSS `<string>`. NÃO será reparado, e em particular os caracteres usados .	An empty string.
`cor`	`<color>`	O valor do atributo é analisado como hash (3- ou 6-valores hash) ou a palavra-chave. Precisa ser um valor `<string>` CSS válido. Os espaços do lead e trail são retirados.	corAtual
`url`	`<url>`	The attribute value is parsed as a string that is used inside a CSS `url()`function. Relative URL are resolved relatively to the original document, not relatively to the style sheet. Leading and trailing spaces are stripped.	The url `about:invalid` that points to a non-existent document with a generic error condition.
`integer`	`<integer>`	The attribute value is parsed as a CSS `<integer>`. If it is not valid, that is not an integer or out of the range accepted by the CSS property, the default value is used. Leading and trailing spaces are stripped.	`0`, or, if `0` is not a valid value for the property, the property's minimum value.
`number`	`<number>`	The attribute value is parsed as a CSS `<number>`. If it is not valid, that is not a number or out of the range accepted by the CSS property, the default value is used. Leading and trailing spaces are stripped.	`0`, or, if `0` is not a valid value for the property, the property's minimum value.
`length`	`<length>`	The attribute value is parsed as a CSS `<length>` dimension, that is including the unit (e.g. `12.5em`). If it is not valid, that is not a length or out of the range accepted by the CSS property, the default value is used. If the given unit is a relative length,	`0`, or, if `0` is not a valid value for the property, the property's minimum value.

Palavra-chave	Tipo Associado	Comentário	Valor Padrão
		`attr()`computes it to an absolute length. Leading and trailing spaces are stripped.	
`em`, `ex`, `px`, `rem`, `vw`, `vh`, `vmin`, `vmax`, `mm`, `cm`, `in`, `pt`, or `pc`	`<length>`	The attribute value is parsed as a CSS `<number>`, that is without the unit (e.g. `12.5`), and interpreted as a `<length>` with the specified unit. If it is not valid, that is not a number or out of the range accepted by the CSS property, the default value is used. If the given unit is a relative length, `attr()`computes it to an absolute length. Leading and trailing spaces are stripped.	`0`, or, if `0` is not a valid value for the property, the property's minimum value.
`angle`	`<angle>`	The attribute value is parsed as a CSS `<angle>` dimension, that is including the unit (e.g. `30.5deg`). If it is not valid, that is not an angle or out of the range accepted by the CSS property, the default value is used. Leading and trailing spaces are stripped.	`0deg`, or, if `0deg` is not a valid value for the property, the property's minimum value.
`deg`, `grad`, `rad`	`<angle>`	The attribute value is parsed as a CSS `<number>`, that is without the unit (e.g. `12.5`), and interpreted as an `<angle>` with the specified unit. If it is not valid, that is not a number or out of the range accepted by the CSS property, the default value is used. Leading and trailing spaces are stripped.	`0deg`, or, if `0deg` is not a valid value for the property, the property's minimum value.
`time`	`<time>`	The attribute value is parsed as a CSS `<time>` dimension, that is including the unit (e.g. `30.5ms`). If it is not valid, that is not a time or out of the range accepted by the CSS property, the default value is used. Leading and trailing spaces are stripped.	`0s`, or, if `0s` is not a valid value for the property, the property's minimum value.
`s`, `ms`	`<time>`	The attribute value is parsed as a CSS `<number>`, that is without the unit (e.g. `12.5`), and interpreted as an`<time>` with the specified unit. If it is not valid, that is not a number or out of the range accepted by the CSS property, the default value is used. Leading and trailing spaces are stripped.	`0s`, or, if `0s` is not a valid value for the property, the property's minimum value.
`frequency`	`<frequency>`	The attribute value is parsed as a CSS `<frequency>` dimension, that is including the unit (e.g. `30.5kHz`). If it is not valid, that is not a frequency or out of the range accepted by the CSS property, the default value is used.	`0Hz`, or, if `0Hz` is not a valid value for the property, the property's minimum value.
`Hz`, `kHz`	`<frequency>`	The attribute value is parsed as a CSS `<number>`, that is without the unit (e.g. `12.5`), and interpreted as a `<frequency>` with the specified unit. If it is not valid, that is not a number or out of the range accepted by the CSS property, the default value is used. Leading and trailing spaces are stripped.	`0Hz`, or, if `0Hz` is not a valid value for the property, the property's minimum value.
`%`	`<percentage>`	The attribute value is parsed as a CSS `<number>`, that is without the unit (e.g. `12.5`), and interpreted as a `<percentage>`. If it is not valid, that is not a number or out of the range accepted by the CSS property, the default value is used. If the given value is used as a length, `attr()`computes it to an absolute length. Leading and trailing spaces are stripped.	`0%`, or, if `0%` is not a valid value for the property, the property's minimum value.

`<fallback>`

The value to be used if the associated attribute is missing or contains an invalid value. The fallback value must be valid where `attr()` is used, even if it is not used, and must not contain another `attr()` expression. If `attr()` is not the sole component value of a property, its `<fallback>` value must be of the type defined by `<type-or-unit>`. If not set, CSS will use the default value defined for each `<type-or-unit>`.

Sintaxe formal

```
<attr()> =
  attr( <attr-name> <attr-type>? , <declaration-value>? )

<attr-name> =
  [ <ident-token>? '|' ]? <ident-token>

<attr-type> =
  type( <syntax> )  |
```

```
  raw-string        |
  number            |
  <attr-unit>

<syntax> =
  '*'                                                |
  <syntax-component> [ <syntax-combinator> <syntax-component> ]*  |
  <syntax-string>

<attr-unit> =
  <custom-ident>

<syntax-component> =
  <syntax-single-component> <syntax-multiplier>?  |
  '<' transform-list '>'

<syntax-combinator> =
  '|'

<syntax-string> =
  <string>

<syntax-single-component> =
  '<' <syntax-type-name> '>'  |
  <ident>

<syntax-multiplier> =
  '#'  |
  '+'

<syntax-type-name> =
  angle               |
  color               |
  custom-ident        |
  image               |
  integer             |
  length              |
  length-percentage   |
  number              |
  percentage          |
  resolution          |
  string              |
  time                |
  url                 |
  transform-function
```

Esta sintaxe reflete o padrão mais recente conforme CSS Values and Units Module Level 5. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

HTML

```
<p data-foo="hello">world</p>
```

CSS

```
p::before {
  content: attr(data-foo) " ";
}
```

Resultado

<blend-mode>

O tipo básico de dados CSS **<blend-mode>** (modo de mesclagem) descreve como as cores devem aparecer, quando objetos se sobrepõem. Usa-se nas propriedades `background-blend-mode` e `mix-blend-mode`.

Para cada pixel sobreposto, o **blend-mode** obtém o valor da cor do pixel da frente e do pixel do fundo, faz um cálculo com esses valores e devolve um novo valor de cor.

Syntaxe

O tipo básico de dados **<blend-mode>** é definido usando uma palavra chave escolhida da lista abaixo.

Valores

`normal`

A cor firnal é a do pixel de cima, não importando a cor dor o pixel de baixo. O efeito é como duas folhas de papel opacas se sobrepondo.

```
<div id="div"></div>
```

```
#div {
  width: 300px;
  height: 300px;
  background: url("br.png"), url("tr.png");
  background-blend-mode: normal;
}
```

`multiply`

A cor final é o resultado da multiplicação das cores de cima e de baixo. Quando uma das camadas for preta, a cor final será preta, enquanto uma camada branca não altera a cor final. O efeito é como duas imagens impressas sobrepostas em um filme transparente.

```
<div id="div"></div>
```

```
#div {
  width: 300px;
  height: 300px;
  background: url("br.png"), url("tr.png");
  background-blend-mode: multiply;
}
```

`screen`

A cor final é o resultado do inverso de cada cor, multiplicados um pelo outro e depois, o inverso desse resultado. Uma camada preta não altera o resultado final, enquanto uma camada branca resulta na cor final branca. O efeito é como o de duas imagens sobrepostas por um projetor, sobre uma tela de projeção.

```
<div id="div"></div>
```

```
#div {
  width: 300px;
  height: 300px;
  background: url("br.png"), url("tr.png");
  background-blend-mode: screen;
}
```

`overlay`

A cor final será o resultado de `multiply`, se a cor de baixo for mais escura ou `screen`, se a cor de baixo for mais clara. Este `blend-mode` é equivalente a `hard-light` mas com as camadas invertidas.

```
<div id="div"></div>
```

```
#div {
  width: 300px;
  height: 300px;
  background: url("br.png"), url("tr.png");
  background-blend-mode: overlay;
}
```

`darken`

A cor final é composta pelos valores mais escuros dos canais de cores de cada uma das camadas.

```
<div id="div"></div>
```

```
#div {
  width: 300px;
  height: 300px;
  background: url("br.png"), url("tr.png");
```

```
  background-blend-mode: darken;
}
```

`lighten`

A cor final é composta pelos valores mais claros de cada um dos canais de cores das camadas.

```
<div id="div"></div>
```

```
#div {
  width: 300px;
  height: 300px;
  background: url("br.png"), url("tr.png");
  background-blend-mode: lighten;
}
```

`color-dodge`

A cor final é o resultado da divisão da cor de baixo pelo inverso da cor de cima. Um fundo preto não altera as cores. Uma cor de cima inversa a cor de baixo, resulta em uma cor bastante luminosa. Este `blend-mode` é similar a `screen`, mas a cor de cima precisa ser apenas tão clara quanto o inverso da cor de fundo para deixar o resultado muito luminoso.

```
<div id="div"></div>
```

```
#div {
  width: 300px;
  height: 300px;
  background: url("br.png"), url("tr.png");
  background-blend-mode: color-dodge;
}
```

`color-burn`

A cor final é o inverso da cor do fundo, dividido pelo valor da cor da frente e então inverte-se o resultado final. Se a cor de cima for branca, não haverá alteração. Se a cor de cima for o inverso da cor de baixo, resultará em uma cor preta. Este `blend-mode` é similar ao `multiply`, mas a cor de cima precisa apenas ser tão escura quanto o inverso da cor de baixo para fazer o resultado final ser preto.

```
<div id="div"></div>
```

```
#div {
  width: 300px;
  height: 300px;
  background: url("br.png"), url("tr.png");
  background-blend-mode: color-burn;
}
```

`hard-light`

A cor final é o resultado do `multiply` se a cor de cima for a mais escura, ou o resultado de `screen` se a cor de cima for mais clara. Este `blend-mode` é equivalente ao `overlay` mas com as camadas invertidas. O efeito é o de iluminar o fundo com um holofote muito forte e intenso.

```
<div id="div"></div>
```

```
#div {
  width: 300px;
  height: 300px;
  background: url("br.png"), url("tr.png");
  background-blend-mode: hard-light;
}
```

`soft-light`

O resultado final é similar ao de `hard-light`, porém menos intenso. O efeito é o de iluminar o fundo com um holofote difuso.

```
<div id="div"></div>
```

```
#div {
  width: 300px;
```

```
  height: 300px;
  background: url("br.png"), url("tr.png");
  background-blend-mode: soft-light;
}
```

`difference`

The final color is the result of subtracting the darker of the two colors from the lighter one. A black layer has no effect, while a white layer inverts the other layer's color.

```
<div id="div"></div>
```

```
#div {
  width: 300px;
  height: 300px;
  background: url("br.png"), url("tr.png");
  background-blend-mode: difference;
}
```

`exclusion`

The final color is similar to `difference`, but with less contrast. As with `difference`, a black layer has no effect, while a white layer inverts the other layer's color.

```
<div id="div"></div>
```

```
#div {
  width: 300px;
  height: 300px;
  background: url("br.png"), url("tr.png");
  background-blend-mode: exclusion;
}
```

`hue`

The final color has the *hue* of the top color, while using the *saturation* and *luminosity* of the bottom color.

```
<div id="div"></div>
```

```
#div {
  width: 300px;
  height: 300px;
  background: url("br.png"), url("tr.png");
  background-blend-mode: hue;
}
```

`saturation`

The final color has the *saturation* of the top color, while using the *hue* and *luminosity* of the bottom color. A pure gray backdrop, having no saturation, will have no effect.

```
<div id="div"></div>
```

```
#div {
  width: 300px;
  height: 300px;
  background: url("br.png"), url("tr.png");
  background-blend-mode: saturation;
}
```

`color`

The final color has the *hue* and *saturation* of the top color, while using the *luminosity* of the bottom color. The effect preserves gray levels and can be used to colorize the foreground.

```
<div id="div"></div>
```

```
#div {
  width: 300px;
  height: 300px;
  background: url("br.png"), url("tr.png");
  background-blend-mode: color;
}
```

`luminosity`

The final color has the *luminosity* of the top color, while using the *hue* and *saturation* of the bottom color. This blend mode is equivalent to `color`, but with the layers swapped.

```
<div id="div"></div>
```

```
#div {
  width: 300px;
  height: 300px;
  background: url("br.png"), url("tr.png");
  background-blend-mode: luminosity;
}
```

Interpolation of blend modes

Changes between blends mode are not interpolated. Any change occurs immediately.

See also

- Properties that use this data type: `background-blend-mode`, `mix-blend-mode`
- Blend modes

calc()

A função **`calc()`** CSS permite você executar cálculos quando especificar os valores de propriedades no CSS. Pode ser utilizada em qualquer lugar , como `<length>`, `<frequency>`, `<angle>`, `<time>`, `<percentage>`, `<number>`, e `<integer>` é permitido.

```
/* propriedade: calc(expressão) */
width: calc(100% - 80px);
```

Syntax

A função `calc()` recebe uma simples expressão como parâmetro e o resultado desta expressão é utilizado como valor. Pode ser uma simples expressão, combinando os seguintes operadores, utilizando padrão operator precedence rules:

`+`

Adição.

`-`

Subtração.

`*`

Multiplicação. Pelo menos um dos argumentos deve ser um `<number>`.

`/`

Divisão. O operador da direita deve ser um `<number>`.

Os operandos na expressão podem ser qualquer valor de sintaxe `<length>`. Você pode usar unidades diferentes para cada valor em sua expressão, se desejar. Você também pode usar parênteses para estabelecer a ordem de computação quando necessário.

Notas

- Divisão por 0 (zero) resulta em um erro gerado pelo HTML parser.
- Os operadores + e - devem estar cercados por **espaço em branco**. Por exemplo, calc (50% -8px) será analisado como uma porcentagem seguida por um comprimento negativo - uma expressão inválida — enquanto calc (50% - 8px) é uma porcentagem seguida por um operador de subtração e um comprimento. Da mesma forma, calc (8px + -50%) é tratado como um comprimento seguido por um operador de adição e uma porcentagem negativa.

- Os operadores `*` e `/` não requerem espaço em branco, mas adicioná-lo para consistência é permitido e recomendado.
- Expressões matemáticas envolvendo porcentagens de larguras e alturas em colunas de tabela, grupos de coluna de tabela, linhas de tabela, grupos de linhas de tabela e células de tabela em tabelas de layout automáticas e fixas podem ser tratadas como se `auto` tivesse sido especificado.
- É permitido aninhar funções `calc ()`, em cujo caso as internas são tratadas como parênteses simples.

Formal syntax

```
<calc()> =
  calc( <calc-sum> )

<calc-sum> =
  <calc-product> [ [ '+' | '-' ] <calc-product> ]*

<calc-product> =
  <calc-value> [ [ '*' | / ] <calc-value> ]*

<calc-value> =
  <number>        |
  <dimension>     |
  <percentage>    |
  <calc-keyword>  |
  ( <calc-sum> )

<calc-keyword> =
  e          |
  pi         |
  infinity   |
  -infinity  |
  NaN
```

Esta sintaxe reflete o padrão mais recente conforme CSS Values and Units Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

Posicionando um objeto na tela usando margin

`calc()` torna mais fácil posicionar um objeto com uma margem definida. Nesse exemplo, o CSS cria um banner que se estende pela janela, com um espaço de 40px entre os dois lados do banner e das bordas da janela:

```
.banner {
  position: absolute;
  left: 40px;
  width: calc(100% - 80px);
  border: solid black 1px;
  box-shadow: 1px 2px;
  background-color: yellow;
  padding: 6px;
  text-align: center;
  box-sizing: border-box;
}
```

```
<div class="banner">This is a banner!</div>
```

Dimensionar campos de formulário automaticamente para caber em seu contêiner

Outro caso para `calc()` é ajudar a garantir que os campos do formulário caibam no espaço disponível, sem expandir para além da borda do seu contêiner, enquanto mantém uma margem apropriada.

Veja-mos no CSS:

```
input {
  padding: 2px;
  display: block;
  width: calc(100% - 1em);
}

#formbox {
  width: calc(100% / 6);
  border: 1px solid black;
```

```
  padding: 4px;
}
```

Aqui, o próprio formulário é estabelecido para utilizar 1/6 da largura da janela disponível. Então, para garantir que os campos de entrada mantenham um tamanho apropriado, utilizamos `calc()` novamente para estabelecer que eles devem ter a largura de seu contêiner menos 1em. Então, o seguinte HTML utiliza este CSS:

```
<form>
  <div id="formbox">
    <label>Type something:</label>
    <input type="text" />
  </div>
</form>
```

calc( ) inserido em variáveis de CSS

Você também pode utilizar `calc()` com variáveis de CSS. Considere o seguinte código:

```
.foo {
  --widthA: 100px;
  --widthB: calc(var(--widthA) / 2);
  --widthC: calc(var(--widthB) / 2);
  width: var(--widthC);
}
```

Depois que todas as variáveis forem expandidas, o valor de `widthC` será `calc( calc( 100px / 2) / 2)`, então quando for atribuído à propriedade de largura `.foo`, todos os `calc()` internos (não importa o quão profundamente atribuídos) serão nivelados para apenas parênteses, de modo que o valor da propriedade `width` será eventualmente `calc( ( 100px / 2) / 2)`, i.e. `25px`. Resumindo: um `calc()` dentro de um `calc()` é idêntico à parênteses.

Questões de acessibilidade

Quando `calc()` é usado para controlar o tamanho do texto, certifique-se de que um dos valores inclui uma unidade de comprimento relativo, por exemplo:

```
h1 {
  font-size: calc(1.5rem + 3vw);
}
```

Isso garante que o tamanho do texto será redimensionado se a página for ampliada.

- MDN Understanding WCAG, Guideline 1.4 explanations
- Understanding Success Criterion 1.4.4 | W3C Understanding WCAG 2.0

Veja também

- Firefox 4: CSS3 calc() ☆ Mozilla Hacks – the Web developer blog

<color>

Sumário

O tipo de dado CSS `<color>` indica uma cor no espaço sRGB. Uma cor pode ser descrita em qualquer uma destas formas:

- usando uma *palavra-chave*
- usando o sistema de coordenada-cúbica RGB (via #-hexadecimal ou das notações funcional `rgb()` e `rgba()`)
- usando o sistema de coordenada-cilíndrica HSL (via `hsl()` e notações funcionais `hsla()`)

Observe que a lista de valores de cores aceitas foi estendida à medida que a especificação evoluiu, culminando com as cores CSS3 mais recentes.

Associado à cor no espaço sRGB, um valor <color> também pode consistir de uma coordenada de *valor de transparência* canal-alfa, Indicando como a cor deve se composta com sua cor de fundo.

Embora os valores de cores CSS sejam definidos com precisão, eles podem aparecer de forma diferente em dispositivos de saída diferentes. A maioria deles não está calibrada e alguns navegadores não suportam o perfil de cores dos dispositivos de saída. Sem estes, a renderização de cores pode variar muito.

> **Nota:** A recomendação WCAG 2.0 do W3C aconselha fortemente aos autores web não usarem cor como o único meio para transmitir uma mensagem específica, ação ou resultado. Alguns usuários têm problemas em distinguir cores e as informações transmitidas podem não ser compreendidas. Claro, isso não impede o uso da cor, mas o seu uso apenas como o único meio para descrever certas coisas (veja Cor e contraste de cor para mais informações).

Interpolação

Valores do tipo de dados CSS `<color>` podem ser interpolados para realizar animações ou para criar valores <gradient>. Nesse caso, eles são interpolados em cada um de seus componentes vermelho, verde, azul, cada um manipulado como um número real, de ponto flutuante. Observe que a interpolação de cores ocorre no espaço de cores sRGBA alfa-pré-multiplicado para evitar que cores cinzentas inesperadas apareçam. Nas animações, a velocidade da interpolação é determinada pela função de temporização associada à animação.

Valores

Existem várias maneiras de descrever um valor `<color>`.

Palavras-chave de cores

As palavras-chave de cor são identificadores não sensíveis a maiúsculas e minúsculas que representam uma cor específica, ex.: `red`, `blue`, `brown`, ou `lightseagreen`. O nome descreve a cor, embora seja principalmente artificial. A lista de valores aceitos variou muito ao longo das diferentes especificações:

- CSS nível 1 aceita apenas16 cores básicas, denominadas *cores VGA* já que foram tiradas do conjunto de cores exibíveis das placas gráficas VGA.
- CSS nível 2 Adicionou a palavra-chave `orange`.
- Desde o início, os navegadores aceitaram outras cores, principalmente a lista de cores nomeadas X11 já que alguns dos primeiros navegadores foram aplicações X11, embora com algumas diferenças. O SVG 1.0 foi o primeiro padrão a definir formalmente essas palavras-chave; As cores no CSS nível 3 também formalmente definiram essas palavras-chave. Elas são muitas vezes referidas como *palavras-chave de cor estendida*, *cores X11* ou *cores SVG*.

Existem algumas restrições a serem consideradas ao usar palavras-chave:

- Com exceção das 16 cores básicas comuns com HTML, as outras não podem ser usadas em HTML. O HTML vai converter esses valores desconhecidos com um algoritmo específico levando a cores completamente diferentes. Essas palavras-chave só devem ser usadas em SVG & CSS.
- Palavras-chave desconhecidas tornam a propriedade CSS inválida. Propriedades inválidas sendo ignoradas, a cor não terá efeito. Esse é um comportamento diferente do HTML.
- Nenhuma palavra-chave de core definida em CSS têm qualquer transparência — eles são cores simples, sólidas.
- Várias palavras-chave indicam as mesmas cores:
  - `darkgray` / `darkgrey`
  - `darkslategray` / `darkslategrey`
  - `dimgray` / `dimgrey`
  - `lightgray` / `lightgrey`
  - `lightslategray` / `lightslategrey`
  - `gray` / `grey`
  - `slategray` / `slategrey`
- Embora os nomes das palavras-chave tenham sido tomadas pelos nomes de cores X11 habituais, a cor pode divergir da cor do sistema correspondente em sistemas X11 como estes são adaptados para o hardware específico pelo fabricante.

Especificação	Cor	Palavra-chave	valores hex RGB	Resultado
CSS Level 1		`black`	`#000000`	
		`silver`	`#c0c0c0`	
		`gray`	`#808080`	
		`white`	`#ffffff`	
		`maroon`	`#800000`	
		`red`	`#ff0000`	

Especificação	Cor	Palavra-chave	valores hex RGB	Resultado
		purple	#800080	
		fuchsia	#ff00ff	
		green	#008000	
		lime	#00ff00	
		olive	#808000	
		yellow	#ffff00	
		navy	#000080	
		blue	#0000ff	
		teal	#008080	
		aqua	#00ffff	
CSS Level 2 (Revision 1)		orange	#ffa500	
CSS Color Module Level 3		aliceblue	#f0f8ff	
		antiquewhite	#faebd7	
		aquamarine	#7fffd4	
		azure	#f0ffff	
		beige	#f5f5dc	
		bisque	#ffe4c4	
		blanchedalmond	#ffebcd	
		blueviolet	#8a2be2	
		brown	#a52a2a	
		burlywood	#deb887	
		cadetblue	#5f9ea0	
		chartreuse	#7fff00	
		chocolate	#d2691e	
		coral	#ff7f50	
		cornflowerblue	#6495ed	
		cornsilk	#fff8dc	
		crimson	#dc143c	
		cyan	#00ffff	
		darkblue	#00008b	
		darkcyan	#008b8b	
		darkgoldenrod	#b8860b	
		darkgray	#a9a9a9	
		darkgreen	#006400	
		darkgrey	#a9a9a9	
		darkkhaki	#bdb76b	
		darkmagenta	#8b008b	
		darkolivegreen	#556b2f	
		darkorange	#ff8c00	
		darkorchid	#9932cc	
		darkred	#8b0000	

Especificação	Cor	Palavra-chave	valores hex RGB	Resultado
		darksalmon	#e9967a	
		darkseagreen	#8fbc8f	
		darkslateblue	#483d8b	
		darkslategray	#2f4f4f	
		darkslategrey	#2f4f4f	
		darkturquoise	#00ced1	
		darkviolet	#9400d3	
		deeppink	#ff1493	
		deepskyblue	#00bfff	
		dimgray	#696969	
		dimgrey	#696969	
		dodgerblue	#1e90ff	
		firebrick	#b22222	
		floralwhite	#fffaf0	
		forestgreen	#228b22	
		gainsboro	#dcdcdc	
		ghostwhite	#f8f8ff	
		gold	#ffd700	
		goldenrod	#daa520	
		greenyellow	#adff2f	
		grey	#808080	
		honeydew	#f0fff0	
		hotpink	#ff69b4	
		indianred	#cd5c5c	
		indigo	#4b0082	
		ivory	#fffff0	
		khaki	#f0e68c	
		lavender	#e6e6fa	
		lavenderblush	#fff0f5	
		lawngreen	#7cfc00	
		lemonchiffon	#fffacd	
		lightblue	#add8e6	
		lightcoral	#f08080	
		lightcyan	#e0ffff	
		lightgoldenrodyellow	#fafad2	
		lightgray	#d3d3d3	
		lightgreen	#90ee90	
		lightgrey	#d3d3d3	
		lightpink	#ffb6c1	
		lightsalmon	#ffa07a	
		lightseagreen	#20b2aa	

Especificação	Cor	Palavra-chave	valores hex RGB	Resultado
		lightskyblue	#87cefa	
		lightslategray	#778899	
		lightslategrey	#778899	
		lightsteelblue	#b0c4de	
		lightyellow	#ffffe0	
		limegreen	#32cd32	
		linen	#faf0e6	
		mediumaquamarine	#66cdaa	
		mediumblue	#0000cd	
		mediumorchid	#ba55d3	
		mediumpurple	#9370db	
		mediumseagreen	#3cb371	
		mediumslateblue	#7b68ee	
		mediumspringgreen	#00fa9a	
		mediumturquoise	#48d1cc	
		mediumvioletred	#c71585	
		midnightblue	#191970	
		mintcream	#f5fffa	
		mistyrose	#ffe4e1	
		moccasin	#ffe4b5	
		navajowhite	#ffdead	
		oldlace	#fdf5e6	
		olivedrab	#6b8e23	
		orangered	#ff4500	
		orchid	#da70d6	
		palegoldenrod	#eee8aa	
		palegreen	#98fb98	
		paleturquoise	#afeeee	
		palevioletred	#db7093	
		papayawhip	#ffefd5	
		peachpuff	#ffdab9	
		peru	#cd853f	
		pink	#ffc0cb	
		plum	#dda0dd	
		powderblue	#b0e0e6	
		rosybrown	#bc8f8f	
		royalblue	#4169e1	
		saddlebrown	#8b4513	
		salmon	#fa8072	
		sandybrown	#f4a460	
		seagreen	#2e8b57	

Especificação	Cor	Palavra-chave	valores hex RGB	Resultado
		seashell	#fff5ee	
		sienna	#a0522d	
		skyblue	#87ceeb	
		slateblue	#6a5acd	
		slategray	#708090	
		slategrey	#708090	
		snow	#fffafa	
		springgreen	#00ff7f	
		steelblue	#4682b4	
		tan	#d2b48c	
		thistle	#d8bfd8	
		tomato	#ff6347	
		turquoise	#40e0d0	
		violet	#ee82ee	
		wheat	#f5deb3	
		whitesmoke	#f5f5f5	
		yellowgreen	#9acd32	
CSS Color Module Level 4		rebeccapurple	#663399	

A cor `rebeccapurple` é equivalente à cor `#639`, e mais informações sobre por que foi introduzida pode ser encontrada neste post no blog Codepen por Trezy "Honoring a Great Man" (em inglês)

<code>palavra-chave transparent</code>

A palavra-chave `transparent` representa uma cor totalmente transparente, isto é, a cor vista será a cor de fundo. Tecnicamente, é um preto com canal alfa no seu valor mínimo e é um atalho para `rgba(0,0,0,0)`.

> **Nota: Nota histórica** A palavra-chave `transparent` não era uma cor verdadeira no CSS nível 2 (Revisão 1). Era uma palavra-chave específica que poderia ser usada no lugar de um valor `<color>` regular em duas propriedades CSS: `background` e `border`. Ela foi adicionada essencialmente para permitir a substituição de cores sólidas herdadas.
>
> Com o apoio da opacidade através de canais alfal, `transparent` foi redefinida como uma cor verdadeira no Cores CSS nível 3 permitindo seu uso em qualquer lugar onde for exigido um valor para `<color>`, como a propriedade `color`.

<code>palavra-chave currentcolor</code>

A palavra-chave `currentcolor` representa o valor calculado da propriedade `color` dos elementos . Ela permite que as propriedades de cor sejam herdadas por propriedades ou propriedades de elemento filho que não a herdam por padrão.

Ela também pode ser usada em propriedades que herdam o valor calculado da propriedade `color` do elemento e será equivalente à palavra-chave `inherit` nesses elementos, se houver.

Exemplo ao vivo

A cor da linha (uma div preenchida com cor) se adapta à cor de sua propriedade `color`, herdada de seu pai.

Exemplo 1

```
<div style="color:darkred">
  A cor deste texto é a mesma da linha:
  <div style="background:currentcolor; height:1px"></div>
  Mais algum texto.
</div>
```

Exemplo 2

```
<div style="color:blue; border-bottom: 1px dashed currentcolor;">
  A cor deste texto é a mesma da linha:
  <div style="background:currentcolor; height:1px"></div>
  Mais algum texto.
</div>
```

<code>rgb()</code>

> **Nota:** Na especificação Módulo de cor CSS Color nível 4, a rgba() foi definida como uma função herdada com gramática e comportamento idênticos à rgb(); na verdade, um apelido. Mais para frente, ambas podem aceitar exatamente os mesmos parâmetros.

RGB colors can be expressed through both hexadecimal (prefixed with `#`) and functional (`rgb()`, `rgba()`) notations.

Hexadecimal notation: `#RGB[A]` or`#RRGGBB[AA]`

> `R` (red), `G` (green), `B` (blue), and `A` (alpha) are hexadecimal characters (0–9, A–F). `A` is optional. For example, `#ff0000` is equivalent to `#ff0000ff`. The three-digit notation (`#RGB`) is a shorter version of the six-digit form (`#RRGGBB`). For example, `#f09` is the same color as `#ff0099`. Likewise, the four-digit RGB notation (`#RGBA`) is a shorter version of the eight-digit form (`#RRGGBBAA`). For example, `#0f38` is the same color as `#00ff3388`.

`rgb()` or `rgba()`: `rgb[a](R, G, B[, A])` or `rgb[a](R G B[ / A])`

> `R` (red), `G` (green), and `B` (blue) can be either `<number>`s or `<percentage>`s, where the number `255` corresponds to `100%`. `A` (alpha) can be a `<number>` between `0` and `1`, or a `<percentage>`, where the number `1` corresponds to `100%` (full opacity).

```
/* These examples all specify the same RGB color: */

#f03
#F03
#ff0033
#FF0033
rgb(255,0,51)
rgb(255, 0, 51)
rgb(255, 0, 51.2) /* ERROR! Don't use fractions, use integers */
rgb(100%,0%,20%)
rgb(100%, 0%, 20%)
rgb(100%, 0, 20%) /* ERROR! Don't mix up integer and percentage notation */

/* whitespace syntax */
rgb(255 0 0)

/* optional alpha value */
rgb(255,0,0,0.4)  /* 40% opaque red */
rgb(255,0,0,40%)  /* 40% opaque red with percentage value for alpha */
rgb(255 0 0 / 0.4)  /* 40% opaque red */
rgb(255 0 0 / 40%)  /* 40% opaque red with percentage value for alpha */
```

<code>hsl()</code>

> **Nota:** In the CSS Color Module Level 4 spec, `hsla()` has been defined as a legacy function with identical grammar and behaviour to `hsl()`; in effect, an alias. Going forward, both can accept exactly the same parameters.

Colors can also be defined via hue, saturation, and lightness, or HSL, by using the `hsl()` functional notation. The advantage of HSL over RGB is that it is far more intuitive: you can guess at the colors you want, and then tweak. It is also easier to create sets of matching colors (by keeping the hue the same and varying the lightness/darkness, and saturation).

Hue is represented as an angle of the color circle (i.e. the rainbow represented in a circle). This angle is given as a unitless `<number>`. By definition **red=0=360**, and the other colors are spread around the circle, so **green=120**, **blue=240**, etc. As an angle, it implicitly wraps around such that -120=240 and 480=120.

Saturation and lightness are represented as percentages. `100%` is full **saturation**, and `0%` is a shade of grey. `100%` **lightness** is white, `0%` lightness is black, and `50%` lightness is "normal."

```
hsl(0,  100%,50%)    /* red */
hsl(30, 100%,50%)
```

```
hsl(60, 100%,50%)
hsl(90, 100%,50%)
hsl(120,100%,50%)    /* green */
hsl(150,100%,50%)
hsl(180,100%,50%)
hsl(210,100%,50%)
hsl(240,100%,50%)    /* blue */
hsl(270,100%,50%)
hsl(300,100%,50%)
hsl(330,100%,50%)
hsl(360,100%,50%)    /* red */

hsl(120,100%,25%)    /* dark green */
hsl(120,100%,50%)    /* green */
hsl(120,100%,75%)    /* light green */

hsl(120,100%,50%)    /* green */
hsl(120, 67%,50%)
hsl(120, 33%,50%)
hsl(120,  0%,50%)

hsl(120, 60%,70%)    /* pastel green */

/* whitespace syntax */
hsl(120 60% 70%)  /* pastel green */

/* angle values for hue component; also accepts rad, grad, turn */
hsl(120deg 60% 70%)  /* pastel green */

/* optional alpha value */
hsl(240,100%,50%,0.05)  /* 5% opaque blue */
hsl(240,100%,50%,5%)  /* 5% opaque blue with percentage value for alpha */
hsl(240 100% 50% / 0.05) /* 5% opaque blue */
hsl(240 100% 50% / 5%) /* 5% opaque blue with percentage value for alpha */
```

<code>rgba()</code>

RGB colors can be expressed through both hexadecimal (prefixed with `#`) and functional (`rgb()`, `rgba()`) notations.

Hexadecimal notation: `#RGB[A]` or`#RRGGBB[AA]`

> `R` (red), `G` (green), `B` (blue), and `A` (alpha) are hexadecimal characters (0–9, A–F). `A` is optional. For example, `#ff0000` is equivalent to `#ff0000ff`. The three-digit notation (`#RGB`) is a shorter version of the six-digit form (`#RRGGBB`). For example, `#f09` is the same color as `#ff0099`. Likewise, the four-digit RGB notation (`#RGBA`) is a shorter version of the eight-digit form (`#RRGGBBAA`). For example, `#0f38` is the same color as `#00ff3388`.

`rgb()` or `rgba()`: `rgb[a](R, G, B[, A])` or `rgb[a](R G B[ / A])`

> `R` (red), `G` (green), and `B` (blue) can be either `<number>`s or `<percentage>`s, where the number `255` corresponds to `100%`. `A` (alpha) can be a `<number>` between `0` and `1`, or a `<percentage>`, where the number `1` corresponds to `100%` (full opacity).

```
#f030               /* 0% opaque red */
#F03F               /* full opaque red */
#ff003300           /* 0% opaque red */
#FF003388           /* 50% opaque red */
rgba(255,0,0,0.1)   /* 10% opaque red */
rgba(255,0,0,0.4)   /* 40% opaque red */
rgba(255,0,0,0.7)   /* 70% opaque red */
rgba(255,0,0,  1)   /* full opaque red */

/* whitespace syntax */
rgba(255 0 0 / 0.4)  /* 40% opaque red */

/* percentage value for alpha */
rgba(255 0 0 / 40%)  /* 40% opaque red */
```

> **Nota:** In the CSS Color Module Level 4 spec, `rgba()` has been defined as a legacy function with identical grammar and behaviour to `rgb()`; in effect, an alias. Going forward, both can accept exactly the same parameters.

<code>hsla()</code>

Colors can be defined in the hue-saturation-lightness-alpha model (HSLa) using the `hsla()` functional notation. HSLa extends the HSL color model to include the alpha channel, allowing specification of the opacity of a color. **a** means opacity: 0=transparent; 1=opaque;

```
hsla(240,100%,50%,0.05)   /* 5% opaque blue */
hsla(240,100%,50%, 0.4)   /* 40% opaque blue */
hsla(240,100%,50%, 0.7)   /* 70% opaque blue */
hsla(240,100%,50%,   1)   /* full opaque blue */

/* whitespace syntax */
hsla(240 100% 50% / 0.05) /* 5% opaque blue */

/* percentage value for alpha */
hsla(240 100% 50% / 5%) /* 5% opaque blue */

/* angle values for hue component; also accepts rad, grad, turn */
hsla(240deg 100% 50% / 5%) /* 5% opaque blue */
hsla(240deg,100%,50%, 0.4)  /* 40% opaque blue */
```

> **Nota:** In the CSS Color Module Level 4 spec, `hsla()` has been defined as a legacy function with identical grammar and behaviour to `hsl()`; in effect, an alias. Going forward, both can accept exactly the same parameters.

System Colors

Not all system colors are supported on all systems. for use on public web pages.

ActiveBorder

: Active window border.

ActiveCaption

: Active window caption. Should be used with `CaptionText` as foreground color.

AppWorkspace

: Background color of multiple document interface.

Background

: Desktop background.

ButtonFace

: Face background color for 3-D elements that appear 3-D due to one layer of surrounding border. Should be used with the `ButtonText` foreground color.

ButtonHighlight

: The color of the border facing the light source for 3-D elements that appear 3-D due to that layer of surrounding border.

ButtonShadow

: The color of the border away from the light source for 3-D elements that appear 3-D due to that layer of surrounding border.

ButtonText

: Text on push buttons. Should be used with the `ButtonFace` or `ThreeDFace` background color.

CaptionText

: Text in caption, size box, and scrollbar arrow box. Should be used with the `ActiveCaption` background color.

GrayText

: Grayed (disabled) text.

Highlight

: Item(s) selected in a control. Should be used with the `HighlightText` foreground color.

HighlightText

Text of item(s) selected in a control. Should be used with the `Highlight` background color.

InactiveBorder

Inactive window border.

InactiveCaption

Inactive window caption. Should be used with the `InactiveCaptionText` foreground color.

InactiveCaptionText

Color of text in an inactive caption. Should be used with the `InactiveCaption` background color.

InfoBackground

Background color for tooltip controls. Should be used with the `InfoText` foreground color.

InfoText

Text color for tooltip controls. Should be used with the `InfoBackground` background color.

Menu

Menu background. Should be used with the `MenuText` or `-moz-MenuBarText` foreground color.

MenuText

Text in menus. Should be used with the `Menu` background color.

Scrollbar

Background color of scroll bars.

ThreeDDarkShadow

The color of the darker (generally outer) of the two borders away from the light source for 3-D elements that appear 3-D due to two concentric layers of surrounding border.

ThreeDFace

The face background color for 3-D elements that appear 3-D due to two concentric layers of surrounding border. Should be used with the `ButtonText` foreground color.

ThreeDHighlight

The color of the lighter (generally outer) of the two borders facing the light source for 3-D elements that appear 3-D due to two concentric layers of surrounding border.

ThreeDLightShadow

The color of the darker (generally inner) of the two borders facing the light source for 3-D elements that appear 3-D due to two concentric layers of surrounding border.

ThreeDShadow

The color of the lighter (generally inner) of the two borders away from the light source for 3-D elements that appear 3-D due to two concentric layers of surrounding border.

Window

Window background. Should be used with the `WindowText` foreground color.

WindowFrame

Window frame.

WindowText

Text in windows. Should be used with the `Window` background color.

Mozilla System Color Extensions

- -moz-ButtonDefault
  - : The border color that goes around buttons that represent the default action for a dialog box.
- -moz-ButtonHoverFace
  - : The background color of a button that the mouse pointer is over (which would be `ThreeDFace` or `ButtonFace` when the mouse pointer is not over it). Should be used with the `-moz-ButtonHoverText` foreground color.
- -moz-ButtonHoverText
  - : The text color of a button that the mouse pointer is over (which would be ButtonText when the mouse pointer is not over it). Should be used with the `-moz-ButtonHoverFace background` color.
- -moz-CellHighlight
  - : Background color for selected item in a tree widget. Should be used with the `-moz-CellHighlightText` foreground color. See also `-moz-html-CellHighlight`.
- -moz-CellHighlightText
  - : Text color for a selected item in a tree. Should be used with the `-moz-CellHighlight background` color. See also `-moz-html-CellHighlightText`.
- -moz-Combobox
  - : Background color for combo-boxes. Should be used with the `-moz-ComboboxText` foreground color. In versions prior to 1.9.2, use `-moz-Field` instead.
- -moz-ComboboxText
  - : Text color for combo-boxes. Should be used with the `-moz-Combobox` background color. In versions prior to 1.9.2, use `-moz-FieldText` instead.
- -moz-Dialog
  - : Background color for dialog boxes. Should be used with the `-moz-DialogText` foreground color.
- -moz-DialogText
  - : Text color for dialog boxes. Should be used with the `-moz-Dialog` background color.
- -moz-dragtargetzone

  -moz-EvenTreeRow
  - : Background color for even-numbered rows in a tree. Should be used with the `-moz-FieldText` foreground color. In Gecko versions prior to 1.9, use `-moz-Field`. See also `-moz-OddTreeRow`.
- -moz-Field
  - : Text field background color. Should be used with the `-moz-FieldText` foreground color.
- -moz-FieldText
  - : Text field text color. Should be used with the `-moz-Field`, `-moz-EvenTreeRow`, or `-moz-OddTreeRow` background color.
- -moz-html-CellHighlight
  - : Background color for highlighted item in HTML `<select>`s. Should be used with the `-moz-html-CellHighlightText` foreground color. Prior to Gecko 1.9, use `-moz-CellHighlight`.
- -moz-html-CellHighlightText
  - : Text color for highlighted items in HTML `<select>`s. Should be used with the `-moz-html-CellHighlight` background color. Prior to Gecko 1.9, use `-moz-CellHighlightText`.
- -moz-mac-accentdarkestshadow

  -moz-mac-accentdarkshadow

-moz-mac-accentface

-moz-mac-accentlightesthighlight

-moz-mac-accentlightshadow

-moz-mac-accentregularhighlight

-moz-mac-accentregularshadow

-moz-mac-chrome-active

- -moz-mac-chrome-inactive
- -moz-mac-focusring

-moz-mac-menuselect

-moz-mac-menushadow

-moz-mac-menutextselect

-moz-MenuHover

  - : Background color for hovered menu items. Often similar to `Highlight`. Should be used with the `-moz-MenuHoverText` or `-moz-MenuBarHoverText` foreground color.

- -moz-MenuHoverText
  - : Text color for hovered menu items. Often similar to `HighlightText`. Should be used with the `-moz-MenuHover` background color.
- -moz-MenuBarText
  - : Text color in menu bars. Often similar to `MenuText`. Should be used on top of `Menu` background.
- -moz-MenuBarHoverText
  - : Color for hovered text in menu bars. Often similar to `-moz-MenuHoverText`. Should be used on top of `-moz-MenuHover` background.
- -moz-nativehyperlinktext
  - : Default platform hyperlink color.
- -moz-OddTreeRow
  - : Background color for odd-numbered rows in a tree. Should be used with the `-moz-FieldText` foreground color. In Gecko versions prior to 1.9, use `-moz-Field`. See also `-moz-EvenTreeRow`.
- -moz-win-communicationstext
  - : Should be used for text in objects with `-moz-appearance: -moz-win-communications-toolbox;`.
- -moz-win-mediatext
  - : Should be used for text in objects with `-moz-appearance: -moz-win-media-toolbox`.

Mozilla Color Preference Extensions

-moz-activehyperlinktext

User's preference for text color of active links. Should be used with the default document background color.

-moz-default-background-color

User's preference for the document background color.

-moz-default-color

User's preference for the text color.

-moz-hyperlinktext

User's preference for the text color of unvisited links. Should be used with the default document background color.

-moz-visitedhyperlinktext

User's preference for the text color of visited links. Should be used with the default document background color.

Ver também

- The `opacity` property, allowing to define the transparency of color at the element level.
- The `color`, `background-color`, `border-color`, `outline-color`, `text-shadow`, `box-shadow` properties.

Tipos básicos de dados CSS

Tipos básicos de dados CSS definem valores típicos (incluindo palavras chaves e unidades) aceitos pelas propriedades e funções do CSS. Eles são um tipo especial de tipo de valor do componente.

Na sintaxe formal, os tipos de dados são indicados pela palavra chave aplicada entre os sinais de desigualdade "<" e ">".

Referência

- `<angle>`
- `<basic-shape>`
- `<blend-mode>`
- `<color>`
- `<custom-ident>`
- `<filter-function>`
- `<flex>`
- `<frequency>`
- `<gradient>`
- `<image>`
- `<integer>`
- `<length>`
- `<number>`
- `<percentage>`
- `<position>`
- `<ratio>`
- `<resolution>`
- `<shape-box>`
- `<single-transition-timing-function>`
- `<string>`
- `<time>`
- `<transform-function>`
- `<url>`

<display-box>

Essas palavras-chave definem se um elemento gera caixas de exibição.

Sintaxe

`contents`

Esses elementos não produzem uma caixa específica por si mesmos. Eles são substituídos por sua pseudo-caixa e suas caixas filho. Observe que a especificação CSS Display Level 3 define como o `contents`

o valor deve afetar "elementos incomuns" - elementos que não são renderizados apenas pelos conceitos da caixa CSS, como elementos substituídos. Consulte o Apêndice B: Efeitos da exibição: conteúdo em Elementos incomuns para obter mais detalhes.

Devido a um erro nos navegadores, atualmente isso remove o elemento da árvore de acessibilidade - os leitores de tela não analisam o que está dentro. Consulte a seção Preocupações com acessibilidade abaixo para obter mais detalhes.

`none`

Desativa a exibição de um elemento para que não tenha efeito no layout (o documento é renderizado como se o elemento não existisse). Todos os elementos descendentes também têm a exibição desativada. Para que um elemento ocupe o espaço que normalmente ocuparia, mas sem renderizar nada, use o `visibility` propriedade em vez disso.

Sintaxe formal

```
<display-box> =
  contents  |
  none
```

Esta sintaxe reflete o padrão mais recente conforme CSS Display Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

Neste primeiro exemplo, o parágrafo com uma classe de segredo é definido como `display: none`; a caixa e qualquer conteúdo agora não são renderizados.

display: none

HTML

```
<p>Visible text</p>
<p class="secret">Invisible text</p>
```

CSS

```
p.secret {
  display: none;
}
```

Resultado

display: contents

Neste exemplo, o externo `<div>` possui uma borda vermelha de 2 pixels e uma largura de 300 px. No entanto, também tem`display: contents` especificado, portanto, este `<div>` não será renderizado, a borda e a largura não serão mais aplicadas e o elemento filho será exibido como se o pai nunca tivesse existido.

HTML

```
<div class="outer">
  <div>Inner div.</div>
</div>
```

CSS

```
.outer {
  border: 2px solid red;
  width: 300px;
  display: contents;
}

.outer > div {
  border: 1px solid green;
}
```

Resultado

Preocupações com acessibilidade

As implementações atuais na maioria dos navegadores removerão da árvore de acessibilidade qualquer elemento com um `display` valor de `contents`. Isso fará com que o elemento - e em algumas versões do navegador, seus elementos descendentes - não sejam mais anunciados pela tecnologia de leitura de tela. Esse é um comportamento incorreto de acordo com a especificação CSSWG.

- Marcação mais acessível com display: conteúdo | Hidde de Vries
- Exibição: conteúdo não é uma redefinição de CSS | Adrian Roselli

Compatibilidade com navegadores

Veja também

- `display`
    - `<display-outside>`

    - `<display-inside>`
    - `<display-listitem>`
    - `<display-internal>`
    - `<display-legacy>`

- Exibição: conteúdo não é uma redefinição de CSS | Adrian Roselli
- Marcação mais acessível com display: contents - hiddedevries.nl

<display-inside>

Essas palavras-chave especificam o interior do elemento`display` type, que define o tipo de contexto de formatação que apresenta seu conteúdo (supondo que seja um elemento não substituído). Essas palavras-chave são usadas como valores do `display` e pode ser usada para fins legados como uma única palavra-chave ou conforme definido na especificação do Nível 3, juntamente com um valor do `<display-outside>` palavras-chave.

Sintaxe

O `<display-inside>` O valor é definido usando uma palavra-chave da lista abaixo:

`flow`

O elemento expõe seu conteúdo usando o layout de fluxo (layout em bloco e em linha). Se o seu tipo de exibição externo for

`inline` ou `run-in`,

e estiver participando de um contexto de formatação em bloco ou em linha, gera uma caixa em linha. Caso contrário, ele gera uma caixa de contêiner de bloco.

Dependendo do valor de outras propriedades como

`position`, `float`, ou `overflow`) e se ele próprio participa de um contexto de formatação em bloco ou embutido, ele estabelece um novo contexto de formatação de bloco (BFC) para seu conteúdo ou integra seu conteúdo ao contexto de formatação pai.

`flow-root`

O elemento gera uma caixa de elemento de bloco que estabelece um novo contexto de formatação de bloco, definindo onde está a raiz da formatação.

`table`

Esses elementos se comportam como HTML`<table>` elementos. Ele define uma caixa no nível do bloco.

`flex`

O elemento se comporta como um elemento de bloco e expõe seu conteúdo de acordo com o modelo flexbox.

`grid`

O elemento se comporta como um elemento de bloco e expõe seu conteúdo de acordo com o modelo de grade.

`ruby`

O elemento se comporta como um elemento embutido e apresenta seu conteúdo de acordo com o modelo de formatação ruby. Ele se comporta como o HTML correspondente`<ruby>` elementos.

> **Nota:** Navegadores que suportam a sintaxe de dois valores, encontrando apenas o valor interno, como quando `display: flex` or `display: grid` especificado, definirá seu valor externo como `block`. Isso resultará no comportamento esperado; por exemplo, se você especificar um elemento a ser `display: grid`, você esperaria que a caixa criada no contêiner da grade fosse uma caixa no nível do bloco.

Sintaxe formal

```
<display-inside> =
  flow       |
  flow-root  |
  table      |
  flex       |
  grid       |
  ruby
```

Esta sintaxe reflete o padrão mais recente conforme CSS Display Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplo

Neste exemplo, a caixa pai foi fornecida `display: flow-root` e assim estabelece um novo BFC e contém o item flutuado.

HTML

```
<div class="box">
  <div class="float">I am a floated box!</div>
  <p>I am content inside the container.</p>
</div>
```

CSS

```
.box {
  background-color: rgb(224, 206, 247);
  border: 5px solid rebeccapurple;
  display: flow-root;
}

.float {
  float: left;
  width: 200px;
  height: 150px;
  background-color: white;
  border: 1px solid black;
  padding: 10px;
}
```

Resultado

Compatibilidade com navegadores

Suporte a vários valores de palavras-chave

Suporte de fluxo-raiz

Suporte de tabela

Suporte de grade

Suporte de flex

Suporte de ruby

Veja também

- `display`
  - `<display-outside>`
  - `<display-listitem>`
  - `<display-internal>`
  - `<display-box>`
  - `<display-legacy>`
- Conceitos básicos do Flexbox
- Conceitos básicos de layout de grade

`<display-internal>`

Alguns modelos de layout, como `table` e `ruby` Alguns modelos de layout, como possuem uma estrutura interna complexa, com várias funções diferentes que seus filhos e descendentes podem preencher. Esta página define os valores de exibição "internos", que só têm significado nesse modo de layout específico.

Sintaxe

Salvo indicação em contrário, o tipo de exibição interno e o tipo de exibição externo de elementos que usam esses valores de exibição são definidos para a palavra-chave fornecida.

`table-row-group`

Esses elementos se comportam como `<tbody>` HTML elementos.

`table-header-group`

Esses elementos se comportam como `<thead>` HTML elementos.

`table-footer-group`

Esses elementos se comportam como `<tfoot>` HTML elementos.

`table-row`

Esses elementos se comportam como `<tr>` HTML elementos.

`table-cell`

Esses elementos se comportam como `<td>` HTML elementos.

`table-column-group`

Esses elementos se comportam como `<colgroup>` HTML elementos.

`table-column`

Esses elementos se comportam como `<col>` HTML elementos.

`table-caption`

Esses elementos se comportam como `<caption>` HTML elementos.

`ruby-base`

Esses elementos se comportam como `<rb>` HTML elementos.

`ruby-text`

Esses elementos se comportam como `<rt>` HTML elementos.

`ruby-base-container`

Esses elementos se comportam como `<rbc>` HTML elementos gerado como caixas anônimas.

`ruby-text-container`

Esses elementos se comportam como `<rtc>` HTML elementos.

Compatibilidade com navegadores

Suporte de valores de tabela

`table`, `table-cell`, `table-column`, `table-column-group`, `table-footer-group`, `table-header-group`, `table-row`, and `table-row-group`

Suporte de valores de rubi

`ruby`, `ruby-base`, `ruby-base-container`, `ruby-text`, and `ruby-text-container`

Veja também

- `display`
  - `<display-outside>`
  - `<display-inside>`
  - `<display-listitem>`
  - `<display-box>`
  - `<display-legacy>`

<display-legacy>

O CSS 2 usou uma sintaxe de palavra-chave única para o `display` , exigindo palavras-chave separadas para variantes em nível de bloco e em linha do mesmo modo de layout. Esta página detalha esses valores.

Sintaxe

`inline-block`

O elemento gera uma caixa de elemento de bloco que fluirá com o conteúdo circundante, como se fosse uma única caixa embutida (se comportando como um elemento substituído).

É equivalente a `inline flow-root`.

`inline-table`

O `inline-table` O valor não possui um mapeamento direto em HTML. Ele se comporta como um HTML`<table>` elemento, mas como uma caixa embutida, em vez de uma caixa no nível do bloco. Dentro da caixa da tabela há um contexto em nível de bloco.

É equivalente a `inline table`.

`inline-flex`

O elemento se comporta como um elemento embutido e apresenta seu conteúdo de acordo com o modelo flexbox.

É equivalente a `inline flex`.

`inline-grid`

O elemento se comporta como um elemento embutido e apresenta seu conteúdo de acordo com o modelo de grade.

É equivalente a `inline grid`.

Sintaxe formal

```
<display-legacy> =
  inline-block  |
  inline-table  |
  inline-flex   |
  inline-grid
```

Esta sintaxe reflete o padrão mais recente conforme CSS Display Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplo

No exemplo abaixo, estamos criando um contêiner flex inline com a palavra-chave herdada inline-flex.

HTML

```
<div class="container">
  <div>Flex Item</div>
  <div>Flex Item</div>
</div>

Not a flex item
```

CSS

```
.container {
  display: inline-flex;
}
```

Result

Na nova sintaxe, o contêiner flex inline seria criado usando dois valores, inline para o tipo de exibição externo e flex para o tipo de exibição interno.

```
.container {
  display: inline flex;
}
```

Compatibilidade com navegadores

Suporte de<code>inline-block</code>

Suporte de<code>inline-table</code>

Suporte de<code>inline-flex</code>

Suporte de<code>inline-grid</code>

Veja também

- `display`
  - `<display-outside>`
  - `<display-inside>`
  - `<display-listitem>`
  - `<display-internal>`
  - `<display-box>`

<display-listitem>

O `list-item` palavra-chave faz com que o elemento gere um`::marker` pseudoelemento com o conteúdo especificado por sua `list-style` propriedades (por exemplo, um marcador) junto com uma caixa principal do tipo especificado para seu próprio conteúdo.

Sintaxe

Um valor único de `list-item` fará com que o elemento se comporte como um item da lista. Isso pode ser usado junto com`list-style-type` e `list-style-position`.

`list-item` também pode ser combinado com qualquer `<display-outside>` palavra-chave e o `flow` ou `flow-root` `<display-inside>` palavras-chave.

> **Nota:** Nos navegadores que suportam a sintaxe de dois valores, se nenhum valor interno for especificado, o padrão será `flow`. Se nenhum valor externo for especificado, a caixa principal terá um tipo de exibição externo de`block`.

Sintaxe formal

```
<display-listitem> =
  <display-outside>?     &&
  [ flow | flow-root ]?  &&
  list-item

<display-outside> =
  block   |
  inline  |
  run-in
```

Esta sintaxe reflete o padrão mais recente conforme CSS Display Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplo

HTML

```
<div class="fake-list">I will display as a list item</div>
```

CSS

```
.fake-list {
  display: list-item;
  list-style-position: inside;
}
```

Resultado

Compatibilidade com navegadores

Veja também

- `display`
  - `<display-outside>`
  - `<display-inside>`
  - `<display-internal>`
  - `<display-box>`
  - `<display-legacy>`

<display-outside>

O `<display-outside>` as palavras-chave especificam a parte externa do elemento `display` type, que é essencialmente seu papel no layout de fluxo. Essas palavras-chave são usadas como valores do `display` e pode ser usada para fins legados como uma única palavra-chave ou conforme definido na especificação do Nível 3, juntamente com um valor do `<display-inside>` palavras-chave.

Sintaxe

O `<display-outside>` O valor é definido usando uma palavra-chave da lista abaixo:

Valor

`block`

O elemento gera uma caixa de elemento de bloco, gerando quebras de linha antes e depois do elemento, quando no fluxo normal.

`inline`

O elemento gera uma ou mais caixas de elementos embutidos que não geram quebras de linha antes ou depois de si mesmas. No fluxo normal, o próximo elemento estará na mesma linha se houver espaço

`run-in`

O elemento gera uma caixa de introdução. Se o irmão adjacente do elemento definido como`display: run-in` box é uma caixa de bloco, o `run-in` box se torna a primeira caixa embutida da caixa de bloco a seguir.

Os elementos de entrada funcionam como linhas ou blocos, dependendo dos elementos circundantes. Ou seja: Se a caixa de introdução contém uma caixa de bloco, o mesmo que bloco. Se uma caixa de bloco segue a caixa de introdução, a caixa de introdução torna-se a primeira caixa embutida da caixa de bloco. Se uma caixa embutida se seguir, a caixa de introdução se tornará uma caixa de bloco.

> **Nota:** Navegadores que suportam a sintaxe de dois valores, encontrando apenas o valor externo, como quando `display: block` ou `display: inline` especificado, definirá o valor interno como`flow`. Isso resultará no comportamento esperado; por exemplo, se você especificar um elemento a ser bloco, seria de esperar que os filhos desse elemento participassem do layout de fluxo normal em bloco e em linha.

Sintaxe formal

```
<display-outside> =
  block   |
  inline  |
  run-in
```

Esta sintaxe reflete o padrão mais recente conforme CSS Display Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplo

No exemplo a seguir, os elementos de extensão (normalmente exibidos como elementos embutidos) são definidos como `display: block` e, assim, divida em novas linhas e expanda para preencher seu contêiner na dimensão embutida.

HTML

```
<span>span 1</span> <span>span 2</span>
```

CSS

```
span {
  display: block;
  border: 1px solid rebeccapurple;
}
```

Resultado

Compatibilidade com navegadores

Veja também

- `display`
  - `<display-inside>`
  - `<display-listitem>`
  - `<display-internal>`
  - `<display-box>`
  - `<display-legacy>`
- Layout de bloco e embutido no fluxo normal
- Contextos de formatação explicados

element

> **Experimental: Esta é uma tecnologia experimental**
> Verifique a tabela de compatibilidade entre Navegadores cuidadosamente antes de usar essa funcionalidade em produção.

A função CSS **`element()`** define uma `<image>` gerada de um elemento HTML qualquer. Essa imagem é ***dinamica***, significando que se o elemento HTML for mudado, a propriedade CSS usando este valor será atualizada automaticamente.

Um cenario particularmente util para usar esta função seria colocar uma imagem em um elemento `<canvas>`, e então usar-lo como um *background*.

Em navegadores Gecko, você pode usar o metodo não padronizado `document.mozSetImageElement()` para mudar o elemento sendo usado como *background* por um elemento de *background* selecionado por CSS.

Sintaxe

```
element( id )
```

Onde:

`id`

É o ID de um elemento para usar como background, especificado usando o atributo HTML *#id* no elemento.

Exemplos

Esses exemplos podem ser vistos em tempo real em versões do Firefox que suportam `-moz-element()`.

Um exemplo de alguma maneira real

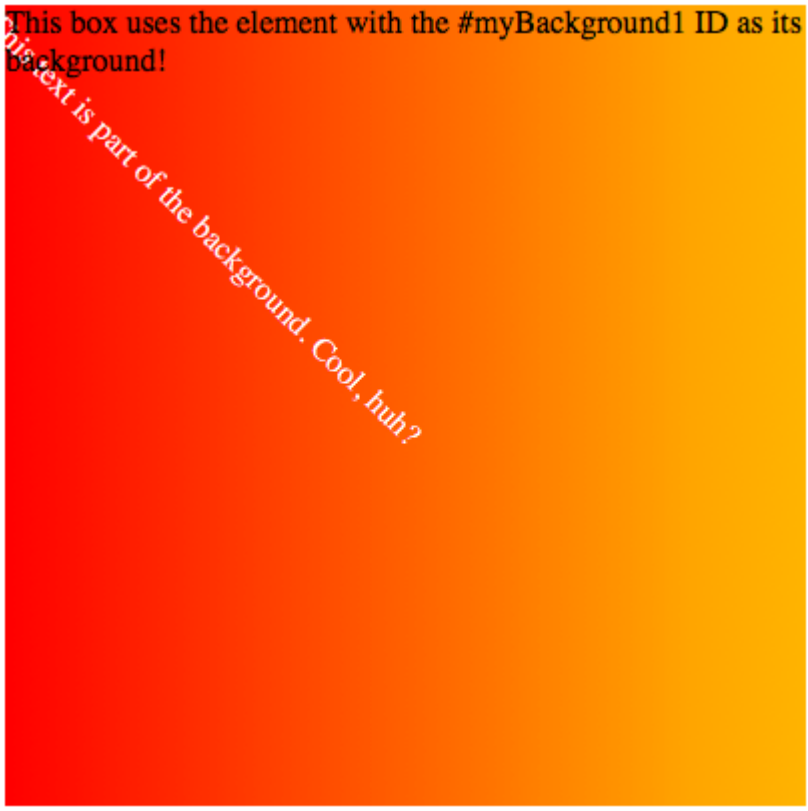
Esse exemplo usa uma `<div>` escondida como *background*. O elemento de *background* tem um gradiente, mas tambem um texto que é usado como parte do *background*.

```
<div
  style="width:400px; height:400px; background:-moz-element(#myBackground1) no-repeat;">
  <p>This box uses the element with the #myBackground1 ID as its background!</p>
</div>

<div style="overflow:hidden; height:0;">
  <div
    id="myBackground1"
    style="width:1024px; height:1024px; background-image: linear-gradient(to right, red, orange, yellow, white);">
    <p style="transform-origin:0 0; transform: rotate(45deg); color:white;">
      This text is part of the background. Cool, huh?
    </p>
```

```
  </div>
</div>
```

A `<div>` com ID "myBackground1" é usada como *background*, mantendo o paragrafo "This box uses the element with the #myBackground1 ID as its background!".

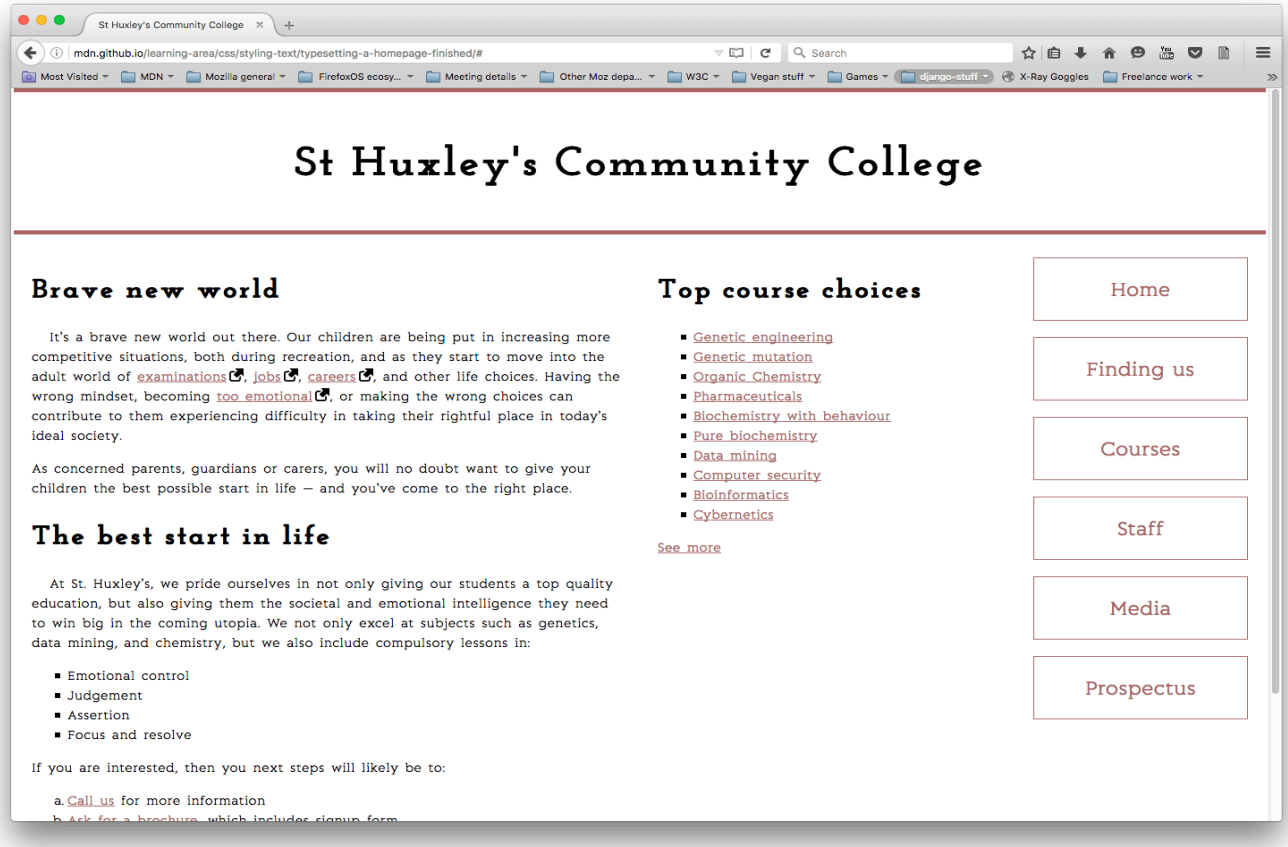


Um exemplo ainda mais bizarro

Esse exemplo usa um elemento `<button>` se repetindo como *background*. Isso mostra que você pode usar qualquer elemento como background, mas não mostra necessariamente boas praticas de design.

```
<div
  style="width:400px; height:100px; background:-moz-element(#myBackground2);"></div>

<div style="overflow:hidden; height:0;">
  <button id="myBackground2" type="button">Evil button!</button>
</div>
```



Veja tambem

- `document.mozSetImageElement()`

env()

A função CSS **`env()`** pode ser usada para inserir o valor de uma variável de ambiente definida pelo agente do usuário em seu CSS, de uma maneira semelhante à função `var()` e propriedades personalizadas. A diferença é que, além de serem definidas pelo agente do usuário em vez de serem definidas pelo usuário, as variáveis de ambiente têm escopo global para um documento, enquanto as propriedades personalizadas têm um escopo para o(s) elemento(s) no(s) qual(is) é/são declarado(s).

```
body {
  padding: env(safe-area-inset-top, 20px) env(safe-area-inset-right, 20px)
    env(safe-area-inset-bottom, 20px) env(safe-area-inset-left, 20px);
}
```

Além disso, ao contrário das propriedades personalizadas, que não podem ser usadas fora do escopo das declarações, a função `env()` pode ser usada em qualquer parte do valor de uma propriedade ou em qualquer parte de um *descriptor* (por exemplo, regras Media). À medida que a especificação evolui, essa também pode ser usada em outros lugares, como em seletores.

Originalmente fornecido pelo navegador iOS para permitir que os desenvolvedores coloquem seu conteúdo em uma *safe area* (área segura) na janela de visualização, os valores de `safe-area-inset-*` definidos na especificação podem ser usados para ajudar a garantir que o conteúdo seja visível até mesmo para usuários que usam uma tela não retangular.

Sintaxe

```
/* Usando os quatro valores de inserção safe area sem valores de fallback */
env(safe-area-inset-top);
env(safe-area-inset-right);
env(safe-area-inset-bottom);
env(safe-area-inset-left);

/* Usando-os com valores de fallback */
env(safe-area-inset-top, 20px);
env(safe-area-inset-right, 1em);
```

```
env(safe-area-inset-bottom, 0.5vh);
env(safe-area-inset-left, 1.4rem);
```

Valores

`safe-area-inset-top`, `safe-area-inset-right`, `safe-area-inset-bottom`, `safe-area-inset-left`

> As variáveis `safe-area-inset-*` são quatro variáveis de ambiente que definem um retângulo por seus valores de inserção: *top, right, bottom* e *left* a partir da borda da janela de visualização, no qual é seguro colocar o conteúdo sem o risco de ser cortado pela forma de um visor não retangular. Para janelas de visualização retangulares, como o monitor de um laptop comum, seu valor é igual a zero. Para telas não retangulares - como um visor de um relógio redondo - os quatro valores definidos pelo agente do usuário formam um retângulo de modo que todo o conteúdo dentro do retângulo seja visível.

> **Nota:** Ao contrário de outras propriedades CSS, os nomes de propriedades definidos pelo agente do usuário fazem distinção entre maiúsculas e minúsculas.

Sintaxe formal

```
<env()> =
  env( <custom-ident> <integer [0,∞]>* , <declaration-value>? )

<integer> =
  <number-token>
```

Esta sintaxe reflete o padrão mais recente conforme CSS Environment Variables Module Level 1, CSS Values and Units Module Level 4. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

Exemplos

O exemplo abaixo faz uso do segundo parâmetro opcional de `env()`, que permite fornecer um valor de *fallback* caso a variável de ambiente não esteja disponível.

```
<p>
  Se a função <code>env()</code> é suportada em seu navegador, o texto deste
  parágrafo deverá estar na borda <i>top</i>, <i>right</i> e <i>bottom</i>, ter
  50px de preenchimento entre a borda e o texto. O CSS é equivalente ao
  preenchimento: <code>padding: 0 0 0 50px</code>, porque, ao contrário de
  outras propriedades CSS, os nomes de propriedades do agente do usuário
  diferenciam letras maiúsculas de minúsculas.
</p>
```

```
p {
  width: 300px;
  border: 2px solid red;
  padding: env(safe-area-inset-top, 50px) env(safe-area-inset-right, 50px)
    env(safe-area-inset-bottom, 50px) env(SAFE-AREA-INSET-LEFT, 50px);
}
```

Exemplo com valores

```
padding: env(
  safe-area-inset-bottom,
  50px
); /* zero para todos agentes do usuário que são retangulares*/
padding: env(
  Safe-area-inset-bottom,
  50px
); /* 50px porque as propriedades do agente do usuário diferenciam maiúsculas de minúsculas */
padding: env(
  x,
  50px 20px
); /* como se estivesse preenchendo: '50px 20px' que foram definidos, porque x não é uma variável de ambiente válida */
padding: env(
  x,
  50px,
  20px
); /* ignorado '50px, 20px' porque não é um valor de preenchimento válido e x não é uma variável de ambiente válida */
```

A sintaxe *fallback*, como de propriedades customizadas, permite vírgulas. Mas se o valor da propriedade não suportar vírgulas, o valor não é válido.

> **Nota:** As propriedades do agente do usuário não são redefinidas pela propriedade all.

Veja também

- `var(…)`
- CSS Custom Properties for Cascading Variables
- Custom Properties (--*)
- Using CSS custom properties (variables)

grayscale()

O **`grayscale()`** é uma função do CSS que modifica a imagem adicionada para as cores branco e preto . Esse resultado é um efeito "filtro". `<filter-function>`.

Experimente

```
filter: grayscale(0);
```

```
filter: grayscale(0.2);
```

```
filter: grayscale(60%);
```

```
filter: grayscale(1);
```

```
<section id="default-example">
  <img
    class="transition-all"
    id="example-element"
    src="/shared-assets/images/examples/firefox-logo.svg"
    width="200" />
</section>
```

Sintaxe

```
grayscale(quantidade)
```

Parâmetros

`quantidade`

A quantia da conversão é especificada em `<number>` (número) ou `<percentage>` (porcentagem). Um valor de `100%` indica que a cor está compeltamente dentro da escala de cinza. Um valor de `0%` não altera as cores da imagem, que permanecerá igual a original. Valores entre `0%` e `100%` são multiplicadores lineares no efeito cinza, modificando a escala de cores conforme a quantidade determinada. A quantia em número é equivalente a em porcentagem.

```
grayscale(0)     /* Não afeta a imagem */
grayscale(.7)    /* .7 => 70% de branco&preto */
grayscale(100%)  /* Completamente em branco&preto */
```

Veja também

- `<filter-function>`

opacity()

A função CSS **`opacity()`** aplica transparência às amostras na imagem inputada. Seu resultado é uma `<filter-function>`.

> **Nota:** Essa função é similar à propriedade `opacity` mais estabelecida. A diferença é que com filtros, alguns browsers fornecem aceleração de hardware para uma melhor performance.

Sintaxe

```
opacity(amount)
```

Parâmetros

`amount`

A quantidade de conversão, especificada com um `<number>` ou uma `<percentage>`. Um valor de `0%` é completamente transparente, enquanto que um valor de `100%` mantém o input inalterado. Valores entre `0%` e `100%` são multiplicadores lineares do efeito. O valor de lacuna para interpolação é `1`.

Exemplos

```
opacity(0%)   /* Completamente transparente */
opacity(50%)  /* 50% transparente */
opacity(1)    /* Nenhum efeito */
```

Ver também

- `<filter-function>`
- Propriedade CSS `opacity`

sepia()

A função CSS **`sepia()`** converte a imagem inputada em sépia, dando uma aparência mais amarelada/marrom e quente. Seu resultado é uma `<filter-function>`.

Sintaxe

```
sepia(amount)
```

Parâmetros

`amount`

A quantidade de conversão, especificada por um `<number>` ou uma `<percentage>`. Um valor de `100%` é completamente sépia, enquanto que um valor de `0%` mantém o input inalterado. Valores entre `0%` e `100%` são multiplicadores lineares do efeito. O valor de lacuna para interpolação é `0`.

Exemplos

```
sepia(0)     /* Sem efeito */
sepia(.65)   /* 65% sépia */
sepia(100%)  /* Completamente sépia */
```

Ver também

- `<filter-function>`

<image>

O tipo de data CSS **`<image>`** representa uma imagem bi-dimensional. Existem dois tipos de imagens: imagens planas, referenciada por um `<url>`, e imagens geradas dinamicamente, geradas por `<gradient>` ou `element()`. Imagens podem ser usadas em inumeras propriedades CSS, como `background-image`, `border-image`, `content`, `cursor`, e `list-style-image`.

Tipos de imagens

CSS pode lidar com os seguintes tipos de imagens:

- Imagens com *dimensões intrínsecas* (tamanho natural), tipo um JPEG, PNG, ou outro formato rasterizado.
- Imagens com *multiplas dimensões intrínsecas,* existente em várias versões dentro de um único arquivo, como alguns formatos .ico. (Nesse caso, a dimensões intrínsecas serão a largura da imagem na área e a proporção mais semelhante à caixa contendo.)
- Imagens sem dimensões intrínsecas mas com um aspecto intrínseco entre a largura e altura, como um SVG ou outro vetor.
- Imagens com nenhuma dimensões intrínsecas, e nenhuma relação de aspecto intrínseco como um gradiente CSS.

CCS determina um tamanho concreto do objeto usando (1) *suas dimensões intrínsecas;* (2) *seu tamanho especificado,* definido por propriedades CSS como `width`, `height`, ou `background-size`; e (3) *é o tamanho padrão,* determinado pelo tipo de propriedade em que a imagem é usada:

Tipo de objeto	Tamanho do objeto padrão
`background-image`	O tamanho da área de posicionamento do fundo do elemento.
`list-style-image`	O tamanho de um carácter `1em`
`border-image`	O tamanho da área da imagem da borda do elemento
`cursor`	O tamanho definido pelo navegador correspondente ao tamanho normal do cursor no sistema do cliente
`border-image-source`	?
`mask-image`	?
`shape-outside`	?
`mask-border-source`	?
Substituí o conteúdo, como quando combinando `content` com um pseudo-elemento (`::after` ou `::before`)	Um 300px × 150px retângulo

O tamanho do objeto concreto é calculado usando o seguinte algoritimo:

- Se o tamanho especificado define *tanto a largura quanto a altura,* esses valores serão usado no tamanho concreto do objeto.
- Se o tamanho especificado define apenas a largura ou a altura, o valor que falta é determind se o valor espeficiado ado usando a relação intrínseca, se existir algum, as dimensões intrínsecas se o valoer espeficicado combinar, ou o tamanho do objeto padrão para esse valor ausente.
- Se o tamanho especificado define nem largura ou altura, o tamanho concreto é calculado de modo que corresponda à proporção intrínseca da imagem mas sem exceder o tamanho padrão do objeto em qualquer dimensão. Se a imagem não tiver relação de aspecto intrínseco, o relação de aspecto intrínseco do objeto é aplicado para ser usado; se esse objeto for vazio, a largura ou altura que faltam são retirados do tamanho de objeto padrão.

> **Nota:** Não são todos os navegadores que suportam cada tipo de imagem em cada propriedade. Veja a seção compatibilidade dos navegadores para mais detalhes.

Sintaxe

O tipo de data `<image>` pode ser representado por qualquer item seguinte:

- Uma imagem denotada pelo tipo de data `<url>`
- Um tipo de data `<gradient>`
- Uma parte da página web, definida pela função `element()`

Exemplos

Imagens válidas

```
url(test.jpg)              /* Uma <url>, enquanto test.jpg é uma imagem real */
linear-gradient(blue, red)  /* Um <gradient> */
element(#realid)           /* Uma parte da página web, referenciada por uma função element() se "realid" for um ID existente na página
*/
```

Imagens inválidas

```
cervin.jpg        /* Um arquivo imagem deve ser definido usando a função url(). */
url(report.pdf)   /* Um arquivo apontado pela função url() deve ser uma imagem. */
element(#fakeid)  /* Um elemento ID deve ser um ID existente na página. */
```

Veja também

- `<gradient>`
- `element()`

initial

A palavra-chave CSS **`initial`** CSS se aplica ao valor inicial (ou padrão) de uma propriedade para um elemento. Este valor inicial é definido pelo navegador. Este valor pode ser aplicado em qualquer propriedade CSS.

Isto inclui também o atalho CSS `all`, no qual o `initial` pode ser utilizado para restaurar todas as propriedades CSS para o seu estado inicial.

> **Nota:** Em propriedades herdadas, O valor inicial pode ser inesperado. Neste caso, considere o uso dos termos {cssxref("inherit")}}, `unset`, ou `revert`.

Exemplo

HTML

```
<p>
  <span>Este texto é vermelho.</span>
  <em>Este texto está em sua cor inicial (normalmente preto).</em>
  <span>Este texto está em vermelho de novo :) </span>
</p>
```

CSS

```
p {
  color: red;
}

em {
  color: initial;
}
```

Veja também

- Utilize `unset` para definir uma propriedade para o seu valor herdado caso haja herança, ou para o seu valor inicial, caso não haja..
- Utilize `revert` para redefinir uma propriedade ao valor estabelecido pela folha de estilo user-agent (ou por seus estilos de usuário, se existir algum).
- Utilize `inherit` para fazer uma propriedade de elemento a mesma de seu elemento pai.
- A propriedade `all` permite que você redefina todas as propriedades para os seus estados initial, inherited, reverted, or unset de uma vez.

<number>

O tipo de dado CSS **`<number>`** representa um número, sendo esse ou um inteiro ou um componente fracional.

Sintaxe

A sintaxe de `<number>` estende a sintaxe de `<integer>`. Um valor fracional é representado por um **`.`** seguido de um ou mais dígitos decimais e pode ser anexado a número inteiro. Não existe uma unidade associada aos números.

Interpolação

Quando animado, os valores de `<number>` são interpolados como números reais e de ponto flutuante. A velocidade da interpolação é determinada pelo tempo da função associada à animação.

Exemplos

Números Válidos

```
12          Um <integer> puro é também um <number>.
4.01        Fração positiva
-456.8      Fração negativa
0.0         Zero
+0.0        Zero inciado por um +
-0.0        Zero iniciado por um -
.60         Um número fracional sem ser iniciado por zero
10e3        Notação científica
-3.4e-2     Notação científica complexa
```

Números Inválidos

```
12.         Números decimais devem ser seguidos por pelo menos um dígito.
+-12.2      Só um sinal +/- é permitido.
12.1.1      Só um ponto decimal é permitido.
```

Veja também

- `<integer>`

<time>

O tipo de dado CSS **`<time>`** representa um valor de tempo expresso em segundo ou milissegundos. É usado em animações(`animation`), transições(`transition`) e propriedades relacionadas.

Sintaxe

O tipo dado `<time>` consiste de um elemento `<number>` seguido por uma das unidades listadas abaixo. Opcionalmente, pode ser precedido por um sinal de `+` ou `-`. Como em todas as dimensões, não há espaço entre uma unidade literal e o número.

> **Nota:** Embora o número `0` seja sempre o mesmo, independente da unidade, essa última não pode ser omitida. Em outras palavras, `0` é invalido e não representa `0s` ou `0ms`.

Unidades

`s`

Representa um tempo em segundos. Exemplos: `0s`, `1.5s`, `-60s`.

`ms`

Representa um tempo em milissegundos. Exemplos: `0ms`, `150.25ms`, `-60000ms`.

> **Nota:** A conversão entre `s` e `ms` segue a proporção `1s = 1000ms`.

Exemplos

Unidades de Tempo Válidas

```
12s         Inteiro Positivo
-456ms      Inteiro Negativo
4.3ms       Decimal
14mS        A unidade é case-insentive, embora letras iniciais maiúsculas (capital letters) não sejam recomendadas.
+0s         Zero com um + no inicio e uma unidade
-0ms        Zero com um - no inicio e uma unidade
```

Unidades de Tempo Inválidas

```
0           Embora seja permitido o uso do zero sem unidade de medida por <length>s é invalido para <time>s.
12.0        Este é um <number>, não um <time>, pois está faltando a unidade de medida.
7 ms        Não é permitido espaço entre o número e a unidade de medida.
```

perspective()

A função CSS **`perspective()`** define uma transformação que configura a distância entre o usuário e o plano z=0. Seu resultado é um dado do tipo `<transform-function>`.

Sintaxe

A distância de perspectiva usada por`perspective()` é especificada por um valor `<length>`, que representa a distância entre o usuário e o plano z=0. Um valor positivo faz o elemento parecer mais perto do usuário, e um valor negativo o faz parecer mais longe.

```
perspective(d)
```

Valores

d

É um `<length>` representando a distância do usuário até o plano z=0. Se for 0 ou um valor negativo, nenhuma transformação de perspectiva é aplicada.

Coordenadas cartesianas em $\mathbb{R}^2$	**Coordenadas homogêneas em $\mathbb{RP}^2$**	**Coordenadas cartesianas em $\mathbb{R}^3$**	**Coordenadas homogêneas em $\mathbb{RP}^3$**
Essa transformação se aplica ao espaço 3D e não pode ser representada no plano.		Essa não é uma transformação linear em $\mathbb{R}^3$, e não pode ser representada usando uma matriz de coordenadas cartesianas.	$\begin{matrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -1/d & 1 \end{matrix}$

Exemplos

HTML

```
<p>Sem perspectiva:</p>
<div class="no-perspective-box">
  <div class="face front">A</div>
  <div class="face top">B</div>
  <div class="face left">C</div>
</div>

<p>Com perspectiva (7.5cm):</p>
<div class="perspective-box">
  <div class="face front">A</div>
  <div class="face top">B</div>
  <div class="face left">C</div>
</div>
```

CSS

```
.face {
  position: absolute;
  width: 100px;
  height: 100px;
  line-height: 100px;
  font-size: 100px;
  text-align: center;
}

.no-perspective-box {
  width: 100px;
  height: 100px;
  transform-style: preserve-3d;
  transform: rotateX(-15deg) rotateY(15deg);
  margin-left: 100px;
}

.perspective-box {
  width: 100px;
  height: 100px;
  transform-style: preserve-3d;
  transform: perspective(7.5cm) rotateX(-15deg) rotateY(15deg);
  margin-left: 100px;
}

.top {
  background-color: skyblue;
  transform: rotateX(90deg) translate3d(0, 0, 50px);
}

.left {
  background-color: pink;
  transform: rotateY(-90deg) translate3d(0, 0, 50px);
}

.front {
  background-color: limegreen;
  transform: translate3d(0, 0, 50px);
}
```

Resultado

Compatibilidade com navegadores

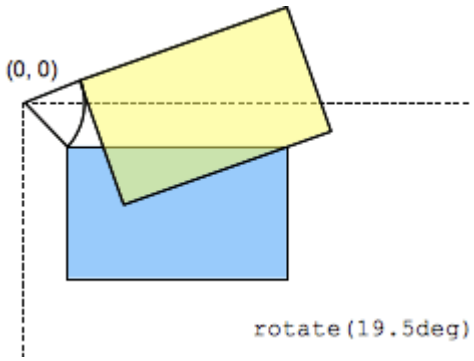
Veja o tipo de dado `<transform-function>` para informações de compatibilidade.

Ver também

- `transform`
- `<transform-function>`

rotate()

A função CSS **`rotate()`** define uma transformação que gira um elemento em torno de um ponto fixo no plano 2D, sem deformá-lo. O resultado é um tipo de dados `<transform-function>`.



O eixo de rotação passa por uma origem, definido pela propriedade CSS `transform-origin`.

Sintaxe

A quantidade de rotação criada por `rotate()` é especificado por um `<angle>`. Se positivo, o movimento será no sentido horário; Se negativo, ela será no sentido anti-horário. Uma rotação de 180° é chamada de *point reflection* (reflexão do ponto).

```
rotate(a)
```

Valores

`a`

É um `<angle>` representando o ângulo da rotação. Um ângulo positivo indica uma rotação no sentido horário, um ângulo negativo no sentido anti-horário.

Coordenadas cartesianas em $\mathbb{R}^2$	**Coordenadas homogêneas em $\mathbb{RP}^2$**	**Coordenadas cartesianas em $\mathbb{R}^3$**	**Coordenadas homogêneas em $\mathbb{RP}^3$**
$\begin{matrix}\cos(a) & -\sin(a) \\ \sin(a) & \cos(a)\end{matrix}$	$\begin{matrix}\cos(a) & -\sin(a) & 0 \\ \sin(a) & \cos(a) & 0 \\ 0 & 0 & 1\end{matrix}$ `[cos(a) sin(a) -sin(a) cos(a) 0 0]`	$\begin{matrix}\cos(a) & -\sin(a) & 0 \\ \sin(a) & \cos(a) & 0 \\ 0 & 0 & 1\end{matrix}$	$\begin{matrix}\cos(a) & -\sin(a) & 0 & 0 \\ \sin(a) & \cos(a) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1\end{matrix}$

Exemplos

HTML

```
<div>Normal</div>
<div class="rotated">Rotated</div>
```

CSS

```
div {
  width: 80px;
  height: 80px;
  background-color: skyblue;
}

.rotated {
```

```
  transform: rotate(45deg); /* Equal to rotateZ(45deg) */
  background-color: pink;
}
```

Resultado

Compatibilidade do navegador

Por favor, veja o tipo de dados `<transform-function>` para informações de compatibilidade.

Veja também

- `transform`
- `<transform-function>`
- `rotate3d()`

rotate3d()

A função CSS **`rotate3d()`** define uma transformação que gira um elemento em torno de um eixo fixo no espaço 3D, sem deformá-lo. Seu resultado é um dado do tipo `<transform-function>`.

No espaço 3D, rotações têm três graus de liberdade que, juntos, descrevem um único eixo de rotação. O eixo de rotação é definido por um vetor [x, y, z] e passado pela origem (como definido pela propriedade `transform-origin`). Se, como especificado, o vetor não for *normalizado* (isto é, se a soma dos quadrados das suas três coordenadas não for 1), o user agent irá normalizá-lo internamente. Um vetor não-normalizável, como o vetor nulo [0, 0, 0], fará com que a rotação seja ignorada, mas sem invalidar toda a propriedade CSS.

> **Nota:** Diferente de rotações no plano 2D, a composição de rotações 3D normalmente não é comutativa. Em outras palavras, a ordem na qual as rotações são aplicadas impacta o resultado.

Sintaxe

A quantidade de rotação criada por `rotate3d()` é especificada por três `<number>` e um `<angle>`. Os `<number>` representam as coordenadas x, y e z do vetor, denotando o eixo de rotação. O `<angle>` representa o ângulo de rotação; se positivo, o movimento será no sentido horário; se negativo, será no sentido anti-horário.

```
rotate3d(x, y, z, a)
```

Valores

`x`

: É um `<number>` descrevendo a coordenada x do vetor denotando o eixo de rotação que pode ser entre 0 e 1.

`y`

: É um `<number>` descrevendo a coordenada y do vetor denotando o eixo de rotação que pode ser entre 0 e 1.

`z`

: É um `<number>` descrevendo a coordenada z do vetor denotando o eixo de rotação que pode ser entre 0 e 1.

`a`

: É um `<angle>` representando o ângulo de rotação. Um ângulo positivo denota uma rotação no sentido horário, e um negativo no sentido anti-horário.

Coordenadas cartesianas em $\mathbb{R}^2$	Essa transformação se aplica ao plano 3D e não pode ser representada em um plano.
Coordenadas homogêneas em $\mathbb{RP}^2$	

Coordenadas cartesianas em $\mathbb{R}^3$	$1+(1 - \cos(a))(x^2 - 1)$ $-z \cdot \sin(a) + xy \cdot (1 - \cos(a))$	$z \cdot \sin(a)+xy(1 - \cos(a))$ $1+(1-\cos(a))(y2-1)$	$-y \cdot \sin(a) + xz \cdot (1 - \cos(a))$ $x \cdot \sin(a) + yz \cdot (1 - \cos(a))$	ysin(a) + xz(1-cos(a)) 0	-xsin(a)+yz(1-cos(a)) 0	1+(1-cos(a))(z2-1) 0	t 1
Coordenadas homogêneas em $\mathbb{RP}^3$							

Exemplos

Rotacionando no eixo y

HTML

```
<div>Normal</div>
<div class="rotated">Rotacionado</div>
```

CSS

```
body {
  perspective: 800px;
}

div {
  width: 80px;
  height: 80px;
  background-color: skyblue;
}

.rotated {
  transform: rotate3d(0, 1, 0, 60deg);
  background-color: pink;
}
```

Resultado

Rotacionando em um eixo customizado

HTML

```
<div>Normal</div>
<div class="rotated">Rotacionado</div>
```

CSS

```
body {
  perspective: 800px;
}

div {
  width: 80px;
  height: 80px;
  background-color: skyblue;
}

.rotated {
  transform: rotate3d(1, 2, -1, 192deg);
  background-color: pink;
}
```

Resultado

Compatibilidade com navegadores

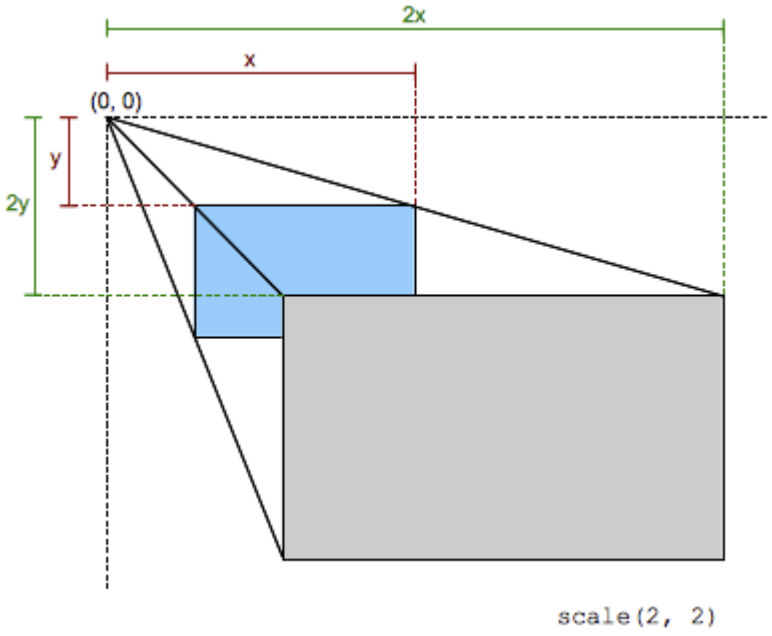
Veja o tipo de dado `<transform-function>` para informações de compatibilidade.

Ver também

- `transform`
- `<transform-function>`

scale()

A função CSS **`scale()`** define uma transformação que redimensiona um elemento no plano 2D. Como o redimensionamento é definido por um vetor, ele pode transformar as dimensões verticais e horizontais em escalas diferentes. Seu resultado é um dado do tipo `<transform-function>`.



Essa transformação de redimensionamento é caracterizada por um vetor bidimensional. Suas coordenadas definem o quanto cada direção deve ser redimensionada. Se as duas coordenadas forem iguais, o redimensionamento é uniforme (*isotrópico*) e a proporção do elemento é preservada (isto é uma transformação homotética).

Quando o valor de uma coordenada está fora do alcance [-1, 1], o elemento cresce ao longo daquela dimensão; quando está dentro, ele encolhe. Se for negativo, o resultado é um ponto de reflexão naquela dimensão. O valor 1 não tem efeito.

> **Nota:** A função `scale()` apenas redimensiona em 2D Para redimensionar em 3D, use `scale3d()` ao invés.

Sintaxe

A função `scale()` é especificada com um ou dois valores, que representam a quantidade de redimensionamento a ser aplicada em cada direção.

```
scale(sx)

scale(sx, sy)
```

Valores

`sx`

Um `<number>` representando a abscissa do vetor de redimensionamento.

`sy`

Um `<number>` representando a ordenada do vetor de redimensionamento. Se não for definida, seu valor padrão é`sx`, resultando em um redimensionamento uniforme que preserva a proporção do elemento.

Coordenadas cartesianas em $\mathbb{R}^2$	Coordenadas homogêneas em $\mathbb{RP}^2$	Coordenadas cartesianas em $\mathbb{R}^3$	Coordenadas homogêneas em $\mathbb{RP}^3$
$\begin{matrix} sx & 0 \\ 0 & sy \end{matrix}$	$\begin{matrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{matrix}$ `[sx 0 0 sy 0 0]`	$\begin{matrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{matrix}$	$\begin{matrix} sx & 0 & 0 & 0 \\ 0 & sy & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$

Exemplos

Redimensionando as dimensões X e Y juntas

HTML

```
<div>Normal</div>
<div class="scaled">Redimensionado</div>
```

CSS

```
div {
  width: 80px;
  height: 80px;
  background-color: skyblue;
}

.scaled {
  transform: scale(0.7); /* Equal to scaleX(0.7) scaleY(0.7) */
  background-color: pink;
}
```

Resultado

Redimensionando dimensões X e Y separadamente e transladando a origem

HTML

```
<div>Normal</div>
<div class="scaled">Redimensionado</div>
```

CSS

```
div {
  width: 80px;
  height: 80px;
  background-color: skyblue;
}

.scaled {
  transform: scale(2, 0.5); /* Igual a scaleX(2) scaleY(0.5) */
  transform-origin: left;
  background-color: pink;
}
```

Resultado

Compatibilidade com navegadores

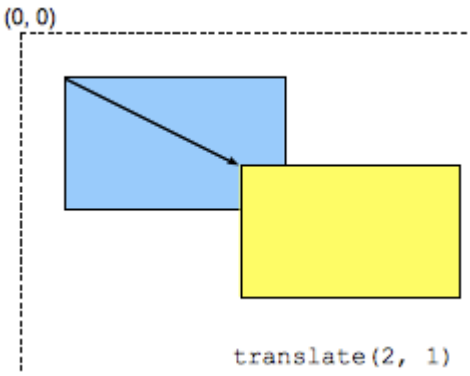
Veja o dado do tipo `<transform-function>` para informações de compatibilidade.

Ver também

- `transform`
- `<transform-function>`
- `scale3d()`

translate()

A função CSS **`translate()`** reposiciona um elemento na direção horizontal e/ou vertical. O seu resultado é um tipo de dado `<transform-function>`.



Esta transformação é caracterizada por um vetor bidimensional. Suas coordenadas definem o quanto o elemento se move em cada direção.

Sintaxe

```
/* Valores <length-percentage> únicos */
transform: translate(200px);
transform: translate(50%);

/* Valores <length-percentage> duplos */
transform: translate(100px, 200px);
transform: translate(100px, 50%);
transform: translate(30%, 200px);
transform: translate(30%, 50%);
```

Valores

Valores `<length-percentage>` únicos

Este valor é um `<length>` ou `<percentage>` representando a abcissa (horizontal, coordenada x) do vetor de translação. A ordenada (vertical, coordenada y) do vetor de translação será definida como `0`. Por exemplo, `translate(2)` é equivalente a `translate(2, 0)`. Um valor percentual refere-se à largura da caixa de referência definida pela propriedade `transform-box`.

Valores `<length-percentage>` duplos

Este valor descreve dois `<length>` ou valores `<percentage>` representando tanto a abcissa (coordenada x) quanto a ordenada (coordenada y) do vetor de translação. Uma porcentagem como primeiro valor refere-se à largura, como segunda parte da altura da caixa de referência definida pela propedade `transform-box`.

Coordenadas cartesianas em $\mathbb{R}^2$	**Coordenadas homogêneas em $\mathbb{RP}^2$**	**Coordenadas cartesianas em $\mathbb{R}^3$**	**Coordenadas homogêneas em $\mathbb{RP}^3$**
Uma translação não é uma transformação linear em $\mathbb{R}^2$ e não pode ser representada usando uma matriz de coordenadas cartesianas.	$\begin{matrix}1 & 0 & tx\\ 0 & 1 & ty\\ 0 & 0 & 1\end{matrix}$ `[1 0 0 1 tx ty]`	$\begin{matrix}1 & 0 & tx\\ 0 & 1 & ty\\ 0 & 0 & 1\end{matrix}$	$\begin{matrix}1 & 0 & 0 & tx\\ 0 & 1 & 0 & ty\\ 0 & 0 & 1 & 0\\ 0 & 0 & 0 & 1\end{matrix}$

Sintaxe formal

```
translate(<length-percentage> , <length-percentage>?)
```

Exemplos

Usando uma conversão de eixo único

HTML

```
<div>Estático</div>
<div class="moved">Movido</div>
<div>Estático</div>
```

CSS

```
div {
  width: 60px;
  height: 60px;
  background-color: skyblue;
}

.moved {
  transform: translate(
    10px
  ); /* Igual a: translateX(10px) ou translate(10px, 0) */
  background-color: pink;
}
```

Resultado

Combinando a translação do eixo y e do eixo x

HTML

```
<div>Estático</div>
<div class="moved">Movido</div>
<div>Estático</div>
```

CSS

```
div {
  width: 60px;
  height: 60px;
  background-color: skyblue;
}

.moved {
  transform: translate(10px, 10px);
  background-color: pink;
}
```

Resultado

Compatibilidade com navegadores

Verifique o tipo de dado `<transform-function>` para informações de compatibilidade.

Veja também

- `transform`
- `<transform-function>`

var()

A função CSS **`var()`** pode ser usada para inserir o valor de uma custom property (propriedade personalizada) substituindo qualquer parte do valor de outra propriedade.

```
var(--header-color, blue);
```

A função `var()` não pode ser usada em nomes de propriedades, seletores ou qualquer outra coisa além de valores de propriedades. (Isso geralmente produz sintaxe inválida, ou então um valor cujo significado não tem conexão com a variável.)

Sintaxe

O primeiro argumento da função é o nome da propriedade personalizada a ser substituída. O segundo argumento opcional da função serve como valor alternativo. Se a propriedade personalizada referenciada pelo primeiro argumento for inválida, a função usará o segundo valor.

```
<var()> =
  var( <custom-property-name> , <declaration-value>? )
```

Esta sintaxe reflete o padrão mais recente conforme CSS Custom Properties for Cascading Variables Module Level 2. Nem todos os navegadores podem ter implementado todas as partes. Consulte Compatibilidade do navegador para informações de suporte.

> **Nota:** A sintaxe do argumento alternativo, assim como as propriedades personalizadas, permite o uso de vírgulas. Por exemplo, `var(--foo, red, blue)` define como argumento alternativo `red, blue`; isto é, qualquer coisa entre a primeira vírgula e o fim da função é considerado como valor do como argumento alternativo.

Valores

<custom-property-name>

O nome da propriedade personalizada referenciada é representada por um identificador que começa com dois traços. As propriedades personalizadas são exclusivamente para uso por autores e usuários; O CSS nunca lhes dará um significado além do que é apresentado aqui.

<declaration-value>

O valor do argumento alternativo é usado caso a propriedade personalizada seja inválida no contexto utilizado. Este valor pode conter qualquer caractere, exceto alguns caracteres com significado especial como novas linhas, colchetes de fechamento perdidos, ex.`)`, `]`, ou `}`, ponto e vírgula de primeiro nível, ou sinais de exclamação.

Exemplos

```
:root {
  --main-bg-color: pink;
}

body {
  background-color: var(--main-bg-color);
}
```

```
/* Argumento alternativo */
/* No estilo de componente: */
.component .header {
  color: var(--header-color, blue);
}

.component .text {
  color: var(--text-color, black);
}

/* No estilo de grandes aplicações: */
.component {
  --text-color: #080; /* header-color não está definido e, desse modo, permanece azul, o valor do argumento alternativo */
}
```

Veja também

- Utilizando variáveis CSS